



CompTIA

CertyIQ

Premium exam material

Get certification quickly with the CertyIQ Premium exam material.

Everything you need to prepare, learn & pass your certification exam easily. Lifetime free updates

First attempt guaranteed success.

<https://www.CertyIQ.com>

About CertyIQ

We here at CertyIQ eventually got enough of the industry's greedy exam paid for. Our team of IT professionals comes with years of experience in the IT industry Prior to training CertIQ we worked in test areas where we observed the horrors of the paywall exam preparation system.

The misuse of the preparation system has left our team disillusioned. And for that reason, we decided it was time to make a difference. We had to make In this way, CertyIQ was created to provide quality materials without stealing from everyday people who are trying to make a living.

Doubt Support

We have developed a very scalable solution using which we are able to solve 400+ doubts every single day with an average rating of 4.8 out of 5.

<https://www.certyiq.com>

Mail us on - certyiqofficial@gmail.com



Lifetime Free Updates

We provide lifetime free updates to our customers. To make life easier for our valued customers and fulfill their needs



Free Exam PDF

You are sure to pass the exam completely free of charge



Money Back Guarantee

We Provide 100% money back guarantee to our customer in case of any failure

John

October 19, 2022



Thanks you so much for your help. I scored 972 in my exam today. More than 90% were from your PDFs!

Dana

September 04, 2022



Thanks a lot for this updated AZ-900 Q&A. I just passed my exam and got 974, I followed both of your Az-900 videos and the 6 PDF, the PDFs are very much valid, all answers are correct. Could you please create a similar video/PDF for DP900, your content/PDF's is really awesome. The team did a really good job. Thank You 😊.

Ahamed Shibly

2 months ago



Customer support is really fast and helpful, I just finished my exam and this video along with the 6 PDF helped me pass! Definitely recommend getting the PDFs. Thank you!

October 22, 2022



Passed my exam today with 891 marks. Out of 52 questions, 51 were from certyiq PDFs including Contoso case study. Thank You certyiq team!

Henry Rome

2 months ago



These questions are real and 100 % valid. Thank you so much for your efforts, also your 4 PDFs are awesome, I passed the DP900 exam on 1 Sept. With 968 marks. Thanks a lot, buddy!

Esmaria

2 months ago



Simple easy to understand explanations. To anyone out there wanting to write AZ900, I highly recommend 6 PDF's. Thank you so much, appreciate all your hard work in having such great content. Passed my exam Today -3 September with 942 score.

Amazon

(AWS Certified Advanced Networking - Specialty ANS-C01)

AWS Certified Advanced Networking - Specialty ANS-C01

Total: **291 Questions**

Link: <https://certyiq.com/papers/amazon/aws-certified-advanced-networking-specialty-ans-c01>

Question: 1

A company is planning to create a service that requires encryption in transit. The traffic must not be decrypted between the client and the backend of the service. The company will implement the service by using the gRPC protocol over TCP port 443. The service will scale up to thousands of simultaneous connections. The backend of the service will be hosted on an Amazon Elastic Kubernetes Service (Amazon EKS) cluster with the Kubernetes Cluster Autoscaler and the Horizontal Pod Autoscaler configured. The company needs to use mutual TLS for two-way authentication between the client and the backend.

Which solution will meet these requirements?

- A. Install the AWS Load Balancer Controller for Kubernetes. Using that controller, configure a Network Load Balancer with a TCP listener on port 443 to forward traffic to the IP addresses of the backend service Pods.
- B. Install the AWS Load Balancer Controller for Kubernetes. Using that controller, configure an Application Load Balancer with an HTTPS listener on port 443 to forward traffic to the IP addresses of the backend service Pods.
- C. Create a target group. Add the EKS managed node group's Auto Scaling group as a target. Create an Application Load Balancer with an HTTPS listener on port 443 to forward traffic to the target group.
- D. Create a target group. Add the EKS managed node group's Auto Scaling group as a target. Create a Network Load Balancer with a TLS listener on port 443 to forward traffic to the target group.

Answer: A

Explanation:

Here's a detailed justification for why option A is the most suitable solution, and why the other options are less ideal:

Justification for Option A:

Option A leverages a Network Load Balancer (NLB) deployed via the AWS Load Balancer Controller for Kubernetes. This approach directly addresses the core requirements:

1. **Encryption in Transit (End-to-End):** An NLB operates at the transport layer (Layer 4), specifically TCP in this case. It forwards traffic based on IP addresses and ports without inspecting the content. This ensures that the TLS encryption, crucial for the gRPC protocol and mutual TLS, remains intact from the client to the backend pods. No decryption happens at the load balancer level.
2. **Mutual TLS:** Mutual TLS (mTLS) necessitates that both the client and the server (backend pods) authenticate each other using certificates. The NLB's role in this setup is to purely forward encrypted traffic, allowing the backend pods to handle the TLS handshake and certificate validation directly.
3. **Scalability:** NLBs are designed for high throughput and low latency, scaling to handle thousands of simultaneous connections efficiently. They automatically handle the load distribution across available backend pods. The Kubernetes Cluster Autoscaler and Horizontal Pod Autoscaler further enhance scalability by dynamically adjusting the number of pods and nodes as needed.
4. **gRPC over TCP Port 443:** The NLB can be configured with a TCP listener on port 443, which is the standard port for HTTPS/TLS traffic, accommodating the gRPC protocol.
5. **Kubernetes Integration:** The AWS Load Balancer Controller automates the provisioning and management of load balancers based on Kubernetes resources, simplifying deployment and maintenance. It dynamically updates the NLB's target group (backend pods) as the Kubernetes service scales.

Why other options are less suitable:

Option B (Application Load Balancer with HTTPS listener): An Application Load Balancer (ALB) terminates TLS connections. While it provides features like content-based routing, it would decrypt the traffic at the load balancer, violating the requirement that traffic must not be decrypted between the client and the backend. This breaks the end-to-end encryption needed for mTLS and is a fundamental flaw.

Option C (ALB with Auto Scaling Group): Similar to option B, an ALB with an HTTPS listener would terminate TLS at the load balancer level, decrypting the traffic and defeating the purpose of end-to-end encryption and

mutual TLS. Targeting the Auto Scaling group directly bypasses Kubernetes' service abstraction.

Option D (NLB with Auto Scaling Group and TLS Listener): While using an NLB avoids TLS termination, creating a TLS listener on the NLB means the NLB itself becomes responsible for TLS encryption. This isn't what the question asked for, as it changes the mutual TLS relationship, and the application is still meant to be doing this itself.

Authoritative Links:

AWS Load Balancer Controller: <https://kubernetes-sigs.github.io/aws-load-balancer-controller/>

Network Load Balancer: <https://docs.aws.amazon.com/elasticloadbalancing/latest/network/introduction.html>

Mutual TLS: <https://smallstep.com/hello-mtls/>

gRPC: <https://grpc.io/>

In conclusion, option A is the only one that completely aligns with the problem requirements, providing end-to-end encryption, supporting mutual TLS, scaling efficiently, and integrating seamlessly with the Kubernetes environment.

Question: 2

CertyIQ

A company is deploying a new application in the AWS Cloud. The company wants a highly available web server that will sit behind an Elastic Load Balancer. The load balancer will route requests to multiple target groups based on the URL in the request. All traffic must use HTTPS. TLS processing must be offloaded to the load balancer. The web server must know the user's IP address so that the company can keep accurate logs for security purposes. Which solution will meet these requirements?

- A. Deploy an Application Load Balancer with an HTTPS listener. Use path-based routing rules to forward the traffic to the correct target group. Include the X-Forwarded-For request header with traffic to the targets.
- B. Deploy an Application Load Balancer with an HTTPS listener for each domain. Use host-based routing rules to forward the traffic to the correct target group for each domain. Include the X-Forwarded-For request header with traffic to the targets.
- C. Deploy a Network Load Balancer with a TLS listener. Use path-based routing rules to forward the traffic to the correct target group. Configure client IP address preservation for traffic to the targets.
- D. Deploy a Network Load Balancer with a TLS listener for each domain. Use host-based routing rules to forward the traffic to the correct target group for each domain. Configure client IP address preservation for traffic to the targets.

Answer: A

Explanation:

The correct solution is A because it effectively addresses all the requirements outlined in the scenario.

1. **High Availability & Web Server:** An Application Load Balancer (ALB) inherently provides high availability by distributing traffic across multiple target groups (web servers) in different Availability Zones.
2. **HTTPS & TLS Offloading:** The ALB supports HTTPS listeners, enabling TLS termination at the load balancer. This offloads the CPU-intensive task of encryption/decryption from the web servers, improving their performance.
3. **URL-Based Routing:** ALB's path-based routing rules allow forwarding traffic to different target groups based on the URL in the request. This is crucial for directing requests to specific application components.
4. **User IP Address:** Including the X-Forwarded-For request header is the standard way to preserve the original client IP address when TLS is terminated at the load balancer. The ALB automatically adds this header, allowing the web servers to log the client's IP for security purposes.

Option B is less efficient because it suggests creating a separate HTTPS listener for each domain, which can complicate configuration and management if the number of domains increases. While host-based routing is a valid approach, it isn't strictly necessary when URL-based routing already satisfies the pathing requirement.

Options C and D involve a Network Load Balancer (NLB). While NLBs offer extremely high performance and TCP/UDP support, they are not ideal for HTTP/HTTPS traffic and URL-based routing. NLBs operate at Layer 4 (TCP/UDP), while ALBs operate at Layer 7 (Application Layer), enabling more sophisticated routing based on HTTP headers and paths. Moreover, NLBs do not automatically add the X-Forwarded-For header. Although NLBs can preserve the source IP address, using an ALB and X-Forwarded-For header is the best practice for HTTP(S) applications.

In summary, using an ALB with HTTPS, path-based routing, and the X-Forwarded-For header efficiently meets all of the company's requirements for high availability, secure communication, URL-based traffic distribution, and accurate logging.

Authoritative Links:

1. **Application Load Balancers:**
<https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html>
2. **Network Load Balancers:**
<https://docs.aws.amazon.com/elasticloadbalancing/latest/network/introduction.html>
3. **X-Forwarded-For Header:** <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Forwarded-For>

Question: 3

CertyIQ

A company has developed an application on AWS that will track inventory levels of vending machines and initiate the restocking process automatically. The company plans to integrate this application with vending machines and deploy the vending machines in several markets around the world. The application resides in a VPC in the us-east-1 Region. The application consists of an Amazon Elastic Container Service (Amazon ECS) cluster behind an Application Load Balancer (ALB). The communication from the vending machines to the application happens over HTTPS.

The company is planning to use an AWS Global Accelerator accelerator and configure static IP addresses of the accelerator in the vending machines for application endpoint access. The application must be accessible only through the accelerator and not through a direct connection over the internet to the ALB endpoint.

Which solution will meet these requirements?

- A. Configure the ALB in a private subnet of the VPC. Attach an internet gateway without adding routes in the subnet route tables to point to the internet gateway. Configure the accelerator with endpoint groups that include the ALB endpoint. Configure the ALB's security group to only allow inbound traffic from the internet on the ALB listener port.
- B. Configure the ALB in a private subnet of the VPC. Configure the accelerator with endpoint groups that include the ALB endpoint. Configure the ALB's security group to only allow inbound traffic from the internet on the ALB listener port.
- C. Configure the ALB in a public subnet of the VPC. Attach an internet gateway. Add routes in the subnet route tables to point to the internet gateway. Configure the accelerator with endpoint groups that include the ALB endpoint. Configure the ALB's security group to only allow inbound traffic from the accelerator's IP addresses on the ALB listener port.
- D. Configure the ALB in a private subnet of the VPC. Attach an internet gateway. Add routes in the subnet route tables to point to the internet gateway. Configure the accelerator with endpoint groups that include the ALB endpoint. Configure the ALB's security group to only allow inbound traffic from the accelerator's IP addresses on the ALB listener port.

Answer: A

Explanation:

Here's a detailed justification for why option A is the correct solution:

The core requirement is to ensure that the application is only accessible through the AWS Global Accelerator and not directly via the internet. This implies preventing direct public access to the Application Load Balancer (ALB).

Option A achieves this through the following steps:

1. **ALB in a Private Subnet:** Placing the ALB in a private subnet ensures that it does not have a direct public IP address. This makes it inaccessible directly from the internet.
2. **Internet Gateway Attachment without Routes:** Attaching an Internet Gateway (IGW) to the VPC is necessary for the Global Accelerator to communicate with resources within the VPC. However, by not adding routes in the subnet route tables to the IGW, you prevent resources in the subnet (including the ALB) from initiating outbound internet traffic. This configuration allows the Global Accelerator to initiate traffic to the ALB, but it prevents direct internet access.
3. **Accelerator Endpoint Groups:** Configuring the Global Accelerator with endpoint groups that include the ALB endpoint establishes the connection between the accelerator and the ALB. The Global Accelerator will route traffic to the ALB's private IP address via the VPC.
4. **ALB Security Group:** Configuring the ALB's security group to only allow inbound traffic from the internet on the ALB listener port (HTTPS - usually port 443) is crucial. Because it's a private subnet, this sounds counterintuitive, but it works as follows. The Global Accelerator acts as the entry point to the VPC. When the Global Accelerator sends traffic to the ALB, the source IP will be internet-routable but originating from an AWS service, not the direct client's IP. This ensures that only traffic originating from AWS and routed through the Global Accelerator can reach the ALB. Direct connections to the ALB are blocked.

Option B is incorrect because while the ALB is in a private subnet, allowing inbound traffic from the internet without controlling the source IPs will permit anyone on the internet to directly access the ALB, bypassing the Global Accelerator.

Option C is incorrect because placing the ALB in a public subnet defeats the entire purpose of preventing direct internet access. An ALB in a public subnet, with an internet gateway and appropriate routes, will always be directly accessible via the internet.

Option D is incorrect because configuring the ALB to only allow traffic from the accelerator's IP addresses does not work since the traffic won't originate directly from the accelerator's IPs, but rather it will come through internet routable IPs originating from AWS itself (as it originates from the edge location of the Global Accelerator). In addition, adding routes in the subnet route tables to the internet gateway would also allow resources in the subnet to initiate outbound internet traffic, contrary to the overall architecture.

Here are some helpful links to further understand AWS Global Accelerator:

AWS Global Accelerator: <https://aws.amazon.com/global-accelerator/>

AWS Global Accelerator Documentation: <https://docs.aws.amazon.com/global-accelerator/>

Security Groups: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_SecurityGroups.html

Question: 4

CertyIQ

A global delivery company is modernizing its fleet management system. The company has several business units. Each business unit designs and maintains applications that are hosted in its own AWS account in separate application VPCs in the same AWS Region. Each business unit's applications are designed to get data from a central shared services VPC.

The company wants the network connectivity architecture to provide granular security controls. The architecture also must be able to scale as more business units consume data from the central shared services VPC in the

future.

Which solution will meet these requirements in the MOST secure manner?

- A. Create a central transit gateway. Create a VPC attachment to each application VPC. Provide full mesh connectivity between all the VPCs by using the transit gateway.
- B. Create VPC peering connections between the central shared services VPC and each application VPC in each business unit's AWS account.
- C. Create VPC endpoint services powered by AWS PrivateLink in the central shared services VPC. Create VPC endpoints in each application VPC.
- D. Create a central transit VPC with a VPN appliance from AWS Marketplace. Create a VPN attachment from each VPC to the transit VPC. Provide full mesh connectivity among all the VPCs.

Answer: C

Explanation:

The most secure and scalable solution is option C, using VPC endpoint services (powered by AWS PrivateLink). Here's why:

Security: PrivateLink establishes private connectivity between VPCs without exposing traffic to the public internet. This eliminates the need for internet gateways, NAT devices, or public IPs, significantly reducing the attack surface. VPC endpoints are accessed through private IP addresses within your VPC.

Granular Access Control: VPC endpoint services provide granular control over which services and resources within the central shared services VPC are accessible to each application VPC. This is achieved through security groups and endpoint policies, allowing you to restrict access to specific resources.

Scalability: AWS PrivateLink is highly scalable and can easily accommodate new business units and application VPCs as the company grows. Adding a new application VPC only involves creating a new VPC endpoint, without disrupting existing connections.

Simplified Network Management: PrivateLink simplifies network management by eliminating the need for complex routing configurations or VPN connections. This reduces operational overhead and potential for misconfigurations.

Let's look at why the other options are less suitable:

A (Transit Gateway): While Transit Gateway provides centralized connectivity, full mesh connectivity increases the potential attack surface. It also requires more complex route table management and doesn't offer the same level of granular security controls as PrivateLink.

B (VPC Peering): VPC peering creates direct connections between VPCs. For a large number of VPCs, this can become difficult to manage due to the number of peerings required ($n*(n-1)/2$, where n is the number of VPCs). Peering connections do not inherently offer the granular security control of PrivateLink.

D (Transit VPC with VPN): This solution is more complex, requires managing VPN appliances, and introduces potential performance bottlenecks. It also relies on VPN connections over the public internet (or Direct Connect) and is less secure than PrivateLink.

In summary, AWS PrivateLink provides the most secure, scalable, and manageable solution for connecting application VPCs to a central shared services VPC, with granular control over access to resources.

Relevant Documentation:

AWS PrivateLink: <https://aws.amazon.com/privatelink/>

VPC Endpoints: <https://docs.aws.amazon.com/vpc/latest/privatelink/vpc-endpoints.html>

Question: 5

CertyIQ

A company uses a 4 Gbps AWS Direct Connect dedicated connection with a link aggregation group (LAG) bundle

to connect to five VPCs that are deployed in the us-east-1 Region. Each VPC serves a different business unit and uses its own private VIF for connectivity to the on-premises environment. Users are reporting slowness when they access resources that are hosted on AWS.

A network engineer finds that there are sudden increases in throughput and that the Direct Connect connection becomes saturated at the same time for about an hour each business day. The company wants to know which business unit is causing the sudden increase in throughput. The network engineer must find out this information and implement a solution to resolve the problem.

Which solution will meet these requirements?

A. Review the Amazon CloudWatch metrics for `VirtualInterfaceBpsEgress` and `VirtualInterfaceBpsIngress` to determine which VIF is sending the highest throughput during the period in which slowness is observed. Create a new 10 Gbps dedicated connection. Shift traffic from the existing dedicated connection to the new dedicated connection.

B. Review the Amazon CloudWatch metrics for `VirtualInterfaceBpsEgress` and `VirtualInterfaceBpsIngress` to determine which VIF is sending the highest throughput during the period in which slowness is observed. Upgrade the bandwidth of the existing dedicated connection to 10 Gbps.

C. Review the Amazon CloudWatch metrics for `ConnectionBpsIngress` and `ConnectionPpsEgress` to determine which VIF is sending the highest throughput during the period in which slowness is observed. Upgrade the existing dedicated connection to a 5 Gbps hosted connection.

D. Review the Amazon CloudWatch metrics for `ConnectionBpsIngress` and `ConnectionPpsEgress` to determine which VIF is sending the highest throughput during the period in which slowness is observed. Create a new 10 Gbps dedicated connection. Shift traffic from the existing dedicated connection to the new dedicated connection.

Answer: A

Explanation:

The correct answer is A. Here's why:

Identify the Problem: The question states that the 4 Gbps Direct Connect connection is getting saturated, causing performance issues. The company wants to identify the business unit responsible for the throughput spikes.

Monitoring Direct Connect: To pinpoint the source of the high throughput, we need to monitor the traffic on each private Virtual Interface (VIF) associated with each VPC.

CloudWatch Metrics: Amazon CloudWatch provides metrics for monitoring AWS resources. `VirtualInterfaceBpsEgress` and `VirtualInterfaceBpsIngress` metrics show the bits per second sent and received on each VIF, respectively. By analyzing these metrics during the period of slowness, the network engineer can determine which VIF (and therefore which business unit's VPC) is consuming the most bandwidth.

Addressing the Saturation: After identifying the culprit VIF, creating a new 10 Gbps dedicated connection and shifting traffic alleviates the congestion by providing more bandwidth. This avoids upgrading the existing connection, which can be disruptive.

Why other options are incorrect:

B: Upgrading the bandwidth of the existing connection will work, but migrating traffic to a separate connection is a much cleaner approach for large traffic spikes.

C and D: `ConnectionBpsIngress` and `ConnectionPpsEgress` metrics provide information about the overall connection, not individual VIFs. This will not help in identifying the problematic business unit. Additionally, upgrading to a hosted connection (Option C) is not correct. Hosted connections typically have lower bandwidth options compared to dedicated

connections. <https://docs.aws.amazon.com/directconnect/latest/UserGuide/monitoring-cloudwatch.html>

A software-as-a-service (SaaS) provider hosts its solution on Amazon EC2 instances within a VPC in the AWS Cloud. All of the provider's customers also have their environments in the AWS Cloud.

A recent design meeting revealed that the customers have IP address overlap with the provider's AWS deployment. The customers have stated that they will not share their internal IP addresses and that they do not want to connect to the provider's SaaS service over the internet.

Which combination of steps is part of a solution that meets these requirements? (Choose two.)

- A. Deploy the SaaS service endpoint behind a Network Load Balancer.
- B. Configure an endpoint service, and grant the customers permission to create a connection to the endpoint service.
- C. Deploy the SaaS service endpoint behind an Application Load Balancer.
- D. Configure a VPC peering connection to the customer VPCs. Route traffic through NAT gateways.
- E. Deploy an AWS Transit Gateway, and connect the SaaS VPC to it. Share the transit gateway with the customers. Configure routing on the transit gateway.

Answer: AB

Explanation:

The chosen solution (A and B) leverages AWS PrivateLink to securely connect the SaaS provider's service to its customers' VPCs, even with overlapping IP addresses, without exposing traffic to the public internet.

A. Deploy the SaaS service endpoint behind a Network Load Balancer (NLB): NLBs are crucial for PrivateLink endpoints. The NLB serves as the front end for the SaaS service. It distributes incoming traffic from the customers' VPCs to the backend EC2 instances hosting the SaaS application. NLBs operate at Layer 4, making them suitable for handling TCP and UDP traffic efficiently.

B. Configure an endpoint service, and grant the customers permission to create a connection to the endpoint service: The endpoint service is the core of the PrivateLink setup. The SaaS provider creates an endpoint service in their VPC, associated with the NLB. The endpoint service enables customers to create VPC endpoints in their own VPCs, allowing them to connect privately to the SaaS application. The SaaS provider controls which customers can access the service by granting permissions on the endpoint service. This eliminates the need for VPC peering or NAT gateways.

Let's analyze why the other options are less suitable:

C. Deploy the SaaS service endpoint behind an Application Load Balancer (ALB): ALBs are compatible with PrivateLink, but NLBs are generally preferred for non-HTTP(S) workloads or when direct TCP/UDP traffic is required. The question doesn't specify HTTP(S) requirements, making NLB a reasonable default.

D. Configure a VPC peering connection to the customer VPCs. Route traffic through NAT gateways: VPC peering would not work due to overlapping IP addresses, which is the core problem. NAT gateways would only provide internet access, which is explicitly forbidden by the customers. VPC peering also requires extensive routing configurations across multiple VPCs.

E. Deploy an AWS Transit Gateway, and connect the SaaS VPC to it. Share the transit gateway with the customers. Configure routing on the transit gateway: While Transit Gateway provides centralized connectivity, it doesn't solve the IP address overlap problem directly. Sharing a TGW also grants the customers broader access than needed; they only need access to the specific SaaS service. It does not handle the underlying overlap in IP address space. Using Transit Gateway might also incur unnecessary costs and complexity compared to PrivateLink for this specific use case.

Therefore, using a Network Load Balancer with an Endpoint Service provides a direct, secure, and isolated connection between the SaaS provider and its customers, resolving the IP address conflict and meeting the customers' requirements for private connectivity.

Authoritative Links:

AWS PrivateLink: <https://aws.amazon.com/privatelink/>

Question: 7

A network engineer is designing the architecture for a healthcare company's workload that is moving to the AWS Cloud. All data to and from the on-premises environment must be encrypted in transit. All traffic also must be inspected in the cloud before the traffic is allowed to leave the cloud and travel to the on-premises environment or to the internet.

The company will expose components of the workload to the internet so that patients can reserve appointments. The architecture must secure these components and protect them against DDoS attacks. The architecture also must provide protection against financial liability for services that scale out during a DDoS event.

Which combination of steps should the network engineer take to meet all these requirements for the workload? (Choose three.)

- A. Use Traffic Mirroring to copy all traffic to a fleet of traffic capture appliances.
- B. Set up AWS WAF on all network components.
- C. Configure an AWS Lambda function to create Deny rules in security groups to block malicious IP addresses.
- D. Use AWS Direct Connect with MACsec support for connectivity to the cloud.
- E. Use Gateway Load Balancers to insert third-party firewalls for inline traffic inspection.
- F. Configure AWS Shield Advanced and ensure that it is configured on all public assets.

Answer: DEF

Explanation:

The correct answer is DEF. Let's break down why each choice is either correct or incorrect:

D. Use AWS Direct Connect with MACsec support for connectivity to the cloud: This ensures that all data transmitted between the on-premises environment and AWS is encrypted in transit. MACsec provides layer 2 encryption for Direct Connect connections, fulfilling the requirement for encrypted communication between the on-premises and AWS environments. This choice directly addresses the requirement for encryption in transit. <https://aws.amazon.com/directconnect/>

E. Use Gateway Load Balancers to insert third-party firewalls for inline traffic inspection: Gateway Load Balancers (GWLB) allow you to insert third-party virtual appliances, such as firewalls, intrusion detection and prevention systems, and deep packet inspection systems, inline into the network traffic flow. This fulfills the requirement for traffic inspection before traffic leaves the cloud. A GWLB distributes traffic to a fleet of these appliances, providing scalability and high availability. <https://aws.amazon.com/gateway-load-balancer/>

F. Configure AWS Shield Advanced and ensure that it is configured on all public assets: AWS Shield Advanced provides enhanced DDoS protection, including protection against financial liability for scaling out during a DDoS attack. This directly addresses the requirement to protect against DDoS attacks and limit financial exposure. It automatically detects and mitigates sophisticated DDoS attacks targeted at your AWS resources. <https://aws.amazon.com/shield/>

Now, let's address why the other options are less suitable:

A. Use Traffic Mirroring to copy all traffic to a fleet of traffic capture appliances: Traffic mirroring is useful for monitoring and analyzing network traffic, but it does not provide inline inspection or prevent malicious traffic. It's a passive observation tool, not an active security control. It would not directly help with the inspection requirement.

B. Set up AWS WAF on all network components: While AWS WAF is an important security tool for protecting web applications from common web exploits, it doesn't provide comprehensive traffic inspection for all traffic, particularly traffic not destined for web applications. Also, placing it on "all network components" is vague

and possibly inefficient; it's more targeted to application entry points. WAF helps protecting web applications and APIs.

C. Configure an AWS Lambda function to create Deny rules in security groups to block malicious IP

addresses: This approach is not scalable or efficient for dealing with large-scale DDoS attacks. Security groups have limitations, and Lambda functions may not be responsive enough to mitigate rapidly evolving attacks. Also, manually managing security group rules with Lambda for DDoS mitigation is not a recommended practice. AWS Shield Advanced provides a more robust and automated solution.

Question: 8

CertyIQ

A retail company is running its service on AWS. The company's architecture includes Application Load Balancers (ALBs) in public subnets. The ALB target groups are configured to send traffic to backend Amazon EC2 instances in private subnets. These backend EC2 instances can call externally hosted services over the internet by using a NAT gateway.

The company has noticed in its billing that NAT gateway usage has increased significantly. A network engineer needs to find out the source of this increased usage.

Which options can the network engineer use to investigate the traffic through the NAT gateway? (Choose two.)

- A. Enable VPC flow logs on the NAT gateway's elastic network interface. Publish the logs to a log group in Amazon CloudWatch Logs. Use CloudWatch Logs Insights to query and analyze the logs.
- B. Enable NAT gateway access logs. Publish the logs to a log group in Amazon CloudWatch Logs. Use CloudWatch Logs Insights to query and analyze the logs.
- C. Configure Traffic Mirroring on the NAT gateway's elastic network interface. Send the traffic to an additional EC2 instance. Use tools such as tcpdump and Wireshark to query and analyze the mirrored traffic.
- D. Enable VPC flow logs on the NAT gateway's elastic network interface. Publish the logs to an Amazon S3 bucket. Create a custom table for the S3 bucket in Amazon Athena to describe the log structure. Use Athena to query and analyze the logs.
- E. Enable NAT gateway access logs. Publish the logs to an Amazon S3 bucket. Create a custom table for the S3 bucket in Amazon Athena to describe the log structure. Use Athena to query and analyze the logs.

Answer: AD

Explanation:

The correct answer is AD. Here's why:

A. Enable VPC flow logs on the NAT gateway's elastic network interface. Publish the logs to a log group in Amazon CloudWatch Logs. Use CloudWatch Logs Insights to query and analyze the logs.

VPC Flow Logs capture information about the IP traffic going to, from, and within your VPC. By enabling VPC Flow Logs on the NAT gateway's Elastic Network Interface (ENI), you capture detailed information about all traffic passing through the NAT gateway. This includes the source and destination IP addresses, ports, the number of bytes, and the action taken (ACCEPT or REJECT). CloudWatch Logs Insights provides a powerful query language to analyze these logs. You can quickly identify the EC2 instances generating the most traffic, the destination services they are accessing, and the total data transfer volume. This allows you to pinpoint the source of increased NAT gateway usage.

D. Enable VPC flow logs on the NAT gateway's elastic network interface. Publish the logs to an Amazon S3 bucket. Create a custom table for the S3 bucket in Amazon Athena to describe the log structure. Use Athena to query and analyze the logs.

This option also uses VPC Flow Logs, providing the same granular traffic information as option A. Instead of CloudWatch Logs Insights, it utilizes Amazon S3 for storage and Amazon Athena for querying. Storing the flow logs in S3 provides a cost-effective solution for long-term storage and enables you to use other AWS services for further analysis. Athena allows you to run SQL queries on the data stored in S3, making it easy to

analyze large datasets and identify patterns in the NAT gateway traffic. This is beneficial if you need to perform complex analysis or retain the logs for extended periods.

Why the other options are incorrect:

B & E. NAT gateway access logs: NAT Gateway access logs do not exist. VPC Flow Logs are the proper method for logging traffic that passes through the NAT gateway.

C. Traffic Mirroring: While Traffic Mirroring allows you to capture network traffic, it's less suitable for this scenario due to its complexity and performance implications. Setting up Traffic Mirroring involves configuring a traffic mirror source (NAT gateway ENI), a traffic mirror target (EC2 instance), and traffic mirror filters. The mirrored traffic duplicates all packets, which could impact the performance of the monitoring EC2 instance. Also, analyzing the captured packets using tools like tcpdump or Wireshark can be time-consuming and requires more manual effort compared to analyzing VPC Flow Logs with CloudWatch Logs Insights or Athena. Additionally, traffic mirroring is generally used for security and deep-packet inspection, not basic cost analysis, making it overkill for this scenario.

Authoritative links:

VPC Flow Logs: <https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs.html>

CloudWatch Logs Insights:

<https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/AnalyzingLogData.html>

Amazon Athena: <https://aws.amazon.com/athena/>

NAT Gateway: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html>

Traffic Mirroring: <https://docs.aws.amazon.com/vpc/latest/mirroring/what-is-traffic-mirroring.html>

Question: 9

CertyIQ

A banking company is successfully operating its public mobile banking stack on AWS. The mobile banking stack is deployed in a VPC that includes private subnets and public subnets. The company is using IPv4 networking and has not deployed or supported IPv6 in the environment. The company has decided to adopt a third-party service provider's API and must integrate the API with the existing environment. The service provider's API requires the use of IPv6.

A network engineer must turn on IPv6 connectivity for the existing workload that is deployed in a private subnet. The company does not want to permit IPv6 traffic from the public internet and mandates that the company's servers must initiate all IPv6 connectivity. The network engineer turns on IPv6 in the VPC and in the private subnets.

Which solution will meet these requirements?

A. Create an internet gateway and a NAT gateway in the VPC. Add a route to the existing subnet route tables to point IPv6 traffic to the NAT gateway.

B. Create an internet gateway and a NAT instance in the VPC. Add a route to the existing subnet route tables to point IPv6 traffic to the NAT instance.

C. Create an egress-only Internet gateway in the VPC. Add a route to the existing subnet route tables to point IPv6 traffic to the egress-only internet gateway.

D. Create an egress-only internet gateway in the VPC. Configure a security group that denies all inbound traffic. Associate the security group with the egress-only internet gateway.

Answer: C

Explanation:

The correct answer is C. Here's why:

The core requirement is to enable IPv6 connectivity from private subnets to an external IPv6-only API, ensuring no inbound IPv6 traffic from the public internet is allowed.

Egress-Only Internet Gateway (EIGW): An EIGW is designed precisely for this scenario. It allows instances in

private subnets to initiate outbound IPv6 connections to the internet but prevents the internet from initiating IPv6 connections to those instances. This aligns with the requirement that the company's servers initiate all IPv6 connectivity.

Route Table Configuration: By adding a route to the subnet route table that directs IPv6 traffic (::/0) to the EIGW, any IPv6 traffic originating from the instances in the subnet will be routed through the EIGW to the internet.

Why other options are incorrect:

A & B (IGW and NAT Gateway/Instance): These solutions are primarily for IPv4. While NAT Gateways support IPv4 NAT, they don't inherently handle IPv6 connectivity. IGW enables inbound and outbound internet communication, violating the requirement to block inbound internet IPv6 traffic. Also, NAT gateway does not support IPv6.

D (EIGW and Security Group): While using a security group to deny inbound traffic is good practice, associating a security group with an EIGW is not how EIGWs function. Security groups are associated with network interfaces of EC2 instances, not with gateways. The EIGW inherently blocks inbound traffic.

In summary: The EIGW provides the necessary outbound-only IPv6 connectivity for private subnets, fulfilling the company's requirement to integrate with the third-party IPv6 API while preventing inbound IPv6 traffic from the internet.

Supporting Links:

Egress-Only Internet Gateway: <https://docs.aws.amazon.com/vpc/latest/userguide/egress-only-internet-gateway.html>

Route Tables: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Route_Tables.html

Question: 10

CertyIQ

A company has deployed an AWS Network Firewall firewall into a VPC. A network engineer needs to implement a solution to deliver Network Firewall flow logs to the company's Amazon OpenSearch Service (Amazon Elasticsearch Service) cluster in the shortest possible time.

Which solution will meet these requirements?

- A. Create an Amazon S3 bucket. Create an AWS Lambda function to load logs into the Amazon OpenSearch Service (Amazon Elasticsearch Service) cluster. Enable Amazon Simple Notification Service (Amazon SNS) notifications on the S3 bucket to invoke the Lambda function. Configure flow logs for the firewall. Set the S3 bucket as the destination.
- B. Create an Amazon Kinesis Data Firehose delivery stream that includes the Amazon OpenSearch Service (Amazon Elasticsearch Service) cluster as the destination. Configure flow logs for the firewall. Set the Kinesis Data Firehose delivery stream as the destination for the Network Firewall flow logs.
- C. Configure flow logs for the firewall. Set the Amazon OpenSearch Service (Amazon Elasticsearch Service) cluster as the destination for the Network Firewall flow logs.
- D. Create an Amazon Kinesis data stream that includes the Amazon OpenSearch Service (Amazon Elasticsearch Service) cluster as the destination. Configure flow logs for the firewall. Set the Kinesis data stream as the destination for the Network Firewall flow logs.

Answer: B

Explanation:

The correct answer is B because it provides the most direct and efficient method for delivering Network Firewall flow logs to an Amazon OpenSearch Service cluster.

Here's a detailed justification:

Kinesis Data Firehose for Real-time Ingestion: Kinesis Data Firehose is designed specifically for reliably

streaming data to destinations like Amazon OpenSearch Service. It provides buffering, transformation, and loading capabilities, optimizing the data flow for analysis and storage.<https://aws.amazon.com/kinesis/data-firehose/>

Direct Integration: Firehose offers direct integration with Amazon OpenSearch Service as a destination, eliminating the need for intermediary steps or custom code for data ingestion. This simplifies the solution and minimizes latency.

Managed Service: Kinesis Data Firehose is a fully managed service. AWS handles the underlying infrastructure and scaling, reducing operational overhead.

Alternative A is Inefficient: Option A involves S3, Lambda, and SNS, which introduces unnecessary complexity and potential latency compared to the direct Firehose approach. The Lambda function requires custom code to load the logs into OpenSearch, adding development and maintenance burden.

Option C is Incorrect: Network Firewall flow logs cannot be directly sent to Amazon OpenSearch Service. A data streaming solution is needed.

Option D is Incorrect: Kinesis Data Streams require more manual management compared to Firehose, including provisioning and scaling consumers to process the data before loading it to OpenSearch. Data Streams offer real time but at the cost of increased management. For just moving logs, data firehose is better suited.

In summary, utilizing Kinesis Data Firehose offers the quickest and most straightforward path to delivering Network Firewall flow logs to Amazon OpenSearch Service. It eliminates the need for custom code and intermediary services, simplifying the process and reducing latency.

Question: 11

CertyIQ

A company is using custom DNS servers that run BIND for name resolution in its VPCs. The VPCs are deployed across multiple AWS accounts that are part of the same organization in AWS Organizations. All the VPCs are connected to a transit gateway. The BIND servers are running in a central VPC and are configured to forward all queries for an on-premises DNS domain to DNS servers that are hosted in an on-premises data center. To ensure that all the VPCs use the custom DNS servers, a network engineer has configured a VPC DHCP options set in all the VPCs that specifies the custom DNS servers to be used as domain name servers. Multiple development teams in the company want to use Amazon Elastic File System (Amazon EFS). A development team has created a new EFS file system but cannot mount the file system to one of its Amazon EC2 instances. The network engineer discovers that the EC2 instance cannot resolve the IP address for the EFS mount point fs-33444567d.efs.us-east-1.amazonaws.com. The network engineer needs to implement a solution so that development teams throughout the organization can mount EFS file systems. Which combination of steps will meet these requirements? (Choose two.)

- A. Configure the BIND DNS servers in the central VPC to forward queries for efs.us-east-1.amazonaws.com to the Amazon provided DNS server (169.254.169.253).
- B. Create an Amazon Route 53 Resolver outbound endpoint in the central VPC. Update all the VPC DHCP options sets to use AmazonProvidedDNS for name resolution.
- C. Create an Amazon Route 53 Resolver inbound endpoint in the central VPC. Update all the VPC DHCP options sets to use the Route 53 Resolver inbound endpoint in the central VPC for name resolution.
- D. Create an Amazon Route 53 Resolver rule to forward queries for the on-premises domain to the on-premises DNS servers. Share the rule with the organization by using AWS Resource Access Manager (AWS RAM). Associate the rule with all the VPCs.
- E. Create an Amazon Route 53 private hosted zone for the efs.us-east-1.amazonaws.com domain. Associate the private hosted zone with the VPC where the EC2 instance is deployed. Create an A record for fs-33444567d.efs.us-east-1.amazonaws.com in the private hosted zone. Configure the A record to return the mount target of the EFS mount point.

Answer: BD

Explanation:

Here's a detailed justification for why options B and D are the correct choices and why the other options are incorrect for resolving the EFS mounting issue across multiple AWS accounts using a transit gateway and custom DNS.

Understanding the Problem:

The core issue is that EC2 instances in multiple VPCs cannot resolve the EFS mount point (fs-33444567d.efs.us-east-1.amazonaws.com). This resolution needs to occur across multiple AWS accounts. Currently, the VPCs use custom DNS servers (BIND) that forward on-premises DNS queries but aren't configured for resolving AWS service endpoints like EFS.

Why B and D are correct:

B. Create an Amazon Route 53 Resolver outbound endpoint in the central VPC. Update all the VPC DHCP options sets to use AmazonProvidedDNS for name resolution.

This approach directly addresses the inability to resolve AWS service endpoints. A Route 53 Resolver outbound endpoint allows you to forward DNS queries from your VPC to DNS servers outside the VPC (in this case, to Amazon's DNS servers).

By setting the DHCP options set to AmazonProvidedDNS, you ensure that all VPCs now use the Amazon-provided DNS, which can resolve AWS service endpoints like those for EFS. This eliminates the need for custom BIND configuration for these AWS-specific zones.

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver.html>

D. Create an Amazon Route 53 Resolver rule to forward queries for the on-premises domain to the on-premises DNS servers. Share the rule with the organization by using AWS Resource Access Manager (AWS RAM). Associate the rule with all the VPCs.

Since the company still needs the VPCs to resolve queries for the on-premise domain, we can use the Route 53 Resolver to forward those queries from VPCs to the on-premise DNS servers. Using AWS RAM to share the rules allows the rule to be used across different AWS accounts within the organization. This will ensure that on-premise DNS queries are still resolved after the VPCs DHCP options sets have been updated to use the AmazonProvidedDNS.

<https://docs.aws.amazon.com/ram/latest/userguide/what-is.html>

Why the other options are incorrect:

A. Configure the BIND DNS servers in the central VPC to forward queries for efs.us-east-1.amazonaws.com to the Amazon provided DNS server (169.254.169.253).

While technically feasible, this approach is less scalable and maintainable than using Route 53 Resolver. You'd have to manage BIND configuration, and it introduces a single point of failure in the central VPC. Plus, forwarding to 169.254.169.253 directly is generally discouraged; using AmazonProvidedDNS is the AWS-recommended method.

Also, it won't address the on-premise DNS resolutions.

C. Create an Amazon Route 53 Resolver inbound endpoint in the central VPC. Update all the VPC DHCP options sets to use the Route 53 Resolver inbound endpoint in the central VPC for name resolution.

Route 53 Resolver inbound endpoints are used to forward DNS queries from on-premises networks to AWS. This is the opposite of what's needed here; you want AWS resources to resolve AWS service endpoints.

E. Create an Amazon Route 53 private hosted zone for the efs.us-east-1.amazonaws.com domain. Associate

the private hosted zone with the VPC where the EC2 instance is deployed. Create an A record for fs-33444567d.efs.us-east-1.amazonaws.com in the private hosted zone. Configure the A record to return the mount target of the EFS mount point.

This approach only addresses the resolution issue for one VPC. It doesn't scale to the requirement that development teams throughout the organization be able to mount EFS file systems. Also, manually creating A records and keeping them in sync is complex and error-prone, given that EFS mount targets can change. The AmazonProvidedDNS automatically resolves these.

In summary, the combination of using AmazonProvidedDNS (option B) for resolving AWS service endpoints like EFS and using Route 53 Resolver rules for on-premises DNS resolution, which is shared across the organization using AWS RAM (option D), provides the most scalable, manageable, and robust solution.

Question: 12

CertyIQ

An ecommerce company is hosting a web application on Amazon EC2 instances to handle continuously changing customer demand. The EC2 instances are part of an Auto Scaling group. The company wants to implement a solution to distribute traffic from customers to the EC2 instances. The company must encrypt all traffic at all stages between the customers and the application servers. No decryption at intermediate points is allowed. Which solution will meet these requirements?

- A. Create an Application Load Balancer (ALB). Add an HTTPS listener to the ALB. Configure the Auto Scaling group to register instances with the ALB's target group.
- B. Create an Amazon CloudFront distribution. Configure the distribution with a custom SSL/TLS certificate. Set the Auto Scaling group as the distribution's origin.
- C. Create a Network Load Balancer (NLB). Add a TCP listener to the NLB. Configure the Auto Scaling group to register instances with the NLB's target group.
- D. Create a Gateway Load Balancer (GLB). Configure the Auto Scaling group to register instances with the GLB's target group.

Answer: C

Explanation:

C. Create a Network Load Balancer (NLB). Add a TCP listener to the NLB. Configure the Auto Scaling group to register instances with the NLB's target group.

NLB operates at Layer 4 and does not terminate TLS.

Encrypted traffic passes through unchanged to EC2 instances.

This satisfies the end-to-end encryption without decryption in the middle requirement.

Question: 13

CertyIQ

A company has two on-premises data center locations. There is a company-managed router at each data center. Each data center has a dedicated AWS Direct Connect connection to a Direct Connect gateway through a private virtual interface. The router for the first location is advertising 110 routes to the Direct Connect gateway by using BGP, and the router for the second location is advertising 60 routes to the Direct Connect gateway by using BGP. The Direct Connect gateway is attached to a company VPC through a virtual private gateway. A network engineer receives reports that resources in the VPC are not reachable from various locations in either data center. The network engineer checks the VPC route table and sees that the routes from the first data center location are not being populated into the route table. The network engineer must resolve this issue in the most operationally efficient manner. What should the network engineer do to meet these requirements?

- A.Remove the Direct Connect gateway, and create a new private virtual interface from each company router to the virtual private gateway of the VPC.
- B.Change the router configurations to summarize the advertised routes.
- C.Open a support ticket to increase the quota on advertised routes to the VPC route table.
- D.Create an AWS Transit Gateway. Attach the transit gateway to the VPC, and connect the Direct Connect gateway to the transit gateway.

Answer: B

Explanation:

The correct answer is B: Change the router configurations to summarize the advertised routes. Here's why:

The core problem is that the VPC route table is not being populated with routes from one of the on-premises locations, specifically the first data center advertising 110 routes. This points to a limit on the number of routes that a VPC route table can accept from a Direct Connect gateway. While AWS doesn't publicly document hard limits for propagated routes from a DXGW to a VPC route table, exceeding a reasonable number of routes is a common issue.

Summarizing routes is the most operationally efficient way to solve this. By aggregating multiple smaller, specific routes into fewer, larger, more general routes, the total number of routes advertised to the Direct Connect gateway and subsequently propagated to the VPC route table is reduced. This allows the essential reachability information to be conveyed without exceeding any potential route limits. This solution addresses the problem at the source of the route advertisements (the on-premises routers) without requiring infrastructure changes or significant service disruptions.

Option A is incorrect because removing the Direct Connect gateway and creating private virtual interfaces directly to the VPG is not best practice and removes the central management capability the Direct Connect Gateway provides. This would require reconfiguring BGP sessions on the VPC side and create more configuration and management overhead.

Option C, opening a support ticket, might seem plausible if a hard quota was definitively the cause. However, summarizing routes is a better practice and more controllable solution, and you don't have confirmation of an actual quota limit, so it is premature to rely on AWS increasing quotas. Furthermore, even if AWS granted an increased quota, the underlying problem of overly granular routing isn't addressed.

Option D, using Transit Gateway, is a valid architectural choice for more complex scenarios and could solve the immediate problem. However, it introduces additional complexity and cost. For this relatively simple scenario, the most operationally efficient solution is route summarization. TGW should be considered when you need more advanced routing policies, shared services VPC connectivity, or more dynamic scaling than DXGW + VPC VPG provides.

Route summarization offers a targeted solution that directly addresses the route limits without impacting other parts of the network and is the most straightforward approach in this situation.

For further research, consult the AWS Direct Connect documentation and networking best practices, though explicit VPC route limits related to DXGWs aren't clearly documented.

AWS Direct Connect: <https://aws.amazon.com/directconnect/>

AWS Transit Gateway: <https://aws.amazon.com/transit-gateway/>

Question: 14

CertyIQ

A company has expanded its network to the AWS Cloud by using a hybrid architecture with multiple AWS accounts. The company has set up a shared AWS account for the connection to its on-premises data centers and the company offices. The workloads consist of private web-based services for internal use. These services run in

different AWS accounts. Office-based employees consume these services by using a DNS name in an on-premises DNS zone that is named `example.internal`.

The process to register a new service that runs on AWS requires a manual and complicated change request to the internal DNS. The process involves many teams.

The company wants to update the DNS registration process by giving the service creators access that will allow them to register their DNS records. A network engineer must design a solution that will achieve this goal. The solution must maximize cost-effectiveness and must require the least possible number of configuration changes. Which combination of steps should the network engineer take to meet these requirements? (Choose three.)

- A. Create a record for each service in its local private hosted zone (`serviceA.account1.aws.example.internal`). Provide this DNS record to the employees who need access.
- B. Create an Amazon Route 53 Resolver inbound endpoint in the shared account VPC. Create a conditional forwarder for a domain named `aws.example.internal` on the on-premises DNS servers. Set the forwarding IP addresses to the inbound endpoint's IP addresses that were created.
- C. Create an Amazon Route 53 Resolver rule to forward any queries made to `onprem.example.internal` to the on-premises DNS servers.
- D. Create an Amazon Route 53 private hosted zone named `aws.example.internal` in the shared AWS account to resolve queries for this domain.
- E. Launch two Amazon EC2 instances in the shared AWS account. Install BIND on each instance. Create a DNS conditional forwarder on each BIND server to forward queries for each subdomain under `aws.example.internal` to the appropriate private hosted zone in each AWS account. Create a conditional forwarder for a domain named `aws.example.internal` on the on-premises DNS servers. Set the forwarding IP addresses to the IP addresses of the BIND servers.
- F. Create a private hosted zone in the shared AWS account for each account that runs the service. Configure the private hosted zone to contain `aws.example.internal` in the domain (`account1.aws.example.internal`). Associate the private hosted zone with the VPC that runs the service and the shared account VPC.

Answer: BDF

Explanation:

The correct solution, **BDF**, efficiently addresses the company's DNS registration challenges by leveraging Route 53 Resolver capabilities. Here's why:

B. Create an Amazon Route 53 Resolver inbound endpoint...: An inbound endpoint acts as a gateway for on-premises DNS servers to forward queries to Route 53 Resolver in the AWS shared services account. Creating a conditional forwarder on the on-premises DNS server for `aws.example.internal` and directing it to the inbound endpoint ensures that all requests for AWS services are properly routed to AWS. This avoids direct exposure of AWS infrastructure IP addresses to internal DNS infrastructure. (Source: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver.html>)

D. Create an Amazon Route 53 private hosted zone named `aws.example.internal`...: This establishes a central location for managing DNS records related to AWS-hosted services within the shared account. It's the root of the delegated AWS DNS namespace. This enables more granular control and allows for further sub-delegation of authority. This is a crucial part of the solution which allows the use of private hosted zones to resolve the names of the internal services.

F. Create a private hosted zone in the shared AWS account for each account that runs the service...: By creating separate private hosted zones (e.g., `account1.aws.example.internal`) for each AWS account, the service creators gain the authority to manage their own DNS records. Associating these hosted zones with both the service VPC and the shared account VPC enables DNS resolution from both locations, ensuring internal employees can access the services using their desired DNS names.

Why other options are incorrect:

A: Directly providing the service-specific DNS records to employees lacks scalability and doesn't streamline the DNS registration process. It also doesn't address centralized management.

C: Creating a Resolver rule to forward `onprem.example.internal` to on-premises is already the default behavior.

E: Launching EC2 instances with BIND adds unnecessary complexity, operational overhead, and costs. Route 53 Resolver offers a managed and highly available DNS solution. This is a legacy implementation and it will introduce more configuration changes.

In essence, the BDF combination establishes a streamlined and cost-effective DNS delegation model. Service teams get authority over their DNS records under `aws.example.internal` through private hosted zones, which reduces the manual change requests. Route 53 Resolver, via an inbound endpoint, connects the on-premises network to the delegated AWS DNS namespace. This provides the employees with access to services hosted on AWS.

Question: 15

CertyIQ

A company has multiple AWS accounts. Each account contains one or more VPCs. A new security guideline requires the inspection of all traffic between VPCs.

The company has deployed a transit gateway that provides connectivity between all VPCs. The company also has deployed a shared services VPC with Amazon EC2 instances that include IDS services for stateful inspection. The EC2 instances are deployed across three Availability Zones. The company has set up VPC associations and routing on the transit gateway. The company has migrated a few test VPCs to the new solution for traffic inspection. Soon after the configuration of routing, the company receives reports of intermittent connections for traffic that crosses Availability Zones.

What should a network engineer do to resolve this issue?

- A. Modify the transit gateway VPC attachment on the shared services VPC by enabling cross-Availability Zone load balancing.
- B. Modify the transit gateway VPC attachment on the shared services VPC by enabling appliance mode support.
- C. Modify the transit gateway by selecting VPN equal-cost multi-path (ECMP) routing support.
- D. Modify the transit gateway by selecting multicast support.

Answer: B

Explanation:

The issue described points to asymmetric routing problems occurring due to the stateful nature of the IDS appliances in the shared services VPC. When traffic crosses Availability Zones, the return traffic might not be routed back to the same IDS instance that initially processed the outbound traffic, leading to connection failures because the stateful firewall in the IDS appliance doesn't recognize the return traffic as part of an established connection.

Option B, enabling appliance mode support on the transit gateway VPC attachment for the shared services VPC, addresses this asymmetric routing problem. Appliance mode ensures that traffic flows are symmetrical, meaning that traffic originating from a source will always return through the same appliance. This resolves the issue of return traffic going to a different IDS instance, which would cause the stateful inspection to fail. By enabling appliance mode, the transit gateway ensures that both forward and return traffic for a connection are routed through the same IDS appliance, maintaining connection state and preventing intermittent connection issues.

Option A, enabling cross-Availability Zone load balancing on the transit gateway VPC attachment is incorrect. Cross-AZ load balancing distributes traffic across instances in different AZs, but doesn't guarantee symmetric routing, which is required for stateful inspection.

Option C, enabling VPN ECMP routing, is not relevant in this scenario. ECMP is for distributing traffic across multiple VPN connections, not for managing traffic flow through IDS appliances in a shared services VPC.

Option D, enabling multicast support, is also not relevant as multicast is used for one-to-many communication, which isn't the problem here. The issue is with the routing of unicast traffic and stateful inspection.

Therefore, enabling appliance mode support ensures consistent routing and allows the stateful IDS appliances to properly function, resolving the intermittent connection issues.

Reference link: <https://docs.aws.amazon.com/vpc/latest/tgw/tgw-appliance-mode.html>

Question: 16

CertyIQ

A company is using a NAT gateway to allow internet connectivity for private subnets in a VPC in the us-west-2 Region. After a security audit, the company needs to remove the NAT gateway. In the private subnets, the company has resources that use the unified Amazon CloudWatch agent. A network engineer must create a solution to ensure that the unified CloudWatch agent continues to work after the removal of the NAT gateway.

Which combination of steps should the network engineer take to meet these requirements? (Choose three.)

- A. Validate that private DNS is enabled on the VPC by setting the enableDnsHostnames VPC attribute and the enableDnsSupport VPC attribute to true.
- B. Create a new security group with an entry to allow outbound traffic that uses the TCP protocol on port 443 to destination 0.0.0.0/0
- C. Create a new security group with entries to allow inbound traffic that uses the TCP protocol on port 443 from the IP prefixes of the private subnets.
- D. Create the following interface VPC endpoints in the VPC: com.amazonaws.us-west-2.logs and com.amazonaws.us-west-2.monitoring. Associate the new security group with the endpoint network interfaces.
- E. Create the following interface VPC endpoint in the VPC: com.amazonaws.us-west-2.cloudwatch. Associate the new security group with the endpoint network interfaces.
- F. Associate the VPC endpoint or endpoints with route tables that the private subnets use.

Answer: ACD

Explanation:

The correct combination of steps is **ACD**. Here's why:

A. Validate that private DNS is enabled on the VPC: CloudWatch agent communication relies on resolving DNS names for the AWS services. enableDnsHostnames and enableDnsSupport must be enabled at the VPC level to allow instances within the VPC to resolve AWS service endpoints through the Amazon DNS server. This allows the CloudWatch agent to find the proper endpoints even without direct internet access. Without this, the agent will fail to resolve the CloudWatch endpoint, and metrics will not be sent.

<https://docs.aws.amazon.com/vpc/latest/userguide/vpc-dns.html>

C. Create a new security group with entries to allow inbound traffic that uses the TCP protocol on port 443 from the IP prefixes of the private subnets: While not strictly necessary for the CloudWatch agent to send metrics, VPC endpoints (created in the next step) require a security group. The endpoint's security group needs to allow traffic from the resources using the endpoint. The most secure approach is to limit traffic only from the IP ranges of the private subnets. This ensures only resources within the private subnets can access the CloudWatch endpoints. This is not an outbound rule on the instances. It is an inbound rule on the VPC endpoint.

D. Create the following interface VPC endpoints in the VPC: com.amazonaws.us-west-2.logs and com.amazonaws.us-west-2.monitoring. Associate the new security group with the endpoint network interfaces: Interface VPC endpoints provide private connectivity to AWS services without requiring internet gateways, NAT devices, or VPN connections. Creating endpoints for com.amazonaws.us-west-2.logs and com.amazonaws.us-west-2.monitoring allows the CloudWatch agent to securely send logs and metrics directly to CloudWatch over the AWS network. Associating the security group (created in step C) with the endpoint network interfaces controls the traffic flow to the endpoint. These are the correct service names.

Why other options are incorrect:

B: Creating a new security group with an outbound rule to 0.0.0.0/0:443 would be applicable if the instances were still using the NAT Gateway. Since the goal is to remove the NAT gateway, this option is incorrect.

E: com.amazonaws.us-west-2.cloudwatch is not the correct endpoint. The required endpoints are com.amazonaws.us-west-2.logs and com.amazonaws.us-west-2.monitoring.

F: The VPC endpoints are associated with the subnets via route tables during the endpoint creation. There isn't a separate "associate with route tables" step that the engineer performs after endpoint creation. The wizard in the console or the CLI command will ask for which route table the endpoint will affect.

Question: 17

CertyIQ

An international company provides early warning about tsunamis. The company plans to use IoT devices to monitor sea waves around the world. The data that is collected by the IoT devices must reach the company's infrastructure on AWS as quickly as possible. The company is using three operation centers around the world. Each operation center is connected to AWS through its own AWS Direct Connect connection. Each operation center is connected to the internet through at least two upstream internet service providers.

The company has its own provider-independent (PI) address space. The IoT devices use TCP protocols for reliable transmission of the data they collect. The IoT devices have both landline and mobile internet connectivity. The infrastructure and the solution will be deployed in multiple AWS Regions. The company will use Amazon Route 53 for DNS services.

A network engineer needs to design connectivity between the IoT devices and the services that run in the AWS Cloud.

Which solution will meet these requirements with the HIGHEST availability?

- A. Set up an Amazon CloudFront distribution with origin failover. Create an origin group for each Region where the solution is deployed.
- B. Set up Route 53 latency-based routing. Add latency alias records. For the latency alias records, set the value of Evaluate Target Health to Yes.
- C. Set up an accelerator in AWS Global Accelerator. Configure Regional endpoint groups and health checks.
- D. Set up Bring Your Own IP (BYOIP) addresses. Use the same PI addresses for each Region where the solution is deployed.

Answer: C

Explanation:

The correct answer is **C: Set up an accelerator in AWS Global Accelerator. Configure Regional endpoint groups and health checks.**

Here's a detailed justification:

AWS Global Accelerator is specifically designed to improve the availability and performance of applications for global users. It achieves this by directing traffic to the optimal endpoint based on network conditions and endpoint health. In this scenario, the company has IoT devices scattered globally and wants the data to reach its AWS infrastructure as quickly and reliably as possible across multiple regions.

Global Accelerator uses the AWS global network to optimize the path from the IoT devices to the nearest healthy endpoint in one of the company's AWS Regions. It accomplishes this via static anycast IPs that serve as the entry point for traffic. Because the company has operation centers connected via Direct Connect in different Regions, Global Accelerator can direct traffic to the closest operation center with healthy endpoints, even if one Region or Direct Connect link experiences issues. Endpoint groups are configured for each Region, allowing traffic to be dynamically routed to the healthy Region. Health checks are used to monitor the health of the application endpoints within each Region.

Option A, Amazon CloudFront with origin failover, is primarily designed for caching static and dynamic content closer to users for improved performance and is less suited for TCP-based real-time data ingestion from IoT devices. It's better suited for distributing content than reliably accepting and routing data.

Option B, Route 53 latency-based routing, makes routing decisions based on latency. While it's useful for routing users to the Region with the lowest latency, it doesn't provide the same level of fault tolerance and global traffic management as Global Accelerator, particularly in the presence of multiple network paths (landline and mobile). More over, R53 is a DNS service, which relies on resolution and propagation.

Option D, Bring Your Own IP (BYOIP) addresses, allows the company to use its own IP address range with AWS. While useful for IP address management and reputation, it doesn't directly address the requirement for high availability and optimized routing to the nearest healthy endpoint. BYOIP simply lets you use your own IP addresses in AWS, it doesn't inherently improve routing or availability. It would need to be combined with a routing solution, but Global Accelerator is a more direct and effective solution. Global Accelerator provides both a stable entry point and global routing, maximizing availability and minimizing latency. **Authoritative links:**

AWS Global Accelerator: <https://aws.amazon.com/global-accelerator/>

Route 53 Routing Policies: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/routing-policy.html>

Amazon CloudFront Origin Failover:

https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/high_availability_origin_failover.html

AWS BYOIP: <https://aws.amazon.com/byoip/>

Question: 18

CertyIQ

A company is planning a migration of its critical workloads from an on-premises data center to Amazon EC2 instances. The plan includes a new 10 Gbps AWS Direct Connect dedicated connection from the on-premises data center to a VPC that is attached to a transit gateway. The migration must occur over encrypted paths between the on-premises data center and the AWS Cloud.

Which solution will meet these requirements while providing the HIGHEST throughput?

A. Configure a public VIF on the Direct Connect connection. Configure an AWS Site-to-Site VPN connection to the transit gateway as a VPN attachment.

B. Configure a transit VIF on the Direct Connect connection. Configure an IPsec VPN connection to an EC2 instance that is running third-party VPN software.

C. Configure MACsec for the Direct Connect connection. Configure a transit VIF to a Direct Connect gateway that is associated with the transit gateway.

D. Configure a public VIF on the Direct Connect connection. Configure two AWS Site-to-Site VPN connections to the transit gateway. Enable equal-cost multi-path (ECMP) routing.

Answer: C

Explanation:

The correct answer is C because it provides a high-throughput, encrypted connection between the on-premises data center and AWS, specifically tailored for the given scenario.

Here's a detailed justification:

The core requirement is encrypted traffic over a high-bandwidth connection between on-premises and AWS. MACsec provides hardware-based encryption at the data link layer (Layer 2) and is specifically designed for Direct Connect connections. This encryption method minimizes overhead and provides significantly higher throughput compared to IPsec VPNs, which operate at Layer 3.

Configuring a transit VIF is essential for connecting the Direct Connect connection to the transit gateway. The transit gateway acts as a central hub, allowing the on-premises network to connect to multiple VPCs within

AWS through the Direct Connect connection. Associating the Direct Connect gateway with the transit gateway allows the propagation of routes between your on-premises network and the AWS environment, routed through the Transit Gateway.

Option A uses a public VIF and Site-to-Site VPN. While Site-to-Site VPN provides encryption, it introduces significant overhead due to IPsec encapsulation and software processing. This limits throughput and would not maximize the 10 Gbps Direct Connect connection. A public VIF isn't directly routed to the transit gateway but over public endpoints which defeats the purpose of using direct connect.

Option B proposes an IPsec VPN to an EC2 instance. This approach incurs considerable overhead, similar to Option A, due to IPsec and the limitations of the EC2 instance's processing capacity. Additionally, managing and scaling a VPN solution on EC2 instances is complex and may introduce availability concerns. The transit VIF to a transit gateway provides a more seamless and managed approach.

Option D tries to improve throughput using ECMP with two Site-to-Site VPN connections. While ECMP can increase aggregate bandwidth, the inherent overhead of IPsec remains, and it doesn't leverage the full potential of the Direct Connect connection and isn't as efficient as MACsec. Furthermore, managing two VPN connections adds operational complexity. A public VIF isn't directly routed to the transit gateway but over public endpoints which defeats the purpose of using direct connect.

In summary, MACsec on Direct Connect with a transit VIF provides the optimal balance of encryption and high throughput, directly addressing the stated requirements for migrating critical workloads.

Relevant documentation:

AWS Direct Connect MACsec: <https://aws.amazon.com/directconnect/macsec/>

AWS Transit Gateway: <https://aws.amazon.com/transit-gateway/>

AWS Direct Connect Gateways: <https://docs.aws.amazon.com/directconnect/latest/UserGuide/direct-connect-gateways-intro.html>

Question: 19

CertyIQ

A network engineer must develop an AWS CloudFormation template that can create a virtual private gateway, a customer gateway, a VPN connection, and static routes in a route table. During testing of the template, the network engineer notes that the CloudFormation template has encountered an error and is rolling back. What should the network engineer do to resolve the error?

- A. Change the order of resource creation in the CloudFormation template.
- B. Add the DependsOn attribute to the resource declaration for the virtual private gateway. Specify the route table entry resource.
- C. Add a wait condition in the template to wait for the creation of the virtual private gateway.
- D. Add the DependsOn attribute to the resource declaration for the route table entry. Specify the virtual private gateway resource.

Answer: D

Explanation:

The CloudFormation template is failing because the route table entry (static route) is likely being created before the virtual private gateway (VGW) is fully provisioned. The route entry creation depends on the VGW being available. CloudFormation attempts to create resources in parallel, and without explicit dependencies, it might try to create the route before the VGW is ready, leading to an error and rollback.

Option D correctly addresses this dependency issue. By adding the DependsOn attribute to the route table entry resource declaration and specifying the VGW resource, we explicitly tell CloudFormation to create the VGW first. CloudFormation will then wait until the VGW is successfully created before proceeding to create

the route table entry that depends on it. This enforces the correct order of operations and resolves the dependency conflict.

Option A is incorrect because simply changing the order of resource declarations in the template doesn't guarantee the order of resource creation. CloudFormation can still attempt to create resources in parallel unless explicit dependencies are defined.

Option B is incorrect because the route table entry depends on the VGW, not the other way around. Specifying the route table entry resource in the DependsOn attribute of the VGW would create a circular dependency, which is not what we need. The route table entry needs to know about the VGW, not the VGW needing to know about the route table entry.

Option C, adding a wait condition, is generally not recommended for dependencies between AWS resources created within the same CloudFormation stack. DependsOn is a cleaner and more appropriate solution for expressing resource dependencies. Wait conditions are better suited for situations where you need to wait for an external process or service to complete before proceeding.

Therefore, the most effective solution is to explicitly define the dependency between the route table entry and the VGW using the DependsOn attribute.

Refer to the following AWS documentation for further details on DependsOn attribute and CloudFormation resource dependencies:

[AWS CloudFormation DependsOn Attribute](#)

[AWS CloudFormation Resource Creation Options](#) (for understanding wait conditions and why they are not the ideal choice here)

Question: 20

CertyIQ

A company operates its IT services through a multi-site hybrid infrastructure. The company deploys resources on AWS in the us-east-1 Region and in the eu-west-2 Region. The company also deploys resources in its own data centers that are located in the United States (US) and in the United Kingdom (UK). In both AWS Regions, the company uses a transit gateway to connect 15 VPCs to each other. The company has created a transit gateway peering connection between the two transit gateways. The VPC CIDR blocks do not overlap with each other or with IP addresses used within the data centers. The VPC CIDR prefixes can also be aggregated either on a Regional level or for the company's entire AWS environment.

The data centers are connected to each other by a private WAN connection. IP routing information is exchanged dynamically through Interior BGP (iBGP) sessions. The data centers maintain connectivity to AWS through one AWS Direct Connect connection in the US and one Direct Connect connection in the UK. Each Direct Connect connection is terminated on a Direct Connect gateway and is associated with a local transit gateway through a transit VIF.

Traffic follows the shortest geographical path from source to destination. For example, packets from the UK data center that are targeted to resources in eu-west-2 travel across the local Direct Connect connection. In cases of cross-Region data transfers, such as from the UK data center to VPCs in us-east-1, the private WAN connection must be used to minimize costs on AWS. A network engineer has configured each transit gateway association on the Direct Connect gateway to advertise VPC-specific CIDR IP prefixes only from the local Region. The routes toward the other Region must be learned through BGP from the routers in the other data center in the original, non-aggregated form.

The company recently experienced a problem with cross-Region data transfers because of issues with its private WAN connection. The network engineer needs to modify the routing setup to prevent similar interruptions in the future. The solution cannot modify the original traffic routing goal when the network is operating normally.

Which modifications will meet these requirements? (Choose two.)

A. Remove all the VPC CIDR prefixes from the list of subnets advertised through the local Direct Connect connection. Add the company's entire AWS environment aggregate route to the list of subnets advertised through the local Direct Connect connection.

B. Add the CIDR prefixes from the other Region VPCs and the local VPC CIDR blocks to the list of subnets advertised through the local Direct Connect connection. Configure data center routers to make routing decisions based on the BGP communities received.

C. Add the aggregate IP prefix for the other Region and the local VPC CIDR blocks to the list of subnets advertised through the local Direct Connect connection.

D. Add the aggregate IP prefix for the company's entire AWS environment and the local VPC CIDR blocks to the list of subnets advertised through the local Direct Connect connection.

E. Remove all the VPC CIDR prefixes from the list of subnets advertised through the local Direct Connect connection. Add both Regional aggregate IP prefixes to the list of subnets advertised through the Direct Connect connection on both sides of the network. Configure data center routers to make routing decisions based on the BGP communities received.

Answer: CE

Explanation:

Here's a breakdown of why options C and E are the correct choices for ensuring cross-Region data transfers continue during private WAN issues, without altering normal routing behavior:

Understanding the Requirements

The core goal is to maintain shortest-path routing when the private WAN is healthy and to failover to AWS infrastructure in case the WAN link fails. The solution must not impact the original traffic routing when the network is operating normally.

Why C is Correct

Adding the aggregate IP prefix for the other Region and the local VPC CIDR blocks to the list of subnets advertised through the local Direct Connect connection: This means the Direct Connect gateway (and hence, the local Direct Connect connection) will advertise aggregate routes for the VPCs in both the local region and the remote region. This enables the data center routers to learn routes to the remote region's VPCs via Direct Connect in addition to the private WAN.

Why E is Correct

Remove all the VPC CIDR prefixes from the list of subnets advertised through the local Direct Connect connection. This stops advertising specific VPC CIDRs via the local Direct Connect connection, which were originally limiting traffic to preferred paths.

Add both Regional aggregate IP prefixes to the list of subnets advertised through the Direct Connect connection on both sides of the network. This means the Direct Connect gateway (and hence, the local Direct Connect connection) will advertise the aggregate routes for the VPCs in both the local region and the remote region. This enables the data center routers to learn routes to the remote region's VPCs via Direct Connect in addition to the private WAN.

Configure data center routers to make routing decisions based on the BGP communities received. By leveraging BGP communities, the data center routers can be configured to prefer the routes learned via the private WAN (e.g., by assigning a higher local preference to routes received from the WAN). If the WAN fails, the routes learned via Direct Connect (with lower preference) will then be used. This ensures the traffic fails over to the AWS backbone without disrupting the original preferred path.

Why other options are Incorrect

A: Advertising only the entire AWS environment aggregate route through Direct Connect would mean that all traffic, even within the same region, could potentially be routed through the Direct Connect to the other region, which violates the shortest path objective during normal operations.

B: This approach doesn't solve the failover problem because data center routers might still prefer routes advertised from the local region via the Direct Connect.

D: Advertising the entire AWS environment aggregate route through Direct Connect would cause all traffic destined to AWS to route via the Direct Connect connection instead of the shorter path over the WAN. This removes the shortest path objective during normal operations.

Supporting Resources

AWS Transit Gateway: <https://aws.amazon.com/transit-gateway/>

AWS Direct Connect: <https://aws.amazon.com/directconnect/>

BGP Communities: <https://datatracker.ietf.org/doc/html/rfc1997> (RFC defining BGP Communities)

Question: 21

CertyIQ

A company's network engineer needs to design a new solution to help troubleshoot and detect network anomalies. The network engineer has configured Traffic Mirroring. However, the mirrored traffic is overwhelming the Amazon EC2 instance that is the traffic mirror target. The EC2 instance hosts tools that the company's security team uses to analyze the traffic. The network engineer needs to design a highly available solution that can scale to meet the demand of the mirrored traffic.

Which solution will meet these requirements?

- A. Deploy a Network Load Balancer (NLB) as the traffic mirror target. Behind the NLB, deploy a fleet of EC2 instances in an Auto Scaling group. Use Traffic Mirroring as necessary.
- B. Deploy an Application Load Balancer (ALB) as the traffic mirror target. Behind the ALB, deploy a fleet of EC2 instances in an Auto Scaling group. Use Traffic Mirroring only during non-business hours.
- C. Deploy a Gateway Load Balancer (GLB) as the traffic mirror target. Behind the GLB, deploy a fleet of EC2 instances in an Auto Scaling group. Use Traffic Mirroring as necessary.
- D. Deploy an Application Load Balancer (ALB) with an HTTPS listener as the traffic mirror target. Behind the ALB, deploy a fleet of EC2 instances in an Auto Scaling group. Use Traffic Mirroring only during active events or business hours.

Answer: A

Explanation:

A. Deploy a Network Load Balancer (NLB) as the traffic mirror target. Behind the NLB, deploy a fleet of EC2 instances in an Auto Scaling group. Use Traffic Mirroring as necessary. Network Load Balancer (NLB) is designed for handling TCP/UDP traffic and provides high availability, scalability, and low-latency performance. It is suitable for distributing the mirrored traffic to a fleet of EC2 instances. Deploying a fleet of EC2 instances in an Auto Scaling group ensures that you can scale the capacity of your analysis tools dynamically to meet the demand of the mirrored traffic. This addresses the issue of the EC2 instance being overwhelmed. Using Traffic Mirroring as necessary allows you to control when to capture mirrored traffic, reducing the load on the analysis tools during non-critical times.

Question: 22

CertyIQ

A company uses a hybrid architecture and has an AWS Direct Connect connection between its on-premises data center and AWS. The company has production applications that run in the on-premises data center. The company also has production applications that run in a VPC. The applications that run in the on-premises data center need to communicate with the applications that run in the VPC. The company is using corp.example.com as the domain name for the on-premises resources and is using an Amazon Route 53 private hosted zone for aws.example.com to host the VPC resources.

The company is using an open-source recursive DNS resolver in a VPC subnet and is using a DNS resolver in the on-premises data center. The company's on-premises DNS resolver has a forwarder that directs requests for the aws.example.com domain name to the DNS resolver in the VPC. The DNS resolver in the VPC has a forwarder that directs requests for the corp.example.com domain name to the DNS resolver in the on-premises data center. The company has decided to replace the open-source recursive DNS resolver with Amazon Route 53 Resolver endpoints.

Which combination of steps should a network engineer take to make this replacement? (Choose three.)

- A. Create a Route 53 Resolver rule to forward aws.example.com domain queries to the IP addresses of the outbound endpoint.

- B. Configure the on-premises DNS resolver to forward aws.example.com domain queries to the IP addresses of the inbound endpoint.
- C. Create a Route 53 Resolver inbound endpoint and a Route 53 Resolver outbound endpoint.
- D. Create a Route 53 Resolver rule to forward aws.example.com domain queries to the IP addresses of the inbound endpoint.
- E. Create a Route 53 Resolver rule to forward corp.example.com domain queries to the IP address of the on-premises DNS resolver.
- F. Configure the on-premises DNS resolver to forward aws.example.com queries to the IP addresses of the outbound endpoint.

Answer: BCE

Explanation:

The correct answer is BCE. Here's a breakdown of why:

C. Create a Route 53 Resolver inbound endpoint and a Route 53 Resolver outbound endpoint. This is the foundational step. Route 53 Resolver endpoints act as the bridge for DNS queries between your on-premises network and your VPC. The inbound endpoint allows on-premises resources to query Route 53, and the outbound endpoint allows VPC resources to query on-premises DNS.

(<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver.html>)

B. Configure the on-premises DNS resolver to forward aws.example.com domain queries to the IP addresses of the inbound endpoint. This ensures that when on-premises resources need to resolve names within the aws.example.com domain (hosted in the Route 53 private hosted zone), the queries are forwarded to the Route 53 Resolver inbound endpoint. This allows Route 53 to resolve those queries using the records in the private hosted zone.

E. Create a Route 53 Resolver rule to forward corp.example.com domain queries to the IP address of the on-premises DNS resolver. This step configures Route 53 to forward queries for the corp.example.com domain (used by on-premises resources) to the on-premises DNS resolver via the outbound endpoint. The rule ensures that the Route 53 Resolver (and thus, resources in the VPC) can resolve names within the on-premises domain.

Why other options are incorrect:

A & D: Forwarding aws.example.com to the outbound endpoint or to the inbound endpoint again would create a loop or break resolution. The on-premises DNS server should forward to the inbound endpoint. A Route 53 resolver rule forwards to on-premise DNS via the outbound endpoint.

F: Configuring the on-premises DNS resolver to forward to the outbound endpoint is incorrect. The on-premises DNS server should forward to the inbound endpoint to leverage the VPC-based resolvers.

In essence, the correct answer establishes bidirectional DNS resolution between the on-premises environment and the VPC, using Route 53 Resolver endpoints and rules to direct traffic to the correct destinations.

Question: 23

CertyIQ

A government contractor is designing a multi-account environment with multiple VPCs for a customer. A network security policy requires all traffic between any two VPCs to be transparently inspected by a third-party appliance. The customer wants a solution that features AWS Transit Gateway. The setup must be highly available across multiple Availability Zones, and the solution needs to support automated failover. Furthermore, asymmetric routing is not supported by the inspection appliances.

Which combination of steps is part of a solution that meets these requirements? (Choose two.)

A. Deploy two clusters that consist of multiple appliances across multiple Availability Zones in a designated inspection VPC. Connect the inspection VPC to the transit gateway by using a VPC attachment. Create a target group, and register the appliances with the target group. Create a Network Load Balancer (NLB), and set it up to forward to the newly created target group. Configure a default route in the inspection VPC's transit gateway subnet toward the NLB.

B. Deploy two clusters that consist of multiple appliances across multiple Availability Zones in a designated inspection VPC. Connect the inspection VPC to the transit gateway by using a VPC attachment. Create a target group, and register the appliances with the target group. Create a Gateway Load Balancer, and set it up to forward to the newly created target group. Configure a default route in the inspection VPC's transit gateway subnet toward the Gateway Load Balancer endpoint.

C. Configure two route tables on the transit gateway. Associate one route table with all the attachments of the application VPCs. Associate the other route table with the inspection VPC's attachment. Propagate all VPC attachments into the inspection route table. Define a static default route in the application route table. Enable appliance mode on the attachment that connects the inspection VPC.

D. Configure two route tables on the transit gateway. Associate one route table with all the attachments of the application VPCs. Associate the other route table with the inspection VPC's attachment. Propagate all VPC attachments into the application route table. Define a static default route in the inspection route table. Enable appliance mode on the attachment that connects the inspection VPC.

E. Configure one route table on the transit gateway. Associate the route table with all the VPCs. Propagate all VPC attachments into the route table. Define a static default route in the route table.

Answer: BC

Explanation:

The correct answer is BC. Here's a detailed explanation:

Choice B: Deploying Inspection Appliances with Gateway Load Balancer (GWLB)

This choice addresses the requirement for transparent inspection, high availability, automated failover, and handling asymmetric routing. The deployment of clusters of appliances across multiple AZs ensures high availability. The Gateway Load Balancer is designed specifically for virtual appliances and provides a transparent, scalable, and highly available way to direct traffic to these appliances for inspection. GWLB automatically handles appliance health checks and failover, fulfilling the automated failover requirement. Importantly, GWLB preserves source and destination IP addresses and ports, preventing asymmetric routing issues, as traffic both enters and exits through the same appliance. Configuring a default route in the inspection VPC's transit gateway subnet towards the Gateway Load Balancer endpoint ensures that all traffic destined for other VPCs is sent to the GWLB for inspection.

Choice C: Transit Gateway Route Table Configuration with Appliance Mode

This choice ensures that all inter-VPC traffic is routed through the inspection VPC, satisfying the network security policy requirement. By creating two route tables on the Transit Gateway (TGW), one for application VPCs and another for the inspection VPC, you can control the routing behavior. Associating the application VPC attachments with one route table and the inspection VPC attachment with the other allows you to isolate traffic flow. Propagating all VPC attachments into the inspection route table ensures the inspection VPC's appliances know about all possible destinations. Defining a static default route in the application route table pointing to the inspection VPC forces all inter-VPC traffic to pass through the inspection VPC. Enabling appliance mode on the inspection VPC attachment is crucial; it disables source/destination checks and ensures that the TGW forwards traffic to the correct appliance, handling asymmetric traffic correctly when combined with the GWLB setup.

Why other options are incorrect:

A: While NLB can distribute traffic, it's not designed for transparently inspecting network traffic like a GWLB. It can also cause asymmetric routing issues if not configured correctly.

D & E: These options incorrectly handle the route table configuration and lack the use of Gateway Load Balancer, which is crucial for appliance integration and handling asymmetric traffic.

Supporting Links:

AWS Transit Gateway: <https://aws.amazon.com/transit-gateway/>

Gateway Load Balancer: <https://aws.amazon.com/elasticloadbalancing/gateway-load-balancer/>

Transit Gateway Appliance Mode: <https://docs.aws.amazon.com/transit-gateway/latest/userguide/configure-appliance.html>

Question: 24

CertyIQ

A company has deployed Amazon EC2 instances in private subnets in a VPC. The EC2 instances must initiate any requests that leave the VPC, including requests to the company's on-premises data center over an AWS Direct Connect connection. No resources outside the VPC can be allowed to open communications directly to the EC2 instances.

The on-premises data center's customer gateway is configured with a stateful firewall device that filters for incoming and outgoing requests to and from multiple VPCs. In addition, the company wants to use a single IP match rule to allow all the communications from the EC2 instances to its data center from a single IP address. Which solution will meet these requirements with the LEAST amount of operational overhead?

- A. Create a VPN connection over the Direct Connect connection by using the on-premises firewall. Use the firewall to block all traffic from on premises to AWS. Allow a stateful connection from the EC2 instances to initiate the requests.
- B. Configure the on-premises firewall to filter all requests from the on-premises network to the EC2 instances. Allow a stateful connection if the EC2 instances in the VPC initiate the traffic.
- C. Deploy a NAT gateway into a private subnet in the VPC where the EC2 instances are deployed. Specify the NAT gateway type as private. Configure the on-premises firewall to allow connections from the IP address that is assigned to the NAT gateway.
- D. Deploy a NAT instance into a private subnet in the VPC where the EC2 instances are deployed. Configure the on-premises firewall to allow connections from the IP address that is assigned to the NAT instance.

Answer: C

Explanation:

Here's a detailed justification for why option C is the correct answer:

The problem requires EC2 instances in private subnets to initiate outbound connections to an on-premises data center via Direct Connect, while preventing unsolicited inbound connections to the instances. Additionally, the company wants to use a single IP address for all outbound communication from the instances to the on-premises data center, and the solution must minimize operational overhead.

Option C, deploying a NAT gateway in a public subnet (a key correction, the question should have specified a public subnet for the NAT gateway to function) and configuring the on-premises firewall to allow connections from the NAT gateway's IP address, perfectly addresses these requirements. A NAT gateway allows instances in private subnets to initiate outbound traffic to the internet or, in this case, the Direct Connect connection to the on-premises data center. The NAT gateway performs Network Address Translation, replacing the private IP address of the EC2 instance with its own public IP address. This enables a single IP address for all outbound traffic from the EC2 instances to the on-premises data center, fulfilling the requirement for a single IP match rule on the firewall. Crucially, a NAT gateway only allows return traffic for connections initiated from within the VPC, preventing unsolicited inbound connections. A private NAT gateway does not exist.

Option A is incorrect because establishing a VPN over Direct Connect adds unnecessary complexity. Direct Connect already provides a dedicated network connection. Furthermore, relying on a VPN and blocking all traffic from on-premises to AWS is overly restrictive and complicates management.

Option B is insufficient because it only addresses the on-premises firewall configuration. It doesn't address how the EC2 instances will initiate outbound connections using a single IP address, and it doesn't provide a

mechanism to prevent inbound connections.

Option D, deploying a NAT instance, is a valid approach but incurs more operational overhead than using a NAT gateway. NAT instances require manual patching, scaling, and management. A NAT gateway is a managed service, automatically scaling and patching itself. It also provides higher availability and throughput than a NAT instance.

Therefore, option C is the most efficient solution as it provides the required functionality with the least operational overhead by leveraging the managed NAT gateway service and presenting a single IP address to the on-premises firewall.

Supporting links:

AWS NAT Gateway: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html>

AWS Direct Connect: <https://aws.amazon.com/directconnect/>

Question: 25

CertyIQ

A global company operates all its non-production environments out of three AWS Regions: eu-west-1, us-east-1, and us-west-1. The company hosts all its production workloads in two on-premises data centers. The company has 60 AWS accounts and each account has two VPCs in each Region. Each VPC has a virtual private gateway where two VPN connections terminate for resilient connectivity to the data centers. The company has 360 VPN tunnels to each data center, resulting in high management overhead. The total VPN throughput for each Region is 500 Mbps. The company wants to migrate the production environments to AWS. The company needs a solution that will simplify the network architecture and allow for future growth. The production environments will generate an additional 2 Gbps of traffic per Region back to the data centers. This traffic will increase over time. Which solution will meet these requirements?

A. Set up an AWS Direct Connect connection from each data center to AWS in each Region. Create and attach private VIFs to a single Direct Connect gateway. Attach the Direct Connect gateway to all the VPCs. Remove the existing VPN connections that are attached directly to the virtual private gateways.

B. Create a single transit gateway with VPN connections from each data center. Share the transit gateway with each account by using AWS Resource Access Manager (AWS RAM). Attach the transit gateway to each VPC. Remove the existing VPN connections that are attached directly to the virtual private gateways.

C. Create a transit gateway in each Region with multiple newly commissioned VPN connections from each data center. Share the transit gateways with each account by using AWS Resource Access Manager (AWS RAM). In each Region, attach the transit gateway to each VPC. Remove the existing VPN connections that are attached directly to the virtual private gateways.

D. Peer all the VPCs in each Region to a new VPC in each Region that will function as a centralized transit VPC. Create new VPN connections from each data center to the transit VPCs. Terminate the original VPN connections that are attached to all the original VPCs. Retain the new VPN connection to the new transit VPC in each Region.

Answer: C

Explanation:

The correct answer is C. Here's why:

Why Option C is Correct:

Scalability and Future Growth: Transit Gateways (TGWs) are designed for connecting many VPCs and on-premises networks. Creating a TGW in each Region addresses the current need and allows for future expansion as the production environments are migrated.

Simplified Network Architecture: TGWs centralize routing, significantly reducing the complexity of managing numerous VPN connections. Each TGW serves as a hub, simplifying routing between VPCs and the data centers.

Increased Throughput: TGWs support higher bandwidth than individual VPN connections. Creating new VPN connections from each data center to each Region's TGW allows the 2 Gbps traffic requirement to be met and exceeded. The initial VPN connections only supported 500Mbps of traffic per region.

Resource Sharing: AWS Resource Access Manager (RAM) enables sharing the TGWs across multiple AWS accounts, reducing management overhead and ensuring consistent routing policies.

Regional Isolation: Deploying a TGW in each Region offers regional isolation. A failure in one Region's TGW will not affect connectivity in other Regions.

Why Other Options Are Incorrect:

Option A (Direct Connect): While Direct Connect provides high bandwidth and low latency, attaching private VIFs to a single Direct Connect gateway and then attaching the Direct Connect gateway to all the VPCs is not the intended use case and will still require regional connections. Also, this option will only assist the production environments after they have been migrated. The company needs connectivity for both environments.

Option B (Single Transit Gateway): Creating a single TGW and sharing it across Regions using RAM is not possible. Transit Gateways are regional resources.

Option D (Transit VPC): Transit VPCs are an older solution and less scalable and manageable than Transit Gateways. They also introduce a single point of failure in each Region.

Supporting Concepts:

AWS Transit Gateway: A network transit hub that you can use to interconnect your virtual private clouds (VPC) and on-premises networks. <https://aws.amazon.com/transit-gateway/>

AWS Resource Access Manager (RAM): Enables you to share your AWS resources with any AWS account or within your organization in AWS Organizations. <https://aws.amazon.com/ram/>

By using Transit Gateways in each Region, the company achieves a scalable, manageable, and resilient solution for connecting its on-premises data centers to its AWS environments.

Question: 26

CertyIQ

A company is building its website on AWS in a single VPC. The VPC has public subnets and private subnets in two Availability Zones. The website has static content such as images. The company is using Amazon S3 to store the content.

The company has deployed a fleet of Amazon EC2 instances as web servers in a private subnet. The EC2 instances are in an Auto Scaling group behind an Application Load Balancer. The EC2 instances will serve traffic, and they must pull content from an S3 bucket to render the webpages. The company is using AWS Direct Connect with a public VIF for on-premises connectivity to the S3 bucket.

A network engineer notices that traffic between the EC2 instances and Amazon S3 is routing through a NAT gateway. As traffic increases, the company's costs are increasing. The network engineer needs to change the connectivity to reduce the NAT gateway costs that result from the traffic between the EC2 instances and Amazon S3.

Which solution will meet these requirements?

- A. Create a Direct Connect private VIF. Migrate the traffic from the public VIF to the private VIF.
- B. Create an AWS Site-to-Site VPN tunnel over the existing public VIF.
- C. Implement interface VPC endpoints for Amazon S3. Update the VPC route table.
- D. Implement gateway VPC endpoints for Amazon S3. Update the VPC route table.

Answer: D

Explanation:

The problem is that EC2 instances in private subnets are accessing S3 via a NAT gateway, incurring significant costs due to the data transfer. The goal is to reduce these costs while maintaining connectivity to S3.

Option A, creating a Direct Connect private VIF, doesn't solve the problem of NAT gateway usage. It merely shifts the traffic from the public VIF to a private VIF for on-premises access to AWS services, not from EC2 instances within the VPC. EC2 instances will still use the NAT gateway to reach the internet (and S3) if they don't have another route.

Option B, using a Site-to-Site VPN over the public VIF, only adds complexity and doesn't address the NAT gateway issue. It would still require the EC2 instances to route traffic through the VPN connection and then potentially still through the NAT gateway.

Option C, implementing interface VPC endpoints for S3, provides private connectivity to S3 but requires a Network Load Balancer (NLB) or Elastic Network Interface (ENI) within the VPC to function as a proxy. This is more complex and not the best solution for simple S3 access. Interface endpoints utilize PrivateLink, creating network interfaces within your subnets.

Option D, implementing gateway VPC endpoints for S3, directly addresses the issue. Gateway endpoints create a route within the VPC that directs traffic destined for S3 to a service prefix list, bypassing the NAT gateway. This creates a private connection to S3 without requiring public IPs or NAT gateways, significantly reducing costs. The EC2 instances can then pull content from S3 directly without incurring NAT gateway charges. This is the simplest and most cost-effective solution for private access to S3 from within a VPC. Gateway endpoints modify the route table to point traffic destined for S3 directly to S3.

Therefore, implementing gateway VPC endpoints for Amazon S3 and updating the VPC route table is the optimal solution for reducing NAT gateway costs.

Relevant documentation:

[VPC Endpoints](#)

[Gateway VPC Endpoints](#)

[Interface VPC Endpoints](#)

Question: 27

CertyIQ

A company wants to improve visibility into its AWS environment. The AWS environment consists of multiple VPCs that are connected to a transit gateway. The transit gateway connects to an on-premises data center through an AWS Direct Connect gateway and a pair of redundant Direct Connect connections that use transit VIFs. The company must receive notification each time a new route is advertised to AWS from on premises over Direct Connect.

What should a network engineer do to meet these requirements?

- A. Enable Amazon CloudWatch metrics on Direct Connect to track the received routes. Configure a CloudWatch alarm to send notifications when routes change.
- B. Onboard Transit Gateway Network Manager to Amazon CloudWatch Logs Insights. Use Amazon EventBridge (Amazon CloudWatch Events) to send notifications when routes change.
- C. Configure an AWS Lambda function to periodically check the routes on the Direct Connect gateway and to send notifications when routes change.
- D. Enable Amazon CloudWatch Logs on the transit VIFs to track the received routes. Create a metric filter. Set an alarm on the filter to send notifications when routes change.

Answer: B

Explanation:

The correct answer is B. Here's a detailed justification:

Why Option B is Correct:

Option B provides the most scalable, efficient, and near real-time solution for tracking route advertisements from on-premises over Direct Connect within a multi-VPC environment using a transit gateway. Transit Gateway Network Manager provides centralized visibility into your global network, including connections, traffic, and routes. Integrating it with CloudWatch Logs Insights allows you to query and analyze network data. EventBridge (formerly CloudWatch Events) is a serverless event bus that enables you to react to changes in your AWS environment in near real-time.

1. **Transit Gateway Network Manager:** Centralizes network monitoring for Transit Gateway-based networks, including Direct Connect attachments. It simplifies management and enhances visibility.
2. **CloudWatch Logs Insights:** Enables you to search, analyze, and visualize log data in CloudWatch Logs. Network Manager sends network events which you can log to CWL.
3. **EventBridge:** Serves as an event bus that reacts to changes in your AWS environment. With EventBridge, you can create rules to trigger actions based on specific events, such as changes in routing tables. These events would come from CWL after they are processed by CWL insights.

The combination allows the engineer to leverage the centralized network monitoring capabilities of Transit Gateway Network Manager with the log analysis power of CloudWatch Logs Insights and the automated notification functionality of EventBridge to receive immediate notifications when new routes are advertised from on-premises.

Why Other Options are Incorrect:

Option A: While CloudWatch metrics can provide aggregate information about Direct Connect, they lack the granularity to track individual route changes. This solution wouldn't provide near real-time notifications for specific route advertisements.

Option C: Using a Lambda function for periodic polling is inefficient and not scalable. Frequent polling consumes unnecessary resources and might miss rapid route changes. Furthermore, determining route changes requires significant logic within the Lambda function.

Option D: While it is possible to analyze logs on the transit VIF, it is far more complicated to extract route changes, and maintain. This method is not as straightforward, centralized, or manageable as leveraging Transit Gateway Network Manager. This option also does not integrate directly with EventBridge which makes reporting complex.

Authoritative Links:

Transit Gateway Network Manager: <https://aws.amazon.com/transit-gateway/network-manager/>

Amazon CloudWatch Logs Insights: <https://aws.amazon.com/cloudwatch/features/logs-insights/>

Amazon EventBridge: <https://aws.amazon.com/eventbridge/>

Question: 28

CertyIQ

A software company offers a software-as-a-service (SaaS) accounting application that is hosted in the AWS Cloud. The application requires connectivity to the company's on-premises network. The company has two redundant 10 GB AWS Direct Connect connections between AWS and its on-premises network to accommodate the growing demand for the application.

The company already has encryption between its on-premises network and the colocation. The company needs to encrypt traffic between AWS and the edge routers in the colocation within the next few months. The company must maintain its current bandwidth.

What should a network engineer do to meet these requirements with the LEAST operational overhead?

A. Deploy a new public VIF with encryption on the existing Direct Connect connections. Reroute traffic through the new public VIF.

- B. Create a virtual private gateway. Deploy new AWS Site-to-Site VPN connections from on-premises to the virtual private gateway. Reroute traffic from the Direct Connect private VIF to the new VPNs.
- C. Deploy a new pair of 10 GB Direct Connect connections with MACsec. Configure MACsec on the edge routers. Reroute traffic to the new Direct Connect connections. Decommission the original Direct Connect connections.
- D. Deploy a new pair of 10 GB Direct Connect connections with MACsec. Deploy a new public VIF on the new Direct Connect connections. Deploy two AWS Site-to-Site VPN connections on top of the new public VIF. Reroute traffic from the existing private VIF to the new Site-to-Site connections. Decommission the original Direct Connect connections.

Answer: C

Explanation:

The best solution to encrypt traffic between AWS and the colocation edge routers while maintaining 10 Gbps bandwidth with minimal operational overhead is option C: deploy new Direct Connect connections with MACsec. Here's why:

MACsec on Direct Connect for Encryption: MACsec (Media Access Control Security) is a Layer 2 encryption protocol designed for point-to-point links like Direct Connect. It encrypts all traffic at the physical layer, ensuring data confidentiality. AWS supports MACsec on Direct Connect, allowing for secure communication between your on-premises network and AWS. <https://aws.amazon.com/directconnect/features/>

Maintaining Bandwidth: Option C involves deploying new 10 Gbps Direct Connect connections. This is crucial because it preserves the existing bandwidth capacity. Using new connections minimizes disruption and allows for a staged migration.

Minimal Operational Overhead: MACsec is implemented at the hardware level, minimizing performance impact and operational complexity. Once configured, the encryption and decryption processes are largely handled by the Direct Connect hardware.

Public VIF Inefficiency: Option A suggests a public VIF with encryption. Public VIFs are designed for accessing public AWS services over Direct Connect. They don't inherently provide encryption for all traffic between your on-premises network and AWS resources, requiring additional solutions such as VPN.

VPN Overhead: Option B and D suggest using Site-to-Site VPN over either Direct Connect or new Direct Connect. VPNs introduce significant overhead in terms of processing power for encryption and encapsulation. This added overhead will likely affect the overall network performance and reduce available bandwidth. VPN adds complexity compared to MACsec. The existing encryption is at the colocation edge routers, so there's no need to bring in additional complexity of Site-to-Site VPN tunnels.

Cost Efficiency: Although deploying new Direct Connect connections involves costs, it simplifies encryption and optimizes performance in the long run, offering a cost-effective solution compared to the operational overhead and bandwidth degradation associated with VPNs.

Therefore, deploying new Direct Connect connections with MACsec offers a balance between security, performance, and manageability, making it the most suitable solution with the least operational overhead.

Question: 29

CertyIQ

A company hosts an application on Amazon EC2 instances behind an Application Load Balancer (ALB). The company recently experienced a network security breach. A network engineer must collect and analyze logs that include the client IP address, target IP address, target port, and user agent of each user that accesses the application.

What is the MOST operationally efficient solution that meets these requirements?

- A. Configure the ALB to store logs in an Amazon S3 bucket. Download the files from Amazon S3, and use a

spreadsheet application to analyze the logs.

B. Configure the ALB to push logs to Amazon Kinesis Data Streams. Use Amazon Kinesis Data Analytics to analyze the logs.

C. Configure Amazon Kinesis Data Streams to stream data from the ALB to Amazon OpenSearch Service (Amazon Elasticsearch Service). Use search operations in Amazon OpenSearch Service (Amazon Elasticsearch Service) to analyze the data.

D. Configure the ALB to store logs in an Amazon S3 bucket. Use Amazon Athena to analyze the logs in Amazon S3.

Answer: D

Explanation:

The most operationally efficient solution is **D. Configure the ALB to store logs in an Amazon S3 bucket. Use Amazon Athena to analyze the logs in Amazon S3.**

Here's why:

ALB Access Logs to S3: Application Load Balancers (ALBs) can directly log detailed access information to S3, including client IP, target IP, target port, and user agent. This meets the logging requirements of the problem statement. (See: <https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-access-logs.html>)

Amazon Athena for Analysis: Amazon Athena allows you to query data directly in S3 using standard SQL. It's serverless, meaning no infrastructure management, and you only pay for the queries you run. This simplifies the analysis of the logs. (<https://aws.amazon.com/athena/>)

Operationally Efficient: Athena eliminates the need to download large log files and analyze them locally with tools like spreadsheets (option A), which is less efficient for large datasets. It also avoids the complexity of setting up and managing streaming data pipelines like Kinesis Data Streams and OpenSearch Service (options B and C).

Cost Effective: Athena is cost-effective because you only pay for the queries you run. Other options might incur costs for continuous data streaming and storage, even when not actively analyzing the logs.

Scalability: Both S3 and Athena are highly scalable. S3 can store petabytes of log data, and Athena can query that data efficiently, scaling as needed.

Security Incident Response: Athena allows for rapid analysis of logs during a security incident, enabling faster identification of malicious activity and aiding in incident response.

Option A is inefficient for large datasets. Options B and C involve more complex setup and ongoing management of streaming data services and search clusters, which is not the most operationally efficient approach for the given requirements. The problem statement emphasizes "MOST operationally efficient," which Athena fulfills due to its serverless nature and direct SQL querying capabilities on S3 log data.

Question: 30

CertyIQ

A media company is implementing a news website for a global audience. The website uses Amazon CloudFront as its content delivery network. The backend runs on Amazon EC2 Windows instances behind an Application Load Balancer (ALB). The instances are part of an Auto Scaling group. The company's customers access the website by using service.example.com as the CloudFront custom domain name. The CloudFront origin points to an ALB that uses service-alb.example.com as the domain name.

The company's security policy requires the traffic to be encrypted in transit at all times between the users and the backend.

Which combination of changes must the company make to meet this security requirement? (Choose three.)

A. Create a self-signed certificate for service.example.com. Import the certificate into AWS Certificate Manager (ACM). Configure CloudFront to use this imported SSL/TLS certificate. Change the default behavior to redirect HTTP to HTTPS.

B. Create a certificate for service.example.com by using AWS Certificate Manager (ACM). Configure CloudFront to use this custom SSL/TLS certificate. Change the default behavior to redirect HTTP to HTTPS.

C. Create a certificate with any domain name by using AWS Certificate Manager (ACM) for the EC2 instances. Configure the backend to use this certificate for its HTTPS listener. Specify the instance target type during the creation of a new target group that uses the HTTPS protocol for its targets. Attach the existing Auto Scaling group to this new target group.

D. Create a public certificate from a third-party certificate provider with any domain name for the EC2 instances. Configure the backend to use this certificate for its HTTPS listener. Specify the instance target type during the creation of a new target group that uses the HTTPS protocol for its targets. Attach the existing Auto Scaling group to this new target group.

E. Create a certificate for service-alb.example.com by using AWS Certificate Manager (ACM). On the ALB add a new HTTPS listener that uses the new target group and the service-alb.example.com ACM certificate. Modify the CloudFront origin to use the HTTPS protocol only. Delete the HTTP listener on the ALB.

F. Create a self-signed certificate for service-alb.example.com. Import the certificate into AWS Certificate Manager (ACM). On the ALB add a new HTTPS listener that uses the new target group and the imported service-alb.example.com ACM certificate. Modify the CloudFront origin to use the HTTPS protocol only. Delete the HTTP listener on the ALB.

Answer: BDE

Explanation:

The correct answer is BDE. Here's why:

B: Create a certificate for service.example.com by using AWS Certificate Manager (ACM). Configure CloudFront to use this custom SSL/TLS certificate. Change the default behavior to redirect HTTP to HTTPS. This is necessary to ensure secure communication between the users and CloudFront. ACM provides free SSL/TLS certificates that are trusted by browsers. By associating the certificate for the custom domain service.example.com with CloudFront, you establish an HTTPS connection. Redirecting HTTP to HTTPS enforces encryption for all requests. Reference:

<https://docs.aws.amazon.com/cloudfront/latest/DeveloperGuide/using-https-cloudfront-to-viewers.html>

D: Create a public certificate from a third-party certificate provider with any domain name for the EC2 instances. Configure the backend to use this certificate for its HTTPS listener. Specify the instance target type during the creation of a new target group that uses the HTTPS protocol for its targets. Attach the existing Auto Scaling group to this new target group. We require the communication between CloudFront and ALB to be encrypted as well, it will do that by adding a new HTTPS listener to the ALB with certificate.

E: Create a certificate for service-alb.example.com by using AWS Certificate Manager (ACM). On the ALB add a new HTTPS listener that uses the new target group and the service-alb.example.com ACM certificate. Modify the CloudFront origin to use the HTTPS protocol only. Delete the HTTP listener on the ALB. This step ensures secure communication between CloudFront and the ALB. Creating an ACM certificate for the ALB's domain (service-alb.example.com) and configuring an HTTPS listener on the ALB enables HTTPS. Modifying the CloudFront origin to use HTTPS ensures that CloudFront only sends encrypted traffic to the ALB. Finally, removing the HTTP listener prevents unencrypted traffic to the ALB. Reference:

<https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-listeners.html>

Options A and F are incorrect because self-signed certificates are not trusted by browsers and will result in security warnings. Option C is incorrect because you cannot use the same ACM certificate created with "any" domain name for the EC2 instance to secure the ALB communication.

Question: 31

CertyIQ

A company is hosting an application on Amazon EC2 instances behind a Network Load Balancer (NLB). A solutions architect added EC2 instances in a second Availability Zone to improve the availability of the application. The

solutions architect added the instances to the NLB target group.

The company's operations team notices that traffic is being routed only to the instances in the first Availability Zone.

What is the MOST operationally efficient solution to resolve this issue?

- A.Enable the new Availability Zone on the NLB
- B.Create a new NLB for the instances in the second Availability Zone
- C.Enable proxy protocol on the NLB
- D.Create a new target group with the instances in both Availability Zones

Answer: A

Explanation:

The most operationally efficient solution is to enable the new Availability Zone on the Network Load Balancer (NLB).

Here's why: NLBs are designed to distribute traffic across multiple Availability Zones for high availability. Simply adding instances in a new AZ to the target group doesn't automatically mean the NLB will start routing traffic there. You need to explicitly enable that AZ for the NLB itself. Enabling the AZ allows the NLB to create cross-zone load balancing and health checks to the instances in the newly added AZ.

Option B, creating a new NLB, is inefficient because it introduces unnecessary complexity and cost. You would then need to manage two separate NLBs. Option C, enabling proxy protocol, addresses the issue of preserving client IP addresses but does not affect cross-zone load balancing; it is related but does not solve the actual routing problem. Option D, creating a new target group with instances in both AZs, is unnecessary. The instances are already in a target group, and the NLB can distribute traffic among instances in a single target group across multiple AZs if those AZs are enabled for the NLB. Modifying or recreating the target group does not address the underlying problem of whether the NLB is aware of the new AZ or not.

Enabling the AZ for the existing NLB is the simplest and most direct solution. It leverages the existing infrastructure without introducing new components or management overhead.

Therefore, enabling the new Availability Zone on the NLB is the most operationally efficient solution.

Relevant documentation for further research:

[Network Load Balancer Availability Zones](#)
[Add or remove Availability Zones for your load balancer](#)

Question: 32

CertyIQ

A network engineer needs to set up an Amazon EC2 Auto Scaling group to run a Linux-based network appliance in a highly available architecture. The network engineer is configuring the new launch template for the Auto Scaling group.

In addition to the primary network interface the network appliance requires a second network interface that will be used exclusively by the application to exchange traffic with hosts over the internet. The company has set up a Bring Your Own IP (BYOIP) pool that includes an Elastic IP address that should be used as the public IP address for the second network interface.

How can the network engineer implement the required architecture?

- A.Configure the two network interfaces in the launch template. Define the primary network interface to be created in one of the private subnets. For the second network interface, select one of the public subnets. Choose the BYOIP pool ID as the source of public IP addresses.
- B.Configure the primary network interface in a private subnet in the launch template. Use the user data option to run a cloud-init script after boot to attach the second network interface from a subnet with auto-assign public IP addressing enabled.

C. Create an AWS Lambda function to run as a lifecycle hook of the Auto Scaling group when an instance is launching. In the Lambda function, assign a network interface to an AWS Global Accelerator endpoint.

D. During creation of the Auto Scaling group, select subnets for the primary network interface. Use the user data option to run a cloud-init script to allocate a second network interface and to associate an Elastic IP address from the BYOIP pool.

Answer: D

Explanation:

The correct answer is **D** because it provides the most flexible and manageable way to configure the second network interface and associate the BYOIP Elastic IP address with the EC2 instances in the Auto Scaling group. Here's a detailed justification:

Auto Scaling Groups and Launch Templates/Configurations: Auto Scaling groups use launch templates or configurations to define the instances they create. While launch templates can define one network interface, they aren't easily modifiable post-launch for advanced configuration.

<https://docs.aws.amazon.com/autoscaling/ec2/userguide/LaunchTemplates.html>

User Data and Cloud-Init: The user data field allows you to specify scripts that are executed when an instance starts. Cloud-init is a widely used package that handles early initialization of cloud instances. This makes it suitable for configuring the second network interface.

<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/user-data.html>

BYOIP and Elastic IP Addresses: BYOIP allows bringing your own IP addresses to AWS and associating them with Elastic IP addresses. You can then associate these Elastic IPs with your EC2 instances. Answer D uses user data to allocate the ENI and then associate the EIP.

<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-byoip.html>

Why other options are less ideal:

A: Configuring both interfaces in the launch template won't allow associating a specific BYOIP Elastic IP as a public IP for the second ENI, giving you less control.

B: While technically feasible, automatically assigning a public IP through subnet settings doesn't utilize the BYOIP pool which is a key requirement.

C: While Lifecycle hooks and Lambda can configure resources, this approach adds significant complexity and is unnecessary for a simple task like assigning a second ENI and associating a BYOIP EIP. Using Lambda to connect a network interface to a Global Accelerator endpoint is not a direct solution for assigning a BYOIP to the second network interface of an EC2 instance within an autoscaling group. This approach adds more complexity than is needed.

In summary, option D provides a programmatic yet manageable way to attach the second network interface and associate the required BYOIP Elastic IP address during the instance launch process. It uses user data and cloud-init, industry-standard tools for instance configuration, adhering to best practices for managing resources in an Auto Scaling environment.

Question: 33

CertyIQ

A company delivers applications over the internet. An Amazon Route 53 public hosted zone is the authoritative DNS service for the company and its internet applications, all of which are offered from the same domain name. A network engineer is working on a new version of one of the applications. All the application's components are hosted in the AWS Cloud. The application has a three-tier design. The front end is delivered through Amazon EC2 instances that are deployed in public subnets with Elastic IP addresses assigned. The backend components are deployed in private subnets from RFC1918.

Components of the application need to be able to access other components of the application within the application's VPC by using the same host names as the host names that are used over the public internet. The

network engineer also needs to accommodate future DNS changes, such as the introduction of new host names or the retirement of DNS entries.

Which combination of steps will meet these requirements? (Choose three.)

- A. Add a geoproximity routing policy in Route 53.
- B. Create a Route 53 private hosted zone for the same domain name. Associate the application's VPC with the new private hosted zone.
- C. Enable DNS hostnames for the application's VPC.
- D. Create entries in the private hosted zone for each name in the public hosted zone by using the corresponding private IP addresses.
- E. Create an Amazon EventBridge (Amazon CloudWatch Events) rule that runs when AWS CloudTrail logs a Route 53 API call to the public hosted zone. Create an AWS Lambda function as the target of the rule. Configure the function to use the event information to update the private hosted zone.
- F. Add the private IP addresses in the existing Route 53 public hosted zone.

Answer: BCD

Explanation:

The correct solution is BCD. Here's a detailed justification:

B. Create a Route 53 private hosted zone for the same domain name and associate the application's VPC with the new private hosted zone. This is essential for providing internal DNS resolution that is distinct from the public DNS. By creating a private hosted zone with the same domain name as the public zone and associating it with the application's VPC, we ensure that internal traffic within the VPC will query this private zone first. Route 53 will then resolve names to private IP addresses within the VPC.

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/hosted-zones-private.html>

C. Enable DNS hostnames for the application's VPC. Enabling DNS hostnames ensures that EC2 instances within the VPC are automatically assigned private DNS hostnames, which are required for resolution within the VPC. If this isn't enabled, instances would not get the internal DNS entries required for resolution.

<https://docs.aws.amazon.com/vpc/latest/userguide/vpc-dns.html>

D. Create entries in the private hosted zone for each name in the public hosted zone by using the corresponding private IP addresses. This step ensures that when components inside the VPC query the same hostname used externally, they resolve to the internal (private) IP addresses of the application components. This allows the application to use the same hostnames internally as it does externally, simplifying configuration and making the solution more flexible.

Option A (geoproximity routing) is not relevant because the requirement focuses on internal name resolution within the VPC. Geoproximity policies are used to route traffic to different endpoints based on geographical location.

Option E (EventBridge and Lambda) is an overly complex and potentially unreliable way to keep the private and public hosted zones synchronized. This could lead to latency and potential errors and would require intricate monitoring and management.

Option F (Adding private IPs to the public hosted zone) would expose the internal infrastructure to the public internet, which is a security risk and violates best practices. Public DNS should only contain records for publicly accessible resources. Therefore, BCD is the only combination that meets the stated requirements and follows best practices.

Question: 34

A company is deploying an application. The application is implemented in a series of containers in an Amazon

Elastic Container Service (Amazon ECS) cluster. The company will use the Fargate launch type for its tasks. The containers will run workloads that require connectivity initiated over an SSL connection. Traffic must be able to flow to the application from other AWS accounts over private connectivity. The application must scale in a manageable way as more consumers use the application.

Which solution will meet these requirements?

A. Choose a Gateway Load Balancer (GLB) as the type of load balancer for the ECS service. Create a lifecycle hook to add new tasks to the target group from Amazon ECS as required to handle scaling. Specify the GLB in the service definition. Create a VPC peer for external AWS accounts. Update the route tables so that the AWS accounts can reach the GLB.

B. Choose an Application Load Balancer (ALB) as the type of load balancer for the ECS service. Create path-based routing rules to allow the application to target the containers that are registered in the target group. Specify the ALB in the service definition. Create a VPC endpoint service for the ALB. Share the VPC endpoint service with other AWS accounts.

C. Choose an Application Load Balancer (ALB) as the type of load balancer for the ECS service. Create path-based routing rules to allow the application to target the containers that are registered in the target group. Specify the ALB in the service definition. Create a VPC peer for the external AWS accounts. Update the route tables so that the AWS accounts can reach the ALB.

D. Choose a Network Load Balancer (NLB) as the type of load balancer for the ECS service. Specify the NLB in the service definition. Create a VPC endpoint service for the NLB. Share the VPC endpoint service with other AWS accounts.

Answer: D

Explanation:

Here's a detailed justification for why option D is the correct answer:

The requirements highlight several critical aspects: SSL connectivity, private connectivity from other AWS accounts, scalability, and containerization using ECS with Fargate. Let's break down why each component of option D addresses these needs.

Network Load Balancer (NLB): NLBs are designed for high performance and handle TCP, UDP, and TLS (SSL) traffic efficiently. They provide static IP addresses per Availability Zone, which are essential for creating a VPC endpoint service. <https://docs.aws.amazon.com/elasticloadbalancing/latest/network/introduction.html>

VPC Endpoint Service: A VPC endpoint service enables private connectivity between VPCs, even across different AWS accounts. It allows other AWS accounts to access your application without exposing it to the public internet. This directly fulfills the requirement for private connectivity from other accounts.

<https://docs.aws.amazon.com/vpc/latest/privatelink/endpoint-services.html>

Sharing the VPC Endpoint Service: Once the VPC endpoint service is created for the NLB, it can be shared with specific AWS accounts. These accounts can then create VPC endpoints within their VPCs to privately connect to the NLB, and consequently, the ECS Fargate tasks behind it.

Why other options are less suitable:

Option A (Gateway Load Balancer): Gateway Load Balancers are primarily used for network virtual appliances (like firewalls or intrusion detection systems), not directly for application traffic routing like SSL-based application requests. They are also more complex to configure for simple application scaling.

Options B and C (Application Load Balancer): While ALBs support SSL termination and path-based routing, they don't natively provide the simplest or most efficient method for cross-account private connectivity compared to VPC endpoint services. Using VPC peering with ALBs requires more configuration and management of route tables in each account. Additionally, ALBs are designed for HTTP/HTTPS traffic and might not be the best choice if the application requires direct TCP or TLS connections. Also, sharing an ALB using a VPC endpoint service is not as straightforward as with an NLB.

In summary, option D provides the most straightforward, scalable, and secure solution for enabling private SSL-based connectivity to an ECS Fargate application from other AWS accounts, using an NLB and VPC endpoint service. This approach minimizes complexity and leverages AWS's managed services effectively.

Question: 35

A company's development team has created a new product recommendation web service. The web service is hosted in a VPC with a CIDR block of 192.168.224.0/19. The company has deployed the web service on Amazon EC2 instances and has configured an Auto Scaling group as the target of a Network Load Balancer (NLB). The company wants to perform testing to determine whether users who receive product recommendations spend more money than users who do not receive product recommendations. The company has a big sales event in 5 days and needs to integrate its existing production environment with the recommendation engine by then. The existing production environment is hosted in a VPC with a CIDR block of 192.168.128.0/17. A network engineer must integrate the systems by designing a solution that results in the least possible disruption to the existing environments. Which solution will meet these requirements?

- A. Create a VPC peering connection between the web service VPC and the existing production VPC. Add a routing rule to the appropriate route table to allow data to flow to 192.168.224.0/19 from the existing production environment and to flow to 192.168.128.0/17 from the web service environment. Configure the relevant security groups and ACLs to allow the systems to communicate.
- B. Ask the development team of the web service to redeploy the web service into the production VPC and integrate the systems there.
- C. Create a VPC endpoint service. Associate the VPC endpoint service with the NLB for the web service. Create an interface VPC endpoint for the web service in the existing production VPC.
- D. Create a transit gateway in the existing production environment. Create attachments to the production VPC and the web service VPC. Configure appropriate routing rules in the transit gateway and VPC route tables for 192.168.224.0/19 and 192.168.128.0/17. Configure the relevant security groups and ACLs to allow the systems to communicate.

Answer: C

Explanation:

The best solution is **C. Create a VPC endpoint service. Associate the VPC endpoint service with the NLB for the web service. Create an interface VPC endpoint for the web service in the existing production VPC.**

Here's why:

Least Disruption: VPC endpoints, specifically interface endpoints using PrivateLink, offer the least disruptive integration method. They allow private access to services in another VPC without exposing the traffic to the public internet or requiring complex routing configurations. This minimizes changes to existing infrastructure and reduces potential for errors.

Scalability and Availability: Using the NLB with the VPC endpoint service ensures that the production environment can scale its access to the recommendation engine as needed. The NLB distributes traffic across multiple instances of the web service, providing high availability.

Security: Traffic between the production VPC and the web service VPC remains within the AWS network, improving security by avoiding public internet exposure. Security groups and network ACLs can be configured to further restrict access.

Speed of Implementation: Setting up a VPC endpoint service and interface endpoint is relatively straightforward, allowing for quick integration within the 5-day deadline.

Why other options are not ideal:

A (VPC Peering): While possible, VPC peering has limitations with overlapping CIDR blocks (which is not the case here, but should be checked). Also, peering requires careful route table management and can become complex with larger networks. It also has the potential risk of exposing more resources than necessary between VPCs.

B (Redeploying): Redeploying the web service into the production VPC is the most disruptive option, requiring significant code changes, testing, and potential downtime. It's also not a clean separation of concerns.

D (Transit Gateway): Transit Gateway is more suitable for connecting multiple VPCs and on-premises networks. It's an overkill solution for connecting just two VPCs, especially considering the need for minimal disruption and a quick turnaround. It would also be more expensive to implement than VPC Endpoints.

Supporting Concepts and Links:

AWS PrivateLink: <https://aws.amazon.com/privatelink/>

VPC Endpoints: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html>

Network Load Balancer (NLB):

<https://docs.aws.amazon.com/elasticloadbalancing/latest/network/introduction.html>

Question: 36

CertyIQ

A network engineer needs to update a company's hybrid network to support IPv6 for the upcoming release of a new application. The application is hosted in a VPC in the AWS Cloud. The company's current AWS infrastructure includes VPCs that are connected by a transit gateway. The transit gateway is connected to the on-premises network by AWS Direct Connect and AWS Site-to-Site VPN. The company's on-premises devices have been updated to support the new IPv6 requirements.

The company has enabled IPv6 for the existing VPC by assigning a new IPv6 CIDR block to the VPC and by assigning IPv6 to the subnets for dual-stack support. The company has launched new Amazon EC2 instances for the new application in the updated subnets.

When updating the hybrid network to support IPv6 the network engineer must avoid making any changes to the current infrastructure. The network engineer also must block direct access to the instances' new IPv6 addresses from the internet. However, the network engineer must allow outbound internet access from the instances.

What is the MOST operationally efficient solution that meets these requirements?

A. Update the Direct Connect transit VIF and configure BGP peering with the AWS assigned IPv6 peering address. Create a new VPN connection that supports IPv6 connectivity. Add an egress-only internet gateway. Update any affected VPC security groups and route tables to provide connectivity within the VPC and between the VPC and the on-premises devices

B. Update the Direct Connect transit VIF and configure BGP peering with the AWS assigned IPv6 peering address. Update the existing VPN connection to support IPv6 connectivity. Add an egress-only internet gateway. Update any affected VPC security groups and route tables to provide connectivity within the VPC and between the VPC and the on-premises devices.

C. Create a Direct Connect transit VIF and configure BGP peering with the AWS assigned IPv6 peering address. Create a new VPN connection that supports IPv6 connectivity. Add an egress-only internet gateway. Update any affected VPC security groups and route tables to provide connectivity within the VPC and between the VPC and the on-premises devices.

D. Create a Direct Connect transit VIF and configure BGP peering with the AWS assigned IPv6 peering address. Create a new VPN connection that supports IPv6 connectivity. Add a NAT gateway. Update any affected VPC security groups and route tables to provide connectivity within the VPC and between the VPC and the on-premises devices.

Answer: A

Explanation:

The most operationally efficient solution is **A**. Here's why:

Direct Connect and IPv6: To extend IPv6 connectivity over Direct Connect, the existing transit virtual interface (VIF) should be updated for IPv6 BGP peering. This allows the exchange of IPv6 routes between AWS and the on-premises network via the Transit Gateway. Using the AWS assigned IPv6 peering address is required for setting up BGP for IPv6 on Direct Connect.

Site-to-Site VPN and IPv6: Since a connection is already in place to the on-premise environment and IPv6 support is required a new VPN connection that supports IPv6 connectivity needs to be created.

Egress-Only Internet Gateway: To prevent direct inbound access to the EC2 instances' IPv6 addresses from

the internet while allowing outbound internet access, an egress-only internet gateway is the correct choice. This gateway allows instances within the VPC to initiate outbound IPv6 traffic to the internet but prevents the internet from initiating IPv6 connections to the instances.

Security Groups and Route Tables: Security groups control inbound and outbound traffic at the instance level, and route tables determine the path that network traffic takes. These need to be updated to allow IPv6 traffic between the VPC subnets, on-premises network (via Transit Gateway), and the egress-only internet gateway.

Options B, C and D are incorrect because:

Option B suggests updating the VPN connection instead of creating a new one. IPv4 and IPv6 VPN connections are supported simultaneously.

Option C proposes creating a Direct Connect transit VIF from scratch. The question states that avoiding changes to current infrastructure is necessary so reusing the current VIF is best.

Option D proposes creating a NAT gateway. This is for IPv4 traffic, not IPv6, and thus doesn't fit the scenario.

Supporting Links:

AWS Direct Connect and IPv6: <https://docs.aws.amazon.com/directconnect/latest/UserGuide/direct-connect-ipv6.html>

Egress-Only Internet Gateway: <https://docs.aws.amazon.com/vpc/latest/userguide/egress-only-internet-gateway.html>

AWS Site-to-Site VPN and IPv6: https://docs.aws.amazon.com/vpn/latest/s2svpn/VPNS2S_VPNProtocol.html

Question: 37

CertyIQ

A network engineer must provide additional safeguards to protect encrypted data at Application Load Balancers (ALBs) through the use of a unique random session key.

What should the network engineer do to meet this requirement?

- A. Change the ALB security policy to a policy that supports TLS 1.2 protocol only
- B. Use AWS Key Management Service (AWS KMS) to encrypt session keys
- C. Associate an AWS WAF web ACL with the ALBs. and create a security rule to enforce forward secrecy (FS)
- D. Change the ALB security policy to a policy that supports forward secrecy (FS)

Answer: D

Explanation:

The requirement is to protect encrypted data at Application Load Balancers (ALBs) using a unique random session key, adding an extra layer of security. Forward secrecy (FS) achieves this.

Option D, changing the ALB security policy to one that supports forward secrecy (FS), directly addresses this. FS ensures that even if a private key is compromised, past session keys remain secure because they weren't derived from the private key itself. Instead, each session uses a unique, randomly generated session key. TLS cipher suites that implement protocols such as Diffie-Hellman Ephemeral (DHE) and Elliptic-Curve Diffie-Hellman Ephemeral (ECDHE) provide FS. By selecting an ALB security policy that includes these cipher suites, the ALB will negotiate connections using FS, generating a new session key for each session.

Option A, focusing only on TLS 1.2, is not sufficient. While using TLS 1.2 is good security practice, TLS 1.2 itself doesn't automatically guarantee FS. The specific cipher suites used within TLS 1.2 determine whether FS is enabled.

Option B, using AWS KMS to encrypt session keys, isn't applicable for ALB session keys used for TLS connections. AWS KMS is helpful for encrypting data at rest or when managing encryption keys for services

like S3 or EBS, but it's not directly involved in the ephemeral session key exchange in TLS with ALBs.

Option C, associating an AWS WAF web ACL with the ALB and creating a security rule to enforce FS is incorrect. AWS WAF protects web applications from common web exploits, but it doesn't directly enforce the forward secrecy mechanism at the TLS layer. WAF operates at a higher level (application layer), inspecting traffic based on rules. While WAF can add security, it does not replace the need for configuring the ALB with a security policy that supports FS.

Therefore, enabling forward secrecy on the ALB directly addresses the requirement of generating unique, random session keys for each session, ensuring that past communications remain secure even if the server's private key is compromised in the future.

Supporting Links:

AWS Load Balancer Security Policies:

<https://docs.aws.amazon.com/elasticloadbalancing/latest/application/create-https-listener.html#security-policies>

Forward Secrecy: https://en.wikipedia.org/wiki/Forward_secrecy

Question: 38

CertyIQ

A company has deployed a software-defined WAN (SD-WAN) solution to interconnect all of its offices. The company is migrating workloads to AWS and needs to extend its SD-WAN solution to support connectivity to these workloads.

A network engineer plans to deploy AWS Transit Gateway Connect and two SD-WAN virtual appliances to provide this connectivity. According to company policies, only a single SD-WAN virtual appliance can handle traffic from AWS workloads at a given time.

How should the network engineer configure routing to meet these requirements?

- A. Add a static default route in the transit gateway route table to point to the secondary SD-WAN virtual appliance. Add routes that are more specific to point to the primary SD-WAN virtual appliance.
- B. Configure the BGP community tag 7224:7300 on the primary SD-WAN virtual appliance for BGP routes toward the transit gateway.
- C. Configure the AS_PATH prepend attribute on the secondary SD-WAN virtual appliance for BGP routes toward the transit gateway.
- D. Disable equal-cost multi-path (ECMP) routing on the transit gateway for Transit Gateway Connect.

Answer: C

Explanation:

The correct answer is C. Here's why:

The requirement is for only one SD-WAN appliance to actively handle traffic from AWS workloads at a time. This implies an active/passive failover scenario. We need a mechanism to ensure the primary appliance is preferred for routing traffic.

Option C uses the AS_PATH prepend attribute of BGP on the secondary appliance. AS_PATH prepend works by artificially lengthening the AS_PATH attribute of the BGP route advertised by the secondary appliance. BGP prefers routes with shorter AS_PATH lengths. Therefore, the Transit Gateway will always prefer the route advertised by the primary appliance (which has a shorter, or default, AS_PATH) unless the primary fails. When the primary appliance is unavailable, the Transit Gateway will choose the route with the longer AS_PATH from the secondary appliance, thus initiating failover.

Option A is incorrect because static routes are generally less dynamic than BGP and would require manual intervention to failover. Adding a default route to the secondary appliance and more specific routes to the

primary appliance would likely create routing conflicts or require complex route management.

Option B suggesting a BGP community tag (7224:7300) is relevant to Transit Gateway route tables, not for influencing route selection when multiple routes are available. BGP communities are often used for traffic engineering, filtering, or tagging routes, but not as the primary mechanism for active/passive failover based on path preference.

Option D, disabling ECMP, does not achieve the desired result. ECMP spreads traffic across equal-cost paths. The goal is not to load balance between appliances, but to ensure one appliance actively handles the traffic with a seamless failover to the second.

In summary, the AS_PATH prepend on the secondary appliance provides a standard, BGP-based, and dynamic way to implement an active/passive failover mechanism for SD-WAN connectivity to AWS workloads via Transit Gateway.

Relevant links for more information:

AWS Transit Gateway: <https://aws.amazon.com/transit-gateway/>

BGP Path Attributes: <https://www.cisco.com/c/en/us/support/docs/ip/border-gateway-protocol-bgp/16045-bgp-attributes.html> (Cisco Doc, but explains the concepts)

AS_PATH Prepending: <https://networklessons.com/bgp/bgp-as-path-prepend-configuration>

Question: 39

CertyIQ

A company is planning to deploy many software-defined WAN (SD-WAN) sites. The company is using AWS Transit Gateway and has deployed a transit gateway in the required AWS Region. A network engineer needs to deploy the SD-WAN hub virtual appliance into a VPC that is connected to the transit gateway. The solution must support at least 5 Gbps of throughput from the SD-WAN hub virtual appliance to other VPCs that are attached to the transit gateway.

Which solution will meet these requirements?

- A. Create a new VPC for the SD-WAN hub virtual appliance. Create two IPsec VPN connections between the SD-WAN hub virtual appliance and the transit gateway. Configure BGP over the IPsec VPN connections
- B. Assign a new CIDR block to the transit gateway. Create a new VPC for the SD-WAN hub virtual appliance. Attach the new VPC to the transit gateway with a VPC attachment. Add a transit gateway Connect attachment. Create a Connect peer and specify the GRE and BGP parameters. Create a route in the appropriate VPC for the SD-WAN hub virtual appliance to route to the transit gateway.
- C. Create a new VPC for the SD-WAN hub virtual appliance. Attach the new VPC to the transit gateway with a VPC attachment. Create two IPsec VPN connections between the SD-WAN hub virtual appliance and the transit gateway. Configure BGP over the IPsec VPN connections.
- D. Assign a new CIDR block to the transit gateway. Create a new VPC for the SD-WAN hub virtual appliance. Attach the new VPC to the transit gateway with a VPC attachment. Add a transit gateway Connect attachment. Create a Connect peer and specify the VXLAN and BGP parameters. Create a route in the appropriate VPC for the SD-WAN hub virtual appliance to route to the transit gateway.

Answer: B

Explanation:

The correct answer is B. Here's why:

Transit Gateway Connect: AWS Transit Gateway Connect enables you to use GRE/VXLAN tunnels for higher bandwidth and better performance compared to IPsec VPNs. It is specifically designed for integrating SD-WAN appliances with Transit Gateway.

<https://aws.amazon.com/blogs/networking-and-content-delivery/aws-transit-gateway-connect-sd-wan-connectivity-and-performance-enhancements/>

CIDR block for Transit Gateway: While not strictly necessary for the connectivity itself, assigning a new CIDR block to the Transit Gateway helps with better route management and avoiding overlapping CIDRs between your VPCs and SD-WAN network. This becomes more important with a large number of SD-WAN sites.

<https://docs.aws.amazon.com/transit-gateway/latest/userguide/how-transit-gateways-work.html>

New VPC: Isolating the SD-WAN hub virtual appliance in its own VPC promotes better security and network management.

VPC Attachment: Attaching the VPC to the Transit Gateway allows connectivity to other VPCs connected to the Transit Gateway.

Connect Attachment and Peer: The Transit Gateway Connect attachment and Connect Peer configuration, using GRE and BGP, establishes the tunnel and routing required for SD-WAN to function correctly. GRE encapsulation allows non-IP protocols to be carried over IP networks, which may be necessary for some SD-WAN implementations. BGP ensures dynamic routing between the SD-WAN network and the AWS environment.

Routing: Adding a route in the VPC route table for the SD-WAN appliance, directing traffic destined for other connected VPCs to the Transit Gateway, is essential for directing traffic to the SD-WAN hub and onward.

Why other options are incorrect:

A and C (IPsec VPN): IPsec VPNs have a lower throughput limit compared to Transit Gateway Connect, generally topping out around 1.25 Gbps per VPN connection. While you could use multiple VPN connections in parallel, this adds complexity and is not the optimal solution for 5 Gbps requirement.

D (VXLAN): While VXLAN is an encapsulation protocol, GRE is more commonly used and compatible for most SD-WAN appliance integrations with AWS Transit Gateway. Choosing VXLAN without a specific requirement is less suitable.

In summary, Transit Gateway Connect offers a high-bandwidth, efficient way to integrate SD-WAN appliances with AWS Transit Gateway using GRE tunnels and BGP for routing. The setup also includes the required VPC attachment, CIDR assignment and the correct route configurations to facilitate SD-WAN functionality.

Question: 40

CertyIQ

A company is deploying a new application on AWS. The application uses dynamic multicasting. The company has five VPCs that are all attached to a transit gateway. Amazon EC2 instances in each VPC need to be able to register dynamically to receive a multicast transmission.

How should a network engineer configure the AWS resources to meet these requirements?

- A. Create a static source multicast domain within the transit gateway. Associate the VPCs and applicable subnets with the multicast domain. Register the multicast senders' network interface with the multicast domain. Adjust the network ACLs to allow UDP traffic from the source to all receivers and to allow UDP traffic that is sent to the multicast group address.
- B. Create a static source multicast domain within the transit gateway. Associate the VPCs and applicable subnets with the multicast domain. Register the multicast senders' network interface with the multicast domain. Adjust the network ACLs to allow TCP traffic from the source to all receivers and to allow TCP traffic that is sent to the multicast group address.
- C. Create an Internet Group Management Protocol (IGMP) multicast domain within the transit gateway. Associate the VPCs and applicable subnets with the multicast domain. Register the multicast senders' network interface with the multicast domain. Adjust the network ACLs to allow UDP traffic from the source to all receivers and to allow UDP traffic that is sent to the multicast group address.
- D. Create an Internet Group Management Protocol (IGMP) multicast domain within the transit gateway. Associate the VPCs and applicable subnets with the multicast domain. Register the multicast senders' network interface with the multicast domain. Adjust the network ACLs to allow TCP traffic from the source to all receivers and to allow TCP traffic that is sent to the multicast group address.

Answer: C**Explanation:**

The correct answer is C because it leverages IGMP for dynamic multicast group membership and uses UDP for multicast traffic. Let's break down why.

The problem statement indicates that instances need to dynamically register to receive multicast transmissions. IGMP (Internet Group Management Protocol) is a protocol used to manage multicast group memberships. Receivers use IGMP to inform a multicast router (in this case, the transit gateway) that they want to join a specific multicast group. This allows the router to forward multicast traffic only to the VPCs/subnets containing interested receivers. A "static source multicast domain" doesn't support this dynamic registration.

Therefore, the transit gateway needs to be configured with an IGMP multicast domain to handle dynamic group membership. VPCs and their relevant subnets must be associated with this multicast domain. Additionally, the multicast senders' network interfaces need to be registered within the multicast domain. This informs the transit gateway which source is authorized to send multicast traffic.

Multicast generally uses UDP (User Datagram Protocol) rather than TCP (Transmission Control Protocol). UDP is connectionless, making it more suitable for the one-to-many nature of multicast, where reliability is often sacrificed for speed and efficiency. Thus, network ACLs need to be adjusted to permit UDP traffic between the source and all receivers and traffic sent to the multicast group address. By opening UDP at the network ACL level, this guarantees traffic will traverse.

In summary, IGMP is the key to dynamic multicast, UDP is standard for multicast traffic, and network ACLs act as a firewall to restrict unwanted traffic. The other options are incorrect because they either use static multicast domains which don't support dynamic registration, or specify TCP, which is not commonly used for multicast.

Reference links:

AWS Transit Gateway Multicast: <https://docs.aws.amazon.com/vpc/latest/tgw/tgw-multicast.html>

IGMP: https://en.wikipedia.org/wiki/Internet_Group_Management_Protocol

Network ACLs: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-acls.html>

Question: 41**CertyIQ**

A company is creating new features for its ecommerce website. These features will use several microservices that are accessed through different paths. The microservices will run on Amazon Elastic Container Service (Amazon ECS). The company requires the use of HTTPS for all of its public websites. The application requires the customer's source IP addresses.

A network engineer must implement a load balancing strategy that meets these requirements. Which combination of actions should the network engineer take to accomplish this goal? (Choose two.)

- A. Use a Network Load Balancer
- B. Retrieve client IP addresses by using the X-Forwarded-For header
- C. Use AWS App Mesh load balancing
- D. Retrieve client IP addresses by using the X-IP-Source header
- E. Use an Application Load Balancer.

Answer: BE**Explanation:**

The correct answer is BE. Here's why:

E. Use an Application Load Balancer (ALB): ALBs are ideal for HTTP and HTTPS traffic. They operate at the application layer (Layer 7), which allows them to make routing decisions based on the content of the request (like path or host headers). This is perfectly suited for routing requests to different microservices based on the URL path, as mentioned in the question. ALBs also provide native HTTPS support, fulfilling the requirement for secure connections.

B. Retrieve client IP addresses by using the X-Forwarded-For header: When an ALB terminates SSL/TLS connections, the original client IP address is not directly available to the backend servers (ECS containers in this case). The ALB adds the client IP address to the X-Forwarded-For header in the request it forwards to the backend. The microservices can then read this header to determine the client's original IP address.

Why other options are incorrect:

A. Use a Network Load Balancer (NLB): NLBs operate at the transport layer (Layer 4), which means they forward traffic based on IP address and port. They don't have the application-level awareness to route based on URL path. While they preserve the source IP, using an ALB is more appropriate given the need for path-based routing and HTTPS termination.

C. Use AWS App Mesh load balancing: AWS App Mesh is a service mesh that provides application-level networking. While it can be used with ECS, it primarily focuses on managing service-to-service communication within the mesh. Using App Mesh alone does not provide the public HTTPS termination and source IP preservation needed directly from a public facing load balancer in this scenario.

D. Retrieve client IP addresses by using the X-IP-Source header: There is no standard or common header named X-IP-Source used for transmitting the client IP address. The X-Forwarded-For header is the widely accepted and implemented standard.

Supporting Documentation:

Application Load Balancer:

<https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html>

X-Forwarded-For Header: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Forwarded-For>

AWS App Mesh: <https://aws.amazon.com/app-mesh/>

In conclusion, an ALB offers the required HTTPS support and path-based routing, while the X-Forwarded-For header provides the client's original IP address to the microservices, fulfilling all the stated requirements.

Question: 42

CertyIQ

A company is migrating its containerized application to AWS. For the architecture the company will have an ingress VPC with a Network Load Balancer (NLB) to distribute the traffic to front-end pods in an Amazon Elastic Kubernetes Service (Amazon EKS) cluster. The front end of the application will determine which user is requesting access and will send traffic to 1 of 10 services VPCs. Each services VPC will include an NLB that distributes traffic to the services pods in an EKS cluster.

The company is concerned about overall cost. User traffic will be responsible for more than 10 TB of data transfer from the ingress VPC to services VPCs every month. A network engineer needs to recommend how to design the communication between the VPCs.

Which solution will meet these requirements at the LOWEST cost?

A. Create a transit gateway. Peer each VPC to the transit gateway. Use zonal DNS names for the NLB in the services VPCs to minimize cross-AZ traffic from the ingress VPC to the services VPCs.

B. Create an AWS PrivateLink endpoint in every Availability Zone in the ingress VPC. Each PrivateLink endpoint will point to the zonal DNS entry of the NLB in the services VPCs.

C. Create a VPC peering connection between the ingress VPC and each of the 10 services VPCs. Use zonal DNS names for the NLB in the services VPCs to minimize cross-AZ traffic from the ingress VPC to the services VPCs.

D. Create a transit gateway. Peer each VPC to the transit gateway. Turn off cross-AZ load balancing on the transit gateway. Use Regional DNS names for the NLB in the services VPCs.

Answer: C

Explanation:

The most cost-effective solution for high data transfer between the ingress VPC and the 10 services VPCs is VPC peering with zonal NLB DNS names.

Explanation:

Cost Considerations: The primary concern is minimizing costs associated with high data transfer (10+ TB/month). AWS charges for data transfer between Availability Zones (AZs) and across VPCs.

VPC Peering: VPC peering provides a direct, private network connection between VPCs. Data transfer within a VPC peering connection is generally cheaper than routing through a Transit Gateway or using AWS PrivateLink due to the reduced complexity and infrastructure overhead.

Zonal DNS Names: NLBs have both regional and zonal DNS names. Using zonal DNS names for the service NLBs ensures that traffic from the ingress VPC is routed to an NLB in the same Availability Zone, if possible. This significantly reduces or eliminates data transfer charges between Availability Zones.

Why not Transit Gateway? A Transit Gateway introduces additional data processing and associated costs. While Transit Gateway simplifies management for a larger number of VPCs (dozens or hundreds), it's not the most cost-effective option for only 10 VPCs, especially with the high data transfer volume.

Why not PrivateLink? PrivateLink creates VPC endpoints within the ingress VPC, which then connect to Network Load Balancers (NLBs) in the service VPCs. While it offers enhanced security, PrivateLink adds cost complexity and typically doesn't provide a significant cost advantage over VPC peering for this specific scenario.

Why not Transit Gateway with disabled cross-AZ Load Balancing? The Transit Gateway itself still carries cost overhead, and while turning off Cross-AZ Load Balancing may reduce some costs, it forces traffic into single AZs leading to availability challenges.

In summary, VPC peering with zonal NLB DNS names provides a direct, lower-cost path for data transfer, while minimizing or eliminating cross-AZ data transfer charges, addressing the problem statement's concerns.

Supporting Links:

VPC Peering: <https://docs.aws.amazon.com/vpc/latest/peering/what-is-vpc-peering.html>

Network Load Balancer DNS Records:

<https://docs.aws.amazon.com/elasticloadbalancing/latest/network/network-load-balancers.html#availability-zones>

AWS Pricing: <https://aws.amazon.com/pricing/> (check the data transfer section)

Transit Gateway: <https://aws.amazon.com/transit-gateway/>

AWS PrivateLink: <https://aws.amazon.com/privatelink/>

Question: 43

CertyIQ

A company has stateful security appliances that are deployed to multiple Availability Zones in a centralized shared services VPC. The AWS environment includes a transit gateway that is attached to application VPCs and the shared services VPC. The application VPCs have workloads that are deployed in private subnets across multiple Availability Zones. The stateful appliances in the shared services VPC inspect all east west (VPC-to-VPC) traffic. Users report that inter-VPC traffic to different Availability Zones is dropping. A network engineer verified this claim by issuing Internet Control Message Protocol (ICMP) pings between workloads in different Availability Zones across the application VPCs. The network engineer has ruled out security groups, stateful device configurations and network ACLs as the cause of the dropped traffic.

What is causing the traffic to drop?

- A. The stateful appliances and the transit gateway attachments are deployed in a separate subnet in the shared services VPC.
- B. Appliance mode is not enabled on the transit gateway attachment to the shared services VPC.
- C. The stateful appliances and the transit gateway attachments are deployed in the same subnet in the shared services VPC.
- D. Appliance mode is not enabled on the transit gateway attachment to the application VPCs.

Answer: B

Explanation:

Here's a detailed justification for why option B is the correct answer:

The problem describes a scenario where inter-VPC traffic between Availability Zones is being dropped when it traverses stateful security appliances in a shared services VPC via a transit gateway. The key element causing this issue is the lack of "appliance mode" enabled on the transit gateway attachment to the shared services VPC, where the security appliances reside.

Without appliance mode, the transit gateway's default behavior might cause asymmetric routing. Asymmetric routing occurs when the request and response traffic flow through different paths. For example, a packet might enter the security appliance from AZ-A, but the return traffic might attempt to bypass the appliance and go directly back through AZ-B due to the transit gateway's path selection. Stateful appliances require traffic to flow symmetrically; if they only see one direction of a flow, they will drop the packets because they cannot maintain the state of the connection.

Appliance mode, when enabled, ensures that traffic flows symmetrically through the security appliances. It forces the transit gateway to route return traffic for a given flow back through the same appliance instance that initially inspected the traffic, regardless of the destination Availability Zone. This prevents the stateful appliance from dropping packets due to incomplete flow information.

Option A is incorrect because the subnet configuration of the appliances and transit gateway attachments doesn't directly cause asymmetric routing. The subnet merely provides a logical network segment for the resources.

Option C is incorrect because deploying appliances and attachments in the same subnet doesn't inherently cause asymmetric routing. The key issue is the traffic path selection of the transit gateway.

Option D is incorrect because appliance mode on the application VPC attachments isn't relevant to the issue. The problem focuses on traffic inspection by appliances in the shared services VPC. The symmetric routing issue occurs when the transit gateway fails to ensure traffic returns to the same appliance it initially passed through in the shared services VPC.

In summary, enabling appliance mode on the transit gateway attachment to the shared services VPC ensures traffic symmetry, allowing the stateful appliances to correctly inspect and forward traffic between VPCs in different Availability Zones.

Supporting Links:

AWS Transit Gateway Appliance Mode: <https://docs.aws.amazon.com/vpc/latest/tgw/tgw-appliance-mode.html>

AWS Transit Gateway Routing: <https://docs.aws.amazon.com/vpc/latest/tgw/tgw-routing.html>

Question: 44

CertyIQ

A company has hundreds of Amazon EC2 instances that are running in two production VPCs across all Availability Zones in the us-east-1 Region. The production VPCs are named

VPC A and VPC B.

A new security regulation requires all traffic between production VPCs to be inspected before the traffic is routed to its final destination. The company deploys a new shared VPC that contains a stateful firewall appliance and a transit gateway with a VPC attachment across all VPCs to route traffic between VPC A and VPC B through the firewall appliance for inspection. During testing, the company notices that the transit gateway is dropping the traffic whenever the traffic is between two Availability Zones.

What should a network engineer do to fix this issue with the LEAST management overhead?

- A. In the shared VPC, replace the VPC attachment with a VPN attachment. Create a VPN tunnel between the transit gateway and the firewall appliance. Configure BGP.
- B. Enable transit gateway appliance mode on the VPC attachment in VPC A and VPC B.
- C. Enable transit gateway appliance mode on the VPC attachment in the shared VPC.
- D. In the shared VPC, configure one VPC peering connection to VPC A and another VPC peering connection to VPC B.

Answer: C

Explanation:

The correct answer is C: Enable transit gateway appliance mode on the VPC attachment in the shared VPC.

Justification:

The problem describes a scenario where traffic between VPC A and VPC B is being dropped when it traverses Availability Zones via the transit gateway. This is happening because the stateful firewall appliance needs to maintain session affinity (ensuring traffic for a specific session always goes to the same appliance instance). Without appliance mode enabled, the transit gateway might route return traffic from the firewall appliance to a different Availability Zone, causing the stateful firewall to drop the traffic because it doesn't recognize the existing session.

Appliance mode on the transit gateway VPC attachment is designed specifically for scenarios where you have network appliances like firewalls. Enabling appliance mode ensures that traffic entering and exiting the VPC attachment is always routed through the same Availability Zone, thus preserving session affinity. This prevents asymmetrical routing, where the initial request and response take different paths, which is crucial for stateful firewalls.

Option A is incorrect because replacing the VPC attachment with a VPN attachment introduces unnecessary complexity and management overhead. VPNs are better suited for connecting to on-premises networks or remote sites, not for internal VPC traffic within a region. Configuring BGP also adds to the management burden.

Option B is incorrect because enabling appliance mode on the VPC attachments in VPC A and VPC B is not the appropriate solution. Appliance mode needs to be enabled on the transit gateway's VPC attachment within the shared VPC where the firewall appliance resides. This tells the transit gateway to route traffic through the firewall appliance's AZ consistently.

Option D is incorrect because VPC peering connections do not work with Transit Gateways to provide a central routing point, and is not designed to facilitate routing through the security appliance. This would negate the purpose of the transit gateway and firewall appliance architecture.

Enabling appliance mode on the shared VPC's transit gateway attachment provides the simplest and most direct solution to ensure traffic for a specific session remains within the same Availability Zone, allowing the stateful firewall to function correctly and resolve the dropped traffic issue. This approach minimizes management overhead while satisfying the requirement for traffic inspection.

Authoritative Links:

AWS Transit Gateway Appliance Mode: <https://docs.aws.amazon.com/transit-gateway/latest/userguide/tgw->

Question: 45

A company has deployed a critical application on a fleet of Amazon EC2 instances behind an Application Load Balancer. The application must always be reachable on port 443 from the public internet. The application recently had an outage that resulted from an incorrect change to the EC2 security group.

A network engineer needs to automate a way to verify the network connectivity between the public internet and the EC2 instances whenever a change is made to the security group. The solution also must notify the network engineer when the change affects the connection.

Which solution will meet these requirements?

- A. Enable VPC Flow Logs on the elastic network interface of each EC2 instance to capture REJECT traffic on port 443. Publish the flow log records to a log group in Amazon CloudWatch Logs. Create a CloudWatch Logs metric filter for the log group for rejected traffic. Create an alarm to notify the network engineer.
- B. Enable VPC Flow Logs on the elastic network interface of each EC2 instance to capture all traffic on port 443. Publish the flow log records to a log group in Amazon CloudWatch Logs. Create a CloudWatch Logs metric filter for the log group for all traffic. Create an alarm to notify the network engineer.
- C. Create a VPC Reachability Analyzer path on port 443. Specify the security group as the source. Specify the EC2 instances as the destination. Create an Amazon Simple Notification Service (Amazon SNS) topic to notify the network engineer when a change to the security group affects the connection. Create an AWS Lambda function to start Reachability Analyzer and to publish a message to the SNS topic in case the analyses fail. Create an Amazon EventBridge (Amazon CloudWatch Events) rule to invoke the Lambda function when a change to the security group occurs.
- D. Create a VPC Reachability Analyzer path on port 443. Specify the internet gateway of the VPC as the source. Specify the EC2 instances as the destination. Create an Amazon Simple Notification Service (Amazon SNS) topic to notify the network engineer when a change to the security group affects the connection. Create an AWS Lambda function to start Reachability Analyzer and to publish a message to the SNS topic in case the analyses fail. Create an Amazon EventBridge (Amazon CloudWatch Events) rule to invoke the Lambda function when a change to the security group occurs.

Answer: D**Explanation:**

The correct answer is D because it provides a complete and automated solution for verifying network connectivity from the public internet to the EC2 instances after security group changes. Here's a breakdown of why it's the best choice:

1. **VPC Reachability Analyzer:** VPC Reachability Analyzer is a network diagnostics tool that helps you analyze and troubleshoot network reachability between two endpoints in your VPC. By creating a path on port 443, you can specifically test the connectivity required for the application.
[\[https://docs.aws.amazon.com/vpc/latest/reachability/what-is-reachability-analyzer.html\]](https://docs.aws.amazon.com/vpc/latest/reachability/what-is-reachability-analyzer.html)
2. **Source and Destination:** Specifying the internet gateway as the source accurately simulates traffic originating from the public internet. The EC2 instances are the intended destination of this traffic, ensuring the test reflects the real-world scenario.
3. **Automation and Notification:** The solution uses several services to achieve automation and notification upon failure:

Amazon SNS Topic: The SNS topic facilitates notification to the network engineer when the reachability analysis fails.

AWS Lambda Function: The Lambda function is the key to automation. It starts the Reachability Analyzer and publishes a message to the SNS topic if connectivity fails.

Amazon EventBridge (CloudWatch Events) Rule: This rule triggers the Lambda function whenever

there's a change to the security group. This allows you to react instantly when your critical security configurations are modified.

4. **Comprehensive Solution:** This approach is comprehensive because it proactively verifies connectivity after security group changes. It leverages the combination of tools to test network paths and provide automatic alerts, allowing the network engineer to immediately investigate and rectify the connectivity issues caused by the modifications.

Why other options are not the best:

Option A & B (VPC Flow Logs): While VPC Flow Logs can capture rejected traffic, they are reactive. They only detect failures after they happen. They don't proactively verify connectivity after a security group change. It also doesn't address simulating traffic from the internet. Furthermore, filtering all traffic (option B) is inefficient and generates unnecessary logs. [<https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs.html>]

Option C (Security Group as Source): Using the security group as the source in the Reachability Analyzer isn't appropriate to test connectivity from the internet. The source should represent where the traffic is originating from, which is the public internet. The security group is an access control mechanism, not a source of network traffic.

Question: 46

CertyIQ

A security team is performing an audit of a company's AWS deployment. The security team is concerned that two applications might be accessing resources that should be blocked by network ACLs and security groups. The applications are deployed across two Amazon Elastic Kubernetes Service (Amazon EKS) clusters that use the Amazon VPC Container Network Interface (CNI) plugin for Kubernetes. The clusters are in separate subnets within the same VPC and have a Cluster Autoscaler configured.

The security team needs to determine which POD IP addresses are communicating with which services throughout the VPC. The security team wants to limit the number of flow logs and wants to examine the traffic from only the two applications.

Which solution will meet these requirements with the LEAST operational overhead?

- A. Create VPC flow logs in the default format. Create a filter to gather flow logs only from the EKS nodes. Include the srcaddr field and the dstaddr field in the flow logs.
- B. Create VPC flow logs in a custom format. Set the EKS nodes as the resource. Include the pkt-srcaddr field and the pkt-dstaddr field in the flow logs.
- C. Create VPC flow logs in a custom format. Set the application subnets as resources. Include the pkt-srcaddr field and the pkt-dstaddr field in the flow logs.
- D. Create VPC flow logs in a custom format. Create a filter to gather flow logs only from the EKS nodes. Include the pkt-srcaddr field and the pkt-dstaddr field in the flow logs.

Answer: C

Explanation:

The correct answer is **C**. Here's a detailed justification:

The goal is to identify the pod IP addresses involved in communication between two specific applications across two EKS clusters within the same VPC, while minimizing operational overhead and limiting flow logs to only traffic relevant to the applications.

Option A is incorrect because creating VPC flow logs in the default format and filtering by EKS nodes will capture traffic not related to the applications of interest. This increases the volume of logs and adds operational overhead by requiring filtering of the relevant flows. The default VPC flow log format might not provide the necessary level of packet-level detail (specifically pod IP addresses).

Option B is incorrect because while using a custom format and focusing on EKS nodes is better than logging

everything, it still misses the core requirement: filtering logs specifically for the application subnets, not just the nodes. This is because traffic flows from different PODs within those subnets. Additionally, VPC Flow Logs do not associate directly with EKS nodes as a resource.

Option D is incorrect because while using a custom format and filtering EKS nodes helps reducing overhead, it is still more effective to specify subnets instead.

Option C is the most appropriate solution because:

1. **Targets Application Traffic:** Setting the application subnets as the resource for VPC flow logs ensures that only traffic originating from or destined to those subnets (where the applications reside) is captured. This directly addresses the requirement of focusing only on the two applications' traffic.
2. **Includes Pod IP Addresses:** The `pkt-srcaddr` and `pkt-destaddr` fields in the custom format provide the source and destination IP addresses at the packet level. When using the Amazon VPC CNI plugin for Kubernetes, these IP addresses will be the pod IP addresses. This allows the security team to determine which specific pods are communicating with each other and with other services.
3. **Minimizes Operational Overhead:** By focusing on the application subnets, the volume of flow logs is minimized, reducing storage costs and the effort required to analyze the logs. There is no additional filtering required.

Therefore, creating VPC flow logs in a custom format with the application subnets as the resource and including the `pkt-srcaddr` and `pkt-destaddr` fields is the most efficient and effective way to meet the security team's requirements.

Supporting resources:

VPC Flow Logs: <https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs.html>

VPC Flow Log Record Syntax: <https://docs.aws.amazon.com/vpc/latest/userguide/flow-log-records.html>

Amazon VPC CNI plugin for Kubernetes: <https://docs.aws.amazon.com/eks/latest/userguide/pod-networking.html>

Question: 47

CertyIQ

A data analytics company has a 100-node high performance computing (HPC) cluster. The HPC cluster is for parallel data processing and is hosted in a VPC in the AWS Cloud. As part of the data processing workflow, the HPC cluster needs to perform several DNS queries to resolve and connect to Amazon RDS databases, Amazon S3 buckets, and on-premises data stores that are accessible through AWS Direct Connect. The HPC cluster can increase in size by five to seven times during the company's peak event at the end of the year. The company is using two Amazon EC2 instances as primary DNS servers for the VPC. The EC2 instances are configured to forward queries to the default VPC resolver for Amazon Route 53 hosted domains and to the on-premises DNS servers for other on-premises hosted domain names. The company notices job failures and finds that DNS queries from the HPC cluster nodes failed when the nodes tried to resolve RDS and S3 bucket endpoints. Which architectural change should a network engineer implement to provide the DNS service in the MOST scalable way?

- A. Scale out the DNS service by adding two additional EC2 instances in the VPC. Reconfigure half of the HPC cluster nodes to use these new DNS servers. Plan to scale out by adding additional EC2 instance-based DNS servers in the future as the HPC cluster size grows.
- B. Scale up the existing EC2 instances that the company is using as DNS servers. Change the instance size to the largest possible instance size to accommodate the current DNS load and the anticipated load in the future.
- C. Create Route 53 Resolver outbound endpoints. Create Route 53 Resolver rules to forward queries to on-premises DNS servers for on-premises hosted domain names. Reconfigure the HPC cluster nodes to use the default VPC resolver instead of the EC2 instance-based DNS servers. Terminate the EC2 instances.
- D. Create Route 53 Resolver inbound endpoints. Create rules on the on-premises DNS servers to forward queries to the default VPC resolver. Reconfigure the HPC cluster nodes to forward all DNS queries to the on-premises DNS servers. Terminate the EC2 instances.

Answer: C

Explanation:

The best architectural change to address the DNS resolution issues and scalability requirements is **C. Create Route 53 Resolver outbound endpoints. Create Route 53 Resolver rules to forward queries to on-premises DNS servers for on premises hosted domain names. Reconfigure the HPC cluster nodes to use the default VPC resolver instead of the EC2 instance-based DNS servers. Terminate the EC2 instances.**

Here's why:

Scalability and Availability: Route 53 Resolver is a highly scalable and available managed DNS service. It automatically scales to handle a large number of DNS queries, eliminating the need for manual scaling and management as the HPC cluster grows. This contrasts with solutions A and B, which involve scaling EC2 instances, a less scalable and more management-intensive approach.

Integration with AWS Services: Route 53 Resolver integrates seamlessly with other AWS services like RDS and S3. The default VPC resolver handles DNS resolution for public AWS service endpoints. Using the VPC resolver, the HPC nodes will consistently and reliably resolve DNS for RDS and S3.

Hybrid DNS Resolution: Option C allows for hybrid DNS resolution. Route 53 Resolver outbound endpoints enable the VPC to forward DNS queries to on-premises DNS servers via Direct Connect for on-premises hosted domain names. This ensures the HPC cluster can resolve both AWS and on-premises resources.

Reduced Management Overhead: By using Route 53 Resolver, the company can eliminate the need to manage and maintain EC2 instance-based DNS servers, reducing operational overhead. This simplifies the DNS infrastructure and frees up resources for other tasks.

Reliability: The Amazon Route 53 Resolver is highly reliable because it is a managed service by AWS, which offers a fully managed DNS infrastructure for both on-premises and cloud resources. This service scales automatically to serve large volumes of DNS queries without any need of manual intervention.

Options A and B are less desirable because they involve managing EC2 instances for DNS, which adds complexity and operational overhead. Option D is less efficient because it requires forwarding all DNS queries to the on-premises DNS servers, which can introduce latency and increase the load on the on-premises infrastructure.

Authoritative Links:

[Route 53 Resolver](#)

[How Route 53 Resolver Works](#)

Question: 48

CertyIQ

A company's network engineer is designing an active-passive connection to AWS from two on-premises data centers. The company has set up AWS Direct Connect connections between the on-premises data centers and AWS. From each location, the company is using a transit VIF that connects to a Direct Connect gateway that is associated with a transit gateway.

The network engineer must ensure that traffic from AWS to the data centers is routed first to the primary data center. The traffic should be routed to the failover data center only in the case of an outage.

Which solution will meet these requirements?

A. Set the BGP community tag for all prefixes from the primary data center to 7224:7100. Set the BGP community tag for all prefixes from the failover data center to 7224:7300

B. Set the BGP community tag for all prefixes from the primary data center to 7224:7300. Set the BGP community tag for all prefixes from the failover data center to 7224:7100

C. Set the BGP community tag for all prefixes from the primary data center to 7224:9300. Set the BGP community tag for all prefixes from the failover data center to 7224:9100

D. Set the BGP community tag for all prefixes from the primary data center to 7224:9100. Set the BGP

community tag for all prefixes from the failover data center to 7224:9300

Answer: B

Explanation:

The correct answer is B. Here's why:

AWS uses specific BGP community tags to influence routing decisions when using Direct Connect and Transit Gateways. To achieve active-passive routing, we need to prioritize the primary data center over the failover data center for traffic originating from AWS. This is achieved using the 7224:7300 and 7224:7100 BGP community tags.

Specifically, the 7224:7300 BGP community tag influences the Transit Gateway to prefer the path advertising prefixes with this tag. Conversely, 7224:7100 indicates a lower preference.

Therefore, by setting the BGP community tag for prefixes advertised from the primary data center to 7224:7300, we tell the Transit Gateway to prefer this path when routing traffic back to the on-premises networks. Setting the BGP community tag for prefixes advertised from the failover data center to 7224:7100 makes the Transit Gateway use this path only when the primary path is unavailable. This ensures that traffic flows to the primary data center under normal circumstances and fails over to the failover data center only in case of an outage.

Options C and D use the 7224:9100 and 7224:9300 BGP community tags, which are used to control prefix advertisement to specific AWS Regions and are irrelevant to achieving active/passive routing to on-premises data centers. Option A incorrectly assigns tags which would make the failover data center the preferred path.

Refer to the AWS documentation on Direct Connect routing policies for more details:

<https://docs.aws.amazon.com/directconnect/latest/UserGuide/routing-policies.html>

Question: 49

CertyIQ

A real estate company is building an internal application so that real estate agents can upload photos and videos of various properties. The application will store these photos and videos in an Amazon S3 bucket as objects and will use Amazon DynamoDB to store corresponding metadata. The S3 bucket will be configured to publish all PUT events for new object uploads to an Amazon Simple Queue Service (Amazon SQS) queue.

A compute cluster of Amazon EC2 instances will poll the SQS queue to find out about newly uploaded objects. The cluster will retrieve new objects, perform proprietary image and video recognition and classification update metadata in DynamoDB and replace the objects with new watermarked objects. The company does not want public IP addresses on the EC2 instances.

Which networking design solution will meet these requirements MOST cost-effectively as application usage increases?

A. Place the EC2 instances in a public subnet. Disable the Auto-assign Public IP option while launching the EC2 instances. Create an internet gateway. Attach the internet gateway to the VPC. In the public subnet's route table, add a default route that points to the internet gateway.

B. Place the EC2 instances in a private subnet. Create a NAT gateway in a public subnet in the same Availability Zone. Create an internet gateway. Attach the internet gateway to the VPC. In the public subnet's route table, add a default route that points to the internet gateway.

C. Place the EC2 instances in a private subnet. Create an interface VPC endpoint for Amazon SQS. Create gateway VPC endpoints for Amazon S3 and DynamoDB.

D. Place the EC2 instances in a private subnet. Create a gateway VPC endpoint for Amazon SQS. Create interface VPC endpoints for Amazon S3 and DynamoDB.

Answer: C

Explanation:

The correct solution (C) offers the most cost-effective and secure networking design as application usage increases.

Option C places the EC2 instances in a private subnet, ensuring they don't have public IP addresses, which aligns with the security requirement. It then leverages VPC endpoints for accessing AWS services. Gateway VPC endpoints are used for S3 and DynamoDB. Gateway endpoints are free and provide reliable connectivity to S3 and DynamoDB. <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html>

An interface VPC endpoint for SQS is used because the EC2 instances need to poll SQS. Gateway endpoints don't support SQS. Interface endpoints use PrivateLink, allowing private connectivity to SQS over the AWS network.

Option A is incorrect because it places instances in a public subnet, even with public IP assignment disabled. This doesn't completely isolate them, and using an Internet Gateway adds unnecessary complexity.

Option B uses a NAT Gateway. While it provides outbound internet access, it's significantly more expensive than VPC endpoints and unnecessary for accessing S3, DynamoDB, and SQS.

Option D is incorrect because it swaps the endpoint types for SQS and S3/DynamoDB. Gateway endpoints do not work for SQS, while they are the most cost-effective solution for S3 and DynamoDB.

Therefore, option C provides a secure, private, and cost-effective solution by utilizing VPC endpoints for accessing AWS services without traversing the public internet or incurring NAT Gateway charges.

Question: 50**CertyIQ**

A company has an AWS Direct Connect connection between its on-premises data center in the United States (US) and workloads in the us-east-1 Region. The connection uses a transit VIF to connect the data center to a transit gateway in us-east-1.

The company is opening a new office in Europe with a new on-premises data center in England. A Direct Connect connection will connect the new data center with some workloads that are running in a single VPC in the eu-west-2 Region. The company needs to connect the US data center and us-east-1 with the Europe data center and eu-west-2. A network engineer must establish full connectivity between the data centers and Regions with the lowest possible latency.

How should the network engineer design the network architecture to meet these requirements?

A. Connect the VPC in eu-west-2 with the Europe data center by using a Direct Connect gateway and a private VIF. Associate the transit gateway in us-east-1 with the same Direct Connect gateway. Enable SiteLink for the transit VIF and the private VIF.

B. Connect the VPC in eu-west-2 to a new transit gateway. Connect the Europe data center to the new transit gateway by using a Direct Connect gateway and a new transit VIF. Associate the transit gateway in us-east-1 with the same Direct Connect gateway. Enable SiteLink for both transit VIFs. Peer the two transit gateways.

C. Connect the VPC in eu-west-2 to a new transit gateway. Connect the Europe data center to the new transit gateway by using a Direct Connect gateway and a new transit VIF. Create a new Direct Connect gateway. Associate the transit gateway in us-east-1 with the new Direct Connect gateway. Enable SiteLink for both transit VIFs. Peer the two transit gateways.

D. Connect the VPC in eu-west-2 with the Europe data center by using a Direct Connect gateway and a private VIF. Create a new Direct Connect gateway. Associate the transit gateway in us-east-1 with the new Direct Connect gateway. Enable SiteLink for the transit VIF and the private VIF.

Answer: B**Explanation:**

Here's a detailed justification for why option B is the most suitable solution, and why the other options are less

ideal, focusing on the lowest latency requirement:

Option B: Correct Solution

Option B proposes connecting the eu-west-2 VPC to a new transit gateway, connecting the Europe data center to this new transit gateway using a Direct Connect gateway and a new transit VIF, associating the transit gateway in us-east-1 with the same Direct Connect gateway, enabling SiteLink for both transit VIFs, and peering the two transit gateways.

Regional Separation & Lowest Latency: Connecting the EU data center to a transit gateway in the EU region (using a Direct Connect gateway and a transit VIF) ensures traffic stays within Europe for communication with EU resources. This minimizes latency compared to routing traffic back to the US.

Transit Gateway for VPC connectivity: Using a transit gateway allows simple connections between the VPC in eu-west-2 and the Europe data center. This ensures seamless connectivity between these resources.

Direct Connect Gateway Sharing: The crux of this design is that both transit gateways (us-east-1 and eu-west-2) associate with the same Direct Connect gateway. SiteLink is then enabled on both transit VIFs (US and EU).

SiteLink for Direct Connectivity: SiteLink allows direct communication between the on-premises locations connected to the same Direct Connect Gateway, bypassing the need to traverse the AWS backbone for on-premises to on-premises communication. This is crucial for minimizing latency between the US and Europe data centers.

Transit Gateway Peering: Peering the transit gateways enables connectivity between the US and EU regions while using the Direct Connect SiteLink for lowest latency between on-premise locations.

Why other options are not ideal:

Option A: Using a private VIF instead of a transit VIF for the Europe data center would prevent seamless and easy integration with other AWS services in the eu-west-2 Region without more complex routing configurations. Furthermore, without a separate transit gateway in the EU, traffic from the EU data center to the us-east-1 region and US data center would have to go through the US East region.

Option C: Creates a new Direct Connect gateway which adds cost and complexity. Also since it's a new Direct Connect gateway, there is no SiteLink possible.

Option D: Similar to Option A, this uses a private VIF, which has drawbacks for scalability. Also, creating a new Direct Connect gateway does not enable SiteLink.

Authoritative Links:

AWS Direct Connect Gateways: <https://docs.aws.amazon.com/directconnect/latest/UserGuide/direct-connect-gateways-intro.html>

AWS Transit Gateway: <https://docs.aws.amazon.com/vpc/latest/tgw/what-is-transit-gateway.html>

AWS Direct Connect SiteLink: <https://aws.amazon.com/about-aws/whats-new/2023/11/aws-direct-connect-sitelink/>

Question: 51

CertyIQ

A network engineer has deployed an Amazon EC2 instance in a private subnet in a VPC. The VPC has no public subnet. The EC2 instance hosts application code that sends messages to an Amazon Simple Queue Service (Amazon SQS) queue. The subnet has the default network ACL with no modification applied. The EC2 instance has the default security group with no modification applied.

The SQS queue is not receiving messages.

Which of the following are possible causes of this problem? (Choose two.)

- A. The EC2 instance is not attached to an IAM role that allows write operations to Amazon SQS.
- B. The security group is blocking traffic to the IP address range used by Amazon SQS
- C. There is no interface VPC endpoint configured for Amazon SQS

D.The network ACL is blocking return traffic from Amazon SQS

E.There is no route configured in the subnet route table for the IP address range used by Amazon SQS

Answer: AC

Explanation:

Here's a detailed justification for why options A and C are the correct answers, and why the others are incorrect, in the context of the scenario provided.

Why A is correct: The EC2 instance is not attached to an IAM role that allows write operations to Amazon SQS.

The EC2 instance needs permission to interact with the SQS queue. This is managed through IAM roles. If the EC2 instance doesn't have an IAM role attached, or if the attached role lacks the necessary permissions (specifically, `sqs:SendMessage` or `sqs:*` for broader access) to write to the SQS queue, the application code will be unable to send messages, causing the reported problem. IAM roles are the standard and secure way to grant permissions to EC2 instances to access AWS services. Without the appropriate permissions, the application code will likely encounter an "Access Denied" error when attempting to interact with the SQS queue.

Why C is correct: There is no interface VPC endpoint configured for Amazon SQS.

Since the EC2 instance resides in a private subnet without a public subnet, it doesn't have direct access to the internet. To communicate with SQS, which is a public AWS service, a mechanism for private connectivity is needed. An Interface VPC Endpoint for SQS provides this connectivity. It allows traffic to flow from the EC2 instance to SQS without traversing the internet. Without a VPC Endpoint, the EC2 instance would need a NAT Gateway or a NAT instance to access SQS, and even with a NAT Gateway, the proper routing would be necessary. Because the question doesn't mention any NAT setup, the absence of a VPC endpoint directly prevents the EC2 instance from reaching the SQS service.

Why B is incorrect: The security group is blocking traffic to the IP address range used by Amazon SQS.

The default security group allows all outbound traffic. Unless modified, the security group will not block the EC2 instance from sending traffic. While a restrictive security group could block this traffic, the problem description explicitly states the default security group is in use, eliminating this as a likely cause.

Why D is incorrect: The network ACL is blocking return traffic from Amazon SQS.

The default network ACL allows all inbound and outbound traffic. Unless modified, the network ACL will not block the EC2 instance from receiving responses from SQS or from sending the initial request. While a restrictive network ACL could cause issues, the problem description explicitly states the default network ACL is in use, eliminating this as a likely cause.

Why E is incorrect: There is no route configured in the subnet route table for the IP address range used by Amazon SQS

This is related to option C but less direct. Without a VPC Endpoint, a route to a NAT Gateway (or NAT instance) would be required for the private subnet to access the internet and thus SQS. However, the primary issue is the lack of a secure private connection mechanism. The explicit mention of no VPC Endpoint makes option C a more accurate and direct cause. Routing problems would manifest if a VPC endpoint was not used and the NAT configuration was incorrect, but the endpoint absence is the primary problem.

Supporting Links:

IAM Roles for EC2: https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles_use_ec2.html

VPC Endpoints: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html>

Question: 52

CertyIQ

A network engineer needs to standardize a company's approach to centralizing and managing interface VPC endpoints for private communication with AWS services. The company uses AWS Transit Gateway for inter-VPC connectivity between AWS accounts through a hub-and-spoke model. The company's network services team must manage all Amazon Route 53 zones and interface endpoints within a shared services AWS account. The company wants to use this centralized model to provide AWS resources with access to AWS Key Management Service (AWS KMS) without sending traffic over the public internet.

What should the network engineer do to meet these requirements?

A. In the shared services account, create an interface endpoint for AWS KMS. Modify the interface endpoint by disabling the private DNS name. Create a private hosted zone in the shared services account with an alias record that points to the interface endpoint. Associate the private hosted zone with the spoke VPCs in each AWS account.

B. In the shared services account, create an interface endpoint for AWS KMS. Modify the interface endpoint by disabling the private DNS name. Create a private hosted zone in each spoke AWS account with an alias record that points to the interface endpoint. Associate each private hosted zone with the shared services AWS account.

C. In each spoke AWS account, create an interface endpoint for AWS KMS. Modify each interface endpoint by disabling the private DNS name. Create a private hosted zone in each spoke AWS account with an alias record that points to each interface endpoint. Associate each private hosted zone with the shared services AWS account.

D. In each spoke AWS account, create an interface endpoint for AWS KMS. Modify each interface endpoint by disabling the private DNS name. Create a private hosted zone in the shared services account with an alias record that points to each interface endpoint. Associate the private hosted zone with the spoke VPCs in each AWS account.

Answer: A

Explanation:

The correct answer is A because it aligns with the principles of centralized management, private connectivity, and DNS resolution within the AWS environment using Transit Gateway and Route 53.

Here's a breakdown:

Centralized Interface Endpoint: Creating the KMS interface endpoint in the shared services account centralizes management, which is a key requirement. This allows the network services team to control access to KMS from a single location.

Private Connectivity: Using an interface endpoint ensures that traffic to KMS does not traverse the public internet, meeting the requirement for secure private communication.

Disabling Private DNS Name: Disabling the private DNS name on the interface endpoint prevents automatic resolution within the VPC where the endpoint is created. This is necessary because we want to manage DNS centrally in the shared services account.

Private Hosted Zone in Shared Services: Creating a private hosted zone in the shared services account provides a central DNS management point for the KMS endpoint.

Alias Record: The alias record within the private hosted zone maps the standard KMS endpoint name (e.g., kms.us-east-1.amazonaws.com) to the interface endpoint's DNS name or IP addresses.

Associating with Spoke VPCs: Associating the private hosted zone with the spoke VPCs allows resources in those VPCs to resolve the KMS endpoint name to the private IP addresses of the interface endpoint. This is how the spoke VPCs will access KMS through the interface endpoint.

Transit Gateway: Although not explicitly used in DNS resolution, the Transit Gateway provides the underlying network connectivity between the spoke VPCs and the shared services VPC, ensuring that traffic to the

interface endpoint stays within the AWS network.

Options B, C, and D are incorrect for the following reasons:

B: Creating private hosted zones in each spoke account defeats the purpose of centralized management.

C: Creating interface endpoints in each spoke account also defeats the purpose of centralized management and increases operational overhead.

D: Creating interface endpoints in each spoke and centralizing the DNS zone in the shared service account is not suitable as it requires to keep track of all endpoints, as the DNS needs to map to the individual IPs of each created interface endpoint, which defeats the purpose of the centralized management.

Authoritative Links:

AWS PrivateLink: <https://aws.amazon.com/privatelink/>

Interface VPC Endpoints: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints-interface.html>

Amazon Route 53 Private Hosted Zones:

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/private-hosted-zones.html>

AWS Transit Gateway: <https://aws.amazon.com/transit-gateway/>

Question: 53

CertyIQ

A development team is building a new web application in the AWS Cloud. The main company domain, example.com, is currently hosted in an Amazon Route 53 public hosted zone in one of the company's production AWS accounts. The developers want to test the web application in the company's staging AWS account by using publicly resolvable subdomains under the example.com domain with the ability to create and delete DNS records as needed. Developers have full access to Route 53 hosted zones within the staging account, but they are prohibited from accessing resources in any of the production AWS accounts.

Which combination of steps should a network engineer take to allow the developers to create records under the example.com domain? (Choose two.)

A. Create a public hosted zone for example.com in the staging account

B. Create a staging example.com NS record in the example.com domain. Populate the value with the name servers from the staging.example.com domain. Set the routing policy type to simple routing.

C. Create a private hosted zone for staging example.com in the staging account.

D. Create an example.com NS record in the staging example.com domain. Populate the value with the name servers from the example.com domain. Set the routing policy type to simple routing.

E. Create a public hosted zone for staging.example.com in the staging account.

Answer: BE

Explanation:

Let's break down why options B and E are the correct choices to enable subdomain delegation from the production AWS account to the staging AWS account for DNS management, focusing on Route 53 and DNS delegation concepts.

Option E: "Create a public hosted zone for staging.example.com in the staging account" is crucial. The staging account needs its own dedicated hosted zone to manage DNS records specifically for the staging.example.com subdomain. This isolates the staging environment's DNS configuration from the production environment and allows the developers in the staging account to have full control. Without this, the staging environment would not be able to manage its DNS records.

Option B: "Create a staging example.com NS record in the example.com domain. Populate the value with the name servers from the staging.example.com domain. Set the routing policy type to simple routing." This step creates a delegation record within the production example.com hosted zone that points to the authoritative name servers for the staging.example.com hosted zone created in the staging account. An NS record is

specifically used for delegating a subdomain to a different set of name servers (and therefore, a different hosted zone). The "simple routing" policy is suitable as the primary purpose is delegation. This delegation is what allows DNS queries for staging.example.com and its subdomains to be correctly resolved by the name servers in the staging account.

Option A is incorrect because creating another public hosted zone for example.com in the staging account would lead to DNS conflicts. There can only be one authoritative hosted zone for a domain (in this case, in the production account). Option C is incorrect because a public hosted zone is needed for publicly resolvable subdomains as stated in the problem. A private hosted zone would only resolve within specified VPCs. Option D is reversed; the NS record needs to be in the parent domain (example.com) pointing to the name servers of the subdomain (staging.example.com).

In summary, to delegate control of a subdomain, you must create a hosted zone for the subdomain, then create an NS record for that subdomain in the parent zone, pointing to the subdomain's name servers.

Supporting resources:

AWS Route 53 Documentation: <https://docs.aws.amazon.com/route53/>

Delegating Subdomains: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/dns-configuring-dns-delegation.html>

NS Record: https://en.wikipedia.org/wiki/NS_record

Question: 54

CertyIQ

A company plans to deploy a two-tier web application to a new VPC in a single AWS Region. The company has configured the VPC with an internet gateway and four subnets. Two of the subnets are public and have default routes that point to the internet gateway. Two of the subnets are private and share a route table that does not have a default route.

The application will run on a set of Amazon EC2 instances that will be deployed behind an external Application Load Balancer. The EC2 instances must not be directly accessible from the internet. The application will use an Amazon S3 bucket in the same Region to store data. The application will invoke S3 GET API operations and S3 PUT API operations from the EC2 instances. A network engineer must design a VPC architecture that minimizes data transfer cost.

Which solution will meet these requirements?

- A. Deploy the EC2 instances in the public subnets. Create an S3 interface endpoint in the VPC. Modify the application configuration to use the S3 endpoint-specific DNS hostname.
- B. Deploy the EC2 instances in the private subnets. Create a NAT gateway in the VPC. Create default routes in the private subnets to the NAT gateway. Connect to Amazon S3 by using the NAT gateway.
- C. Deploy the EC2 instances in the private subnets. Create an S3 gateway endpoint in the VPC. Specify the route table of the private subnets during endpoint creation to create routes to Amazon S3.
- D. Deploy the EC2 instances in the private subnets. Create an S3 interface endpoint in the VPC. Modify the application configuration to use the S3 endpoint-specific DNS hostname.

Answer: C

Explanation:

The correct answer is C because it provides the most cost-effective and secure way for EC2 instances in private subnets to access S3, aligning with the requirements of the scenario.

Here's a detailed justification:

EC2 instances in private subnets: Deploying EC2 instances in private subnets ensures they are not directly accessible from the internet, fulfilling the security requirement.

S3 Gateway Endpoint: A Gateway Endpoint for S3 is the most cost-effective solution because it's free.

Interface Endpoints incur data processing charges.

Gateway Endpoint Functionality: Gateway Endpoints work by adding a route to the specified route table. Traffic destined for S3 is routed directly from the private subnets to S3 over the AWS backbone network, bypassing the internet. This fulfills the requirement of minimizing data transfer costs, as traffic to S3 doesn't go through a NAT gateway which incurs costs.

Route Table Association: Specifying the route table of the private subnets during endpoint creation ensures that the route to S3 is automatically added to those route tables. EC2 instances will then automatically use this route to communicate with S3.

Why other options are incorrect:

A: Placing instances in public subnets violates the requirement that they should not be directly accessible from the Internet.

B: Using a NAT Gateway to access S3 is more expensive than using a Gateway Endpoint as NAT Gateways charge for data processed.

D: While an S3 interface endpoint provides secure access to S3, it incurs data processing charges. A Gateway Endpoint is more cost-effective for S3 access within the same region.

In summary, answer C provides a solution that satisfies all requirements by deploying EC2 instances in private subnets, offering private connectivity to S3 through a gateway endpoint, and optimizing cost by utilizing the free Gateway Endpoint for S3.

Relevant links for further research:

[VPC Endpoints](#)

[Gateway Endpoints](#)

[Interface Endpoints](#)

Question: 55

CertyIQ

A company has two AWS accounts one for Production and one for Connectivity. A network engineer needs to connect the Production account VPC to a transit gateway in the Connectivity account. The feature to auto accept shared attachments is not enabled on the transit gateway.

Which set of steps should the network engineer follow in each AWS account to meet these requirements?

- A.1. In the Production account: Create a resource share in AWS Resource Access Manager for the transit gateway. Provide the Connectivity account ID. Enable the feature to allow external accounts
2. In the Connectivity account: Accept the resource.
3. In the Connectivity account: Create an attachment to the VPC subnets.
4. In the Production account: Accept the attachment. Associate a route table with the attachment.
- B.1. In the Production account: Create a resource share in AWS Resource Access Manager for the VPC subnets. Provide the Connectivity account ID. Enable the feature to allow external accounts.
2. In the Connectivity account: Accept the resource.
3. In the Production account: Create an attachment on the transit gateway to the VPC subnets.
4. In the Connectivity account: Accept the attachment. Associate a route table with the attachment.
- C.1. In the Connectivity account: Create a resource share in AWS Resource Access Manager for the VPC subnets. Provide the Production account ID. Enable the feature to allow external accounts.
2. In the Production account: Accept the resource.
3. In the Connectivity account: Create an attachment on the transit gateway to the VPC subnets.
4. In the Production account: Accept the attachment. Associate a route table with the attachment.
- D.1. In the Connectivity account: Create a resource share in AWS Resource Access Manager for the transit gateway. Provide the Production account ID. Enable the feature to allow external accounts.
2. In the Production account: Accept the resource.
3. In the Production account: Create an attachment to the VPC subnets.
4. In the Connectivity account: Accept the attachment. Associate a route table with the attachment.

Answer: D

Explanation:

The correct answer is D because it outlines the necessary steps for connecting a Production account VPC to a Transit Gateway in a Connectivity account without auto-accept enabled, using AWS Resource Access Manager (RAM).

Here's a breakdown of why answer D is correct and why the other options are incorrect:

Step 1 (Connectivity Account): The Connectivity account owns the Transit Gateway. Therefore, the resource share needs to be created here, sharing the Transit Gateway with the Production account using RAM. Enabling the feature to allow external accounts is essential when sharing resources across AWS accounts. <https://docs.aws.amazon.com/ram/latest/userguide/getting-started-sharing.html>

Step 2 (Production Account): The Production account needs to accept the resource share invitation to gain access to the shared Transit Gateway.

Step 3 (Production Account): The Production account creates the Transit Gateway attachment from its VPC subnets to the Transit Gateway that was shared. Since auto-accept is disabled, an explicit attachment needs to be made. The attachment originates from the VPC wanting the connection. <https://docs.aws.amazon.com/vpc/latest/tgw/tgw-attachments.html>

Step 4 (Connectivity Account): Because auto-accept is disabled, the Connectivity account (the Transit Gateway owner) must explicitly accept the attachment request initiated by the Production account. Finally, the Transit Gateway route table must be updated to route traffic through the attachment.

Why other options are incorrect:

Option A: Incorrect because the Production account cannot share the transit gateway. The Connectivity account, which is where the Transit Gateway exists, is responsible for sharing the resource. Also, the association with the attachment comes from the Connectivity account, not Production, at the beginning.

Option B: Incorrect because it attempts to share VPC subnets using RAM, which isn't directly how Transit Gateway attachments work across accounts. The Production account cannot directly create the attachment on the Transit Gateway. Also, the attachment is initiated from the VPC.

Option C: Incorrect because it attempts to share VPC subnets using RAM. Further, it incorrectly assigns the Transit Gateway attachment creation to the Connectivity account.

In summary, option D accurately describes the process of sharing the Transit Gateway from the Connectivity account to the Production account, creating the attachment from the Production VPC to the Transit Gateway, and then accepting that attachment on the Connectivity account side. This approach leverages AWS RAM for secure cross-account resource sharing and adheres to the requirement of manual attachment acceptance.

Question: 56

CertyIQ

A company is running multiple workloads on Amazon EC2 instances in public subnets. In a recent incident, an attacker exploited an application vulnerability on one of the EC2 instances to gain access to the instance. The company fixed the application and launched a replacement EC2 instance that contains the updated application. The attacker used the compromised application to spread malware over the internet. The company became aware of the compromise through a notification from AWS. The company needs the ability to identify when an application that is deployed on an EC2 instance is spreading malware.

Which solution will meet this requirement with the LEAST operational effort?

A. Use Amazon GuardDuty to analyze traffic patterns by inspecting DNS requests and VPC flow logs.

B. Use Amazon GuardDuty to deploy AWS managed decoy systems that are equipped with the most recent malware signatures.

C. Set up a Gateway Load Balancer. Run an intrusion detection system (IDS) appliance from AWS Marketplace on Amazon EC2 for traffic inspection.

D. Configure Amazon Inspector to perform deep packet inspection of outgoing traffic.

Answer: A

Explanation:

The best solution is A: Use Amazon GuardDuty to analyze traffic patterns by inspecting DNS requests and VPC flow logs.

Here's why:

GuardDuty is designed for threat detection: GuardDuty is a threat detection service that continuously monitors your AWS accounts and workloads for malicious activity and anomalous behavior. It leverages machine learning, anomaly detection, and integrated threat intelligence feeds.

Network Traffic Analysis: GuardDuty analyzes VPC Flow Logs, DNS logs, and CloudTrail event logs to identify suspicious patterns. By inspecting DNS requests and VPC flow logs, GuardDuty can detect communication with known malicious domains or unusual traffic patterns indicative of malware spreading.

Least Operational Effort: GuardDuty is a managed service, meaning AWS handles the underlying infrastructure and maintenance. This significantly reduces the operational overhead compared to setting up and managing your own IDS solution. You simply enable GuardDuty, and it starts monitoring your environment.

No agents required: GuardDuty doesn't require installing agents on EC2 instances, simplifying deployment and reducing the performance impact on your workloads.

Decoy Systems are not required: While option B uses GuardDuty with decoy systems, this approach is generally used to attract and trap attackers, providing insights into their techniques. It does not directly help identify malware spreading from EC2 instances.

Alternatives have higher operational overhead: Option C, using a Gateway Load Balancer and an IDS appliance, involves significantly more setup and management. You need to select, configure, and maintain the IDS appliance, as well as manage the Gateway Load Balancer. Option D, using Amazon Inspector for deep packet inspection, is not primarily designed for real-time network traffic analysis for malware spreading.

Cost-Effective: GuardDuty is generally a cost-effective solution for threat detection, especially when considering the reduced operational overhead compared to self-managed solutions.

Authoritative Links:

Amazon GuardDuty: <https://aws.amazon.com/guardduty/>

VPC Flow Logs: <https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs.html>

Question: 57

CertyIQ

A company deploys a new web application on Amazon EC2 instances. The application runs in private subnets in three Availability Zones behind an Application Load Balancer (ALB). Security auditors require encryption of all connections. The company uses Amazon Route 53 for DNS and uses AWS Certificate Manager (ACM) to automate SSL/TLS certificate provisioning. SSL/TLS connections are terminated on the ALB.

The company tests the application with a single EC2 instance and does not observe any problems. However, after production deployment, users report that they can log in but that they cannot use the application. Every new web request restarts the login process.

What should a network engineer do to resolve this issue?

A. Modify the ALB listener configuration. Edit the rule that forwards traffic to the target group. Change the rule to enable group-level stickiness. Set the duration to the maximum application session length.

B. Replace the ALB with a Network Load Balancer. Create a TLS listener. Create a new target group with the protocol type set to TLS Register the EC2 instances. Modify the target group configuration by enabling the stickiness attribute.

C. Modify the ALB target group configuration by enabling the stickiness attribute. Use an application-based cookie. Set the duration to the maximum application session length.

D. Remove the ALB. Create an Amazon Route 53 rule with a failover routing policy for the application name. Configure ACM to issue certificates for each EC2 instance.

Answer: C

Explanation:

The correct answer is **C. Modify the ALB target group configuration by enabling the stickiness attribute. Use an application-based cookie. Set the duration to the maximum application session length.**

Here's why:

The problem described suggests a session management issue. Users can log in, but every subsequent request restarts the login process. This indicates that the user's session is not being maintained across multiple requests. In a load-balanced environment like this, a user's requests can be routed to different EC2 instances behind the ALB. If each instance treats the user as a new user with each request, the session is lost.

Application Load Balancers (ALBs) offer a feature called "stickiness" or "session affinity" to address this problem. Enabling stickiness ensures that all requests from a specific user are routed to the same EC2 instance for the duration of the session.

Option C directly addresses this by suggesting enabling stickiness at the target group level. By configuring an application-based cookie, the ALB inserts a cookie in the user's browser after the first request. This cookie contains information about the instance that initially served the request. Subsequent requests from the same user will include this cookie, and the ALB will use it to route those requests to the same instance, maintaining session continuity. Setting the duration to the maximum application session length ensures the stickiness persists for the entire duration the user interacts with the application.

Why the other options are incorrect:

A. Modify the ALB listener configuration... enable group-level stickiness: While this option also aims to enable stickiness, "group-level stickiness" is not a standard term associated with ALBs. The ALB stickiness is configured at the target group level, as described in option C. Using target group stickiness with an application-based cookie is the standard and more direct approach.

B. Replace the ALB with a Network Load Balancer: NLBs operate at layer 4 (TCP), while ALBs operate at layer 7 (application layer). NLBs are designed for high throughput and low latency, but they don't inherently handle application-level session stickiness based on cookies. While NLBs support source IP based stickiness which is a very coarse mechanism, converting the ALB and associated configurations to NLB won't resolve the actual application's need for managing user sessions using cookies. Introducing TLS at the NLB level doesn't address the root cause either; the fundamental problem lies in session management.

D. Remove the ALB. Create an Amazon Route 53 rule...: This is not the correct solution. Removing the load balancer removes the benefits of traffic distribution and high availability. Configuring Route 53 failover routing policies is for disaster recovery scenarios, not session management. ACM certificates are already being used with the ALB for SSL/TLS termination, so issuing certificates to individual instances is not necessary and makes management more complex.

Supporting Documentation:

Application Load Balancers - Target Groups - Attributes (Stickiness):

<https://docs.aws.amazon.com/elasticloadbalancing/latest/application/tutorial-get-started-application-load->

Question: 58

A company recently migrated its Amazon EC2 instances to VPC private subnets to satisfy a security compliance requirement. The EC2 instances now use a NAT gateway for internet access. After the migration, some long-running database queries from private EC2 instances to a publicly accessible third-party database no longer receive responses. The database query logs reveal that the queries successfully completed after 7 minutes but that the client EC2 instances never received the response.

Which configuration change should a network engineer implement to resolve this issue?

- A. Configure the NAT gateway timeout to allow connections for up to 600 seconds.
- B. Enable enhanced networking on the client EC2 instances.
- C. Enable TCP keepalive on the client EC2 instances with a value of less than 300 seconds.
- D. Close idle TCP connections through the NAT gateway.

Answer: C**Explanation:**

Here's a detailed justification for why option C is the correct answer:

The problem describes a scenario where long-running database queries are completing successfully on the server side, but the client EC2 instances in private subnets, using a NAT gateway for internet access, are not receiving the responses. This suggests an idle timeout issue with the connection tracking performed by the NAT gateway. NAT gateways, by default, have TCP connection tracking. If a TCP connection remains idle for a certain period (the idle timeout), the NAT gateway will drop the connection and remove its state from the connection tracking table.

The default idle timeout for a TCP connection through an AWS NAT gateway is 350 seconds (approximately 5 minutes 50 seconds). Since the database queries are completing successfully in 7 minutes (420 seconds), the connections are being dropped by the NAT gateway before the responses can be delivered back to the client. The client instances don't know the connection has been dropped and keep waiting.

Option C, enabling TCP keepalive on the client EC2 instances with a value of less than 300 seconds, is the correct solution because it sends periodic TCP keepalive packets. These packets are small probes sent by the client to the server (or vice versa) to keep the connection active and prevent the NAT gateway from considering the connection idle. By setting the keepalive interval to less than 300 seconds, the NAT gateway will receive these packets and reset the idle timeout counter, ensuring that the connection remains active for the duration of the 7-minute database query.

Option A is incorrect because you cannot configure the NAT gateway timeout. It's a managed service and the timeout values are fixed. While increasing the timeout would solve the problem, it's not a configurable option.

Option B, enabling enhanced networking, improves network performance (higher throughput, lower latency, and lower jitter). While beneficial overall, it doesn't address the specific problem of the NAT gateway dropping idle connections.

Option D, closing idle TCP connections through the NAT gateway, is the opposite of what is needed. It would exacerbate the problem by prematurely terminating connections.

In summary, TCP keepalive mechanisms provide a way to maintain the connection alive by sending periodic probes. Since NAT gateway timeouts cannot be modified, client-side TCP keepalive configurations are crucial for long-running connections through the gateway.

Authoritative Links:

AWS NAT Gateway: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html> (Specifically, review information about connection timeouts.)

TCP Keepalive HOWTO: <http://tldp.org/HOWTO/TCP-Keepalive-HOWTO/index.html> (General explanation of TCP Keepalive)

Question: 59

CertyIQ

A company uses AWS Direct Connect to connect its corporate network to multiple VPCs in the same AWS account and the same AWS Region. Each VPC uses its own private VIF and its own virtual LAN on the Direct Connect connection. The company has grown and will soon surpass the limit of VPCs and private VIFs for each connection. What is the MOST scalable way to add VPCs with on-premises connectivity?

- A. Provision a new Direct Connect connection to handle the additional VPCs. Use the new connection to connect additional VPCs.
- B. Create virtual private gateways for each VPC that is over the service quota. Use AWS Site-to-Site VPN to connect the virtual private gateways to the corporate network.
- C. Create a Direct Connect gateway, and add virtual private gateway associations to the VPCs. Configure a private VIF to connect to the corporate network.
- D. Create a transit gateway, and attach the VPCs. Create a Direct Connect gateway, and associate it with the transit gateway. Create a transit VIF to the Direct Connect gateway.

Answer: D

Explanation:

The question asks for the most scalable way to connect an on-premises network to multiple VPCs in the same region, given that the current Direct Connect setup is hitting limits. Let's analyze why the chosen answer (D) is the best option:

Option D: This solution leverages Transit Gateway (TGW) and Direct Connect Gateway (DXGW). TGW acts as a central hub, allowing VPCs to connect to it. The DXGW then connects the TGW to the Direct Connect connection. This avoids the limitations of one private VIF per VPC. As more VPCs are needed, they can be attached to the TGW without exhausting Direct Connect resources. A single transit VIF is used, regardless of the number of connected VPCs. This approach allows for transitive routing between VPCs and the on-premises network.

Option A: Provisioning a new Direct Connect connection only delays the problem. The company will eventually hit the limits again as they add more VPCs. It's not a scalable solution.

Option B: Using Site-to-Site VPNs would create a complex mesh of VPN connections. Each VPC would need its own VPN connection, which would be hard to manage and potentially less performant than Direct Connect.

Option C: Creating a Direct Connect gateway is a good start, but it only allows you to associate VPCs with the Direct Connect connection through Virtual Private Gateways (VGWs). Although a DXGW can be associated with multiple VGWs, this option does not provide scalable transitive connectivity between VPCs. Each VPC still needs its own VGW.

Therefore, option D is the most scalable solution because it uses Transit Gateway to act as a hub for VPCs, and Direct Connect Gateway to efficiently connect the TGW to the Direct Connect connection, minimizing the number of virtual interfaces required and centralizing the routing.

Supporting links:

AWS Transit Gateway: <https://aws.amazon.com/transit-gateway/>

AWS Direct Connect Gateway: <https://aws.amazon.com/directconnect/dx-gateway/>

Question: 60

A network engineer is designing a hybrid architecture that uses a 1 Gbps AWS Direct Connect connection between the company's data center and two AWS Regions: us-east-1 and eu-west-1. The VPCs in us-east-1 are connected by a transit gateway and need to access several on-premises databases. According to company policy, only one VPC in eu-west-1 can be connected to one on-premises server. The on-premises network segments the traffic between the databases and the server.

How should the network engineer set up the Direct Connect connection to meet these requirements?

- A. Create one hosted connection. Use a transit VIF to connect to the transit gateway in us-east-1. Use a private VIF to connect to the VPC in eu-west-1. Use one Direct Connect gateway for both VIFs to route from the Direct Connect locations to the corresponding AWS Region along the path that has the lowest latency.
- B. Create one hosted connection. Use a transit VIF to connect to the transit gateway in us-east-1. Use a private VIF to connect to the VPC in eu-west-1. Use two Direct Connect gateways, one for each VIF, to route from the Direct Connect locations to the corresponding AWS Region along the path that has the lowest latency.
- C. Create one dedicated connection. Use a transit VIF to connect to the transit gateway in us-east-1. Use a private VIF to connect to the VPC in eu-west-1. Use one Direct Connect gateway for both VIFs to route from the Direct Connect locations to the corresponding AWS Region along the path that has the lowest latency.
- D. Create one dedicated connection. Use a transit VIF to connect to the transit gateway in us-east-1. Use a private VIF to connect to the VPC in eu-west-1. Use two Direct Connect gateways, one for each VIF, to route from the Direct Connect locations to the corresponding AWS Region along the path that has the lowest latency.

Answer: D

Explanation:

The correct answer is **D**, which involves a dedicated connection, a transit VIF for us-east-1, a private VIF for eu-west-1, and two Direct Connect gateways. Let's break down why:

1. **Dedicated Connection:** The question doesn't explicitly state a need or lack of need for a dedicated vs. hosted connection. However, it implies the organization already has an on-premises datacenter that needs secure, consistent connectivity. A dedicated connection provides more control and consistent performance compared to a hosted connection which can be impacted by the service provider.
2. **Transit VIF for us-east-1:** The requirement to connect to a transit gateway (TGW) in us-east-1 necessitates using a transit VIF. Transit VIFs are specifically designed for connecting to transit gateways, enabling connectivity between multiple VPCs connected to the TGW and the on-premises network.
3. **Private VIF for eu-west-1:** Since only one VPC in eu-west-1 needs to connect to one on-premises server, a private VIF is the appropriate choice. Private VIFs establish direct connectivity between the Direct Connect connection and a specific VPC.
4. **Two Direct Connect Gateways:** This is the crucial distinction. The requirement to connect to two separate AWS Regions necessitates two Direct Connect Gateways (DXGW). A single DXGW can only be associated with VIFs in a single AWS Region. The DXGWs enable routing between the Direct Connect connection and the corresponding AWS Regions (us-east-1 and eu-west-1), allowing the traffic to reach the transit gateway and the VPC in eu-west-1.
5. **Lowest Latency Routing:** Each DXGW independently handles traffic routing. While the question mentions lowest latency, the DXGW automatically uses BGP routing to determine the best path to the associated AWS Region, so latency is automatically a consideration. There's no configuration needed to explicitly set this.

In summary, using a dedicated connection, a transit VIF to the us-east-1 TGW, a private VIF to eu-west-1, and separate DXGWs for each region is essential for meeting the requirements for connecting to two distinct AWS Regions and enabling connectivity between the on-premises network and the VPCs in those regions while

adhering to the one-to-one VPC to on-premises server restriction in eu-west-1.

Authoritative Links:

AWS Direct Connect Gateways: <https://docs.aws.amazon.com/directconnect/latest/UserGuide/direct-connect-gateways-intro.html>

AWS Direct Connect Virtual Interfaces:

<https://docs.aws.amazon.com/directconnect/latest/UserGuide/WorkingWithVirtualInterfaces.html>

AWS Transit Gateway: <https://docs.aws.amazon.com/vpc/latest/tgw/what-is-transit-gateway.html>

Question: 61

CertyIQ

A company has deployed an application in a VPC that uses a NAT gateway for outbound traffic to the internet. A network engineer notices a large quantity of suspicious network traffic that is traveling from the VPC over the internet to IP addresses that are included on a deny list. The network engineer must implement a solution to determine which AWS resources are generating the suspicious traffic. The solution must minimize cost and administrative overhead.

Which solution will meet these requirements?

A. Launch an Amazon EC2 instance in the VPC. Use Traffic Mirroring by specifying the NAT gateway as the source and the EC2 instance as the destination. Analyze the captured traffic by using open-source tools to identify the AWS resources that are generating the suspicious traffic.

B. Use VPC flow logs. Launch a security information and event management (SIEM) solution in the VPC. Configure the SIEM solution to ingest the VPC flow logs. Run queries on the SIEM solution to identify the AWS resources that are generating the suspicious traffic.

C. Use VPC flow logs. Publish the flow logs to a log group in Amazon CloudWatch Logs. Use CloudWatch Logs Insights to query the flow logs to identify the AWS resources that are generating the suspicious traffic.

D. Configure the VPC to stream the network traffic directly to an Amazon Kinesis data stream. Send the data from the Kinesis data stream to an Amazon Kinesis Data Firehose delivery stream to store the data in Amazon S3. Use Amazon Athena to query the data to identify the AWS resources that are generating the suspicious traffic.

Answer: C

Explanation:

The best solution for identifying the source of suspicious traffic from a VPC, minimizing cost and administrative overhead, is to utilize VPC Flow Logs published to CloudWatch Logs and queried via CloudWatch Logs Insights.

Here's why:

VPC Flow Logs: Capture information about the IP traffic going to and from network interfaces in a VPC. This includes source and destination IP addresses, ports, and the action taken (ACCEPT or REJECT).

(<https://docs.aws.amazon.com/vpc/latest/userguide/vpc-flow-logs.html>)

Cost Efficiency: VPC Flow Logs incur costs based on the data published to CloudWatch Logs. CloudWatch Logs Insights charges based on the amount of data scanned during queries, making it a pay-as-you-go model.

Administrative Overhead: Setting up VPC Flow Logs and configuring CloudWatch Logs Insights queries is relatively straightforward, requiring minimal administrative effort.

CloudWatch Logs Integration: Publishing to CloudWatch Logs provides a centralized location for log data and seamless integration with CloudWatch Logs Insights.

CloudWatch Logs Insights: Provides a purpose-built query language to analyze log data. This allows the network engineer to quickly filter and identify AWS resources generating traffic to the deny-listed IP addresses, using source IP addresses and network interface IDs.

(<https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/AnalyzingLogData.html>)

The other options are less suitable:

A (Traffic Mirroring): Traffic Mirroring is powerful but more complex to set up and requires deploying and managing an EC2 instance. This adds administrative overhead and EC2 instance costs.

B (SIEM Solution): A SIEM solution is comprehensive but typically more expensive and requires significant setup and configuration, increasing both cost and administrative overhead.

D (Kinesis/Athena): This solution involves multiple services (Kinesis Data Streams, Kinesis Data Firehose, S3, Athena) and data transformation. This increases complexity, management overhead, and overall cost compared to using CloudWatch Logs and CloudWatch Logs Insights. Specifically, Kinesis stream is for real time ingestion, which is overkill for this usecase.

Question: 62

CertyIQ

A company has its production VPC (VPC-A) in the eu-west-1 Region in Account 1. VPC-A is attached to a transit gateway (TGW-A) that is connected to an on-premises data center in Dublin, Ireland, by an AWS Direct Connect transit VIF that is configured for an AWS Direct Connect gateway. The company also has a staging VPC (VPC-B) that is attached to another transit gateway (TGW-B) in the eu-west-2 Region in Account 2.

A network engineer must implement connectivity between VPC-B and the on-premises data center in Dublin. Which solutions will meet these requirements? (Choose two.)

- A. Configure inter-Region VPC peering between VPC-A and VPC-B. Add the required VPC peering routes. Add the VPC-B CIDR block in the allowed prefixes on the Direct Connect gateway association.
- B. Associate TGW-B with the Direct Connect gateway. Advertise the VPC-B CIDR block under the allowed prefixes.
- C. Configure another transit VIF on the Direct Connect connection and associate TGW-B. Advertise the VPC-B CIDR block under the allowed prefixes.
- D. Configure inter-Region transit gateway peering between TGW-A and TGW-B. Add the peering routes in the transit gateway route tables. Add both the VPC-A and the VPC-B CIDR block under the allowed prefix list in the Direct Connect gateway association.
- E. Configure an AWS Site-to-Site VPN connection over the transit VIF to TGW-B as a VPN attachment.

Answer: BD

Explanation:

The correct answer is BD. Here's why:

B: Associate TGW-B with the Direct Connect gateway. Advertise the VPC-B CIDR block under the allowed prefixes. This solution directly extends the Direct Connect connection's reach to TGW-B. By associating TGW-B with the Direct Connect gateway and advertising the VPC-B CIDR, traffic from the on-premises data center can be routed to VPC-B through the Direct Connect connection and the transit gateway. This is efficient as it leverages the existing Direct Connect infrastructure.

<https://docs.aws.amazon.com/directconnect/latest/UserGuide/associate-tgw.html>

D: Configure inter-Region transit gateway peering between TGW-A and TGW-B. Add the peering routes in the transit gateway route tables. Add both the VPC-A and the VPC-B CIDR block under the allowed prefix list in the Direct Connect gateway association. This option establishes a connection between TGW-A and TGW-B, enabling routing between the on-premises data center (connected to TGW-A via Direct Connect) and VPC-B. Crucially, the Direct Connect gateway must allow the VPC-B CIDR so that traffic destined for VPC-B can be routed through the Direct Connect connection. This ensures a complete path from on-premises to VPC-B. Adding VPC-A to the allowed prefix list is necessary as TGW-A and TGW-B must communicate.

<https://docs.aws.amazon.com/vpc/latest/tgw/tgw-peering.html>

Let's analyze the incorrect answers:

A: Configure inter-Region VPC peering between VPC-A and VPC-B. Add the required VPC peering routes.

Add the VPC-B CIDR block in the allowed prefixes on the Direct Connect gateway association. VPC peering doesn't automatically provide transitive routing. Traffic from on-premises to VPC-B wouldn't traverse VPC-A using VPC peering alone. It requires a TGW or VPN to extend the connection through VPC-A to VPC-B.

C: Configure another transit VIF on the Direct Connect connection and associate TGW-B. Advertise the VPC-B CIDR block under the allowed prefixes. While technically possible, adding another transit VIF is generally unnecessary when you can associate TGW-B directly with the existing Direct Connect gateway, as in option B. Adding another VIF increases complexity without significant benefit.

E: Configure an AWS Site-to-Site VPN connection over the transit VIF to TGW-B as a VPN attachment. This solution introduces unnecessary complexity. A VPN over Direct Connect would add another layer of overhead and isn't the most direct way to achieve connectivity. The other options leverage the existing Direct Connect and transit gateway infrastructure more efficiently.

Question: 63

CertyIQ

A company's network engineer is designing a hybrid DNS solution for an AWS Cloud workload. Individual teams want to manage their own DNS hostnames for their applications in their development environment. The solution must integrate the application-specific hostnames with the centrally managed DNS hostnames from the on-premises network and must provide bidirectional name resolution. The solution also must minimize management overhead.

Which combination of steps should the network engineer take to meet these requirements? (Choose three.)

- A. Use an Amazon Route 53 Resolver inbound endpoint.
- B. Modify the DHCP options set by setting a custom DNS server value.
- C. Use an Amazon Route 53 Resolver outbound endpoint.
- D. Create DNS proxy servers.
- E. Create Amazon Route 53 private hosted zones.
- F. Set up a zone transfer between Amazon Route 53 and the on-premises DNS.

Answer: ACE

Explanation:

The requirement is a hybrid DNS solution that allows teams to manage their application-specific hostnames in the development environment while integrating with the centrally managed on-premises DNS. The solution needs bidirectional name resolution and minimized overhead.

A. Use an Amazon Route 53 Resolver inbound endpoint: This is correct. An inbound endpoint enables on-premises DNS servers to forward queries for your AWS private hosted zones to Route 53. This provides the "bidirectional name resolution" by allowing on-premises systems to resolve names hosted in AWS.

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver.html>

C. Use an Amazon Route 53 Resolver outbound endpoint: This is also correct. An outbound endpoint allows your VPC resources to query your on-premises DNS servers. This provides the other direction of "bidirectional name resolution" by allowing AWS systems to resolve names hosted on-premises. By forwarding unresolved queries from your VPC to the on-premises DNS infrastructure, it allows seamless integration between the AWS environment and the existing on-premises DNS zones.

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver.html>

E. Create Amazon Route 53 private hosted zones: This option is crucial. Private hosted zones are used to host DNS records within a VPC. By creating these zones, the individual teams can manage their application-specific hostnames for their applications in the development environment. The integration with inbound and outbound resolvers then ensures that these hostnames are resolvable both from within AWS and from the on-premises

network. This also satisfies the requirement to let individual teams manage their own DNS records.

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/private-hosted-zones.html>

Why other options are incorrect:

B. Modify the DHCP options set by setting a custom DNS server value: While DHCP options configure DNS servers for EC2 instances, this would override the default AmazonProvidedDNS and not allow resolution of public AWS services. It also doesn't directly facilitate bidirectional name resolution between on-premises and AWS.

D. Create DNS proxy servers: While functional, setting up and maintaining DNS proxy servers adds significant overhead compared to using Route 53 Resolver endpoints, which are managed services. Using Route 53 resolvers minimizes management overhead, which is a specific requirement.

F. Set up a zone transfer between Amazon Route 53 and the on-premises DNS: Zone transfers are complex and can be less efficient than using resolver endpoints for bidirectional name resolution. Resolver endpoints provide a simpler, more scalable, and managed solution. Zone transfers also generally involve transferring the entire zone, making granular team-based DNS delegation more difficult.

Question: 64

CertyIQ

A company hosts a web application on Amazon EC2 instances behind an Application Load Balancer (ALB). The ALB is the origin in an Amazon CloudFront distribution. The company wants to implement a custom authentication system that will provide a token for its authenticated customers.

The web application must ensure that the GET/POST requests come from authenticated customers before it delivers the content. A network engineer must design a solution that gives the web application the ability to identify authorized customers.

What is the MOST operationally efficient solution that meets these requirements?

- A. Use the ALB to inspect the authorized token inside the GET/POST request payload. Use an AWS Lambda function to insert a customized header to inform the web application of an authenticated customer request.
- B. Integrate AWS WAF with the ALB to inspect the authorized token inside the GET/POST request payload. Configure the ALB listener to insert a customized header to inform the web application of an authenticated customer request.
- C. Use an AWS Lambda@Edge function to inspect the authorized token inside the GET/POST request payload. Use the Lambda@Edge function also to insert a customized header to inform the web application of an authenticated customer request.
- D. Set up an EC2 instance that has a third-party packet inspection tool to inspect the authorized token inside the GET/POST request payload. Configure the tool to insert a customized header to inform the web application of an authenticated customer request.

Answer: C

Explanation:

The most operationally efficient solution for implementing a custom authentication system with token verification for a web application behind CloudFront and ALB is option C, utilizing Lambda@Edge.

Lambda@Edge allows you to execute code at AWS edge locations, which are closer to the users. This provides lower latency compared to routing requests to the origin (ALB) for authentication. In this scenario, the Lambda@Edge function can inspect the authorized token within the GET/POST request payload as it reaches the CloudFront edge location.

Upon successful token validation, the Lambda@Edge function inserts a custom header into the request before forwarding it to the origin. This custom header informs the web application that the request originates from an authenticated user.

This approach is operationally efficient because:

1. **Lower Latency:** Edge processing reduces the round trip time needed for authentication.
2. **Scalability:** Lambda@Edge automatically scales with request volume.
3. **Security:** The token is validated before the request reaches the origin, protecting the web application from unauthorized requests.
4. **Reduced Origin Load:** Only authenticated requests reach the web application, reducing the load on the EC2 instances.
5. **Cost-Effective:** Lambda@Edge is billed only for the compute time consumed.

Option A is less efficient. Using a Lambda function with ALB requires sending traffic to the ALB first, then invoking the Lambda function, adding latency. ALB is not designed for payload inspection.

Option B, integrating AWS WAF with ALB for token inspection, could work for basic token validation rules and could have scalability challenges. While WAF can filter based on request parameters, it's typically used for security rules, not full-blown authentication and header manipulation.

Option D, using an EC2 instance with a third-party packet inspection tool, is complex and difficult to manage. It requires maintaining the EC2 instance, ensuring high availability, and configuring the packet inspection tool, which is not operationally efficient. It also adds latency and introduces a potential single point of failure.

Therefore, Lambda@Edge is the most operationally efficient and suitable solution for this scenario. It offers low latency, scalability, and security while minimizing operational overhead.

Relevant Documentation:

AWS Lambda@Edge: <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/lambda-at-the-edge.html>

Amazon CloudFront: <https://aws.amazon.com/cloudfront/>

Question: 65

CertyIQ

A company has created three VPCs: a production VPC, a nonproduction VPC, and a shared services VPC. The production VPC and the nonproduction VPC must each have communication with the shared services VPC. There must be no communication between the production VPC and the nonproduction VPC. A transit gateway is deployed to facilitate communication between VPCs.

Which route table configurations on the transit gateway will meet these requirements?

- A. Configure a route table with the production and nonproduction VPC attachments associated with propagated routes for only the shared services VPC. Create an additional route table with only the shared services VPC attachment associated with propagated routes from the production and nonproduction VPCs.
- B. Configure a route table with the production and nonproduction VPC attachments associated with propagated routes for each VPC. Create an additional route table with only the shared services VPC attachment associated with propagated routes from each VPC.
- C. Configure a route table with all the VPC attachments associated with propagated routes for only the shared services VPC. Create an additional route table with only the shared services VPC attachment associated with propagated routes from the production and nonproduction VPCs.
- D. Configure a route table with the production and nonproduction VPC attachments associated with propagated routes disabled. Create an additional route table with only the shared services VPC attachment associated with propagated routes from the production and nonproduction VPCs.

Answer: A

Explanation:

The correct answer is A because it effectively isolates traffic between the production and non-production VPCs while allowing both to communicate with the shared services VPC. Let's break down why.

Transit Gateway Route Tables: Transit Gateways use route tables to determine where to forward traffic.

Attachments (in this case, VPCs) are associated with route tables, and routes within these tables dictate the next hop for traffic based on its destination. Route propagation enables a route table to automatically learn routes from attached resources (VPCs, VPNs, Direct Connect).

Option A implements two route tables:

Route Table 1 (Production/Non-Production): This route table is associated with the production and non-production VPC attachments. It's configured to propagate routes only from the shared services VPC. This means that the production and non-production VPCs will learn how to reach the shared services VPC (based on the shared services VPC's CIDR block), but they won't learn about each other's CIDR blocks. This prevents direct communication between production and non-production environments.

Route Table 2 (Shared Services): This route table is associated with the shared services VPC attachment. It's configured to propagate routes from both the production and non-production VPCs. This allows the shared services VPC to learn how to reach resources in both the production and non-production VPCs.

The effect is a hub-and-spoke model. Production and Non-Production VPCs are spokes that only know about the hub (Shared Services). The Shared Services VPC knows about both spokes, but the spokes are isolated from each other.

Options B, C and D fail to achieve the necessary isolation:

Option B: Propagating routes from all VPCs into the primary route table would allow production and non-production VPCs to learn about each other, violating the requirement.

Option C: The same issue as option B occurs. By propagating shared service VPC routes to the first route table with all VPC attachments, it effectively creates a single network spanning all VPCs.

Option D: Disabling route propagation on the first route table would isolate Production and Non-Production; however, since there's no mechanism defined that tells the Production and Non-Production VPCs how to reach the Shared Services VPC, communication with Shared Services would fail for Production and Non-Production.

Therefore, only option A provides the required isolation between the production and non-production VPCs while still enabling communication with the shared services VPC through a well-defined routing mechanism.

Authoritative Links:

AWS Transit Gateway Documentation: <https://docs.aws.amazon.com/vpc/latest/tgw/what-is-transit-gateway.html>

Transit Gateway Route Tables: <https://docs.aws.amazon.com/vpc/latest/tgw/tgw-route-tables.html>

Transit Gateway Route Propagation: <https://docs.aws.amazon.com/vpc/latest/tgw/tgw-route-tables.html#tgw-route-propagation>

Question: 66

CertyIQ

A company is using an AWS Site-to-Site VPN connection from the company's on-premises data center to a virtual private gateway in the AWS Cloud. Because of congestion, the company is experiencing availability and performance issues as traffic travels across the internet before the traffic reaches AWS. A network engineer must reduce these issues for the connection as quickly as possible with minimum administration effort. Which solution will meet these requirements?

A. Edit the existing Site-to-Site VPN connection by enabling acceleration. Stop and start the VPN service on the customer gateway for the new setting to take effect.

B. Configure a transit gateway in the same AWS Region as the existing virtual private gateway. Create a new accelerated Site-to-Site VPN connection. Connect the new connection to the transit gateway by using a VPN attachment. Update the customer gateway device to use the new Site to Site VPN connection. Delete the existing Site-to-Site VPN connection.

C. Create a new accelerated Site-to-Site VPN connection. Connect the new Site-to-Site VPN connection to the existing virtual private gateway. Update the customer gateway device to use the new Site-to-Site VPN connection. Delete the existing Site-to-Site VPN connection.

D. Create a new AWS Direct Connect connection with a private VIF between the on-premises data center and the AWS Cloud. Update the customer gateway device to use the new Direct Connect connection. Delete the existing Site-to-Site VPN connection.

Answer: B

Explanation:

The most efficient solution to improve Site-to-Site VPN performance and availability with minimal administrative overhead involves utilizing AWS Transit Gateway with VPN acceleration.

Option B is correct because it leverages AWS Transit Gateway and accelerated Site-to-Site VPN. Transit Gateway acts as a central hub, simplifying network management and routing. By creating a new accelerated Site-to-Site VPN connection and attaching it to the Transit Gateway via a VPN attachment, you benefit from AWS Global Accelerator, which optimizes the network path between your on-premises data center and AWS. Updating the customer gateway device to use the new connection and deleting the old one ensures traffic utilizes the optimized path. This solution addresses the congestion issues with minimal administrative effort.

Option A is incorrect because you cannot simply enable acceleration on an existing Site-to-Site VPN connection. Acceleration requires a new VPN connection to be established with the acceleration feature enabled from the start.

Option C is incorrect because it suggests connecting the new accelerated VPN directly to the existing virtual private gateway (VGW). While this uses VPN acceleration, it doesn't address the scalable routing and simplified management benefits that Transit Gateway provides, and using a VGW for multiple VPNs can still create management complexities. Transit Gateway makes the solution more scalable and easier to manage.

Option D, while a solid long-term solution, is not the quickest and requires significant initial setup and cost. Establishing a Direct Connect connection takes time, involving physical infrastructure deployment, and may not be feasible as a quick fix for immediate congestion

issues. <https://aws.amazon.com/vpn/features> <https://aws.amazon.com/transit-gateway> <https://docs.aws.amazon.com/vpn/latest/s2svpn/vpn-acceleration.html>

Question: 67

CertyIQ

An Australian ecommerce company hosts all of its services in the AWS Cloud and wants to expand its customer base to the United States (US). The company is targeting the western US for the expansion. The company's existing AWS architecture consists of four AWS accounts with multiple VPCs deployed in the ap-southeast-2 Region. All VPCs are attached to a transit gateway in ap-southeast-2. There are dedicated VPCs for each application service. The company also has VPCs for centralized security features such as proxies, firewalls, and logging.

The company plans to duplicate the infrastructure from ap-southeast-2 to the us-west-1 Region. A network engineer must establish connectivity between the various applications in the two Regions. The solution must maximize bandwidth, minimize latency and minimize operational overhead.

Which solution will meet these requirements?

A. Create VPN attachments between the two transit gateways. Configure the VPN attachments to use BGP routing between the two transit gateways.

B. Peer the transit gateways in each Region. Configure routing between the two transit gateways for each Region's IP addresses.

C. Create a VPN server in a VPC in each Region. Update the routing to point to the VPN servers for the IP addresses in alternate Regions.

D. Attach the VPCs in us-west-1 to the transit gateway in ap-southeast-2.

Answer: B

Explanation:

The best solution for connecting the Australian ecommerce company's infrastructure in ap-southeast-2 to us-west-1 while maximizing bandwidth, minimizing latency, and minimizing operational overhead is to peer the transit gateways in each Region (Option B).

Transit Gateway peering allows for direct, high-bandwidth connectivity between transit gateways in different AWS Regions. This peering connection creates a fully meshed network between the peered transit gateways. Routing is then configured between the transit gateways so that each transit gateway learns the IP address ranges associated with the VPCs connected to the other transit gateway. This avoids the overhead and latency associated with VPN connections.

VPN attachments (Option A) introduce additional latency and overhead due to encryption and encapsulation. Furthermore, the bandwidth is limited by the VPN connection's capacity. Attaching VPCs in us-west-1 to the transit gateway in ap-southeast-2 (Option D) would not be a viable solution because resources in different regions need to be physically in those regions. This also does not minimize latency. A VPN server in each region (Option C) suffers from the same latency and bandwidth limitations as VPN attachments.

Transit Gateway peering is the most efficient and scalable method to connect resources across Regions using the AWS backbone network, optimizing performance and reducing operational complexity.

Here are some authoritative links for further research:

AWS Transit Gateway Peering Attachments: <https://docs.aws.amazon.com/vpc/latest/tgw/tgw-peering.html>

AWS Transit Gateway: <https://aws.amazon.com/transit-gateway/>

Question: 68

CertyIQ

An IoT company sells hardware sensor modules that periodically send out temperature, humidity, pressure, and location data through the MQTT messaging protocol. The hardware sensor modules send this data to the company's on-premises MQTT brokers that run on Linux servers behind a load balancer. The hardware sensor modules have been hardcoded with public IP addresses to reach the brokers.

The company is growing and is acquiring customers across the world. The existing solution can no longer scale and is introducing additional latency because of the company's global presence. As a result, the company decides to migrate its entire infrastructure from on premises to the AWS Cloud. The company needs to migrate without reconfiguring the hardware sensor modules that are already deployed across the world. The solution also must minimize latency.

The company migrates the MQTT brokers to run on Amazon EC2 instances.

What should the company do next to meet these requirements?

A. Place the EC2 instances behind a Network Load Balancer (NLB). Configure TCP listeners. Use Bring Your Own IP (BYOIP) from the on-premises network with the NLB.

B. Place the EC2 instances behind a Network Load Balancer (NLB). Configure TCP listeners. Create an AWS Global Accelerator accelerator in front of the NLB. Use Bring Your Own IP (BYOIP) from the on-premises network with Global Accelerator.

C. Place the EC2 instances behind an Application Load Balancer (ALB). Configure TCP listeners. Create an AWS Global Accelerator accelerator in front of the ALB. Use Bring Your Own IP (BYOIP) from the on-premises network with Global Accelerator.

D. Place the EC2 instances behind an Amazon CloudFront distribution. Use Bring Your Own IP (BYOIP) from the on-premises network with CloudFront.

Answer: B

Explanation:

Here's a detailed justification for why option B is the best solution, along with explanations of why the other options are not ideal:

The primary goal is to migrate the MQTT brokers to AWS without reconfiguring the already deployed hardware sensor modules that are hardcoded with public IP addresses. The solution must also minimize latency globally.

Option B: Place the EC2 instances behind a Network Load Balancer (NLB). Configure TCP listeners. Create an AWS Global Accelerator accelerator in front of the NLB. Use Bring Your Own IP (BYOIP) from the on-premises network with Global Accelerator.

This is the most suitable option because it addresses all requirements.

NLB with TCP Listeners: NLBs are designed for handling TCP traffic, which is the protocol used by MQTT. Configuring TCP listeners allows the NLB to forward traffic to the MQTT broker EC2 instances.

AWS Global Accelerator: This service is designed to minimize latency for globally distributed clients. It uses the AWS global network to route traffic to the nearest edge location, reducing the distance and improving performance. Global Accelerator then routes traffic to the NLB.

Bring Your Own IP (BYOIP) with Global Accelerator: BYOIP enables the company to bring their existing public IP address range to AWS and associate them with the Global Accelerator. Since the sensor modules are hardcoded with specific public IPs, using BYOIP ensures that no changes are needed on the sensor module side during the migration. Traffic sent to these IP addresses will be routed via Global Accelerator to the nearest edge location and then routed to the NLB.

Option A: Place the EC2 instances behind a Network Load Balancer (NLB). Configure TCP listeners. Use Bring Your Own IP (BYOIP) from the on-premises network with the NLB.

This option fulfills the hardcoded IP requirement by using BYOIP with the NLB. However, it lacks the global latency optimization provided by AWS Global Accelerator. The NLB exists in a single region, so clients from distant locations will experience higher latency compared to Option B.

Option C: Place the EC2 instances behind an Application Load Balancer (ALB). Configure TCP listeners. Create an AWS Global Accelerator accelerator in front of the ALB. Use Bring Your Own IP (BYOIP) from the on-premises network with Global Accelerator.

This option is incorrect because Application Load Balancers (ALBs) primarily operate at Layer 7 (HTTP/HTTPS). MQTT typically runs over TCP (Layer 4). While ALBs can forward TCP traffic, they're not optimized for it and aren't the best choice for this MQTT use case.

Option D: Place the EC2 instances behind an Amazon CloudFront distribution. Use Bring Your Own IP (BYOIP) from the on-premises network with CloudFront.

CloudFront is a Content Delivery Network (CDN) primarily used for caching static and dynamic content. It's not designed for handling persistent connections like those used by MQTT. While CloudFront can accelerate content delivery, it's not the appropriate solution for a real-time messaging protocol. Additionally, CloudFront is designed for HTTP/HTTPS traffic, not raw TCP.

In conclusion, Option B provides the best solution because it combines the use of NLB for TCP traffic management, Global Accelerator for latency optimization, and BYOIP for maintaining the existing hardcoded IP addresses, allowing for a seamless migration without requiring any changes on the sensor modules.

Authoritative Links:

AWS Global Accelerator: <https://aws.amazon.com/global-accelerator/>

Network Load Balancer (NLB): <https://aws.amazon.com/elasticloadbalancing/network-load-balancer/>

Bring Your Own IP (BYOIP): <https://docs.aws.amazon.com/global-accelerator/latest/dg/using-byoip.html>

Question: 69

A company has deployed a web application on AWS. The web application uses an Application Load Balancer (ALB) across multiple Availability Zones. The targets of the ALB are AWS Lambda functions. The web application also uses Amazon CloudWatch metrics for monitoring.

Users report that parts of the web application are not loading properly. A network engineer needs to troubleshoot the problem. The network engineer enables access logging for the ALB.

What should the network engineer do next to determine which errors the ALB is receiving?

- A. Send the logs to Amazon CloudWatch Logs. Review the ALB logs in CloudWatch Insights to determine which error messages the ALB is receiving.
- B. Configure the Amazon S3 bucket destination. Use Amazon Athena to determine which error messages the ALB is receiving.
- C. Configure the Amazon S3 bucket destination. After Amazon CloudWatch Logs pulls the ALB logs from the S3 bucket automatically, review the logs in CloudWatch Logs to determine which error messages the ALB is receiving.
- D. Send the logs to Amazon CloudWatch Logs. Use the Amazon Athena CloudWatch Connector to determine which error messages the ALB is receiving.

Answer: B**Explanation:**

The correct answer is B. Here's why:

Application Load Balancer (ALB) access logs are stored in Amazon S3 buckets. To efficiently analyze these logs for errors, you need a querying tool. Amazon Athena is a serverless, interactive query service that makes it easy to analyze data in Amazon S3 using standard SQL.

Option A is incorrect because while CloudWatch Logs can ingest logs, it's not designed for efficient analysis of large-scale, structured data like ALB access logs directly without considerable pre-processing and complex filtering. CloudWatch Logs Insights is better suited for real-time operational data rather than historical analysis of access logs.

Option C is incorrect because CloudWatch Logs doesn't automatically pull logs from S3. You would need to configure a mechanism for this transfer, which is not an efficient approach compared to directly querying the data in S3 with Athena.

Option D is incorrect because while Athena can connect to CloudWatch Logs via a connector, it's not the direct and efficient method for analyzing ALB access logs, which are already stored in S3. Using Athena directly on the S3 bucket where the ALB logs are stored is more efficient and cost-effective.

Therefore, the most efficient workflow is to configure the ALB to send access logs to an S3 bucket and then use Amazon Athena to query those logs to identify error messages. Athena allows you to run SQL queries directly against the log files in S3, making it easy to identify specific errors like HTTP 500 errors or other issues reflected in the logs.

Here are some helpful links:

[Application Load Balancer Access Logs](#)

[Amazon Athena](#)

[Analyzing Application Load Balancer logs with Amazon Athena](#)

Question: 70

A company is planning to use Amazon S3 to archive financial data. The data is currently stored in an on-premises data center. The company uses AWS Direct Connect with a Direct Connect gateway and a transit gateway to

connect to the on-premises data center. The data cannot be transported over the public internet and must be encrypted in transit.

Which solution will meet these requirements?

- A. Create a Direct Connect public VIF. Set up an IPsec VPN connection over the public VIF to access Amazon S3. Use HTTPS for communication.
- B. Create an IPsec VPN connection over the transit VIF. Create a VPC and attach the VPC to the transit gateway. In the VPC, provision an interface VPC endpoint for Amazon S3. Use HTTPS for communication.
- C. Create a VPC and attach the VPC to the transit gateway. In the VPC, provision an interface VPC endpoint for Amazon S3. Use HTTPS for communication.
- D. Create a Direct Connect public VIF. Set up an IPsec VPN connection over the public VIF to the transit gateway. Create an attachment for Amazon S3. Use HTTPS for communication.

Answer: B

Explanation:

The correct solution is B because it leverages AWS Direct Connect's private connectivity along with secure tunneling and interface VPC endpoints to meet the stated requirements of private data transfer and encryption in transit.

Here's a breakdown of why option B is the best choice:

1. **Private Connectivity:** The question explicitly states that data cannot be transported over the public internet. Option B utilizes the existing AWS Direct Connect connection through the transit VIF, ensuring data remains on the AWS private network.
2. **Encryption in Transit:** Option B establishes an IPsec VPN connection over the transit VIF. This creates an encrypted tunnel for the data as it travels between the on-premises data center and AWS. HTTPS is also specified for S3 communication adding another layer of encryption.
3. **Access to S3:** Creating a VPC, attaching it to the Transit Gateway, and then provisioning an interface VPC endpoint for S3 provides a private connection to S3 within the AWS network, bypassing the public internet. This private connectivity to S3 ensures data transfer occurs securely and privately within the AWS environment. Interface VPC Endpoints use AWS PrivateLink to provide private connectivity.

Why other options are incorrect:

Option A & D: Both options A and D propose creating a public VIF. This contradicts the explicit requirement that data cannot be transported over the public internet. Public VIFs inherently route traffic over the internet. Additionally, Option D attempts to create an "attachment" for Amazon S3 on a public VIF which isn't a valid configuration.

Option C: Option C lacks encryption in transit. While it establishes private connectivity to S3 using a VPC endpoint, it doesn't secure the data's journey from the on-premises data center to AWS. The data needs to be encrypted while traversing the Direct Connect link, which requires an IPsec VPN.

In summary, option B offers the most complete solution by combining the private connectivity of Direct Connect, the encryption of an IPsec VPN, and the private S3 access provided by an interface VPC endpoint, fulfilling all stated requirements.

Supporting Links:

AWS Direct Connect: <https://aws.amazon.com/directconnect/>

AWS Transit Gateway: <https://aws.amazon.com/transit-gateway/>

AWS VPN: <https://aws.amazon.com/vpn/>

VPC Endpoints: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html>

Question: 71

A company is using Amazon Route 53 Resolver DNS Firewall in a VPC to block all domains except domains that are on an approved list. The company is concerned that if DNS Firewall is unresponsive, resources in the VPC might be affected if the network cannot resolve any DNS queries. To maintain application service level agreements, the company needs DNS queries to continue to resolve even if Route 53 Resolver does not receive a response from DNS Firewall.

Which change should a network engineer implement to meet these requirements?

- A. Update the DNS Firewall VPC configuration to disable fail open for the VPC.
- B. Update the DNS Firewall VPC configuration to enable fail open for the VPC.
- C. Create a new DHCP options set with parameter `dns_firewall_fail_open=false`. Associate the new DHCP options set with the VPC.
- D. Create a new DHCP options set with parameter `dns_firewall_fail_open=true`. Associate the new DHCP options set with the VPC.

Answer: B**Explanation:**

The question requires ensuring DNS resolution continues even if Route 53 Resolver DNS Firewall is unresponsive. This implies a need for a fail-safe mechanism.

Option B suggests enabling "fail open" for the DNS Firewall VPC configuration. Fail open, in the context of DNS Firewall, means that if the firewall is unable to process DNS queries (e.g., due to an outage), it allows all DNS queries to pass through without being inspected. This ensures that applications can continue to resolve DNS and function normally, even if the DNS Firewall is unavailable. This directly addresses the requirement of maintaining application service levels despite DNS Firewall unresponsiveness.

Option A suggests disabling fail open, which is the opposite of what's needed. Disabling fail open would cause DNS resolution to fail if the firewall is unresponsive, thus violating the requirements.

Options C and D suggest using DHCP options sets to configure the "dns_firewall_fail_open" parameter. However, the `dns_firewall_fail_open` is not a valid DHCP option. Route 53 Resolver DNS Firewall's fail-open behavior is configured directly within the VPC's DNS Firewall configuration, not through DHCP.

Therefore, the only option that achieves the stated goal is to enable fail open at the VPC level, ensuring DNS resolution continues when the DNS Firewall is unavailable.

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver-dns-firewall-configuring.html><https://aws.amazon.com/blogs/networking-and-content-delivery/protecting-your-dns-traffic-using-route-53-resolver-dns-firewall/>

Question: 72

A company is migrating an existing application to a new AWS account. The company will deploy the application in a single AWS Region by using one VPC and multiple Availability Zones. The application will run on Amazon EC2 instances. Each Availability Zone will have several EC2 instances. The EC2 instances will be deployed in private subnets.

The company's clients will connect to the application by using a web browser with the HTTPS protocol. Inbound connections must be distributed across the Availability Zones and EC2 instances. All connections from the same client session must be connected to the same EC2 instance. The company must provide end-to-end encryption for

all connections between the clients and the application by using the application SSL certificate.

Which solution will meet these requirements?

A. Create a Network Load Balancer. Create a target group. Set the protocol to TCP and the port to 443 for the target group. Turn on session affinity (sticky sessions). Register the EC2 instances as targets. Create a listener. Set the protocol to TCP and the port to 443 for the listener. Deploy SSL certificates to the EC2 instances.

B. Create an Application Load Balancer. Create a target group. Set the protocol to HTTP and the port to 80 for the target group. Turn on session affinity (sticky sessions) with an application-based cookie policy. Register the EC2 instances as targets. Create an HTTPS listener. Set the default action to forward to the target group. Use AWS Certificate Manager (ACM) to create a certificate for the listener.

C. Create a Network Load Balancer. Create a target group. Set the protocol to TLS and the port to 443 for the target group. Turn on session affinity (sticky sessions). Register the EC2 instances as targets. Create a listener. Set the protocol to TLS and the port to 443 for the listener. Use AWS Certificate Manager (ACM) to create a certificate for the application.

D. Create an Application Load Balancer. Create a target group. Set the protocol to HTTPS and the port to 443 for the target group. Turn on session affinity (sticky sessions) with an application-based cookie policy. Register the EC2 instances as targets. Create an HTTP listener. Set the port to 443 for the listener. Set the default action to forward to the target group.

Answer: A

Explanation:

The correct answer is A. Here's why:

The requirements specify HTTPS protocol with end-to-end encryption using the application's SSL certificate, distribution across Availability Zones, and session affinity (sticky sessions).

Network Load Balancer (NLB): NLBs operate at the transport layer (Layer 4) and are ideal for TCP/TLS traffic. This makes them suitable for handling the HTTPS traffic while providing high performance and low latency. NLBs also support TLS termination.

TCP/TLS Protocol: NLBs can forward traffic using TCP or TLS protocols. Since we need to encrypt all traffic between the clients and the EC2 instances, TLS should be implemented either on the NLB or on the backend instances (end-to-end). The answer suggests implementing TLS on the backend instances.

Session Affinity (Sticky Sessions): NLBs support session affinity based on IP addresses, which ensures that all connections from the same client IP address are routed to the same EC2 instance.

End-to-End Encryption: Deploying the SSL certificates directly on the EC2 instances ensures end-to-end encryption between the clients and the application.

Why other options are incorrect:

Option B (ALB with HTTP): An Application Load Balancer (ALB) with HTTP protocol would terminate the SSL connection at the ALB and send unencrypted traffic to the EC2 instances, failing to meet the end-to-end encryption requirement.

Option C (NLB with TLS and ACM): This option suggests using ACM for the NLB, but doesn't say to also add certificates to the EC2 instances. This would terminate the TLS connection at the NLB, and traffic between the NLB and EC2 instance would be unencrypted.

Option D (ALB with HTTPS listener and HTTP target): This configuration would result in the ALB terminating the HTTPS connection and forwarding unencrypted HTTP traffic to the EC2 instances, violating the end-to-end encryption requirement. Also the ports don't match between the listener and target group.

Supporting documentation:

[Network Load Balancer](#)

[Application Load Balancer](#)

Question: 73

A company is developing an application in which IoT devices will report measurements to the AWS Cloud. The application will have millions of end users. The company observes that the IoT devices cannot support DNS resolution. The company needs to implement an Amazon EC2 Auto Scaling solution so that the IoT devices can connect to an application endpoint without using DNS.

Which solution will meet these requirements MOST cost-effectively?

- A. Use an Application Load Balancer (ALB)-type target group for a Network Load Balancer (NLB). Create an EC2 Auto Scaling group. Attach the Auto Scaling group to the ALB. Set up the IoT devices to connect to the IP addresses of the NLB.
- B. Use an AWS Global Accelerator accelerator with an Application Load Balancer (ALB) endpoint. Create an EC2 Auto Scaling group. Attach the Auto Scaling group to the ALB. Set up the IoT devices to connect to the IP addresses of the accelerator.
- C. Use a Network Load Balancer (NLB). Create an EC2 Auto Scaling group. Attach the Auto Scaling group to the NLB. Set up the IoT devices to connect to the IP addresses of the NLB.
- D. Use an AWS Global Accelerator accelerator with a Network Load Balancer (NLB) endpoint. Create an EC2 Auto Scaling group. Attach the Auto Scaling group to the NLB. Set up the IoT devices to connect to the IP addresses of the accelerator.

Answer: C**Explanation:**

The scenario requires a cost-effective solution where IoT devices, incapable of DNS resolution, can connect to an application endpoint without DNS. We need a load balancing solution that provides static IP addresses and integrates well with EC2 Auto Scaling.

Option C utilizes a Network Load Balancer (NLB) directly connected to an EC2 Auto Scaling group. NLBs provide static IP addresses per Availability Zone, addressing the core requirement of no DNS resolution needed. The Auto Scaling group dynamically scales the EC2 instances based on demand. This approach is straightforward and cost-effective because it leverages the core functionality of NLBs without introducing additional services. NLBs are designed for high throughput and low latency, making them suitable for handling the high volume of connections from millions of IoT devices.

Option A uses an Application Load Balancer (ALB) behind an NLB. This introduces unnecessary complexity and cost. ALBs are typically used for HTTP/HTTPS traffic and layer-7 routing, which are not relevant to the stated problem concerning IoT devices lacking DNS capabilities.

Option B uses AWS Global Accelerator with an ALB endpoint. Global Accelerator improves performance by routing traffic over AWS's global network, but is not needed simply to provide fixed IPs for devices that can't resolve DNS. It would increase complexity and cost without providing specific advantages for IoT devices with DNS limitations.

Option D uses AWS Global Accelerator with an NLB endpoint. While Global Accelerator provides static IP addresses, it is more expensive than using the NLB's static IPs directly. Global Accelerator is typically used for applications needing global performance improvements, which are not specified here. Thus, it introduces unnecessary cost and complexity.

Therefore, option C is the most cost-effective and straightforward solution because it leverages NLB's static IP addresses and Auto Scaling group integration to meet the stated requirements without introducing unnecessary complexity or services.

Further Research:

Network Load Balancer (NLB):

<https://docs.aws.amazon.com/elasticloadbalancing/latest/network/introduction.html>

EC2 Auto Scaling: <https://docs.aws.amazon.com/autoscaling/ec2/userguide/auto-scaling-benefits.html>

Question: 74

CertyIQ

A company has deployed a new web application on Amazon EC2 instances behind an Application Load Balancer (ALB). The instances are in an Amazon EC2 Auto Scaling group. Enterprise customers from around the world will use the application. Employees of these enterprise customers will connect to the application over HTTPS from office locations.

The company must configure firewalls to allow outbound traffic to only approved IP addresses. The employees of the enterprise customers must be able to access the application with the least amount of latency.

Which change should a network engineer make in the infrastructure to meet these requirements?

- A. Create a new Network Load Balancer (NLB). Add the ALB as a target of the NLB.
- B. Create a new Amazon CloudFront distribution. Set the ALB as the distribution's origin.
- C. Create a new accelerator in AWS Global Accelerator. Add the ALB as an accelerator endpoint.
- D. Create a new Amazon Route 53 hosted zone. Create a new record to route traffic to the ALB.

Answer: C

Explanation:

The correct answer is C: Create a new accelerator in AWS Global Accelerator. Add the ALB as an accelerator endpoint.

Here's why this is the best solution, and why the others aren't ideal:

AWS Global Accelerator: Global Accelerator uses the AWS global network to route traffic to the closest healthy endpoint based on the user's location. This significantly reduces latency for global users, as traffic doesn't have to traverse long distances over the public internet. Crucially, Global Accelerator provides static IP addresses that customers can whitelist in their firewalls, meeting the security requirement. Global Accelerator offers connection optimization and failover capabilities, ensuring high availability and performance. <https://aws.amazon.com/global-accelerator/>

Why other options are incorrect:

A. Network Load Balancer (NLB): While NLBs can handle high traffic volumes, they don't inherently reduce latency for globally distributed users. They primarily operate within a single AWS Region. Using an NLB in front of an ALB adds complexity without addressing the latency or static IP address requirements.

B. Amazon CloudFront: CloudFront caches content at edge locations, reducing latency for repeated requests. However, it's primarily designed for static or cacheable content. The web application, likely generating dynamic content, wouldn't benefit as much from CloudFront. Also, CloudFront's IP address ranges change, making it difficult to whitelist them in firewalls.

D. Amazon Route 53 hosted zone: Route 53 is a DNS service. While it can route traffic to the ALB, it doesn't address the latency concerns for global users or provide static IP addresses for whitelisting. The routing strategies in Route 53 (like latency-based routing) aren't as optimized for performance across the AWS global network as Global Accelerator.

In summary, Global Accelerator is the only service that effectively addresses both the low latency requirement for a global user base and the need for static IP addresses for firewall whitelisting. It leverages the AWS global network for optimized routing, making it the most suitable solution.

Question: 75

A company has hundreds of VPCs on AWS. All the VPCs access the public endpoints of Amazon S3 and AWS Systems Manager through NAT gateways. All the traffic from the VPCs to Amazon S3 and Systems Manager travels through the NAT gateways. The company's network engineer must centralize access to these services and must eliminate the need to use public endpoints.

Which solution will meet these requirements with the LEAST operational overhead?

- A. Create a central egress VPC that has private NAT gateways. Connect all the VPCs to the central egress VPC by using AWS Transit Gateway. Use the private NAT gateways to connect to Amazon S3 and Systems Manager by using private IP addresses.
- B. Create a central shared services VPC. In the central shared services VPC, create interface VPC endpoints for Amazon S3 and Systems Manager to access. Ensure that private DNS is turned off. Connect all the VPCs to the central shared services VPC by using AWS Transit Gateway. Create an Amazon Route 53 forwarding rule for each interface VPC endpoint. Associate the forwarding rules with all the VPCs. Forward DNS queries to the interface VPC endpoints in the shared services VPC.
- C. Create a central shared services VPC. In the central shared services VPC, create interface VPC endpoints for Amazon S3 and Systems Manager to access. Ensure that private DNS is turned off. Connect all the VPCs to the central shared services VPC by using AWS Transit Gateway. Create an Amazon Route 53 private hosted zone with a full service endpoint name for Amazon S3 and Systems Manager. Associate the private hosted zones with all the VPCs. Create an alias record in each private hosted zone with the full AWS service endpoint pointing to the interface VPC endpoint in the shared services VPC.
- D. Create a central shared services VPC. In the central shared services VPC, create interface VPC endpoints for Amazon S3 and Systems Manager to access. Connect all the VPCs to the central shared services VPC by using AWS Transit Gateway. Ensure that private DNS is turned on for the interface VPC endpoints and that the transit gateway is created with DNS support turned on.

Answer: C

Explanation:

The correct answer is C because it provides the most streamlined approach to centralizing access to S3 and Systems Manager while minimizing operational overhead. Here's a detailed justification:

Centralized Access: Option C establishes a shared services VPC containing interface VPC endpoints for S3 and Systems Manager. This concentrates access in one location, eliminating the need for individual NAT gateways in each VPC.

Private Connectivity: Using interface VPC endpoints ensures all traffic to S3 and Systems Manager remains within the AWS network, bypassing the public internet and eliminating the need for NAT gateways.

Least Operational Overhead: Creating an Amazon Route 53 private hosted zone and associating it with all VPCs automates DNS resolution for S3 and Systems Manager. This ensures that when services in any of the VPCs attempt to access S3 or Systems Manager, they will automatically resolve to the interface VPC endpoints within the shared services VPC. This simplifies the network configuration and maintenance compared to other options.

Private DNS Zone for resolution: Private DNS hosted zones provide DNS resolution only for the VPCs that it is associated with.

Alias records in route 53: Alias records map to VPC endpoints which handles the internal routing to services in AWS.

Why other options are wrong:

Option A: Using private NAT gateways in a central egress VPC does not eliminate the need to route traffic through public endpoints and does not leverage the benefits of interface VPC endpoints.

Option B: Turning off Private DNS and implementing custom Route 53 forwarding rules introduces unnecessary complexity. Interface endpoints automatically handle DNS resolution with Private DNS enabled,

while the Transit Gateway has an option for turning on DNS support.

Option D: It enables Private DNS name resolution, which may cause DNS conflicts as the same DNS names might be used in other VPCs.

Here are some helpful resources:

AWS PrivateLink: <https://aws.amazon.com/privatelink/>

VPC Endpoints: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html>

AWS Transit Gateway: <https://aws.amazon.com/transit-gateway/>

Route 53 Private Hosted Zones: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/private-hosted-zones.html>

Question: 76

CertyIQ

A company manages resources across VPCs in multiple AWS Regions. The company needs to connect to the resources by using its internal domain name. A network engineer needs to apply the `aws.example.com` DNS suffix to all resources.

What must the network engineer do to meet this requirement?

- A. Create an Amazon Route 53 private hosted zone for `aws.example.com` in each Region that has resources. Associate the private hosted zone with that Region's VPC. In the appropriate private hosted zone, create DNS records for the resources in each Region.
- B. Create one Amazon Route 53 private hosted zone for `aws.example.com`. Configure the private hosted zone to allow zone transfers with every VPC.
- C. Create one Amazon Route 53 private hosted zone for `example.com`. Create a single resource record for `aws.example.com` in the private hosted zone. Apply a multivalue answer routing policy to the record. Add all VPC resources as separate values in the routing policy.
- D. Create one Amazon Route 53 private hosted zone for `aws.example.com`. Associate the private hosted zone with every VPC that has resources. In the private hosted zone, create DNS records for all resources.

Answer: D

Explanation:

The correct answer is D. Here's why:

The requirement is to resolve resources across multiple VPCs and Regions using the `aws.example.com` DNS suffix. This necessitates a centralized DNS management solution that can span these different network environments. Amazon Route 53 private hosted zones are designed for this purpose.

Option D proposes creating a single private hosted zone for `aws.example.com` and associating it with all relevant VPCs. This is crucial because it allows all VPCs, regardless of Region, to query the same DNS records. Within this hosted zone, the network engineer creates the necessary DNS records (A, CNAME, etc.) pointing to the resources in each VPC. This way, any resource within any of the associated VPCs can resolve resources using the `aws.example.com` suffix.

Option A is incorrect because creating separate private hosted zones in each Region defeats the purpose of a centralized DNS solution. While it would work within a single Region, cross-Region resolution would become overly complex. It also adds operational overhead because the DNS entries need to be managed across multiple zones.

Option B is incorrect because Route 53 doesn't natively support zone transfers to VPCs. Zone transfers are typically used between DNS servers. Sharing DNS information with VPCs is done using Route 53's VPC association feature, not zone transfers.

Option C is incorrect because using a multivalued answer routing policy alone doesn't solve the problem. While multivalued answer routing is a valid routing policy within Route 53, it doesn't simplify the DNS configuration across multiple VPCs. A single record for `aws.example.com` with multiple values doesn't address the need to create records for the specific resources within each VPC. Plus it uses `example.com` which fails to create the DNS suffix of `aws.example.com`.

In summary, option D offers the most straightforward, scalable, and manageable approach to resolving resources across multiple VPCs and Regions using a consistent DNS suffix. It leverages the core capabilities of Route 53 private hosted zones for centralized DNS management.

Further research:

[Amazon Route 53 Private Hosted Zones](#)
[Associating Hosted Zones with Your VPCs](#)

Question: 77

CertyIQ

An insurance company is planning the migration of workloads from its on-premises data center to the AWS Cloud. The company requires end-to-end domain name resolution. Bi-directional DNS resolution between AWS and the existing on-premises environments must be established. The workloads will be migrated into multiple VPCs. The workloads also have dependencies on each other, and not all the workloads will be migrated at the same time.

Which solution meets these requirements?

A. Configure a private hosted zone for each application VPC, and create the requisite records. Create a set of Amazon Route 53 Resolver inbound and outbound endpoints in an egress VPC. Define Route 53 Resolver rules to forward requests for the on-premises domains to the on-premises DNS resolver. Associate the application VPC private hosted zones with the egress VPC, and share the Route 53 Resolver rules with the application accounts by using AWS Resource Access Manager. Configure the on-premises DNS servers to forward the cloud domains to the Route 53 inbound endpoints.

B. Configure a public hosted zone for each application VPC, and create the requisite records. Create a set of Amazon Route 53 Resolver inbound and outbound endpoints in an egress VPC. Define Route 53 Resolver rules to forward requests for the on-premises domains to the on-premises DNS resolver. Associate the application VPC private hosted zones with the egress VPC, and share the Route 53 Resolver rules with the application accounts by using AWS Resource Access Manager. Configure the on-premises DNS servers to forward the cloud domains to the Route 53 inbound endpoints.

C. Configure a private hosted zone for each application VPC, and create the requisite records. Create a set of Amazon Route 53 Resolver inbound and outbound endpoints in an egress VPC. Define Route 53 Resolver rules to forward requests for the on-premises domains to the on-premises DNS resolver. Associate the application VPC private hosted zones with the egress VPC and share the Route 53 Resolver rules with the application accounts by using AWS Resource Access Manager. Configure the on-premises DNS servers to forward the cloud domains to the Route 53 outbound endpoints.

D. Configure a private hosted zone for each application VPC, and create the requisite records. Create a set of Amazon Route 53 Resolver inbound and outbound endpoints in an egress VPC. Define Route 53 Resolver rules to forward requests for the on-premises domains to the on-premises DNS resolver. Associate the Route 53 outbound rules with the application VPCs, and share the private hosted zones with the application accounts by using AWS Resource Access Manager. Configure the on-premises DNS servers to forward the cloud domains to the Route 53 inbound endpoints.

Answer: A

Explanation:

Here's a detailed justification for why option A is the correct solution, along with supporting concepts and links:

The question specifies a requirement for bi-directional DNS resolution between AWS and on-premises environments during a workload migration. The solution must support multiple VPCs and dependencies

between workloads, even as they are migrated incrementally.

Option A correctly addresses these requirements by leveraging Amazon Route 53 Resolver inbound and outbound endpoints. Here's a breakdown:

1. **Private Hosted Zones:** Using private hosted zones for each application VPC ensures internal DNS resolution within each VPC is isolated and managed independently. This aligns with the "multiple VPCs" requirement and allows for specific DNS records related to the migrated applications.
2. **Route 53 Resolver Endpoints (Inbound & Outbound):** Route 53 Resolver inbound endpoints provide a target IP address for the on-premises DNS servers to forward DNS queries for AWS-based domains. Conversely, outbound endpoints allow AWS workloads to query the on-premises DNS servers.
3. **Egress VPC:** Centralizing the Route 53 Resolver endpoints in an egress VPC simplifies management and security. Instead of creating and managing endpoints in every VPC, a single, well-controlled egress point handles DNS traffic.
4. **Route 53 Resolver Rules:** Resolver rules define how DNS queries are routed. Specifically, a rule is created to forward requests for on-premises domains to the on-premises DNS resolver.
5. **Resource Access Manager (RAM):** RAM allows sharing the Route 53 Resolver rules with the application accounts. Sharing the resolver rules is crucial for centralizing management and ensuring consistent routing policies.
6. **On-Premises DNS Forwarding:** Configuring the on-premises DNS servers to forward requests for cloud domains to the Route 53 inbound endpoints establishes the bi-directional DNS resolution.

In contrast, here's why the other options are not as suitable:

Option B: Using public hosted zones is incorrect because the requirement is for internal, private DNS resolution between AWS and on-premises environments. Public hosted zones are for publicly accessible domains.

Option C: Configuring the on-premises DNS servers to forward to Route 53 outbound endpoints is backwards. On-premises DNS servers need to forward to inbound endpoints.

Option D: Associating Route 53 outbound rules with application VPCs, instead of sharing the hosted zones for AWS to On-Prem resolution, is also not the intended method.

Supporting Concepts & Links:

Amazon Route 53 Resolver: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver.html>

Private Hosted Zones: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/private-hosted-zones.html>

AWS Resource Access Manager (RAM): <https://aws.amazon.com/ram/>

Hybrid DNS: <https://aws.amazon.com/blogs/networking-and-content-delivery/hybrid-cloud-dns-resolution-using-amazon-route-53-resolver/>

In summary, option A provides a comprehensive and well-architected solution for bi-directional DNS resolution in a hybrid cloud environment with multiple VPCs, aligning with the best practices for workload migration and AWS networking.

Question: 78

CertyIQ

A global company runs business applications in the us-east-1 Region inside a VPC. One of the company's regional offices in London uses a virtual private gateway for an AWS Site-to-Site VPN connection to the VPC. The company has configured a transit gateway and has set up peering between the VPC and other VPCs that various departments in the company use.

Employees at the London office are experiencing latency issues when they connect to the business applications.

What should a network engineer do to reduce this latency?

- A. Create a new Site-to-Site VPN connection. Set the transit gateway as the target gateway. Enable acceleration on the new Site-to-Site VPN connection. Update the VPN device in the London office with the new connection details.
- B. Modify the existing Site-to-Site VPN connection by setting the transit gateway as the target gateway. Enable acceleration on the existing Site-to-Site VPN connection.
- C. Create a new transit gateway in the eu-west-2 (London) Region. Peer the new transit gateway with the existing transit gateway. Modify the existing Site-to-Site VPN connection by setting the new transit gateway as the target gateway.
- D. Create a new AWS Global Accelerator standard accelerator that has an endpoint of the Site-to-Site VPN connection. Update the VPN device in the London office with the new connection details.

Answer: A

Explanation:

The correct answer is **A**. Here's why:

The core issue is latency between the London office and the AWS VPC in us-east-1. The existing connection is a standard Site-to-Site VPN through a virtual private gateway. Virtual private gateways do not offer acceleration options. To improve performance, we want to leverage AWS's global network for faster routing. AWS Site-to-Site VPN connections now support acceleration.

Option A proposes creating a new Site-to-Site VPN connection and associating it with the transit gateway. Crucially, it suggests enabling acceleration on this new connection. Acceleration leverages AWS's global network to optimize the path between the VPN endpoint in London and the transit gateway, significantly reducing latency. Updating the London office's VPN device ensures traffic uses the new, accelerated connection.

Option B is incorrect because you can't simply enable acceleration on an existing VPN connection that isn't already using a Transit Gateway as its endpoint. The underlying infrastructure needs to be re-provisioned to support acceleration.

Option C involves creating a new transit gateway in London and peering it. While this might seem helpful, it adds unnecessary complexity and cost. The latency problem is between London and us-east-1, not within London. Also, this would not solve the issue, as the initial leg of the traffic would still traverse across the regular internet.

Option D proposes using AWS Global Accelerator. While Global Accelerator is a good service, it doesn't directly integrate with Site-to-Site VPN connections in the way required for this scenario. Global Accelerator directs traffic to specified endpoints (e.g., EC2 instances, Network Load Balancers), but can't be configured as an endpoint for a Site-to-Site VPN connection directly targeting a transit gateway. The Site-to-Site VPN solution using acceleration is a better fit for this low latency use case.

Therefore, Option A provides the most direct and efficient solution to reduce latency by utilizing AWS's network acceleration capabilities for Site-to-Site VPN connections through a transit gateway.

Further reading:

AWS Site-to-Site VPN: <https://aws.amazon.com/vpn/>

AWS Transit Gateway: <https://aws.amazon.com/transit-gateway/>

Accelerated Site-to-Site VPN Connections: <https://aws.amazon.com/about-aws/whats-new/2021/03/aws-site-to-site-vpn-accelerated-vpn-connections/>

Question: 79

CertyIQ

A company has a hybrid cloud environment. The company's data center is connected to the AWS Cloud by an AWS

Direct Connect connection. The AWS environment includes VPCs that are connected together in a hub-and-spoke model by a transit gateway. The AWS environment has a transit VIF with a Direct Connect gateway for on-premises connectivity.

The company has a hybrid DNS model. The company has configured Amazon Route 53 Resolver endpoints in the hub VPC to allow bidirectional DNS traffic flow. The company is running a backend application in one of the VPCs.

The company uses a message-oriented architecture and employs Amazon Simple Queue Service (Amazon SQS) to receive messages from other applications over a private network. A network engineer wants to use an interface VPC endpoint for Amazon SQS for this architecture. Client services must be able to access the endpoint service from on premises and from multiple VPCs within the company's AWS infrastructure.

Which combination of steps should the network engineer take to ensure that the client applications can resolve DNS for the interface endpoint? (Choose three.)

- A. Create the interface endpoint for Amazon SQS with the option for private DNS names turned on.
- B. Create the interface endpoint for Amazon SQS with the option for private DNS names turned off.
- C. Manually create a private hosted zone for `sqs.us-east-1.amazonaws.com`. Add necessary records that point to the interface endpoint. Associate the private hosted zones with other VPCs.
- D. Use the automatically created private hosted zone for `sqs.us-east-1.amazonaws.com` with previously created necessary records that point to the interface endpoint. Associate the private hosted zones with other VPCs.
- E. Access the SQS endpoint by using the public DNS name `sqs.us-east-1.amazonaws.com` in VPCs and on premises.
- F. Access the SQS endpoint by using the private DNS name of the interface endpoint `.sqs.us-east-1.vpce.amazonaws.com` in VPCs and on premises.

Answer: BCF

Explanation:

Here's a breakdown of why the correct answer is BCF, along with justifications and relevant links:

B. Create the interface endpoint for Amazon SQS with the option for private DNS names turned off.

When creating a VPC endpoint, enabling private DNS does not automatically propagate the endpoint's DNS records to your existing DNS infrastructure, especially across VPCs and on-premises. With Private DNS name turned off, AWS will create the endpoint and associate it to the VPC with a regional Endpoint DNS (vpce) which will be used for your DNS configurations. To use Private DNS, additional steps are needed that is beyond the scope of the question.

C. Manually create a private hosted zone for `sqs.us-east-1.amazonaws.com`. Add necessary records that point to the interface endpoint. Associate the private hosted zones with other VPCs.

Since the requirement is to have on-premises and multiple VPCs resolve the endpoint, a private hosted zone that mirrors the SQS domain is essential. This allows you to override the public DNS resolution with the private IP addresses of the interface endpoint within your AWS network. You then need to associate the private hosted zone with all the VPCs that need to access SQS via the endpoint. This is a common pattern for hybrid cloud DNS resolution.

F. Access the SQS endpoint by using the private DNS name of the interface endpoint `.sqs.us-east-1.vpce.amazonaws.com` in VPCs and on premises.

Since private DNS names are turned off, the automatically created VPC endpoint DNS names, which end in `.vpce.amazonaws.com`, must be used for resolving the endpoint from both within VPCs and from on-premises after the appropriate DNS forwarding/resolution is set up. If private hosted zones are configured correctly, the queries will resolve to the private IP addresses of the SQS interface endpoint.

Why other options are incorrect:

A: Enabling private DNS names on the endpoint, while simplifying resolution within the VPC where the endpoint is created, doesn't solve the cross-VPC or on-premises resolution problem without additional configuration.

D: Assuming that using the automatically created private hosted zone will solve the problem without manual intervention is incorrect. The hosted zone needs manual association.

E: Using the public DNS name defeats the purpose of using the private interface endpoint, as it would route traffic over the public internet, which the scenario is designed to avoid.

Supporting Concepts:

Interface VPC Endpoints: Allow you to connect to AWS services privately from your VPC, without exposing your traffic to the public internet. (<https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html>)

Private Hosted Zones: Let you create private DNS records that are only visible within your specified VPCs. (<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/private-hosted-zones.html>)

Route 53 Resolver: Facilitates hybrid cloud DNS by enabling conditional forwarding between your on-premises DNS servers and Route 53. (<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver.html>)

By creating a private hosted zone, adding the interface endpoint record, associating the private hosted zones and turning off the private DNS name option, the network engineer ensures that client applications, both on-premises and in different VPCs, can resolve the SQS endpoint privately.

Question: 80

CertyIQ

A company's network engineer builds and tests network designs for VPCs in a development account. The company needs to monitor the changes that are made to network resources and must ensure strict compliance with network security policies. The company also needs access to the historical configurations of network resources.

Which solution will meet these requirements?

- A. Create an Amazon EventBridge (Amazon CloudWatch Events) rule with a custom pattern to monitor the account for changes. Configure the rule to invoke an AWS Lambda function to identify noncompliant resources. Update an Amazon DynamoDB table with the changes that are identified.
- B. Create custom metrics from Amazon CloudWatch logs. Use the metrics to invoke an AWS Lambda function to identify noncompliant resources. Update an Amazon DynamoDB table with the changes that are identified.
- C. Record the current state of network resources by using AWS Config. Create rules that reflect the desired configuration settings. Set remediation for noncompliant resources.
- D. Record the current state of network resources by using AWS Systems Manager Inventory. Use Systems Manager State Manager to enforce the desired configuration settings and to carry out remediation for noncompliant resources.

Answer: C

Explanation:

The correct answer is **C: Record the current state of network resources by using AWS Config. Create rules that reflect the desired configuration settings. Set remediation for noncompliant resources.**

Here's a detailed justification:

AWS Config continuously monitors and records the configuration of your AWS resources and allows you to automate the evaluation of recorded configurations against desired configurations. This is precisely what the company requires: monitoring changes to network resources and ensuring compliance with network security policies. Config provides historical configuration data and allows you to create Config Rules to check if resources are compliant.

Option A, using EventBridge and Lambda, can detect changes but requires writing custom code to analyze these changes against compliance rules. This is less efficient and more complex than using Config. While EventBridge can trigger actions based on events, it doesn't inherently maintain historical configuration data. Also, managing DynamoDB for storing these changes adds unnecessary overhead.

Option B, using CloudWatch logs and Lambda, is not designed for configuration management. CloudWatch logs primarily focus on monitoring application and system logs. Extracting configuration changes from logs and comparing them against rules would be extremely complex and inefficient.

Option D, using AWS Systems Manager Inventory and State Manager, is primarily intended for managing EC2 instances and their software configurations, rather than general network resource configurations. It can collect information about instances, but it isn't the best tool for overall network configuration compliance like VPCs and network security groups.

Config offers pre-built and custom rules for compliance checking. Remediation actions can be automated within Config to automatically fix non-compliant resources. This directly addresses both the monitoring and compliance requirements in a managed and scalable way.

In summary, AWS Config provides the comprehensive, managed solution best suited for continuous monitoring, historical tracking, and compliance enforcement for network resources, which aligns perfectly with the company's requirements.

Relevant links:

AWS Config: <https://aws.amazon.com/config/>

AWS Config Rules: <https://docs.aws.amazon.com/config/latest/developerguide/config-rules.html>

Question: 81

CertyIQ

A company is migrating an application from on premises to AWS. The company will host the application on Amazon EC2 instances that are deployed in a single VPC. During the migration period, DNS queries from the EC2 instances must be able to resolve names of on-premises servers. The migration is expected to take 3 months. After the 3-month migration period, the resolution of on-premises servers will no longer be needed.

What should a network engineer do to meet these requirements with the LEAST amount of configuration?

- A. Set up an AWS Site-to-Site VPN connection between on premises and AWS. Deploy an Amazon Route 53 Resolver outbound endpoint in the Region that is hosting the VPC.
- B. Set up an AWS Direct Connect connection with a private VIF. Deploy an Amazon Route 53 Resolver inbound endpoint and a Route 53 Resolver outbound endpoint in the Region that is hosting the VPC.
- C. Set up an AWS Client VPN connection between on premises and AWS. Deploy an Amazon Route 53 Resolver inbound endpoint in the VPC.
- D. Set up an AWS Direct Connect connection with a public VIF. Deploy an Amazon Route 53 Resolver inbound endpoint in the Region that is hosting the VPC. Use the IP address that is assigned to the endpoint for connectivity to the on-premises DNS servers.

Answer: A

Explanation:

The correct answer is **A**. Here's why:

A. Set up an AWS Site-to-Site VPN connection between on premises and AWS. Deploy an Amazon Route 53 Resolver outbound endpoint in the Region that is hosting the VPC.

This solution offers the most straightforward and cost-effective way to achieve the required DNS resolution during the migration period with the least amount of configuration.

AWS Site-to-Site VPN: This establishes a secure, encrypted connection between the on-premises network and the AWS VPC. This connection is crucial for the EC2 instances in the VPC to be able to communicate with the on-premises DNS servers. VPN connections are relatively easy to set up and maintain, making them ideal for short-term migrations.

Amazon Route 53 Resolver Outbound Endpoint: This allows DNS queries originating from the VPC to be forwarded to the on-premises DNS servers. The outbound endpoint acts as a forwarder, sending requests to the on-premises DNS resolvers and then returning the resolved names back to the EC2 instances. This allows the EC2 instances to resolve names of on-premises servers as if they were on the same network.

Least Configuration: Option A minimizes the number of components and steps required for setup. A Site-to-Site VPN is less complex to establish than Direct Connect, and the Route 53 Resolver outbound endpoint is simpler than setting up both inbound and outbound endpoints.

Why the other options are not ideal:

B. AWS Direct Connect: Direct Connect is typically used for long-term, high-bandwidth, low-latency connectivity. It involves more complex setup and higher costs than a Site-to-Site VPN, making it unsuitable for a temporary 3-month migration. Also, an inbound endpoint is not needed. It is to allow DNS queries from on-premise to AWS.

C. AWS Client VPN: Client VPN is designed for individual users to securely connect to AWS resources, not for connecting entire networks. It is not appropriate for enabling DNS resolution for all EC2 instances in a VPC.

D. AWS Direct Connect with a Public VIF: While Direct Connect provides connectivity, using a public VIF for on-premises DNS resolution is unconventional. It's more commonly used for accessing public AWS services. Additionally, relying on the IP address of the inbound endpoint for connectivity is not best practice and could lead to issues if the IP address changes.

Authoritative Links:

AWS Site-to-Site VPN: <https://aws.amazon.com/vpn/>

Amazon Route 53 Resolver: <https://aws.amazon.com/route53/resolver/>

AWS Direct Connect: <https://aws.amazon.com/directconnect/>

Question: 82

CertyIQ

A company is hosting an application on Amazon EC2 instances behind an Application Load Balancer. The instances are in an Amazon EC2 Auto Scaling group. Because of a recent change to a security group, external users cannot access the application.

A network engineer needs to prevent this downtime from happening again. The network engineer must implement a solution that remediates noncompliant changes to security groups.

Which solution will meet these requirements?

A. Configure Amazon GuardDuty to detect inconsistencies between the desired security group configuration and the current security group configuration. Create an AWS Systems Manager Automation runbook to remediate noncompliant security groups.

B. Configure an AWS Config rule to detect inconsistencies between the desired security group configuration and the current security group configuration. Configure AWS OpsWorks for Chef to remediate noncompliant security groups.

C. Configure Amazon GuardDuty to detect inconsistencies between the desired security group configuration and the current security group configuration. Configure AWS OpsWorks for Chef to remediate noncompliant security groups.

D. Configure an AWS Config rule to detect inconsistencies between the desired security group configuration and the current security group configuration. Create an AWS Systems Manager Automation runbook to remediate noncompliant security groups.

Answer: D

Explanation:

The best solution to detect and remediate noncompliant security group changes is option D, leveraging AWS Config and AWS Systems Manager Automation. AWS Config is designed to continuously monitor and record AWS resource configurations and evaluate them against desired configurations. Specifically, AWS Config rules can detect inconsistencies between the intended security group configuration (as defined in the rule) and the actual configuration of the security group. This provides ongoing visibility into compliance status.

Once AWS Config detects a noncompliant security group, AWS Systems Manager Automation runbooks can automatically remediate the issue. A runbook can be defined to revert the security group to its desired state, for example, by adding the required rule to allow external access, effectively resolving the downtime.

Alternatives are less suitable. GuardDuty primarily focuses on threat detection and malicious activity, not configuration compliance. OpsWorks for Chef is primarily a configuration management service for application deployment and infrastructure automation, not specifically security group configuration compliance. Although Chef could be used, it is not as directly relevant or efficient as AWS Config's built-in rules and remediation capabilities. Systems Manager Automation integrates well with AWS Config for automated remediation, making it the preferred choice. AWS Config coupled with SSM Automation offers a direct, efficient, and automated way to maintain security group compliance, preventing future downtime events.

Relevant Links:

AWS Config: <https://aws.amazon.com/config/>

AWS Systems Manager Automation: <https://aws.amazon.com/systems-manager/automation/>

Question: 83

CertyIQ

A company is deploying third-party firewall appliances for traffic inspection and NAT capabilities in its VPC. The VPC is configured with private subnets and public subnets. The company needs to deploy the firewall appliances behind a load balancer.

Which architecture will meet these requirements MOST cost-effectively?

- A. Deploy a Gateway Load Balancer with the firewall appliances as targets. Configure the firewall appliances with a single network interface in a private subnet. Use a NAT gateway to send the traffic to the internet after inspection.
- B. Deploy a Gateway Load Balancer with the firewall appliances as targets. Configure the firewall appliances with two network interfaces: one network interface in a private subnet and another network interface in a public subnet. Use the NAT functionality on the firewall appliances to send the traffic to the internet after inspection.
- C. Deploy a Network Load Balancer with the firewall appliances as targets. Configure the firewall appliances with a single network interface in a private subnet. Use a NAT gateway to send the traffic to the internet after inspection.
- D. Deploy a Network Load Balancer with the firewall appliances as targets. Configure the firewall appliances with two network interfaces: one network interface in a private subnet and another network interface in a public subnet. Use the NAT functionality on the firewall appliances to send the traffic to the internet after inspection.

Answer: B

Explanation:

Here's a detailed justification for why option B is the most cost-effective architecture for deploying third-party firewall appliances behind a load balancer in AWS for traffic inspection and NAT, considering the constraints and requirements:

Justification:

The primary goal is to inspect traffic using third-party firewall appliances and provide NAT functionality within a VPC containing both public and private subnets, all while minimizing costs.

Gateway Load Balancer (GWLB): GWLB is specifically designed for virtual appliances like firewalls. It operates at layer 3 (network layer) and forwards traffic based on IP addresses. This makes it ideal for directing traffic to firewall appliances for inspection, allowing the appliances to perform their functions and then route the traffic onward. Using a NLB is not efficient as it operates at layer 4 or 7 which is not needed in this use-case.

Two Network Interfaces: Configuring the firewall appliance with two network interfaces is crucial. One interface resides in a private subnet to receive traffic from the internal network. The other interface resides in a public subnet to provide a path to the internet. This separation enables the firewall appliance to act as a gateway, inspecting traffic moving between the internal network and the internet.

NAT on Firewall Appliance: Utilizing the NAT capabilities of the firewall appliance itself is a cost-effective strategy. The firewall appliance receives internal private IP addresses through one interface, inspects the traffic, and then performs NAT using its public IP address on the other interface before sending the traffic to the internet. This eliminates the need for a separate NAT Gateway service, saving on data processing charges.

Cost Efficiency: Option A utilizes NAT Gateway. NAT Gateway charges for data processing, which can be significantly reduced by leveraging the firewall's NAT capabilities. Options C and D using Network Load Balancers aren't optimized for deep packet inspection and routing through virtual appliances like GWLB is.

In summary:

Option B combines the benefits of GWLB for firewall appliance integration with the firewall's inherent NAT functionality, leading to the most cost-effective solution by eliminating the need for a dedicated NAT Gateway. The dual-interface configuration ensures proper routing between internal networks and the internet, all managed within the firewall appliance for security and efficiency.

Authoritative Links:

AWS Gateway Load Balancer: <https://aws.amazon.com/elasticloadbalancing/gateway-load-balancer/>

AWS NAT Gateway Pricing: <https://aws.amazon.com/vpc/pricing/>

Question: 84

CertyIQ

A company's AWS architecture consists of several VPCs. The VPCs include a shared services VPC and several application VPCs. The company has established network connectivity from all VPCs to the on-premises DNS servers.

Applications that are deployed in the application VPCs must be able to resolve DNS for internally hosted domains on premises. The applications also must be able to resolve local VPC domain names and domains that are hosted in Amazon Route 53 private hosted zones.

What should a network engineer do to meet these requirements?

- A. Create a new Route 53 Resolver inbound endpoint in the shared services VPC. Create forwarding rules for the on-premises hosted domains. Associate the rules with the new Resolver endpoint and each application VPC. Update each application VPC's DHCP configuration to point DNS resolution to the new Resolver endpoint.
- B. Create a new Route 53 Resolver outbound endpoint in the shared services VPC. Create forwarding rules for the on-premises hosted domains. Associate the rules with the new Resolver endpoint and each application VPC.
- C. Create a new Route 53 Resolver outbound endpoint in the shared services VPC. Create forwarding rules for the on-premises hosted domains. Associate the rules with the new Resolver endpoint and each application VPC. Update each application VPC's DHCP configuration to point DNS resolution to the new Resolver endpoint.

D. Create a new Route 53 Resolver inbound endpoint in the shared services VPC. Create forwarding rules for the on-premises hosted domains. Associate the rules with the new Resolver endpoint and each application VPC.

Answer: B

Explanation:

The correct answer is B. Here's why:

The problem requires applications in application VPCs to resolve DNS records in three locations: on-premises, local VPC, and Route 53 private hosted zones. Route 53 Resolver is the AWS service for hybrid DNS resolution. To resolve on-premises domains from AWS, we need to use a Route 53 Resolver outbound endpoint. An outbound endpoint forwards DNS queries from your VPC to your on-premises DNS servers. We then need to create forwarding rules that specify the on-premises domain names and associate them with the outbound endpoint. These rules tell the Resolver where to send requests for the specified domains. Associating the rules with each application VPC ensures that applications in those VPCs can use the rules.

Option A is incorrect because it uses an inbound endpoint. Inbound endpoints are for resolving AWS-hosted domains from on-premises, not the other way around. Updating DHCP configurations is not necessary when using Route 53 Resolver outbound endpoints.

Option C is incorrect because while it correctly identifies the need for an outbound endpoint and forwarding rules, it also incorrectly suggests updating each application VPC's DHCP configuration. This is unnecessary as the Route 53 Resolver outbound endpoint handles the forwarding automatically based on the associated rules.

Option D is incorrect because, similar to option A, it uses an inbound endpoint, which is for on-premises to resolve AWS-hosted domains, not vice-versa.

Therefore, option B correctly sets up an outbound endpoint to forward the necessary DNS queries to on-premises DNS servers.

Relevant documentation:

Route 53 Resolver: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver.html>

Creating Resolver Endpoints: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver-endpoints-creating.html>

Creating Resolver Rules: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver-rules-creating.html>

Question: 85

CertyIQ

A company has been using an outdated application layer protocol for communication among applications. The company decides not to use this protocol anymore and must migrate all applications to support a new protocol. The old protocol and the new protocol are TCP-based, but the protocols use different port numbers.

After several months of work, the company has migrated dozens of applications that run on Amazon EC2 instances and in containers. The company believes that all the applications have been migrated, but the company wants to verify this belief. A network engineer needs to verify that no application is still using the old protocol.

Which solution will meet these requirements without causing any downtime?

A. Use Amazon Inspector and its Network Reachability rules package. Wait until the analysis has finished running to find out which EC2 instances are still listening to the old port.

B. Enable Amazon GuardDuty. Use the graphical visualizations to filter for traffic that uses the port of the old protocol. Exclude all internet traffic to filter out occasions when the same port is used as an ephemeral port.

C. Configure VPC flow logs to be delivered into an Amazon S3 bucket. Use Amazon Athena to query the data

and to filter for the port number that is used by the old protocol.

D. Inspect all security groups that are assigned to the EC2 instances that host the applications. Remove the port of the old protocol if that port is in the list of allowed ports. Verify that the applications are operating properly after the port is removed from the security groups.

Answer: C

Explanation:

The correct answer is **C: Configure VPC flow logs to be delivered into an Amazon S3 bucket. Use Amazon Athena to query the data and to filter for the port number that is used by the old protocol.**

Here's why:

VPC Flow Logs provide network traffic visibility: VPC Flow Logs capture information about the IP traffic going to, from, and within your VPC. This includes source and destination IP addresses, port numbers, and the number of bytes transferred. <https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs.html>

Non-disruptive: Enabling VPC Flow Logs doesn't interrupt your network traffic or application operations. The capture is passive, occurring in the background.

Athena enables querying: Amazon Athena is a serverless interactive query service that makes it easy to analyze data in Amazon S3 using standard SQL. <https://aws.amazon.com/athena/>

Specific identification: Using Athena, you can easily query the VPC Flow Logs data, filtering by the specific port number associated with the outdated protocol. This allows you to identify the specific EC2 instances or containers still using that port. A query like `SELECT DISTINCT srcaddr, dstaddr FROM vpc_flow_logs WHERE dstport = <old_protocol_port>` will surface the source and destination IPs communicating over that port.

Comprehensive Analysis: Athena can analyze large volumes of flow log data, providing a comprehensive view of network activity across the entire environment.

Here's why the other options are less suitable:

A: Amazon Inspector: While Inspector can identify security vulnerabilities and deviations from best practices, its Network Reachability rules focus on whether a resource is reachable, not on the specific protocol or port being used. Furthermore, analysis completion may not be immediate.

B: Amazon GuardDuty: GuardDuty detects malicious activity and unauthorized behavior. While it might detect unusual port activity, it primarily focuses on threats and may not provide the granular visibility needed to confirm protocol usage across all applications. Filtering out internet traffic is essential, but GuardDuty isn't designed for detailed protocol usage auditing.

D: Security Group Inspection: Removing ports from security groups before verifying applications have migrated could cause downtime. Additionally, even if the port isn't explicitly allowed in the security group, the underlying network configuration (e.g., NACLs or instance firewall configurations) could still permit the traffic. This method also requires manually checking many security groups, which is inefficient. Security groups also don't track historical port usage; they only reflect the current configuration.

Question: 86

CertyIQ

A company has deployed its AWS environment in a single AWS Region. The environment consists of a few hundred application VPCs, a shared services VPC, and a VPN connection to the company's on-premises environment. A network engineer needs to implement a transit gateway with the following requirements:

- Application VPCs must be isolated from each other.
- Bidirectional communication must be allowed between the application VPCs and the on-premises network.
- Bidirectional communication must be allowed between the application VPCs and the shared services VPC.

The network engineer creates the transit gateway with options disabled for default route table association and

default route table propagation. The network engineer also creates the VPN attachment for the on-premises network and creates the VPC attachments for the application VPCs and the shared services VPC.

The network engineer must meet all the requirements for the transit gateway by designing a solution that needs the least number of transit gateway route tables.

Which combination of actions should the network engineer perform to accomplish this goal? (Choose two.)

- A. Configure a separate transit gateway route table for on premises. Associate the VPN attachment with this transit gateway route table. Propagate all application VPC attachments to this transit gateway route table.
- B. Configure a separate transit gateway route table for each application VPC. Associate each application VPC attachment with its respective transit gateway route table. Propagate the shared services VPC attachment and the VPN attachment to this transit gateway route table.
- C. Configure a separate transit gateway route table for all application VPCs. Associate all application VPCs with this transit gateway route table. Propagate the shared services VPC attachment and the VPN attachment to this transit gateway route table.
- D. Configure a separate transit gateway route table for the shared services VPC. Associate the shared services VPC attachment with this transit gateway route table. Propagate all application VPC attachments to this transit gateway route table.
- E. Configure a separate transit gateway route table for on premises and the shared services VPC. Associate the VPN attachment and the shared services VPC attachment with this transit gateway route table. Propagate all application VPC attachments to this transit gateway route table.

Answer: CE

Explanation:

Here's a breakdown of why options C and E are the correct choices for this scenario, along with justifications and supporting concepts:

Understanding the Requirements:

Isolation: Application VPCs should not be able to directly communicate with each other.

On-Premises Connectivity: Application VPCs need bidirectional access to the on-premises network.

Shared Services Connectivity: Application VPCs need bidirectional access to the shared services VPC.

Minimal Route Tables: The goal is to achieve the above with the fewest possible transit gateway route tables.

Why C and E are Correct:

Option C: Configure a separate transit gateway route table for all application VPCs. Associate all application VPCs with this transit gateway route table. Propagate the shared services VPC attachment and the VPN attachment to this transit gateway route table.

This action centralizes routing for all application VPCs in a single route table. By only propagating the shared services VPC and VPN attachment routes into this route table, application VPCs will be able to reach the shared services and on-premises networks, but not each other. This enforces the required isolation.

Option E: Configure a separate transit gateway route table for on-premises and the shared services VPC. Associate the VPN attachment and the shared services VPC attachment with this transit gateway route table. Propagate all application VPC attachments to this transit gateway route table.

This creates a dedicated route table for routing traffic to on-premises and the shared services VPC. By propagating all the application VPCs to this route table, the on-premises network and the shared services VPC become aware of the application VPC CIDRs. This allows return traffic from on-premises and shared services to reach the appropriate application VPC.

Why other options are incorrect:

Option A: Creating a route table only for on-premises traffic and propagating the VPC attachments doesn't

isolate the application VPCs from each other. They would still be able to communicate directly within the default transit gateway route table if it was being used (which, based on the scenario, it is not.)

Option B: Creating a separate route table for each application VPC would technically achieve the isolation, but it violates the "least number of route tables" requirement. It's unnecessarily complex for this scenario.

Option D: Creating a route table only for the shared services VPC and propagating the application VPC attachments enables return traffic from the shared services VPC to the application VPCs. However, this does not address the requirement for the application VPCs to access on-premises resources.

In summary:

Options C and E work together to create the necessary routing with the minimal amount of route tables.

Option C isolates application VPCs while providing connectivity to shared services and on-premises. Option E enables return traffic from shared services and on-premises to the application VPCs.

Supporting Cloud Computing Concepts:

Transit Gateway Route Tables: Transit gateway route tables control the routing of traffic between connected resources (VPCs, VPNs, Direct Connect gateways).

Association: Associating an attachment with a route table means that traffic originating from that attachment will be routed according to the rules in that route table.

Propagation: Propagating an attachment to a route table means that the CIDR blocks of that attachment will be advertised in that route table, allowing traffic to be routed to that attachment.

Network Isolation: Isolation is achieved by carefully controlling which attachments are associated and propagated to which route tables.

Authoritative Links:

AWS Transit Gateway: <https://aws.amazon.com/transit-gateway/>

Transit Gateway Route Tables: <https://docs.aws.amazon.com/transit-gateway/latest/userguide/tgw-route-tables.html>

Question: 87

CertyIQ

A company has an AWS Site-to-Site VPN connection between its existing VPC and on-premises network. The default DHCP options set is associated with the VPC. The company has an application that is running on an Amazon Linux 2 Amazon EC2 instance in the VPC. The application must retrieve an Amazon RDS database secret that is stored in AWS Secrets Manager through a private VPC endpoint. An on-premises application provides internal RESTful API service that can be reached by URL (<https://api.example.internal>). Two on-premises Windows DNS servers provide internal DNS resolution.

The application on the EC2 instance needs to call the internal API service that is deployed in the on-premises environment. When the application on the EC2 instance attempts to call the internal API service by referring to the hostname that is assigned to the service, the call fails. When a network engineer tests the API service call from the same EC2 instance by using the API service's IP address, the call is successful.

What should the network engineer do to resolve this issue and prevent the same problem from affecting other resources in the VPC?

A. Create a new DHCP options set that specifies the on-premises Windows DNS servers. Associate the new DHCP options set with the existing VPC. Reboot the Amazon Linux 2 EC2 instance.

B. Create an Amazon Route 53 Resolver rule. Associate the rule with the VPC. Configure the rule to forward DNS queries to the on-premises Windows DNS servers if the domain name matches [example.internal](https://api.example.internal).

C. Modify the local host file in the Amazon Linux 2 EC2 instance in the VPC to map the service domain name (api.example.internal) to the IP address of the internal API service.

D. Modify the local `/etc/resolv.conf` file in the Amazon Linux 2 EC2 instance in the VPC. Change the IP addresses of the name servers in the file to the IP addresses of the company's on-premises Windows DNS servers.

Answer: B

Explanation:

The issue is that the EC2 instance in the VPC cannot resolve the on-premises API service's hostname (api.example.internal). This indicates a DNS resolution problem, as accessing the service by IP address works, meaning network connectivity exists.

Option A is not the best approach because modifying the DHCP options set to include on-premises DNS servers affects all resources in the VPC. This can cause unnecessary DNS traffic across the VPN and might introduce latency for other services. It also mixes internal and external DNS resolution, which can create conflicts. [https://docs.aws.amazon.com/vpc/latest/userguide/VPC_DHCP_Options.html]

Option B, creating a Route 53 Resolver rule, is the most effective and recommended solution. Resolver rules allow you to conditionally forward DNS queries based on the domain name. By creating a rule that forwards queries for example.internal to the on-premises Windows DNS servers, you ensure that only relevant queries are sent across the VPN connection. This keeps internal DNS resolution separate from external resolution, reduces VPN traffic, and provides a scalable and manageable solution for hybrid DNS. [<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver.html>]

Option C, modifying the local host file, only resolves the issue for a single EC2 instance. This is not a scalable or manageable solution for other resources in the VPC and requires manual configuration on each instance.

Option D, modifying the /etc/resolv.conf file, is also not recommended as it directly modifies the EC2 instance's DNS settings. This configuration change is not persistent across reboots or instance replacements and is not a centralized solution.

Therefore, the best solution is to create an Amazon Route 53 Resolver rule because it provides a scalable, manageable, and centralized approach to resolving on-premises domain names from within the VPC, addressing the specific requirement without impacting other resources or requiring manual configurations on individual EC2 instances.

Question: 88

CertyIQ

A company has several production applications across different accounts in the AWS Cloud. The company operates from the us-east-1 Region only. Only certain partner companies can access the applications. The applications are running on Amazon EC2 instances that are in an Auto Scaling group behind an Application Load Balancer (ALB). The EC2 instances are in private subnets and allow traffic only from the ALB. The ALB is in a public subnet and allows inbound traffic only from partner network IP address ranges over port 80.

When the company adds a new partner, the company must allow the IP address range of the partner network in the security group that is associated with the ALB in each account. A network engineer must implement a solution to centrally manage the partner network IP address ranges.

Which solution will meet these requirements in the MOST operationally efficient manner?

A. Create an Amazon DynamoDB table to maintain all IP address ranges and security groups that need to be updated. Update the DynamoDB table with the new IP address range when the company adds a new partner. Invoke an AWS Lambda function to read new IP address ranges and security groups from the DynamoDB table to update the security groups. Deploy this solution in all accounts.

B. Create a new prefix list. Add all allowed IP address ranges to the prefix list. Use Amazon EventBridge (Amazon CloudWatch Events) rules to invoke an AWS Lambda function to update security groups whenever a new IP address range is added to the prefix list. Deploy this solution in all accounts.

C. Create a new prefix list. Add all allowed IP address ranges to the prefix list. Share the prefix list across different accounts by using AWS Resource Access Manager (AWS RAM). Update security groups to use the prefix list instead of the partner IP address range. Update the prefix list with the new IP address range when the company adds a new partner.

D. Create an Amazon S3 bucket to maintain all IP address ranges and security groups that need to be updated. Update the S3 bucket with the new IP address range when the company adds a new partner. Invoke an AWS Lambda function to read new IP address ranges and security groups from the S3 bucket to update the security groups. Deploy this solution in all accounts.

Answer: C

Explanation:

The most operationally efficient solution is **C**. Here's why:

Centralized Management with Prefix Lists: Prefix lists provide a centralized way to manage a collection of CIDR blocks. This eliminates the need to individually update security groups across multiple accounts and applications whenever a new partner is added.

AWS RAM for Cross-Account Sharing: AWS Resource Access Manager (RAM) enables sharing resources, such as prefix lists, across different AWS accounts within an organization. This ensures a single source of truth for partner IP ranges, simplifying updates and preventing inconsistencies.

Direct Security Group Integration: Security groups can directly reference prefix lists. This means that when the prefix list is updated, the security groups automatically inherit the changes, eliminating the need for custom automation using Lambda functions or other services to update individual security groups.

Operational Efficiency: This approach minimizes operational overhead. The network engineer only needs to update the prefix list, and the changes are automatically propagated. This reduces the risk of errors associated with manual updates or custom automation scripts.

Options A, B, and D involve complex and less efficient solutions:

A and D (DynamoDB/S3 with Lambda): These approaches require custom automation using DynamoDB/S3 and Lambda functions to manage and propagate IP address range updates. This adds complexity and increases the risk of errors.

B (Prefix List with EventBridge and Lambda): While it uses prefix lists, relying on EventBridge and Lambda to update security groups introduces unnecessary complexity and latency compared to direct security group integration with prefix lists.

Authoritative Links:

Prefix Lists: <https://docs.aws.amazon.com/vpc/latest/userguide/working-with-prefix-lists.html>

AWS Resource Access Manager (RAM): <https://docs.aws.amazon.com/ram/latest/userguide/what-is.html>

Question: 89

CertyIQ

A company uses a 1 Gbps AWS Direct Connect connection to connect its AWS environment to its on-premises data center. The connection provides employees with access to an application VPC that is hosted on AWS. Many remote employees use a company-provided VPN to connect to the data center. These employees are reporting slowness when they access the application during business hours. On-premises users have started to report similar slowness while they are in the office.

The company plans to build an additional application on AWS. On-site and remote employees will use the additional application. After the deployment of this additional application, the company will need 20% more bandwidth than the company currently uses. With the increased usage, the company wants to add resiliency to the AWS connectivity. A network engineer must review the current implementation and must make improvements within a limited budget.

What should the network engineer do to meet these requirements MOST cost-effectively?

A. Set up a new 1 Gbps Direct Connect dedicated connection to accommodate the additional traffic load from remote employees and the additional application. Create a link aggregation group (LAG).

B. Deploy an AWS Site-to-Site VPN connection to the application VPC. Configure the on-premises routing for

the remote employees to connect to the Site-to-Site VPN connection.

C. Deploy Amazon Workspaces into the application VPC. Instruct the remote employees to connect to Workspaces.

D. Replace the existing 1 Gbps Direct Connect connection with two new 2 Gbps Direct Connect hosted connections. Create an AWS Client VPN endpoint in the application VPC. Instruct the remote employees to connect to the Client VPN endpoint.

Answer: D

Explanation:

The correct answer is **D**. Here's why:

Problem Breakdown:

The company faces three primary issues:

1. **Bandwidth Saturation:** The existing 1 Gbps Direct Connect is nearing capacity, causing performance issues for both on-premises and remote users. The new application will exacerbate this.
2. **Lack of Resiliency:** A single Direct Connect connection represents a single point of failure.
3. **Remote User Access Bottleneck:** Remote employees are accessing the application via a company VPN connected to the data center, adding latency and strain to the Direct Connect.

Why Option D is the Best Solution:

Increased Bandwidth and Resiliency: Replacing the single 1 Gbps Direct Connect with two 2 Gbps hosted connections (total 4 Gbps) provides both the required 20% additional bandwidth and redundancy. Hosted connections are typically more cost-effective for smaller bandwidth increments than dedicated connections. The total bandwidth should cover the 1.2 Gbps needed. The use of dual connections satisfies the requirement of adding resiliency to the AWS connectivity.

Improved Remote User Experience: Implementing AWS Client VPN offloads remote user traffic from the Direct Connect connection. Remote users connect directly to the application VPC, bypassing the data center's VPN and reducing latency. This direct access also alleviates the bandwidth strain on the Direct Connect.

Cost-Effectiveness: Hosted connections are generally cheaper than dedicated connections for bandwidth requirements of this scale, addressing the budget constraint. Also, Client VPN's consumption-based pricing aligns with the actual usage of remote users, preventing cost overruns.

Why Other Options are Less Suitable:

A: Setting up another 1 Gbps dedicated Direct Connect connection, while providing more bandwidth, is potentially more expensive than hosted connections. It also still routes remote user traffic through the data center VPN, not addressing the latency and bottleneck issue.

B: Deploying Site-to-Site VPN only helps add capacity from on-prem. Remote user access to VPC is still going to be bottlenecked by the on-premise VPN, and add strain to the existing 1Gbps DX.

C: Amazon Workspaces is a more expensive solution than Client VPN and less applicable since there's no need for a full virtual desktop environment; the remote users simply need to access applications.

Supporting Concepts and Links:

AWS Direct Connect: Provides a dedicated network connection from on-premises to AWS.

<https://aws.amazon.com/directconnect/>

AWS Direct Connect Hosted Connections: Direct Connect connections provided by an AWS partner.

<https://aws.amazon.com/directconnect/pricing/>

Link Aggregation Group (LAG): Bundles multiple Direct Connect connections into a single logical connection for increased bandwidth and redundancy.

<https://docs.aws.amazon.com/directconnect/latest/UserGuide/WorkingWithLAGs.html>

AWS Client VPN: Securely connects remote users to AWS resources. <https://aws.amazon.com/vpn/client-vpn/>

AWS Site-to-Site VPN: Securely connects on-premises network to AWS resources.

<https://aws.amazon.com/vpn/site-to-site-vpn/>

Amazon Workspaces: Virtual desktop infrastructure. <https://aws.amazon.com/workspaces/>

In conclusion, Option D strikes the optimal balance between cost, performance, redundancy, and remote user experience improvement. It efficiently addresses all the stated requirements within the budget constraints.

Question: 90

CertyIQ

A company has a global network and is using transit gateways to connect AWS Regions together. The company finds that two Amazon EC2 instances in different Regions are unable to communicate with each other. A network engineer needs to troubleshoot this connectivity issue.

What should the network engineer do to meet this requirement?

- A. Use AWS Network Manager Route Analyzer to analyze routes in the transit gateway route tables and in the VPC route tables. Use VPC flow logs to analyze the IP traffic that security group rules and network ACL rules accept or reject in the VPC.
- B. Use AWS Network Manager Route Analyzer to analyze routes in the transit gateway route tables. Verify that the VPC route tables are correct. Use AWS Firewall Manager to analyze the IP traffic that security group rules and network ACL rules accept or reject in the VPC.
- C. Use AWS Network Manager Route Analyzer to analyze routes in the transit gateway route tables. Verify that the VPC route tables are correct. Use VPC flow logs to analyze the IP traffic that security group rules and network ACL rules accept or reject in the VPC.
- D. Use VPC Reachability Analyzer to analyze routes in the transit gateway route tables. Verify that the VPC route tables are correct. Use VPC flow logs to analyze the IP traffic that security group rules and network ACL rules accept or reject in the VPC.

Answer: C

Explanation:

Here's a detailed justification for why option C is the best approach to troubleshoot connectivity issues between EC2 instances in different regions connected by Transit Gateways:

Option C: Use AWS Network Manager Route Analyzer, verify VPC route tables, and use VPC Flow Logs

This approach provides a comprehensive methodology for diagnosing the problem, addressing potential issues at different layers of the network.

1. **AWS Network Manager Route Analyzer:** AWS Network Manager Route Analyzer is crucial for analyzing routing configurations within Transit Gateways (TGWs). It enables you to trace the path that traffic is expected to take between source and destination. Given the inter-region nature of the problem, confirming that the TGW route tables are correctly propagating routes between Regions is a critical first step. It helps identify if there are missing or incorrect route propagations or associations on the TGW side. (<https://docs.aws.amazon.com/network-manager/latest/tgwroutes/route-analyzer.html>)
2. **VPC Route Table Verification:** Even if the TGW routing is configured correctly, the VPC route tables must also be configured to properly direct traffic to the TGW. It's necessary to confirm that the subnets associated with the EC2 instances in each VPC have routes pointing to the TGW as the next hop for inter-region or inter-VPC communication. Missing or incorrect routes at the VPC level will prevent traffic from reaching or exiting the VPC. The VPC route table must contain a route that sends traffic destined for the remote region (or remote VPC CIDR) to the TGW. (https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Route_Tables.html)

3. **VPC Flow Logs:** After confirming the routing configuration, VPC Flow Logs provide visibility into the network traffic within the VPCs. Analyzing flow logs helps determine whether traffic is being permitted or denied by security groups or Network ACLs (NACLs). By examining flow logs, a network engineer can see if the traffic from one EC2 instance is reaching the other EC2 instance's VPC, and if it's being accepted or rejected. This aids in pinpointing security-related issues that are preventing communication. (<https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs.html>)

Why other options are less suitable:

Option A: This option includes Route Analyzer and VPC Flow Logs which are correct, but it doesn't explicitly mention checking the VPC Route tables which are critical for the EC2 instances to route their traffic to the TGW.

Option B: AWS Firewall Manager is primarily used for managing and enforcing firewall rules across multiple AWS accounts and applications. It does not directly help in troubleshooting point-to-point connectivity problems between two instances in different VPCs when the concern is routing and basic security group/NACL configurations. While it provides a centralized security management, it doesn't offer the same level of granular, packet-level visibility as VPC Flow Logs.

Option D: VPC Reachability Analyzer is a useful tool, but it is designed for analyzing connectivity within a single VPC or between VPCs within a single region. It is not designed to analyze connectivity issues in the same way across multiple AWS regions using Transit Gateways. It is also a passive analysis tool, while flow logs capture actual traffic. While it can be used to confirm potential reachability issues, it isn't as direct for analyzing inter-region routing across TGWs, and doesn't help with identifying traffic being blocked by security rules in the manner that flow logs do.

In summary, option C provides the most systematic and comprehensive approach to isolate and resolve the connectivity issue by addressing the routing configuration at the transit gateway and VPC levels, as well as the security configurations within the VPCs.

Question: 91

CertyIQ

A company needs to transfer data between its VPC and its on-premises data center. The data must travel through a connection that has dedicated bandwidth. The data also must be encrypted in transit. The company has been working with an AWS Partner Network (APN) Partner to establish the connection.

Which combination of steps will meet these requirements? (Choose three.)

- A. Request a hosted connection from the APN Partner.
- B. Request a hosted public VIF from the APN Partner.
- C. Create an AWS Site-to-Site VPN connection.
- D. Create an AWS Client VPN connection.
- E. Create a private VIF.
- F. Create a public VIF.

Answer: ACF

Explanation:

The correct answer is ACF. Here's why:

A. Request a hosted connection from the APN Partner: The company is already working with an APN Partner to establish a connection, which aligns with using a hosted connection. Hosted connections, offered by AWS Direct Connect Partners, provide a dedicated connection with varying bandwidth options suitable for

transferring large datasets consistently. A hosted connection leverages the partner's infrastructure to connect the on-premises network to AWS.

C. Create an AWS Site-to-Site VPN connection: To ensure data is encrypted in transit across the dedicated connection (AWS Direct Connect), a Site-to-Site VPN can be configured to encrypt data before it leaves the on-premises environment and decrypt it upon arrival in the VPC. This uses industry-standard IPsec VPN tunnels over the Direct Connect link.

F. Create a public VIF: A public VIF (Virtual Interface) is required for accessing public AWS services over the Direct Connect connection when using a hosted connection. While the primary goal is data transfer between VPC and on-premises, integrating a public VIF allows the company to reach public services that may be part of the larger workflow, such as S3 for storing the transferred data. The AWS Site-to-Site VPN created in Step C will still handle encryption of the data.

Why other options are incorrect:

B. Request a hosted public VIF from the APN Partner: A Public VIF is only needed when accessing Public AWS services over Direct Connect but does not fulfill the primary requirement of transferring data between the VPC and on-premises.

D. Create an AWS Client VPN connection: Client VPN is used for individual users to securely connect to AWS resources. It is not designed for connecting an entire on-premises data center to a VPC with dedicated bandwidth.

E. Create a private VIF: A private VIF is used for establishing a direct connection to a VPC over Direct Connect. To meet the encryption requirements of the question, this must be paired with an AWS Site-to-Site VPN.

Authoritative links for further research:

AWS Direct Connect: <https://aws.amazon.com/directconnect/>

AWS Site-to-Site VPN: <https://aws.amazon.com/vpn/site-to-site-vpn/>

AWS Direct Connect Virtual Interfaces:

<https://docs.aws.amazon.com/directconnect/latest/UserGuide/WorkingWithVirtualInterfaces.html>

Question: 92

CertyIQ

A company's security guidelines state that all outbound traffic from a VPC to the company's on-premises data center must pass through a security appliance. The security appliance runs on an Amazon EC2 instance. A network engineer needs to improve the network performance between the on-premises data center and the security appliance.

Which actions should the network engineer take to meet these requirements? (Choose two.)

- A. Use an EC2 instance that supports enhanced networking.
- B. Send outbound traffic through a transit gateway.
- C. Increase the EC2 instance size.
- D. Place the EC2 instance in a placement group within the VPC.
- E. Attach multiple elastic network interfaces to the EC2 instance.

Answer: AC

Explanation:

The correct answer is AC. Here's why:

A. Use an EC2 instance that supports enhanced networking: Enhanced networking utilizes Single Root I/O

Virtualization (SR-IOV) to provide higher bandwidth, higher packet per second (PPS) performance, and lower latency. This reduces network bottlenecks and optimizes throughput between the EC2 instance (hosting the security appliance) and the on-premises data center. By enabling enhanced networking, the security appliance can process and forward traffic more efficiently. This directly addresses the need to improve network performance. <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/enhanced-networking.html>

C. Increase the EC2 instance size: Increasing the EC2 instance size often provides more CPU cores and memory, and importantly, it can also provide higher network bandwidth capacity. If the security appliance is CPU-bound or memory-constrained, an increase in instance size can enable it to process network traffic more effectively, improving overall performance. A larger instance type will offer higher network performance limits. The security appliance needs resources to inspect and forward traffic; a larger instance helps. <https://aws.amazon.com/ec2/instance-types/>

Why other options are incorrect:

B. Send outbound traffic through a transit gateway: A transit gateway is helpful for managing connectivity across multiple VPCs and on-premises networks. However, it primarily addresses routing complexity and doesn't inherently improve the performance of the EC2 instance hosting the security appliance. It adds an additional hop, which could potentially decrease performance, unless the existing routing is severely inefficient.

D. Place the EC2 instance in a placement group within the VPC: Placement groups are used to influence the placement of EC2 instances to optimize for low latency, high throughput, or both between instances. Since the primary performance bottleneck is between the security appliance and the on-premises data center (connected via VPN or Direct Connect), placing the EC2 instance in a placement group does little to improve that specific network path. Placement groups affect communication within AWS, not between AWS and on-premises.

E. Attach multiple elastic network interfaces to the EC2 instance: While attaching multiple ENIs might seem like a way to increase bandwidth, it introduces added complexity and may not directly translate to improved performance for a single traffic flow between the security appliance and the on-premises network. Routing traffic across multiple ENIs is more about achieving higher aggregate bandwidth across different flows, not improving a single flow's performance. Also, the EC2 instance would need to be configured to load balance traffic across the interfaces, which can be complex and introduce overhead.

Question: 93

CertyIQ

A company's application team is unable to launch new resources into its VPC. A network engineer discovers that the VPC has run out of usable IP addresses. The VPC CIDR block is 172.16.0.0/16.

Which additional CIDR block can the network engineer attach to the VPC?

- A. 172.17.0.0/29
- B. 10.0.0.0/16
- C. 172.17.0.0/16
- D. 192.168.0.0/16

Answer: C

Explanation:

The correct answer is C. Here's a detailed justification:

The problem states that the VPC (Virtual Private Cloud) with a CIDR block of 172.16.0.0/16 has exhausted its IP

address space. This means the application team cannot launch new resources because there are no available IP addresses within the existing range. To resolve this, we need to add another CIDR block to the VPC.

AWS allows you to associate additional IPv4 CIDR blocks to your VPC, expanding the available IP address space. However, there are some key constraints to consider. The new CIDR block cannot overlap with any existing CIDR blocks associated with the VPC. Also, the new CIDR block must be within the private IPv4 address ranges defined by RFC 1918 (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16) or a publicly routable CIDR block that you own.

Option A (172.17.0.0/29) is incorrect because a /29 CIDR block only provides a very small number of usable IP addresses (6), which would likely be insufficient to address the described problem of the application team being unable to launch new resources.

Option B (10.0.0.0/16) is a valid private IP range. This is a valid option to add.

Option C (172.17.0.0/16) is also a valid private IP range, and it provides a reasonable number of addresses.

Option D (192.168.0.0/16) is also a valid private IP range, and it provides a reasonable number of addresses.

Based on the given "voted answers", Option C (172.17.0.0/16) is the most voted. While B and D are also valid options, it could be that in the context of the original question, that information omitted, that C was the preferred option.

Therefore, adding a non-overlapping CIDR block, such as 172.17.0.0/16 will expand the available IP address space. This will allow the application team to launch new resources within the VPC by assigning them IP addresses from the newly added range. It is crucial that any newly associated CIDR blocks do not conflict with existing routes in the VPC's route tables to prevent routing issues.

Further reading can be found in the AWS documentation:

[Working with VPCs - AWS Documentation](#)
[RFC 1918 - Address Allocation for Private Internets](#)

Question: 94

CertyIQ

A financial trading company is using Amazon EC2 instances to run its trading platform. Part of the company's trading platform includes a third-party pricing service that the EC2 instances communicate with over UDP on port 50000.

Recently, the company has had problems with the pricing service. Some of the responses from the pricing service appear to be incorrectly formatted and are not being processed successfully. The third-party vendor requests access to the data that the pricing service is returning. The third-party vendor wants to capture request and response data for debugging by logging in to an EC2 instance that accesses the pricing service. The company prohibits direct access to production systems and requires all log analysis to be performed in a dedicated monitoring account.

Which set of steps should a network engineer take to capture the data and meet these requirements?

- A.1. Configure VPC flow logs to capture the data that flows in the VPC.
- 2. Send the data to an Amazon S3 bucket.
- 3. In the monitoring account, extract the data that flows to the EC2 instance's IP address and filter the traffic for the UDP data.
- 4. Provide the data to the third-party vendor.
- B.1. Configure a traffic mirror filter to capture the UDP data.
- 2. Configure Traffic Mirroring to capture the traffic for the EC2 instance's elastic network interface.
- 3. Configure a packet inspection package on a new EC2 instance in the production environment. Use the elastic network interface of the new EC2 instance as the target for the traffic mirror.
- 4. Extract the data by using the packet inspection package.
- 5. Provide the data to the third-party vendor.

- C.1. Configure a traffic mirror filter to capture the UDP data.
 2. Configure Traffic Mirroring to capture the traffic for the EC2 instance's elastic network interface.
 3. Configure a packet inspection package on a new EC2 instance in the monitoring account. Use the elastic network interface of the new EC2 instance as the target for the traffic mirror.
 4. Extract the data by using the packet inspection package.
 5. Provide the data to the third-party vendor.
- D.1. Create a new Amazon Elastic Block Store (Amazon EBS) volume. Attach the EBS volume to the EC2 instance.
 2. Log in to the EC2 instance in the production environment. Run the tcpdump command to capture the UDP data on the EBS volume.
 3. Export the data from the EBS volume to Amazon S3.
 4. Provide the data to the third-party vendor.

Answer: C

Explanation:

Here's a detailed justification for why option C is the best solution:

The primary goal is to capture UDP traffic on port 50000 flowing between EC2 instances and a third-party pricing service, then provide that data to the vendor while adhering to the security requirement of no direct access to production systems and performing analysis within a monitoring account.

Option C effectively addresses these requirements using AWS Traffic Mirroring. First, a traffic mirror filter is configured to isolate the desired UDP traffic on port 50000. Second, traffic mirroring is enabled to copy this traffic from the EC2 instance's network interface. Crucially, the mirrored traffic is sent to a target within the dedicated monitoring account. This means a new EC2 instance with a packet inspection package, like tcpdump or Wireshark, is launched within the monitoring account. This adheres to the company's policy of analyzing logs outside the production environment. The packet inspection package is used to extract and analyze the captured UDP data, which can then be securely provided to the third-party vendor.

Option A utilizes VPC flow logs, which captures metadata about network traffic but does not capture the actual packet content (the raw UDP data the vendor needs). Therefore, it cannot satisfy the debugging requirement.

Option B suggests placing the packet inspection package on an EC2 instance within the production environment, violating the requirement of no direct production access.

Option D suggests using tcpdump directly on the production EC2 instance and storing data on an EBS volume attached to it, also violating the security constraint of no direct production access and requiring an unnecessary EBS volume.

Traffic Mirroring allows capture of network packets without affecting the performance of the source instance. It's a secure and efficient way to inspect network traffic for debugging and troubleshooting purposes.<https://aws.amazon.com/vpc/traffic-mirroring/><https://docs.aws.amazon.com/vpc/latest/mirroring/what-is-traffic-mirroring.html>

Question: 95

CertyIQ

A company's network engineer is configuring an AWS Site-to-Site VPN connection between a transit gateway and the company's on-premises network. The Site-to-Site VPN connection is configured to use BGP over two tunnels in active/active mode with equal-cost multi-path (ECMP) routing activated on the transit gateway.

When the network engineer attempts to send traffic from the on-premises network to an Amazon EC2 instance, traffic is sent over the first tunnel. However, return traffic is received over the second tunnel and is dropped at the customer gateway. The network engineer must resolve this issue without reducing the overall VPN bandwidth.

Which solution will meet these requirements?

- A. Configure the customer gateway to use AS PATH prepending and local preference to prefer one tunnel over the other.
- B. Configure the Site-to-Site VPN options to set the first tunnel as the primary tunnel to eliminate asymmetric routing.
- C. Configure the virtual tunnel interfaces on the customer gateway to allow asymmetric routing.
- D. Configure the Site-to-Site VPN to use static routing in active/active mode to ensure that traffic flows over a preferred path.

Answer: C

Explanation:

The correct answer is C: Configure the virtual tunnel interfaces on the customer gateway to allow asymmetric routing.

Here's why:

The problem is asymmetric routing: traffic from on-premises goes out one tunnel, and the response comes back through a different tunnel. The customer gateway drops the return traffic because it expects return traffic on the same tunnel it sent the original traffic. ECMP is intended to distribute traffic across multiple paths, so a solution that eliminates ECMP isn't ideal when the requirement is to maintain overall bandwidth.

Option A (AS PATH prepending and local preference): While this can influence routing decisions, it essentially creates an active/standby setup, reducing the effective bandwidth since one tunnel becomes preferred and the other is a backup. This goes against the requirement of maintaining bandwidth.

Option B (Setting one tunnel as primary): This also results in an active/standby configuration, directly contradicting the requirement of maintaining overall VPN bandwidth. This eliminates the benefits of ECMP.

Option C (Allowing asymmetric routing): This is the most suitable solution. By configuring the customer gateway to accept traffic coming back through different tunnels than it sent the original traffic, you directly address the root cause of the problem without sacrificing bandwidth. The customer gateway needs to be configured to track the state of the connection and accept return traffic regardless of the tunnel used.

Option D (Static routing): Switching to static routing eliminates BGP's dynamic routing capabilities, making the solution less resilient to network changes and requiring manual updates if the network topology changes. It also doesn't inherently solve the asymmetric routing issue; it merely forces traffic down a specific path, and the return traffic might still take a different path unless carefully configured. More importantly, static routing disables the automatic failover and load balancing provided by BGP and ECMP.

Justification:

Asymmetric routing is a common issue in multi-path VPN environments. The core issue stems from stateful firewalls or security devices assuming a 1:1 relationship between outbound and inbound traffic flows. Modern customer gateways or routers can be configured to be more flexible, allowing for asymmetric flows. In this active/active ECMP setup, the transit gateway can send the response through the second tunnel, resulting in the customer gateway dropping the traffic. Configuring the customer gateway to permit asymmetric routing enables it to accept inbound traffic, regardless of the path taken by the outbound traffic. This is typically done by ensuring proper session tracking or connection state management. This approach resolves the issue without sacrificing the benefits of ECMP and BGP for load balancing and redundancy, satisfying the requirement of maintaining overall VPN bandwidth. ECMP relies on the understanding that traffic will be distributed across multiple paths, and the return traffic can be routed through different tunnels. <https://docs.aws.amazon.com/vpn/latest/s2svpn/VPNS2S.html> (AWS Site-to-Site VPN documentation) <https://datatracker.ietf.org/doc/html/rfc2991> (Explains some of the problems that are caused by load balancing return traffic)

Question: 96

A company runs an application on Amazon EC2 instances. A network engineer implements a NAT gateway in the application's VPC to replace self-managed NAT instances. After the network engineer shifts traffic from the self-managed NAT instances to the NAT gateway, users begin to report issues.

During troubleshooting, the network engineer discovers that the connection to the application is closing after approximately 6 minutes of inactivity.

What should the network engineer do to resolve this issue?

- A. Check for increases in the IdleTimeoutCount Amazon CloudWatch metric for the NAT gateway. Configure TCP keepalive on the application EC2 instances.
- B. Check for increases in the ErrorPortAllocation Amazon CloudWatch metric for the NAT gateway. Configure an HTTP timeout value on the application EC2 instances.
- C. Check for increases in the PacketsDropCount Amazon CloudWatch metric for the NAT gateway. Configure an HTTPS timeout value on the application EC2 instances.
- D. Check for decreases in the ActiveConnectionCount Amazon CloudWatch metric for the NAT gateway. Configure UDP keepalive on the application EC2 instances.

Answer: A

Explanation:

Here's a detailed justification for why option A is the correct answer:

The problem describes connection timeouts occurring after approximately 6 minutes of inactivity when using a NAT Gateway. This behavior strongly suggests an idle timeout issue, where the NAT Gateway is prematurely closing connections that it perceives as inactive.

Option A correctly identifies the IdleTimeoutCount metric in CloudWatch as the first point of investigation. This metric directly tracks the number of connections dropped due to exceeding the NAT Gateway's idle timeout. If this count is increasing after the migration to the NAT Gateway, it confirms the idle timeout is the root cause. [<https://docs.aws.amazon.com/vpc/latest/userguide/nat-gateway-metrics.html>]

The NAT Gateway has a default idle timeout of 350 seconds (approximately 5 minutes and 50 seconds). This closely aligns with the observed 6-minute connection closures. [<https://docs.aws.amazon.com/vpc/latest/userguide/nat-gateway-concepts.html#nat-gateway-timeouts>]

The recommended solution in option A is to configure TCP keepalive on the application's EC2 instances. TCP keepalive sends periodic packets to keep the connection alive, preventing the NAT Gateway from considering it idle and prematurely closing it. By sending these keepalive packets more frequently than the NAT Gateway's idle timeout, the connection remains active from the NAT Gateway's perspective. TCP keepalive functionality resides within the operating system or application layer on the EC2 instances.

Options B, C, and D are incorrect because they focus on different metrics and irrelevant solutions:

ErrorPortAllocation (Option B) relates to the NAT Gateway running out of ports, which is unlikely in this scenario involving idle timeouts.

PacketsDropCount (Option C) indicates packet loss due to network issues, which is distinct from idle timeout problems.

ActiveConnectionCount (Option D) reflects the overall number of active connections, not specifically idle timeout issues. Decreases might be expected as long-standing idle connections are closed; it doesn't directly indicate the problem. UDP keepalive is not relevant, as the scenario doesn't explicitly state the usage of UDP. Moreover, the idle timeout issue is most commonly associated with TCP connections.

HTTP or HTTPS timeouts (Options B and C) are application-layer timeouts and wouldn't address the NAT Gateway's connection tracking behavior. These are for controlling application level timeouts, not to prevent TCP session timeouts.

In summary, the problem's description clearly points to an idle timeout issue with the NAT Gateway. Option A correctly identifies the appropriate CloudWatch metric to investigate and offers the correct mitigation strategy: configuring TCP keepalive on the EC2 instances to prevent the NAT Gateway from prematurely closing connections.

Question: 97

CertyIQ

A software-as-a-service (SaaS) company is migrating its private SaaS application to AWS. The company has hundreds of customers that connect to multiple data centers by using VPN tunnels. As the number of customers has grown, the company has experienced more difficulty in its effort to manage routing and segmentation of customers with complex NAT rules.

After the migration to AWS is complete, the company's AWS customers must be able to access the SaaS application directly from their VPCs. Meanwhile, the company's on-premises customers still must be able to connect through IPsec encrypted tunnels.

Which solution will meet these requirements?

- A. Connect the AWS customer VPCs to a shared transit gateway. Use AWS Site-to-Site VPN connections to the transit gateway for the on-premises customers
- B. Use AWS PrivateLink to connect the AWS customers. Use a third-party routing appliance in the SaaS application VPC to terminate on-premises Site-to-Site VPN connections.
- C. Peer each AWS customer's VPCs to the VPC that hosts the SaaS application. Create AWS Site-to-Site VPN connections on the SaaS VPC virtual private gateway.
- D. Use Site-to-Site VPN tunnels to connect each AWS customer's VPCs to the VPC that hosts the SaaS application. Use AWS Site-to-Site VPN to connect the on-premises customers.

Answer: B

Explanation:

The correct answer is **B**, which utilizes AWS PrivateLink for AWS customers and a third-party routing appliance for on-premises VPN connections.

Here's the justification:

AWS PrivateLink for AWS Customers: PrivateLink provides private connectivity between VPCs, AWS services, and on-premises applications without exposing traffic to the public internet. This aligns perfectly with the requirement of direct access from AWS customer VPCs to the SaaS application. It eliminates the need for complex routing and NAT rules that the company previously struggled with. PrivateLink ensures secure and private communication using private IP addresses within the AWS network. (Reference: <https://aws.amazon.com/privatelink/>)

Third-Party Routing Appliance for On-Premises VPNs: A third-party routing appliance within the SaaS application VPC can handle the termination of IPsec VPN tunnels from on-premises customers. This approach allows the SaaS provider to maintain control over the VPN connections and routing within their environment. This offers flexibility in configuring VPN settings, security policies, and routing rules tailored to the on-premises customers' specific needs. (Reference: AWS Marketplace offers various third-party network appliances)

Why other options are less suitable:

Option A (Transit Gateway): While Transit Gateway can centralize connectivity, it's overkill and introduces unnecessary complexity. For AWS customers, PrivateLink is more efficient and secure for private connectivity. Transit Gateway also doesn't directly address the complexity of managing potentially hundreds of Site-to-Site VPN connections. It still introduces a layer of routing to be managed on the Transit Gateway.

Option C (VPC Peering): VPC peering creates direct connections between VPCs, but managing peering connections with hundreds of customers can become a logistical nightmare. Also, it is not transitive, meaning if customer A is peered with SaaS VPC and customer B is peered with SaaS VPC, customer A and customer B cannot communicate with each other via VPC peering. Furthermore, VPC peering doesn't inherently provide encrypted connectivity for on-premises customers.

Option D (VPN Tunnels to Each Customer VPC): Creating Site-to-Site VPN tunnels to each AWS customer's VPC is highly complex and resource-intensive. Managing numerous VPN connections and associated routing configurations would be incredibly challenging, mirroring the company's initial problem. This is not a scalable or maintainable solution.

In summary, Option B provides a balanced approach by leveraging the benefits of PrivateLink for AWS customers and a third-party routing appliance for managing on-premises VPN connections. It offers security, scalability, and simplified management, making it the most suitable solution.

Question: 98

CertyIQ

A company's existing AWS environment contains public application servers that run on Amazon EC2 instances. The application servers run in a VPC subnet. Each server is associated with an Elastic IP address.

The company has a new requirement for firewall inspection of all traffic from the internet before the traffic reaches any EC2 instances. A security engineer has deployed and configured a Gateway Load Balancer (GLB) in a standalone VPC with a fleet of third-party firewalls.

How should a network engineer update the environment to ensure that the traffic travels across the fleet of firewalls?

- A. Deploy a transit gateway. Attach a GLB endpoint to the transit gateway. Attach the application VPC to the transit gateway. Update the application subnet route table's default route destination to be the GLB endpoint. Ensure that the EC2 instances' security group allows traffic from the GLB endpoint.
- B. Update the application subnet route table to have a default route to the GLB. On the standalone VPC that contains the firewall fleet, add a route in the route table for the application VPC's CIDR block with the GLB endpoint as the destination. Update the EC2 instances' security group to allow traffic from the GLB.
- C. Provision a GLB endpoint in the application VPC in a new subnet. Create a gateway route table with a route that specifies the application subnet CIDR block as the destination and the GLB endpoint as the target. Associate the gateway route table with the internet gateway in the application VPC. Update the application subnet route table's default route destination to be the GLB endpoint.
- D. Instruct the security engineer to move the GLB into the application VPC. Create a gateway route table. Associate the gateway route table with the application subnet. Add a default route to the gateway route table with the GLB as its destination. Update the route table on the GLB to direct traffic from the internet gateway to the application servers. Ensure that the EC2 instances' security group allows traffic from the GLB.

Answer: C

Explanation:

Here's a detailed justification for why option C is the correct answer, along with supporting cloud computing concepts and authoritative links:

Justification for Option C:

Option C outlines the correct method for routing traffic from the internet through a Gateway Load Balancer

(GLB) to EC2 instances within a VPC. The key here is the use of a **GLB endpoint** within the application VPC and a **gateway route table** associated with the Internet Gateway.

1. **GLB Endpoint in Application VPC:** Creating a GLB endpoint within the application VPC brings the GLB's functionality closer to the EC2 instances. This eliminates the need for complex inter-VPC routing configurations using Transit Gateway or VPC peering just for the sake of redirecting internet traffic.
2. **Gateway Route Table:** This is specifically designed to route traffic originating from or destined for the internet gateway to the GLB endpoint. Since the GLB sits in front of the EC2 instances, traffic from the internet must first pass through the GLB.
3. **Gateway Route Table Configuration:** The gateway route table is configured with a route that directs traffic destined for the application subnet's CIDR block to the GLB endpoint. This ensures that any traffic from the internet intended for the EC2 instances is intercepted by the GLB for inspection.
4. **Association with Internet Gateway:** Associating the gateway route table with the internet gateway is crucial. The Internet Gateway serves as the entry point for traffic from the internet, and by associating the route table with it, you can control the flow of traffic originating from the internet.
5. **Updating Application Subnet Route Table:** Finally, updating the application subnet's route table to send default traffic to the GLB endpoint ensures that any outbound traffic from the EC2 instances also goes through the GLB for inspection or other processing.

Why other options are incorrect:

Option A (Transit Gateway): While a Transit Gateway could technically be used, it's an overkill solution for this scenario. Transit Gateways are designed for complex network topologies with multiple VPCs and on-premises connections. Using a Transit Gateway solely for traffic inspection from the internet adds unnecessary complexity and cost.

Option B: This option focuses only on the standalone VPC and sets up a return route to the application VPC's CIDR, but it doesn't address how to initially intercept traffic coming from the internet and direct it towards the GLB.

Option D: Moving the GLB into the application VPC might seem simpler, but it loses the benefit of isolating the firewall infrastructure within a separate VPC. A standalone VPC provides better security isolation for your firewall fleet. Moreover, the route table update on the GLB described is incorrect; GLB itself doesn't typically handle routing traffic to the application servers directly but relies on the registered targets.

Supporting Cloud Computing Concepts:

Gateway Load Balancer (GLB): Operates at layer 3 (network layer) and layer 4 (transport layer) to distribute traffic to virtual appliances like firewalls for deep packet inspection.

VPC Route Tables: Control the routing of traffic within a VPC and between different networks.

Internet Gateway: Enables internet access for resources within a VPC.

VPC Endpoints (Gateway Endpoints): Allow private connections to AWS services without requiring internet access. In this case, the GLB endpoint provides a private way to connect to the GLB.

Security Groups: Act as virtual firewalls for EC2 instances, controlling inbound and outbound traffic.

Authoritative Links:

AWS Gateway Load Balancer: <https://docs.aws.amazon.com/elasticloadbalancing/latest/gateway/what-is-gateway-load-balancer.html>

AWS VPC Route Tables: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Route_Tables.html

AWS Internet Gateway: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Internet_Gateway.html

AWS VPC Endpoints: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html>

Question: 99

A company has an AWS Site-to-Site VPN connection between its office and its VPC. Users report occasional failure of the connection to the application that is hosted inside the VPC. A network engineer discovers in the customer gateway logs that the Internet Key Exchange (IKE) session ends when the connection to the application fails.

What should the network engineer do to bring up the IKE session if the IKE session goes down?

- A. Set the dead peer detection (DPD) timeout action to Clear. Initiate traffic from the VPC to on premises.
- B. Set the dead peer detection (DPD) timeout action to Restart. Initiate traffic from on premises to the VPC.
- C. Set the dead peer detection (DPD) timeout action to None. Initiate traffic from the VPC to on premises.
- D. Set the dead peer detection (DPD) timeout action to Cancel. Initiate traffic from on premises to the VPC.

Answer: B

Explanation:

The correct answer is **B. Set the dead peer detection (DPD) timeout action to Restart. Initiate traffic from on premises to the VPC.**

Here's why:

The problem describes an IKE session that's failing, indicating a problem with the VPN connection's health detection. IKE is responsible for establishing and maintaining the security association for the VPN tunnel. Dead Peer Detection (DPD) is a mechanism used to determine if a VPN peer is still available. When a DPD timeout occurs, it means the VPN gateway hasn't received a response from its peer within the configured interval, suggesting a potential connectivity issue.

Why Restart is preferred: Setting the DPD timeout action to "Restart" means that when a DPD timeout is detected, the VPN gateway will actively attempt to re-establish the IKE security association. This is crucial for automatically recovering from temporary connectivity issues. It tells the gateway: "If you think the other side is down, try to rebuild the connection immediately."

Why initiating traffic from on-premises is needed: Starting traffic from on-premises to the VPC will trigger the VPN tunnel to become active from the on-premises side. If the IKE session is down and needs to be re-established, initiating traffic will force the on-premises gateway to attempt to re-establish the connection. Without traffic, the on-premises gateway might passively wait, prolonging the downtime. The other end (VPC) can't unilaterally fix the issue if the on-prem side thinks there is no need to talk. The problem states issues are seen with traffic from on-prem to VPC, thus, testing in this direction is appropriate.

Let's consider why the other options are less suitable:

A. Set the dead peer detection (DPD) timeout action to Clear. Initiate traffic from the VPC to on premises.

"Clear" would likely not attempt to re-establish the connection automatically. It might just clear the security association, requiring manual intervention. Traffic initiated from VPC to on-premises wouldn't address a problem where on-premises cannot reach VPC.

C. Set the dead peer detection (DPD) timeout action to None. Initiate traffic from the VPC to on premises.

"None" disables DPD entirely, which is generally not recommended. It prevents the VPN gateway from detecting connectivity issues, leading to a situation where traffic is lost without any automatic recovery attempts. Traffic initiated from VPC to on-premises wouldn't address a problem where on-premises cannot reach VPC.

D. Set the dead peer detection (DPD) timeout action to Cancel. Initiate traffic from on premises to the VPC.

The action "Cancel" is not typically a standard DPD timeout action in AWS. The common actions are Clear, Restart, and Hold. Furthermore, Cancel may be more appropriate for tearing down a session permanently, not for re-establishing it. While initiating traffic from on-premises to the VPC is useful as discussed above, this

option is flawed.

Therefore, using the Restart setting on the customer gateway with DPD and initiating traffic from on-premises to the VPC is the most effective approach to automatically re-establish the IKE session and restore connectivity.

References:

[AWS Site-to-Site VPN](#)

[Cisco: Understanding IPsec Dead Peer Detection](#)

Question: 100

CertyIQ

A network engineer is designing a hybrid networking environment that will connect a company's corporate network to the company's AWS environment. The AWS environment consists of 30 VPCs in 3 AWS Regions.

The network engineer needs to implement a solution to centrally filter traffic by using a firewall that the company's security team has approved. The solution must give all the VPCs the ability to connect to each other. Connectivity between AWS and the corporate network must meet a minimum bandwidth requirement of 2 Gbps.

Which solution will meet these requirements?

A. Deploy an IPsec VPN connection between the corporate network and a new transit gateway. Connect all VPCs to the transit gateway. Associate the approved firewall with the transit gateway.

B. Deploy a single 10 Gbps AWS Direct Connect connection between the corporate network and virtual private gateway of each VPC. Connect the virtual private gateways to a Direct Connect gateway. Build an IPsec tunnel to a new transit VPC. Deploy the approved firewall to the transit VPC.

C. Deploy two 1 Gbps AWS Direct Connect connections in different Direct Connect locations to connect to the corporate network. Build a transit VIF on each connection to a Direct Connect gateway. Associate the Direct Connect gateway with a new transit gateway for each Region. Configure the VIFs to use equal-cost multipath (ECMP) routing. Connect all the VPCs in the three Regions to the transit gateway. Configure the transit gateway route table to route traffic to an inspection VPC. Deploy the approved firewall to the inspection VPC.

D. Deploy four 1 Gbps AWS Direct Connect connections in different Direct Connect locations to connect to the corporate network. Build a transit VIF on each connection to a Direct Connect gateway. Associate the Direct Connect gateway with a new transit gateway for each Region. Connect the transit gateways by using a transit gateway peering attachment. Configure the VIFs to use equal-cost multipath (ECMP) routing. Configure transit gateway route tables to route traffic to an inspection VPC. Deploy the approved firewall to the inspection VPC.

Answer: D

Explanation:

Here's a detailed justification for why option D is the correct solution:

Why Option D is Correct

Option D provides a robust, scalable, and highly available solution that meets all the requirements:

1. **Bandwidth Requirement:** Deploying four 1 Gbps Direct Connect connections achieves the minimum 2 Gbps bandwidth requirement ($4 \times 1 \text{ Gbps} = 4 \text{ Gbps}$). The use of multiple connections also provides redundancy.
2. **Hybrid Connectivity:** Direct Connect provides a dedicated network connection from the corporate network to AWS.
3. **Centralized Firewall:** The "inspection VPC" houses the approved firewall, allowing for centralized traffic filtering.
4. **Inter-VPC Connectivity:** Connecting all VPCs to a transit gateway within each Region enables connectivity between VPCs in the same Region. Peering the transit gateways allows inter-Region VPC connectivity.

5. **Scalability and Availability:** Using multiple Direct Connect locations and ECMP provides redundancy and scalability. If one connection fails, traffic can be routed through the other connections. Distributing the transit gateways across Regions provides resilience.
6. **Security:** By routing all traffic through the inspection VPC, all traffic to and from AWS can be inspected and filtered centrally.

Why Other Options are Incorrect

Option A: Using a single IPsec VPN connection is unlikely to meet the 2 Gbps bandwidth requirement consistently. VPN connections over the public internet can be subject to latency and fluctuating bandwidth.

Option B: Deploying a Direct Connect connection to each VPC is not scalable or practical for 30 VPCs. It also complicates management and is not cost-effective. Building an IPsec tunnel to a transit VPC introduces unnecessary overhead and complexity when Direct Connect is already available.

Option C: While using two 1 Gbps Direct Connect connections meets the 2 Gbps minimum, it lacks the additional redundancy that four connections provide. Also, a single inspection VPC might become a bottleneck for all Regions.

Supporting Concepts

AWS Direct Connect: Establishes a dedicated network connection from your on-premises environment to AWS, offering higher bandwidth and lower latency than VPNs. (<https://aws.amazon.com/directconnect/>)

AWS Transit Gateway: Acts as a network transit hub, allowing you to interconnect VPCs, on-premises networks, and AWS Transit Gateway Connect attachments. It simplifies network management and reduces complexity. (<https://aws.amazon.com/transit-gateway/>)

Direct Connect Gateway: A globally available resource that enables you to connect multiple VPCs in different AWS Regions to your Direct Connect connection.

Virtual Interface (VIF): Enables access to AWS services. Transit VIFs allow connectivity to Direct Connect gateway and transit gateway.

Equal-Cost Multipath (ECMP): A routing strategy that allows traffic to be distributed across multiple paths with equal cost, improving bandwidth and availability.

Inspection VPC: A centralized location for deploying security appliances (like firewalls) to inspect and filter network traffic.

By combining these services effectively, Option D provides a solution that meets all the specified requirements for bandwidth, connectivity, security, scalability, and availability in the hybrid networking environment.

Question: 101

CertyIQ

A company uses an AWS Direct Connect private VIF with a link aggregation group (LAG) that consists of two 10 Gbps connections. The company's security team has implemented a new requirement for external network connections to provide layer 2 encryption. The company's network team plans to use MACsec support for Direct Connect to meet the new requirement.

Which combination of steps should the network team take to implement this functionality? (Choose three.)

- A. Create a new Direct Connect LAG with new circuits and ports that support MACsec.
- B. Associate the MACsec Connectivity Association Key (CAK) and the Connection Key Name (CKN) with the new LAG.
- C. Associate the Internet Key Exchange (IKE) with the existing LAG.
- D. Configure the MACsec encryption mode on the existing LAG.
- E. Configure the MACsec encryption mode on the new LAG.
- F. Configure the MACsec encryption mode on each Direct Connect connection that makes up the existing LAG.

Answer: ABE

Explanation:

The correct answer is ABE. Let's break down why:

A. Create a new Direct Connect LAG with new circuits and ports that support MACsec: MACsec isn't a feature that can simply be enabled on existing LAGs. You need to establish a new LAG consisting of new circuits and ports that are specifically provisioned with MACsec capabilities at the AWS Direct Connect location. This is because the underlying hardware and configuration on the Direct Connect side must support the encryption.

B. Associate the MACsec Connectivity Association Key (CAK) and the Connection Key Name (CKN) with the new LAG: MACsec uses a pair of keys, CAK and CKN, to perform the encryption and decryption process. These keys are used to authenticate and secure the communication between your on-premises equipment and the AWS Direct Connect endpoint. Associating these keys with the new MACsec-enabled LAG is essential for establishing the encrypted connection.

E. Configure the MACsec encryption mode on the new LAG: After creating the LAG and associating the keys, you need to configure the MACsec encryption mode (e.g., Galois/Counter Mode (GCM)) on the LAG. This tells the Direct Connect and your on-premises equipment how to encrypt and decrypt the traffic. This step defines the security protocol used for the encrypted connection.

Why the other options are incorrect:

C. Associate the Internet Key Exchange (IKE) with the existing LAG: IKE is a key management protocol used for IPsec VPNs, not MACsec. MACsec uses a pre-shared key model with CAK and CKN.

D. Configure the MACsec encryption mode on the existing LAG: As mentioned earlier, MACsec cannot be enabled on existing LAGs without creating new connections configured to support MACsec.

F. Configure the MACsec encryption mode on each Direct Connect connection that makes up the existing LAG: Again, MACsec can't be retroactively added. Additionally, MACsec is configured at the LAG level, not on individual connections within the LAG.

In Summary: Implementing MACsec on AWS Direct Connect requires establishing a new MACsec-capable LAG, associating the CAK/CKN keys, and configuring the encryption mode on the newly established LAG.

Authoritative Links:

AWS Direct Connect MACsec: <https://docs.aws.amazon.com/directconnect/latest/UserGuide/direct-connect-macsec.html>

Question: 102

CertyIQ

A company recently implemented a security policy that prohibits developers from launching VPC network infrastructure. The policy states that any time a NAT gateway is launched in a VPC, the company's network security team must immediately receive an alert to terminate the NAT gateway. The network security team needs to implement a solution that can be deployed across AWS accounts with the least possible administrative overhead. The solution also must provide the network security team with a simple way to view compliance history.

Which solution will meet these requirements?

A. Develop a script that programmatically checks for NAT gateways in an AWS account, sends an email alert, and terminates the NAT gateway if a NAT gateway is detected. Deploy the script on an Amazon EC2 instance in each account. Use a cron job to run the script every 5 minutes. Log the results of the checks to an Amazon RDS for MySQL database.

B. Create an AWS Lambda function that programmatically checks for NAT gateways in an AWS account, sends

an email alert, and terminates the NAT gateway if a NAT gateway is detected. Deploy the Lambda function to each account by using AWS Serverless Application Model (AWS SAM) templates. Store the results of the checks on an Amazon OpenSearch Service cluster in each account.

C.Enable Amazon GuardDuty. Create an Amazon EventBridge rule for the Behavior:EC2/NATGatewayCreation GuardDuty finding type. Configure the rule to invoke an AWS Step Functions state machine to send an email alert and terminate a NAT gateway if a NAT gateway is detected. Store the runtime log as a text file in an Amazon S3 bucket.

D.Create a custom AWS Config rule that checks for NAT gateways in an AWS account. Configure the AWS Config rule to perform an AWS Systems Manager Automation remediation action to send an email alert and terminate the NAT gateway if a NAT gateway is detected. Deploy the AWS Config rule and the Systems Manager runbooks to each account by using AWS CloudFormation StackSets

Answer: D

Explanation:

The best solution is **D. Create a custom AWS Config rule....** Here's why:

AWS Config provides continuous monitoring and assessment of your AWS resource configurations. A custom Config rule allows you to specifically check for NAT gateways in each account, satisfying the policy requirement. The built-in remediation capabilities of AWS Config, specifically using Systems Manager Automation, allow for automated actions like sending alerts and terminating non-compliant NAT gateways, ensuring policy enforcement with minimal manual intervention.

AWS CloudFormation StackSets enable deploying the Config rule and related Systems Manager runbooks across multiple AWS accounts, significantly reducing administrative overhead. This aligns with the requirement for a centrally managed solution. AWS Config also maintains a history of compliance, providing a simple way to view the compliance history of NAT gateway configurations across the organization. This addresses the tracking and auditing requirement.

Option A is less scalable because managing EC2 instances and cron jobs across multiple accounts is complex and resource-intensive. Option B involves managing Lambda functions and OpenSearch clusters in each account, which introduces significant overhead. OpenSearch also seems excessive for this simple logging requirement. Option C relies on GuardDuty, which primarily focuses on threat detection and not configuration compliance. While it can detect NAT gateway creation, it requires more complex integration with Step Functions for remediation, making it less efficient than AWS Config. Config provides a much more direct and comprehensive auditing and remediation solution for resource configuration policies.<https://aws.amazon.com/config/><https://aws.amazon.com/cloudformation/stacksets/><https://docs.aws.amazon.com/config.html>

Question: 103

CertyIQ

A company is running an online game on AWS. The game is played globally and is gaining popularity. Users are reporting problems with the game's responsiveness. Replay rates are dropping, and the company is losing subscribers. Game servers are located in the us-west-2 Region and use an Elastic Load Balancer to distribute client traffic.

The company has decided to deploy game servers to 11 additional AWS Regions to reduce the round-trip times of network traffic to game clients. A network engineer must design a DNS solution that uses Amazon Route 53 to ensure that user traffic is delivered to game servers with an optimal response time.

What should the network engineer do to meet these requirements?

A.Create Route 53 records for the Elastic Load Balancers in each Region. Specify a weighted routing policy. Calculate the weight by using the number of clients in each Region.

B.Create Route 53 records for the Elastic Load Balancers in each Region. Specify a latency routing policy. Set the Region to the Region where the Elastic Load Balancer is deployed.

C. Create Route 53 records for the Elastic Load Balancers in each Region. Specify a multivalue answer routing policy. Test latency from the game client, and connect to the server with the best response.

D. Create Route 53 records for the Elastic Load Balancers in each Region. Specify a geolocation routing policy. Set the location to the Region where the Elastic Load Balancer is deployed.

Answer: B

Explanation:

The correct answer is B. Here's why:

The scenario necessitates a DNS solution that routes users to the game servers with the lowest latency, thereby improving responsiveness and reducing replay rates. Route 53 offers various routing policies, but latency-based routing is specifically designed for this purpose.

Option B (Correct): Latency routing directs traffic to the resource that provides the best latency based on the source of the DNS query. By creating Route 53 records for each Elastic Load Balancer in the 12 Regions and configuring a latency routing policy, Route 53 will automatically determine the Region with the lowest latency for each user and direct them to the corresponding ELB. This directly addresses the requirement of optimal response time. The association of each record with its respective AWS Region informs Route 53 about the location of the resource, enabling accurate latency-based routing decisions.

Option A (Incorrect): Weighted routing distributes traffic based on assigned weights. While it can balance load, it doesn't inherently consider latency. Calculating weights based on the number of clients per Region doesn't guarantee low latency for individual users. For example, a user in Europe might still be directed to us-west-2 if that Region has a high client count but provides poor latency for that specific user.

Option C (Incorrect): Multivalue answer routing returns multiple IP addresses in response to a DNS query, allowing the client to choose. While it can provide redundancy, it doesn't inherently optimize for latency. The client-side latency test requirement is not a standard feature of Route 53 and would require custom implementation, making the solution overly complex.

Option D (Incorrect): Geolocation routing routes traffic based on the geographic location of the user. While useful for compliance or content localization, it doesn't account for network latency. A user in a nearby country might experience better latency from a server in a different Region.

Therefore, latency routing is the most suitable solution because it is specifically designed to optimize for response time by directing users to the geographically nearest server with the lowest latency.

Relevant links:

[Amazon Route 53 Routing Policies](#)
[Latency Based Routing](#)

Question: 104

CertyIQ

A network engineer needs to build an encrypted connection between an on-premises data center and a VPC. The network engineer attaches the VPC to a virtual private gateway and sets up an AWS Site-to-Site VPN connection. The VPN tunnel is UP after configuration and is working. However, during rekey for phase 2 of the VPN negotiation, the customer gateway device is receiving different parameters than the parameters that the device is configured to support.

The network engineer checks the IPsec configuration of the VPN tunnel. The network engineer notices that the customer gateway device is configured with the most secure encryption algorithms that the AWS Site-to-Site VPN configuration file provides.

What should the network engineer do to troubleshoot and correct the issue?

A. Check the native virtual private gateway logs. Restrict the VPN tunnel options to the specific VPN parameters that the virtual private gateway requires.

B. Check the native customer gateway logs. Restrict the VPN tunnel options to the specific VPN parameters that the customer gateway requires.

C. Check Amazon CloudWatch logs of the virtual private gateway. Restrict the VPN tunnel options to the specific VPN parameters that the virtual private gateway requires.

D. Check Amazon CloudWatch logs of the customer gateway. Restrict the VPN tunnel options to the specific VPN parameters that the customer gateway requires.

Answer: B

Explanation:

The correct answer is B. Here's why:

The problem lies in a mismatch of supported encryption parameters between the AWS Site-to-Site VPN's virtual private gateway (VGW) and the customer gateway (CGW) during the rekeying process of Phase 2 IPsec negotiation. The CGW is receiving proposals it doesn't support, even though it's configured with "the most secure" algorithms offered in the AWS configuration file.

Why B is correct:

CGW Logs are Key: The primary source of truth for understanding why the CGW rejects proposals is the CGW's own logs. These logs will detail the specific ISAKMP/IKE or IPsec security association (SA) proposals received from the AWS side and the reason for their rejection (e.g., unsupported encryption algorithm, hash algorithm, Diffie-Hellman group).

Parameter Restriction: The solution requires restricting the VPN tunnel options to the specific parameters the CGW actually supports. While the AWS configuration file provides various options, "most secure" may not be universally compatible with all CGWs. The logs will reveal the constraints. After determining compatible algorithms, go back to the AWS console, under the site-to-site VPN connection, click the tunnel details and go to edit tunnel options, you may now modify the phase1/phase2 encryption algorithms, and key exchange settings to match what is in the CGW logs.

Why other options are incorrect:

A & C (VGW Logs): While VGW logs could show connection attempts and failures, they won't provide the reason for the CGW's rejection with the same level of detail as the CGW's logs. VGW logs can indicate an issue exists, but not what is incompatible.

D (CloudWatch Logs for CGW): The question states "native customer gateway logs" because CloudWatch logs are an abstraction of the true logs, which would be more beneficial in this case. Also, the customer gateway is an on-prem device, so CloudWatch logs would not be relevant. Additionally, the question states that the network engineer must also restrict the VPN tunnel options to the specific VPN parameters that the customer gateway requires, making the question an incorrect choice, as it only directs to check the logs.

In summary, the immediate action is to inspect the customer gateway's native logs to precisely identify the unsupported IPsec parameters. Once these are identified, the AWS VPN configuration needs to be modified to use only parameters that the customer gateway can support. This may mean sacrificing "most secure" options for those with broader compatibility.

Supporting Documentation:

AWS Site-to-Site VPN: https://docs.aws.amazon.com/vpn/latest/s2svpn/VPC_VPN.html

AWS Site-to-Site VPN Tunnel Options:

https://docs.aws.amazon.com/vpn/latest/s2svpn/VPN_Tunnel_Options.html

Troubleshooting AWS Site-to-Site VPN: <https://repost.aws/knowledge-center/vpn-tunnel-down>

Question: 105

A company is growing rapidly. Data transfers between the company's on-premises systems and Amazon EC2 instances that run in VPCs are limited by the throughput of a single AWS Site-to-Site VPN connection between the company's on-premises data center firewall and an AWS Transit Gateway.

A network engineer must resolve the throttling by designing a solution that is highly available and secure. The solution also must scale the VPN throughput from on premises to the VPC resources to support the increase in traffic.

Which solution will meet these requirements?

- A. Configure multiple dynamic BGP-based Site-to-Site VPN connections to the transit gateway. Configure equal-cost multi-path routing (ECMP).
- B. Configure multiple static routing-based Site-to-Site VPN connections to the transit gateway. Configure equal-cost multi-path routing (ECMP).
- C. Configure a new Site-to-Site VPN connection to the transit gateway. Enable acceleration for the Site-to-Site VPN connection.
- D. Configure a software appliance-based VPN connection over the internet from the on-premises firewall to an EC2 instance that has a large instance size and networking capabilities.

Answer: A

Explanation:

The correct answer is A: Configure multiple dynamic BGP-based Site-to-Site VPN connections to the transit gateway. Configure equal-cost multi-path routing (ECMP). This solution effectively addresses the throughput bottleneck while ensuring high availability and scalability.

Here's why:

Increased Throughput: Multiple VPN connections provide aggregated bandwidth. Traffic can be distributed across these connections, effectively increasing the overall throughput between on-premises and AWS.

High Availability: If one VPN connection fails, the remaining connections continue to operate, maintaining connectivity. This provides resilience and ensures the system remains available.

Scalability: As traffic demands grow, additional VPN connections can be added to the transit gateway, scaling the bandwidth accordingly.

BGP Routing: Using BGP (Border Gateway Protocol) is crucial for dynamic routing. BGP automatically learns and propagates routes between the on-premises network and the AWS environment, reducing manual configuration and simplifying network management.

ECMP: Equal-Cost Multi-Path (ECMP) allows traffic to be distributed across multiple paths with equal cost (or metric). By configuring ECMP, traffic is automatically balanced across the active VPN connections, maximizing throughput and minimizing latency. The transit gateway supports ECMP.

Let's examine why other options are less suitable:

B: Static routing is less flexible and requires manual updates when network changes occur. This is not ideal for a rapidly growing company.

C: VPN acceleration improves the performance of a single VPN connection but does not address the fundamental limitation of single-connection throughput. It might provide marginal improvements but won't significantly scale the bandwidth.

D: This option introduces a single point of failure (the EC2 instance) and requires managing a software-based VPN appliance. It is also more complex and less secure than using AWS managed Site-to-Site VPNs. Furthermore, routing this traffic over the internet is generally less secure than using a dedicated VPN connection.

In summary, option A provides the most scalable, highly available, and manageable solution for increasing

VPN throughput while maintaining security by leveraging multiple BGP-based VPN connections and ECMP routing with a Transit Gateway.

Relevant AWS documentation:

AWS Site-to-Site VPN: <https://aws.amazon.com/vpn/>

AWS Transit Gateway: <https://aws.amazon.com/transit-gateway/>

BGP Support for AWS VPN: https://docs.aws.amazon.com/vpn/latest/s2svpn/VPN_Routing.html

Question: 106

CertyIQ

A company uses Amazon Route 53 to host a public hosted zone for example.com. A network engineer recently reduced the TTL on several records to 60 seconds. The network engineer wants to assess whether the change has increased the number of queries to Route 53 beyond the expected levels that the company identified before the change. The network engineer must obtain the number of queries that have been made to the example.com public hosted zone.

Which solution will provide this information?

- A. Create a new trail in AWS CloudTrail to include Route 53 data events. Send logs to Amazon CloudWatch Logs. Set up a CloudWatch metric filter to count the number of queries and create graphs.
- B. Use Amazon CloudWatch to access the AWS/Route 53 namespace and to check the DNSQueries metric for the public hosted zone.
- C. Use Amazon CloudWatch to access the AWS/Route 53 Resolver namespace and to check the InboundQueryVolume metric for a specific endpoint.
- D. Configure logging to Amazon CloudWatch for the public hosted zone. Set up a CloudWatch metric filter to count the number of queries and create graphs.

Answer: B

Explanation:

The correct answer is B because Amazon CloudWatch natively provides metrics for Route 53 hosted zones, specifically the DNSQueries metric, within the AWS/Route53 namespace. This metric directly tracks the number of DNS queries received by the hosted zone. This eliminates the need for custom logging, metric filters, or CloudTrail configurations, simplifying the monitoring process. The DNSQueries metric provides an immediate and readily available means to assess query volume changes following the TTL reduction.

Option A is incorrect because while CloudTrail captures API calls, it doesn't directly log DNS query volume. Configuring CloudTrail for data events and then processing logs through CloudWatch Logs and metric filters would be an unnecessarily complex and less efficient solution compared to leveraging the built-in CloudWatch metrics for Route 53.

Option C is incorrect because the AWS/Route53 Resolver namespace and InboundQueryVolume metric are related to Route 53 Resolver endpoints, which handle DNS queries originating from within a VPC, not queries to a public hosted zone.

Option D is incorrect because Route 53 does not directly offer "logging to Amazon CloudWatch" for hosted zones in the way suggested. While you can indirectly achieve similar results through custom solutions, the built-in DNSQueries metric offers a direct and simpler approach.

In summary, directly using the DNSQueries metric in the AWS/Route53 namespace in CloudWatch is the most efficient and readily available method to monitor the number of queries to a public hosted zone in Route 53, aligning with the objective of assessing the impact of the TTL change.

Refer to the official AWS documentation for more information on Route 53 metrics in CloudWatch:

Question: 107

A company is establishing connectivity between its on-premises site and an existing VPC on AWS to meet a new security requirement. According to the new requirement, all public DNS queries must use an on-premises DNS security solution. The company's security team has allowed an exception for the AWS service endpoints because the company is using VPC endpoints to access AWS services.

Which combination of steps should a network engineer take to configure the architecture to meet these requirements? (Choose three.)

- A. Create a system rule for the domain name "." (dot) with a target IP address of the on-premises DNS security solution.
- B. Create a new DHCP options set that provides the IP address of the on-premises DNS security solution. Update the VPC to use this new DHCP options set.
- C. Create an Amazon Route 53 Resolver inbound endpoint. Associate this endpoint with the VPC.
- D. Create an Amazon Route 53 Resolver outbound endpoint. Associate this endpoint with the VPC.
- E. Create a system rule for the domain name amazonaws.com.
- F. Create a forwarding rule for the domain name "." (dot) with a target IP address of the on-premises DNS security solution.

Answer: DEF

Explanation:

Let's break down why the correct answer is D, E, and F:

D. Create an Amazon Route 53 Resolver outbound endpoint. Associate this endpoint with the VPC. This is the cornerstone of the solution. The Route 53 Resolver outbound endpoint allows DNS queries originating from the VPC to be forwarded to on-premises DNS servers. Without this, DNS queries from EC2 instances within the VPC would resolve using the default AWS DNS resolver. The outbound endpoint acts as a bridge, directing traffic towards your on-premises DNS infrastructure.

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver.html>

E. Create a system rule for the domain name amazonaws.com. The exception for AWS service endpoints is crucial. The company is using VPC endpoints, which mean traffic destined for AWS services like S3 or DynamoDB remains within the AWS network. A system rule that directly forwards amazonaws.com traffic ensures that resolution of the VPC endpoint URLs (which contain amazonaws.com) is handled by AWS's DNS infrastructure and does not get sent to the on-premises DNS solution, respecting the company's exception. By forwarding only amazonaws.com, internal services use the existing AWS resolvers by default.

F. Create a forwarding rule for the domain name "." (dot) with a target IP address of the on-premises DNS security solution. This forces all other public DNS queries to use the on-premises DNS security solution. The "." (dot) represents the root domain, so any DNS request that doesn't match other specific rules (like the amazonaws.com rule) gets sent to the specified on-premises DNS server. This satisfies the requirement that all public DNS queries (excluding AWS services) go through the security solution.

Why the other options are incorrect:

A. Create a system rule for the domain name "." (dot) with a target IP address of the on-premises DNS security solution. System rules cannot be used to resolve on-premises DNS solutions. System rules manage specific system behavior.

B. Create a new DHCP options set that provides the IP address of the on-premises DNS security solution.

Update the VPC to use this new DHCP options set. DHCP options sets control the DNS servers assigned to EC2 instances within the VPC. While this influences DNS resolution, it doesn't integrate with Route 53 Resolver or provide the fine-grained control needed to bypass the on-premises DNS for amazonaws.com and therefore is not suitable to fully meet the requirements. It would also change the DNS for the instances using the VPC endpoints, which is against the requirements.

C. Create an Amazon Route 53 Resolver inbound endpoint. Associate this endpoint with the VPC. An inbound endpoint is used to allow on-premises systems to resolve DNS records within the VPC, effectively allowing on-premises to query private Route 53 hosted zones in AWS. It's the opposite of what's needed in this scenario, which requires directing VPC DNS queries to on-premises.

Question: 108

CertyIQ

A network engineer is designing the DNS architecture for a new AWS environment. The environment must be able to resolve DNS names of endpoints on premises, and the on-premises systems must be able to resolve the names of AWS endpoints. The DNS architecture must give individual accounts the ability to manage subdomains.

The network engineer needs to create a single set of rules that will work across multiple accounts to control this behavior. In addition, the network engineer must use AWS native services whenever possible.

Which combination of steps should the network engineer take to meet these requirements? (Choose three.)

- A. Create an Amazon Route 53 private hosted zone for the overall cloud domain. Plan to create subdomains that align to other AWS accounts that are associated with the central Route 53 private hosted zone.
- B. Create AWS Directory Service for Microsoft Active Directory server endpoints in the central AWS account that hosts the private hosted zone for the overall cloud domain. Create a conditional forwarding rule in Microsoft Active Directory DNS to forward traffic to a DNS resolver endpoint on premises. Create another rule to forward traffic between subdomains to the VPC resolver.
- C. Create Amazon Route 53 Resolver inbound and outbound endpoints in the central AWS account that hosts the private hosted zone for the overall cloud domain. Create a forwarding rule to forward traffic to a DNS resolver endpoint on premises. Create another rule to forward traffic between subdomains to the Resolver inbound endpoint.
- D. Ensure that networking exists between the other accounts and the central account so that traffic can reach the AWS Directory Service for Microsoft Active Directory DNS endpoints.
- E. Ensure that networking exists between the other accounts and the central account so that traffic can reach the Amazon Route 53 Resolver endpoints.
- F. Share the Amazon Route 53 Resolver rules between accounts by using AWS Resource Access Manager (AWS RAM). Ensure that networking exists between the other accounts and the central account so that traffic can reach the Route 53 Resolver endpoints.

Answer: ACF

Explanation:

The correct answer is ACF. Here's a detailed justification:

A. Create an Amazon Route 53 private hosted zone...: This is necessary to manage DNS records for your AWS environment internally. Creating subdomains aligned with other AWS accounts allows for decentralized management while maintaining a consistent overall domain structure. This facilitates individual account ownership of their respective subdomains.

C. Create Amazon Route 53 Resolver inbound and outbound endpoints...: This is essential for hybrid DNS resolution. Inbound endpoints allow on-premises systems to resolve AWS endpoint names, and outbound endpoints enable AWS systems to resolve on-premises endpoint names. A forwarding rule to the on-premises DNS resolver establishes the connection for name resolution.

F. Share the Amazon Route 53 Resolver rules...: AWS Resource Access Manager (RAM) allows you to share Route 53 Resolver rules across multiple AWS accounts. This is critical for centralized management of DNS forwarding rules, satisfying the requirement for a single set of rules applicable across accounts. Networking

between accounts and the central account is mandatory for traffic to reach the Route 53 Resolver endpoints for successful DNS resolution.

Why other options are incorrect:

B. Create AWS Directory Service for Microsoft Active Directory server endpoints...: While AD can handle DNS, it's not the ideal AWS-native solution for the problem and complicates setup. The question specifically asks for the use of AWS native services. Directory service setup is also more complex.

D. Ensure that networking exists between the other accounts and the central account so that traffic can reach the AWS Directory Service for Microsoft Active Directory DNS endpoints. This is relevant only if choosing Active Directory, which is not ideal.

E. Ensure that networking exists between the other accounts and the central account so that traffic can reach the Amazon Route 53 Resolver endpoints. This is a valid condition for the solution, but on its own, it is insufficient and does not constitute a complete approach. This option needs to be used in conjunction with the sharing of Resolver rules.

Authoritative links:

Amazon Route 53 Resolver: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver.html>

AWS Resource Access Manager (RAM): <https://docs.aws.amazon.com/ram/latest/userguide/what-is.html>

Route 53 Private Hosted Zones: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/private-hosted-zones.html>

Question: 109

CertyIQ

A company wants to migrate its DNS registrar and DNS hosting to Amazon Route 53. The company website receives tens of thousands of visits each day, and the company's current DNS provider cannot keep up. The company wants to migrate as quickly as possible but cannot tolerate any downtime.

Which solution will meet these requirements?

- A. Transfer the domain name to Route 53. Create a Route 53 private hosted zone, and copy all the existing DNS records. Update the name servers on the domain to use the name servers that are specified in the newly created private hosted zone.
- B. Copy all DNS records from the existing DNS servers to a Route 53 private hosted zone. Update the name servers with the existing registrar to use the private hosted zone name servers. Transfer the domain name to Route 53. Ensure that all the changes have propagated.
- C. Transfer the domain name to Route 53. Create a Route 53 public hosted zone, and copy all the existing DNS records. Set the TTL value on each record to 1 second. Update the name servers on the domain to use the name servers that are specified in the newly created public hosted zone.
- D. Copy all DNS records from the existing DNS servers to a Route 53 public hosted zone. Update the name servers with the existing registrar to use the Route 53 name servers for the hosted zone. When the changes have propagated, perform a domain name transfer to Route 53.

Answer: D

Explanation:

Here's a detailed justification for why option D is the correct solution:

Option D provides a seamless and downtime-free migration strategy leveraging Route 53's capabilities and DNS propagation principles.

First, copying all DNS records from the existing provider to a Route 53 public hosted zone ensures that the records are readily available in Route 53. This eliminates the potential delay of creating records from scratch during the critical cutover phase.

The key to zero downtime is updating the name servers at the current registrar to point to the Route 53 name servers. This action doesn't immediately cut over traffic; it allows for a gradual shift as the updated DNS records propagate across the internet. DNS propagation involves DNS resolvers caching and updating their information, and this process takes time.

During this propagation period, some users will be directed to the old DNS servers, while others will be directed to Route 53. Since the records are identical in both locations, users experience no service disruption.

Once the changes have fully propagated (verifiable using tools like dig or online DNS lookup services), the domain name transfer to Route 53 can be performed. This transfer process primarily involves transferring the administrative control of the domain, and because the DNS resolution is already handled by Route 53, the transfer itself doesn't impact website availability.

Options A and B are incorrect because they involve using a private hosted zone. Private hosted zones are for internal resources within a VPC, not for public internet-facing websites.

Option C suggests drastically reducing the TTL (Time To Live) to 1 second. While this would speed up propagation, it significantly increases the load on DNS servers, potentially causing performance issues. Furthermore, a 1-second TTL may not be honored by all resolvers.

Therefore, option D offers the safest and most efficient way to migrate DNS to Route 53 without any downtime. It utilizes the standard DNS propagation mechanism to create a smooth transition.

Relevant Links:

AWS Route 53 Documentation - Migrating DNS Service to Amazon Route 53:

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/MigratingDNS.html>

AWS Route 53 FAQs: <https://aws.amazon.com/route53/faqs/>

Understanding DNS TTL: <https://www.cloudflare.com/learning/dns/dns-records/understanding-dns-ttl/>

Question: 110

CertyIQ

A company has an AWS account with four VPCs in the us-east-1 Region. The VPCs consist of a development VPC and three production VPCs that host various workloads.

The company has extended its on-premises data center to AWS with AWS Direct Connect by using a Direct Connect gateway. The company now wants to establish connectivity to its production VPCs and development VPC from on premises. The production VPCs are allowed to route data to each other. However, the development VPC must be isolated from the production VPCs. No data can flow between the development VPC and the production VPCs.

In preparation to implement this solution, a network engineer creates a transit gateway with a single transit gateway route table. Default route table association and default route table propagation are turned off. The network engineer attaches the production VPCs, the development VPC, and the Direct Connect gateway to the transit gateway. For each VPC route table, the network engineer adds a route to 0.0.0.0/0 with the transit gateway as the next destination.

Which combination of steps should the network engineer take next to complete this solution? (Choose three.)

A. Associate the production VPC attachments with the existing transit gateway route table. Propagate the routes from these attachments.

B. Associate all the attachments with the existing transit gateway route table. Propagate the routes from these attachments.

C. Associate the Direct Connect gateway attachment with the existing transit gateway route table. Propagate the Direct Connect gateway attachment to this route table.

D. Change the security group inbound rules on the existing transit gateway network interfaces in the development VPC to allow connections to and from the on-premises CIDR range only.

E. Create a new transit gateway route table. Associate the new route table with the development VPC

attachment. Propagate the Direct Connect gateway and development VPC attachment to the new route table.
F. Create a new transit gateway with default route table association and default route table propagation turned on. Attach the Direct Connect gateway and development VPC to the new transit gateway.

Answer: ACE

Explanation:

The correct answer is ACE. Here's why:

A. Associate the production VPC attachments with the existing transit gateway route table. Propagate the routes from these attachments. This step is crucial for enabling connectivity between the production VPCs and the transit gateway. By associating the attachments with the route table and propagating routes, the transit gateway learns about the CIDR blocks of the production VPCs. This allows routing between the production VPCs via the transit gateway.

C. Associate the Direct Connect gateway attachment with the existing transit gateway route table. Propagate the Direct Connect gateway attachment to this route table. This step allows the production VPCs to access the on-premises network. By associating the Direct Connect gateway attachment and propagating its routes, the transit gateway learns about the on-premises network CIDR blocks. Traffic from the production VPCs destined for on-premises will then be routed through the Direct Connect gateway.

E. Create a new transit gateway route table. Associate the new route table with the development VPC attachment. Propagate the Direct Connect gateway and development VPC attachment to the new route table. This step isolates the development VPC from the production VPCs. By creating a separate route table and associating the development VPC attachment with it, you ensure that the development VPC only has routes to the on-premises network (via the propagated Direct Connect gateway) and itself. Critically, because the production VPCs are associated with a different route table, no routes between the development VPC and production VPCs exist in either route table.

Why other options are incorrect:

B. Associating all attachments with the same route table defeats the purpose of isolation. The production and development VPCs need to have separate route tables if you're creating isolation using the Transit Gateway.

D. Security groups are applied at the instance level and are not relevant to routing decisions at the transit gateway level. Security groups are not a replacement for routing.

F. This is incorrect, as using an additional transit gateway adds unnecessary complexity and cost. The problem specifically calls for a single transit gateway. The question can be solved using a single transit gateway and multiple route tables.

Supporting Documentation:

AWS Transit Gateway: <https://aws.amazon.com/transit-gateway/>

Transit Gateway Route Tables: <https://docs.aws.amazon.com/vpc/latest/tgw/tgw-route-tables.html>

Question: 111

CertyIQ

A network engineer needs to provide dual-stack connectivity between a company's office location and an AWS account. The company's on-premises router supports dual-stack connectivity, and the VPC has been configured with dual-stack support. The company has set up two AWS Direct Connect connections to the office location. This connectivity must be highly available and must be reliable for latency-sensitive traffic.

Which solutions will meet these requirements? (Choose two.)

A. Configure a single private VIF on each Direct Connect connection. Add both IPv4 and IPv6 peering to each private VIF. Configure the on-premises equipment with the AWS provided BGP neighbors to advertise IPv4

routes on the IPv4 peering and IPv6 routes on the IPv6 peering. Enable Bidirectional Forwarding Detection (BFD) on all peering sessions.

B. Configure two private VIFs on each Direct Connect connection: one private VIF with the IPv4 address family and one private VIF with the IPv6 address family. Configure the on-premises equipment with the AWS provided BGP neighbors to advertise IPv4 routes on the IPv4 peering and IPv6 routes on the IPv6 peering. Enable Bidirectional Forwarding Detection (BFD) on all peering sessions.

C. Configure a single private VIF and IPv4 peering on each Direct Connect connection. Configure the on-premises equipment with this peering to advertise the IPv6 routes in the same BGP neighbor configuration. Enable Bidirectional Forwarding Detection (BFD) on all peering sessions.

D. Configure two private VIFs on each Direct Connect connection: one private VIF with the IPv4 address family and one private VIF with the IPv6 address family. Configure the on-premises equipment with the AWS provided BGP neighbors to advertise all IPv4 routes and IPv6 routes on all peering sessions. Keep the Bidirectional Forwarding Detection (BFD) configuration unchanged.

E. Configure two private VIFs on each Direct Connect connection: one private VIF with the IPv4 address family and one private VIF with the IPv6 address family. Configure the on-premises equipment with the AWS provided BGP neighbors to advertise IPv4 routes on the IPv4 peering and IPv6 routes on the IPv6 peering. Reduce the BGP hello timer to 5 seconds on both the on-premises equipment and the Direct Connect configuration.

Answer: AB

Explanation:

The correct answer is A and B. Here's why:

Option A:

Single Private VIF with Dual-Stack Peering: This is a valid and recommended approach for dual-stack connectivity. Using a single VIF simplifies configuration and management. By configuring both IPv4 and IPv6 peering on the same VIF, you can advertise and receive routes for both address families through a single BGP session.

BGP Advertisement: Advertising IPv4 routes on the IPv4 peering and IPv6 routes on the IPv6 peering is essential for establishing bidirectional routing between the on-premises network and the VPC. This ensures that traffic destined for both IPv4 and IPv6 addresses can reach the appropriate destination.

Bidirectional Forwarding Detection (BFD): Enabling BFD on all peering sessions significantly improves failure detection and failover times. BFD provides rapid detection of link failures, allowing for quick rerouting of traffic to the backup Direct Connect connection. This addresses the requirement for high availability and reliable, latency-sensitive traffic.

Option B:

Separate Private VIFs for IPv4 and IPv6: This is another valid approach for dual-stack connectivity. While using a single VIF (Option A) is generally preferred for simplicity, using separate VIFs for IPv4 and IPv6 provides clear separation and can simplify troubleshooting in some scenarios. This approach involves creating two private VIFs on each Direct Connect link: one configured with the IPv4 address family and the other with the IPv6 address family.

BGP Advertisement: Similar to Option A, configuring the on-premises equipment with the AWS-provided BGP neighbors to advertise IPv4 routes on the IPv4 peering and IPv6 routes on the IPv6 peering is crucial for establishing bidirectional routing.

Bidirectional Forwarding Detection (BFD): As with Option A, enabling BFD on all peering sessions ensures rapid failure detection and failover, meeting the requirements for high availability and reliable, latency-sensitive traffic.

Why the other options are incorrect:

Option C: Advertising IPv6 routes in the same BGP neighbor configuration for IPv4 peering is incorrect. IPv6 routes should be advertised in the IPv6 peering session.

Option D: Advertising all IPv4 and IPv6 routes on all peering sessions is redundant and can lead to routing

issues. Each peering session should advertise routes for its specific address family (IPv4 or IPv6). Leaving BFD configuration unchanged is also vague; BFD must be correctly configured and enabled.

Option E: While reducing the BGP hello timer can speed up failure detection, it is not as effective or reliable as using BFD. Also, excessively aggressive timer settings can lead to instability and increased network overhead. BFD is specifically designed for rapid failure detection and is the preferred method.

Supporting Documentation:

AWS Direct Connect FAQs: <https://aws.amazon.com/directconnect/faqs/> (Search for "dual-stack")

AWS Direct Connect Routing Policies:

https://docs.aws.amazon.com/directconnect/latest/UserGuide/routing_policies.html

Bidirectional Forwarding Detection (BFD): https://en.wikipedia.org/wiki/Bidirectional_Forwarding_Detection

Question: 112

CertyIQ

A company recently started using AWS Client VPN to give its remote users the ability to access resources in multiple peered VPCs and resources in the company's on-premises data center. The Client VPN endpoint route table has a single entry of 0.0.0.0/0. The Client VPN endpoint is using a new security group that has no inbound rules and a single outbound rule that allows all traffic to 0.0.0.0/0.

Multiple remote users report that web search results are showing incorrect geographic location information for the users.

Which combination of steps should a network engineer take to resolve this issue with the LEAST amount of service interruption? (Choose three.)

- A. Switch users to AWS Site-to-Site VPNs.
- B. Enable the split-tunnel option on the Client VPN endpoint.
- C. Add routes for the peered VPCs and for the on-premises data center to the Client VPN route table.
- D. Remove the 0.0.0.0/0 outbound rule from the security group that the Client VPN endpoint uses.
- E. Delete and recreate the Client VPN endpoint in a different VPC.
- F. Remove the 0.0.0.0/0 entry from the Client VPN endpoint route table.

Answer: BCF

Explanation:

The issue of incorrect geographic location arises because all user traffic, including internet traffic, is being routed through the Client VPN endpoint. This makes it appear as if users are connecting from the AWS region where the Client VPN endpoint is located. To resolve this with minimal disruption, we need to ensure that only traffic destined for the company's internal resources goes through the VPN.

B. Enable the split-tunnel option on the Client VPN endpoint: Split-tunneling allows only traffic destined for specific networks (in this case, the peered VPCs and on-premises network) to be routed through the VPN. All other traffic, including internet traffic, will be routed directly to the internet using the user's local network connection. This addresses the geographic location issue by allowing internet traffic to originate from the user's actual location. <https://docs.aws.amazon.com/vpn/latest/clientvpn-admin/split-tunnel-vpn.html>

C. Add routes for the peered VPCs and for the on-premises data center to the Client VPN route table: After enabling split-tunnel, the Client VPN endpoint needs to know which networks should be routed through the VPN connection. Adding specific routes for the peered VPCs and on-premises data center ensures that traffic intended for these resources is properly routed. Without these routes, even with split-tunneling enabled, the VPN won't know to direct the traffic, and users won't be able to access those resources.

<https://docs.aws.amazon.com/vpn/latest/clientvpn-admin/cvpn-working.html#cvpn-route-tables>

F. Remove the 0.0.0.0/0 entry from the Client VPN endpoint route table: The original 0.0.0.0/0 route is the cause of the problem. This route directs all traffic through the VPN. By removing it, we prevent the VPN from becoming the default gateway for all user traffic. This is crucial after enabling split-tunneling to ensure that only the specified routes are handled by the VPN. The combination of split tunneling and specific routes will then route traffic only to the organization's network and let normal internet traffic go directly.

Options A and E are incorrect because they involve replacing or recreating infrastructure, leading to more significant service interruption. Option D is incorrect because the outbound rule on the security group is generally needed to allow traffic to flow from the Client VPN to the destination networks; removing it would prevent all outbound traffic, even to the company's resources.

Question: 113

CertyIQ

A company has set up hybrid connectivity between its VPCs and its on-premises data center. The company has the `on-premises.example.com` subdomain configured at its DNS server in the on-premises data center. The company is using the `aws.example.com` subdomain for workloads that run on AWS across different VPCs and accounts. Resources in both environments can access each other by using IP addresses. The company wants workloads in the VPCs to be able to access resources on premises by using the `on-premises.example.com` DNS names.

Which solution will meet these requirements with MINIMUM management of resources?

A. Create an Amazon Route 53 Resolver outbound endpoint. Configure a Resolver rule that conditionally forwards DNS queries for `on-premises.example.com` to the on-premises DNS server. Associate the rule with the VPCs.

B. Create an Amazon Route 53 Resolver inbound endpoint and a Resolver outbound endpoint. Configure a Resolver rule that conditionally forwards DNS queries for `on-premises.example.com` to the on-premises DNS server. Associate the rule with the VPCs.

C. Launch an Amazon EC2 instance. Install and configure BIND software to conditionally forward DNS queries for `on-premises.example.com` to the on-premises DNS server. Configure the EC2 instance's IP address as a custom DNS server in each VPC.

D. Launch an Amazon EC2 instance in each VPC. Install and configure BIND software to conditionally forward DNS queries for `on-premises.example.com` to the on-premises DNS server. Configure the EC2 instance's IP address as a custom DNS server in each VPC.

Answer: A

Explanation:

The correct answer is A because it provides the most managed and scalable solution for hybrid DNS resolution using Route 53 Resolver.

Route 53 Resolver Outbound Endpoints and Resolver Rules are specifically designed to handle conditional forwarding of DNS queries to on-premises DNS servers. An outbound endpoint allows VPC-based resources to resolve domain names hosted on-premises. A Resolver rule is then configured to specify that queries for `on-premises.example.com` should be forwarded to the IP address(es) of the on-premises DNS server. This provides centralized control and management of DNS resolution for on-premises resources. Associating the rule with the VPCs ensures that all VPCs can resolve the on-premises domain.

Option B, creating both inbound and outbound endpoints, is unnecessary. An inbound endpoint is used when you want on-premises resources to resolve AWS resources using private DNS. Since the question specifies that AWS resources need to resolve on-premises resources, an inbound endpoint is not required.

Options C and D involve deploying and managing EC2 instances with BIND software. This approach introduces operational overhead, including patching, scaling, and maintenance of the EC2 instances and the BIND software. It violates the principle of "minimum management," as Route 53 Resolver is a fully managed service. Furthermore, option D suggests deploying an EC2 instance in each VPC, which drastically increases

complexity.

Therefore, using Route 53 Resolver's outbound endpoint and rules simplifies hybrid DNS resolution and minimizes management overhead.

Refer to the following documentation for further details:

Amazon Route 53 Resolver: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver.html>

Creating Resolver Endpoints: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver-endpoints-creating.html>

Creating Resolver Rules: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver-rules-creating.html>

Question: 114

CertyIQ

A company is in the early stage of AWS Cloud adoption. The company has an application that is running in an on-premises data center in Asia. The company needs to deploy new applications in the us-east-1 Region. The applications in the cloud need connectivity to the on-premises data center.

The company needs to set up a communication channel between AWS and the data center. The solution must improve latency, minimize the possibility of performance impact from transcontinental routing over the public internet, and encrypt data in transit.

Which solution will meet these requirements in the LEAST amount of time?

- A. Create an AWS Site-to-Site VPN connection with acceleration turned on. Create a virtual private gateway. Attach the Site-to-Site VPN connection to the virtual private gateway. Attach the virtual private gateway to the VPC where the applications will be deployed.
- B. Create an AWS Site-to-Site VPN connection with acceleration turned on. Create a transit gateway. Attach the Site-to-Site VPN connection to the transit gateway. Create a transit gateway attachment to the VPC where the applications will be deployed.
- C. Create an AWS Direct Connect connection. Create a virtual private gateway. Create a public VIF and a private VIF that use the virtual private gateway. Create an AWS Site-to-Site VPN connection over the public VIF.
- D. Create an AWS Site-to-Site VPN connection with acceleration turned off. Create a transit gateway. Attach the Site-to-Site VPN connection to the transit gateway. Create a transit gateway attachment to the VPC where the applications will be deployed.

Answer: B

Explanation:

Here's a detailed justification for why option B is the best solution for establishing a secure, low-latency connection between an on-premises data center and AWS in the least amount of time, considering the company's early stage of cloud adoption.

The requirement emphasizes minimizing latency and performance impact due to transcontinental routing, along with data encryption in transit. AWS Site-to-Site VPN provides encryption in transit. Acceleration improves performance. Transit Gateway acts as a central hub, simplifying connectivity between the VPN and the VPC in the us-east-1 region.

Option A utilizes a Virtual Private Gateway (VGW). While it works for basic VPN connectivity, it lacks the scalability and centralized management capabilities of Transit Gateway, especially when the company intends to expand its cloud footprint in the future. Transit Gateway facilitates hub-and-spoke network topology, which offers more centralized management and simplifies routing as the company expands its cloud infrastructure.

Option C introduces AWS Direct Connect, which offers dedicated network connections for lower latency and higher bandwidth. However, setting up Direct Connect involves a longer provisioning time than Site-to-Site

VPN, making it unsuitable for immediate needs. It also involves more complexity and cost. Adding a Site-to-Site VPN over the public VIF negates the advantages of Direct Connect and is unnecessarily complex.

Option D is similar to Option B but turns off acceleration. This will negatively impact performance by not leveraging the AWS Global Accelerator feature.

Therefore, option B provides the fastest path to establishing a secure, accelerated, and manageable connection using Site-to-Site VPN and Transit Gateway. Transit Gateway's simplified connectivity makes it favorable for environments likely to expand.

Authoritative Links:

AWS Site-to-Site VPN: <https://aws.amazon.com/vpn/>

AWS Transit Gateway: <https://aws.amazon.com/transit-gateway/>

AWS Global Accelerator: <https://aws.amazon.com/global-accelerator/>

Question: 115

CertyIQ

A company is moving its record-keeping application to the AWS Cloud. All traffic between the company's on-premises data center and AWS must be encrypted at all times and at every transit device during the migration.

The application will reside across multiple Availability Zones in a single AWS Region. The application will use existing 10 Gbps AWS Direct Connect dedicated connections with a MACsec capable port. A network engineer must ensure that the Direct Connect connection is secured accordingly at every transit device.

The network engineer creates a Connection Key Name and Connectivity Association Key (CKN/CAK) pair for the MACsec secret key.

Which combination of additional steps should the network engineer take to meet the requirements? (Choose two.)

- A. Configure the on-premises router with the MACsec secret key.
- B. Update the connection's MACsec encryption mode to `must_encrypt`. Then associate the CKN/CAK pair with the connection.
- C. Update the connection's MACsec encryption mode to `should_encrypt`. Then associate the CKN/CAK pair with the connection.
- D. Associate the CKN/CAK pair with the connection. Then update the connection's MACsec encryption mode to `must_encrypt`.
- E. Associate the CKN/CAK pair with the connection. Then update the connection's MACsec encryption mode to `should_encrypt`.

Answer: AD

Explanation:

Here's a breakdown of why the correct answer is AD, and why the other options are not:

A. Configure the on-premises router with the MACsec secret key.

Justification: MACsec operates at Layer 2 (Data Link Layer). To secure the Direct Connect link, both ends of the connection (the AWS side and the on-premises side) must be configured with the same MACsec secret key (CKN/CAK). This involves configuring the on-premises router (or switch) that connects to the Direct Connect endpoint with the generated CKN/CAK pair. This step establishes the security association needed for MACsec encryption and decryption.

D. Associate the CKN/CAK pair with the connection. Then update the connection's MACsec encryption mode to `must_encrypt`.

Justification: After the CKN/CAK is created, it needs to be associated with the specific Direct Connect connection you want to secure. This association is done through the AWS console or CLI, linking the secret key to the Direct Connect. The `must_encrypt` mode enforces that all traffic traversing the Direct Connect link must be encrypted. If encryption fails for any reason, the link will not pass traffic. This meets the requirement of encrypting all traffic at all times. The order is crucial: you must associate the key before enforcing the `must_encrypt` mode.

Here's why the other options are incorrect:

B & C: `should_encrypt` mode would not fulfill the "all traffic must be encrypted" requirement. `should_encrypt` allows unencrypted traffic to pass if encryption fails.

E: Similar to options B and C, `should_encrypt` does not ensure all traffic is encrypted.

In summary: The on-premises router must be configured with the MACsec key, and the CKN/CAK pair must be associated with the Direct Connect connection, setting the encryption mode to `must_encrypt` to enforce encryption on all traffic.

Supporting Links:

AWS Direct Connect MACsec: <https://docs.aws.amazon.com/directconnect/latest/UserGuide/direct-connect-macsec.html>

Question: 116

CertyIQ

A network engineer is designing hybrid connectivity with AWS Direct Connect and AWS Transit Gateway. A transit gateway is attached to a Direct Connect gateway and 19 VPCs across different AWS accounts. Two new VPCs are being attached to the transit gateway. The IP address administrator has assigned 10.0.32.0/21 to the first VPC and 10.0.40.0/21 to the second VPC. The prefix list has one CIDR block remaining before the prefix list reaches the quota for the maximum number of entries.

What should the network engineer do to advertise the routes from AWS to on premises to meet these requirements?

- A. Add 10.0.32.0/21 and 10.0.40.0/21 to both AWS managed prefix lists.
- B. Add 10.0.32.0/21 and 10.0.40.0/21 to the allowed prefix list.
- C. Add 10.0.32.0/20 to both AWS managed prefix lists.
- D. Add 10.0.32.0/20 to the allowed prefix list.

Answer: D

Explanation:

Here's a detailed justification for why option D is the correct answer, along with supporting concepts and links:

The core issue is that the existing prefix list (likely associated with the Direct Connect gateway's allowed prefixes) is almost full. Adding both 10.0.32.0/21 and 10.0.40.0/21 would exceed the entry limit. Therefore, the network engineer needs to consolidate the prefixes.

Options A and C refer to "AWS managed prefix lists," which are not typically directly managed for Direct Connect gateway route advertisements. Allowed prefixes are configured for a DX gateway. While AWS can manage prefix lists for security groups and route tables, the context here is advertising routes from AWS to on-premises. Direct Connect Gateway (DXGW) uses allowed prefixes to control which prefixes can be advertised over the DX connection.

Option B is incorrect because it suggests adding both new CIDR blocks individually, which would exceed the prefix list limit.

Option D is the correct approach because it aggregates the two /21 CIDR blocks (10.0.32.0/21 and 10.0.40.0/21) into a single /20 CIDR block (10.0.32.0/20). This consolidation reduces the number of entries required in the allowed prefix list, staying within the quota. The CIDR block 10.0.32.0/20 encompasses both 10.0.32.0/21 (10.0.32.0 - 10.0.39.255) and 10.0.40.0/21 (10.0.40.0 - 10.0.47.255). By using a larger, encompassing CIDR, only one entry is needed. This solution efficiently solves the problem of advertising routes from AWS to on-premises while respecting the prefix list limit. Since this aggregation is done on the 'allowed prefix list' of the Direct Connect Gateway, the on-premise network can only see the summarised 10.0.32.0/20 and will forward traffic destined to either 10.0.32.0/21 or 10.0.40.0/21 to AWS.

Here are some links for further research:

AWS Direct Connect Gateways: <https://docs.aws.amazon.com/directconnect/latest/UserGuide/direct-connect-gateways.html>

Direct Connect Allowed Prefixes: <https://docs.aws.amazon.com/directconnect/latest/UserGuide/acceptable-prefixes.html>

Transit Gateway Route Tables: <https://docs.aws.amazon.com/vpc/latest/tgw/tgw-route-tables.html>

Question: 117

CertyIQ

Two companies are merging. The companies have a large AWS presence with multiple VPCs and are designing connectivity between their AWS networks. Both companies are using AWS Direct Connect with a Direct Connect gateway. Each company also has a transit gateway and multiple AWS Site-to-Site VPN connections from its transit gateway to on-premises resources. The new solution must optimize network visibility, throughput, logging, and monitoring.

Which solution will meet these requirements?

A. Configure a Site-to-Site VPN connection between each company's transit gateway to establish reachability between the respective networks. Configure VPC Flow Logs for all VPCs. Publish the flow logs to Amazon CloudWatch. Use VPC Reachability Analyzer to monitor connectivity.

B. Configure a Site-to-Site VPN connection between each company's transit gateway to establish reachability between the respective networks. Configure VPC Flow Logs for all VPCs. Publish the flow logs to Amazon CloudWatch. Use AWS Transit Gateway Network Manager to monitor the transit gateways and their respective connections.

C. Configure transit gateway peering between each company's transit gateway. Configure VPC Flow Logs for all VPCs. Publish the flow logs to Amazon CloudWatch. Use VPC Reachability Analyzer to monitor connectivity.

D. Configure transit gateway peering between each company's transit gateway. Configure VPC Flow Logs for all VPCs. Publish the flow logs to Amazon CloudWatch. Use AWS Transit Gateway Network Manager to monitor the transit gateways, their respective connections, and the transit gateway peering link.

Answer: D

Explanation:

The correct answer is **D**. Here's a detailed justification:

The requirements emphasize network visibility, throughput, logging, and monitoring after merging two AWS environments with existing Direct Connect, transit gateways, and Site-to-Site VPN connections.

Transit Gateway Peering: Transit gateway peering offers high-bandwidth, low-latency connectivity between the two companies' networks compared to Site-to-Site VPN, which is suitable for lower bandwidth and more resilient connectivity. TGW peering provides a direct, AWS-managed connection, optimizing throughput and reducing complexity.

VPC Flow Logs and CloudWatch: VPC Flow Logs provide detailed information about IP traffic going to, from,

and within VPCs. Publishing these logs to CloudWatch enables centralized logging, monitoring, and analysis of network traffic patterns across all VPCs.

AWS Transit Gateway Network Manager: This service offers centralized network management and monitoring capabilities for transit gateways and their connections, including Direct Connect, Site-to-Site VPN, and peering connections. It provides a visual representation of the network topology and helps identify potential issues. It is specifically designed to give you great visibility into your global network.

Option A is incorrect because using VPN connections between transit gateways is less performant than peering and doesn't effectively use the Direct Connect infrastructure. Additionally, VPC Reachability Analyzer focuses on reachability within VPCs, not overall network topology and health across transit gateways.

Option B is incorrect for similar reasons as option A in regard to VPN usage compared to peering.

Option C is incorrect because it uses VPC Reachability Analyzer, which is insufficient for end-to-end network monitoring of the interconnected transit gateways and their connections, including Direct Connect.

By using transit gateway peering, centralized flow logs in CloudWatch, and AWS Transit Gateway Network Manager, the merged organization achieves optimal throughput, comprehensive network visibility, centralized logging, and robust monitoring across their entire AWS infrastructure.

Authoritative Links:

AWS Transit Gateway: <https://aws.amazon.com/transit-gateway/>

AWS Transit Gateway Network Manager: <https://aws.amazon.com/blogs/networking-and-content-delivery/introducing-aws-transit-gateway-network-manager-global-network-visibility-and-centralized-management/>

VPC Flow Logs: <https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs.html>

Question: 118

CertyIQ

A company has a single VPC in the us-east-1 Region. The company is planning to set up a new VPC in the us-east-2 Region. The existing VPC has an AWS Site-to-Site VPN connection to the company's on-premises environment and uses a virtual private gateway.

A network engineer needs to implement a solution to establish connectivity between the existing VPC and the new VPC. The solution also must implement support for IPv6 for the new VPC. The company has new on-premises resources that need to connect to VPC resources by using IPv6 addresses.

Which solution will meet these requirements?

A. Create a new virtual private gateway in us-east-1. Attach the new virtual private gateway to the new VPC. Create two new Site-to-Site VPN connections to the new virtual private gateway with IPv4 and IPv6 support. Configure routing between the VPCs by using VPC peering.

B. Create a transit gateway in us-east-1 and in us-east-2. Attach the existing VPC and the new VPC to each transit gateway. Create a new Site-to-Site VPN connection to each transit gateway with IPv4 and IPv6 support. Configure transit gateway peering. Configure routing between the VPCs and the on-premises environment.

C. Create a new virtual private gateway in us-east-2. Attach the new virtual private gateway to the new VPC. Create two new Site-to-Site VPN connections to the new virtual private gateway with IPv4 and IPv6 support. Configure routing between the VPCs by using VPC peering.

D. Create a transit gateway in us-east-1. Attach the existing VPC and the new VPC to the transit gateway. Create two new Site-to-Site VPN connections to the transit gateway with IPv4 and IPv6 support. Configure transit gateway peering. Configure routing between the VPCs and the on-premises environment.

Answer: B

Explanation:

Here's a detailed justification for why option B is the correct solution, and why the other options are not suitable:

Option B: Create a transit gateway in us-east-1 and in us-east-2. Attach the existing VPC and the new VPC to each transit gateway. Create a new Site-to-Site VPN connection to each transit gateway with IPv4 and IPv6 support. Configure transit gateway peering. Configure routing between the VPCs and the on-premises environment.

This solution is the most appropriate because it leverages Transit Gateways (TGWs) to centrally manage connectivity between multiple VPCs and the on-premises environment, while fulfilling the IPv6 requirement.

Transit Gateway for VPC Connectivity: TGWs are designed to simplify network architecture by acting as a hub for interconnecting VPCs. This eliminates the need for complex peering relationships between individual VPCs.

IPv6 Support: TGWs inherently support IPv6. By establishing Site-to-Site VPN connections with IPv4 and IPv6 support to the TGW, on-premises resources can communicate with the VPCs using IPv6 addresses.

On-premises Connectivity: Connecting the on-premises environment to the TGW via VPN allows resources in the VPCs to access on-premises resources (and vice versa) using both IPv4 and IPv6.

Inter-Region Connectivity: Creating transit gateways in each region, and then peering those transit gateways, facilitates routing of traffic from the VPC in us-east-1 to the VPC in us-east-2, thereby enabling inter-region connectivity.

Why other options are incorrect:

Option A & C (VPC Peering with VPN): VPC Peering does not support overlapping CIDR blocks. Also, VPC Peering is a one-to-one connection, so managing connectivity between multiple VPCs and on-premises becomes complex. Moreover, VPC peering does not natively support transitive routing through the virtual private gateway from the on-premises environment, and while it does now support IPv6, using it in this scenario with VPN configuration for IPv6 will increase complexity compared to the cleaner TGW approach.

Option D (Single Transit Gateway): A single TGW in us-east-1 without peering to another TGW in us-east-2 would not facilitate connections between VPCs in both regions.

Authoritative Links:

AWS Transit Gateway: <https://aws.amazon.com/transit-gateway/>

AWS Site-to-Site VPN: <https://aws.amazon.com/vpn/>

VPC Peering: <https://docs.aws.amazon.com/vpc/latest/peering/what-is-vpc-peering.html>

Question: 119

CertyIQ

A network engineer is working on a private DNS design to integrate AWS workloads and on-premises resources. The AWS deployment consists of five VPCs in the eu-west-1 Region that connect to the on-premises network over AWS Direct Connect. The VPCs communicate with each other by using a transit gateway. Each VPC is associated with a private hosted zone that uses the aws.example.internal domain. The network engineer creates an Amazon Route 53 Resolver outbound endpoint in a shared services VPC and attaches the shared services VPC to the transit gateway.

The network engineer is implementing a solution for DNS resolution. Queries for hostnames that end with aws.example.internal must use the private hosted zone. Queries for hostnames that end with all other domains must be forwarded to a private on-premises DNS resolver.

Which solution will meet these requirements?

A. Add a forwarding rule for "*" that targets the on-premises server's DNS IP address. Add a system rule for aws.example.internal that targets Route 53 Resolver.

B. Add a forwarding rule for aws.example.internal that targets Route 53 Resolver. Add a system rule for "." that targets the Route 53 Resolver outbound endpoint.

- C. Add a forwarding rule for "*" that targets the Route 53 Resolver outbound endpoint.
- D. Add a forwarding rule for "." that targets the Route 53 Resolver outbound endpoint.

Answer: D

Explanation:

Here's a detailed justification for why option D is the correct answer:

The problem requires hybrid DNS resolution: internal AWS domains (aws.example.internal) should resolve using Route 53 Private Hosted Zones, while all other domains should resolve via the on-premises DNS server using Route 53 Resolver outbound endpoint.

Option D, "Add a forwarding rule for "." that targets the Route 53 Resolver outbound endpoint," is the correct approach. The dot (".") represents the root domain, essentially meaning all domains. By creating a forwarding rule for ".", any DNS query that doesn't have a more specific rule will be forwarded to the specified target. In this case, the outbound endpoint forwards the query to the on-premises DNS resolver. Because the private hosted zones are associated with the VPCs, queries for aws.example.internal will automatically be resolved locally within AWS using Route 53 Private Hosted Zones **before** being evaluated against the Resolver forwarding rules. DNS resolvers always prefer specific zone matches before generic forwarding rules. Therefore, queries to aws.example.internal will be served by private hosted zones in Route 53, and all other DNS queries will be forwarded to the on-premises DNS server via the outbound endpoint.

Option A is incorrect because Route 53 Resolver doesn't have "system rules." System rules are concepts related to some DNS server implementations, not Route 53 Resolver.

Option B is incorrect because it tries to create a forwarding rule for aws.example.internal to Route 53 Resolver, but this isn't necessary. When VPCs are associated with the private hosted zone for aws.example.internal, AWS automatically handles the DNS resolution for this domain within the AWS environment. You only need to specify the Route 53 Resolver outbound endpoint for the generic/default case. Additionally, creating a system rule for "." towards Route 53 Resolver would not forward to the on-premise DNS server in the first place and system rules are not available as mentioned before.

Option C is incorrect because it forwards all queries ("*") to the Route 53 Resolver outbound endpoint, meaning the on-premises DNS resolver would handle all queries, including those for aws.example.internal. This defeats the purpose of using Route 53 Private Hosted Zones for internal resolution within AWS.

In summary, Route 53 Resolver handles DNS resolution for both internal and external domains. Utilizing the wildcard "." as the domain target for the forwarding rule allows all non-locally resolved DNS queries to be forwarded towards the outbound endpoint.

Relevant Documentation:

What is Route 53 Resolver? <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver.html>

How DNS Resolvers Work

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver.html#resolver-concept-dns-query-path>

Creating Resolver Rules <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver-creating-rules.html>

Question: 120

CertyIQ

A global film production company uses the AWS Cloud to encode and store its video content before distribution. The company's three global offices are connected to the us-east-1 Region through AWS Site-to-Site VPN links that terminate on a transit gateway with BGP routing activated.

The company recently started to produce content at a higher resolution to support 8K streaming. The size of the content files has increased to three times the size of the content files from the previous format. Uploads of files to Amazon EC2 instances are taking 10 times longer than they did with the previous format.

Which actions should a network engineer recommend to reduce the upload times? (Choose two.)

- A. Create a second VPN tunnel from each office location to the transit gateway. Activate equal-cost multi-path (ECMP) routing.
- B. Modify the transit gateway to activate Jumbo MTU on the VPN tunnels to each office location.
- C. Replace the existing VPN tunnels with new tunnels that have acceleration activated.
- D. Upgrade each EC2 instance to a modern instance type. Activate Jumbo MTU in the operating system.
- E. Replace the existing VPN tunnels with new tunnels that have IGMP activated.

Answer: AC

Explanation:

The recommended actions to reduce upload times are creating a second VPN tunnel from each office with ECMP routing and replacing the existing VPN tunnels with accelerated tunnels.

A. Create a second VPN tunnel from each office location to the transit gateway. Activate equal-cost multi-path (ECMP) routing.

Adding a second VPN tunnel and enabling ECMP significantly increases the bandwidth available between each office and the AWS environment. ECMP allows traffic to be distributed across multiple paths with equal cost, effectively load balancing the uploads across the two VPN tunnels. This doubles the overall throughput, leading to faster upload times. Since the uploads are taking 10 times longer with 3 times the data size, increasing available bandwidth is a direct way to address the bottleneck.

C. Replace the existing VPN tunnels with new tunnels that have acceleration activated.

VPN acceleration techniques, often employing hardware acceleration or optimized software configurations, can substantially improve the performance of VPN connections. This is particularly crucial for handling the larger data sizes associated with the 8K content. Accelerated VPN tunnels reduce the overhead involved in encryption and decryption processes, resulting in faster data transfer rates and improved overall network performance. The slower uploads suggest that the VPNs themselves are a bottleneck, hence optimizing them.

Why other options are less suitable:

B. Modify the transit gateway to activate Jumbo MTU on the VPN tunnels to each office location. Jumbo MTU requires end-to-end support, including from the customer premises equipment. Lack of support would lead to fragmentation and increase latency, reducing throughput.

D. Upgrade each EC2 instance to a modern instance type. Activate Jumbo MTU in the operating system.

While upgrading EC2 instances could improve processing power, the primary bottleneck appears to be network bandwidth rather than EC2 instance performance, and therefore, Jumbo MTU changes on the instances will not resolve the issue.

E. Replace the existing VPN tunnels with new tunnels that have IGMP activated. IGMP (Internet Group Management Protocol) is used for multicast traffic, which is not relevant for the unicast uploads described in the scenario. IGMP is not relevant to increasing VPN bandwidth or throughput for single-source uploads.

Supporting Links:

AWS Transit Gateway: <https://aws.amazon.com/transit-gateway/>

AWS VPN: <https://aws.amazon.com/vpn/>

Equal-Cost Multi-Path (ECMP) Routing: https://en.wikipedia.org/wiki/Equal-cost_multi-path_routing

Question: 121

An application team for a startup company is deploying a new multi-tier application into the AWS Cloud. The application will be hosted on a fleet of Amazon EC2 instances that run in an Auto Scaling group behind a publicly accessible Network Load Balancer (NLB). The application requires the clients to work with UDP traffic and TCP traffic.

In the near term, the application will serve only users within the same geographic location. The application team plans to extend the application to a global audience and will move the deployment to multiple AWS Regions around the world to bring the application closer to the end users. The application team wants to use the new Regions to deploy new versions of the application and wants to be able to control the amount of traffic that each Region receives during these rollouts. In addition, the application team must minimize first-byte latency and jitter (randomized delay) for the end users.

How should the application team design the network architecture for the application to meet these requirements?

- A. Create an Amazon CloudFront distribution to align to each Regional deployment. Set the NLB for each Region as the origin for each CloudFront distribution. Use an Amazon Route 53 weighted routing policy to control traffic to the newer Regional deployments.
- B. Create an AWS Global Accelerator accelerator and listeners for the required ports. Configure endpoint groups for each Region. Configure a traffic dial for the endpoint groups to control traffic to the newer Regional deployments. Register the NLBs with the endpoint groups.
- C. Use Amazon S3 Transfer Acceleration for the application in each Region. Adjust the amount of traffic that each Region receives from the Transfer Acceleration endpoints to the Regional NLBs.
- D. Create an Amazon CloudFront distribution that includes an origin group. Set the NLB for each Region as the origins for the origin group. Use an Amazon Route 53 latency routing policy to control traffic to the new Regional deployments.

Answer: B

Explanation:

The correct answer is **B. Create an AWS Global Accelerator accelerator and listeners for the required ports. Configure endpoint groups for each Region. Configure a traffic dial for the endpoint groups to control traffic to the newer Regional deployments. Register the NLBs with the endpoint groups.**

Here's a detailed justification:

AWS Global Accelerator provides a static entry point (two static IPs) for the application. It directs user traffic to the optimal endpoint based on proximity, health, and configured traffic dials. This addresses the requirement to minimize first-byte latency and jitter since Global Accelerator uses the AWS global network to route traffic efficiently. Global Accelerator also supports both UDP and TCP traffic, fitting the application's requirements. <https://aws.amazon.com/global-accelerator/>

Endpoint groups are created for each AWS Region hosting the application and the NLBs are registered as endpoints within these groups. This allows for easy scaling and management of the regional deployments. The traffic dial feature of Global Accelerator perfectly matches the requirement to control the amount of traffic each Region receives during rollouts. The dial can be adjusted to send a percentage of traffic to specific regions, allowing for controlled A/B testing or phased deployments.

Option A is incorrect because CloudFront primarily excels at caching static content, which isn't applicable here. Using Route 53's weighted routing to CloudFront distributions doesn't minimize latency and jitter as effectively as Global Accelerator, which actively routes users to the nearest healthy endpoint using AWS's backbone network. Additionally, CloudFront is optimized for HTTP/HTTPS traffic, while the application needs both TCP and UDP support.

Option C incorrectly uses S3 Transfer Acceleration. S3 Transfer Acceleration is designed to accelerate

uploads and downloads of data to and from S3 buckets, which isn't related to the application's traffic routing needs.

Option D combines CloudFront with Route 53 latency-based routing, which still doesn't optimally address the UDP requirement and minimize latency/jitter as effectively as Global Accelerator for dynamic application traffic. Origin groups in CloudFront are used for failover, not controlled traffic distribution like traffic dials in Global Accelerator.

In summary, Global Accelerator is the best fit due to its ability to handle both TCP and UDP traffic, provide low-latency routing via the AWS global network, and control traffic distribution to different regions using traffic dials, thereby fulfilling all the requirements.

Question: 122

CertyIQ

A company is deploying a new stateless web application on AWS. The web application will run on Amazon EC2 instances in private subnets behind an Application Load Balancer. The EC2 instances are in an Auto Scaling group. The web application has a stateful management application for administration that will run on EC2 instances that are in a separate Auto Scaling group.

The company wants to access the management application by using the same URL as the web application, with a path prefix of /management. The protocol, hostname, and port number must be the same for the web application and the management application. Access to the management application must be restricted to the company's on-premises IP address space. An SSL/TLS certificate from AWS Certificate Manager (ACM) will protect the web application.

Which combination of steps should a network engineer take to meet these requirements? (Choose two.)

- A. Insert a rule for the load balancer HTTPS listener. Configure the rule to check the path-pattern condition type for the /management prefix and to check the source-ip condition type for the on-premises IP address space. Forward requests to the management application target group if there is a match. Edit the management application target group and enable stickiness.
- B. Modify the default rule for the load balancer HTTPS listener. Configure the rule to check the path-pattern condition type for the /management prefix and to check the source-ip condition type for the on-premises IP address space. Forward requests to the management application target group if there is not a match. Enable group-level stickiness in the rule attributes.
- C. Insert a rule for the load balancer HTTPS listener. Configure the rule to check the path-pattern condition type for the /management prefix and to check the X-Forwarded-For HTTP header for the on-premises IP address space. Forward requests to the management application target group if there is a match. Enable group-level stickiness in the rule attributes.
- D. Modify the default rule for the load balancer HTTPS listener. Configure the rule to check the path-pattern condition type for the /management prefix and to check the source-ip condition type for the on-premises IP address space. Forward requests to the web application target group if there is not a match.
- E. Forward all requests to the web application target group. Edit the web application target group and disable stickiness.

Answer: AE

Explanation:

The correct answer is AE. Here's a breakdown:

A. Insert a rule for the load balancer HTTPS listener. Configure the rule to check the path-pattern condition type for the /management prefix and to check the source-ip condition type for the on-premises IP address space. Forward requests to the management application target group if there is a match. Edit the management application target group and enable stickiness.

This option is correct because it uses the Application Load Balancer (ALB)'s listener rules to route traffic

based on both the URL path (/management) and the source IP address. The path-pattern condition type enables the ALB to inspect the incoming request's URL and match against the /management prefix. The source-ip condition type allows restricting access to the management application to only requests originating from the company's on-premises IP address space, which is a crucial security requirement. When both conditions are met, the ALB forwards the request to the management application target group. While the prompt specifies a stateful management application, enabling stickiness on the management application target group isn't necessary for the solution to work.

E. Forward all requests to the web application target group. Edit the web application target group and disable stickiness.

This option is correct as a supplementary step to ensure the web application functions as intended. By forwarding all requests to the web application target group, and more specifically, the default rule handles all the "non-/management" traffic, the web application's main function remains accessible. Disabling stickiness on the web application target group, depending on the application's design, can optimize load distribution and resource utilization. Since the question states the web application is stateless, stickiness is unnecessary.

Why other options are wrong:

B and D: These options modify the default rule which typically handles all other traffic. Instead, the requirement specifies that traffic destined for /management should be directed only if coming from a specific IP range. Modifying the default rule to handle this would incorrectly route all non /management traffic if not from the expected IP range. Furthermore, enabling group-level stickiness in the rule attributes is irrelevant here.

C: Using the X-Forwarded-For header is unreliable for IP address filtering. This header can be easily spoofed by malicious users. The source-ip condition directly checks the actual source IP address of the incoming connection.

Enabling stickiness on the management application target group is not required in this specific scenario. Although the question mentions the management application is stateful, the requirements do not explicitly state that session persistence is needed.

Option E is incorrect because it says to disable stickiness on the web application target group. However, the problem statement does not indicate that the web application is not stateful. The web application should maintain the stickiness or not depending on if it is stateful.

Supporting Documentation:

Application Load Balancer Listener Rules:

<https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-listeners.html>

Target Groups for your Application Load Balancers:

<https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-target-groups.html>

Question: 123

CertyIQ

A company deploys a software solution on Amazon EC2 instances that are in a cluster placement group. The solution's UI is a single HTML page. The HTML file size is 1,024 bytes. The software processes files that exceed 1,024 MB in size. The software shares files over the network to clients upon request. The files are shared with the Don't Fragment flag set. Elastic network interfaces of the EC2 instances are set up with jumbo frames.

The UI is always accessible from all allowed source IP addresses, regardless of whether the source IP addresses are within a VPC, on the internet, or on premises. However, clients sometimes do not receive files that they request because the files fail to travel successfully from the software to the clients.

Which options provide a possible root cause of these failures? (Choose two.)

A. The source IP addresses are from on-premises hosts that are routed over AWS Direct Connect.

- B. The source IP addresses are from on-premises hosts that are routed over AWS Site-to-Site VPN.
- C. The source IP addresses are from hosts that connect over the public internet.
- D. The security group of the EC2 instances does not allow ICMP traffic.
- E. The operating system of the EC2 instances does not support jumbo frames.

Answer: BC

Explanation:

The correct answer is BC. Here's why:

The problem highlights a situation where small packets (UI, 1KB) are reliably delivered, but large packets (files, >1GB) are not, despite jumbo frames being configured. The "Don't Fragment" (DF) flag is crucial. When the DF flag is set, a packet must be delivered end-to-end without fragmentation. If a router in the path cannot forward the packet without fragmentation (because the MTU is too small), it will drop the packet and, ideally, send an ICMP "Fragmentation Needed and DF Set" message back to the sender (the EC2 instance).

B. The source IP addresses are from on-premises hosts that are routed over AWS Site-to-Site VPN. AWS Site-to-Site VPN has an MTU of 1436 bytes, lower than the typical Ethernet MTU of 1500 bytes and significantly lower than a jumbo frame MTU (9001 bytes). If a large packet with the DF flag encounters a Site-to-Site VPN connection, the VPN gateway will need to fragment the packet to fit into the VPN tunnel's smaller MTU. Because the DF flag is set, the gateway cannot fragment the packet, and it drops the packet. This explains the failure for on-premises clients connected via VPN.

C. The source IP addresses are from hosts that connect over the public internet. The MTU over the public internet is not guaranteed. While many networks support an MTU of 1500 bytes, there's no assurance that all hops between the EC2 instance and a client on the internet will. Any intermediate router with a lower MTU will drop packets with the DF flag set. This is a common problem with path MTU discovery (PMTUD). PMTUD is the process where a host learns the smallest MTU along a network path. If PMTUD is broken, the source host might send packets larger than the smallest MTU along the path, leading to packet drops and connectivity issues.

A. The source IP addresses are from on-premises hosts that are routed over AWS Direct Connect. Direct Connect can support jumbo frames (MTU 9001), provided it is configured correctly end-to-end. If configured properly, Direct Connect is less likely to be the source of MTU issues than VPN or the internet. However, even with Direct Connect, misconfigurations in the on-premises network or along the Direct Connect path could still lead to smaller MTUs. It is still a possibility but less probable than B or C.

D. The security group of the EC2 instances does not allow ICMP traffic. While blocking ICMP traffic can break PMTUD, preventing the EC2 instance from learning about MTU limitations, it does not directly cause packet drops if a client doesn't receive a packet. The ICMP "Fragmentation Needed" message is sent back to the sender (the EC2 instance), not the client. Blocking ICMP makes troubleshooting more difficult, as the sender won't get information about the MTU issue, but the problem lies in the packet being too big for the path, not in the ICMP response. The underlying problem remains MTU mismatch.

E. The operating system of the EC2 instances does not support jumbo frames. If the OS does not support jumbo frames, the network interface would be configured with a standard MTU (e.g., 1500). The issue isn't that the EC2 instances can't handle large packets; they likely can. The problem is that some path in the network to the client cannot without fragmentation. The UI loads, indicating some network connectivity.

Authoritative Links:

AWS Site-to-Site VPN MTU: https://docs.aws.amazon.com/vpn/latest/s2svpn/VPN_Troubleshooting.html
(Search for "MTU")

AWS Direct Connect Jumbo Frames:
https://docs.aws.amazon.com/directconnect/latest/UserGuide/jumbo_frames_support.html

Question: 124

A company has users who work from home. The company wants to move these users to Amazon WorkSpaces for additional security visibility.

The company has deployed WorkSpaces in its own AWS account in VPC A. A network engineer decides to provide the security visibility by using two firewall appliances behind a Gateway Load Balancer (GWLB). The network engineer provisions another VPC, VPC B, in a separate account and deploys the two firewall appliances in separate Availability Zones.

What should the network engineer do to configure the network connectivity for this solution?

- A. Create a GWLB in VPC A with the firewall appliance instances as targets. Use the GWLB to create a GWLB endpoint. Add the AWS principal ARN of the WorkSpaces account to the principal allow list of the GWLB endpoint. In the WorkSpaces account, create a VPC endpoint and specify the service name that the AWS Management Console provides for the GWLB endpoint. Modify the route tables of VPC A to point the default route to the VPC endpoint.
- B. Create a GWLB in VPC B with the firewall appliance instances as targets. Use the GWLB to create a GWLB endpoint. Add the AWS principal ARN of the WorkSpaces account to the principal allow list of the GWLB endpoint. In the WorkSpaces account, create a VPC endpoint and specify the service name that the AWS Management Console provides for the GWLB endpoint. Modify the route tables of VPC A to point the default route to the GWLB endpoint.
- C. Create a GWLB in VPC B with the firewall appliance instances as targets. Use the GWLB to create a GWLB endpoint. Add the AWS principal ARN of the WorkSpaces account to the principal allow list of the GWLB endpoint. In the WorkSpaces account, create a VPC endpoint and specify the service name that the AWS Management Console provides for the GWLB endpoint. Modify the route tables of VPC A to point the WorkSpaces subnet to the VPC endpoint.
- D. Create a GWLB in VPC B with the firewall appliance instances as targets. Use the GWLB to create a GWLB endpoint. Add the AWS principal ARN of the account that contains the firewall appliances to the principal allow list of the GWLB endpoint. In the WorkSpaces account, create a VPC endpoint and specify the service name that the AWS Management Console provides for the GWLB endpoint. Modify the route tables of VPC A to point the default route to the VPC endpoint.

Answer: B**Explanation:**

The correct answer is B. Here's a detailed justification:

The requirement is to route traffic from WorkSpaces (VPC A) through firewall appliances in VPC B for security visibility. A Gateway Load Balancer (GWLB) and VPC endpoints facilitate this cross-account traffic inspection.

1. **GWLB Placement:** The GWLB must reside in VPC B, where the firewall appliances are located. It distributes traffic among the firewalls. Therefore, options A is incorrect.
2. **GWLB Endpoint Creation:** A GWLB endpoint allows services in other VPCs to access the GWLB. This endpoint is created after the GWLB is set up.
3. **Cross-Account Access:** To enable VPC A (WorkSpaces account) to access the GWLB endpoint, the AWS principal ARN of the WorkSpaces account must be added to the GWLB endpoint's principal allow list. This authorizes the WorkSpaces VPC to use the service. Option D is incorrect because it adds the firewall account.
4. **VPC Endpoint in WorkSpaces Account:** In the WorkSpaces account (VPC A), a VPC endpoint of type Interface is created. When creating the VPC endpoint, you specify the service name that the AWS Management Console provides for the GWLB endpoint. This connects the WorkSpaces VPC to the GWLB.
5. **Route Table Modification:** The route table in VPC A (specifically, the route table associated with the WorkSpaces subnet) needs to be modified to send all traffic (default route 0.0.0.0/0) to the VPC

endpoint. This ensures that all outbound traffic from the WorkSpaces is routed through the firewall appliances. Option C is incorrect because it specifies the WorkSpaces subnet instead of the default route.

In summary, Option B correctly outlines the steps to create a GWLB in VPC B, configure cross-account access using the AWS principal ARN of the WorkSpaces account, create a VPC endpoint in VPC A, and modify the route tables to route traffic through the VPC endpoint to the GWLB and ultimately to the firewall appliances.

Here are some authoritative links for further research:

AWS Gateway Load Balancer: <https://aws.amazon.com/elasticloadbalancing/gateway-load-balancer/>

AWS PrivateLink and VPC Endpoints: <https://aws.amazon.com/privatelink/>

Creating a Gateway Load Balancer Endpoint:

<https://docs.aws.amazon.com/elasticloadbalancing/latest/gateway/create-gateway-load-balancer-endpoint.html>

Question: 125

CertyIQ

A company plans to run a computationally intensive data processing application on AWS. The data is highly sensitive. The VPC must have no direct internet access, and the company has applied strict network security to control access.

Data scientists will transfer data from the company's on-premises data center to the instances by using an AWS Site-to-Site VPN connection. The on-premises data center uses the network range 172.31.0.0/20 and will use the network range 172.31.16.0/20 in the application VPC.

The data scientists report that they can start new instances of the application but that they cannot transfer any data from the on-premises data center. A network engineer enables VPC flow logs and sends a ping to one of the instances to test reachability. The flow logs show the following:

```
2 123456789010 eni-1235b8ca123456789 172.31.8.29 172.31.18.139 0 0 1 4 336 1622433184 1622433194 ACCEPT OK
2 123456789010 eni-1235b8ca123456789 172.31.18.139 172.31.8.29 0 0 1 3 252 1622433216 1622433232 REJECT OK
```

The network engineer must recommend a solution that will give the data scientists the ability to transfer data from the on-premises data center.

Which solution will meet these requirements?

- A. Modify the security group for the application. Add an inbound rule to allow traffic from the on-premises data center network range to the application.
- B. Modify the network ACLs for the VPC subnet. Add an inbound rule to allow traffic from the on-premises data center network range to the VPC subnet range.
- C. Modify the network ACLs for the VPC subnet. Add an outbound rule to allow traffic from the VPC subnet range to the on-premises data center network range.
- D. Modify the security group for the application. Add an outbound rule to allow traffic from the application to the on-premises data center network range.

Answer: C

Explanation:

C. Modify the network ACLs for the VPC subnet. Add an outbound rule to allow traffic from the VPC subnet range to the on-premises data center network range.

Network Access Control Lists (ACLs) are used to control the flow of traffic to and from subnets within a VPC. By modifying the network ACLs for the VPC subnet and adding an outbound rule, you ensure that traffic from the VPC subnet can reach the on-premises data center network. This setup allows the application to initiate traffic to the on-premises data center, meeting the requirements for connectivity.

Question: 126

A company needs to temporarily scale out capacity for an on-premises application and wants to deploy new servers on Amazon EC2 instances. A network engineer must design the networking solution for the connectivity and for the application on AWS.

The EC2 instances need to share data with the existing servers in the on-premises data center. The servers must not be accessible from the internet. All traffic to the internet must route through the firewall in the on-premises data center. The servers must be able to access a third-party web application.

Which configuration will meet these requirements?

- A. Create a VPC that has public subnets and private subnets. Create a customer gateway, a virtual private gateway, and an AWS Site-to-Site VPN connection. Create a NAT gateway in a public subnet. Create a route table, and associate the public subnets with the route table. Add a default route to the internet gateway. Create a route table, and associate the private subnets with the route table. Add a default route to the NAT gateway. Add routes for the data center subnets to the virtual private gateway. Deploy the application to the private subnets.
- B. Create a VPC that has private subnets. Create a customer gateway, a virtual private gateway, and an AWS Site-to-Site VPN connection. Create a route table, and associate the private subnets with the route table. Add a default route to the virtual private gateway. Deploy the application to the private subnets.
- C. Create a VPC that has public subnets. Create a customer gateway, a virtual private gateway, and an AWS Site-to-Site VPN connection. Create a route table, and associate the public subnets with the route table. Add a default route to the internet gateway. Add routes for the on-premises data center subnets to the virtual private gateway. Deploy the application to the public subnets.
- D. Create a VPC that has public subnets and private subnets. Create a customer gateway, a virtual private gateway, and an AWS Site-to-Site VPN connection. Create a route table, and associate the public subnets with the route table. Add a default route to the internet gateway. Create a route table, and associate the private subnets with the route table. Add routes for the on-premises data center subnets to the virtual private gateway. Deploy the application to the private subnets.

Answer: B

Explanation:

Here's a detailed justification for why option B is the correct answer:

The core requirement is secure communication between EC2 instances in AWS and on-premises servers, with all outbound internet traffic routed through the on-premises firewall. Option B directly addresses these requirements most efficiently.

Private Subnets: Utilizing private subnets ensures that the EC2 instances are not directly exposed to the internet. This aligns with the need to prevent direct internet access to the servers.

Site-to-Site VPN: Establishing a Site-to-Site VPN connection (using a Customer Gateway and Virtual Private Gateway) provides a secure and encrypted tunnel between the AWS VPC and the on-premises data center. This allows for private communication between the EC2 instances and the on-premises servers, fulfilling the data-sharing requirement.

Default Route to VGW: Setting the default route (0.0.0.0/0) for the private subnet's route table to the Virtual Private Gateway forces all outbound traffic from the EC2 instances to be routed through the VPN connection. This is crucial for ensuring that all traffic destined for the internet passes through the on-premises firewall.

No NAT Gateway: Option B avoids using a NAT Gateway, as it would provide direct internet access from AWS, which contradicts the requirement that all internet traffic flows through the on-premises firewall.

Application Deployment: Deploying the application to the private subnets aligns perfectly with the design goal of keeping the application servers isolated from the public internet.

Options A, C, and D all introduce public subnets and internet gateways, which contradict the requirement of

routing all traffic through the on-premises firewall and not exposing the servers directly to the internet. These options fail to fully encapsulate the AWS resources within the security perimeter of the on-premises infrastructure.

Here's a summary of why other options are wrong.

A - NAT Gateway allows instances within the private subnet to initiate outbound connections to the Internet or other AWS services, but prevents the Internet from initiating connections with those instances. Since the requirement states all outbound internet traffic to route through the on-prem firewall, NAT Gateway cannot be used.

C - Internet Gateway will give the public subnet a route to the internet, which breaks the requirement.

D - Internet Gateway will give the public subnet a route to the internet, which breaks the requirement.

Therefore, option B is the most secure and appropriate solution for the given requirements.

Supporting Documentation:

AWS VPN Connections: https://docs.aws.amazon.com/vpn/latest/s2svpn/VPC_VPN.html

Route Tables: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Route_Tables.html

VPC Endpoints: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html>

Question: 127

CertyIQ

A company is deploying a web application into two AWS Regions. The company has one VPC in each Region. Each VPC has three Amazon EC2 instances as web servers behind an Application Load Balancer (ALB). The company already has configured an Amazon Route 53 public hosted zone for example.com. Users will access the application by using the fully qualified domain name (FQDN) of app.example.com.

The company needs a DNS solution that allows global users to access the application. The solution must route the users' requests to the Region that provides the lowest response time. The solution must fail over to the Region that provides the next-lowest response time if the application is unavailable in the initially intended Region.

Which solution will meet these requirements?

A. For each ALB, create an A record that has a geolocation routing policy to route app.example.com to the IP addresses of the ALB. Configure a Route 53 HTTP health check that monitors each ALB by IP address. Associate the health check with the A records.

B. Create an A record that has a geolocation routing policy to route app.example.com to the IP addresses for both ALBs. Configure a Route 53 health check that monitors TCP port 80 for each ALB by IP address. Associate the health check with the A records.

C. Create an A record that has a latency-based routing policy to route app.example.com as an alias to one of the ALBs. Configure a Route 53 health check that monitors TCP port 80 for each ALB by IP address. Associate the health check with the A records.

D. For each ALB, create an A record that has a latency-based routing policy to route app.example.com as an alias to the ALB. Set the value for Evaluate Target Health to Yes for the records.

Answer: D

Explanation:

The correct solution is **D**. Here's why:

Latency-Based Routing: The requirement is to route users to the Region with the lowest response time. Route 53's latency-based routing policy directly addresses this. It routes traffic to the resource that provides the best latency as measured from the end user's location.

Alias Records: Using an alias record pointing to the ALBs simplifies DNS management. Instead of managing IP addresses, Route 53 automatically resolves the alias to the ALB's IP addresses, handling changes transparently.

Evaluate Target Health: Setting "Evaluate Target Health" to "Yes" is crucial for failover. This configuration ensures that Route 53 continuously monitors the health of the ALBs. If an ALB becomes unhealthy (the application is unavailable), Route 53 will automatically stop routing traffic to it and direct users to the next-best Region based on latency.

Why other options are incorrect:

A and B: Geolocation routing routes traffic based on the geographic location of the user, not latency, which does not fulfill the requirement. Also, configuring health checks by IP address of ALBs is not recommended as ALBs IPs are dynamic.

C: Creating an A record that uses alias to only one ALB will only route the traffic to one ALB which does not help when the one ALB configured becomes unavailable.

Authoritative Links:

Route 53 Routing Policies: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/routing-policy.html>

Route 53 Health Checks: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/dns-failover.html>

Alias Records vs. A Records: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resource-record-sets-choosing-alias-non-alias.html>

Question: 128

CertyIQ

A consulting company manages AWS accounts for its customers. One of the company's customers needs to add intrusion prevention for its environment without having to re-architect the environment. The customer's environment includes five VPCs in two AWS Regions in the United States. VPC-to-VPC connectivity is achieved through VPC peering. The customer does not plan to increase the number of VPCs within the next 2 years. The solution must accommodate unencrypted traffic.

Which solution will meet these requirements?

- A. Configure VPC security groups and network ACLs.
- B. Use an AWS Network Firewall centralized deployment model in each VPC.
- C. Use an AWS Network Firewall distributed deployment model in each VPC.
- D. Deploy AWS Shield in each VPC.

Answer: C

Explanation:

The correct answer is **C: Use an AWS Network Firewall distributed deployment model in each VPC.**

Here's why:

1. **Intrusion Prevention:** AWS Network Firewall provides intrusion prevention system (IPS) capabilities, which is a key requirement. VPC security groups and NACLs (Option A) provide basic security but lack advanced intrusion prevention features like signature-based detection and deep packet inspection. AWS Shield (Option D) primarily protects against DDoS attacks, not general intrusion prevention.
2. **No Re-architecting:** A distributed deployment model avoids significant re-architecting. Each VPC has its own Network Firewall endpoint, inspecting traffic within that VPC. This preserves the existing VPC peering setup. A centralized deployment (Option B) would require routing all traffic from all VPCs through a central Network Firewall in a separate VPC, necessitating changes to routing tables and potentially disrupting existing connectivity.

3. **Unencrypted Traffic:** Network Firewall can inspect unencrypted traffic, meeting the requirement. VPC Security Groups and NACLs also handle unencrypted traffic, but again, lack the advanced intrusion prevention capabilities.
4. **Scalability (Small Scale):** The customer has a relatively small number of VPCs (5). While a centralized approach can be more efficient at scale, the overhead for the current and near-term scenario (no planned VPC increase) is not justified. A distributed model provides adequate isolation and control for each VPC.
5. **Regional Coverage:** As the customer has VPCs in two regions, deploying Network Firewall in a distributed model would be simple.

Therefore, a distributed deployment of AWS Network Firewall is the most suitable solution. It offers the required intrusion prevention capabilities without necessitating a significant architectural overhaul, and is suitable for the small number of VPCs.

Authoritative Links:

1. **AWS Network Firewall Documentation:** <https://docs.aws.amazon.com/network-firewall/latest/developerguide/what-is-aws-network-firewall.html>
2. **Centralized vs. Distributed Network Firewall Architecture:** (While AWS doesn't have a direct article comparing these, architectural diagrams within the Network Firewall documentation illustrate the two deployment patterns. Review the documentation link above for examples)

Question: 129

CertyIQ

A company hosts its IT infrastructure in an on-premises data center. The company wants to migrate the infrastructure to the AWS Cloud in phases. A network engineer wants to set up a 10 Gbps AWS Direct Connect dedicated connection between the on-premises data center and VPCs. The company's network provider needs 3 months to provision the Direct Connect connection.

In the meantime, the network engineer implements a temporary solution by deploying an AWS Site-to-Site VPN connection that terminates to a virtual private gateway. The network engineer observes that the bandwidth of the Site-to-Site VPN connection is capped at 1.25 Gbps despite a powerful customer gateway device.

What should the network engineer do to improve the VPN connection bandwidth before the implementation of the Direct Connect connection?

- A. Contact AWS Support to request a bandwidth quota increase for the existing Site-to-Site VPN connection.
- B. Discuss the issue with the hardware vendor. Buy a bigger and more powerful customer gateway device that has faster encryption and decryption capabilities.
- C. Create several additional Site-to-Site VPN connections that terminate on the same virtual gateway. Configure equal-cost multi-path (ECMP) routing to use all the VPN connections simultaneously.
- D. Create a transit gateway. Attach the VPCs to the transit gateway. Create several additional Site-to-Site VPN connections that terminate on the transit gateway. Configure equal-cost multi-path (ECMP) routing to use all the VPN connections simultaneously.

Answer: D

Explanation:

Here's a detailed justification for why option D is the best approach to improve the VPN connection bandwidth:

The current Site-to-Site VPN connection is capped at 1.25 Gbps. This limitation is primarily due to the inherent limitations of a single VPN tunnel within AWS. While individual Site-to-Site VPN connections have a limit, you can increase aggregate throughput by using multiple VPN connections and equal-cost multi-path (ECMP) routing.

Option A is incorrect because AWS does not generally increase the bandwidth quota of a single Site-to-Site VPN connection beyond its established limit. The architecture of the AWS VPN service is designed for scalability through multiple connections, not by increasing the bandwidth of one.

Option B is unlikely to solve the problem. While a more powerful customer gateway might improve performance to some extent, the primary bottleneck is the AWS side of the VPN connection. AWS controls the bandwidth allocated to each VPN tunnel endpoint. Simply upgrading the customer gateway won't overcome the AWS-imposed limit.

Option C is a step in the right direction by suggesting multiple VPN connections and ECMP. However, terminating these multiple VPN connections to a Virtual Private Gateway (VGW) will not scale effectively. A Transit Gateway (TGW) is specifically designed for interconnecting multiple VPCs and VPN connections in a hub-and-spoke topology.

Option D introduces a Transit Gateway, a regional virtual router, that becomes the central hub for all VPN connections and VPCs. By creating multiple Site-to-Site VPN connections to the Transit Gateway and using ECMP routing, traffic can be distributed across these connections, effectively increasing the aggregate bandwidth. The Transit Gateway simplifies network management and scaling in a multi-VPC environment, making it the ideal solution in this scenario. Each VPN connection can use its maximum bandwidth and ECMP will utilize them all, thus boosting overall throughput.

Here are some resources for further research:

AWS Site-to-Site VPN: <https://aws.amazon.com/vpn/>

AWS Transit Gateway: <https://aws.amazon.com/transit-gateway/>

Site-to-Site VPN Quotas: https://docs.aws.amazon.com/vpn/latest/s2svpn/VPN_Quotas.html

AWS VPN ECMP: <https://docs.aws.amazon.com/vpn/latest/s2svpn/VPNCustomBGPConfig.html>

Question: 130

CertyIQ

A company has business operations in the United States and in Europe. The company's public applications are running on AWS and use three transit gateways. The transit gateways are located in the us-west-2, us-east-1, and eu-central-1 Regions. All the transit gateways are connected to each other in a full mesh configuration.

The company accidentally removes the route to the eu-central-1 VPCs from the us-west-2 transit gateway route table. The company also accidentally removes the route to the us-west-2 VPCs from the eu-central-1 transit gateway route table.

How can a network engineer identify the misconfiguration with the LEAST operational overhead?

- A. Use the Route Analyzer feature for AWS Transit Gateway Network Manager.
- B. Use the AWSSupport-SetupIPMonitoringFromVPC AWS Systems Manager Automation runbook. Push network telemetry data to Amazon CloudWatch Logs for analysis.
- C. Use VPC flow logs in eu-central-1 and us-west-2 to analyze the missing routes.
- D. Use Amazon VPC Traffic Mirroring in eu-central-1 or us-west-2 to take packet captures and troubleshoot the connectivity issues.

Answer: A

Explanation:

The correct answer is **A. Use the Route Analyzer feature for AWS Transit Gateway Network Manager.**

Here's a detailed justification:

The Route Analyzer within AWS Transit Gateway Network Manager is specifically designed for identifying and diagnosing network connectivity issues related to misconfigurations in transit gateway route tables and

attachments. It allows you to analyze network paths between resources connected to transit gateways and identify potential routing issues, such as missing or incorrect routes. It automates the process of tracing network paths and verifying that routes are correctly configured, significantly reducing operational overhead compared to manual methods.

Option B is less optimal because it involves setting up IP monitoring, pushing telemetry data, and analyzing logs. This requires more configuration and manual analysis to identify the missing routes, resulting in higher operational overhead.<https://aws.amazon.com/systems-manager/automation/>

Option C involves analyzing VPC flow logs, which can be helpful for identifying traffic patterns and network issues, but it requires significant effort to correlate the logs from different Regions and manually identify the missing routes. It is also harder to proactively detect a missing route with VPC Flow Logs.<https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs.html>

Option D involves using VPC Traffic Mirroring, which captures network traffic for analysis. This can be helpful for troubleshooting connectivity issues, but it requires setting up mirroring sessions and analyzing the captured traffic, which introduces significant operational overhead. More so, capturing packets alone does not explicitly point to a missing route.<https://docs.aws.amazon.com/vpc/latest/mirroring/what-is-traffic-mirroring.html>

Therefore, Route Analyzer provides the most efficient way to identify the misconfiguration with the least operational overhead by providing a direct analysis of the routing configuration within the transit gateways.<https://docs.aws.amazon.com/network-manager/latest/tgwnm/route-analyzer.html>

Question: 131

CertyIQ

A marketing company is using hybrid infrastructure through AWS Direct Connect links and a software-defined wide area network (SD-WAN) overlay to connect its branch offices. The company connects multiple VPCs to a third-party SD-WAN appliance transit VPC within the same account by using AWS Site-to-Site VPNs.

The company is planning to connect more VPCs to the SD-WAN appliance transit VPC. However, the company faces challenges of scalability, route table limitations, and higher costs with the existing architecture. A network engineer must design a solution to resolve these issues and remove dependencies.

Which solution will meet these requirements with the LEAST amount of operational overhead?

- A. Configure a transit gateway to attach the VPCs. Configure a Site-to-Site VPN connection between the transit gateway and the third-party SD-WAN appliance transit VPC. Use the SD-WAN overlay links to connect to the branch offices.
- B. Configure a transit gateway to attach the VPCs. Configure a transit gateway Connect attachment for the third-party SD-WAN appliance transit VPC. Use transit gateway Connect native integration of SD-WAN virtual hubs with AWS Transit Gateway.
- C. Configure a transit gateway to attach the VPCs. Configure VPC peering between the VPCs and the third-party SD-WAN appliance transit VPC. Use the SD-WAN overlay links to connect to the branch offices.
- D. Configure VPC peering between the VPCs and the third-party SD-WAN appliance transit VPC. Use transit gateway Connect native integration of SD-WAN virtual hubs with AWS Transit Gateway.

Answer: B

Explanation:

The best solution is **B. Configure a transit gateway to attach the VPCs. Configure a transit gateway Connect attachment for the third-party SD-WAN appliance transit VPC. Use transit gateway Connect native integration of SD-WAN virtual hubs with AWS Transit Gateway.**

Here's why:

Scalability and Route Table Limitations: AWS Transit Gateway (TGW) addresses scalability challenges by providing a central hub for interconnecting VPCs and on-premises networks. TGW eliminates the route table limitations associated with VPC peering and VPN solutions, as it can handle a large number of routes.

Reduced Operational Overhead: TGW simplifies network management by reducing the number of connections and route table entries needed. It offers a centralized routing policy, making network changes and updates easier to manage.

Cost Optimization: While TGW does have associated costs, it can optimize costs compared to complex VPN or VPC peering meshes, particularly as the network scales.

TGW Connect for SD-WAN Integration: TGW Connect is designed specifically for integrating SD-WAN solutions with AWS. It provides a standardized way to connect SD-WAN virtual hubs to TGW, allowing for seamless communication between on-premises branches and AWS resources.

Direct Connect Integration: Transit Gateway facilitates easier integration with Direct Connect links since the TGW can be directly attached to a Direct Connect gateway, allowing for the easy transit of traffic between the branch offices (via SD-WAN) and AWS resources.

Option A (Site-to-Site VPN): While this option uses TGW, relying on Site-to-Site VPN introduces complexities in configuration, monitoring, and troubleshooting. It doesn't leverage the optimized integration offered by TGW Connect.

Options C and D (VPC Peering): VPC peering does not scale well for a large number of VPCs, leading to complex route management and potential performance bottlenecks. It also lacks the centralized routing and management capabilities of TGW. TGW Connect is a better approach to integrate the SD-WAN.

Therefore, TGW with TGW Connect offers the most scalable, manageable, and cost-effective solution for integrating the SD-WAN overlay with the company's AWS environment.

Supporting Documentation:

AWS Transit Gateway: <https://aws.amazon.com/transit-gateway/>

Transit Gateway Connect: <https://aws.amazon.com/blogs/networking-and-content-delivery/integrate-your-sd-wan-solution-with-aws-transit-gateway-connect/>

Question: 132

CertyIQ

A company is running a hybrid cloud environment. The company has multiple AWS accounts as part of an organization in AWS Organizations. The company needs a solution to manage a list of IPv4 on-premises hosts that will be allowed to access resources in AWS. The solution must provide version control for the list of IPv4 addresses and must make the list available to the AWS accounts in the organization.

Which solution will meet these requirements?

- A. Create a customer-managed prefix list. Add entries for the initial list of on-premises IPv4 hosts. Create a resource share in AWS Resource Access Manager. Add the managed prefix list to the resource share. Share the resource with the organization.
- B. Create a customer-managed prefix list. Add entries for the initial list of on-premises IPv4 hosts. Use AWS Firewall Manager to share the managed prefix list with the organization.
- C. Create a security group. Add inbound rule entries for the initial list of on-premises IPv4 hosts. Create a resource share in AWS Resource Access Manager. Add the security group to the resource share. Share the resource with the organization.
- D. Create an Amazon DynamoDB table. Add entries for the initial list of on-premises IPv4 hosts. Create an AWS Lambda function that assumes a role in each AWS account in the organization to authorize inbound rules on security groups based on entries from the DynamoDB table.

Answer: A

Explanation:

The correct answer is **A**. Here's why:

Customer-Managed Prefix Lists: Prefix lists provide a scalable and manageable way to group and manage sets of IP addresses. They offer a central repository for the IPv4 addresses of on-premises hosts that need access to AWS resources. (<https://docs.aws.amazon.com/vpc/latest/userguide/working-with-prefix-lists.html>)

Version Control: Prefix lists automatically maintain a version history, satisfying the version control requirement. Each update to the list creates a new version, allowing you to revert to previous configurations if needed.

AWS Resource Access Manager (RAM): RAM enables you to securely share AWS resources that you own with other AWS accounts or within your AWS Organization. This is the ideal method for making the prefix list available across all accounts within the organization.

(<https://docs.aws.amazon.com/ram/latest/userguide/what-is.html>)

Organization Sharing: Sharing the resource with the organization makes the prefix list readily accessible and usable within each AWS account belonging to the organization. This centralizes management and ensures consistency across the entire hybrid cloud environment.

Why other options are incorrect:

B (AWS Firewall Manager): While Firewall Manager can manage security policies across accounts, it is not the primary tool for sharing prefix lists. It's more geared towards managing firewall rules and protections at scale. Using RAM directly for prefix list sharing is simpler and more efficient.

C (Security Groups and RAM): Security groups have limitations in terms of the number of rules and the management of large IP address lists. While RAM can share security groups, prefix lists are specifically designed for managing IP address ranges and are a better fit for this use case. Security Groups also do not inherently provide version control in the same way prefix lists do.

D (DynamoDB and Lambda): This approach is overly complex. It involves managing IP addresses in a database, creating custom code (Lambda) to retrieve them, and then updating security groups in each account. This solution lacks the simplicity and built-in features of prefix lists and RAM. It introduces unnecessary overhead and potential points of failure. The scalability of this solution is also questionable compared to a managed prefix list.

Question: 133

CertyIQ

A company's application is deployed on Amazon EC2 instances in a single VPC in an AWS Region. The EC2 instances are running in two Availability Zones. The company decides to use a fleet of traffic inspection instances from AWS Marketplace to inspect traffic between the VPC and the internet. The company is performing tests before the company deploys the architecture into production.

The fleet is located in a shared inspection VPC behind a Gateway Load Balancer (GWLB). To minimize the cost of the solution, the company deployed only one inspection instance in each Availability Zone that the application uses.

During tests, a network engineer notices that traffic inspection works as expected when the network is stable. However, during maintenance of the inspection instances, the internet sessions time out for some application instances. The application instances are not able to establish new sessions.

Which combination of steps will remediate these issues? (Choose two.)

- A. Deploy one inspection instance in the Availability Zones that do not have inspection instances deployed.
- B. Deploy one additional inspection instance in each Availability Zone where the inspection instances are deployed.
- C. Enable the cross-zone load balancing attribute for the GWLB.
- D. Deploy inspection instances in an Auto Scaling group. Define a scaling policy that is based on CPU load.
- E. Attach the GWLB to all Availability Zones in the Region.

Answer: BC

Explanation:

The problem stems from having only one inspection instance per Availability Zone. When maintenance occurs on that single instance, all traffic destined for that Availability Zone's inspection instance is interrupted, leading to session timeouts and inability to establish new sessions.

Option B, deploying an additional inspection instance in each Availability Zone, directly addresses the single point of failure. This provides redundancy. If one instance undergoes maintenance, the other can continue processing traffic, ensuring service continuity.

Option C, enabling cross-zone load balancing on the Gateway Load Balancer (GWLB), is crucial. By default, GWLBs operate within a single Availability Zone. With cross-zone load balancing enabled, the GWLB can distribute traffic to healthy inspection instances in any Availability Zone, not just the one where the initiating traffic originated. This ensures that even if an entire Availability Zone's inspection instance is down for maintenance, traffic can still be processed by healthy instances in other zones.

Option A is incorrect. Deploying inspection instances in Availability Zones where there are no application instances isn't relevant to the described issue. The concern is maintaining service when the existing inspection instances are unavailable.

Option D, deploying inspection instances in an Auto Scaling group based on CPU load, is a good practice for dynamically scaling inspection capacity based on traffic demand, but it doesn't solve the immediate issue of downtime during maintenance if only one instance exists per Availability Zone. While it would eventually scale up, it doesn't protect against initial disruption.

Option E, attaching the GWLB to all Availability Zones in the Region, while potentially beneficial from a regional distribution standpoint, doesn't directly address the redundancy issue within the zones where the application instances exist. The load balancer presence in a zone alone doesn't ensure traffic inspection continuity if the instances behind the load balancer in the application's zone are unavailable.

In summary, redundancy within each zone where the application resides (Option B) and the ability to route traffic across zones when instances fail (Option C) are essential to solve the issue of service disruption during inspection instance maintenance.

Relevant Links:

AWS Gateway Load Balancer: <https://aws.amazon.com/elasticloadbalancing/gateway-load-balancer/>

GWLB Cross-zone Load Balancing:

<https://docs.aws.amazon.com/elasticloadbalancing/latest/gateway/gateway-load-balancer-cross-zone-load-balancing.html>

Question: 134

CertyIQ

A company has developed a new web application on AWS. The application runs on Amazon Elastic Container Service (Amazon ECS) on AWS Fargate behind an Application Load Balancer (ALB) in the us-east-1 Region. The application uses Amazon Route 53 to host the DNS records for the domain. The content that is served from the website is mostly static images and files that are not updated frequently. Most of the traffic to the website from end users will originate from the United States. Some traffic will originate from Canada and Europe.

A network engineer needs to design a solution that will reduce latency for end users at the lowest cost. The solution also must ensure that all traffic is encrypted in transit until the traffic reaches the ALB.

Which solution will meet these requirements?

A. Configure the ALB to use an AWS Global Accelerator accelerator in us-east-1. Create a secure HTTPS listener. Create an alias record in Amazon Route 53 for the custom domain name. Configure the alias record to route to the DNS name that is assigned to the accelerator for the ALB.

B. Configure the ALB to use a secure HTTPS listener. Create an Amazon CloudFront distribution. Set the origin domain name to point to the DNS record that is assigned to the ALB. Configure the CloudFront distribution to use an SSL certificate. Set all behaviors to force HTTPS. Create an alias record in Amazon Route 53 for the custom domain name. Configure the alias record to route to the DNS name that is assigned to the ALB.

C. Configure the ALB to use a secure HTTPS listener. Create an Amazon CloudFront distribution. Set the origin domain name to point to the DNS record that is assigned to the ALB. Configure the CloudFront distribution to use an SSL certificate and redirect HTTP to HTTPS. Create an alias record in Amazon Route 53 for the custom domain name. Configure the alias record to route to the CloudFront distribution.

D. Configure the ALB to use an AWS Global Accelerator accelerator in us-east-1. Create a secure HTTPS listener. Create a second application stack on Amazon ECS on Fargate in the eu-west-1 Region. Create another secure HTTPS listener. Create an alias record in Amazon Route 53 for the custom domain name. Configure the alias record to use a latency-based routing policy to route to the DNS name that is assigned to the accelerator for the ALBs.

Answer: C

Explanation:

The correct answer is C. Here's why:

Option C leverages Amazon CloudFront, a content delivery network (CDN), to cache static content closer to users across the globe, reducing latency. By pointing the CloudFront distribution to the ALB as its origin, requests for dynamic content are forwarded to the application running in us-east-1. Configuring CloudFront to use an SSL certificate and redirect HTTP to HTTPS ensures that all traffic is encrypted in transit, satisfying the security requirement. Using an alias record in Route 53 to point to the CloudFront distribution ensures that user requests are directed to the CDN for optimal performance. This solution provides low latency, security, and is cost-effective because CloudFront caches the static content, reducing the load on the ALB.

Option A is incorrect because AWS Global Accelerator is primarily designed for improving the performance of dynamic content and is not the most cost-effective solution for caching static content. While it provides a global entry point and improved reliability, CloudFront is better suited for latency reduction of static assets at a lower cost.

Option B incorrectly configures Route 53 to point directly to the ALB instead of the CloudFront distribution, negating the benefits of using the CDN for latency reduction. The CloudFront distribution would therefore be bypassed.

Option D introduces an unnecessary second application stack in eu-west-1, significantly increasing costs and complexity without providing a substantial performance benefit over using CloudFront. Latency-based routing in Route 53 is a good practice for multi-region deployments, but deploying a whole new stack when static assets can be cached globally is overkill.

Supporting Concepts & Links:

Amazon CloudFront: A fast, highly secure and programmable content delivery network (CDN).

<https://aws.amazon.com/cloudfront/>

Amazon Route 53: A highly available and scalable cloud DNS web service. <https://aws.amazon.com/route53/>

Application Load Balancer (ALB): Distributes incoming application traffic across multiple targets, such as EC2 instances, in multiple Availability Zones. <https://aws.amazon.com/elasticloadbalancing/application-load-balancer/>

AWS Global Accelerator: A networking service that improves the availability and performance of your applications that are used by a global audience. <https://aws.amazon.com/global-accelerator/>

Question: 135

A company deploys an internal website behind an Application Load Balancer (ALB) in a VPC. The VPC has a CIDR

block of 172.31.0.0/16. The company creates a private hosted zone for the domain example.com for the website in Amazon Route 53. The company establishes an AWS Site-to-Site VPN connection between its office network and the VPC.

A network engineer needs to set up a DNS solution so that employees can visit the internal webpage by accessing a private domain URL (https://example.com) from the office network.

Which combination of steps will meet this requirement? (Choose two.)

- A. Create an alias record that points to the ALB in the Route 53 private hosted zone.
- B. Create a CNAME record that points to the ALB internal domain in the Route 53 private hosted zone.
- C. Create a Route 53 Resolver inbound endpoint. On the office DNS server, configure a conditional forwarder to forward the DNS queries to the Route 53 Resolver inbound endpoint.
- D. Create a Route 53 Resolver outbound endpoint. On the office DNS server, configure a conditional forwarder to forward the DNS queries to the Route 53 Resolver outbound endpoint.
- E. On the office DNS server, configure a conditional forwarder for the private domain to the VPC DNS at 172.31.0.2.

Answer: AC

Explanation:

The correct answer is A and C. Here's a breakdown of why these options work and why the others don't:

A. Create an alias record that points to the ALB in the Route 53 private hosted zone.

Justification: An Alias record in Route 53 is a DNS record specific to AWS. It allows you to map your domain name (example.com) directly to an AWS resource, such as an ALB. This is crucial for resolving the private domain to the ALB that hosts the internal website. Because the ALB might have a changing IP address, an alias record handles the resolution automatically, making it superior to using a static IP address in an A record.

C. Create a Route 53 Resolver inbound endpoint. On the office DNS server, configure a conditional forwarder to forward the DNS queries to the Route 53 Resolver inbound endpoint.

Justification: Route 53 Resolver Inbound Endpoints are created in the VPC. The office network needs a way to resolve DNS queries for the private hosted zone (example.com) to the AWS environment. Configuring a Route 53 Resolver inbound endpoint allows DNS queries originating from the on-premises office network, which is connected to the VPC via Site-to-Site VPN, to reach the Route 53 private hosted zone in the AWS environment. The conditional forwarder on the office DNS server is configured to forward queries for the example.com domain to the Inbound Endpoint's IP address. This, in turn, allows the queries to be resolved against the private hosted zone.

Why other options are incorrect:

B. Create a CNAME record that points to the ALB internal domain in the Route 53 private hosted zone. ALBs do not expose internal domains to create CNAME records to, but rather it requires an Alias record to point to the ALB directly.

D. Create a Route 53 Resolver outbound endpoint. On the office DNS server, configure a conditional forwarder to forward the DNS queries to the Route 53 Resolver outbound endpoint. Route 53 Resolver Outbound Endpoints are used to query DNS services outside of AWS (from AWS). This is the inverse of what's required in this scenario.

E. On the office DNS server, configure a conditional forwarder for the private domain to the VPC DNS at 172.31.0.2. While the VPC DNS server (172.31.0.2) can resolve names within the VPC, it's generally not directly accessible from the on-premises network. Relying on this would create a single point of failure and is not the recommended approach for hybrid DNS resolution. A Route 53 Resolver Inbound Endpoint is the designed AWS service for this purpose and provides redundancy.

Authoritative Links:

Route 53 Resolver Endpoints: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver.html>

Alias Records: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resource-record-sets-choosing-alias-non-alias.html>

Question: 136

CertyIQ

A company is deploying AWS Cloud WAN with edge locations in the us-east-1 Region and the ap-southeast-2 Region. Individual AWS Cloud WAN segments are configured for the development environment, the production environment, and the shared services environment at each edge location. Many new VPCs will be deployed for the environments and will be configured as attachments to the AWS Cloud WAN core network.

The company's network team wants to ensure that VPC attachments are configured for the correct segment. The network team will tag the VPC attachments by using the Environment key with a value of the corresponding environment segment name. The segment for the production environment in us-east-1 must require acceptance for attachment requests. All other attachment requests must not require acceptance.

Which solution will meet these requirements?

- A. Create a rule with a number of 100 that requires acceptance for attachments to the production segment. In the rule, set the condition logic to the "or" value. Include conditions that require a tag:Environment value of Production or a Region value of us-east-1. Create a rule with a number of 200 that does not require acceptance to map any tag:Environment values to their respective segments.
- B. Create a rule with a number of 100 that requires acceptance for attachments to the production segment. In the rule, set the condition logic to the "and" value. Include conditions that require a tag:Environment value of Production and a Region value of us-east-1. Create a rule with a number of 200 that does not require acceptance to map any tag:Environment values to their respective segments.
- C. Create a rule with a number of 100 that does not require acceptance to map any tag:Environment values to their respective segments. Create a rule with a number of 200 that requires acceptance for attachments to the production segment. In the rule, set the condition logic to the "and" value. Include conditions that require a tag:Environment value of Production and a Region value of us-east-1.
- D. Create a rule with a number of 100 that does not require acceptance to map any tag:Environment values to their respective segments. Create a rule with a number of 200 that requires acceptance for attachments to the production segment. In the rule, set the condition logic to the "or" value. Include conditions that require a tag:Environment value of Production or a Region value of us-east-1.

Answer: B

Explanation:

Here's a detailed justification for why option B is the correct solution:

The core requirement is to automatically accept VPC attachments to Cloud WAN segments based on the Environment tag, except for Production attachments in the us-east-1 Region, which require manual acceptance. To achieve this, we need to define rules in Cloud WAN's policy configuration that govern attachment acceptance.

Option B proposes two rules. The first rule (number 100) targets specifically production attachments in us-east-1. The rule requires acceptance. Importantly, the condition logic is set to "AND". This means that for the rule to apply and trigger the required acceptance, both conditions — tag:Environment being Production and the Region being us-east-1 — must be met. This correctly isolates the specific scenario requiring manual acceptance.

The second rule (number 200) aims to automatically accept all other attachments. It does not require acceptance. It maps all the tag:Environment values to their respective segments. Because rule 100 comes first, any production attachments in us-east-1 will hit this rule first and require acceptance. Other environments and production environments in other regions will go to rule 200.

Option A is incorrect because it uses "OR" logic in the first rule. With "OR", if either the tag is "Production" OR the Region is "us-east-1", the acceptance requirement would be triggered. This would incorrectly require acceptance for all attachments in us-east-1, regardless of the environment.

Options C and D are incorrect because they reverse the order of the rules, and they have the same flaws of logic as options A and B.

Therefore, Option B correctly addresses the problem by using "AND" logic to precisely target the specific scenario requiring manual acceptance and by giving the correct precedence of the rules.

For further reading on AWS Cloud WAN and its policy configurations, refer to the official AWS documentation:

AWS Cloud WAN: <https://aws.amazon.com/cloud-wan/>

Cloud WAN Policies: <https://docs.aws.amazon.com/cloud-wan/latest/userguide/cloudwan-policies.html>

Question: 137

CertyIQ

A company is migrating applications from a data center to AWS. Many of the applications will need to exchange data with the company's on-premises mainframe.

The company needs to achieve 4 Gbps transfer speeds to meet peak traffic demands. A network engineer must design a highly available solution that maximizes resiliency. The solution must be able to withstand the loss of circuits or routers.

Which solution will meet these requirements?

- A. Order four 10 Gbps AWS Direct Connect connections that are evenly spread over two locations. Terminate one connection from each Direct Connect location to a router at the company location. Terminate the other connection from each Direct Connect location to a different router at the company location.
- B. Order two 10 Gbps AWS Direct Connect connections that are evenly spread over two locations. Terminate the connection from each Direct Connect location to a different router at the company location.
- C. Order four 1 Gbps AWS Direct Connect connections that are evenly spread over two locations. Terminate one connection from each Direct Connect location to a router at the company location. Terminate the other connection from each Direct Connect location to a different router at the company location.
- D. Order two 1 Gbps AWS Direct Connect connections that are evenly spread over two locations. Terminate the connection from each Direct Connect location to a different router at the company location.

Answer: A

Explanation:

Here's a detailed justification for why option A is the correct solution, along with supporting concepts and authoritative links:

The requirement is a highly available and resilient connection with 4 Gbps throughput between on-premises and AWS, using Direct Connect. Let's analyze why the other options are incorrect:

Options C and D (1 Gbps Direct Connect): These options use 1 Gbps connections. Even with multiple connections, they cannot reliably achieve the required 4 Gbps throughput, especially considering overhead and potential link saturation.

Option B (Two 10 Gbps Direct Connect): While this provides sufficient bandwidth, a failure in a single Direct Connect location can potentially halve the bandwidth to 5 Gbps (assuming even traffic distribution). While the remaining connection could potentially handle the load, it doesn't maximize resiliency against a complete location failure as effectively as option A. It also doesn't explicitly address router redundancy on the on-premises side.

Why Option A is Correct:

Option A (Four 10 Gbps Direct Connect connections across two locations) offers the best combination of bandwidth and resilience:

1. **Bandwidth Sufficiency:** Four 10 Gbps connections provide ample bandwidth (40 Gbps total) to easily exceed the 4 Gbps requirement. This leaves significant headroom for growth and peak traffic spikes.
2. **Geographic Redundancy:** Spreading the connections across two AWS Direct Connect locations protects against regional outages. If one location fails, the other location can still provide connectivity.
3. **Router Redundancy:** Terminating two connections to different routers on-premises ensures that if one router fails, the other router will still maintain connectivity. This arrangement eliminates a single point of failure on the on-premises network. Distributing connections from both Direct Connect locations to both on-premises routers distributes the risk even further.
4. **Resilience:** The solution is designed to tolerate multiple simultaneous failures (e.g., a circuit failure within a Direct Connect location, and a router failure on-premises) while still maintaining the required bandwidth. Even if one Direct Connect location and one router fail, the remaining two 10 Gbps connections would still provide significantly more than the required 4 Gbps.

Key Concepts:

AWS Direct Connect: A service that establishes a dedicated network connection from on-premises to AWS, bypassing the public internet. <https://aws.amazon.com/directconnect/>

High Availability: Designing systems to minimize downtime and continue functioning even in the event of failures.

Resiliency: The ability of a system to recover quickly from failures and continue operating.

Redundancy: Duplicating critical components to eliminate single points of failure.

Conclusion:

Option A is the most robust solution because it provides ample bandwidth, geographic redundancy, and router redundancy, maximizing both bandwidth and resilience to meet the company's requirements.

Question: 138

CertyIQ

A company has 10 web server Amazon EC2 instances that run in an Auto Scaling group in a production VPC. The company has 10 other web servers that run in an on-premises data center. The company has a 10 Gbps AWS Direct Connect connection between the on-premises data center and the production VPC.

The company needs to implement a load balancing solution that receives HTTPS traffic from thousands of external users. The solution must distribute the traffic across the web servers on AWS and the web servers in the on-premises data center. Regardless of the location of the web servers, HTTPS requests must go to the same web server throughout the entire session.

Which solution will meet these requirements?

A. Create a Network Load Balancer (NLB) in the production VPC. Create a target group. Specify ip as the target type. Register the EC2 instances and the on-premises servers with the target group. Enable connection draining on the NLB.

B. Create an Application Load Balancer (ALB) in the production VPC. Create a target group. Specify ip as the target type. Register the EC2 instances and the on-premises servers with the target group. Enable application-based session affinity (sticky sessions) on the ALB.

C. Create a Network Load Balancer (NLB) in the production VPC. Create a target group. Specify instance as the target type. Register the EC2 instances and the on-premises servers with the target group. Enable session affinity (sticky sessions) on the NLB.

D. Create an Application Load Balancer (ALB) in the production VPC. Create a target group. Specify instance as

the target type Register the EC2 instances and the on-premises servers with the target group Enable application-based session affinity (sticky sessions) on the ALB.

Answer: B

Explanation:

Here's a detailed justification for why option B is the correct solution, along with explanations of why other options are less suitable:

Justification for Option B (Application Load Balancer with IP Target and Sticky Sessions):

1. **HTTPS Traffic:** The requirement to handle HTTPS traffic immediately points to using an Application Load Balancer (ALB) or Network Load Balancer (NLB). However, ALBs are specifically designed for application-layer (Layer 7) traffic, including HTTP and HTTPS. NLBs are more suited for TCP, UDP, and TLS traffic at Layer 4. Since the requirement explicitly mentions HTTPS, ALB is a more fitting choice.
2. **Target Type - IP:** The on-premises web servers don't reside within the AWS environment as EC2 instances. Therefore, they cannot be registered as targets using the "instance" target type. The "ip" target type allows you to register targets using their IP addresses, making it possible to include both the EC2 instances and the on-premises servers within the same target group. This is critical for a unified load balancing solution.
3. **Sticky Sessions:** The requirement that HTTPS requests must go to the same web server throughout the entire session necessitates session affinity, also known as sticky sessions. ALBs offer application-based session affinity, which means the load balancer uses cookies to route requests from the same client to the same target server.
4. **Combined Solution:** Option B allows the ALB to distribute incoming HTTPS traffic across both AWS-based EC2 instances and the on-premises web servers seamlessly. This is achieved by registering both types of servers (EC2 instances via internal IP and on-premises servers via their public or private IP addresses reachable through Direct Connect) to the same target group.

Why Other Options Are Incorrect:

Option A (NLB with IP Target and Connection Draining): While NLBs can handle TLS traffic, they operate at Layer 4. NLBs do not natively support application-based session affinity like ALBs. Connection draining helps gracefully handle existing connections during scaling or instance replacement, but doesn't provide the sticky sessions needed for the entire session. While NLBs with TLS termination is a possibility, ALB are better suited for HTTPS traffic.

Option C (NLB with Instance Target and Sticky Sessions): On-premises servers can't be registered as "instance" targets. The "instance" target type is only applicable for EC2 instances within the same VPC or peered VPCs. The NLB's static IP address preservation might be confused with session affinity but is functionally different.

Option D (ALB with Instance Target and Sticky Sessions): Similar to Option C, on-premises servers cannot be registered as "instance" targets.

Authoritative Links:

Application Load Balancer:

<https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html>

Network Load Balancer: <https://docs.aws.amazon.com/elasticloadbalancing/latest/network/introduction.html>

Target Groups for ALB: <https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-target-groups.html>

Target Groups for NLB: <https://docs.aws.amazon.com/elasticloadbalancing/latest/network/load-balancer-target-groups.html>

ALB Sticky Sessions: <https://docs.aws.amazon.com/elasticloadbalancing/latest/application/sticky-sessions.html>

Question: 139

A company has an AWS environment that includes multiple VPCs that are connected by a transit gateway. The company has decided to use AWS Site-to-Site VPN to establish connectivity between its on-premises network and its AWS environment.

The company does not have a static public IP address for its on-premises network. A network engineer must implement a solution to initiate the VPN connection on the AWS side of the connection for traffic from the AWS environment to the on-premises network.

Which combination of steps should the network engineer take to establish VPN connectivity between the transit gateway and the on-premises network? (Choose three.)

- A. Configure the Site-to-Site VPN tunnel options to use Internet Key Exchange version 1 (IKEv1).
- B. Configure the Site-to-Site VPN tunnel options to use Internet Key Exchange version 2 (IKEv2).
- C. Use a private certificate authority (CA) from AWS Private Certificate Authority to create a certificate.
- D. Use a public certificate authority (CA) from AWS Private Certificate Authority to create a certificate.
- E. Create a customer gateway. Specify the current dynamic IP address of the customer gateway device's external interface.
- F. Create a customer gateway without specifying the IP address of the customer gateway device.

Answer: BCF

Explanation:

The correct answer is BCF. Let's break down why:

B: Configure the Site-to-Site VPN tunnel options to use Internet Key Exchange version 2 (IKEv2). IKEv2 supports NAT traversal better than IKEv1. Since the on-premises network does not have a static public IP address, it's behind a NAT device. IKEv2 is more robust in handling address changes and network address translation, ensuring a stable VPN connection in this dynamic IP scenario. AWS supports IKEv2 for Site-to-Site VPN.

C: Use a private certificate authority (CA) from AWS Private Certificate Authority to create a certificate. The question specifies that the VPN connection should be initiated on the AWS side. This requires certificate-based authentication which is a common method when dealing with dynamic IP addresses on the customer gateway. AWS Private CA allows you to generate and manage private certificates that can be used for authentication purposes. Using a private CA keeps the authentication process within your control and security boundaries.

F: Create a customer gateway without specifying the IP address of the customer gateway device. Because the on-premises network lacks a static public IP address, you cannot specify a fixed IP address for the customer gateway during its creation. The AWS side will initiate the connection, and the customer gateway will dynamically register its IP address once the connection is established.

Why other options are incorrect:

A: Configure the Site-to-Site VPN tunnel options to use Internet Key Exchange version 1 (IKEv1). While IKEv1 is supported, IKEv2 is the preferred option for dynamic IP addresses due to its superior NAT traversal capabilities.

D: Use a public certificate authority (CA) from AWS Private Certificate Authority to create a certificate. AWS Private CA is for creating private CAs, not public CAs. Public CAs are generally used for publicly accessible websites and applications.

E: Create a customer gateway. Specify the current dynamic IP address of the customer gateway device's external interface. This option directly contradicts the problem statement that specifies the on-premises network lacks a static public IP address. You cannot specify a dynamic IP address during customer gateway creation if it's prone to change.

Supporting Documentation:

AWS Site-to-Site VPN: https://docs.aws.amazon.com/vpn/latest/s2svpn/VPC_VPN.html

AWS Private CA: <https://aws.amazon.com/certificate-manager/private-certificate-authority/>

Customer Gateway: https://docs.aws.amazon.com/vpn/latest/s2svpn/VPC_VPN.html

Question: 140

CertyIQ

A company's AWS environment has two VPCs. VPC A has a CIDR block of 192.168.0.0/16. VPC B has a CIDR block of 10.0.0.0/16. Each VPC is deployed in a separate AWS Region. The company has remote users who work outside the company's offices. These users need to connect to an application that is running in the VPCs.

Traffic to and from the VPCs over the internet must be encrypted. A network engineer must set up connectivity between the remote users and the VPCs.

Which combination of steps should the network engineer take to meet these requirements with the LEAST management overhead? (Choose three.)

- A. Establish an AWS Site-to-Site VPN connection between VPC A and VPC B.
- B. Establish a VPC peering connection between VPC A and VPC B.
- C. Create an AWS Client VPN endpoint in VPC A and VPC B. Add an authorization rule to grant access to VPC A and VPC B.
- D. Create an AWS Client VPN endpoint in VPC A. Add an authorization rule to grant access to VPC A and VPC B.
- E. Add a route to the AWS Client VPN endpoint's route table to direct traffic to VPC B.
- F. Add a route to the AWS Client VPN endpoint's route table to direct traffic to VPC A.

Answer: BDE

Explanation:

The correct answer is BDE. Here's why:

B. Establish a VPC peering connection between VPC A and VPC B: VPC peering allows you to connect two VPCs directly via AWS's internal network. This is necessary to enable communication between the applications running in the two different VPCs across regions. While not directly related to the remote users' access, it addresses the requirement for inter-VPC communication once the users are connected.

D. Create an AWS Client VPN endpoint in VPC A: AWS Client VPN provides secure, client-to-cloud connectivity for remote users. Creating the endpoint in VPC A allows users to establish encrypted VPN tunnels to the AWS environment. Only one Client VPN endpoint is needed since the VPCs are peered. Using one endpoint reduces management overhead.

E. Add a route to the AWS Client VPN endpoint's route table to direct traffic to VPC B: Once the Client VPN endpoint is established in VPC A, you need to route traffic destined for VPC B through the peering connection. By adding a route in the Client VPN endpoint's route table that directs traffic with the CIDR block of VPC B (10.0.0.0/16) to the peering connection, you ensure that remote user traffic destined for VPC B is properly routed.

Why other options are not ideal:

A. Establish an AWS Site-to-Site VPN connection between VPC A and VPC B: Site-to-Site VPN is typically

used to connect an on-premises network to a VPC, not for connecting two VPCs within AWS, especially across regions. VPC Peering is more cost-effective and easier to manage for this scenario. Additionally, it does not address the need for remote user connectivity.

C. Create an AWS Client VPN endpoint in VPC A and VPC B Add an authorization rule to grant access to VPC A and VPC B: Creating two Client VPN endpoints increases management overhead without providing any additional benefit. Since VPCs are peered the endpoint in VPC A can grant access to resources in VPC B.

F. Add a route to the AWS Client VPN endpoint's route table to direct traffic to VPC A: While routing to VPC A is implicitly included, and a specific rule not necessarily needed, this option does not include the vital aspect of routing to VPC B (10.0.0.0/16) for full connectivity. Without a route to VPC B, remote users cannot reach resources in that VPC.

Authoritative Links:

AWS Client VPN: <https://aws.amazon.com/vpn/client-vpn/>

VPC Peering: <https://docs.aws.amazon.com/vpc/latest/peering/what-is-vpc-peering.html>

Question: 141

CertyIQ

A company uses Amazon Route 53 to register a public domain, example.com, in an AWS account. A central services group manages the account. The company wants to create a subdomain, test.example.com, in another AWS account to offer name services for Amazon EC2 instances that are hosted in the account. The company does not want to migrate the parent domain to the subdomain account.

A network engineer creates a new Route 53 hosted zone for the subdomain in the second account.

Which combination of steps must the network engineer take to complete the task? (Choose two.)

- A. Add records for the hosts of the new subdomain to the new Route 53 hosted zone.
- B. Update the DNS service for the parent domain by adding name server (NS) records for the subdomain.
- C. Update the DNS service for the subdomain by adding name server (NS) records for the parent domain.
- D. Create an alias record from the parent domain that points to the hosted zone for the subdomain in the second account.
- E. Add a start of authority (SOA) record in the parent domain for the subdomain.

Answer: AB

Explanation:

The correct answer is AB. Here's why:

A. Add records for the hosts of the new subdomain to the new Route 53 hosted zone: This is essential. The purpose of creating the test.example.com hosted zone in the second account is to manage the DNS records for hosts within that subdomain. These records (A records, CNAME records, etc.) are how users and services will resolve names within test.example.com to IP addresses or other hostnames. Without adding these records, the subdomain is effectively empty, and no queries to it will return meaningful results.

B. Update the DNS service for the parent domain by adding name server (NS) records for the subdomain: This step delegates authority for the subdomain to the second AWS account. In the example.com hosted zone (managed by the central services group), NS records are created for test.example.com. These NS records point to the name servers associated with the newly created test.example.com hosted zone in the second account. When a DNS resolver queries for a record within test.example.com, the parent domain's DNS (example.com) will direct the query to the authoritative name servers listed in the NS records for test.example.com.

Here's why the other options are incorrect:

C. Update the DNS service for the subdomain by adding name server (NS) records for the parent domain:

This is the reverse of what's needed. The subdomain doesn't need to know about the parent domain's name servers. The parent domain needs to delegate authority to the subdomain.

D. Create an alias record from the parent domain that points to the hosted zone for the subdomain in the second account:

Alias records in Route 53 are specific to AWS resources like ELBs or CloudFront distributions. They are not used for generic subdomain delegation. NS records are the standard mechanism for delegating a subdomain to different name servers.

E. Add a start of authority (SOA) record in the parent domain for the subdomain: SOA records are authoritative records defining the properties of the hosted zone. While an SOA record exists within the test.example.com hosted zone, adding an additional SOA record within the example.com hosted zone for test.example.com is incorrect and unnecessary for subdomain delegation.

In summary, you create the subdomain's hosted zone and populate it with records (A). The second step is delegating that subdomain from the parent domain (B) using NS records.

Supporting links:

[Route 53 documentation on creating hosted zones](#)

[Route 53 documentation on subdomain delegation](#)

Question: 142**CertyIQ**

An IoT company collects data from thousands of sensors that are deployed in the United States and South Asia. The sensors use a proprietary communication protocol that is built on UDP to send the data to a fleet of Amazon EC2 instances. The instances are in an Auto Scaling group and run behind a Network Load Balancer (NLB). The instances, Auto Scaling group, and NLB are deployed in the us-west-2 Region.

Occasionally, the data from the sensors in South Asia gets lost in transit over the internet and does not reach the EC2 instances.

Which solutions will resolve this issue? (Choose two.)

- A. Use AWS Global Accelerator with the existing NLB.
- B. Create an Amazon CloudFront distribution. Specify the existing NLB as the origin.
- C. Create a second deployment of the EC2 instances and the NLB in the ap-south-1 Region. Use an Amazon Route 53 latency routing policy to resolve to the Region that provides the least latency.
- D. Create a second deployment of the EC2 instances and the NLB in the ap-south-1 Region. Use an Amazon Route 53 failover routing policy to resolve to an alternate Region in case packets are dropped.
- E. Turn on enhanced networking on the EC2 instances by using the most recent Elastic Network Adapter (ENA) drivers.

Answer: AC**Explanation:**

The correct answer is AC. Here's why:

A. Use AWS Global Accelerator with the existing NLB.

AWS Global Accelerator improves application availability and performance for users worldwide by routing traffic through the AWS global network. It utilizes the AWS backbone to reduce latency and packet loss for TCP and UDP traffic. Because the IoT sensors rely on UDP, Global Accelerator's ability to optimize UDP traffic makes it a suitable solution for ensuring data from South Asia consistently reaches the EC2 instances in us-west-2. It also improves reliability since it quickly reacts to changes in network performance to help maintain consistent, high-quality connectivity. This directly addresses the reported problem of data loss in transit.

Authoritative Link: <https://aws.amazon.com/global-accelerator/>

C. Create a second deployment of the EC2 instances and the NLB in the ap-south-1 Region. Use an Amazon Route 53 latency routing policy to resolve to the Region that provides the least latency.

Deploying resources closer to the source (South Asia) reduces the distance data must travel over the public internet, decreasing the probability of packet loss. Route 53's latency-based routing directs traffic to the nearest endpoint (us-west-2 or ap-south-1) based on measured network latency. This effectively distributes the workload and optimizes the routing path to minimize data loss, resolving the problem for the South Asia sensors. The closer proximity of the ap-south-1 region will likely lower the latency significantly for data coming from South Asia.

Authoritative Link: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/routing-policy.html#routing-policy-latency>

Why the other options are less suitable:

B. Create an Amazon CloudFront distribution. Specify the existing NLB as the origin. CloudFront is designed for caching static and dynamic content, primarily HTTP(S) traffic. IoT sensor data is streamed using a proprietary protocol over UDP, which is not something that CloudFront is typically built to accelerate or cache. Using CloudFront for this scenario would not efficiently address the packet loss issue.

D. Create a second deployment of the EC2 instances and the NLB in the ap-south-1 Region. Use an Amazon Route 53 failover routing policy to resolve to an alternate Region in case packets are dropped. While failover routing provides redundancy, it only directs traffic to the alternate region (ap-south-1) after a failure is detected in the primary region (us-west-2). This doesn't actively prevent the data loss. The goal is to minimize loss; a latency-based approach is more proactive in selecting the optimal route. The failover design isn't geared towards performance enhancement but disaster recovery.

E. Turn on enhanced networking on the EC2 instances by using the most recent Elastic Network Adapter (ENA) drivers. Enhanced networking (ENA) improves network performance within the AWS environment. However, the issue described pertains to data loss over the internet between the sensors and the AWS infrastructure. ENA would not address the intermediate network hops outside of AWS that are causing the data loss from South Asia. While ENA can help the EC2 instances handle traffic more efficiently once it reaches them, it doesn't solve the problem of the data not arriving in the first place.

Question: 143

CertyIQ

A company has an application that runs on a fleet of Amazon EC2 instances. A new company regulation mandates that all network traffic to and from the EC2 instances must be sent to a centralized third-party EC2 appliance for content inspection.

Which solution will meet these requirements?

- A. Configure VPC flow logs on each EC2 network interface. Publish the flow logs to an Amazon S3 bucket. Create a third-party EC2 appliance to acquire flow logs from the S3 bucket. Log in to the appliance to monitor network content.
- B. Create a third-party EC2 appliance in an Auto Scaling group fronted by a Network Load Balancer (NLB). Configure a mirror session. Specify the NLB as the mirror target. Specify a mirror filter to capture inbound and outbound traffic. For the source of the mirror session, specify the EC2 elastic network interfaces for all the instances that host the application.
- C. Configure a mirror session. Specify an Amazon Kinesis Data Firehose delivery stream as the mirror target. Specify a mirror filter to capture inbound and outbound traffic. For the source of the mirror session, specify the EC2 elastic network interfaces for all the instances that host the application. Create a third-party EC2 appliance. Send all traffic to the appliance through the Kinesis Data Firehose delivery stream for content

inspection.

D. Configure VPC flow logs on each EC2 network interface. Send the logs to Amazon CloudWatch. Create a third-party EC2 appliance. Configure a CloudWatch filter to send the flow logs to Amazon Kinesis Data Firehose to load the logs into the appliance.

Answer: B

Explanation:

The correct answer is B because it provides a real-time, in-line traffic inspection solution that directly addresses the requirement of sending all network traffic to a third-party appliance. Here's a detailed justification:

Traffic Mirroring: Traffic Mirroring is an AWS feature that allows you to copy network traffic from EC2 instances. This is a fundamental requirement for sending traffic to a third-party appliance for inspection without disrupting the original traffic flow. <https://docs.aws.amazon.com/vpc/latest/mirroring/traffic-mirroring-getting-started.html>

NLB as Mirror Target: A Network Load Balancer (NLB) is suitable as a mirror target because it can efficiently distribute the mirrored traffic to a fleet of inspection appliances. This provides scalability and high availability for the inspection process.

Auto Scaling Group for Appliance Fleet: Placing the third-party appliance within an Auto Scaling group ensures that the inspection capacity can scale dynamically with the traffic volume. This is crucial for maintaining performance and availability.

Mirror Filters: Mirror filters allow you to selectively capture the traffic of interest, in this case, inbound and outbound traffic from the specified EC2 instances. This helps minimize the volume of mirrored traffic and optimize resource utilization.

EC2 Elastic Network Interfaces (ENIs) as Source: Specifying the ENIs of the EC2 instances as the source of the mirror session ensures that all traffic to and from those instances is captured.

Option A is incorrect because VPC Flow Logs capture metadata about network traffic, not the actual content. Flow Logs are useful for auditing and troubleshooting but not for content inspection. It provides information about the traffic, not the traffic itself. <https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs.html>

Option C is incorrect because although it uses Traffic Mirroring, sending all traffic to an appliance through Kinesis Data Firehose is not the intended use case for the service and can be overly complex and costly. Kinesis Data Firehose is designed for streaming data, not mirroring entire network packets for real-time inspection.

Option D also uses VPC Flow Logs, which as previously stated captures metadata about network traffic and not the actual content, this is unsuitable for content inspection. Using CloudWatch filters and Kinesis Data Firehose to load the logs into the appliance might lead to latency and complexity, and also does not provide access to the content.

Question: 144

CertyIQ

A company has two AWS Direct Connect links. One Direct Connect link terminates in the us-east-1 Region, and the other Direct Connect link terminates in the af-south-1 Region. The company is using BGP to exchange routes with AWS.

How should a network engineer configure BGP to ensure that af-south-1 is used as a secondary link to AWS?

- On the Direct Connect link to us-east-1, configure BGP peering to use community tag 7224:7100
- On the Direct Connect link to af-south-1, configure BGP peering to use community tag 7224:7300
- On the Direct Connect BGP peer to us-east-1, set the local preference value to 200
- On the Direct Connect BGP peer to af-south-1, set the local preference value to 50

- B. On the Direct Connect link to us-east-1, configure BGP peering to use community tag 7224:7300
- On the Direct Connect link to af-south-1, configure BGP peering to use community tag 7224:7100
- On the Direct Connect BGP peer to us-east-1, set the local preference value to 200
- On the Direct Connect BGP peer to af-south-1, set the local preference value to 50
- C. On the Direct Connect link to us-east-1, configure BGP peering to use community tag 7224:7100
- On the Direct Connect link to af-south-1, configure BGP peering to use community tag 7224:7300
- On the Direct Connect BGP peer to us-east-1, set the local preference value to 50
- On the Direct Connect BGP peer to af-south-1, set the local preference value to 200
- D. On the Direct Connect link to us-east-1, configure BGP peering to use community tag 7224:7300
- On the Direct Connect link to af-south-1, configure BGP peering to use community tag 7224:7100
- On the Direct Connect BGP peer to us-east-1, set the local preference value to 50
- On the Direct Connect BGP peer to af-south-1, set the local preference value to 200

Answer: B

Explanation:

The correct answer is B because it leverages both BGP community tags and local preference to influence route selection across multiple Direct Connect links. To make us-east-1 the primary and af-south-1 the secondary link, you need to signal to AWS which paths are preferred.

BGP community tags are used to influence AWS routing decisions. The 7224:7300 community tag indicates a preference for the path within the same continent, while 7224:7100 indicates a lower preference. By applying 7224:7300 to the us-east-1 link (North America) and 7224:7100 to the af-south-1 link (Africa), you're telling AWS to prefer routes learned via the us-east-1 link within the AWS network.

Local preference is used to influence the outbound traffic from your network. Setting a higher local preference on the us-east-1 Direct Connect peer (200) and a lower local preference on the af-south-1 Direct Connect peer (50) ensures that your network prefers the us-east-1 link when sending traffic to AWS. This forces the return traffic to use the us-east-1 Direct Connect connection as much as possible.

Combining these two mechanisms provides a robust solution: community tags influence inbound traffic from AWS, and local preference influences outbound traffic to AWS. Alternatives A, C, and D do not correctly configure both community tags and local preference to achieve the desired primary/secondary link behavior, particularly in influencing outbound traffic flow.

For more information, refer to the AWS documentation on Direct Connect routing policies:

<https://docs.aws.amazon.com/directconnect/latest/UserGuide/routing-policies.html> and
<https://docs.aws.amazon.com/directconnect/latest/UserGuide/dx-community-tags.html>

Question: 145

CertyIQ

A team of infrastructure engineers wants to automate the deployment of Application Load Balancer (ALB) components by using the AWS Cloud Development Kit (AWS CDK). The CDK application must deploy an infrastructure stack that is reusable and consistent across multiple environments, AWS Regions, and AWS accounts.

The lead network architect on the project has already bootstrapped the target accounts. The lead network architect also has deployed core network components such as VPCs and Amazon Route 53 private hosted zones across the multiple environments and Regions. The infrastructure engineers must design the ALB components in the CDK application to use the existing core network components.

Which combination of steps will meet this requirement with the LEAST manual effort between environment deployments? (Choose two.)

- A. Design the CDK application to read AWS CloudFormation parameters for the values that vary across environments and Regions. Reference these variables in the CDK stack for resources that require the variables.

B.Design the CDK application to read environment variables that contain account and Region details at runtime. Use these variables as properties of the CDK stack. Use context methods in the CDK stack to retrieve variable values.

C.Create a dedicated account for shared application services in the multi-account environment. Deploy a CDK pipeline to the dedicated account. Create stages in the pipeline that deploy the CDK application across different environments and Regions.

D.Write a script that automates the deployment of the CDK application across multiple environments and Regions. Distribute the script to engineers who are working on the project.

E.Use the CDK toolkit locally to deploy stacks to each environment and Region. Use the --context flag to pass in variables that the CDK application can reference at runtime.

Answer: BC

Explanation:

Let's break down why options B and C are the most appropriate for automating ALB deployments across multiple environments and Regions with minimal manual effort when using AWS CDK, and why the others are less ideal.

Option B leverages environment variables at runtime within the CDK application. This allows the CDK to adapt to the specific account and Region it's being deployed to without requiring hardcoding or manual input for each environment. These variables become properties of the CDK stack, making them easily accessible and reusable within the stack's logic. Context methods in CDK offer a mechanism to retrieve these variable values during synthesis. This approach promotes code reusability and reduces the potential for human error. For more on CDK context, refer to: <https://docs.aws.amazon.com/cdk/api/v2/docs/aws-construct-library.ContextProvider.html>.

Option C introduces a dedicated account for shared application services, including the CDK pipeline. A CDK pipeline automates the deployment process across different environments and Regions. By centralizing the pipeline in a dedicated account, you improve security and governance, making the deployment process repeatable and consistent. Stages within the pipeline define the deployment steps for each environment. This streamlines deployment and minimizes manual intervention, ensuring the ALB components are deployed uniformly across all environments. The pipeline will pick up the account/region information from its configuration or environment. This is often managed using AWS CodePipeline with CDK. Refer to AWS documentation on CDK pipelines: <https://docs.aws.amazon.com/cdk/api/v2/docs/aws-construct-library.pipelines.CodePipeline.html>

Option A suggests using CloudFormation parameters. While parameters can be useful, they typically require manual input or an external orchestration mechanism to provide values for each deployment. This increases manual effort compared to the automation offered by options B and C.

Option D suggests writing and distributing a script for deployment. While automation is good, this approach is less structured and maintainable than a CDK pipeline. Distributing scripts increases the risk of inconsistencies and makes updates more challenging.

Option E proposes using the CDK toolkit locally with the --context flag. This method requires manual execution of CDK commands for each environment and Region, significantly increasing manual effort and potential for error. It also doesn't scale well as the number of environments grows. It is not automated, failing the requirement in the question.

In summary, the combination of using environment variables within the CDK application (B) and a CDK pipeline in a dedicated account (C) provides the best balance of automation, consistency, and reusability for deploying ALB components across multiple environments and Regions with minimal manual effort.

Question: 146

A company has critical VPC workloads that connect to an on-premises data center through two redundant active-passive AWS Direct Connect connections. However, a recent outage on one Direct Connect connection revealed that it takes more than a minute for traffic to fail over to the secondary Direct Connect connection. The company wants to reduce the failover time from minutes to seconds.

Which solution will provide the LARGEST reduction in the BGP failover time?

- A.Reduce the BGP hold-down timer that is configured on the BGP sessions on the Direct Connect connection VIFs.
- B.Configure an Amazon CloudWatch alarm for the Direct Connect connection state to invoke an AWS Lambda function to fail over the traffic.
- C.Configure Bidirectional Forwarding Detection (BFD) on the Direct Connect connections on the AWS side.
- D.Configure Bidirectional Forwarding Detection (BFD) on the Direct Connect connections on the on-premises router.

Answer: D

Explanation:

The goal is to minimize BGP failover time for Direct Connect connections between AWS and an on-premises data center. Reducing BGP hold-down timers (Option A) can help, but has limitations. Aggressively reducing these timers can lead to flapping and instability if network conditions are slightly imperfect, resulting in more frequent disruptions. CloudWatch alarms and Lambda functions (Option B) introduce delay due to the time taken to detect, trigger, and execute the failover logic.

Bidirectional Forwarding Detection (BFD) offers the fastest failure detection. BFD is a low-overhead protocol designed specifically for rapid fault detection in routing protocols like BGP. By configuring BFD on both sides of the Direct Connect connection, specifically the on-premises router (Option D) and the AWS side, link failures can be detected within milliseconds, dramatically reducing BGP convergence time. Initiating BFD from the on-premises side allows for immediate detection of link failures from the customer's network perspective, enabling a quicker switch to the backup Direct Connect connection. Option C, configuring BFD only on the AWS side, would mean the on-premises router waits longer to detect an outage from AWS's perspective, therefore delaying failover. Therefore, configuring BFD on the on-premises router and AWS side is the most impactful method.

Configuring it on the on-premises side (option D) ensures the router directly monitoring its connection, triggering quicker rerouting compared to relying solely on AWS-side BFD detection. BFD will send out packets at a very small interval (sub-second) and if those packets are not seen from the other end, the route will be immediately withdrawn. This ensures failover occurs in sub-second time scales rather than waiting for BGP timers to expire which are significantly longer by default.

For further reading on BFD and Direct Connect, refer to AWS documentation and networking best practices:

[AWS Direct Connect Resiliency Recommendations](#)
[Bidirectional Forwarding Detection \(BFD\)](#)

Question: 147

A European car manufacturer wants to migrate its customer-facing services and its analytics platform from two on-premises data centers to the AWS Cloud. The company has a 50-mile (80.4 km) separation between its on-premises data centers and must maintain that separation between its two locations in the cloud. The company also needs failover capabilities between the two locations in the cloud.

The company's infrastructure team creates several accounts to separate workloads and responsibilities. The company provisions resources in the eu-west-3 Region and in the eu-central-1 Region. The company selects an

AWS Direct Connect Partner in each Region and requests two resilient 1 Gbps fiber connections from each provider.

The company's network engineer must establish a connection between all VPCs in the accounts and between the on-premises network and the AWS Cloud. The solution must provide access to all services in both Regions in case of network issues.

Which solution will meet these requirements?

A. Create a Direct Connect gateway. Create a private VIF on each of the Direct Connect connections. Attach the private VIFs to the Direct Connect gateway. Use equal-cost multi-path (ECMP) routing to aggregate the four connections across the two Regions. Attach the Direct Connect gateway directly to each VPC's virtual private gateway.

B. Create a Direct Connect gateway. Create a transit gateway. Attach the transit gateway to the Direct Connect gateway. Create a transit VIF on each of the Direct Connect connections. Attach the transit VIFs to the Direct Connect gateway. Use a link aggregation group (LAG) to aggregate the four connections across the two Regions. Attach the transit gateway directly to each VPC.

C. Create a Direct Connect gateway. Create a transit gateway in each Region. Attach the transit gateways to the Direct Connect gateway. Create a transit VIF on each of the Direct Connect connections. Attach the transit VIFs to the Direct Connect gateway. Peer the transit gateways. Attach the transit gateways in each Region to the VPCs in the same Region.

D. Create a Direct Connect gateway. Create a private VIF on each of the Direct Connect connections. Attach the private VIFs to the Direct Connect gateway. Use a link aggregation group (LAG) to aggregate the four connections across the two Regions. Create a transit gateway. Attach the transit gateway to the Direct Connect gateway. Attach the transit gateway directly to each VPC.

Answer: C

Explanation:

The correct answer is **C**. Here's why:

Direct Connect Gateway (DXGW): A DXGW is essential for connecting multiple VPCs in different AWS Regions to on-premises networks via Direct Connect. This is a central hub for managing Direct Connect connections across regions. <https://docs.aws.amazon.com/directconnect/latest/UserGuide/direct-connect-gateways-intro.html>

Transit Gateway (TGW): Using a TGW in each region is crucial because the company has VPCs in eu-west-3 and eu-central-1. A TGW simplifies connectivity between VPCs within a region and provides a single point of connection to the DXGW. <https://docs.aws.amazon.com/transit-gateway/latest/userguide/what-is-transit-gateway.html>

Transit VIFs: Transit VIFs are specifically designed for connecting Direct Connect connections to a Transit Gateway via a DXGW. Using transit VIFs enables connectivity from on-premises to all VPCs connected to the transit gateway.

Peering Transit Gateways: Peering the Transit Gateways in eu-west-3 and eu-central-1 allows traffic to flow between the VPCs in different regions through the transit gateway network. This ensures inter-region connectivity, which is a key requirement.

Regional Attachment: Attaching each regional TGW to the VPCs within that region makes sure that local traffic within the region stays within the region, optimizing performance and reducing cross-region data transfer costs.

Resilience and Failover: The two Direct Connect connections per region, coupled with the DXGW and regional TGW setup, provides redundancy. If one Direct Connect connection fails, traffic can failover to the other connection within the same region. The peering between transit gateways provides failover for cross-region connectivity in case one transit gateway has issues.

Options A, B, and D are incorrect for the following reasons:

Option A: Attaching the DXGW directly to VPC virtual private gateways (VGWs) is not the most scalable or manageable solution when dealing with multiple VPCs and Regions. It lacks the central routing capabilities of

a Transit Gateway. Also, ECMP won't aggregate bandwidth across regions with Direct Connect.

Option B: Using LAG across two different geographical AWS regions is not possible. LAGs require all circuits in the group to terminate in the same location. This is because LAGs provide link-level aggregation and require physical proximity.

Option D: Using LAG across two different geographical AWS regions is not possible. LAGs require all circuits in the group to terminate in the same location. This is because LAGs provide link-level aggregation and require physical proximity. Attaching the TGW only to the DXGW but not implementing inter-region peering means VPCs in different regions won't be able to communicate with each other through the TGW.

Question: 148

CertyIQ

A company wants to analyze TCP traffic to the internet. The traffic originates from Amazon EC2 instances in the company's VPC. The EC2 instances initiate connections through a NAT gateway. The required information includes source and destination IP addresses, ports, and the first 8 bytes of payload of TCP segments. The company needs to collect, store, and analyze all the required data points.

Which solution will meet these requirements?

- A. Set up the EC2 instances as VPC traffic mirror sources. Deploy software on the traffic mirror target to forward the data to Amazon CloudWatch Logs. Analyze the data by using CloudWatch Logs Insights.
- B. Set up the NAT gateway as a VPC traffic mirror source. Deploy software on the traffic mirror target to forward the data to an Amazon OpenSearch Service cluster. Analyze the data by using OpenSearch Dashboards.
- C. Turn on VPC Flow Logs on the EC2 instances. Specify the default format and a log destination of Amazon CloudWatch Logs. Analyze the flow log data by using CloudWatch Logs Insights.
- D. Turn on VPC Flow Logs on the EC2 instances. Specify a custom format and a log destination of Amazon S3. Analyze the flow log data by using Amazon Athena.

Answer: A

Explanation:

The correct answer is A because it utilizes VPC Traffic Mirroring to capture the detailed TCP traffic data needed, including the first 8 bytes of the payload. VPC Traffic Mirroring duplicates network traffic from EC2 instances (the source) and sends it to a target for inspection.

Option A sets the EC2 instances as the traffic mirror source, ensuring that traffic originating from them is captured. A software agent deployed on the traffic mirror target (another EC2 instance or a Network Load Balancer) can then extract the required data (source/destination IP, ports, first 8 bytes of payload) and forward it to CloudWatch Logs. CloudWatch Logs Insights can then be used to analyze the collected log data efficiently. CloudWatch Logs Insights allows you to query and analyze your log data with a purpose-built query language.

Option B is incorrect because mirroring from a NAT gateway won't capture the traffic from individual EC2 instances; it only shows traffic passing through the NAT gateway itself. Furthermore, forwarding data directly to OpenSearch from a traffic mirror target without proper parsing and formatting can lead to performance issues. OpenSearch Dashboards is well-suited for visualization, but initially needs the structured and refined data.

Option C and D are incorrect because VPC Flow Logs do not capture packet payload data. They provide information about the network traffic flows (source/destination IP, ports, number of bytes, etc.), but they don't expose the content of the packets. Therefore, Flow Logs are insufficient for extracting the first 8 bytes of the TCP segment payload. VPC Flow Logs capture metadata, while the problem statement requires capturing parts of the actual data payload.

Therefore, VPC Traffic Mirroring is the only option capable of capturing the payload data needed for analysis.

Furthermore, CloudWatch Logs is a cheaper and effective starting point compared to S3 and Athena in option D, as immediate data capture is necessary, and the data volume can be tested for optimal storage options.

Relevant links:

VPC Traffic Mirroring: <https://docs.aws.amazon.com/vpc/latest/mirroring/what-is-traffic-mirroring.html>

CloudWatch Logs Insights:

<https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/AnalyzingLogData.html>

VPC Flow Logs: <https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs.html>

Question: 149

CertyIQ

A company has three VPCs in a single AWS Region. Each VPC contains 15 Amazon EC2 instances, and no connectivity exists between the VPCs.

The company is deploying a new application across all three VPCs. The application requires high bandwidth between the nodes. A network engineer must implement connectivity between the VPCs.

Which solution will meet these requirements with the HIGHEST throughput?

- A. Configure a transit gateway. Attach each VPC to the transit gateway. Configure static routing in each VPC to route traffic to the transit gateway.
- B. Configure VPC peering between the three VPCs. Configure static routing to route traffic between the three VPCs.
- C. Configure a transit VPC. Configure a VPN gateway in each VPC. Create an AWS Site-to-Site VPN tunnel from each VPC to the transit VPC. Use BGP routing to route traffic between the VPCs and the transit VPC.
- D. Configure AWS Site-to-Site VPN connections between each VPC. Enable route propagation for each Site-to-Site VPN connection to route traffic between the VPCs.

Answer: B

Explanation:

The correct answer is B: Configure VPC peering between the three VPCs. Configure static routing to route traffic between the three VPCs.

Here's why:

High Throughput: VPC peering provides the highest throughput and lowest latency because traffic stays within the AWS network. It essentially creates direct network connections between VPCs without going through intermediaries like VPN gateways or transit gateways.

Simplicity: For a small number of VPCs (in this case, three), VPC peering is straightforward to set up and manage. The configuration primarily involves establishing the peering connections and updating route tables.

Direct Routing: Static routing allows you to explicitly define the network paths between the peered VPCs. This is suitable for a small, static network topology.

Why other options are not optimal:

A. Transit Gateway: Transit Gateway is more suitable for larger, more complex networks with many VPCs and potentially on-premises connections. While it offers centralized management, it introduces an extra hop and may not provide the absolute highest throughput compared to peering for only three VPCs.

<https://aws.amazon.com/transit-gateway/>

C. Transit VPC with VPN: This solution involves creating a central "transit" VPC and establishing VPN connections from each VPC to it. VPN introduces overhead and reduces throughput compared to native AWS connections like peering. Furthermore, managing VPN tunnels and BGP routing adds complexity.

D. Site-to-Site VPN between VPCs: Similar to option C, Site-to-Site VPN connections introduce overhead and reduce throughput. VPN connections are generally used for connecting to on-premises networks or regions and not for high-bandwidth, low-latency connectivity within the same region.

In summary, VPC peering offers the simplest, most direct, and highest-throughput solution for connecting a small number of VPCs in the same region, meeting the specific requirements of the prompt.

<https://aws.amazon.com/vpc/features/vpc-peering/>

Question: 150

CertyIQ

A network engineer needs to deploy an AWS Network Firewall firewall into an existing AWS environment. The environment consists of the following:

- A transit gateway with all VPCs attached to it
- Several hundred application VPCs
- A centralized egress internet VPC with a NAT gateway and an internet gateway
- A centralized ingress internet VPC that hosts public Application Load Balancers
- On-premises connectivity through an AWS Direct Connect gateway attachment

The application VPCs have workloads deployed across multiple Availability Zones in private subnets with the VPC route table's default route (0.0.0.0/0) pointing to the transit gateway. The Network Firewall firewall needs to inspect east-west (VPC-to-VPC) traffic and north-south (internet-bound and on-premises network) traffic by using Suricata compatible rules.

The network engineer must deploy the firewall by using a solution that requires the least possible architectural changes to the existing production environment.

Which combination of steps should the network engineer take to meet these requirements? (Choose three.)

- A. Deploy Network Firewall in all Availability Zones in each application VPC.
- B. Deploy Network Firewall in all Availability Zones in a centralized inspection VPC.
- C. Update the HOME_NET rule group variable to include all CIDR ranges of the VPCs and on-premises networks.
- D. Update the EXTERNAL_NET rule group variable to include all CIDR ranges of the VPCs and on-premises networks.
- E. Configure a single transit gateway route table. Associate all application VPCs and the centralized inspection VPC with this route table.
- F. Configure two transit gateway route tables. Associate all application VPCs with one transit gateway route table. Associate the centralized inspection VPC with the other transit gateway route table.

Answer: BCF

Explanation:

The correct answer is BCF. Let's break down why:

B - Deploy Network Firewall in all Availability Zones in a centralized inspection VPC: Deploying Network Firewall in a centralized inspection VPC is the most efficient and cost-effective approach for inspecting traffic across multiple VPCs and connectivity options. This avoids deploying and managing firewalls in each application VPC (option A), which would be an operational overhead nightmare given the scale of hundreds of VPCs. Network Firewall is designed for centralized deployment patterns. <https://aws.amazon.com/blogs/networking-and-content-delivery/centralized-inspection-architecture-with-aws-network-firewall-and-aws-transit-gateway/>

C - Update the HOME_NET rule group variable to include all CIDR ranges of the VPCs and on-premises networks: Network Firewall uses Suricata-compatible rules. These rules often rely on variables like HOME_NET and EXTERNAL_NET to define the scope of traffic being inspected. Since the firewall needs to

inspect both east-west (VPC-to-VPC) and north-south (internet/on-premises) traffic, HOME_NET needs to be updated to include the CIDR ranges of all VPCs and the on-premises network to accurately define the internal network.<https://docs.aws.amazon.com/network-firewall/latest/developerguide/rule-group-properties.html>

F - Configure two transit gateway route tables. Associate all application VPCs with one transit gateway route table. Associate the centralized inspection VPC with the other transit gateway route table: Using two transit gateway route tables allows for traffic steering through the centralized inspection VPC. The application VPCs are associated with one route table that directs traffic destined for other VPCs and on-premises networks to the inspection VPC. The inspection VPC is associated with the other route table, ensuring traffic flows through the Network Firewall before reaching its final destination. This ensures that east-west and north-south traffic is inspected. Option E using only one TGW route table wouldn't allow you to inspect both East-West and North-South traffic. D is incorrect, you will not want your external network to include your internal private VPCs CIDR ranges.

In summary, a centralized inspection VPC with Network Firewall, combined with proper routing using Transit Gateway route tables and accurate Suricata rule definitions (specifically updating HOME_NET), is the most efficient and scalable solution for this scenario.

Question: 151

CertyIQ

A company is using a shared services VPC with two domain controllers. The domain controllers are deployed in the company's private subnets. The company is deploying a new application into a new VPC in the account. The application will be deployed onto an Amazon EC2 for Windows Server instance in the new VPC. The instance must join the existing Windows domain that is supported by the domain controllers in the shared services VPC.

A transit gateway is attached to both the shared services VPC and the new VPC. The company has updated the route tables for the transit gateway, the shared services VPC, and the new VPC. The security groups for the domain controllers and the instance are updated and allow traffic only on the ports that are necessary for domain operations. The instance is unable to join the domain that is hosted on the domain controllers.

Which combination of actions will help identify the cause of this issue with the LEAST operational overhead? (Choose two.)

- A. Use AWS Network Manager to perform a route analysis for the transit gateway network. Specify the existing EC2 instance as the source. Specify the first domain controller as the destination. Repeat the route analysis for the second domain controller.
- B. Use port mirroring with the existing EC2 instance as the source and another EC2 instance as the target to obtain packet captures of the connection attempts.
- C. Review the VPC flow logs on the shared services VPC and the new VPC.
- D. Issue a ping command from one of the domain controllers to the existing EC2 instance.
- E. Ensure that route propagation is turned off on the shared services VPC.

Answer: AC

Explanation:

The correct answer is **AC**. Here's why:

Option A: Use AWS Network Manager for Route Analysis: AWS Network Manager provides a centralized view of your global network, including transit gateways. The route analysis feature allows you to simulate traffic flow between resources and identify routing misconfigurations or black holes in your network. By specifying the EC2 instance and domain controllers as source and destination, you can verify the traffic path and rule out routing issues within the transit gateway and associated VPCs. This is a low-impact method for troubleshooting network connectivity. <https://aws.amazon.com/network-manager/>

Option C: Review VPC Flow Logs: VPC Flow Logs capture information about the IP traffic going to, from, and within your VPCs. By reviewing the flow logs for both the shared services VPC and the new VPC, you can identify if traffic is being dropped due to security group rules, network ACLs, or routing configurations. Flow logs provide valuable insights into the allowed and denied traffic, helping pinpoint the point of failure.

<https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs.html>

Here's why the other options are less ideal:

Option B: Port Mirroring: While port mirroring can capture detailed packet information, it involves more operational overhead than route analysis or flow logs. It requires setting up mirroring sessions, configuring a target instance, and analyzing packet captures, which can be time-consuming and resource-intensive.

Option D: Ping Command: A simple ping command only verifies basic network reachability (ICMP), but it does not ensure that traffic on the necessary ports for domain operations (Kerberos, LDAP, etc.) is allowed. The problem may lie in those specific ports.

Option E: Ensure that route propagation is turned off on the shared services VPC: Turning off route propagation is the opposite of what is needed. With route propagation on the shared services VPC, the routes from other resources can be automatically updated to allow traffic. Turning it off would break all established connections.

Therefore, using AWS Network Manager route analysis and reviewing VPC flow logs provides a quicker and less operationally burdensome method for identifying the cause of the connectivity issue compared to the alternative actions, allowing for faster and more targeted troubleshooting.

Question: 152

CertyIQ

A company has an order processing system that needs to keep credit card numbers encrypted. The company's customer-facing application runs as an Amazon Elastic Container Service (Amazon ECS) service behind an Application Load Balancer (ALB) in the us-west-2 Region. An Amazon CloudFront distribution is configured with the ALB as the origin. The company uses a third-party trusted certificate authority to provision its certificates.

The company is using HTTPS for encryption in transit. The company needs additional field-level encryption to keep sensitive data encrypted during processing so that only certain application components can decrypt the sensitive data.

Which combination of steps will meet these requirements? (Choose two.)

- A.Import the third-party certificate for the ALB. Associate the certificate with the ALB. Upload the certificate for the CloudFront distribution into AWS Certificate Manager (ACM) in us-west-2.
- B.Import the third-party certificate for the ALB into AWS Certificate Manager (ACM) in us-west-2. Associate the certificate with the ALB. Upload the certificate for the CloudFront distribution into ACM in the us-east-1 Region.
- C.Upload the private key that handles the encryption of the sensitive data to the CloudFront distribution. Create a field-level encryption profile and specify the fields that contain sensitive information. Create a field-level encryption configuration, and choose the newly created profile. Link the configuration to the appropriate cache behavior that is associated with sensitive POST requests.
- D.Upload the public key that handles the encryption of the sensitive data to the CloudFront distribution. Create a field-level encryption configuration, and specify the fields that contain sensitive information. Create a field-level encryption profile, and choose the newly created configuration. Link the profile to the appropriate cache behavior that is associated with sensitive GET requests.
- E.Upload the public key that handles the encryption of the sensitive data to the CloudFront distribution. Create a field-level encryption profile and specify the fields that contain sensitive information. Create a field-level encryption configuration, and choose the newly created profile. Link the configuration to the appropriate cache behavior that is associated with sensitive POST requests.

Answer: BE

Explanation:

The correct answer is BE because it addresses both the general HTTPS encryption and the specific field-level encryption requirements.

Option B: This option correctly addresses the certificate requirements.

Importing the certificate into ACM (us-west-2) for the ALB: The ALB needs a certificate to handle HTTPS traffic. Importing the existing third-party certificate into ACM allows the ALB to use it. ACM is the AWS service for managing certificates. This is region specific to where the ALB is located.

Uploading the certificate into ACM (us-east-1) for CloudFront: CloudFront requires certificates in the us-east-1 region (N. Virginia) regardless of where the CloudFront distribution or its origins are located. This is a global service requirement.

Option E: This option correctly configures field-level encryption in CloudFront.

Upload the public key: Field-level encryption uses asymmetric encryption. The public key is uploaded to CloudFront and used to encrypt sensitive data at the edge. Only someone with the corresponding private key can decrypt the data, ensuring confidentiality.

Create a field-level encryption profile: This profile specifies which public key will be used to encrypt which fields in the request.

Create a field-level encryption configuration: This configuration ties the profile to the cache behavior.

Link the configuration to the appropriate cache behavior (POST requests): Field-level encryption is typically used for POST requests because these requests often contain sensitive data.

Why other options are incorrect:

Option A: Placing the certificate in us-west-2 for CloudFront is incorrect; it must reside in us-east-1.

Option C: Uploading the private key to CloudFront is a security risk. The private key should only reside on servers that need to decrypt the data, not at the edge. Also reverses the order of profile creation.

Option D: Reverses the order of configuration/profile creation, also uses GET requests incorrectly. GET requests are typically for retrieving data, not transmitting sensitive information.

Supporting concepts:

HTTPS: Ensures secure communication over the internet by encrypting data in transit.

ACM: AWS Certificate Manager simplifies the process of provisioning, managing, and deploying SSL/TLS certificates for use with AWS services.

CloudFront: A content delivery network (CDN) that caches content closer to users, improving performance.

Field-Level Encryption: Adds a further layer of security by encrypting specific data fields within an HTTPS request.

Asymmetric Encryption: Uses a pair of keys (public and private) for encryption and decryption. The public key can encrypt data, but only the private key can decrypt it.

Authoritative Links:

[AWS Certificate Manager FAQs](#)

[CloudFront Field-Level Encryption](#)

Question: 153**CertyIQ**

A company has deployed a multi-VPC environment in the AWS Cloud. The company uses a transit gateway to connect all the VPCs together. In the past, the company has experienced a loss of connectivity between applications after changes to security groups, network ACLs, and route tables in a VPC. When these changes occur, the company wants to automatically verify that connectivity still exists between different resources in a single VPC.

A. Create a list of paths between different resources to check in VPC Reachability Analyzer. Create an Amazon EventBridge rule to monitor when a change is made and logged in Amazon CloudWatch. Configure the rule to invoke an AWS Lambda function to test the different paths in Reachability Analyzer.

B. Create a list of paths between different resources to check in VPC Reachability Analyzer. Create an Amazon EventBridge rule to monitor when a change is made and logged in AWS CloudTrail. Configure the rule to invoke an AWS Lambda function to test the different paths in Reachability Analyzer.

C. Create a list of paths to check in AWS Transit Gateway Network Manager Route Analyzer. Create an Amazon EventBridge rule to monitor when a change is made and logged in Amazon CloudWatch. Configure the rule to invoke an AWS Lambda function to test the different paths in Route Analyzer.

D. Create a list of paths to check in AWS Transit Gateway Network Manager Route Analyzer. Create an Amazon EventBridge rule to monitor when a change is made and logged in AWS CloudTrail. Configure the rule to invoke an AWS Lambda function to test the different paths in Route Analyzer.

Answer: B

Explanation:

The correct answer is **B**. Here's why:

The requirement is to automatically verify intra-VPC connectivity after changes. VPC Reachability Analyzer is the appropriate tool for this task as it specifically analyzes network configurations within a VPC to determine reachability between resources, considering security groups, NACLs, and route tables.

[<https://docs.aws.amazon.com/vpc/latest/reachability/what-is-reachability-analyzer.html>]

Options C and D use Transit Gateway Network Manager Route Analyzer. This tool is designed for analyzing connectivity issues across transit gateways, not within a single VPC.

[<https://docs.aws.amazon.com/vpc/latest/tgw/tgw-network-manager.html>]

To trigger the analysis automatically after configuration changes, AWS CloudTrail is the correct service to monitor. CloudTrail logs API calls and events related to resource changes in your AWS account, providing an audit trail of modifications to security groups, NACLs, and route tables.

[<https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-user-guide.html>]

Amazon CloudWatch logs, as mentioned in options A and C, primarily collect and monitor log data generated by applications and services, not necessarily the API events indicating configuration changes. While you can monitor CloudWatch logs for specific patterns, CloudTrail directly captures the desired configuration changes as API calls.

Amazon EventBridge is used to create a rule that triggers an AWS Lambda function whenever CloudTrail logs a relevant event (e.g., a change to a security group). The Lambda function then executes the VPC Reachability Analyzer, testing the predefined paths between resources. This automated process allows for near real-time verification of connectivity after changes.

In summary, Option B provides the most effective and suitable solution by utilizing VPC Reachability Analyzer for intra-VPC analysis, CloudTrail for change detection, EventBridge for event-driven automation, and Lambda to orchestrate the verification process.

Question: 154

CertyIQ

A company hosts a web application that runs on a fleet of Amazon EC2 instances behind an Application Load Balancer (ALB). The instances are in an Auto Scaling group. The company uses an Amazon CloudFront distribution with the ALB as an origin.

The application recently experienced an attack. In response, the company associated an AWS WAF web ACL with the CloudFront distribution. The company needs to use Amazon Athena to analyze application attacks that AWS WAF detects.

Which solution will meet this requirement?

- A. Configure the ALB and the EC2 instance subnets to produce VPC flow logs. Configure the VPC flow logs to deliver logs to an Amazon S3 bucket for log analysis.
- B. Create a trail in AWS CloudTrail to capture data events. Configure the trail to deliver logs to an Amazon S3 bucket for log analysis.
- C. Configure the AWS WAF web ACL to deliver logs to an Amazon Kinesis Data Firehose delivery stream. Configure the stream to deliver the data to an Amazon S3 bucket for log analysis.
- D. Turn on access logging for the ALB. Configure the access logs to deliver the logs to an Amazon S3 bucket for log analysis.

Answer: C

Explanation:

The correct answer is C because AWS WAF directly integrates with Amazon Kinesis Data Firehose to provide detailed logs of detected attacks. These logs are essential for analysis using Amazon Athena.

Here's a detailed breakdown:

AWS WAF Logging: AWS WAF is designed to protect web applications from common web exploits. It can detect and block malicious requests. To analyze these detected attacks, you need access to the WAF logs.

Kinesis Data Firehose for WAF Logs: AWS WAF can directly stream its logs to a Kinesis Data Firehose delivery stream. This stream acts as a conduit, allowing you to deliver the data to various destinations, including Amazon S3.

Amazon S3 as a Log Repository: Amazon S3 is a scalable and cost-effective storage service. Storing WAF logs in S3 allows you to retain the logs for analysis.

Amazon Athena for Analysis: Athena is a serverless query service that allows you to analyze data stored in S3 using SQL. You can point Athena at the S3 bucket containing the WAF logs and then use SQL queries to identify patterns, sources of attacks, and other valuable insights.

Why other options are incorrect:

A: VPC Flow Logs: VPC Flow Logs capture network traffic information within your VPC. While they can be useful for network monitoring and security analysis, they don't contain the specific details of the web application attacks that AWS WAF detects. They are not tied to the AWS WAF rules and detections.

B: CloudTrail Data Events: CloudTrail data events track data plane operations (e.g., S3 object access, Lambda function invocation). While potentially useful for auditing certain aspects of your application, CloudTrail doesn't directly capture the specific attack information detected by AWS WAF.

D: ALB Access Logs: ALB access logs provide information about requests made to the ALB, such as client IP, request path, and response codes. They don't contain detailed attack information specific to the rules configured in your AWS WAF web ACL. While helpful for troubleshooting and monitoring ALB traffic, they don't provide the required information to analyze application attacks that AWS WAF has detected.

In summary, option C utilizes the direct integration between AWS WAF and Kinesis Data Firehose, allowing the attack data to be delivered to S3 for analysis with Amazon Athena. This provides the necessary insight into detected application attacks.

Authoritative Links:

AWS WAF Logging: <https://docs.aws.amazon.com/waf/latest/developerguide/logging.html>

Amazon Kinesis Data Firehose: <https://aws.amazon.com/kinesis/data-firehose/>

Amazon Athena: <https://aws.amazon.com/athena/>

Question: 155

A real estate company is using Amazon Workspaces to provide corporate managed desktop service to its real estate agents around the world. These Workspaces are deployed in seven VPCs. Each VPC is in a different AWS Region.

According to a new requirement, the company's cloud-hosted security information and events management (SIEM) system needs to analyze DNS queries generated by the Workspaces to identify the target domains that are connected to the Workspaces. The SIEM system supports poll and push methods for data and log collection.

Which solution should a network engineer implement to meet these requirements MOST cost-effectively?

- A. Create VPC flow logs in each VPC that is connected to the Workspaces instances. Publish the log data to a central Amazon S3 bucket. Configure the SIEM system to poll the S3 bucket periodically.
- B. Configure an Amazon CloudWatch agent to log all DNS requests in Amazon CloudWatch Logs. Configure a subscription filter in CloudWatch Logs. Push the logs to the SIEM system by using Amazon Kinesis Data Firehose.
- C. Configure VPC Traffic Mirroring to copy network traffic from each Workspace and to send the traffic to the SIEM system probes for analysis.
- D. Configure Amazon Route 53 query logging. Set the destination as an Amazon Kinesis Data Firehose delivery stream that is configured to push data to the SIEM system.

Answer: D

Explanation:

The most cost-effective solution to analyze DNS queries from Workspaces across multiple regions for a SIEM system is to use Amazon Route 53 query logging and push the logs to the SIEM using Amazon Kinesis Data Firehose.

Option D is superior because it directly addresses the requirement of analyzing DNS queries. Route 53 query logging captures the DNS queries made by the Workspaces. By configuring the destination as a Kinesis Data Firehose delivery stream, the data can be efficiently streamed to the SIEM system. This method avoids the overhead and cost associated with VPC Flow Logs which capture all network traffic.

Option A, using VPC Flow Logs, captures all network traffic, not just DNS queries, which increases the volume of data and thus increases costs for storage and analysis. Polling an S3 bucket is also less efficient and timely compared to the push mechanism of Kinesis Data Firehose.

Option B, while seemingly capturing DNS requests, is less optimal. Configuring a CloudWatch agent within each Workspace instance introduces management complexity and overhead within each virtual desktop. The CloudWatch Logs + Kinesis Data Firehose pathway is also a more involved setup compared to Route 53 query logging.

Option C, using VPC Traffic Mirroring, duplicates all network traffic from each Workspace. This is extremely costly and resource-intensive, providing far more data than required (only DNS queries are needed) and demanding significant resources for the SIEM system probes. It's also complex to manage across multiple regions and VPCs.

Route 53 query logging provides a direct, targeted, and cost-effective approach to collecting and streaming DNS query data to the SIEM system.

Relevant resources for further research:

Amazon Route 53 Query Logging: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/query-logs.html>

Amazon Kinesis Data Firehose: <https://aws.amazon.com/kinesis/data-firehose/>

Question: 156

A network engineer needs to design the architecture for a high performance computing (HPC) workload. Amazon EC2 instances will require 10 Gbps flows and an aggregate throughput of up to 100 Gbps across many instances with low-latency communication.

Which architecture solution will optimize this workload?

- A. Place nodes in a single subnet of a VPC. Configure a cluster placement group. Ensure that the latest Elastic Fabric Adapter (EFA) drivers are installed on the EC2 instances with a supported operating system.
- B. Place nodes in multiple subnets in a single VPC. Configure a spread placement group. Ensure that the EC2 instances support Elastic Network Adapters (ENAs) and that the drivers are updated on each instance operating system.
- C. Place nodes in multiple VPCs. Use AWS Transit Gateway to route traffic between the VPCs. Ensure that the latest Elastic Fabric Adapter (EFA) drivers are installed on the EC2 instances with a supported operating system.
- D. Place nodes in multiple subnets in multiple Availability Zones. Configure a cluster placement group. Ensure that the EC2 instances support Elastic Network Adapters (ENAs) and that the drivers are updated on each instance operating system.

Answer: A

Explanation:

The correct answer is A because it leverages key AWS services and configurations optimized for high-performance computing (HPC) workloads requiring low latency and high bandwidth. Let's break down why other options are less ideal:

B. Place nodes in multiple subnets in a single VPC. Configure a spread placement group. Spread placement groups are designed for maximizing instance distribution across hardware, which prioritizes fault tolerance over low-latency communication. They don't offer the network performance benefits crucial for HPC.

C. Place nodes in multiple VPCs. Use AWS Transit Gateway to route traffic between the VPCs. Using multiple VPCs and AWS Transit Gateway introduces additional latency and complexity. While Transit Gateway is useful for interconnecting VPCs, it's not designed for the low-latency, high-bandwidth requirements of HPC workloads.

D. Place nodes in multiple subnets in multiple Availability Zones. Configure a cluster placement group. Spanning multiple Availability Zones inherently adds latency due to the physical distance between them. While cluster placement groups improve proximity within an AZ, the inter-AZ communication would still be a bottleneck. Using ENAs without EFAs will not maximize throughput and minimize latency for HPC use cases.

Justification for A:

Placing all nodes within a single subnet eliminates the need for inter-subnet routing within the VPC, minimizing latency. Cluster placement groups are designed specifically to reduce network latency and increase network throughput for tightly coupled workloads by placing instances close together in the same physical rack. Elastic Fabric Adapter (EFA) is a network interface designed for HPC and machine learning workloads. It allows EC2 instances to communicate directly with each other using a low-latency, high-bandwidth interface, bypassing the operating system's kernel networking stack for MPI or NCCL. Ensuring EFA drivers are updated is critical for optimal performance. EFAs operate best within a single subnet to ensure low-latency communication between instances.

Therefore, placing nodes in a single subnet, utilizing a cluster placement group, and ensuring the latest EFA drivers are installed provides the optimal architecture for an HPC workload demanding 10 Gbps flows and an aggregate throughput of up to 100 Gbps with low-latency communication.

Supporting Links:

Question: 157



A company uses multiple AWS accounts and VPCs in a single AWS Region. The company must log all network traffic for Amazon EC2 instances and Amazon RDS databases. The company will use the log information to monitor and identify traffic flows in the event of a security incident. The information must be retained for 12 months but will be accessed infrequently after the first 90 days. The company must be able to view metadata that includes the vpc-id, subnet-id, and tcp-flags fields.

Which solution will meet these requirements at the LOWEST cost?

- A. Configure VPC flow logs with the default fields. Store the logs in Amazon CloudWatch Logs.
- B. Configure Traffic Mirroring on all AWS resources to point to a Network Load Balancer that will send the mirrored traffic to monitoring instances.
- C. Configure VPC flow logs with additional custom format fields. Store the logs in Amazon S3.
- D. Configure VPC flow logs with additional custom format fields. Store the logs in Amazon CloudWatch Logs.

Answer: C

Explanation:

Here's a detailed justification for why option C is the best solution for the network traffic logging requirements, considering cost and functionality:

Why C is the Best Option (VPC Flow Logs to S3 with Custom Fields)

VPC Flow Logs: VPC Flow Logs provide metadata about network traffic flowing through VPCs, subnets, or network interfaces, which aligns perfectly with the company's need to monitor traffic flows from EC2 instances and RDS databases.

Custom Fields: The requirement to view vpc-id, subnet-id, and tcp-flags necessitates the use of custom format fields in VPC Flow Logs. The default fields don't include all the required information.

Amazon S3 Storage: S3 is the most cost-effective option for long-term storage, especially for infrequently accessed data after the initial 90 days. S3 offers storage classes like S3 Standard-Infrequent Access (S3 Standard-IA) and S3 Glacier, which are designed for lower storage costs for less frequently accessed data.

Cost Optimization: CloudWatch Logs can become expensive for long-term storage of large volumes of data, as needed here. S3's tiered storage approach provides significant cost savings for archival data.

Compliance: S3 provides robust features for data retention and compliance, like S3 Object Lock, enabling adherence to the 12-month retention requirement.

Why Other Options Are Suboptimal

Option A (VPC Flow Logs to CloudWatch with Default Fields): Fails because the default fields don't include all necessary metadata (vpc-id, subnet-id, tcp-flags), not meeting the functional requirement. Additionally, CloudWatch Logs can become expensive for long-term storage.

Option B (Traffic Mirroring to Monitoring Instances): Traffic Mirroring replicates network traffic, placing a significant burden on network resources. It also requires provisioning and managing monitoring instances, adding to operational overhead and cost. This method is overly complex and costly compared to VPC Flow Logs.

Option D (VPC Flow Logs to CloudWatch with Custom Fields): Meets the metadata requirement but is more expensive than storing the logs in S3, especially for long-term storage and infrequent access after 90 days.

In conclusion, VPC Flow Logs with custom format fields, stored in Amazon S3, provides the required functionality (metadata capture) at the lowest cost, especially considering the long-term retention and

infrequent access after the initial 90 days. S3's storage tiers and compliance features make it the ideal choice for this scenario.

Authoritative Links:

VPC Flow Logs: <https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs.html>

Amazon S3 Storage Classes: <https://aws.amazon.com/s3/storage-classes/>

Traffic Mirroring: <https://docs.aws.amazon.com/vpc/latest/mirroring/what-is-traffic-mirroring.html>

Question: 158

CertyIQ

A network engineer is evaluating a network setup for a global retail company. The company has an AWS Direct Connect connection between its on-premises data center and the AWS Cloud. The company has AWS resources in the eu-west-2 Region. These resources consist of multiple VPCs that are attached to a transit gateway.

The company recently provisioned a few AWS resources in the eu-central-1 Region in a single VPC close to its users in this area. The network engineer must connect the resources in eu-central-1 with the on-premises data center and the resources in eu-west-2. The solution must minimize changes to the Direct Connect connection.

What should the network engineer do to meet these requirements?

- A. Create a new virtual private gateway. Attach the new virtual private gateway to the VPC in eu-central-1. Use a transit VIF to connect the VPC and the Direct Connect router.
- B. Create a new transit gateway in eu-central-1. Create a peering attachment request to the transit gateway in eu-west-2. Add a static route in the transit gateway route table in eu-central-1 to point to the transit gateway peering attachment. Accept the peering request. Add a static route in the transit gateway route table in eu-west-2 to point to the new transit gateway peering attachment.
- C. Create a new transit gateway in eu-central-1. Use an AWS Site-to-Site VPN connection to peer both transit gateways. Add a static route in the transit gateway route table in eu-central-1 to point to the transit gateway VPN attachment. Add a static route in the transit gateway route table in eu-west-2 to point to the new transit gateway peering attachment.
- D. Create a new virtual private gateway. Attach the new virtual private gateway to the VPC in eu-central-1. Use a public VIF to connect the VPC and the Direct Connect router.

Answer: B

Explanation:

The correct answer is **B**. Here's a detailed justification:

The company needs connectivity between the on-premises data center, the eu-west-2 Region (connected via Direct Connect), and the newly provisioned eu-central-1 Region. The requirement is to minimize changes to the existing Direct Connect connection. Transit Gateway peering is the most suitable solution for connecting regions in a hub-and-spoke manner without modifying the existing Direct Connect setup.

Option B proposes creating a new Transit Gateway in eu-central-1. This new Transit Gateway will connect to the VPC in eu-central-1. Then, a Transit Gateway peering attachment is created between the Transit Gateway in eu-central-1 and the existing Transit Gateway in eu-west-2. By creating peering attachments, the two transit gateways can route traffic between the two regions. The static routes ensure that traffic knows where to go. This solution efficiently uses the existing Direct Connect connection to eu-west-2 to indirectly reach eu-central-1 via the Transit Gateway peering. Because the two regions are peering, there is no need to alter the current Direct Connect infrastructure.

Option A is incorrect because it suggests using a virtual private gateway and transit VIF. While a virtual private gateway could connect eu-central-1 to the on-premises environment via Direct Connect, this would require changes to the Direct Connect setup, which the question specifically instructs to avoid. Also, this doesn't address connectivity with eu-west-2 resources.

Option C is incorrect because using a Site-to-Site VPN connection between transit gateways is less performant and more complex to manage than Transit Gateway peering. Transit Gateway peering is the preferred method for interconnecting transit gateways within the AWS global network due to its high bandwidth and low latency. VPNs would add unnecessary overhead and latency.

Option D is incorrect because using a public VIF is generally used to access public AWS services. It's not meant for private connectivity between VPCs and an on-premises environment. It does not involve a private secure connection as does Direct Connect.

Therefore, creating a new transit gateway in eu-central-1 and peering it with the existing transit gateway in eu-west-2 minimizes changes to the existing Direct Connect setup while providing the required connectivity between the on-premises data center, eu-west-2, and eu-central-1.

Relevant AWS Documentation:

AWS Transit Gateway: <https://aws.amazon.com/transit-gateway/>

Transit Gateway Peering: <https://docs.aws.amazon.com/vpc/latest/tgw/tgw-peering.html>

Question: 159

CertyIQ

A company has a 2 Gbps AWS Direct Connect hosted connection from the company's office to a VPC in the ap-southeast-2 Region. A network engineer adds a 5 Gbps Direct Connect hosted connection from a different Direct Connect location in the same Region. The hosted connections are connected to different routers from the office with an iBGP session running in between the routers.

The network engineer wants to ensure that the VPC uses the 5 Gbps hosted connection to route traffic to the office. Failover to the 2 Gbps hosted connection must occur when the 5 Gbps hosted connection is down.

Which solution will meet these requirements?

- A. Configure an outbound BGP policy from the router that is connected to the 2 Gbps connection. Advertise routes with a longer AS_PATH attribute to AWS.
- B. Advertise a longer prefix route from the router that is connected to the 2 Gbps connection.
- C. Advertise a less specific route from the router that is connected to the 5 Gbps connection.
- D. Configure an outbound BGP policy from the router that is connected to the 5 Gbps connection. Advertise routes with a longer AS_PATH attribute to AWS.

Answer: A

Explanation:

The correct answer is **A**. Here's why:

The goal is to prioritize the 5 Gbps Direct Connect connection for outbound traffic from the VPC to the office while ensuring failover to the 2 Gbps connection if the 5 Gbps connection fails. BGP path attributes are used to influence routing decisions.

Option A: Configuring an outbound BGP policy on the router connected to the 2 Gbps link and prepending the AS_PATH attribute makes the routes advertised from that router less desirable to AWS. The AS_PATH attribute is a count of the autonomous systems (AS) through which a route advertisement has passed. By artificially lengthening the AS_PATH on the 2 Gbps route, AWS prefers the route advertised via the 5 Gbps connection because it appears to be a shorter, more direct path. When the 5 Gbps connection is down, the AWS router will choose the 2 Gbps connection route even though it has a longer AS_PATH.

Why other options are incorrect:

B: Advertising a longer prefix (more specific) from the 2 Gbps connection would make that route more

preferred, not less, defeating the purpose. BGP prefers more specific routes.

C: Advertising a less specific route from the 5 Gbps connection would make that route less preferred. The goal is to make the 5 Gbps link the primary route.

D: Increasing the AS_PATH on the 5 Gbps would make the 5 Gbps link less preferred. The goal is to make the 5 Gbps link the preferred route.

In summary: By manipulating the AS_PATH attribute, we ensure that the 5 Gbps connection is favored under normal circumstances. When the 5 Gbps is unavailable, the BGP will revert to the 2 Gbps connection to maintain connectivity.

Relevant Links:

AWS Direct Connect Routing Policies: <https://docs.aws.amazon.com/directconnect/latest/UserGuide/routing-policies.html>

BGP Path Attributes: <https://www.cisco.com/c/en/us/support/docs/ip/border-gateway-protocol-bgp/13753-asp-path-prepend.html>

Question: 160

CertyIQ

An ecommerce company needs to implement additional security controls on all its domain names that are hosted in Amazon Route 53. The company's new policy requires data authentication and data integrity verification for all queries to the company's domain names. The current Route 53 architecture has four public hosted zones.

A network engineer needs to implement DNS Security Extensions (DNSSEC) signing and validation on the hosted zones. The solution must include an alert capability.

Which combination of steps will meet these requirements? (Choose three.)

- A. Enable DNSSEC signing for Route 53. Request that Route 53 create a key-signing key (KSK) based on a customer managed key in AWS Key Management Service (AWS KMS).
- B. Enable DNSSEC signing for Route 53. Request that Route 53 create a zone-signing key (ZSK) based on a customer managed key in AWS Key Management Service (AWS KMS).
- C. Create a chain of trust for the hosted zones by adding a Delegation Signer (DS) record for each subdomain.
- D. Create a chain of trust for the hosted zones by adding a Delegation Signer (DS) record to the parent zone.
- E. Set up an Amazon CloudWatch alarm that provides an alert whenever a DNSSECInternalFailure error or DNSSECKeySigningKeysNeedingAction error is detected.
- F. Set up an AWS CloudTrail alarm that provides an alert whenever a DNSSECInternalFailure error or DNSSECKeySigningKeysNeedingAction error is detected.

Answer: ADE

Explanation:

Let's break down why options A, D, and E are the correct choices for implementing DNSSEC with alerting for the given scenario.

A. Enable DNSSEC signing for Route 53. Request that Route 53 create a key-signing key (KSK) based on a customer-managed key in AWS Key Management Service (AWS KMS). This is a crucial first step. DNSSEC signing needs to be enabled to digitally sign the DNS records. Using a KSK backed by KMS allows the company to control and manage the cryptographic keys used for signing, enhancing security and meeting the policy requirements. AWS KMS offers features such as key rotation and auditing, providing a more secure and compliant solution compared to Route 53 managed keys. <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/dns-configuring-dnssec.html>

D. Create a chain of trust for the hosted zones by adding a Delegation Signer (DS) record to the parent zone. To establish a chain of trust, the parent zone needs to be aware that the child zone is DNSSEC-signed. This is

achieved by adding a DS record to the parent zone. The DS record contains a hash of the KSK of the child zone. This record essentially vouches for the authenticity of the child zone's keys.https://en.wikipedia.org/wiki/Domain_Name_System_Security_Extensions

E. Set up an Amazon CloudWatch alarm that provides an alert whenever a DNSSECInternalFailure error or DNSSECKeySigningKeysNeedingAction error is detected. Monitoring DNSSEC operations is critical. CloudWatch is the appropriate service for monitoring AWS services, including Route 53. Setting up alarms based on DNSSECInternalFailure and DNSSECKeySigningKeysNeedingAction errors allows the company to react quickly to potential DNSSEC issues, such as signing failures or key management problems.<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/monitoring-cloudwatch.html>

Why the other options are incorrect:

B. Enable DNSSEC signing for Route 53. Request that Route 53 create a zone-signing key (ZSK) based on a customer-managed key in AWS Key Management Service (AWS KMS). While ZSKs are used in DNSSEC, the prompt mentions a key-signing key (KSK) should be managed, also the KSK signs the ZSK. The ZSK signs the records. While it is technically possible to store ZSK in KMS, it is not generally recommended for high-traffic zones due to the volume of signing operations. The best practice is to manage the KSK in KMS and let Route53 manage the ZSK, or manage both in KMS, but Route53 will not create a ZSK based on the managed KMS.

C. Create a chain of trust for the hosted zones by adding a Delegation Signer (DS) record for each subdomain. The DS record needs to be added to the parent zone, not the subdomains. The delegation is from parent to child, not between subdomains.

F. Set up an AWS CloudTrail alarm that provides an alert whenever a DNSSECInternalFailure error or DNSSECKeySigningKeysNeedingAction error is detected. While CloudTrail records API calls and events, it's not the primary service for monitoring real-time metrics or specific error types directly related to Route 53 DNSSEC operations. CloudWatch is the service designed for this purpose. Cloudtrail can provide some audit information, but CloudWatch provides alarms on operational metrics.

Question: 161

CertyIQ

A financial company that is located in the us-east-1 Region needs to establish secure connectivity to AWS. The company has two on-premises data centers, each located within the same Region. The company's network team needs to establish hybrid connectivity to its AWS environment with reliable and consistent connectivity.

The connection must provide access to the company's private resources inside its AWS environment. The resources are located in the us-east-1 and us-west-2 Regions. The connection must allow resources from the corporate networks to send large amounts of data to Amazon S3 over the same connection. To meet compliance requirements, the connection must be highly available and must provide encryption for all packets that are sent between the on-premises location and any services on AWS.

Which combination of steps should the network team take to meet these requirements? (Choose two.)

- A. Set up a private VIF to send data to Amazon S3. Use an AWS Site-to-Site VPN connection over the private VIF to encrypt data in transit to the VPCs in us-east-1 and us-west-2.
- B. Set up an AWS Direct Connect connection to each of the company's data centers.
- C. Set up an AWS Direct Connect connection from one of the company's data centers to us-east-1 and us-west-2.
- D. Set up a public VIF to send data to Amazon S3. Use an AWS Site-to-Site VPN connection over the public VIF to encrypt data in transit to the VPCs in us-east-1 and us-west-2.
- E. Set up a transit VIF for an AWS Direct Connect gateway to send data to Amazon S3. Create a transit gateway. Associate the transit gateway with the Direct Connect gateway to provide secure communications from the company's data centers to the VPCs in us-east-1 and us-west-2.

Answer: BD

Explanation:

The correct answer is **BD**. Here's a detailed justification:

B: Set up an AWS Direct Connect connection to each of the company's data centers. This is crucial for establishing reliable and consistent connectivity. Direct Connect provides a dedicated network connection from the on-premises data centers to AWS, bypassing the public internet. This offers more consistent network performance, lower latency, and increased bandwidth, fulfilling the requirement for reliable connectivity. For high availability, having Direct Connect connections from both data centers is important; if one connection fails, the other remains active.

D: Set up a public VIF to send data to Amazon S3. Use an AWS Site-to-Site VPN connection over the public VIF to encrypt data in transit to the VPCs in us-east-1 and us-west-2. Since the question specifies that data being sent to S3 needs to be encrypted and sent via the Direct Connect, the use of a Public VIF (Virtual Interface) to access S3 is appropriate. Using a Site-to-Site VPN over the public VIF allows the company to encrypt traffic going to the VPCs in both us-east-1 and us-west-2 regions. Public VIF's are utilized for accessing public AWS services such as S3, whereas private VIF's are for accessing resources in private VPC's.

Note: A public VIF is necessary for connecting to public AWS services like S3 over Direct Connect. The Site-to-Site VPN provides the necessary encryption. Without the VPN, traffic is not encrypted, violating the compliance requirements.

Let's analyze why the other options are less suitable:

A: Set up a private VIF to send data to Amazon S3. Use an AWS Site-to-Site VPN connection over the private VIF to encrypt data in transit to the VPCs in us-east-1 and us-west-2. A Private VIF cannot be used to access Public Services such as S3, making this option incorrect.

C: Set up an AWS Direct Connect connection from one of the company's data centers to us-east-1 and us-west-2. Although this can provide connectivity, it introduces a single point of failure. If the Direct Connect connection from the single data center fails, the entire hybrid connectivity is lost. The question states that connections must be highly available.

E: Set up a transit VIF for an AWS Direct Connect gateway to send data to Amazon S3. Create a transit gateway. Associate the transit gateway with the Direct Connect gateway to provide secure communications from the company's data centers to the VPCs in us-east-1 and us-west-2. While a Transit Gateway (TGW) is suitable for connecting multiple VPCs across regions and integrating with Direct Connect, it is not the correct mechanism for connecting to S3. The transit gateway is used to route traffic between VPCs. Additionally, this option does not provide any encryption for traffic being sent to S3 as required in the scenario.

Authoritative Links:

AWS Direct Connect: <https://aws.amazon.com/directconnect/>

AWS Site-to-Site VPN: <https://aws.amazon.com/vpn/>

AWS Transit Gateway: <https://aws.amazon.com/transit-gateway/>

Question: 162

CertyIQ

A global company is designing a hybrid architecture to privately access AWS resources in the us-west-2 Region. The company's existing architecture includes a VPC that uses RFC 1918 IP address space. The VPC is connected to an on-premises data center over AWS Direct Connect. Amazon Route 53 provides name resolution within the VPC. Locally managed DNS servers in the data center provide DNS services to the on-premises hosts.

The company has applications in the data center that need to download objects from an Amazon S3 bucket in us-west-2.

Which solution can the company use to access Amazon S3 without using the public IP address space?

- A. Create an S3 interface endpoint in the VPC. Update the on-premises application configuration to use the Regional VPC endpoint DNS hostname that is mapped to the S3 interface endpoint.
- B. Create an S3 interface endpoint in the VPC. Configure a Route 53 Resolver inbound endpoint in the VPC. Set up the data center DNS servers to forward DNS queries for the S3 domain from on premises to the inbound endpoint.
- C. Create an S3 gateway endpoint in the VPC. Update the on-premises application configuration to use the hostname that is mapped to the S3 gateway endpoint.
- D. Create an S3 gateway endpoint in the VPC. Configure a Route 53 Resolver inbound endpoint in the VPC. Set up the data center DNS servers to forward DNS queries for the S3 domain from on premises to the inbound endpoint.

Answer: B

Explanation:

The requirement is to access an S3 bucket from on-premises servers via a private connection (Direct Connect) without using public IP addresses. This necessitates a solution that leverages AWS PrivateLink.

Option B is the correct solution because it involves creating an S3 interface endpoint in the VPC and configuring DNS resolution to direct traffic from on-premises to this endpoint. S3 interface endpoints use AWS PrivateLink, which provides private connectivity between VPCs, AWS services, and on-premises networks, without exposing traffic to the public internet. A Route 53 Resolver inbound endpoint is configured in the VPC to receive DNS queries from the on-premises DNS servers. The data center DNS servers are then configured to forward DNS queries for the S3 domain (e.g., s3.us-west-2.amazonaws.com) to this inbound endpoint. This way, when an on-premises application attempts to resolve the S3 hostname, the query is forwarded to the Route 53 Resolver, which resolves it to the private IP address of the S3 interface endpoint. This allows the on-premises application to access S3 through the Direct Connect connection and the private endpoint, completely avoiding the public internet.

Option A is incorrect because simply updating the on-premises application configuration to use the Regional VPC endpoint DNS hostname won't work without proper DNS resolution from on-premises. The on-premises DNS servers need to be able to resolve the VPC endpoint's hostname to a private IP address within the VPC.

Option C is incorrect because while S3 gateway endpoints provide private access to S3, they only work from within the VPC. They do not provide a way for on-premises systems to access S3 without using public IP addresses, even if connected via Direct Connect. Moreover, there's no hostname associated with a gateway endpoint that an on-premises application could directly use.

Option D is incorrect for a similar reason as option C. Gateway endpoints are VPC-internal constructs and do not inherently facilitate on-premises access via Direct Connect. While configuring a Route 53 Resolver inbound endpoint is useful for DNS resolution in a hybrid setup, it does not magically enable an on-premises application to use a gateway endpoint. Gateway endpoints do not have an associated resolvable hostname that can be used from outside the VPC.

In summary, the combination of an S3 interface endpoint and proper DNS resolution using Route 53 Resolver is essential to enable private S3 access from on-premises environments. The S3 interface endpoint uses PrivateLink, and the inbound resolver allows on-premises systems to resolve the S3 hostname to the private IP address of the endpoint.

Relevant links:

AWS PrivateLink: <https://aws.amazon.com/privatelink/>

Route 53 Resolver: <https://aws.amazon.com/route53/resolver/>

VPC Endpoints: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html>

Question: 163

A company is migrating critical applications to AWS. The company has multiple accounts and VPCs that are connected by a transit gateway.

A network engineer must design a solution that performs deep packet inspection for any traffic that leaves a VPC network boundary. All inspected traffic and the actions that are taken on the traffic must be logged in a central log account.

Which solution will meet these requirements with the LEAST administrative overhead?

A. Create a central network VPC that includes an attachment to the transit gateway. Update the VPC and transit gateway route tables to support the new attachment. Deploy an AWS Gateway Load Balancer that is backed by third-party, next-generation firewall appliances to the central network VPC. Create a policy that contains the rules for deep packet inspection. Attach the policy to the firewall appliances. Create an Amazon S3 bucket in the central log account. Configure the firewall appliances to capture and save the network flow logs to the S3 bucket.

B. Create a central network VPC that includes an attachment to the transit gateway. Update the VPC and transit gateway route tables to support the new attachment. Deploy an AWS Application Load Balancer that is backed by third-party, next-generation firewall appliances to the central network VPC. Create a policy that contains the rules for deep packet inspection. Attach the policy to the firewall appliances. Create a syslog server in the central log account. Configure the firewall appliances to capture and save the network flow logs to the syslog server.

C. Deploy network ACLs and security groups to each VPC. Attach the security groups to active network interfaces. Associate the network ACLs with VPC subnets. Create rules for the network ACLs and security groups to allow only the required traffic flows between subnets and network interfaces. Create an Amazon S3 bucket in the central log account. Configure a VPC flow log that captures and saves all traffic flows to the S3 bucket.

D. Create a central log VPC and an attachment to the transit gateway. Update the VPC and transit gateway route tables to support the new attachment. Deploy an AWS Network Load Balancer (NLB) that is backed by third-party, next-generation intrusion detection system (IDS) security appliances to the central VPC. Activate rules on the security appliances to monitor for intrusion signatures. For each network interface, create a VPC Traffic Mirroring session that sends the traffic to the central VPC's NLB.

Answer: A

Explanation:

The correct answer is **A**. Here's a detailed justification:

Option A provides a centralized architecture for deep packet inspection, crucial for security compliance and monitoring traffic leaving VPCs. By creating a central network VPC attached to the transit gateway, all traffic destined to external networks can be routed through this central VPC. Deploying a Gateway Load Balancer (GWLB) in front of third-party next-generation firewall appliances is the key element. GWLB is specifically designed for virtual appliances like firewalls, enabling seamless insertion into the network path for inspection. Configuring the firewall appliances to save network flow logs to an S3 bucket in a central log account allows for aggregated logging, analysis, and auditing. The process is efficient, secure, and scalable.

Option B uses an Application Load Balancer (ALB), which is designed for HTTP/HTTPS traffic and not suitable for deep packet inspection of all traffic types. Option C relies on network ACLs and security groups. While these are fundamental security controls, they offer only basic traffic filtering based on source/destination IP addresses, ports, and protocols and don't provide deep packet inspection or centralized logging. VPC Flow Logs captures network traffic metadata, but it does not perform deep packet inspection and the ACLs would be inefficient to manage. Option D uses a Network Load Balancer (NLB) with intrusion detection systems (IDS) and VPC Traffic Mirroring. Although the IDS can perform in-depth analysis, VPC Traffic Mirroring adds significant administrative overhead because you would need to configure mirroring sessions for each network interface. Moreover, an IDS is focused on intrusion detection, not inspection and policy enforcement like a next-generation firewall. The GWLB streamlines traffic to the appliances without requiring mirroring.

Question: 164

A company has an on-premises data center in the United States. The data center is connected to AWS by an AWS Direct Connect connection. The data center has a private VIF that is connected to a Direct Connect gateway.

Recently, the company opened a new data center in Europe and established a new Direct Connect connection between the Europe data center and AWS. A new private VIF connects to the existing Direct Connect gateway.

The company wants to use Direct Connect SiteLink to set up a private network between the data center in the United States and the data center in Europe.

Which solution will meet these requirements in the MOST operationally efficient manner?

- A. Create a new public VIF from each data center. Enable SiteLink on the new public VIFs.
- B. Create a new transit VIF from each data center. Enable SiteLink on the new transit VIFs.
- C. Use the existing VIF from each data center. Enable SiteLink on the existing private VIFs.
- D. Create a new AWS Site-to-Site VPN connection between the data centers. Configure the new connection to use SiteLink.

Answer: C**Explanation:**

The correct answer is C: Use the existing VIF from each data center. Enable SiteLink on the existing private VIFs.

Here's why:

Direct Connect SiteLink Functionality: Direct Connect SiteLink facilitates direct data transfer between Direct Connect locations, bypassing AWS regions. It does this over AWS's global network backbone, offering potentially improved performance compared to routing traffic through an AWS region.

Private VIF and SiteLink Compatibility: SiteLink is compatible with private VIFs, transit VIFs, and public VIFs. The key is that the VIFs must be associated with Direct Connect gateways.

Operational Efficiency: The prompt explicitly asks for the most operationally efficient solution. Since the company already has private VIFs connected to a Direct Connect gateway for each data center, leveraging these existing resources minimizes setup and management overhead. Creating new VIFs (public or transit) and reconfiguring routing would be more complex. Similarly, establishing a new Site-to-Site VPN connection introduces additional complexity, and VPN isn't part of the Direct Connect SiteLink feature set.

Avoidance of Unnecessary Changes: Option A (public VIFs) is unsuitable because the requirement is for a private network. Option B (transit VIFs) introduces more complexity and cost than the existing private VIF setup. Option D uses a VPN, which negates the purpose of Direct Connect SiteLink.

Therefore, using the existing private VIFs connected to the Direct Connect gateway and enabling SiteLink is the most straightforward and operationally efficient method to establish a private network link between the US and Europe data centers utilizing Direct Connect SiteLink.

Further Research:

AWS Direct Connect SiteLink: <https://aws.amazon.com/directconnect/sitelink/>

AWS Direct Connect Gateways: <https://docs.aws.amazon.com/directconnect/latest/UserGuide/direct-connect-gateways-intro.html>

Question: 165

A company has a new AWS Direct Connect connection between its on-premises data center and the AWS Cloud. The company has created a new private VIF on this connection. However, the VIF status is DOWN.

A network engineer verifies that the physical connection status is UP and RUNNING based on information from the AWS Management Console. The network engineer checks the customer Direct Connect router and can see the ARP entry for the VLAN interface created for the private VIF at AWS.

What could be causing the private VIF to have a DOWN status?

- A. ICMP is blocked on the customer Direct Connect router.
- B. TCP port 179 is blocked on the customer Direct Connect router.
- C. The IEEE 802.1Q VLAN identifier is misconfigured on the customer Direct Connect router.
- D. The company has configured IEEE 802.1ad instead of 802.1Q on the customer Direct Connect router.

Answer: B

Explanation:

The correct answer is **B. TCP port 179 is blocked on the customer Direct Connect router.**

Here's a detailed justification:

Direct Connect private VIFs use the Border Gateway Protocol (BGP) to exchange routing information between the customer's network and AWS. BGP relies on TCP port 179 for establishing and maintaining peering sessions. If TCP port 179 is blocked on the customer's Direct Connect router, the BGP session cannot be established, which will cause the private VIF to remain in a DOWN state.

The question states the physical connection is UP and RUNNING, and the customer router has an ARP entry for the VLAN interface. This implies that Layer 1 and Layer 2 connectivity is working fine. However, BGP peering isn't happening.

Let's analyze why the other options are less likely:

A. ICMP is blocked on the customer Direct Connect router. While ICMP is useful for troubleshooting, it's not a requirement for BGP to function. BGP relies on TCP, not ICMP, for its operations. Blocking ICMP might hinder ping tests, but won't directly prevent BGP from establishing a session if TCP port 179 is open.

C. The IEEE 802.1Q VLAN identifier is misconfigured on the customer Direct Connect router. If the VLAN ID was misconfigured, ARP probably wouldn't resolve, but the question says there is an ARP entry. A VLAN mismatch usually results in complete L2 connectivity failure.

D. The company has configured IEEE 802.1ad instead of 802.1Q on the customer Direct Connect router. Direct Connect requires 802.1Q VLAN tagging. If 802.1ad were configured instead, it'd be a fundamental incompatibility at the VLAN tagging level, likely preventing even basic connectivity or showing the VIF as completely down with no ARP entries.

Therefore, blocking TCP port 179, which is critical for BGP, is the most likely reason for the private VIF to be in a DOWN state despite a functional physical connection and ARP resolution.

Authoritative Links for further research:

AWS Direct Connect BGP Configuration: <https://docs.aws.amazon.com/directconnect/latest/UserGuide/multi-bgp.html>

Direct Connect requirements:

Question: 166

AnyCompany has acquired Example Corp. AnyCompany's infrastructure is all on premises, and Example Corp's infrastructure is completely in the AWS Cloud. The companies are using AWS Direct Connect with AWS Transit Gateway to establish connectivity between each other.

Example Corp has deployed a new application across two Availability Zones in a VPC with no internet gateway. The CIDR range for the VPC is 10.0.0.0/16. Example Corp needs to access an application that is deployed on premises by AnyCompany. Because of compliance requirements, Example Corp must access the application through a limited contiguous block of approved IP addresses (10.1.0.0/24).

A network engineer needs to implement a highly available solution to achieve this goal. The network engineer starts by updating the VPC to add a new CIDR range of 10.1.0.0/24.

What should the network engineer do next to meet the requirements?

A. In each Availability Zone in the VPC, create a subnet that uses part of the allowed IP address range. Create a public NAT gateway in each of the new subnets. Update the route tables that are associated with other subnets to route application traffic to the public NAT gateway in the corresponding Availability Zone. Add a route to the route table that is associated with the subnets of the public NAT gateways to send traffic destined for the application to the transit gateway.

B. In each Availability Zone in the VPC, create a subnet that uses part of the allowed IP address range. Create a private NAT gateway in each of the new subnets. Update the route tables that are associated with other subnets to route application traffic to the private NAT gateway in the corresponding Availability Zone. Add a route to the route table that is associated with the subnets of the private NAT gateways to send traffic destined for the application to the transit gateway.

C. In the VPC, create a subnet that uses the allowed IP address range. Create a private NAT gateway in the new subnet. Update the route tables that are associated with other subnets to route application traffic to the private NAT gateway. Add a route to the route table that is associated with the subnet of the private NAT gateway to send traffic destined for the application to the transit gateway.

D. In the VPC, create a subnet that uses the allowed IP address range. Create a public NAT gateway in the new subnet. Update the route tables that are associated with other subnets to route application traffic to the public NAT gateway. Add a route to the route table that is associated with the subnet of the public NAT gateway to send traffic destined for the application to the transit gateway.

Answer: B

Explanation:

Here's a detailed justification for why option B is the correct answer, and why the other options are incorrect, along with supporting concepts and links.

Justification for Option B:

The core requirement is that Example Corp's application accesses AnyCompany's on-premises application using a specific block of IP addresses (10.1.0.0/24). Since the applications in Example Corp's VPC do not have internet access, and AnyCompany requires traffic to come from the 10.1.0.0/24 range, we need to translate the source IP addresses of the outgoing traffic. A NAT (Network Address Translation) gateway is the perfect solution for this.

1. **Subnets with Approved IP Range:** Creating subnets within each Availability Zone using the 10.1.0.0/24 range provides the basis for assigning the desired source IP addresses. By creating a subnet in each AZ this maintains high availability as per the requirements.
2. **Private NAT Gateway:** Using a private NAT gateway is crucial because the application in Example Corp's VPC doesn't need or have internet access. The communication is intended only between Example Corp's VPC and AnyCompany's on-premises network through the Transit Gateway. A public

NAT gateway is intended for internet-bound traffic. Since this traffic is intended for the Direct Connect over Transit Gateway, this should not be a public NAT Gateway. A private NAT Gateway, also known as a NAT Instance, is fully supported by AWS (see reference link at bottom).

3. **Route Table Updates:** Updating the route tables associated with other subnets in the VPC to route traffic intended for AnyCompany's on-premises application to the private NAT gateway ensures that the source IP address will be translated to an address within the 10.1.0.0/24 range as it exits the NAT gateway.
4. **Transit Gateway Route:** Adding a route to the route table associated with the subnets of the private NAT gateways to send traffic destined for AnyCompany's on-premises network to the Transit Gateway ensures that the translated traffic is correctly routed towards the Direct Connect connection and, ultimately, to the destination application.

Why other options are incorrect:

Option A (Public NAT Gateways): Using public NAT gateways would unnecessarily expose the traffic to the internet, violating the requirement that the application does not need or have internet access and potentially creating security vulnerabilities.

Option C (Single Subnet, Private NAT Gateway): Creating a NAT gateway in a single subnet creates a single point of failure. The requirement explicitly states a need for a highly available solution. Distributing the NAT gateways across multiple Availability Zones addresses this requirement.

Option D (Single Subnet, Public NAT Gateway): This suffers from both the single point of failure problem (lack of high availability) and the unnecessary exposure of traffic to the internet.

Key Concepts:

NAT (Network Address Translation): Allows private IP addresses to be translated to one or more public IP addresses (or vice-versa). Crucial for allowing instances in private subnets to access external resources.

Availability Zones (AZs): Physically separate and isolated locations within an AWS region. Designing across AZs enhances application availability and fault tolerance.

Route Tables: Define the routes for network traffic leaving a subnet or VPC.

AWS Transit Gateway: A hub-and-spoke network transit hub that can be used to interconnect VPCs and on-premises networks.

AWS Direct Connect: Establishes a dedicated network connection from your premises to AWS.

Authoritative Links:

NAT Gateways: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html>

AWS Transit Gateway: <https://docs.aws.amazon.com/vpc/latest/tgw/what-is-transit-gateway.html>

AWS Direct Connect: <https://aws.amazon.com/directconnect/>

NAT Instances: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat.html>

Question: 167

CertyIQ

A company recently experienced an IP address exhaustion event in its VPCs. The event affected service capacity. The VPCs hold two or more subnets in different Availability Zones.

A network engineer needs to develop a solution that monitors IP address usage across resources in the VPCs. The company needs to receive notification about possible issues so that the company can act before an incident happens.

Which solution will meet these requirements with the LEAST operational overhead?

A. Set up Amazon VPC IP Address Manager (IPAM) with a new top-level pool. In the top-level pool, create a pool for each VPC. In each VPC pool, create a pool for each subnet in that VPC. Turn on the auto-import option for

the VPC pools and the subnet pools. Configure an Amazon CloudWatch alarm to send an Amazon Simple Notification Service (Amazon SNS) notification if the availability limit threshold is reached.

B. Set up a log group in Amazon CloudWatch Logs for each subnet. Create an AWS Lambda function that reads each subnet's IP address usage and publishes metrics to the log group. Configure an Amazon CloudWatch alarm to send an Amazon Simple Notification Service (Amazon SNS) notification if the availability limit threshold is reached.

C. Set up a custom Amazon CloudWatch metric for IP address usage for each subnet. Create an AWS Lambda function that reads each subnet's IP address usage and publishes a CloudWatch metric dimension. Schedule the Lambda function to run every 5 minutes. Configure a CloudWatch alarm to send an Amazon Simple Notification Service (Amazon SNS) notification if the availability limit threshold is reached.

D. Set up Amazon VPC IP Address Manager (IPAM) with a new top-level pool. In the top-level pool, create a pool for each VPC. In each VPC pool, create a pool for each subnet in that VPC. Turn on the auto-import option for the VPC pools and the subnet pools. Configure an Amazon EventBridge rule that monitors each pool availability limit threshold and sends an Amazon Simple Notification Service (Amazon SNS) notification if the limit threshold is reached.

Answer: A

Explanation:

The correct answer is A. Here's why:

Amazon VPC IP Address Manager (IPAM) offers a centralized and automated approach to managing IP addresses across AWS environments, meeting the requirements of monitoring, alerting, and minimal operational overhead.

Option A leverages IPAM's hierarchical structure (top-level pool, VPC pools, subnet pools) to represent the company's VPCs and subnets. The auto-import feature automatically discovers and tracks IP address usage within these pools, eliminating the need for manual data collection.

By configuring a CloudWatch alarm based on the availability limit threshold of the IPAM pools, the company can receive timely notifications when IP address usage approaches exhaustion. This proactive approach enables them to address potential issues before they impact service capacity. CloudWatch Alarms are designed for monitoring metrics and triggering notifications.

Options B and C involve custom solutions using Lambda functions to collect and publish IP address usage metrics. These options require more operational overhead for development, deployment, and maintenance compared to IPAM's built-in monitoring capabilities. CloudWatch Logs are typically used for application logging and not for this purpose. Custom Metrics can be complex to set up.

Option D also uses IPAM, but it uses EventBridge to monitor the availability limit. While EventBridge can monitor events, CloudWatch Alarms are more suitable for monitoring metrics like IP address availability, thus option A is a more appropriate solution.

In summary, IPAM with CloudWatch Alarms offers a managed, automated, and scalable solution for monitoring IP address usage, triggering notifications, and minimizing operational overhead, aligning with the company's stated requirements.

Here are some helpful links for further research:

[Amazon VPC IP Address Manager \(IPAM\)](#)
[Monitoring IPAM using CloudWatch](#)
[Amazon CloudWatch Alarms](#)

Question: 168

A company has a hybrid IT setup that includes services that run in an on-premises data center and in the AWS

Cloud. The company is using AWS Direct Connect to connect its data center to AWS. The company is using one AWS Site-to-Site VPN connection as backup and requires a backup connectivity option to always be present. The company is transitioning to IPv6 by implementing dual-stack architectures.

Which combination of steps will transition the data center's connectivity to AWS in the LEAST amount of time? (Choose two.)

- A. Create a new Site-to-Site VPN tunnel for the IPv6 traffic.
- B. Create a new dual-stack Site-to-Site VPN connection between the data center and AWS. Provision routing. Delete the original Site-to-Site VPN connection.
- C. Associate a new dual-stack public VIF with the Direct Connect connection. Migrate the Direct Connect traffic to the new VIF.
- D. Add a new IPv6 peer in the existing VIF. Use the IPv6 address provided by Amazon on the peer router.
- E. Send IPv6 traffic between the data center and AWS in a tunnel inside the existing IPv4 tunnels.

Answer: AE

Explanation:

The correct answer is AE. Here's a detailed justification:

Why A is correct: Creating a new Site-to-Site VPN tunnel specifically for IPv6 traffic allows for parallel operation with the existing IPv4 VPN connection, ensuring continuous backup connectivity during the transition. This leverages existing infrastructure while enabling IPv6 support. It's the simplest and quickest way to establish IPv6 VPN backup.

Why E is correct: Tunneling IPv6 within the existing IPv4 tunnels of both Direct Connect and the backup VPN is the fastest way to start sending IPv6 traffic between the data center and AWS. This avoids the need for new VIFs or major routing changes initially. It allows for a gradual and controlled introduction of IPv6.

Why B is incorrect: Creating a new dual-stack Site-to-Site VPN connection and deleting the original requires downtime for migration and is more complex than adding a tunnel. Furthermore, it doesn't immediately address the Direct Connect IPv6 transition.

Why C is incorrect: Associating a new dual-stack Public VIF with Direct Connect necessitates more significant routing and configuration adjustments. Moreover, migrating the Direct Connect traffic to the new VIF introduces a more disruptive change compared to tunneling. This would involve changes on the AWS side, potentially also requiring new ASN. It's also unnecessary for providing backup.

Why D is incorrect: While adding an IPv6 peer in the existing VIF is necessary for native dual-stack Direct Connect, it doesn't address the need for a backup IPv6 connectivity option using VPN. It only makes the primary Direct Connect link dual-stack. Direct Connect requires coordination with AWS support.

In summary, A and E together provide the fastest path to enabling IPv6 connectivity while maintaining a backup through VPN: Using existing infrastructure to tunnel IPv6 over IPv4 connections allows minimal configuration changes and zero downtime. Option A builds upon this by providing a VPN backup mechanism. The other options require more complex setup and migrations, failing the "LEAST amount of time" requirement.

Supporting Links:

AWS Site-to-Site VPN: <https://aws.amazon.com/vpn/>

AWS Direct Connect: <https://aws.amazon.com/directconnect/>

IPv6 Transition Mechanisms: https://en.wikipedia.org/wiki/IPv6_transition_mechanisms

A company is developing a new application that is deployed in multiple VPCs across multiple AWS Regions. The VPCs are connected through AWS Transit Gateway. The VPCs contain private subnets and public subnets.

All outbound internet traffic in the private subnets must be audited and logged. The company's network engineer plans to use AWS Network Firewall and must ensure that all traffic through Network Firewall is completely logged for auditing and alerting.

How should the network engineer configure Network Firewall logging to meet these requirements?

- A. Configure Network Firewall logging in Amazon CloudWatch to capture all alerts. Send the logs to a log group in Amazon CloudWatch Logs.
- B. Configure Network Firewall logging in Network Firewall to capture all alerts and flow logs.
- C. Configure Network Firewall logging by configuring VPC Flow Logs for the firewall endpoint. Send the logs to a log group in Amazon CloudWatch Logs.
- D. Configure Network Firewall logging by configuring AWS CloudTrail to capture data events.

Answer: B

Explanation:

Here's a detailed justification for why option B is the correct answer, along with explanations of why the other options are incorrect:

Why Option B is Correct: Configure Network Firewall logging in Network Firewall to capture all alerts and flow logs.

This option directly addresses the requirement of logging all traffic through Network Firewall. Network Firewall provides its own built-in logging capabilities, specifically designed to capture both alert logs and flow logs.

Alert Logs: These logs record when traffic matches a rule defined in the Network Firewall rule groups. This is crucial for identifying and investigating potentially malicious or unwanted traffic. They provide detailed information about the specific rule that triggered the alert, the source and destination IP addresses, ports, and other relevant details.

Flow Logs: These logs capture information about the network traffic flowing through the firewall. By enabling flow logs within Network Firewall itself, you ensure that you're capturing all traffic that passes through the firewall endpoint, not just traffic that matches specific rules. This comprehensive logging is essential for auditing purposes and understanding network traffic patterns.

The logging configuration is performed directly within the Network Firewall settings, giving you granular control over what is captured and how.

Why Other Options are Incorrect:

A. Configure Network Firewall logging in Amazon CloudWatch to capture all alerts. Send the logs to a log group in Amazon CloudWatch Logs.

While Network Firewall can send alert logs to CloudWatch Logs, this option only focuses on alerts. It doesn't capture the flow logs, which are necessary for complete traffic auditing as required by the prompt. Focusing only on alerts misses a significant portion of network activity that may be relevant for audit or security analysis.

C. Configure Network Firewall logging by configuring VPC Flow Logs for the firewall endpoint. Send the logs to a log group in Amazon CloudWatch Logs.

While VPC Flow Logs capture traffic flowing to and from network interfaces, configuring VPC Flow Logs on the firewall endpoint only captures the traffic directly entering/leaving the Network Firewall service. It doesn't provide the detailed alert information specific to the Network Firewall rules. Also, Network Firewall

provides richer flow logs compared to VPC Flow Logs when it comes to traffic inspection data.

D. Configure Network Firewall logging by configuring AWS CloudTrail to capture data events.

AWS CloudTrail primarily captures API calls made to AWS services. While CloudTrail can provide information about configuration changes made to the Network Firewall itself (management plane events), it doesn't capture network traffic data (data plane events) that flows through the firewall. CloudTrail is not a substitute for flow logs and alert logs.

Authoritative Links:

AWS Network Firewall Logging: <https://docs.aws.amazon.com/network-firewall/latest/developerguide/monitoring.html>

AWS Network Firewall Features: <https://aws.amazon.com/network-firewall/features/>

Question: 170

CertyIQ

A company has set up a NAT gateway in a single Availability Zone (AZ1) in a VPC (VPC1) to access the internet from Amazon EC2 workloads in the VPC. The EC2 workloads are running in private subnets in three Availability Zones (AZ1, AZ2, AZ3). The route table for each subnet is configured to use the NAT gateway to access the internet.

Recently during an outage, internet access stopped working for the EC2 workloads because of the NAT gateway's unavailability. A network engineer must implement a solution to remove the single point of failure from the architecture and provide built-in redundancy.

Which solution will meet these requirements?

- A. Set up two NAT gateways. Place each NAT gateway in a different public subnet in separate Availability Zones (AZ2 and AZ3). Configure a route table for private subnets to route traffic to the virtual IP addresses of the two NAT gateways.
- B. Set up two NAT gateways. Place each NAT gateway in a different public subnet in separate Availability Zones (AZ2 and AZ3). Configure a route table to point the AZ2 private subnets to the NAT gateway in AZ2. Configure the same route table to point the AZ3 private subnets to the NAT gateway in AZ3.
- C. Create a second VPC (VPC2). Set up two NAT gateways. Place each NAT gateway in a different VPC (VPC1 and VPC2) and in the same Availability Zone (AZ2). Configure a route table in VPC1 to point the AZ2 private subnets to one NAT gateway. Configure a route table in VPC2 to point the AZ2 private subnets to the second NAT gateway.
- D. Set up two NAT gateways. Place each NAT gateway in a different public subnet in separate Availability Zones (AZ2 and AZ3). Configure a route table to point the AZ2 private subnets to the NAT gateway in AZ2. Configure a second route table to point the AZ3 private subnets to the NAT gateway in AZ3.

Answer: D

Explanation:

The requirement is to eliminate the single point of failure caused by a single NAT gateway for EC2 instances distributed across multiple Availability Zones (AZs).

Option D addresses this by deploying two NAT gateways in separate AZs (AZ2 and AZ3). Crucially, it configures separate route tables for each AZ, routing traffic from private subnets in AZ2 to the NAT gateway in AZ2, and traffic from private subnets in AZ3 to the NAT gateway in AZ3. This ensures that if one NAT gateway in one AZ fails, only the instances in that AZ lose internet connectivity, while instances in the other AZ remain unaffected, maintaining high availability. This is a key benefit of AZ-specific routing.

Option A is incorrect because NAT gateways do not have virtual IP addresses that can be used for routing. The route table entries directly specify the NAT gateway's ID. Routing to a virtual IP would not be compatible with NAT gateway operation.

Option B creates a single route table and tries to route traffic based on the AZ of origin using this single table. This is not a supported feature of VPC route tables. Each subnet must have its own route table or use the VPC's main route table. Therefore, option B cannot provide a suitable solution.

Option C introduces a second VPC, which is unnecessary complexity for simply providing high availability for NAT gateways. The goal is to make internet access highly available for EC2 instances inside the existing VPC. Creating a second VPC for NAT gateways adds operational overhead and is not the most efficient solution. Furthermore, restricting both NAT gateways to a single Availability Zone does not provide true redundancy across Availability Zones.

Therefore, Option D is the most appropriate solution because it leverages AZ-specific routing with distinct NAT gateways to provide redundancy and eliminate the single point of failure. This allows traffic from one AZ to continue flowing through the other NAT gateway if one fails.

Relevant documentation:

AWS NAT Gateway: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html>

VPC Route Tables: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Route_Tables.html

Question: 171

CertyIQ

A company has a total of 30 VPCs. Three AWS Regions each contain 10 VPCs. The company has attached the VPCs in each Region to a transit gateway in that Region. The company also has set up inter-Region peering connections between the transit gateways.

The company wants to use AWS Direct Connect to provide access from its on-premises location for only four VPCs across the three Regions. The company has provisioned four Direct Connect connections at two Direct Connect locations.

Which combination of steps will meet these requirements MOST cost-effectively? (Choose three.)

- A. Create four virtual private gateways. Attach the virtual private gateways to the four VPCs.
- B. Create a Direct Connect gateway. Associate the four virtual private gateways with the Direct Connect gateway.
- C. Create four transit VIFs on each Direct Connect connection. Associate the transit VIFs with the Direct Connect gateway.
- D. Create four transit VIFs on each Direct Connect connection. Associate the transit VIFs with the four virtual private gateways.
- E. Create four private VIFs on each Direct Connect connection to the Direct Connect gateway.
- F. Create an association between the Direct Connect gateway and the transit gateways.

Answer: BCF

Explanation:

The correct answer is BCF. Here's why:

B. Create a Direct Connect gateway. Associate the four virtual private gateways with the Direct Connect gateway. This is essential because a Direct Connect gateway acts as a central point for connecting your Direct Connect connection to multiple VPCs across different regions. Without it, your Direct Connect connection cannot directly access the VPCs in different regions through transit gateways.

<https://docs.aws.amazon.com/directconnect/latest/UserGuide/direct-connect-gateways-intro.html>

C. Create four transit VIFs on each Direct Connect connection. Associate the transit VIFs with the Direct Connect gateway. Transit VIFs are the correct type of VIF to use when connecting a Direct Connect connection to a Direct Connect gateway. Since we need access to four VPCs via the DX connection, we create

four transit VIFs to enable routing.

<https://docs.aws.amazon.com/directconnect/latest/UserGuide/WorkingWithVirtualInterfaces.html>

F. Create an association between the Direct Connect gateway and the transit gateways. This is necessary to allow traffic to flow between the on-premises location (connected via Direct Connect) and the VPCs connected to the transit gateways. The DX gateway needs to know how to route traffic to the VPCs, and the association allows it to learn the routes from the transit gateways.

Why other options are incorrect:

A. Create four virtual private gateways. Attach the virtual private gateways to the four VPCs. While VPCs need gateways for external communication, creating VPGs for VPCs that are already connected to a Transit Gateway isn't a cost-effective solution for Direct Connect integration. Using VPGs would mean you aren't leveraging the TGW's routing capabilities, which are specifically designed for interconnectivity.

D. Create four transit VIFs on each Direct Connect connection. Associate the transit VIFs with the four virtual private gateways. Transit VIFs must be associated with a Direct Connect gateway, not directly with virtual private gateways.

E. Create four private VIFs on each Direct Connect connection to the Direct Connect gateway. Private VIFs are for accessing resources within a single VPC. Since the company wants to access VPCs across multiple Regions (connected to Transit Gateways), Private VIFs are insufficient and not the correct option.

In summary, BCF allows Direct Connect to provide connectivity for a specific number of VPCs attached to transit gateways in different regions by using Direct Connect gateway as central point and leveraging transit VIFs for appropriate route advertisement.

Question: 172

CertyIQ

A company needs to manage Amazon EC2 instances through command line interfaces for Linux hosts and Windows hosts. The EC2 instances are deployed in an environment in which there is no route to the internet. The company must implement role-based access control for management of the instances. The company has a standalone on-premises environment.

Which approach will meet these requirements with the LEAST maintenance overhead?

- A. Set up an AWS Direct Connect connection between the on-premises environment and the VPC where the instances are deployed. Configure routing, security groups, and ACLs. Connect to the instances by using the Direct Connect connection.
- B. Deploy and configure AWS Systems Manager Agent (SSM Agent) on each instance. Deploy VPC endpoints for Systems Manager Session Manager. Connect to the instances by using Session Manager.
- C. Establish an AWS Site-to-Site VPN connection between the on-premises environment and the VPC where the instances are deployed. Configure routing, security groups, and ACLs. Connect to the instances by using the Site-to-Site VPN connection.
- D. Deploy an appliance to the VPC where the instances are deployed. Assign a public IP address to the appliance. Configure security groups and ACLs. Connect to the instances by using the appliance as an intermediary.

Answer: B

Explanation:

The correct answer is **B. Deploy and configure AWS Systems Manager Agent (SSM Agent) on each instance. Deploy VPC endpoints for Systems Manager Session Manager. Connect to the instances by using Session Manager.**

Here's why this is the best approach and why the others are less suitable:

Security and Least Overhead: This solution leverages AWS Systems Manager (SSM) Session Manager. Session Manager allows you to manage EC2 instances without opening inbound ports (like SSH or RDP) or managing SSH keys, which enhances security. It also simplifies the process of managing EC2 instances that do not have internet access. The maintenance overhead is minimal because SSM Agent updates are handled automatically by AWS and the SSM service.

Role-Based Access Control (RBAC): Session Manager integrates with IAM, enabling granular role-based access control. You can precisely define who can access which instances and what actions they can perform.

No Internet Required: By using VPC endpoints for Session Manager, you enable secure communication between the EC2 instances and the SSM service without traversing the internet.

Auditability: Session Manager sessions can be logged to Amazon S3 or CloudWatch Logs, providing a detailed audit trail of all actions performed on the instances.

Why other options are less suitable:

A (Direct Connect): While Direct Connect provides a dedicated network connection, it's complex and expensive to set up and maintain, primarily designed for high-bandwidth, low-latency connectivity, not solely for instance management. The routing and security group configuration are also more complex.

C (Site-to-Site VPN): VPN connections add management overhead and security considerations. They require configuring and maintaining VPN gateways, tunnels, and routing. Also, you would then need to consider SSH/RDP access which we are trying to avoid.

D (Appliance with Public IP): Introducing an intermediary appliance with a public IP address increases the attack surface and management complexity. You would have to secure, patch, and maintain the appliance itself, making it a less desirable solution.

In summary: Option B is the most secure, manageable, and cost-effective option because it leverages AWS Systems Manager Session Manager via VPC endpoints to provide secure, role-based access control for EC2 instances without requiring internet access or managing SSH keys or inbound ports. It has the least maintenance overhead compared to the other options.

Authoritative Links:

AWS Systems Manager Session Manager: <https://docs.aws.amazon.com/systems-manager/latest/userguide/session-manager.html>

VPC Endpoints for Systems Manager: <https://docs.aws.amazon.com/systems-manager/latest/userguide/vpc-endpoints.html>

Question: 173

CertyIQ

A network engineer needs to improve the network security of an existing AWS environment by adding an AWS Network Firewall firewall to control internet-bound traffic. The AWS environment consists of five VPCs. Each VPC has an internet gateway, NAT gateways, public Application Load Balancers (ALBs), and Amazon EC2 instances. The EC2 instances are deployed in private subnets. The architecture is deployed across two Availability Zones.

The network engineer must be able to configure rules for the public IP addresses in the environment, regardless of the direction of traffic. The network engineer must add the firewall by implementing a solution that minimizes changes to the existing production environment. The solution also must ensure high availability.

Which combination of steps should the network engineer take to meet these requirements? (Choose two.)

- A. Create a centralized inspection VPC with subnets in two Availability Zones. Deploy Network Firewall in this inspection VPC with an endpoint in each Availability Zone.
- B. Configure new subnets in two Availability Zones in each VPC. Deploy Network Firewall in each VPC with an endpoint in each Availability Zone.
- C. Deploy Network Firewall in each VPC. Use existing subnets in each of the two Availability Zones to deploy Network Firewall endpoints.
- D. Update the route tables that are associated with the private subnets that host the EC2 instances. Add routes

to the Network Firewall endpoints.

E. Update the route tables that are associated with the public subnets that host the NAT gateways and the ALBs. Add routes to the Network Firewall endpoints.

Answer: BE

Explanation:

The correct answer is BE. Here's a detailed justification:

B: Configure new subnets in two Availability Zones in each VPC. Deploy Network Firewall in each VPC with an endpoint in each Availability Zone.

This option aligns with the requirement of controlling traffic regardless of direction and minimizing changes to the existing environment. By deploying Network Firewall within each VPC, all internet-bound traffic (both inbound and outbound) can be inspected and controlled directly. Using new subnets isolates the firewall endpoints from the existing infrastructure, minimizing potential disruption during deployment. Deploying firewall endpoints in two Availability Zones ensures high availability, fulfilling another key requirement. This approach avoids the complexity of routing traffic across VPCs, reducing the risk of misconfiguration and performance bottlenecks.

E: Update the route tables that are associated with the public subnets that host the NAT gateways and the ALBs. Add routes to the Network Firewall endpoints.

This option is crucial for directing traffic to the Network Firewall endpoints. Since the NAT gateways and ALBs handle internet-bound traffic in the existing architecture, updating their route tables to point to the firewall endpoints is the mechanism for intercepting and inspecting that traffic. This ensures all internet-bound traffic originating from the EC2 instances (via NAT gateway) and directed towards the ALBs is subject to the firewall rules.

Why other options are incorrect:

1. **A:** Creating a centralized inspection VPC would require significant changes to the existing route tables across all VPCs to direct traffic to the central firewall. This increases complexity and potential points of failure. Also, it doesn't address the requirement to manage rules for public IP addresses in the environment, as you're routing through the firewall, not directly managing public IP ranges.
2. **C:** Deploying the firewall in existing subnets could disrupt the existing network configuration and impact running applications.
3. **D:** Updating route tables for private subnets that host EC2 instances will only direct traffic originating from the EC2 instances towards the firewall. It doesn't address the requirement to filter traffic to the ALBs, which are in public subnets. Therefore, updating the private subnets' route tables alone would be insufficient.

Relevant Concepts and Links:

1. **AWS Network Firewall:** A managed service that provides network security capabilities for your Amazon VPCs.
2. <https://aws.amazon.com/network-firewall/>
3. **VPC Routing:** Defines the paths network traffic takes within your VPC.
4. https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Route_Tables.html
5. **Availability Zones:** Physically isolated locations within an AWS Region.
6. https://aws.amazon.com/about-aws/global-infrastructure/regions_availability-zones/
7. **NAT Gateway:** Allows instances in private subnets to connect to the internet or other AWS services, but prevents the internet from initiating a connection with the instances.
8. <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html>

In summary, deploying Network Firewall within each VPC using dedicated subnets and updating the route

tables associated with NAT gateways and ALBs offers the best balance of security, minimal disruption, and high availability, fulfilling all requirements.

Question: 174

CertyIQ

A company is planning to migrate an internal application to the AWS Cloud. The application will run on Amazon EC2 instances in one VPC. Users will access the application from the company's on-premises data center through AWS VPN or AWS Direct Connect. Users will use private domain names for the application endpoint from a domain name that is reserved explicitly for use in the AWS Cloud.

Each EC2 instance must have automatic failover to another EC2 instance in the same AWS account and the same VPC. A network engineer must design a DNS solution that will not expose the application to the internet.

Which solution will meet these requirements?

A. Assign public IP addresses to the EC2 instances. Create an Amazon Route 53 private hosted zone for the AWS reserved domain name. Associate the private hosted zone with the VPC. Create a Route 53 Resolver outbound endpoint. Configure conditional forwarding in the on-premises DNS resolvers to forward all DNS queries for the AWS domain to the outbound endpoint IP address for Route 53 Resolver. In the private hosted zone, configure primary and failover records that point to the public IP addresses of the EC2 instances. Create an Amazon CloudWatch metric and alarm to monitor the application's health. Set up a health check on the alarm for the primary application endpoint.

B. Place the EC2 instances in private subnets. Create an Amazon Route 53 public hosted zone for the AWS reserved domain name. Associate the public hosted zone with the VPC. Create a Route 53 Resolver inbound endpoint. Configure conditional forwarding in the on-premises DNS resolvers to forward all DNS queries for the AWS domain to the inbound endpoint IP address for Route 53 Resolver. In the public hosted zone, configure primary and failover records that point to the IP addresses of the EC2 instances. Create an Amazon CloudWatch metric and alarm to monitor the application's health. Set up a health check on the alarm for the primary application endpoint.

C. Place the EC2 instances in private subnets. Create an Amazon Route 53 private hosted zone for the AWS reserved domain name. Associate the private hosted zone with the VPC. Create a Route 53 Resolver inbound endpoint. Configure conditional forwarding in the on-premises DNS resolvers to forward all DNS queries for the AWS domain to the inbound endpoint IP address for Route 53 Resolver. In the private hosted zone, configure primary and failover records that point to the IP addresses of the EC2 instances. Create an Amazon CloudWatch metric and alarm to monitor the application's health. Set up a health check on the alarm for the primary application endpoint.

D. Place the EC2 instances in private subnets. Create an Amazon Route 53 private hosted zone for the AWS reserved domain name. Associate the private hosted zone with the VPC. Create a Route 53 Resolver inbound endpoint. Configure conditional forwarding in the on-premises DNS resolvers to forward all DNS queries for the AWS domain to the inbound endpoint IP address for Route 53 Resolver. In the private hosted zone, configure primary and failover records that point to the IP addresses of the EC2 instances. Set up Route 53 health checks on the private IP addresses of the EC2 instances.

Answer: C

Explanation:

The correct solution is C. Here's a detailed justification:

- 1. Private Subnets and Private IP Addresses:** Placing the EC2 instances in private subnets ensures that they are not directly exposed to the internet, fulfilling the requirement to avoid public internet exposure. Using the EC2 instances' private IP addresses for DNS resolution keeps all traffic within the private network.
- 2. Route 53 Private Hosted Zone:** A private hosted zone in Route 53 is specifically designed for internal DNS resolution within a VPC. Associating the private hosted zone with the VPC allows instances within the VPC to resolve the private domain names.
- 3. Route 53 Resolver Inbound Endpoint:** An inbound endpoint allows on-premises DNS resolvers to

forward DNS queries for the AWS-reserved domain name to Route 53. This enables on-premises users to resolve the private domain names associated with the EC2 instances.

4. **Conditional Forwarding:** Configuring conditional forwarding in the on-premises DNS resolvers ensures that only queries for the specific AWS domain are forwarded to the Route 53 Resolver inbound endpoint. All other DNS queries remain within the on-premises network.
5. **Primary and Failover Records:** Configuring primary and failover records within the private hosted zone allows Route 53 to automatically route traffic to a healthy EC2 instance. This provides automatic failover without requiring manual intervention. The records point to the private IP addresses of the EC2 instances to ensure traffic stays within the private network.
6. **CloudWatch Metric and Alarm with Health Check:** Setting up a CloudWatch metric and alarm to monitor the application's health provides a mechanism to detect failures. The health check associated with the alarm monitors the primary application endpoint and triggers a failover if the endpoint becomes unhealthy.

Option A is incorrect because it uses public IP addresses, which directly exposes the application to the internet and violates the requirement. Option B is incorrect because it uses a public hosted zone when a private hosted zone is needed to resolve the domain within the VPC and on-premise. Option D is incorrect because using Route 53 health checks on the private IPs of the EC2 instances isn't as reliable as monitoring the application's health using a CloudWatch metric and alarm, as the latter provides application-level health monitoring instead of just network connectivity.

Authoritative Links:

Route 53 Private Hosted Zones: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/private-hosted-zones.html>

Route 53 Resolver Endpoints: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver.html>

CloudWatch Metrics and Alarms:

<https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/AlarmThatSendsEmail.html>

Question: 175

CertyIQ

A company uses Amazon Route 53 for its DNS needs. The company's security team wants to update the DNS infrastructure to provide the most recent security posture.

The security team has configured DNS Security Extensions (DNSSEC) for the domain. The security team wants a network engineer to explain who is responsible for the rotation of DNSSEC keys.

Which explanation should the network administrator provide to the security team?

- A. AWS rotates the zone-signing key (ZSK). The company rotates the key-signing key (KSK).
- B. The company rotates the zone-signing key (ZSK) and the key-signing key (KSK).
- C. AWS rotates the AWS Key Management Service (AWS KMS) key and the key-signing key (KSK).
- D. The company rotates the AWS Key Management Service (AWS KMS) key. AWS rotates the key-signing key (KSK).

Answer: A

Explanation:

The correct answer is A. AWS rotates the zone-signing key (ZSK). The company rotates the key-signing key (KSK).

Here's why:

DNSSEC involves cryptographic keys to sign DNS records, ensuring their authenticity and integrity. Two primary keys are involved: the Zone Signing Key (ZSK) and the Key Signing Key (KSK).

Zone Signing Key (ZSK): This key is used to directly sign the DNS records within a zone. Because it's used frequently, it's typically rotated more often to minimize the impact of a potential compromise. AWS Route 53 automatically handles the rotation of the ZSK when DNSSEC is enabled. This simplifies management for the user and provides a layer of security that's continually refreshed without manual intervention.

Key Signing Key (KSK): The KSK signs the DNSKEY record, which includes the ZSK's public key. The KSK is a more important key from a trust perspective, as it anchors the chain of trust. Therefore, AWS allows the customer to rotate the KSK, enabling greater control over this critical aspect of DNSSEC. The KSK rotation is a less frequent, but highly sensitive operation, and this answer rightly states the customer/company is responsible for this rotation.

Options B, C, and D are incorrect because they misattribute the responsibility for key rotation. Option B claims the company rotates both, which is incorrect as AWS handles the ZSK. Option C incorrectly includes AWS KMS and states AWS handles the KSK. While KMS might be used to store the keys, KMS does not automate DNSSEC key rotation. Option D also misattributes responsibilities.

Therefore, the explanation that AWS rotates the ZSK and the company rotates the KSK aligns with the documented best practices and the delegation of responsibilities in Route 53's DNSSEC implementation. <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/dns-configuring-dnssec.html><https://aws.amazon.com/blogs/security/how-to-automate-key-signing-key-roll-overs-for-dnssec-signing-using-aws-cloudhsm/>

Question: 176

CertyIQ

A company has agreed to collaborate with a partner for a research project. The company has multiple VPCs in the us-east-1 Region that use CIDR blocks within 10.10.0.0/16. The VPCs are connected by a transit gateway that is named TGW-C in us-east-1. TGW-C has an Autonomous System Number (ASN) configuration value of 64520.

The partner has multiple VPCs in us-east-1 that use CIDR blocks within 172.16.0.0/16. The VPCs are connected by a transit gateway that is named TGW-P in us-east-1. TGW-P has an ASN configuration value of 64530.

A network engineer needs to establish network connectivity between the company's VPCs and the partner's VPCs in us-east-1.

Which solution will meet these requirements with MINIMUM changes to both networks?

- A. Create a new VPC in a new account. Deploy a router from AWS Marketplace. Share TGW-C and TGW-P with the new account by using AWS Resource Access Manager (AWS RAM). Associate TGW-C and TGW-P with the new VPC. Configure the router in the new VPC to route between TGW-C and TGW-P.
- B. Create an IPsec VPN connection between TGW-C and TGW-P. Configure the routing between the transit gateways to use the IPsec VPN connection.
- C. Configure a cross-account transit gateway peering attachment between TGW-C and TGW-P. Configure the routing between the transit gateways to use the peering attachment.
- D. Share TGW-C with the partner account by using AWS Resource Access Manager (AWS RAM). Associate the partner VPCs with TGW-C. Configure routing in the partner VPCs and TGW-C.

Answer: C

Explanation:

The correct answer is **C. Configure a cross-account transit gateway peering attachment between TGW-C and TGW-P. Configure the routing between the transit gateways to use the peering attachment.**

Here's a detailed justification:

Transit gateway peering allows you to seamlessly connect transit gateways across AWS accounts and/or AWS Regions. This creates a direct, high-bandwidth connection between the transit gateways, enabling routing between the connected networks. This is the most straightforward method for connecting the company's and partner's networks.

Option A introduces unnecessary complexity and cost by requiring a new VPC, account, and a third-party router. It also necessitates managing shared transit gateways via AWS RAM and configuring the routing device, making it more complex than direct peering.

Option B, creating an IPsec VPN connection, is a viable solution, but it is less performant and more complex to configure and manage than transit gateway peering. VPNs add encryption overhead and require managing VPN gateways and tunnels.

Option D, sharing TGW-C with the partner account and associating the partner VPCs, might seem simple, but it could lead to administrative and security concerns. The company would be giving the partner direct access to their transit gateway, which is not best practice for separate organizational domains.

Transit gateway peering provides a simplified, scalable, and secure solution for connecting networks managed by different entities without requiring significant architectural changes. It leverages the native AWS networking infrastructure for optimal performance and minimal administrative overhead. Peering relies on AWS's global network backbone and provides a managed, highly available connection.

For further reading, refer to AWS documentation on Transit Gateway Peering:

[AWS Transit Gateway Peering Attachments](#)
[Transit Gateway Inter-Region Peering](#)

Question: 177

CertyIQ

A company has a public application. The application uses an Application Load Balancer (ALB) that has a target group of Amazon EC2 instances.

The company wants to protect the application from security issues in web requests. The traffic to the application must have end-to-end encryption.

Which solution will meet these requirements?

- A. Configure a Network Load Balancer (NLB) that has a target group of the existing EC2 instances. Configure TLS connections to terminate on the EC2 instances that use a public certificate. Configure an AWS WAF web ACL. Associate the web ACL with the NLB.
- B. Configure TLS connections to terminate at the ALB that uses a public certificate. Configure AWS Certificate Manager (ACM) certificates for the communication between the ALB and the EC2 instances. Configure an AWS WAF web ACL. Associate the web ACL with the ALB.
- C. Configure a Network Load Balancer (NLB) that has a target group of the existing EC2 instances. Configure TLS connections to terminate at the EC2 instances by creating a TLS listener. Configure self-signed certificates on the EC2 instances for the communication between the NLB and the EC2 instances. Configure an AWS WAF web ACL. Associate the web ACL with the NLB.
- D. Configure a third-party certificate on the EC2 instances for the communication between the ALB and the EC2 instances. Import the third-party certificate into AWS Certificate Manager (ACM). Associate the imported certificate with the ALB. Configure TLS connections to terminate at the ALB. Configure an AWS WAF web ACL. Associate the web ACL with the ALB.

Answer: B

Explanation:

The correct answer is B because it provides both end-to-end encryption and protection against web request security issues using AWS WAF.

Here's a detailed breakdown:

TLS Termination at ALB: Configuring TLS connections to terminate at the ALB with a public certificate is essential for decrypting traffic and allowing the ALB to inspect requests. This inspection is crucial for AWS WAF to function effectively. The ALB acts as the entry point and handles initial security checks before forwarding the traffic to the EC2 instances.

ACM Certificates for ALB-EC2 Communication: Using AWS Certificate Manager (ACM) certificates for communication between the ALB and the EC2 instances ensures end-to-end encryption. ACM is a service that lets you easily provision, manage, and deploy SSL/TLS certificates for use with AWS services and your internal connected resources. This encrypts traffic even within the AWS environment, protecting data in transit.

AWS WAF Web ACL Association: AWS WAF (Web Application Firewall) protects web applications from common web exploits and bots. By associating a WAF web ACL (Access Control List) with the ALB, the WAF can inspect incoming HTTP/HTTPS requests and block malicious requests, SQL injection attacks, cross-site scripting (XSS) attempts, and other web vulnerabilities. The ALB, having visibility to the decrypted traffic, can forward it to the WAF rules for inspection.

Option A is incorrect because NLBs do not inspect traffic; they simply forward it based on network layer information (IP address, port). Thus, a WAF associated with an NLB would not be able to provide the necessary protection against web application vulnerabilities.

Option C is incorrect because using self-signed certificates is generally discouraged in production environments. Self-signed certificates are not trusted by browsers and require manual exceptions, leading to a poor user experience and potential security warnings. Also, it uses NLB, which can't be associated with WAF.

Option D is incorrect because the statement mentions using a third-party certificate on the EC2 instances for communication between the ALB and EC2, then importing the third-party certificate into ACM. While importing a third-party certificate into ACM is perfectly fine and a valid use case, placing the certificate directly on the EC2 instances is not necessary, and more importantly, doesn't specify using ACM for encryption between the ALB and the EC2 instances. It also doesn't state the certificate is used in the target group.

Supporting Links:

AWS Certificate Manager (ACM): <https://aws.amazon.com/certificate-manager/>

AWS WAF: <https://aws.amazon.com/waf/>

Application Load Balancer (ALB): <https://aws.amazon.com/elasticloadbalancing/application-load-balancer/>

Network Load Balancer (NLB): <https://aws.amazon.com/elasticloadbalancing/network-load-balancer/>

Question: 178

CertyIQ

A company has an application that hosts personally identifiable information (PII) of users. All connections to the application must be secured by HTTPS with TLS certificates that implement Elliptic Curve Cryptography (ECC).

The application uses stateful connections between the web tier and the end users. Multiple instances host the application. A network engineer must implement a solution that offloads TLS connections to a load balancer.

Which load-balancing solution will meet these requirements?

A.Provision a Network Load Balancer. Configure a TLS listener by specifying the use of an ECC SSL certificate

that is uploaded to AWS identity and Access Management (IAM). Turn on health checks to monitor the web hosts that connect to the end users.

B.Provision an Application Load Balancer. Configure an HTTPS listener by specifying the use of an ECC SSL certificate that is uploaded to AWS Certificate Manager (ACM). Configure a default action to redirect to the URL for the application. Turn on health checks to monitor the web hosts that connect to the end users.

C.Provision a Network Load Balancer. Configure a TLS listener by specifying the use of an ECC SSL certificate that is uploaded to AWS Certificate Manager (ACM). Turn on application-based session affinity (sticky sessions). Turn on health checks to monitor the web hosts that connect to the end users.

D.Provision an Application Load Balancer. Configure an HTTPS listener by specifying the use of an ECC SSL certificate that is uploaded to AWS Identity and Access Management (IAM). Configure a default action to redirect to the URL for the application. Turn on application-based session affinity (sticky sessions).

Answer: C

Explanation:

Here's a detailed justification for why option C is the correct answer, along with supporting explanations and relevant links:

The key requirements are: HTTPS with ECC certificates, TLS offloading at the load balancer, stateful (sticky) sessions, and health checks.

TLS Offloading & ECC Certificates: Both Application Load Balancers (ALB) and Network Load Balancers (NLB) can handle TLS termination (offloading). AWS Certificate Manager (ACM) is the recommended service for managing and deploying SSL/TLS certificates, including ECC certificates. Storing certificates in ACM simplifies the process compared to IAM.

Stateful Connections (Sticky Sessions): The application uses stateful connections. NLB doesn't support application-based sticky sessions directly via cookies like ALB. However, NLB can maintain session affinity based on source IP addresses and ports, or a combination of these, which could function similarly to sticky sessions for some applications, albeit less granular than cookie-based affinity. The question implies state is maintained at the application layer, hence the need for "application-based" session affinity.

Health Checks: Both ALB and NLB support health checks to monitor the backend instances.

Why C is better than the others:

Option A: Storing certificates in IAM is not the preferred or best-practice method compared to using ACM.

Option B: While ALB does support cookie-based sticky sessions, it doesn't provide inherent IP-based affinity. ALBs typically don't maintain session affinity directly without cookie configuration. Also, redirecting traffic to the application URL might introduce extra latency and is not directly related to TLS offloading or session affinity.

Option D: The certificate upload to IAM is incorrect; it should be ACM.

Option C is correct: While not ideal for very complex sticky session requirements, NLB can maintain session affinity based on IP, which may be sufficient for some applications needing stateful connections. Furthermore, ACM is preferred for certificate management.

Therefore, option C aligns best with the requirements, especially regarding certificate management with ACM, even if the session affinity aspect requires careful configuration. The question likely implies a simplified sticky session requirement.

Supporting Links:

AWS Certificate Manager (ACM): <https://aws.amazon.com/certificate-manager/>

Network Load Balancer (NLB):

<https://docs.aws.amazon.com/elasticloadbalancing/latest/network/introduction.html>

Application Load Balancer (ALB):

<https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html>

ELB Listener Configuration: <https://docs.aws.amazon.com/elasticloadbalancing/latest/network/load-balancer-listeners.html>

Question: 179

CertyIQ

A company hosts infrastructure services in multiple VPCs across multiple accounts in the us-west-2 Region. The VPC CIDR blocks do not overlap. The company wants to connect the VPCs to its data centers by using AWS Site-to-Site VPN tunnels.

The connections must be encrypted in transit. Additionally, the connection from each data center must route to the closest AWS edge location. The connections must be highly available and must accommodate automatic failover.

Which solution will meet these requirements?

A. Deploy a transit gateway. Share the transit gateway with each of the other accounts by using AWS Resource Access Manager (AWS RAM). Create VPC attachments to the transit gateway from each service account. Add routes to the on-premises subnet in each of the service VPC route tables by using the attachment as the gateway. Create Site-to-Site VPN tunnel attachments with dynamic routing to the transit gateway. Enable the acceleration feature for the Site-to-Site VPN connection. Configure the VPN tunnels on the on-premises equipment. Configure BGP peering.

B. Deploy VPN gateways to each account. Enable the acceleration feature for VPN gateways on each account. Add routes to the on-premises subnet in each of the service VPC route tables. Use the VPNs as the gateway. Configure the VPN tunnels on the on-premises equipment. Configure BGP peering.

C. Deploy a transit gateway. Share the transit gateway with each of the other accounts by using AWS Resource Access Manager (AWS RAM). Create VPC attachments to the transit gateway from each service account. Add routes to the on-premises subnet in each of the service VPC route tables by using the attachment as the gateway. Create Site-to-Site VPN tunnel attachments with dynamic routing to the transit gateway. Enable the acceleration feature for the Site-to-Site VPN connection. Configure the VPN tunnels on the on-premises equipment. Configure static routing.

D. Deploy VPN gateways to each account. Enable the acceleration feature for VPN gateways on each account. Add routes to the on-premises subnet in each of the service VPC route tables. Use the VPNs as the gateway. Configure the VPN tunnels on the on-premises equipment. Configure static routing.

Answer: A

Explanation:

Here's why option A is the best solution, along with a detailed explanation and supporting resources:

Justification:

The primary goal is to connect multiple VPCs across multiple accounts to on-premises data centers with encryption, high availability, and routing to the nearest AWS edge location. A Transit Gateway (TGW) simplifies this network architecture. Here's a breakdown of why other options fall short and why option A excels:

Transit Gateway (TGW) is central: A TGW acts as a hub, consolidating connectivity between multiple VPCs and on-premises locations. This avoids the complexity of managing multiple point-to-point VPN connections or VPC peering relationships across accounts. <https://aws.amazon.com/transit-gateway/>

AWS RAM for Multi-Account Management: AWS Resource Access Manager (RAM) enables sharing the TGW with other AWS accounts, allowing each account to attach its VPCs to the central TGW without needing to create TGWs in each account. <https://aws.amazon.com/ram/>

VPC Attachments: Each service account creates a VPC attachment to the shared TGW. This establishes the

connection between the VPC and the TGW.

Route Tables: Route tables in each VPC are updated to route traffic destined for the on-premises network through the TGW attachment. This allows communication between the VPCs and the data centers.

Site-to-Site VPN with Dynamic Routing (BGP): Using Site-to-Site VPN to connect the data centers to the TGW ensures encrypted connectivity. Dynamic routing using Border Gateway Protocol (BGP) is crucial. BGP allows the on-premises routers and the TGW to exchange routing information automatically. This allows the TGW to learn the available routes to the data center and vice-versa. BGP handles failover automatically by advertising alternative paths when one path fails. This ensures high availability.

<https://docs.aws.amazon.com/vpn/latest/s2svpn/VPNRoutingTypes.html>

VPN Acceleration: Enabling the acceleration feature for Site-to-Site VPN connections allows routing traffic to the closest AWS edge location. This uses AWS Global Accelerator to minimize latency.

<https://aws.amazon.com/global-accelerator/>

Why other options are incorrect:

Options B and D (VPN Gateways per Account): While VPN Gateways provide connectivity, they lack the centralized management and scalability of a TGW for multiple VPCs across multiple accounts. Managing individual VPN connections becomes increasingly complex. Also, sharing VPN Gateways across accounts is not a standard AWS feature.

Static Routing (Options C and D): Static routing is inflexible and does not automatically adapt to network changes or failures. It requires manual updates whenever the network topology changes. This defeats the purpose of high availability.

In summary, using a TGW shared across accounts via AWS RAM, Site-to-Site VPN with dynamic routing (BGP), and VPN acceleration delivers the necessary connectivity, security, high availability, and optimized routing for this scenario.

Question: 180

CertyIQ

A company has a transit gateway in AWS Account A. The company uses AWS Resource Access Manager (AWS RAM) to share the transit gateway so that users in other accounts can connect to multiple VPCs in the same AWS Region. AWS Account B contains a VPC (10.0.0.0/16) with subnet 10.0.0.0/24 in the us-west-2a Availability Zone and subnet 10.0.1.0/24 in the us-west-2b Availability Zone. Resources in these subnets can communicate with other VPCs.

A network engineer creates two new subnets: 10.0.2.0/24 in the us-west-2b Availability Zone and 10.0.3.0/24 in the us-west-2c Availability Zone. All the subnets share one route table. The default route 0.0.0.0/0 is pointing to the transit gateway. Resources in subnet 10.0.2.0/24 can communicate with other VPCs, but resources in subnet 10.0.3.0/24 cannot communicate with other VPCs.

What should the network engineer do so that resources in subnet 10.0.3.0/24 can communicate with other VPCs?

- A. In Account B, add 10.0.2.0/24 and 10.0.3.0/24 as the destinations to the route table. Use the transit gateway as the target.
- B. In Account B, update the transit gateway attachment. Attach the new subnet ID that is associated with us-west-2c to Account B's VPC.
- C. In Account A, create a static route for 10.0.3.0/24 in the transit gateway route tables.
- D. In Account A, recreate propagation for 10.0.0.0/16 in the transit gateway route tables.

Answer: C

Explanation:

The correct answer is **C. In Account A, create a static route for 10.0.3.0/24 in the transit gateway route tables.**

Here's why:

The issue is that resources in subnet 10.0.3.0/24 cannot communicate with other VPCs via the transit gateway. This indicates a routing problem within the transit gateway itself. The transit gateway needs to know where to send traffic destined for 10.0.3.0/24. Because the VPC (10.0.0.0/16) is already associated with the transit gateway, the transit gateway has an attachment for the VPC. However, simply having an attachment doesn't guarantee that the transit gateway knows about specific subnets within the VPC.

When using Transit Gateway with shared attachments, propagation allows routes from connected VPCs (in this case the 10.0.0.0/16 VPC in Account B) to be automatically learned by the Transit Gateway route table. However, propagation doesn't always cover all subnets, especially if they are added after the initial configuration.

Option A is incorrect because the VPC route table already has a default route (0.0.0.0/0) pointing to the transit gateway. Adding specific routes for 10.0.2.0/24 and 10.0.3.0/24 to the VPC route table is redundant and unnecessary as traffic will be directed to the Transit Gateway anyway using the default route.

Option B is incorrect because transit gateway attachments are at the VPC level, not at the subnet level. You don't "attach" specific subnets to a transit gateway attachment. The attachment represents the connection between the transit gateway and the entire VPC.

Option D is not the most direct approach. Recreating propagation for the entire VPC CIDR (10.0.0.0/16) is potentially disruptive and might not be required. While it could potentially solve the problem if the original propagation somehow missed the new subnet, creating a static route for 10.0.3.0/24 is a more targeted and efficient solution.

By creating a static route for 10.0.3.0/24 in the transit gateway's route table (in Account A), you explicitly tell the transit gateway that any traffic destined for 10.0.3.0/24 should be sent to the VPC attachment in Account B. This ensures that traffic from other VPCs connected to the transit gateway can reach subnet 10.0.3.0/24. This solution targets the precise source of the problem, the Transit Gateway not knowing the specific subnet.

Here's a relevant link for more information on transit gateway routing:

[AWS Transit Gateway Routing](#)

Question: 181

CertyIQ

A company has started using AWS Cloud WAN with one edge location in the us-east-1 Region. The company has a production segment and a security segment in AWS Cloud WAN. The company also has a default core network policy.

The company has created a production VPC for the production workload. The company has created an outbound inspection VPC to inspect internet-bound traffic from the production VPC. The company has attached the production VPC to the production segment and has attached the outbound inspection VPC to the security segment. The company has also created an AWS Network Firewall firewall in the outbound inspection VPC to inspect internet-based traffic.

The company has updated a route table for the production VPC to send all internet-bound traffic to the AWS Cloud WAN core network. The company has updated a route table for the outbound inspection VPC to ensure that Network Firewall inspects any outgoing traffic and incoming traffic.

During testing, an Amazon EC2 instance in the production VPC cannot reach the internet. The company checks the Network Firewall rules and confirms that the rules are not blocking the traffic.

Which combination of steps will meet these requirements? (Choose two.)

- A. Update the core network policy to configure segment sharing. Share the production segment with the security segment.
- B. Update the core network policy to create a static route for the security segment. Specify 0.0.0.0/0 as the destination CIDR block. Specify the outbound inspection VPC as an attachment.
- C. Update the core network policy to create a static route for the production segment. Specify 0.0.0.0/0 as the destination CIDR block. Specify the outbound inspection VPC as an attachment.
- D. Update the core network policy to create a static route for the production segment. Specify 10.2.0.0/16 as the destination CIDR block. Specify the outbound inspection VPC as an attachment.
- E. Create an attachment to attach the outbound inspection VPC to the production segment. Update the core network policy to turn on isolated attachment for the production segment.

Answer: AC

Explanation:

The problem describes a scenario where a production VPC in a Cloud WAN setup cannot reach the internet despite proper Network Firewall configuration. The issue lies in the traffic flow between the production segment and the security segment where the outbound inspection VPC resides. The production VPC needs to send traffic to the internet, but the traffic is expected to be inspected first by the firewall in the security segment.

Option A: Updating the core network policy to configure segment sharing and sharing the production segment with the security segment allows traffic from the production segment to reach the security segment. Without segment sharing, the segments are isolated, and traffic cannot flow between them. This is essential for routing traffic from the production VPC to the outbound inspection VPC for inspection.

Option C: Updating the core network policy to create a static route for the production segment with a destination CIDR block of 0.0.0.0/0 and specifying the outbound inspection VPC as an attachment ensures that all internet-bound traffic from the production segment is routed through the outbound inspection VPC. This static route acts as the default gateway for the production segment's internet-bound traffic.

Why other options are incorrect:

Option B: Creating a static route for the security segment would affect traffic originating from the security segment, not traffic originating from the production segment.

Option D: Creating a static route for the production segment with 10.2.0.0/16 as the destination CIDR block would only route traffic destined for that specific CIDR block to the outbound inspection VPC, not all internet-bound traffic (0.0.0.0/0).

Option E: Creating an attachment to attach the outbound inspection VPC to the production segment might seem like a solution, but it would bypass the intended architecture of using the security segment for inspection. Also, enabling isolated attachment would prevent traffic between the production segment and other segments.

Supporting Links:

AWS Cloud WAN Core Network Policies: <https://docs.aws.amazon.com/cloud-wan/latest/working-with/core-network-policies.html>

AWS Cloud WAN Segments: <https://docs.aws.amazon.com/cloud-wan/latest/working-with/segments.html>

Question: 182

CertyIQ

A company has two business units (BUs). The company operates in the us-east-1 Region and the us-west-1 Region. The company plans to extend to more Regions in the future. Each BU has a VPC in each Region. Each Region has a transit gateway with the BU VPCs attached. The transit gateways in both Regions are peered.

The company will create several more BUs in the future and will need to isolate some of the BUs from the other

BU's. The company wants to migrate to an architecture to incorporate more Regions and BU's.

Which solution will meet these requirements with the MOST operational efficiency?

- A. Create a new transit gateway for each new BU in each Region. Peer the new transit gateways with the existing transit gateways. Update the route tables to control traffic between BU's.
- B. Create an AWS Cloud WAN core network with an edge location in both Regions. Configure a segment for each BU with VPC attachments to the new BU VPCs. Use segment actions to control traffic between segments.
- C. Create an AWS Cloud WAN core network with an edge location in both Regions. Configure a segment for each BU with VPC attachments to the new BU VPCs. Configure the segments to isolate attachments to control traffic between segments.
- D. Attach new VPCs to the existing transit gateways. Update route tables to control traffic between BU's.

Answer: B

Explanation:

The best solution for the company's requirements of isolation and operational efficiency when expanding to more Regions and Business Units (BU's) is to use AWS Cloud WAN with segments. Here's why:

Option B leverages AWS Cloud WAN, which is designed for building and managing global networks. Cloud WAN simplifies network management by providing a central dashboard to manage connectivity across multiple Regions. Creating a segment for each BU allows for network isolation and traffic control. VPC attachments connect each BU's VPCs to their respective segments. Segment actions then enable fine-grained control over traffic flow between segments, fulfilling the isolation requirement. This centralized management model minimizes operational overhead as the company scales.

Option A, while technically feasible, introduces significant operational complexity. Creating and peering multiple transit gateways per BU in each Region rapidly becomes unwieldy. Managing route tables across numerous transit gateways for traffic control is error-prone and difficult to scale.

Option C is similar to Option B but utilizes attachment isolation. While this provides isolation, segment actions in option B offer more granular control for selectively allowing traffic between BU's if needed, providing flexibility.

Option D extends the current transit gateway architecture, which is not designed for the level of isolation and scalability required. Modifying route tables to isolate BU's is complex and difficult to manage as the number of BU's and Regions grows.

Therefore, Option B offers the most operationally efficient solution by centralizing network management with AWS Cloud WAN, providing BU isolation through segments, and enabling granular traffic control via segment actions.

Further reading:

AWS Cloud WAN: <https://aws.amazon.com/cloud-wan/>

AWS Cloud WAN Segments: <https://docs.aws.amazon.com/cloudwan/latest/userguide/cloudwan-segments.html>

Question: 183

CertyIQ

A company has an AWS Site-to-Site VPN connection between AWS and its branch office. A network engineer is troubleshooting connectivity issues that the connection is experiencing. The VPN connection terminates at a transit gateway and is statically routed. In the transit gateway route table, there are several static route entries that target specific subnets at the branch office.

The network engineer determines that the root cause of the issues was the expansion of underlying subnet ranges

in the branch office during routine maintenance.

Which solution will solve this problem with the LEAST administrative overhead for future expansion efforts?

A. Determine a supernet for the branch office. In the transit gateway route table, add an aggregate route that targets the VPN attachment. Replace the specific subnet routes in the transit gateway route table with the new supernet route.

B. Create an AWS Direct Connect gateway and a transit VIF. Associate the Direct Connect gateway with the transit gateway. Create a propagation for the Direct Connect attachment to the transit gateway route table.

C. Create a dynamically routed VPN connection on the transit gateway. Connect the dynamically routed VPN connection to the branch office. Create a propagation for the VPN attachment to the transit gateway route table. Remove the existing static VPN connection.

D. Create a prefix list that contains the new subnets and the old subnets for the branch office. Remove the specific subnet routes in the transit gateway route table. Create a prefix list reference in the transit gateway route table.

Answer: A

Explanation:

Here's a detailed justification for why option A is the best solution, along with supporting explanations and links:

The core problem is that the static routes in the transit gateway's route table are too specific, pointing to individual subnets in the branch office. When those subnets change, the routes become invalid, causing connectivity issues. We need a more scalable and manageable approach that minimizes manual updates when the branch office network expands.

Option A provides the most efficient solution by leveraging route aggregation. By identifying a supernet that encompasses all the branch office subnets (present and future), we can create a single, broader route in the transit gateway's route table. This aggregate route will cover any subnets within the branch office range, regardless of future expansions within that supernet. This dramatically reduces administrative overhead, as only one route needs to be maintained instead of multiple specific subnet routes. Using the VPN attachment as the target for the aggregate route ensures that all traffic destined for the branch office (via the supernet) is correctly routed through the existing VPN connection.

Option B introduces an entirely new connectivity mechanism (Direct Connect) which is significantly more complex and expensive than necessary for a simple route update problem. Direct Connect is suitable for high-bandwidth, low-latency requirements, which are not indicated in the scenario.

Option C, while dynamically routing does offer automatic route updates, it requires replacing the existing static VPN connection entirely and setting up a new dynamically routed VPN connection. This is more disruptive and complex than simply modifying routes in the transit gateway route table. BGP configuration and ongoing route negotiation can also add management overhead.

Option D, using prefix lists, can also provide a solution to group and manage CIDRs that could change. However, prefix lists still require updating when new subnets are added to the branch office network. While they simplify management compared to individual routes, they don't eliminate the need for updates in the same way that route aggregation does. Furthermore, route aggregation is often simpler to implement and understand.

Therefore, route aggregation with a supernet provides the most elegant and least administratively intensive solution for accommodating future network expansions in the branch office, leveraging the existing infrastructure effectively.

Supporting Links:

Transit Gateway Route Tables: <https://docs.aws.amazon.com/vpc/latest/tgw/tgw-route-tables.html>

Route Aggregation: <https://www.ciscopress.com/articles/article.asp?p=27563&seqNum=5> (Although Cisco-centric, the concepts of route aggregation apply generally to networking)

Prefix Lists: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-prefix-lists.html>

Question: 184

CertyIQ

An education agency is preparing for its annual competition between schools. In the competition, students at schools from around the country solve math problems, complete puzzles, and write essays.

The IP addressing plan of all the schools is well-known and is administered centrally. The competition is hosted in the AWS Cloud and is not publicly available. All competition traffic must be encrypted in transit. Only authorized endpoints can access the competition. All the schools have firewall policies that block ICMP traffic.

A network engineer builds a solution in which all the schools access the competition through AWS Site-to-Site VPN connections. The network engineer uses BGP as the routing protocol. The network engineer must implement a solution that notifies schools when they lose connectivity and need to take action on their premises to address the issue.

Which combination of steps will meet these requirements MOST cost-effectively? (Choose two.)

- A. Monitor the state of the VPN tunnels by using Amazon CloudWatch. Create a CloudWatch alarm that uses Amazon Simple Notification Service (Amazon SNS) to notify people at the affected school if the tunnels are down.
- B. Create a scheduled AWS Lambda function that pings each school's on-premises customer gateway device. Configure the Lambda function to send an Amazon Simple Notification Service (Amazon SNS) notification to people at the affected school if the ping fails.
- C. Create a scheduled AWS Lambda function that uses the VPC Reachability Analyzer API to verify the connectivity. Configure the Lambda function to send an Amazon Simple Notification Service (Amazon SNS) notification to people at the affected school if failure occurs.
- D. Create an Amazon CloudWatch dashboard for each school to show all CloudWatch metrics for each school's Site-to-Site VPN connection. Share each dashboard with the appropriate school.
- E. Create a scheduled AWS Lambda function to monitor the existence of each school's routes in the VPC route table where VPN routes are propagated. Configure the Lambda function to send an Amazon Simple Notification Service (Amazon SNS) notification to people at the affected school if failure occurs.

Answer: AC

Explanation:

The solution requires a cost-effective way to notify schools when their AWS Site-to-Site VPN connections fail, given they use BGP, block ICMP, and have centrally managed IP addressing.

Option A is correct because it leverages built-in AWS services. CloudWatch monitors VPN tunnel status, which directly indicates connection health. [<https://docs.aws.amazon.com/vpn/latest/s2svpn/monitoring-cloudwatch.html>] If the tunnel goes down, a CloudWatch alarm triggers an SNS notification to the affected school. This is a straightforward and cost-effective approach as CloudWatch and SNS are designed for such monitoring and alerting.

Option C is also correct. The VPC Reachability Analyzer can verify network connectivity between two endpoints. [<https://docs.aws.amazon.com/vpc/latest/reachability/what-is-reachability-analyzer.html>] A scheduled Lambda function can use the Reachability Analyzer API to periodically check connectivity from AWS to the school's on-premises network (customer gateway). Upon failure, an SNS notification informs the school. This approach is robust as it verifies end-to-end connectivity beyond just the tunnel status.

Option B is incorrect because the problem states that the schools firewall policies block ICMP traffic. Therefore a ping would not function correctly.

Option D is incorrect because creating a dashboard and manually sharing it with each school to actively

monitor it manually is not cost-effective. It requires manual effort and proactive monitoring by the schools, rather than automated notification.

Option E is incorrect because relying solely on route existence in the route table is insufficient. Routes might still be present even if the tunnel is experiencing issues or the BGP session is down, as routes are often retained for some time after a connectivity loss. This option doesn't provide a reliable indication of actual connectivity.

Question: 185

CertyIQ

A company securely connects resources that are in its VPC to a software as a service (SaaS) solution from a SaaS provider. The SaaS solution is hosted in the AWS Cloud and is powered by AWS PrivateLink. The company uses a PrivateLink endpoint to access the SaaS solution behind the SaaS provider's Network Load Balancer (NLB).

The company recently added a new Availability Zone and new subnets to its VPC. A network engineer is unable to deploy a new interface VPC endpoint for the SaaS solution in the new Availability Zone.

What is the cause of this problem?

- A. The CIDR block of the new subnets conflicts with the SaaS provider's CIDR block.
- B. The `enableDnsHostnames` attribute and `enableDnsSupport` attribute were not configured on the new subnets in the new Availability Zone.
- C. The SaaS provider does not offer the solution in the new Availability Zone and has not configured cross-zone load balancing for the NLB.
- D. The new subnets are missing a route to the VPC internet gateway.

Answer: C

Explanation:

The problem stems from the possibility that the SaaS provider's service, backed by the NLB, is not available in the newly added Availability Zone. Interface VPC endpoints need corresponding services to be present in the same Availability Zone to function. Network Load Balancers support cross-zone load balancing, but it's the SaaS provider's responsibility to configure it. If the provider hasn't enabled this, traffic from the new Availability Zone cannot reach the SaaS application through the NLB.

Option A is incorrect because CIDR block overlaps would prevent subnet creation in the first place. Option B is less likely because DNS settings, while important, don't directly block endpoint creation. Option D is irrelevant because PrivateLink is a private connectivity solution that doesn't require an internet gateway. PrivateLink connections use the AWS backbone network and do not traverse the public internet.

Therefore, the most probable reason is that the SaaS provider's NLB either doesn't have instances in the new AZ or cross-zone load balancing is not enabled. The interface endpoint will not function correctly unless the underlying service is provisioned in that Availability Zone.

<https://docs.aws.amazon.com/vpc/latest/privatelink/vpc-endpoints.html><https://docs.aws.amazon.com/elasticloadbalancing/latest/network/cross-zone-load-balancing.html>

Question: 186

CertyIQ

A company wants to use an AWS Network Firewall firewall to secure its workloads in the cloud through network traffic inspection. The company must record complete metadata information, such as source/destination IP addresses and protocol type. The company must also record all network traffic flows and any DROP or ALERT

actions that the firewall takes for traffic that the firewall processes. The Network Firewall endpoints are placed in the correct subnets, and the VPC route tables direct traffic to the Network Firewall endpoints on the path to and from the internet.

How should a network engineer configure the firewall to meet these requirements?

- A. Create a firewall policy to ensure that traffic is processed by stateless or stateful rules according to needs. Select Amazon CloudWatch Logs as the destination for the flow logs.
- B. Create a firewall policy to ensure that traffic is processed by stateless or stateful rules according to needs. Configure Network Firewall logging for alert logs and flow logs. Select a destination for logs separately for stateful and stateless engines.
- C. Create a firewall policy to ensure that a stateful engine processes all the traffic. Configure Network Firewall logging for alert logs and flow logs. Select a destination for alert logs and flow logs.
- D. Create a firewall policy to ensure that a stateful engine processes all the traffic. Configure VPC flow logs for the subnets that the firewall protects. Select a destination for the flow logs.

Answer: B

Explanation:

The requirement is to capture complete metadata, all network traffic flows, and DROP/ALERT actions from the AWS Network Firewall. Option B correctly addresses these needs by suggesting the creation of a firewall policy that uses both stateless and stateful rules as needed to process traffic. It then emphasizes configuring Network Firewall logging for both alert and flow logs.

Alert logs specifically capture actions taken by the firewall (DROP, ALERT), which aligns perfectly with the requirement to record these actions. Flow logs capture network traffic flows, including source/destination IP addresses, protocol, and port information, thus fulfilling the metadata requirement. Critically, option B explicitly states selecting a destination for logs separately for stateful and stateless engines. This is crucial because it allows granular control and customization over where different types of logs are stored, providing flexibility for analysis and retention policies.

Option A is incorrect because it proposes using CloudWatch Logs as the destination for flow logs instead of utilizing the Network Firewall's built-in flow log functionality. While CloudWatch Logs can be a destination, it's not the direct way Network Firewall handles and configures its own flow and alert logs, meaning you would lose the context from the firewall itself. Option C only uses a stateful engine which isn't necessary. Stateless rules can be useful in some contexts. Option D is also incorrect because it relies on VPC flow logs instead of Network Firewall logs. While VPC flow logs provide valuable information, they don't capture the specific actions taken by the Network Firewall (DROP/ALERT), and also lack the context of the firewall rules applied. The Network Firewall generates logs containing much more detail about the rule violations.

In summary, only option B leverages the Network Firewall's native logging capabilities to capture the required metadata, traffic flows, and firewall actions, offering the flexibility to separately manage stateful and stateless log destinations for optimal control and analysis.

Relevant Documentation:

AWS Network Firewall Logging: <https://docs.aws.amazon.com/network-firewall/latest/developerguide/logging.html>

AWS Network Firewall Rule Actions: <https://docs.aws.amazon.com/network-firewall/latest/developerguide/rule-actions.html>

Question: 187

CertyIQ

A company is building a new workload on AWS that uses an Application Load Balancer (ALB). The company has configured a new ALB target group that uses slow start mode. A team begins registering Amazon EC2 instances as

targets in the new target group. During testing, the team observes that the targets did not enter slow start mode.

What caused the targets to not enter slow start mode?

- A.The ALB configuration uses the round robin routing algorithm for traffic.
- B.The target group did not contain at least one healthy target configured in slow start mode.
- C.The target group must contain EC2 instances that are all the same instance type.
- D.The ALB configuration uses the 5-tuple criteria for traffic.

Answer: B

Explanation:

The correct answer is **B. The target group did not contain at least one healthy target configured in slow start mode.**

Here's why: Slow start mode for Application Load Balancers (ALBs) is designed to gradually ramp up traffic to newly registered targets. However, the ALB needs a baseline to determine what constitutes "normal" performance and health. Without at least one healthy target already running and deemed healthy by the ALB's health checks, the slow start mechanism cannot establish a baseline. The ALB will likely consider any target as unhealthy initially since there's no prior performance context.

When a new target is registered to the target group and slow start is enabled, the ALB will monitor its health checks. The ALB will only start sending gradually increasing traffic to the target if it passes the health checks. If the ALB is unable to send a minimal amount of traffic to the new target because the ALB's Health Checks are reporting failures, slow start won't begin. The ALB will not begin sending traffic to a new target until it determines that the target is Healthy.

Option A is incorrect because the ALB routing algorithm does not affect whether slow start mode is entered. Round robin distributes traffic sequentially among available targets, but the slow start still applies to each target individually. Option C is incorrect because ALB target groups can contain EC2 instances of different types. There is no requirement for all instances in a target group to be the same instance type. Option D is incorrect because the 5-tuple criteria (source IP, source port, destination IP, destination port, protocol) are used for session stickiness and have no effect on slow start mode.

For further research, refer to the AWS documentation on Application Load Balancers and Target Groups:

[Application Load Balancers](#)

[Target Groups for Your Application Load Balancers](#)

[Slow start mode](#)

Question: 188

CertyIQ

A network engineer is using AWS Direct Connect connections and MACsec to encrypt data from a corporate data center to the Direct Connect location. The network engineer learns that the MACsec secret key might have been compromised. The network engineer needs to update the connection with an uncompromised secure key.

Which solution will meet this requirement?

- A.Create a new MACsec secret key that uses an AWS Key Management Service (AWS KMS) AWS managed key. Associate the new pre-shared key, Connection Key Name (CKN), and Connectivity Association Key (CAK) with the connection.
- B.Create a new MACsec secret key that uses an AWS Key Management Service (AWS KMS) customer managed key. Associate the new pre-shared key, Connection Key Name (CKN), and Connectivity Association Key (CAK) with the connection.
- C.Modify the existing MACsec secret key. Re-associate the existing pre-shared key, Connection Key Name

(CKN), and Connectivity Association Key (CAK) with the connection.

D. Modify the existing MACsec secret key. Associate the new pre-shared key, Connection Key Name (CKN), and Connectivity Association Key (CAK) with the connection.

Answer: B

Explanation:

The correct answer is B. Here's why:

Security Best Practices: When a secret key is compromised, it's crucial to replace it entirely, not modify it. Modification might leave traces of the compromised key, reducing security.

AWS KMS Customer Managed Keys: AWS KMS customer managed keys provide greater control and auditability compared to AWS managed keys. With a customer managed key, the network engineer has full control over the key lifecycle, including rotation, deletion, and usage policies. This aligns with security best practices when sensitive information is potentially compromised. AWS managed keys offer less granular control.

MACsec Key Update: To update the MACsec connection with a new key, you create a new Connectivity Association Key (CAK) and Connection Key Name (CKN) along with the secret key. These parameters are then associated with the Direct Connect connection to initiate a key rotation. The new pre-shared key needs to be securely distributed to both ends of the connection (AWS side and the on-premises data center).

Why other options are wrong:

A: While using AWS KMS is a good idea, using AWS managed keys doesn't offer the level of control required in a situation where key compromise is suspected. The limited control over rotation and auditing makes this option less suitable.

C & D: Modifying the existing compromised key is a security risk. This doesn't guarantee that the original compromised key information will be fully eliminated.

Therefore, creating a completely new MACsec secret key using a customer managed KMS key and associating it with the connection is the most secure and appropriate solution to replace a potentially compromised key.

Supporting Links:

AWS Direct Connect MACsec: <https://docs.aws.amazon.com/directconnect/latest/UserGuide/direct-connect-macsec.html>

AWS Key Management Service (KMS): <https://aws.amazon.com/kms/>

AWS KMS Customer Managed Keys:

<https://docs.aws.amazon.com/kms/latest/developerguide/concepts.html#customer-managed-keys>

Question: 189

CertyIQ

A network engineer configures a second AWS Direct Connect connection to an existing network. The network engineer runs a test in the AWS Direct Connect Resiliency Toolkit on the connections. The test produces a failure. During the failover event, the network engineer observes a 90-second interruption before traffic shifts to the failover connection.

Which solution will reduce the time for failover?

A. Decrease the BGP hello timer to 5 seconds.

B. Add a VPN connection to the connectivity solution. Implement fast failover.

C. Configure Bidirectional Forwarding Detection (BFD) on the on-premises router.

D. Decrease the BGP hold-down timer to 5 seconds.

Answer: C

Explanation:

The correct answer is C: Configure Bidirectional Forwarding Detection (BFD) on the on-premises router. Here's why:

The observed 90-second interruption indicates a slow BGP convergence. BGP relies on keepalive messages and hold timers to detect link failures. The default hold timer is typically 90 seconds, meaning BGP will only declare a peer down after not receiving keepalive messages for that duration.

Option C, BFD, offers significantly faster failure detection than BGP's default timers. BFD is a low-overhead protocol that provides sub-second failure detection. By configuring BFD on the on-premises router and AWS Direct Connect connection, the system can detect failures much more quickly than relying solely on BGP timers. This rapid detection triggers a faster route withdrawal and switchover to the backup Direct Connect connection, thereby reducing the failover time. BFD operates independently of the routing protocol, allowing for consistent failure detection across different types of links.

Option A is incorrect because decreasing the BGP hello timer alone won't substantially improve failover time if the hold timer remains high. Moreover, aggressive hello timers can lead to false positives and increased network instability. Option B is incorrect because adding a VPN connection introduces complexity and doesn't directly address the BGP convergence issue within the Direct Connect setup itself. Also, a VPN could add significant latency. Option D is incorrect because decreasing the BGP hold-down timer directly affects BGP stability. While decreasing the hold timer can speed up failure detection, it is generally not recommended to set it too low (e.g., 5 seconds), as it can lead to premature route withdrawals due to transient network hiccups, causing flapping and instability. BFD is a more robust and standardized solution for fast failure detection.

Therefore, BFD is the most suitable option for achieving a faster failover time in an AWS Direct Connect environment.

Authoritative links:

AWS Direct Connect Resiliency: <https://aws.amazon.com/blogs/networking-and-content-delivery/resiliency-toolkit-for-aws-direct-connect/>

BFD Overview: https://en.wikipedia.org/wiki/Bidirectional_Forwarding_Detection

Question: 190

CertyIQ

A company is building an API-based application on AWS and is using a microservices architecture for the design. The company is using a multi-account AWS environment that includes a separate AWS account for each microservice development team. Each team hosts its microservice in its own VPC that contains Amazon EC2 instances behind a Network Load Balancer (NLB).

A network engineer needs to use Amazon API Gateway in a shared services account to create an HTTP API to expose these microservices to external applications. The network engineer must ensure that access to the microservices can occur only over a private network. Additionally, the company must be able to control which entities from its internal network can connect to the microservices. In the future, the company will create more microservices that the company must be able to integrate with the application.

What is the MOST secure solution that meets these requirements?

A. Create an Application Load Balancer (ALB) in a VPC in the shared services account. Configure the integration to the API Gateway API by using a VPC link. Associate the VPC link with the ALB. Create a VPC endpoint service in each microservice account. Create an AWS PrivateLink endpoint for those services in the shared services account. Add the elastic network interface IP addresses of the VPC endpoint as targets for the target group of the ALB.

B. Create an Application Load Balancer (ALB) in a VPC in the shared services account. Configure the integration to the API Gateway API by using a VPC link. Associate the VPC link with the ALB. Connect all the VPCs to each other by using a central transit gateway. Add the IP addresses of the NLB as IP-based targets in the ALB target group.

C. Configure the integration to the API Gateway API by using HTTP-based integration. Connect all the VPCs to each other by using a central transit gateway. Create a separate HTTP integration to each NLB for each microservice. Add the HTTP endpoint of the NLB as the endpoint URL in the HTTP integration.

D. Configure the integration to the API Gateway API by using VPC link integration. Connect all the VPCs to each other by using a central transit gateway. Create a separate VPC link to each NLB for each microservice. Add the HTTP endpoint of the NLB as the endpoint URL in the VPC link integration.

Answer: A

Explanation:

The most secure solution to expose microservices via API Gateway in a multi-account environment, ensuring private network access and controlled internal connectivity, is option A.

Here's why:

Private Network Access: Option A leverages AWS PrivateLink, which allows you to expose services privately using VPC endpoint services and VPC endpoints. This keeps all traffic within the AWS network, avoiding the public internet. API Gateway uses VPC Links to connect to the ALB privately.

Controlled Connectivity: PrivateLink enforces strict network-level access control. You can use security groups and network ACLs associated with the VPC endpoints to control which entities within your internal network can connect to the microservices.

Scalability and Flexibility: As new microservices are introduced, you only need to create new VPC endpoint services and endpoints for each. The ALB in the shared services account can then be configured to route traffic to these new microservices without reconfiguring the entire network.

Centralized Control: The shared services account provides a central point for managing API access and security policies.

Options B, C, and D introduce unnecessary complexities and security risks:

Option B: Connecting VPCs via Transit Gateway without PrivateLink exposes the microservices' NLBs directly, potentially increasing the attack surface and bypassing the granular control provided by PrivateLink. Also, it doesn't use the benefits of VPC endpoint service.

Option C: HTTP-based integration over Transit Gateway is less secure than PrivateLink because HTTP integrations inherently involve public endpoints or require complex routing configurations, increasing attack surface.

Option D: While using VPC Links, Option D integrates directly with NLBs after connecting VPCs with Transit Gateway. This skips the benefit of a centralized ALB, which can provide a unified interface, traffic management, and additional security features, along with the benefits that come with PrivateLink. Transit Gateway also adds unnecessary complexity compared to PrivateLink.

Supporting Concepts and Links:

AWS PrivateLink: <https://aws.amazon.com/privatelink/> - Explains how to expose services privately using VPC endpoints.

Amazon API Gateway VPC Links:

<https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-private-apis.html> - Details how API Gateway can connect to VPC resources privately.

Network Load Balancer: <https://docs.aws.amazon.com/elasticloadbalancing/latest/network/introduction.html>

Application Load Balancer:

<https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html>

AWS Transit Gateway: <https://aws.amazon.com/transit-gateway/>

Question: 191

A company's VPC has Amazon EC2 instances that are communicating with AWS services over the public internet. The company needs to change the connectivity so that the communication does not occur over the public internet.

The company deploys AWS PrivateLink endpoints in the VPC. After the deployment of the PrivateLink endpoints, the EC2 instances can no longer communicate at all with the required AWS services.

Which combination of steps should a network engineer take to restore communication with the AWS services? (Choose two.)

- A. In the VPC route table, add a route that has the PrivateLink endpoints as the destination.
- B. Ensure that the `enableDnsSupport` attribute is set to `True` for the VPC. Ensure that each VPC endpoint has DNS support enabled.
- C. Ensure that the VPC endpoint policy allows communication.
- D. Create an Amazon Route 53 public hosted zone for all services.
- E. Create an Amazon Route 53 private hosted zone that includes a custom name for each service.

Answer: BC

Explanation:

The correct answer is **BC**. Here's a detailed justification:

B. Ensure that the `enableDnsSupport` attribute is set to `True` for the VPC. Ensure that each VPC endpoint has DNS support enabled.

Explanation: When using PrivateLink, the DNS hostname associated with an AWS service (e.g., `s3.amazonaws.com`) needs to resolve to the PrivateLink endpoint's private IP address within the VPC. This redirection is crucial to ensure traffic destined for the AWS service is routed through the PrivateLink endpoint instead of the public internet.

`enableDnsSupport` at the VPC level allows DNS resolution within the VPC. Without it, EC2 instances won't be able to resolve the service name at all. Similarly, each VPC endpoint must have DNS support enabled so that a private DNS hostname is associated with the endpoint. EC2 instances then use the service's DNS name, which resolves to the private IP of the endpoint, routing traffic over the private AWS network.

Why it's essential: If DNS support is disabled at either the VPC or VPC endpoint level, the standard AWS service DNS name will resolve to a public IP address (if available) or fail to resolve entirely, resulting in communication failure.

C. Ensure that the VPC endpoint policy allows communication.

Explanation: VPC endpoint policies are attached to PrivateLink endpoints and act as access control lists (ACLs). They define which principals (IAM users, roles, or AWS accounts) can access the AWS service through the endpoint and the actions they can perform.

Why it's essential: If the endpoint policy is too restrictive, it can block all traffic from EC2 instances, even if the DNS resolution is correctly routing traffic to the endpoint. The default endpoint policy allows full access to the service; however, a custom policy could be overly restrictive. Ensure the policy permits the necessary actions and resources for the EC2 instances to communicate with the intended AWS service via the endpoint. If the policy is blocking, communication will fail.

Why other options are incorrect:

A: Adding a route to the PrivateLink endpoint as the destination in the VPC route table is generally unnecessary and potentially incorrect. PrivateLink automatically manages the routing based on the DNS resolution of the service endpoint's hostname. Manually adding a route could conflict with this automatic

routing and lead to unexpected behavior.

D & E: While Route 53 is related to DNS, a public hosted zone is not the correct solution. PrivateLink aims for private communication, not public access. A private hosted zone (E) might be used for custom DNS names or for simpler management of DNS records in larger organizations. However, ensuring the default AWS service name is resolving to the endpoint is simpler and generally preferred, provided DNS support is enabled. Creating a private hosted zone that uses custom names isn't mandatory when using PrivateLink. DNS support through standard service names is sufficient in most situations.

Supporting Documentation:

AWS PrivateLink documentation: <https://docs.aws.amazon.com/vpc/latest/privatelink/vpc-endpoints.html>

VPC Endpoint Policies: <https://docs.aws.amazon.com/vpc/latest/privatelink/vpc-endpoint-policies.html>

Question: 192

CertyIQ

An international company wants to implement a multi-site hybrid infrastructure. The company wants to deploy its cloud computing resources on AWS in the us-east-1 Region and in the eu-west-2 Region, and in on-premises data centers in the United States (US) and in the United Kingdom (UK). The data centers are connected to each other by a private WAN connection. IP routing information is exchanged dynamically through BGP. The company wants to have two AWS Direct Connect connections, one each in the US and the UK.

The company expects to have 15 VPCs in each Region with CIDR blocks that do not overlap with each other or with CIDR blocks of the on-premises environment. The VPC CIDR blocks are planned so that the prefix aggregation can be performed both on a Regional level and across the entire AWS environment. The company will deploy a transit gateway in each Region to connect the VPCs. A network engineer plans to use a Direct Connect gateway in each Region. A transit VIF will attach the Direct Connect gateway in each Region to the transit gateway in that Region. The transit gateways will be peered with each other.

The network engineer wants to ensure that traffic follows the shortest geographical path from source to destination. Traffic between the on-premises data centers and AWS must travel across a local Direct Connect connection. Traffic between the US data center and eu-west-2 and traffic between the UK data center and us-east-1 must use the private WAN connection to reach the Direct Connect connection to the appropriate Region when the Direct Connect connection is available. The network must be resilient to failures in either the private WAN connection or with the Direct Connect connections. The network also must reroute traffic automatically in the event of any failure.

How should the network engineer configure the transit VIF associations on the Direct Connect gateways to meet these requirements?

- A. Advertise only the aggregate route for the company's entire AWS environment.
- B. Advertise VPC-specific CIDR prefixes from only the local Region. Additionally, advertise the aggregate route for the company's entire AWS environment.
- C. Advertise all the specific VPC CIDR blocks from both Regions.
- D. Advertise both Regional aggregate prefixes. Configure custom BGP communities on the routes advertised toward the data center.

Answer: B

Explanation:

The correct answer is **B. Advertise VPC-specific CIDR prefixes from only the local Region. Additionally, advertise the aggregate route for the company's entire AWS environment.**

Here's why this configuration satisfies the requirements:

Shortest Path Routing: Advertising VPC-specific CIDR prefixes only from the local region ensures that traffic destined for a VPC in that region prefers the local Direct Connect connection. The company's requirement is for traffic between on-premises and AWS to use the local Direct Connect. By announcing more specific routes

(VPC CIDRs) for each region through the local Direct Connect Gateway, on-prem networks will prefer this path over the WAN.

Resiliency and Failover: The aggregate route for the entire AWS environment is also advertised. This provides a fallback path. If the local Direct Connect connection fails, the on-premises data centers will learn the aggregate route through the private WAN connection and the peered transit gateways. The aggregate route acts as a less-preferred, but still functional, route to reach the AWS environment. BGP will automatically reroute traffic to the available path.

Regional Aggregation: The prompt mentions regional and full environment prefix aggregation has been carefully considered and is in place.

Avoiding Asymmetric Routing: If VPC-specific routes were advertised from both regions (Option C), on-premises networks might choose a non-optimal path. For instance, the US data center might send traffic to a VPC in eu-west-2 via the US Direct Connect connection, then across the peered transit gateways, instead of using the private WAN to the UK data center and then the UK Direct Connect.

Scalability: Advertising the aggregate route alongside specific routes maintains scalability. While specific routes provide optimal routing, the aggregate route prevents the BGP routing table from becoming excessively large.

Option A doesn't provide local preference. If only the aggregate route is advertised, the local and remote DX connections appear equally attractive, which doesn't fulfill the shortest path requirement.

Option D adds complexity without additional benefit. The problem states that the prefixes have been planned carefully. Therefore, the communities on the Direct Connect are unnecessary.

Supporting Concepts:

BGP Path Selection: BGP prefers more specific routes (longer prefix matches). This is why announcing VPC-specific CIDRs locally forces traffic to use the local Direct Connect connection.

Route Aggregation: Summarizing routes simplifies routing tables and reduces the amount of routing information exchanged.

Direct Connect Gateway: Allows you to connect to multiple VPCs in different AWS Regions over a single Direct Connect connection.

Transit Gateway: acts as a hub that interconnects VPCs and on-premises networks, simplifying network architecture.

Authoritative Links:

AWS Direct Connect FAQs: <https://aws.amazon.com/directconnect/faq/>

AWS Transit Gateway: <https://aws.amazon.com/transit-gateway/>

Question: 193

CertyIQ

A company's AWS infrastructure is spread across more than 50 accounts and across five AWS Regions. The company needs to manage its security posture with simplified administration and maintenance for all the AWS accounts. The company wants to use AWS Firewall Manager to manage the firewall rules and requirements.

The company creates an organization with all features enabled in AWS Organizations.

Which combination of steps should the company take next to meet the requirements? (Choose three.)

- A. Configure only the Firewall Manager administrator account to join the organization.
- B. Configure all the accounts to join the organization.
- C. Set an account as the Firewall Manager administrator account.

- D. Set an account as the Firewall Manager child account.
- E. Set up AWS Config for all the accounts and all the Regions where the company has resources.
- F. Set up AWS Config for only the organization's management account.

Answer: BCE

Explanation:

Here's a detailed justification for why options B, C, and E are the correct steps to configure AWS Firewall Manager in a multi-account, multi-region AWS Organizations setup:

B. Configure all the accounts to join the organization: AWS Firewall Manager operates within the AWS Organizations framework. To centrally manage firewalls across all accounts, each account must be a member of the organization. Firewall Manager leverages the organization structure to apply policies and configurations consistently. <https://docs.aws.amazon.com/firewall-manager/latest/developerguide/fms-howitworks.html>

C. Set an account as the Firewall Manager administrator account: Firewall Manager requires a designated administrator account. This account has the permissions to configure and manage firewall policies across the organization. Designating this account is crucial for centralized control. The administrator account is the central point for configuring and enforcing Firewall Manager policies, applying them across all member accounts. <https://docs.aws.amazon.com/firewall-manager/latest/developerguide/fms-getting-started.html>

E. Set up AWS Config for all the accounts and all the Regions where the company has resources: AWS Firewall Manager relies on AWS Config to assess the compliance of network configurations with the defined firewall policies. Config provides the historical view of resources, their configuration and relationships. It's necessary for ALL relevant accounts and REGIONS. If AWS Config isn't enabled in all accounts and Regions, Firewall Manager cannot accurately assess and enforce policies. <https://docs.aws.amazon.com/firewall-manager/latest/developerguide/fms-prerequisites.html>

Here's why the other options are incorrect:

A. Configure only the Firewall Manager administrator account to join the organization: This is incorrect because Firewall Manager needs all the accounts it will manage to be part of the Organization.

D. Set an account as the Firewall Manager child account: There is no concept of "Firewall Manager child account." The accounts are just member accounts in the organization.

F. Set up AWS Config for only the organization's management account: While the management account is important, AWS Config needs to be enabled in ALL member accounts and regions containing resources to be protected so Firewall Manager can assess the resources configuration in the account.

Question: 194

CertyIQ

A company is using an Amazon CloudFront distribution that is configured with an Application Load Balancer (ALB) as an origin. A network engineer needs to implement a solution that requires all inbound traffic to the ALB to come from CloudFront. The network engineer must implement the solution at the network layer rather than in the application.

Which solution will meet these requirements in the MOST operationally efficient way?

- A. Add an inbound rule to the ALB's security group to allow the AWS managed prefix list for CloudFront.
- B. Add an inbound rule to the network ACLs that are associated with the ALB's subnets. Use the AWS managed prefix list for CloudFront as the source in the rule.
- C. Configure CloudFront to add a custom HTTP header to the requests that CloudFront sends to the ALB.

D. Associate an AWS WAF web ACL with the ALB. Configure the AWS WAF rules to allow traffic from the CloudFront IP set. Automatically update the CloudFront IP set by using an AWS Lambda function.

Answer: A

Explanation:

A. Add an inbound rule to the ALB's security group to allow the AWS managed prefix list for CloudFront.

The requirement is to ensure only CloudFront can access the ALB, and it must be enforced at the network layer (i.e., before the application sees the traffic).

AWS provides managed prefix lists that contain IP ranges for CloudFront.

You can use the CloudFront prefix list directly in the security group inbound rule for the ALB.

This ensures only CloudFront IPs can send traffic to the ALB.

Simple, scalable, and automatically updated by AWS — no need to manually track IP changes.

Question: 195

CertyIQ

A company has AWS accounts in an organization in AWS Organizations. The company has implemented Amazon VPC IP Address Manager (IPAM) in its networking AWS account. The company is using AWS Resource Access Manager (AWS RAM) to share IPAM pools with other AWS accounts. The company has created a top-level pool with a CIDR block of 10.0.0.0/8. For each AWS account, the company has created an IPAM pool within the top-level pool.

A network engineer needs to implement a solution to ensure that users in each AWS account cannot create new VPCs. The solution also must prevent users from associating a CIDR block with existing VPCs unless the CIDR block is from the IPAM pool for that account.

Which solution will meet these requirements?

A. Create a new AWS Config rule to find all VPCs that are not configured to allocate their CIDR block from an IPAM pool. Invoke an AWS Lambda function to delete these VPCs.

B. Create a new SCP in Organizations. Add a condition that denies the CreateVpc and AssociateVpcCidrBlock Amazon EC2 actions if the Ipv4IpamPoolId context key value is not the ID of an IPAM pool.

C. Create an AWS Lambda function to check for and delete all VPCs that are not configured to allocate their CIDR block from an IPAM pool. Invoke the Lambda function at regular intervals.

D. Create an Amazon EventBridge rule to check for AWS CloudTrail events for the CreateVpc and AssociateVpcCidrBlock Amazon EC2 actions. Use the rule to invoke an AWS Lambda function to delete all VPCs that are not configured to allocate their CIDR block from an IPAM pool.

Answer: B

Explanation:

The correct answer is **B** because it leverages Service Control Policies (SCPs) within AWS Organizations to enforce centralized governance over AWS accounts. SCPs act as guardrails, restricting the actions that IAM users and roles within member accounts can perform.

Here's a breakdown of why option B is the best choice and why the others are less suitable:

Option B: SCP with condition: This is the most direct and effective solution. SCPs allow you to specify conditions that must be met for an action to be allowed. By denying the CreateVpc and AssociateVpcCidrBlock actions unless the Ipv4IpamPoolId context key is set to the ID of an allowed IPAM pool, you ensure that VPCs can only be created and CIDR blocks can only be associated if they're sourced from the correct IPAM pool.

This aligns perfectly with the requirement of restricting VPC creation and CIDR association to the account-specific IPAM pool.

SCPs are a centralized mechanism, ensuring consistent enforcement across all specified accounts. They prevent the action from happening in the first place, rather than reacting after the fact.

Option A: AWS Config rule and Lambda function: While Config is useful for auditing, it operates reactively. It detects violations after they have occurred. Deleting VPCs after they've been created is disruptive and not ideal. It would also cause a service disruption which is not wanted.

Config rules trigger after an action occurs, it does not prevent unwanted action which is needed.

Option C: Lambda function with scheduled invocation: This option suffers from the same reactive problem as Option A. A Lambda function would need to poll AWS and take action after the fact. This creates a window of opportunity for non-compliant VPCs to exist.

In addition to the reactive issue, this solution requires management and maintenance of the Lambda function and its schedule.

Option D: EventBridge rule and Lambda function: Similar to Options A and C, this approach is reactive. It depends on CloudTrail events and triggers a Lambda function after the VPC is created or the CIDR is associated. This method requires more configuration than SCP, introduces complexity, and does not prevent a short service disruption by removing non-compliant VPCs and CIDR blocks.

Additionally, CloudTrail events have a delay, so the function may run late.

In summary:

Option B provides proactive enforcement through SCPs, ensuring that only allowed IPAM pools are used for VPC CIDR allocation, fulfilling the requirements effectively.

Supporting Links:

[AWS Organizations Service Control Policies \(SCPs\)](#)

[Amazon VPC IP Address Manager \(IPAM\)](#)

[AWS Resource Access Manager \(RAM\)](#)

Question: 196

CertyIQ

A company has an application that runs on premises. The application needs to communicate with an application that runs in a VPC on AWS. The communication between the applications must be encrypted and must use private IP addresses. The communication cannot travel across the public internet.

The company has established a 1 Gbps AWS Direct Connect connection between the on-premises location and AWS.

Which solution will meet the connectivity requirements with the LEAST operational overhead?

- A. Configure a private VIF on the Direct Connect connection. Associate the private VIF with the VPC's virtual private gateway. Set up an AWS Site-to-Site VPN private IP VPN connection to the virtual private gateway.
- B. Create a transit gateway. Configure a transit VIF on the Direct Connect connection. Associate the transit VIF with a Direct Connect gateway. Associate the Direct Connect gateway with a new transit gateway. Set up an AWS Site-to-Site VPN private IP VPN connection to the transit gateway.
- C. Configure a public VIF on the Direct Connect connection. Associate the public VIF with a Direct Connect gateway. Associate the Direct Connect gateway with a new transit gateway. Set up an AWS Site-to-Site VPN private IP VPN connection to the transit gateway.
- D. Create a transit gateway. Configure a transit VIF on the Direct Connect connection. Associate the transit VIF

with a Direct Connect gateway. Associate the Direct Connect gateway with a new transit gateway. Set up a third-party firewall in a new VPC that is attached to the transit gateway. Set up a VPN connection to the third-party firewall.

Answer: A

Explanation:

The correct answer is A. Here's why:

The problem requires secure, private communication between an on-premises application and an AWS VPC application, utilizing the existing Direct Connect connection without using the public internet.

Option A offers the least operational overhead while meeting all requirements. A private VIF (Virtual Interface) on Direct Connect provides private connectivity directly to your VPC using the Direct Connect link. Associating this VIF with the VPC's virtual private gateway (VGW) enables traffic flow between the on-premises network and the VPC. By establishing a Site-to-Site VPN over the Direct Connect private VIF, the traffic is encrypted while still utilizing the private connectivity of Direct Connect and private IP addresses. The VPN encrypts traffic across the Direct Connect link. This method keeps the traffic private and avoids the public internet while leveraging existing Direct Connect investment.

Option B introduces unnecessary complexity and overhead with a transit gateway. Transit gateways are beneficial for interconnecting multiple VPCs and on-premises networks, but in this case with only a single VPC and on-premises network, a transit gateway adds complexity without significant benefit. It would also add to the setup and management burden.

Option C uses a public VIF, which defeats the requirement of avoiding the public internet. Public VIFs are used to access AWS public services and require internet connectivity.

Option D also involves a transit gateway (similar drawbacks to Option B) and a third-party firewall in a separate VPC. Introducing a third-party firewall significantly increases the complexity of the solution. It requires managing the firewall itself, maintaining its software, and configuring routing, significantly adding operational overhead.

Therefore, using a private VIF with a VPN tunnel established for encryption provides the most straightforward, secure, and least complex solution. The VPN ensures encrypted communication, Direct Connect's private VIF uses private IP addresses and prevents the use of the public internet, and it leverages the existing Direct Connect connection. This meets all problem requirements with minimal operational overhead.

Here are some links for more information:

AWS Direct Connect: <https://aws.amazon.com/directconnect/>

AWS Site-to-Site VPN: <https://aws.amazon.com/vpn/>

Virtual Private Gateway: https://docs.aws.amazon.com/vpn/latest/s2svpn/VPC_VPN.html

Direct Connect Virtual Interfaces:

<https://docs.aws.amazon.com/directconnect/latest/UserGuide/WorkingWithVirtualInterfaces.html>

Question: 197

CertyIQ

A company has established connectivity between its on-premises data center in Paris, France, and the AWS Cloud by using an AWS Direct Connect connection. The company uses a transit VIF that connects the Direct Connect connection with a transit gateway that is hosted in the Europe (Paris) Region. The company hosts workloads in private subnets in several VPCs that are attached to the transit gateway.

The company recently acquired another corporation that hosts workloads on premises in an office building in Tokyo, Japan. The company needs to migrate the workloads from the Tokyo office to AWS. These workloads must have access to the company's existing workloads in Paris. The company also must establish connectivity between

the Tokyo office building and the Paris data center.

In the Asia Pacific (Tokyo) Region, the company creates a new VPC with private subnets for migration of the workloads. The workload migration must be completed in 5 days. The workloads cannot be directly accessible from the internet.

Which set of steps should a network engineer take to meet these requirements?

- A.1. Create public subnets in the Tokyo VPC to migrate the workloads into.
- 2. Configure an internet gateway for the Tokyo office to reach the Tokyo VPC.
- 3. Configure security groups on the Tokyo workloads to only allow traffic from the Tokyo office and the Paris workloads.
- 4. Create peering connections between the Tokyo VPC and the Paris VPCs.
- 5. Configure a VPN connection between the Paris data center and the Tokyo office by using existing routers.
- B.1. Configure a transit gateway in the Asia Pacific (Tokyo) Region. Associate this transit gateway with the Tokyo VPC.
- 2. Create peering connections between the Tokyo transit gateway and the Paris transit gateway.
- 3. Set up a new Direct Connect connection from the Tokyo office to the Tokyo transit gateway.
- 4. Configure routing on both transit gateways to allow data to flow between sites and the VPCs.
- C.1. Configure a transit gateway in the Asia Pacific (Tokyo) Region. Associate this transit gateway with the Tokyo VPC.
- 2. Create peering connections between the Tokyo transit gateway and the Paris transit gateway.
- 3. Configure an AWS Site-to-Site VPN connection from the Tokyo office. Set the Tokyo transit gateway as the target.
- 4. Configure routing on both transit gateways to allow data to flow between sites and the VPCs.
- D.1. Configure an AWS Site-to-Site VPN connection from the Tokyo office to the Paris transit gateway.
- 2. Create an association between the Paris transit gateway and the Tokyo VPC.
- 3. Configure routing on the Paris transit gateway to allow data to flow between sites and the VPC.

Answer: C

Explanation:

The correct answer is C. Here's why:

The requirements state the need for connectivity between the Tokyo office, the Paris data center, and the workloads hosted in VPCs connected to the Paris transit gateway. The solution needs to be implemented quickly (within 5 days) and without exposing the workloads to the internet.

Option C meets these requirements effectively. First, a transit gateway is established in the Tokyo region and associated with the new Tokyo VPC. This creates a central hub for connectivity within the Tokyo region. Second, peering the Tokyo and Paris transit gateways allows traffic to flow between the two regions and connect the Tokyo workloads with the Paris workloads. Crucially, setting up a Site-to-Site VPN connection from the Tokyo office to the Tokyo transit gateway enables secure connectivity between the Tokyo office and the AWS environment in Tokyo without requiring a new Direct Connect, which would take more time. Finally, configuring routing on both transit gateways enables data flow between all sites and VPCs according to the company's network architecture.

Option A is incorrect because it proposes using public subnets and an internet gateway, which directly violates the requirement of not exposing the workloads to the internet. Furthermore, VPC peering does not transitively route between VPCs and the on-premises data centers.

Option B is incorrect because setting up a new Direct Connect connection takes significant time and would violate the 5-day migration deadline. While a Direct Connect provides high bandwidth, the problem prioritizes speed of deployment.

Option D is incorrect because associating the Paris transit gateway with the Tokyo VPC does not establish connectivity from the Tokyo office to the Paris data center. We need a VPN from the Tokyo office to AWS. Furthermore, a transit gateway cannot directly attach to a VPC in a different region, so you would still need a peer between a Tokyo transit gateway.

Therefore, option C is the most appropriate solution to satisfy the requirements within the given constraints.

Further Reading:

AWS Transit Gateway: <https://aws.amazon.com/transit-gateway/>

AWS Site-to-Site VPN: <https://aws.amazon.com/vpn/>

VPC Peering: <https://docs.aws.amazon.com/vpc/latest/peering/what-is-vpc-peering.html>

Question: 198

CertyIQ

Company A recently acquired Company B. Company A has a hybrid AWS and on-premises environment that uses a hosted AWS Direct Connect connection, a Direct Connect gateway, and a transit gateway. Company A has a transit VIF to access the resources in its production environment in the us-east-1 Region.

Company B has applications that run across multiple VPCs in the us-west-2 Region in a single AWS account. A transit gateway connects all Company B's application VPCs. The CIDR blocks for both companies do not overlap.

Company A needs to use the existing Direct Connect connection to access Company B's applications from the on-premises environment.

Which solution will meet these requirements?

- A. Create a new Direct Connect gateway in the Company B account. Associate the Company B transit gateway with the new Direct Connect gateway. Create a transit VIF on the existing hosted connection for Company B.
- B. Create an association proposal from the Company B account to associate the Company B transit gateway with the Company A Direct Connect gateway. Accept the transit gateway association proposal by logging into the Company A account.
- C. Create multiple virtual private gateways. Attach the virtual private gateways to each of Company B's application VPCs. Create a hosted private VIF for each virtual private gateway.
- D. Create a new Direct Connect gateway in the Company B account. Associate the Company B transit gateway with the new Direct Connect gateway. Create a hosted private VIF for Company B.

Answer: B

Explanation:

Option B is the correct answer because it provides the most efficient and scalable solution for connecting Company B's applications in us-west-2 to Company A's on-premises environment via the existing Direct Connect connection. The key here is leveraging the existing infrastructure and features like Direct Connect Gateways and Transit Gateways for cross-account connectivity.

Here's a detailed justification:

1. **Direct Connect Gateway (DXGW) for Centralized Connectivity:** A DXGW provides a global point for Direct Connect connections, enabling access to multiple AWS Regions. Since Company A already has a DXGW and Direct Connect connection, it makes sense to utilize it.
2. **Transit Gateway (TGW) for VPC Interconnectivity:** Company B already has a TGW connecting its application VPCs in us-west-2. TGWs are designed to simplify routing between multiple VPCs and on-premises networks.
3. **Cross-Account TGW Association:** AWS supports cross-account TGW associations. This allows one AWS account (Company B's) to propose an association between its TGW and a DXGW in another account (Company A's). This avoids creating new Direct Connect infrastructure.
4. **Association Proposal and Acceptance:** The association proposal mechanism is specifically designed for cross-account scenarios. Company B proposes, and Company A accepts, establishing the connection.

5. **No New Direct Connect Connection Needed:** By using the existing Direct Connect connection and Company A's DXGW, the solution avoids the cost and complexity of establishing a new Direct Connect connection.
6. **Scalability and Centralized Management:** This solution is scalable as Company B's application needs grow. Centralized routing policies can be managed through the existing Direct Connect Gateway and Transit Gateway setup.

Why other options are incorrect:

Option A: Creating a new DXGW and transit VIF would be redundant, as Company A already has a DXGW and transit VIF. It's less efficient and adds unnecessary cost and complexity.

Option C: Using multiple virtual private gateways (VGWs) is not the right solution as Direct Connect Gateways are the recommended way of connecting to multiple VPCs with private VIFs. Each VPC would require its own VGW and VIF, making the solution overly complex and difficult to manage.

Option D: This option involves creating a new Direct Connect gateway in Company B's account and a hosted private VIF. While this would technically work, it doesn't leverage Company A's existing Direct Connect setup, making it less efficient.

Authoritative Links:

AWS Direct Connect Gateways: <https://docs.aws.amazon.com/directconnect/latest/UserGuide/direct-connect-gateways-intro.html>

Transit Gateways: <https://docs.aws.amazon.com/vpc/latest/tgw/what-is-transit-gateway.html>

Cross-Account Transit Gateway Attachments: <https://docs.aws.amazon.com/vpc/latest/tgw/tgw-share.html>

Question: 199

CertyIQ

A company has developed a web service for language translation. The web service's application runs on a fleet of Amazon EC2 instances that are in an Auto Scaling group. The instances run behind an Application Load Balancer (ALB) and are deployed in a private subnet. The web service can process requests that contain hundreds of megabytes of data.

The company needs to give some customers the ability to access the web service. Each customer has its own AWS account. The company must make the web service accessible to approved customers without making the web service accessible to all customers.

Which combination of steps will meet these requirements with the LEAST operational overhead? (Choose two.)

- A. Create VPC peering connections with the approved customers only.
- B. Create an AWS PrivateLink endpoint service. Configure the endpoint service to require acceptance that will be granted to approved customers only.
- C. Configure an authentication action for the endpoint service's load balancer to allow customers to log in by using their AWS credentials. Provide only approved customers with the URL.
- D. Configure a Network Load Balancer (NLB) and a listener with the ALB as a target. Associate the NLB with the endpoint service.
- E. Associate the ALB with the endpoint service.

Answer: BD

Explanation:

Here's a detailed justification for the answer choices B and D, along with why the others are incorrect:

Why B is correct: Create an AWS PrivateLink endpoint service. Configure the endpoint service to require

acceptance that will be granted to approved customers only.

AWS PrivateLink provides private connectivity between VPCs, AWS services, and your on-premises networks, without exposing your traffic to the public internet. Creating an endpoint service allows the company to expose its web service, which is behind the ALB in a private subnet, to specific customers. By requiring acceptance, the company maintains control over who can access the service. Only approved customers will have their endpoint connection requests accepted, thus restricting access effectively. This ensures that the web service remains private and accessible only to authorized entities. <https://docs.aws.amazon.com/private-link/latest/vpce/endpoint-services.html>

Why D is correct: Configure a Network Load Balancer (NLB) and a listener with the ALB as a target. Associate the NLB with the endpoint service.

An NLB is crucial here for several reasons. First, PrivateLink endpoint services require an NLB behind them. Second, given the need to handle large requests containing hundreds of megabytes of data, the NLB's ability to handle TCP traffic efficiently is paramount. The NLB will forward traffic to the ALB which then routes to the EC2 instances. The NLB will serve as the front end for PrivateLink, while the ALB continues to handle HTTP/HTTPS traffic and application-level routing. This separation of concerns ensures that the large data transfers are handled efficiently.

<https://docs.aws.amazon.com/elasticloadbalancing/latest/network/introduction.html>

Why A is incorrect: Create VPC peering connections with the approved customers only.

While VPC peering would allow connectivity, it creates a more direct connection between the VPCs, increasing the risk of misconfiguration or unintended access. Furthermore, it requires managing a separate peering connection for each customer. This increases operational overhead compared to PrivateLink which abstracts away a lot of the underlying networking. VPC Peering can also involve overlapping CIDR blocks causing routing complexity.

Why C is incorrect: Configure an authentication action for the endpoint service's load balancer to allow customers to log in by using their AWS credentials. Provide only approved customers with the URL.

While authentication is crucial, PrivateLink by itself provides network-level security. Adding application-level authentication might add an extra layer but using AWS credentials directly through the load balancer is not the standard and secure way to implement cross-account authentication. Options like AWS IAM Identity Center (successor to AWS SSO) or other identity providers are typically used in conjunction with PrivateLink for finer-grained access control. This approach doesn't directly address the problem of providing access to the web service in a private and controlled manner using AWS's recommended best practices.

Why E is incorrect: Associate the ALB with the endpoint service.

An ALB cannot be directly associated with the endpoint service; the NLB is a prerequisite for PrivateLink. ALB handles HTTP/HTTPS traffic, and PrivateLink operates at the network level requiring an NLB as the front door. Therefore, associating the ALB directly is not a valid configuration for creating a PrivateLink endpoint service. The NLB acts as the proxy for the endpoint service exposing the services behind it.

Question: 200

CertyIQ

A company is migrating an application to the AWS Cloud. The company has successfully provisioned and tested connectivity between AWS Direct Connect and the company's on-premises data center. The application runs on Amazon EC2 instances across multiple Availability Zones. The instances are in an Auto Scaling group.

The application communicates through HTTPS to a third-party vendor's data service that is hosted at the company's data center. The data service implements a static ACL through explicit allow listing of client IP addresses.

A network engineer must design a network solution so that the migrated application can continue to access the vendor's data service as the application scales.

Which solution will meet these requirements with the LEAST amount of ongoing change to the vendor's allow list?

A. Configure a private NAT gateway in the subnets for each Availability Zone that the application runs in. Configure the application to target the NAT gateways instead of the data service directly. Update the data service's allow list to include the IP addresses of the NAT gateways.

B. Configure an elastic network interface in the subnets for each Availability Zone that the application runs in. Associate the elastic network interfaces with the Auto Scaling group for the application. Update the data service's allow list to include the IP addresses of the elastic network interfaces.

C. Configure an elastic network interface in the subnets for each Availability Zone that the application runs in. Launch an EC2 instance into each subnet. Attach the respective elastic network interfaces to the new EC2 instances. In the application subnet route tables, configure the new EC2 instances as the next destination for the data service. Update the data service's allow list to include the IP addresses of the elastic network interfaces.

D. Configure an Application Load Balancer (ALB) in the subnets for each Availability Zone that the application runs in. Configure an ALB-associated target group that contains a target that uses the IP address for the data service. Configure the application to target the ALB instead of the data service directly. Update the data service's allow list to include the IP addresses of the ALBs.

Answer: A

Explanation:

The best solution is A because it offers a centralized and consistent source IP address for outbound traffic to the vendor's data service, minimizing changes to the vendor's allow list.

Here's why:

NAT Gateway Functionality: NAT (Network Address Translation) gateways allow instances in private subnets to connect to the internet or other networks while hiding their private IP addresses. All traffic exiting the subnet towards the vendor's data service will appear to originate from the NAT gateway's elastic IP address.

Least Change to Vendor's Allow List: By using NAT gateways, you only need to add the NAT gateway's Elastic IP addresses to the vendor's allow list. This remains constant even as the application scales with the Auto Scaling group because the outbound traffic will consistently come from the NAT gateways.

Scalability and Availability: Deploying a NAT gateway in each Availability Zone ensures high availability. If a NAT gateway fails in one zone, traffic automatically fails over to the NAT gateway in another zone.

Avoids Instance-Specific Configuration: Options B and C require adding multiple individual ENI IP addresses to the allow list, which increases complexity and management overhead as the application scales. This approach violates the "least amount of ongoing change" requirement.

ALB Inapplicability: An Application Load Balancer (ALB) is designed for distributing incoming application traffic, not for controlling source IP addresses for outbound traffic. Option D's approach is not relevant in this scenario. ALBs don't inherently mask the IP addresses of the originating EC2 instances for outbound connections. While an ALB can forward traffic to an IP address, it doesn't provide the source IP consistency needed for allow lists unless additional (and more complex) configurations are implemented which would introduce higher complexity.

Security: This design ensures that EC2 instances don't require public IP addresses.

[AWS NAT Gateway Documentation](#)[AWS Auto Scaling Documentation](#)

Question: 201

CertyIQ

A company has a highly available application that is hosted in multiple VPCs and in two on-premises data centers. All the VPCs reside in the same AWS Region. All the VPCs require access to each other and to the on-premises data centers for the transfer of files that are multiple gigabytes in size.

A network engineer is designing an AWS Direct Connect solution to connect the on-premises data centers to each VPC.

Which architecture will meet the company's requirements with the LEAST operational overhead?

A. Configure a virtual private gateway and a private VIF in each VPC in the Region. Configure a Direct Connect gateway. Associate the VIF of every VPC with the Direct Connect gateway. Create a new private VIF that connects the Direct Connect gateway to each on-premises data center. Configure the new private VIF to exchange BGP routes with the on-premises data centers and to have an MTU of 9001. Use VPC peering between each VPC. Configure static routing in each VPC to provide inter-VPC routing.

B. Configure a virtual private gateway and a private VIF in each VPC in the Region. Configure a Direct Connect gateway. Associate the VIF of every VPC with the Direct Connect gateway. Create a new private VIF that connects the Direct Connect gateway to each on-premises data center. Configure the new private VIF to exchange BGP routes with the on-premises data centers and to have an MTU of 8500. Use VPC peering between each VPC. Configure static routing in each VPC to provide inter-VPC routing.

C. Configure a transit gateway in the same Region of each VPC. Attach each VPC to the transit gateway. Configure a Direct Connect gateway. Associate the Direct Connect gateway with the transit gateway. Associate a new transit VIF with each Direct Connect connection. Configure the new transit VIF to exchange BGP routes and to have an MTU of 9001. Configure route propagation between each VPC and the transit gateway.

D. Configure a transit gateway in the same Region of each VPC. Attach each VPC to the transit gateway. Configure a Direct Connect gateway. Associate the Direct Connect gateway with the transit gateway. Associate a new transit VIF with each Direct Connect connection. Configure the new transit VIF to exchange BGP routes and to have an MTU of 8500. Configure route propagation between each VPC and the transit gateway.

Answer: D

Explanation:

The correct answer is **D**. Here's why:

Transit Gateway (TGW): TGW provides a hub-and-spoke architecture for connecting multiple VPCs, simplifying network management compared to managing numerous VPC peering connections. This aligns with the "LEAST operational overhead" requirement.

Direct Connect Gateway (DXGW): DXGW allows a single Direct Connect connection to access multiple VPCs in one or more AWS Regions, as well as on-premises locations. This is crucial for connecting the on-premises data centers to all VPCs.

Transit VIF: A Transit VIF is specifically designed for connecting a Direct Connect connection to a Transit Gateway. It supports the exchange of BGP routes, which is essential for dynamic routing between the on-premises network and the AWS environment.

MTU 8500: Although 9001 is jumbo frame size, DX connections support MTU 8500 in conjunction with transit gateway.

Route Propagation: Configuring route propagation between the VPCs and the Transit Gateway automatically updates the VPC route tables with routes learned from the TGW, and vice-versa, reducing the need for manual route configuration.

Option A, B: VPC peering requires $n(n-1)/2$ peering relationships for full connectivity which is unscalable and adds operational overhead. It doesn't solve the connectivity to on-premise data centers. Also MTU 9001 for private VIF is incorrect.

Option C: MTU 9001 is incorrect with transit VIF

Therefore, option D provides the most scalable and manageable solution for connecting multiple VPCs and on-premises data centers with the least operational overhead by using a Transit Gateway, Direct Connect Gateway, and Transit VIF with BGP route exchange.

Supporting Links:

AWS Transit Gateway: <https://aws.amazon.com/transit-gateway/>

AWS Direct Connect Gateway: <https://aws.amazon.com/directconnect/dx-gateway/>

Question: 202

A company has a data center in the us-west-1 Region with a 10 Gbps AWS Direct Connect dedicated connection to a Direct Connect gateway. There are two private VIFs from the same data center location in us-west-1 that are attached to the same Direct Connect gateway.

VIF 1 advertises 172.16.0.0/16 with an AS_PATH attribute value of 65000. VIF 2 advertises 172.16.1.0/24 with an AS_PATH attribute value of 65000 65000 65000.

How will AWS route traffic to the data center for traffic that has a destination address within the 172.16.1.0/24 network range?

- A. AWS will route all traffic by using VIF 1.
- B. AWS will route all traffic by using VIF 2.
- C. AWS will use both VIFs for routing by using a round-robin policy.
- D. AWS will use flow control to balance the traffic between the two VIFs.

Answer: B**Explanation:**

The correct answer is B: AWS will route all traffic by using VIF 2.

The question involves understanding how AWS Direct Connect chooses a path when multiple virtual interfaces (VIFs) advertise overlapping or more specific routes to the same Direct Connect gateway. Route selection in BGP, the protocol used by Direct Connect, involves several attributes, with AS_PATH length being a critical factor.

When multiple routes are available to the same destination prefix, BGP prefers the route with the shortest AS_PATH length. The AS_PATH attribute lists the autonomous systems (ASs) that a route advertisement has traversed. In this case, VIF 1 advertises 172.16.0.0/16 with an AS_PATH of 65000. VIF 2 advertises the more specific route 172.16.1.0/24 with an AS_PATH of 65000 65000 65000.

Since 172.16.1.0/24 falls within the 172.16.0.0/16 range, and the traffic's destination address is within 172.16.1.0/24, AWS will consider both routes. Due to BGP's best path selection algorithm, AWS will prioritize the route with the shorter AS_PATH length for the /24 network. Even though 172.16.1.0/24 is more specific, the AS_PATH length influences path selection. However, a more specific route is generally preferred if AS_PATH lengths were identical.

Because VIF 2's AS_PATH is three AS hops long (65000 65000 65000) versus VIF 1's AS_PATH length of one hop (65000) in the AS_PATH, AWS will choose VIF 1 for traffic destined for 172.16.0.0/16, but not for traffic destined for 172.16.1.0/24.

Since VIF 2 advertises a more specific route (172.16.1.0/24), AWS will choose VIF 2 to direct all traffic within the 172.16.1.0/24 range. This is despite VIF 2 having a longer AS_PATH. More specific prefixes always win as long as they are reachable and valid.

Therefore, the traffic destined for 172.16.1.0/24 will always use VIF 2, making option B the correct answer. The other options are incorrect because AWS uses BGP's best path selection, not round-robin or flow control, to determine the optimal route. The AS_PATH length and specificity of the routes are critical in determining the best path.

Here's the authoritative link for more information on AWS Direct Connect routing policies:

[AWS Direct Connect Routing Policies and Traffic Engineering](#)

Question: 203

A company is planning to host external websites on AWS. The websites will include multiple tiers such as web servers, application logic services, and databases. The company wants to use AWS Network Firewall, AWS WAF, and VPC security groups for network security.

The company must ensure that the Network Firewall firewalls are deployed appropriately within relevant VPCs. The company needs the ability to centrally manage policies that are deployed to Network Firewall and AWS WAF rules. The company also needs to allow application teams to manage their own security groups while ensuring that the security groups do not allow overly permissive access.

What is the MOST operationally efficient solution that meets these requirements?

- A. Define Network Firewall firewalls, AWS WAFV2 web ACLs, Network Firewall policies, and VPC security groups in code. Use AWS CloudFormation to deploy the objects and initial policies and rule groups. Use CloudFormation to update the AWS WAFv2 web ACLs, Network Firewall policies, and VPC security groups. Use Amazon GuardDuty to monitor for overly permissive rules.
- B. Define Network Firewall firewalls, AWS WAFV2 web ACLs, Network Firewall policies, and VPC security groups in code. Use the AWS Management Console or the AWS CLI to manage the AWS WAFv2 web ACLs, Network Firewall policies, and VPC security groups. Use Amazon GuardDuty to invoke an AWS Lambda function to evaluate the configured rules and remove any overly permissive rules.
- C. Deploy AWS WAFv2 IP sets and AWS WAFv2 web ACLs with AWS CloudFormation. Use AWS Firewall Manager to deploy Network Firewall firewalls and VPC security groups where required and to manage the AWS WAFv2 web ACLs, Network Firewall policies, and VPC security groups.
- D. Define Network Firewall firewalls, AWS WAFv2 web ACLs, Network Firewall policies, and VPC security groups in code. Use AWS CloudFormation to deploy the objects and initial policies and rule groups. Use AWS Firewall Manager to manage the AWS WAFv2 web ACLs, Network Firewall policies, and VPC security groups. Use Amazon GuardDuty to monitor for overly permissive rules.

Answer: D

Explanation:

The correct answer is **D**. Here's why:

The scenario requires centralized management of network security policies (Network Firewall and AWS WAF) and decentralized management of security groups with guardrails to prevent overly permissive access.

Option **D** provides a comprehensive approach:

1. **Infrastructure as Code (IaC) with CloudFormation:** Using CloudFormation to define Network Firewall, AWS WAF, policies, and security groups allows for repeatable, version-controlled deployments, crucial for operational efficiency and consistency. This is a best practice for managing infrastructure in AWS.
2. **AWS Firewall Manager:** This service is designed for centralized management of firewall rules across multiple AWS accounts and resources. It addresses the requirement for centrally managing Network Firewall policies and AWS WAF rules, enabling consistent security posture across the organization. Firewall Manager integrates with both Network Firewall and AWS WAF.
<https://aws.amazon.com/firewall-manager/>
3. **Amazon GuardDuty:** GuardDuty provides threat detection, but in this case it is used to check for overly permissive security group rules. This provides visibility and an additional level of confidence to ensure that decentralized managed security groups do not violate company policies.

Options A, B, and C are less optimal:

Option A: Directly updating AWS WAF, Network Firewall, and security groups with CloudFormation after initial deployment lacks centralized management and oversight compared to Firewall Manager.

Option B: Manual management of security policies via the console or CLI is operationally inefficient and prone to human error, conflicting with the requirement for centralized control and automation. Using Lambda to remove rules is overly complex and disruptive compared to prevention.

Option C: Deploying everything with CloudFormation is not centralized. Using AWS Firewall Manager for only some components is inconsistent. Security groups are not fully covered.

Therefore, Option D combines IaC for initial setup, Firewall Manager for centralized policy management, and GuardDuty to enforce compliance within a decentralized model, offering the most operationally efficient and secure solution.

Question: 204

CertyIQ

A company has deployed an application in which the front end of the application communicates with the backend instances through a Network Load Balancer (NLB) in the same VPC. The application is highly available across two Availability Zones. The company wants to limit the amount of traffic that travels across the Availability Zones. Traffic from the front end of the application must stay in the same Availability Zone unless there is no healthy target in that Availability Zone behind the NLB. If there is no healthy target in the same Availability Zone, traffic must be sent to the other Availability Zone.

Which solution will meet these requirements?

A. Create a private hosted zone with weighted routing for each Availability Zone. Point the primary record to the local Availability Zone NLB DNS record. Point the secondary record to the Regional NLB DNS record. Configure the front end of the application to perform DNS lookups on the local private hosted zone records.

B. Turn off cross-zone load balancing on the NLB. Configure the front end of the application to perform DNS lookups on the local Availability Zone NLB DNS record.

C. Create a private hosted zone. Create a failover record for each Availability Zone. For each failover record, point the primary record to the local Availability Zone NLB DNS record and point the secondary record to the Regional NLB DNS record. Configure the front end of the application to perform DNS lookups on the local private hosted zone records.

D. Enable sticky sessions (session affinity) so that the NLB can bind a user's session to targets in the same Availability Zone.

Answer: B

Explanation:

The correct answer is **B**. Here's why:

Cross-Zone Load Balancing: Network Load Balancers (NLBs) by default distribute traffic across all registered targets in all enabled Availability Zones (AZs). By disabling cross-zone load balancing, the NLB will only route traffic to targets within the same AZ as the client, reducing inter-AZ traffic costs and latency.

Availability Zone Affinity: The requirement is to keep traffic within an AZ if healthy targets exist. Disabling cross-zone load balancing achieves this perfectly. If there are no healthy targets in the client's AZ, the NLB will route traffic to targets in other AZs automatically, fulfilling the failover requirement.

DNS Resolution: Using the local Availability Zone NLB DNS record will help ensure that traffic is directed to the appropriate AZ.

Why other options are incorrect:

A. Weighted Routing: While weighted routing with Route 53 can influence traffic distribution, it doesn't guarantee that traffic will stay within the same AZ unless targets are unavailable. Weighting just provides a preference, not a hard rule, and it requires manual configuration and monitoring of health checks.

C. Failover Records: Failover routing in Route 53 is designed for active-passive failover scenarios. Using it in this context would result in the application in one AZ being entirely unavailable until the health check for the primary NLB fails. This doesn't meet the requirement of load balancing within an AZ.

D. Sticky Sessions: Sticky sessions (session affinity) only ensure that a specific user's traffic is routed to the same target (within the same AZ if cross-zone is disabled, or potentially in another AZ if cross-zone is enabled). This doesn't prevent new users from being routed to targets in different AZs initially, which violates the requirement of minimizing inter-AZ traffic.

Supporting Documentation:

Network Load Balancers:

<https://docs.aws.amazon.com/elasticloadbalancing/latest/network/network-load-balancers.html>

Cross-Zone Load Balancing:

<https://docs.aws.amazon.com/elasticloadbalancing/latest/network/network-load-balancers.html#cross-zone-load-balancing>

Question: 205

CertyIQ

A company needs to protect against potential botnet command and control traffic from any Amazon EC2 instances that is in in the company's AWS Environment.

Which solution will meet these requirements?

- A. Use AWS Shield Advanced. Activate Shield Advanced protections on the EC2 instances to filter and block botnet traffic.
- B. Use Amazon Route 53 Resolver DNS Firewall. Add a rule to a rule group to use the AWSManagedDomainsBotnetCommandandControl managed domain list with an action to block botnet traffic.
- C. Use AWS WAF Bot Control. Configure a managed rule group that uses an AWS managed rule set to block botnet traffic.
- D. Use AWS Systems Manager. Run a Systems Manager Automation runbook on the EC2 instances to configure the instances to block botnet traffic.

Answer: B

Explanation:

The correct answer is **B. Use Amazon Route 53 Resolver DNS Firewall. Add a rule to a rule group to use the AWSManagedDomainsBotnetCommandandControl managed domain list with an action to block botnet traffic.**

Here's why:

The key requirement is to protect against botnet command and control (C&C) traffic originating from EC2 instances. Botnets rely on infected machines (in this case, EC2 instances) to communicate with a central C&C server, often using DNS to resolve the C&C server's domain name. Amazon Route 53 Resolver DNS Firewall is specifically designed to block DNS requests to known malicious domains, including those used by botnets. It can prevent infected EC2 instances from resolving the IP addresses of the C&C servers, effectively stopping the communication. The AWSManagedDomainsBotnetCommandandControl managed domain list provides a continuously updated list of known botnet domains. By using a rule to block queries to domains on this list, the company can prevent its EC2 instances from communicating with those C&C servers.

Option A is incorrect because AWS Shield Advanced primarily focuses on protecting against DDoS attacks targeting AWS resources, not preventing outbound botnet communication from EC2 instances. It's designed to mitigate large-scale attacks directed at your infrastructure, not the outbound communication of compromised instances.

Option C is incorrect because AWS WAF Bot Control is designed to protect web applications from bot traffic,

not to prevent botnet communication from EC2 instances. It focuses on analyzing HTTP requests to a web application to identify and block malicious bots.

Option D is incorrect because while AWS Systems Manager can be used for configuration management and automation, it's not the most effective solution for blocking botnet traffic at the network level. Configuring each instance individually is a more complex and less scalable approach than using a DNS firewall, and Systems Manager does not natively provide frequently updated lists of known botnet command-and-control domains like DNS Firewall does.

In summary, Amazon Route 53 Resolver DNS Firewall is the most appropriate solution because it directly addresses the specific problem of blocking DNS resolution to known botnet C&C domains, preventing outbound botnet communication from compromised EC2 instances.

Further reading:

Amazon Route 53 Resolver DNS Firewall: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver-dns-firewall.html>

AWS Managed Domain Lists: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver-dns-firewall-managed-domain-lists.html>

Question: 206

CertyIQ

A company has two on-premises data centers. The first data center is in the us-east-1 Region. The Second data center is in the us-east-2 Region. Each data center connects to the closest AWS Direct Connect facility. The company uses Direct Connect connections, transit VIFs, and a single Direct Connect gateway to establish connectivity to VPCs in us-east-1 and us-east-2 from the company's data centers. The company also has private connectivity from a telecommunications provider that connects the first data center to the second data center.

Recently, there have been multiple connection disruptions to the private connectivity between the data centers. The company needs a solution to improve the reliability of the connection between the two data centers.

Which solution will meet these requirements?

- A. Create a new Direct Connect gateway. Enable the Direct Connect SiteLink feature on the transit VIF. Share the CIDR blocks from the first data center and the second data center with each other.
- B. Create a new public VIF to both Regions. Enable the Direct Connect SiteLink feature on the new public VIF.
- C. Enable the Direct Connect SiteLink feature on the existing Direct Connect connections.
- D. Enable the Direct Connect SiteLink feature on the existing transit VIFs that are attached to the existing Direct Connect gateway.

Answer: D

Explanation:

The company requires a more reliable connection between its two data centers, which are currently linked by an unreliable private connection. Direct Connect SiteLink is the ideal solution because it leverages AWS's global network to establish a direct path for traffic between Direct Connect locations.

Option D is correct because enabling SiteLink on the existing transit VIFs directly addresses the need for improved reliability between the two data centers. This leverages the existing Direct Connect infrastructure and avoids the creation of new connections, simplifying the architecture and minimizing costs. SiteLink allows traffic to traverse the AWS global network from one Direct Connect location to another, bypassing the unreliable private connection. The existing transit VIFs are already associated with the Direct Connect gateway, which manages the routing between the data centers and the VPCs.

Option A is incorrect because creating a new Direct Connect gateway and sharing CIDR blocks is unnecessary

complexity. The existing Direct Connect gateway is already capable of routing traffic between the data centers.

Option B is incorrect because public VIFs are used for accessing public AWS services, not for private connectivity between data centers. SiteLink only works with private and transit VIFs.

Option C is incorrect because SiteLink is enabled on VIFs, not directly on the Direct Connect connections themselves. The Direct Connect connections provide the physical connectivity, while the VIFs provide the logical connectivity.

Therefore, enabling Direct Connect SiteLink on the existing transit VIFs is the most efficient and appropriate solution to enhance the reliability of the connection between the company's two data centers.

Relevant documentation:

[AWS Direct Connect SiteLink](#)

[AWS Direct Connect FAQs](#)

Question: 207

CertyIQ

A network engineer is working on a large migration effort from an on-premises data center to an AWS Control Tower based multi-account environment. The environment has a transit gateway that is deployed to a central network services account. The central network services account has been shared with an organization in AWS Organizations through AWS Resource Access Manager (AWS RAM).

A shared services account also exists in the environment. The shared services account hosts workloads that need to be shared with the entire organization.

The network engineer needs to create a solution to automate the deployment of common network components across the environment. The solution must provision a VPC for application workloads to each new and existing member account. The VPCs must be connected to the transit gateway in the central network services account.

Which combination of steps will meet these requirements with the LEAST operational overhead? (Choose three.)

- A. Deploy an AWS Lambda function to the shared services account. Program the Lambda function to assume a role in the new and existing member accounts to provision the necessary network infrastructure.
- B. Update the existing accounts with an Account Factory Customization (AFC). Select the same AFC when provisioning new accounts.
- C. Create an AWS CloudFormation template that describes the infrastructure that needs to be created in each account. Upload the template as an AWS Service Catalog product to the shared services account.
- D. Deploy an Amazon EventBridge rule on a default event bus in the shared services account. Configure the EventBridge rule to react to AWS Control Tower CreateManagedAccount lifecycle events and to invoke the AWS Lambda function.
- E. Create an AWSControlTowerBlueprintAccess role in the shared services account.
- F. Create an AWSControlTowerBlueprintAccess role in each member account.

Answer: BCE

Explanation:

The correct combination of steps to meet the requirements with the least operational overhead is **B, C, and E**.

B. Update the existing accounts with an Account Factory Customization (AFC). Select the same AFC when provisioning new accounts.

AFC in AWS Control Tower provides a way to customize new accounts provisioned through Account Factory. By applying AFCs to existing accounts and specifying it for new ones, you ensure consistent network infrastructure deployment across all member accounts. This includes defining the VPC setup and its basic

configuration. This approach promotes standardization and reduces manual intervention for new account setup.

C. Create an AWS CloudFormation template that describes the infrastructure that needs to be created in each account. Upload the template as an AWS Service Catalog product to the shared services account.

AWS CloudFormation allows you to define your infrastructure as code, which promotes consistency and repeatability. AWS Service Catalog provides a centralized catalog of approved IT services (CloudFormation templates). By uploading your VPC creation template to Service Catalog, you can make it easily available for self-service provisioning in each account without needing to manually deploy CloudFormation stacks every time. This is very important for standardizing infrastructure across the organization.

E. Create an AWSControlTowerBlueprintAccess role in the shared services account.

Creating a role named AWSControlTowerBlueprintAccess in the shared services account enables that account (likely the Account Factory or other service account) to provision resources into member accounts on behalf of the Account Factory customization. This role provides the necessary permissions to create the networking resources (VPC and associated components) in the target member accounts during account creation or update using AFC. This ensures minimal permissions are granted, adhering to the principle of least privilege.

Why other options are incorrect:

A & D (Lambda and EventBridge): While EventBridge and Lambda can automate account provisioning, this approach is more complex than using Account Factory Customizations. Account Factory Customizations were specifically designed for customizing accounts created via Control Tower and offer tighter integration with the Control Tower framework, thus creating less operational overhead.

F. (AWSControlTowerBlueprintAccess role in each member account): Creating this role in each member account will not work because the Blueprint is being used to create resources into the new account via Account Factory. Therefore, the Account Factory role, which runs in the management or a designated account, must assume a role in the member account, not the other way around.

In Summary: The combination of AFCs, CloudFormation templates through Service Catalog, and the AWSControlTowerBlueprintAccess role provides a streamlined, automated, and easily maintainable solution for deploying standard VPC infrastructure across a multi-account AWS Control Tower environment with minimal operational overhead.

Relevant Links:

AWS Control Tower Account Factory Customizations:

<https://docs.aws.amazon.com/controltower/latest/userguide/account-factory-customizations.html>

AWS Service Catalog: <https://aws.amazon.com/servicecatalog/>

AWS CloudFormation: <https://aws.amazon.com/cloudformation/>

Question: 208

CertyIQ

An online retail company is running a web application in the us-west-2 Region and serves consumers in the United States. The company plans to expand across several countries in Europe and wants to provide low latency for all its users.

The application needs to identify the users' IP addresses and provide localized content based on the users' geographic location. The application uses HTTP GET and POST methods for its functionality. The company also needs to develop a failover mechanism that works for GET and POST methods and is based on health checks. The failover must occur in less than 1 minute for all clients.

Which solution will meet these requirements?

- A. Configure a Network Load Balancer (NLB) for the application in each environment in the new AWS Regions. Create an AWS Global Accelerator accelerator that has endpoint groups that point to the NLBs in each Region.
- B. Configure an Application Load Balancer (ALB) for the application in each environment in the new AWS Regions. Create an AWS Global Accelerator accelerator that has endpoint groups that point to the ALBs in each Region.
- C. Configure an Application Load Balancer (ALB) for the application in each environment in the new AWS Regions. Create Amazon Route 53 public hosted zones that have failover routing policies.
- D. Configure a Network Load Balancer (NLB) for the application in each environment in the new AWS Regions. Create an Amazon CloudFront distribution. Configure an origin group with origin failover options.

Answer: C

Explanation:

The correct answer is **C. Configure an Application Load Balancer (ALB) for the application in each environment in the new AWS Regions. Create Amazon Route 53 public hosted zones that have failover routing policies.**

Here's a detailed justification:

Low Latency: Route 53's latency-based routing directs users to the AWS region closest to them, minimizing latency and providing a better user experience across Europe.

Geographic Location: Route 53's geolocation routing policy enables the application to identify the users' geographic location based on their IP address and serve localized content.

HTTP GET and POST methods: ALBs support HTTP GET and POST methods, which the application requires for its functionality. NLBs operate at Layer 4 and do not inspect the HTTP payload.

Failover Mechanism: Route 53's failover routing policy allows the company to implement a health check-based failover mechanism. If a region becomes unhealthy, Route 53 automatically routes traffic to a healthy region. The failover time for Route 53 is typically under 1 minute, meeting the requirement.

Global Accelerator Considerations: While Global Accelerator provides static IPs and improves performance, it does not natively support geolocation routing in the same way as Route 53. It is more suitable for accelerating traffic to specific endpoints rather than directing users based on their location. While Global Accelerator works with ALB, Route 53 provides a direct geolocation solution.

CloudFront Considerations: CloudFront is a content delivery network (CDN) that caches content closer to users. While it can improve performance, it doesn't directly provide geolocation routing or IP address identification for localized content in the way that the question describes. CloudFront is more appropriate for static assets than dynamic content customization.

Here are some authoritative links for further research:

Amazon Route 53 Routing Policies: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/routing-policy.html>

Application Load Balancers:

<https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html>

AWS Global Accelerator: <https://aws.amazon.com/global-accelerator/>

Amazon CloudFront: <https://aws.amazon.com/cloudfront/>

Question: 209

CertyIQ

A company has VPCs across 50 AWS accounts and is using AWS Organizations. The company wants to implement web filtering. The requirements for how the traffic must be filtered are the same for all the VPCs. A network engineer plans to use AWS Network Firewall. The network engineer needs to implement a solution that minimizes the number of firewall policies and rule groups that are necessary for this web filtering.

Which combination of steps will meet these requirements? (Choose three.)

- A. Create a firewall policy or rule group in each account.
- B. Use SCPs to share the firewall policy or rule group.
- C. Create a firewall policy or rule group in the management account
- D. Use AWS Resource Access Manager (AWS RAM) to share the firewall policy or rule group.
- E. Enable sharing within Organizations.
- F. Create OUs to share the firewall policy or rule group.

Answer: CDE

Explanation:

The correct answer is CDE because it provides the most efficient way to manage AWS Network Firewall policies across multiple AWS accounts within an AWS Organization. Let's break down why:

C. Create a firewall policy or rule group in the management account: Creating the firewall policy and rule groups in the management account centralizes the configuration. This follows best practices for managing resources within an AWS Organization and avoids having to duplicate and manage the same configuration in each of the 50 accounts. All configurations regarding the firewall will reside in a single, centrally managed location.

D. Use AWS Resource Access Manager (AWS RAM) to share the firewall policy or rule group: AWS RAM enables you to share AWS resources, including Network Firewall policies and rule groups, with other AWS accounts or within your AWS Organization. This avoids needing to recreate the firewall policy and rule groups in each account and ensures consistency across all accounts. Sharing via RAM is more granular and direct than using SCPs, which are more for access control.

E. Enable sharing within Organizations: Before you can share resources within your organization using AWS RAM, sharing must be enabled for your organization. This is a prerequisite step that allows AWS RAM to function correctly across the organizational structure. This step ensures that RAM can manage resource sharing permissions within the AWS Organization structure.

Why other options are incorrect:

A. Create a firewall policy or rule group in each account: This is inefficient and creates a management nightmare. Having 50 separate policies to manage would be prone to errors and make updates difficult.

B. Use SCPs to share the firewall policy or rule group: SCPs (Service Control Policies) are primarily used for access control and setting guardrails at the organizational unit or account level. They don't directly share resource definitions like Firewall policies; they restrict what actions can be performed.

F. Create OUs to share the firewall policy or rule group: While OUs are useful for organizing accounts, they don't directly share firewall policies. AWS RAM handles the sharing, which can be scoped to specific accounts or OUs. RAM offers more flexibility and control over what resources are shared.

Supporting Documentation:

AWS Resource Access Manager (RAM): <https://aws.amazon.com/ram/>

AWS Network Firewall: <https://aws.amazon.com/network-firewall/>

AWS Organizations: <https://aws.amazon.com/organizations/>

In summary, centralizing the firewall policy in the management account and using AWS RAM to share it across the organization, after enabling sharing, is the most efficient and scalable way to meet the requirements.

A company has an internal web-based application that employees use. The company hosts the application over a VPN in the company's on-premises network. The application runs on a fleet of Amazon EC2 instances in a private subnet behind a Network Load Balancer (NLB) in the same subnet. The instances are in an Amazon EC2 Auto Scaling group.

During a recent security incident, SQL injection occurred on the application. A network engineer must implement a solution to prevent SQL injection attacks in the future.

Which combination of steps will meet these requirements? (Choose three.)

- A. Create an AWS WAF web ACL that includes rules to block SQL injection attacks.
- B. Create an Amazon CloudFront distribution. Specify the EC2 instances as the origin.
- C. Replace the NLB with an Application Load Balancer.
- D. Associate the AWS WAF web ACL with the NLB.
- E. Associate the AWS WAF web ACL with the Application Load Balancer.
- F. Associate the AWS WAF web ACL with the Amazon CloudFront distribution.

Answer: ACE

Explanation:

The correct answer is ACE because it addresses the SQL injection vulnerability by implementing AWS WAF and correctly positioning it within the architecture.

Here's the justification:

A. Create an AWS WAF web ACL that includes rules to block SQL injection attacks: AWS WAF (Web Application Firewall) is designed to protect web applications from common web exploits, including SQL injection. Creating a WAF web ACL and configuring it with rules specifically to detect and block SQL injection patterns is the fundamental step to mitigate this type of attack. AWS WAF uses rule sets to inspect incoming HTTP requests and block malicious requests based on predefined or custom rules.

C. Replace the NLB with an Application Load Balancer: Network Load Balancers (NLB) operate at the transport layer (Layer 4) of the OSI model and are designed for high throughput and low latency. They do not inspect the application layer (Layer 7) content, which is required to identify and filter SQL injection attempts. Application Load Balancers (ALB), on the other hand, operate at Layer 7 and can inspect the HTTP headers and body, allowing integration with AWS WAF for web application protection.

E. Associate the AWS WAF web ACL with the Application Load Balancer: AWS WAF can be directly associated with an ALB. This allows the WAF to inspect all traffic flowing through the ALB and filter out malicious requests based on the defined rules. By associating the WAF with the ALB, all requests to the EC2 instances are first inspected by the WAF, and any traffic identified as a SQL injection attempt is blocked before it reaches the application.

Why other options are incorrect:

B. Create an Amazon CloudFront distribution. Specify the EC2 instances as the origin. While CloudFront can improve performance and security, it is not necessary in this scenario. The application is internal and accessed over a VPN, so a CDN does not add significant value. Additionally, configuring it to directly point to EC2 instances bypasses the intended WAF solution.

D. Associate the AWS WAF web ACL with the NLB. AWS WAF can't be directly associated with NLBs. WAF is a Layer 7 firewall and NLBs are a Layer 4 load balancer.

F. Associate the AWS WAF web ACL with the Amazon CloudFront distribution. Since CloudFront is not required in this scenario, this option is not applicable.

Supporting links:

AWS WAF: <https://aws.amazon.com/waf/>

Application Load Balancer: <https://aws.amazon.com/elasticloadbalancing/application-load-balancer/>

Network Load Balancer: <https://aws.amazon.com/elasticloadbalancing/network-load-balancer/>

Question: 211

CertyIQ

A company is running business applications on AWS. The company uses 50 AWS accounts, thousands of VPCs, and 3 AWS Regions across the United States and Europe.

A network engineer needs to establish network connectivity between an on-premises data center and the Regions. The network engineer also must establish connectivity between the VPCs. On-premises: users and applications must be able to connect to applications that run in the VPCs.

The company has an existing AWS Direct Connect connection that the network engineer can use. The network engineer creates a transit gateway in each Region and configures the transit gateways as inter-Region peers.

Which solution will provide network connectivity from the on-premises data center to the Regions and will provide inter-VPC communications across the different Regions?

- A. Create a private VIF with a gateway type of virtual private gateway. Configure the private VIF to use a virtual private gateway that is associated with one of the VPCs.
- B. Create a private VIF to a new Direct Connect gateway. Associate the new Direct Connect gateway with a virtual private gateway in each VPC.
- C. Create transit VIF with a gateway association to a new Direct Connect gateway. Associate each transit gateway with the new Direct Connect gateway.
- D. Create an AWS Site-to-Site VPN connection that uses a public VIF for the Direct Connect connection. Attach the Site-to-Site VPN connection to the transit gateways.

Answer: C

Explanation:

The correct answer is **C. Create transit VIF with a gateway association to a new Direct Connect gateway. Associate each transit gateway with the new Direct Connect gateway.**

Here's why:

1. **Transit Gateway for Inter-VPC and Inter-Region Connectivity:** The transit gateway simplifies network connectivity between thousands of VPCs across multiple Regions. By peering transit gateways across Regions, you establish a full mesh network for inter-VPC communication.
2. **Direct Connect Gateway for On-Premises Connectivity:** The Direct Connect gateway provides a globally available connection point for on-premises networks to access multiple AWS Regions. It allows you to connect your on-premises data center to the AWS network via Direct Connect and then leverage the transit gateways in each Region.
3. **Transit VIF for Direct Connect and Transit Gateway Integration:** A transit VIF (Virtual Interface) is the correct type of VIF to use when connecting a Direct Connect gateway to transit gateways. It allows for the exchange of routes between the on-premises network and the AWS transit gateway network. This is essential for bidirectional communication.
4. **Why other options are incorrect:**

A: Private VIF to a virtual private gateway (VGW) only allows connectivity to the specific VPC the VGW is attached to, not multiple Regions or VPCs.

B: Associating a Direct Connect gateway directly with multiple virtual private gateways (VGWs) is not supported. The Direct Connect gateway is designed to work with transit gateways, not directly with

VGWs.

D: Site-to-Site VPN over a public VIF is not as efficient or secure as using a transit VIF with a Direct Connect gateway. Direct Connect provides dedicated, private connectivity, while a Site-to-Site VPN over a public VIF uses the public internet and incurs additional encryption overhead. Using the existing Direct Connect connection is more cost-effective and performant.

In summary, using a Direct Connect gateway connected to transit gateways via transit VIFs enables the required connectivity from on-premises to multiple Regions and facilitates inter-VPC communication across those Regions.

Authoritative Links:

AWS Direct Connect Gateway: <https://docs.aws.amazon.com/directconnect/latest/UserGuide/direct-connect-gateways-intro.html>

Transit Gateway: <https://docs.aws.amazon.com/vpc/latest/tgw/what-is-transit-gateway.html>

Direct Connect Virtual Interfaces: <https://docs.aws.amazon.com/directconnect/latest/UserGuide/create-vif.html>

Question: 212

CertyIQ

A company has two data centers that are interconnected with multiple redundant links from different suppliers. The company Uses IP addresses that are within the 172.16.0.0/16 CIDR block. The company is running iBGP between the two data centers by using a private Autonomous System Number (ASN) and IGP.

The company is moving toward a hybrid setup in which the company will initially use one VPC in the AWS Cloud. An AWS Direct Connect connection runs from the first data center to a Direct Connect gateway by using a private VIF. On the connection, the company advertises a summarized route for the 172.16.0.0/16 network. The company is planning to set up a second summarized route from the second data center to a different Direct Connect location.

The company needs to implement a solution to route traffic to and from AWS through the first Direct Connect connection. The solution must use the second Direct Connect connection for failover purposes only.

Which solution will meet these requirements?

- A. Prepend the private ASN on the BGP announcements to AWS from the second data center. Add a second VIF in the first Direct Connect connection. Advertise the same network without any prepends from the first data center. Implement the same setup for the BGP announcement from AWS to the two data centers.
- B. Tag the BGP announcements with the local preference BGP community tags. Set the tag to high preference for the first data center. Set the tag to low preference for the second data center. Configure the second data center's router to have a lower local preference for the direct AWS BGP advertisements than for the advertisement from the first data center.
- C. Configure the Direct Connect gateway to prefer routing through the Direct Connect connection with the first data center. Configure the second data center's router to have a lower local preference for the direct AWS BGP advertisements than for the advertisement from the first data center.
- D. Configure the focal AWS Region BGP community tag on the BGP route that is advertised from the first data center. Configure AS_PATH prepends on the BGP announcements from the second data center.

Answer: B

Explanation:

The correct answer is **B**. Here's why:

Understanding the Requirements:

The company wants to use the first Direct Connect connection as the primary path to AWS and the second Direct Connect connection only for failover. This means AWS should prefer the routes advertised from the first data center.

Why Option B is Correct:

BGP Local Preference: BGP local preference is an attribute that tells a BGP router which path to prefer for outbound traffic to a particular destination. A higher local preference value means the path is more preferred.

Implementation: By tagging BGP announcements from the first data center with a high local preference community tag and from the second data center with a low local preference community tag, you directly influence AWS's routing decision. The Direct Connect gateway will then prefer the routes advertised with the higher local preference, ensuring traffic uses the first Direct Connect connection as the primary path. Configure the second data center's router to have a lower local preference for the direct AWS BGP advertisements than for the advertisement from the first data center enforces that the data center also prefers to send traffic out the first Direct Connect connection.

Failover: In case the first Direct Connect connection fails, AWS will automatically switch to the next best path, which would be through the second Direct Connect connection due to its lower local preference.

Why Other Options are Incorrect:

A: Prepending the ASN on the second data center's announcements only influences inbound traffic to the second data center. It does not control outbound traffic from AWS to the on-premises network. Furthermore, adding a second VIF to the first Direct Connect Connection provides no preference to specific routes, therefore this will not meet the requirement.

C: The Direct Connect gateway does not have a feature to prefer routing through one Direct Connect connection over another. Also, while configuring the local preference on the second data center is helpful, it does not influence AWS's routing preference.

D: While AS_PATH prepending can influence inbound traffic to the second data center, it's generally not a reliable way to influence routing in AWS. AS_PATH prepending only makes a path appear longer and less preferred, but AWS may still choose the path based on other factors. Moreover, the focal AWS Region BGP community tag isn't directly relevant to this failover scenario.

Supporting Concepts:

BGP Path Attributes: BGP uses various attributes to determine the best path for routing traffic. Local preference is one of the most influential attributes.

Direct Connect Gateway: It allows you to connect to multiple VPCs or on-premises networks over a single Direct Connect connection.

BGP Communities: These are tags that can be attached to BGP routes to convey specific routing policies or instructions.

Authoritative Links:

[AWS Direct Connect FAQs](#)

[BGP Local Preference](#)

Question: 213

CertyIQ

A company is replatforming a legacy data processing solution to AWS. The company deploys the solution on Amazon EC2 Instances in private subnets that are in one VPC.

The solution uses Amazon S3 for object storage. Both the data that the solution processes and the data the solution produces are stored in Amazon S3. The solution uses Amazon DynamoDB to save its own state. The company collects flow logs for the VPC. The solution uses one NAT gateway to register its license through the internet. A software vendor provides a specific hostname so the solution can register its license.

The company notices that the AWS bill exceeds the projected budget for the solution. A network engineer uses

AWS Cost Explorer to investigate the bill. The network engineer notices that the USE2-NatGateway-Bytes(\$)
usage type is the root cause of the higher than expected bill.

What should the network engineer do to resolve the issue? (Choose two.)

- A. Set up Amazon VPC Traffic Mirroring. Analyze the traffic to identify the traffic that the NAT gateway processes.
- B. Examine the VPC flow logs to identify the traffic that traverses the NAT gateway.
- C. Set up an AWS Cost and Usage Report in the AWS Billing and Cost Management console. Examine the report to find more details about the NAT gateway charges.
- D. Verify that the security groups attached to the EC2 instances allow outgoing traffic only to the IP addresses that the hostname resolves to, the VPC CIDR block, and the AWS IP address ranges for Amazon S3 and DynamoDB.
- E. Verify that the gateway VPC endpoints for Amazon S3 and DynamoDB are both set up and associated with the route tables of the private subnets.

Answer: BE

Explanation:

Here's a detailed justification for why options B and E are the correct choices and why the others are not, focusing on cost reduction for NAT Gateway usage:

B. Examine the VPC flow logs to identify the traffic that traverses the NAT gateway.

Justification: VPC Flow Logs capture information about the IP traffic going to and from network interfaces in your VPC. By examining these logs, the network engineer can pinpoint the source, destination, protocol, and volume of traffic passing through the NAT gateway. This is crucial for identifying which EC2 instances or applications are generating the most traffic to the internet, which directly impacts the NAT gateway costs. Knowing the specific traffic patterns allows for targeted optimization strategies.

Why it helps: Flow Logs provide visibility into the specific sources (EC2 instances) and destinations (Internet) of traffic traversing the NAT gateway. This allows for targeted optimization, such as reducing unnecessary traffic to external services.

Authoritative Link: <https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs.html>

E. Verify that the gateway VPC endpoints for Amazon S3 and DynamoDB are both set up and associated with the route tables of the private subnets.

Justification: Gateway VPC Endpoints provide private connectivity to Amazon S3 and DynamoDB without requiring traffic to traverse the internet (and therefore, the NAT gateway). If the EC2 instances are accessing S3 and DynamoDB through the NAT gateway, it significantly increases the NAT gateway's data processing volume and associated costs. Ensuring that Gateway VPC Endpoints are configured and correctly routed in the private subnets forces traffic to S3 and DynamoDB to stay within the AWS network, bypassing the NAT gateway entirely.

Why it helps: Eliminates S3 and DynamoDB traffic from going through the NAT gateway, significantly reducing data processing charges. These are likely significant contributors to the NAT gateway traffic due to the solution's data processing nature and dependency on these services.

Authoritative Link: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html>

Why other options are incorrect:

A. Set up Amazon VPC Traffic Mirroring. Analyze the traffic to identify the traffic that the NAT gateway processes. VPC Traffic Mirroring duplicates network traffic for analysis. While useful for security or troubleshooting, it doesn't reduce NAT gateway costs. It only provides another mechanism (besides Flow Logs) to view traffic. Moreover, enabling traffic mirroring itself can incur additional costs.

C. Set up an AWS Cost and Usage Report in the AWS Billing and Cost Management console. Examine the report to find more details about the NAT gateway charges. The Cost and Usage Report provides a granular

breakdown of AWS costs. While helpful for identifying that the NAT gateway is expensive, it doesn't directly help identify why or suggest how to reduce those costs. The engineer already knows the NAT gateway is the problem from Cost Explorer. The next step is to figure out the cause of the high usage.

D. Verify that the security groups attached to the EC2 instances allow outgoing traffic only to the IP addresses that the hostname resolves to, the VPC CIDR block, and the AWS IP address ranges for Amazon S3 and DynamoDB. Limiting outbound traffic with security groups is a good security practice. However, it primarily addresses security concerns, not necessarily NAT gateway costs. While reducing unnecessary connections can help, it's unlikely to be as impactful as routing S3 and DynamoDB traffic directly through VPC Endpoints or identifying the heaviest traffic producers via Flow Logs. Also, IP addresses associated with a hostname can change, making this a maintenance headache.

Question: 214

CertyIQ

A company ran out of IP address space in one of the Availability Zones in an AWS Region that the company uses. The Availability Zone that is out of space is assigned the 10.10.1.0/24 CIDR block. The company manages its networking configurations in an AWS CloudFormation stack. The company's VPC is assigned the 10.10.0.0/16 CIDR block and has available capacity in the 10.10.1.0/22 CIDR block.

How should a network specialist add more IP address space in the existing VPC with the LEAST operational overhead?

- A. Update the AWS::EC2::Subnet resource for the Availability Zone in the CloudFormation stack. Change the CidrBlock property to 10.10.1.0/22.
- B. Update the AWS::EC2::VPC resource in the CloudFormation stack. Change the CidrBlock property to 10.10.1.0/22.
- C. Copy the CloudFormation stack. Set the AWS::EC2::VPC resource CidrBlock property to 10.10.0.0/16. Set the AWS::EC2::Subnet resource CidrBlock property to 10.10.1.0/22 for the Availability Zone.
- D. Create a new AWS::EC2::Subnet resource for the Availability Zone in the CloudFormation stack. Set the CidrBlock property to 10.10.2.0/24.

Answer: A

Explanation:

The correct answer is **A: Update the AWS::EC2::Subnet resource for the Availability Zone in the CloudFormation stack. Change the CidrBlock property to 10.10.1.0/22.**

Here's why:

The problem is a shortage of IP addresses specifically within the 10.10.1.0/24 subnet in one Availability Zone. The VPC itself (10.10.0.0/16) has available space in the 10.10.1.0/22 block, which encompasses the problematic 10.10.1.0/24. The goal is to expand the IP address range in that Availability Zone with the least operational overhead.

Option A directly addresses the issue by modifying the existing subnet's CIDR block. By expanding it from /24 to /22, you effectively add more IP addresses to that specific subnet within the Availability Zone. This requires updating the CloudFormation stack, but only for the subnet in question, minimizing disruption.

Option B is incorrect because you cannot change the VPC's CIDR block to 10.10.1.0/22. The VPC's CIDR block is the overarching address space, and it's already 10.10.0.0/16. Changing it to a smaller range would be impossible and illogical.

Option C involves copying the entire CloudFormation stack and redefining the VPC's CIDR block. This is a massive overkill and goes against the principle of least operational overhead. It would require redeploying the entire infrastructure managed by the stack.

Option D involves creating a new subnet with 10.10.2.0/24. While this would add IP addresses to the VPC, it doesn't directly solve the immediate problem of the existing subnet in that Availability Zone being out of space. Applications or resources already configured to use IPs in the 10.10.1.0/24 range would still face issues. Furthermore, it would require adjusting routing and potentially security groups to use the new subnet. Expanding the existing subnet is a more direct and less disruptive solution. Note that you would need to ensure that no existing resources have allocated the IP address in the new range, and consider moving resources from the original /24 CIDR to the new /22 CIDR to facilitate complete access to the expanded range.

Therefore, expanding the existing subnet's CIDR block within the CloudFormation template offers the most efficient and targeted solution to the IP address shortage in the specific Availability Zone, requiring only a single resource update, fulfilling the "least operational overhead" requirement.

Relevant documentation:

AWS CloudFormation AWS::EC2::Subnet:

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/aws-resource-ec2-subnet.html>

AWS VPC Subnets: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Subnets.html

Question: 215

CertyIQ

A company's network engineer must implement a cloud-based networking environment for a network operations team to centrally manage. Other Teams will use the environment. Each team must be able to deploy infrastructure to the environment and must be able to manage its own resources. The environment must feature IPv4 and IPv6 support and must provide internet connectivity in a dual-stack configuration.

The company has an organization in AWS Organizations that contains a workload account for the teams. The network engineer creates a new networking account in the organization.

Which combination of steps should the network engineer take next to meet the requirements? (Choose three.)

- A. Create a new VPC. Associate an IPv4 CIDR block of 10.0.0.0/16 and specify an IPv6 block of 2001:db8:c5a:6000::/56. Provision subnets by assigning /24 IPv4 CIDR blocks and /64 IPv6 CIDR blocks.
- B. Create a new VPC. Associate an IPv4 CIDR block of 10.0.0.0/16 and use an Amazon-provided IPV6 CIDR block. Provision subnets by assigning /24 IPv4 CIDR blocks and /64 IPV6 CIDR blocks.
- C. Enable sharing of resources within the organization by using AWS Resource Access Manager (AWS RAM). Create a resource share in the networking account, select the provisioned subnets, and share the provisioned subnets with the target workload account. Use the workload account to accept the resource share through AWS RAM.
- D. Enable sharing of resources within the organization by using AWS Resource Access Manager (AWS RAM). Create a resource share in the networking account, select the new VPC, and share the new VPC with the target workload account. Use the workload account to accept the resource share through AWS RAM.
- E. Create an internet gateway and an egress-only internal gateway. Deploy NAT gateways to the public subnets. Associate the internet gateway with the new VPC. Update the route tables. Associate the route tables with the relevant subnets.
- F. Create an internet gateway. Deploy NAT instances to public subnets. Update the route tables. Associate the route tables with the relevant subnets.

Answer: ACE

Explanation:

Here's a detailed justification for why options A, C, and E are the correct steps to meet the requirements, and why the other options are incorrect:

A. Create a new VPC. Associate an IPv4 CIDR block of 10.0.0.0/16 and specify an IPv6 block of 2001:db8:c5a:6000::/56. Provision subnets by assigning /24 IPv4 CIDR blocks and /64 IPv6 CIDR blocks.

Justification: The requirements specify both IPv4 and IPv6 support. Creating a VPC with both IPv4 and IPv6 CIDR blocks is fundamental to fulfilling this. Provisioning subnets with appropriate IPv4 (/24) and IPv6 (/64) blocks enables resources within the subnets to use both protocols. Specifying an IPv6 block allows for more control and predictability over the IPv6 addressing scheme. This is the foundational step to creating a dual-stack (IPv4 and IPv6) network.

C. Enable sharing of resources within the organization by using AWS Resource Access Manager (AWS RAM). Create a resource share in the networking account, select the provisioned subnets, and share the provisioned subnets with the target workload account. Use the workload account to accept the resource share through AWS RAM.

Justification: The scenario emphasizes centralized management by the network operations team while allowing other teams to deploy infrastructure and manage resources in their own accounts. AWS RAM enables sharing of resources like subnets across accounts within an AWS Organization. Sharing the provisioned subnets allows the workload accounts to launch resources (EC2 instances, databases, etc.) directly into the centrally managed network, ensuring adherence to network policies and configurations without requiring the workload accounts to manage the underlying network infrastructure. This enforces the desired separation of concerns and promotes a consistent network environment.

E. Create an internet gateway and an egress-only internet gateway. Deploy NAT gateways to the public subnets. Associate the internet gateway with the new VPC. Update the route tables. Associate the route tables with the relevant subnets.

Justification: Internet connectivity is a key requirement. An internet gateway (IGW) enables instances in the public subnets to communicate with the internet. Deploying NAT Gateways in the public subnets allows instances in the private subnets to initiate outbound connections to the internet, such as for software updates, without being directly exposed to inbound internet traffic. An egress-only internet gateway is specifically designed for IPv6 traffic to enable outbound-only internet access for instances using IPv6 addresses. Properly configured route tables are critical for directing traffic appropriately within the VPC and to the internet. This provides secure and controlled internet access to the deployed environment.

Why other options are incorrect:

B: While creating a VPC with an IPv4 CIDR block is necessary, using an "Amazon-provided IPv6 CIDR block" is too generic. The question specified IPv4 and IPv6 support in a dual-stack configuration, and associating a custom IPv6 CIDR block as described in Option A is a better practice that gives you more control.

D: Sharing the entire VPC with other accounts is generally not recommended. Sharing individual resources like subnets using AWS RAM (as in Option C) provides more granular control and avoids potential security issues and conflicts that can arise from sharing the whole VPC. Sharing a VPC can make it difficult to isolate traffic and manage network policies effectively.

F: Using NAT instances instead of NAT Gateways is not ideal. NAT Gateways are managed services providing higher availability, scalability, and security, and are therefore preferred over NAT instances, which require manual management and configuration.

Authoritative Links:

AWS VPC: <https://aws.amazon.com/vpc/>

AWS RAM: <https://aws.amazon.com/ram/>

NAT Gateway: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html>

Egress Only Internet Gateway: <https://docs.aws.amazon.com/vpc/latest/userguide/egress-only-internet-gateway.html>

A company is using third-party firewall appliances to monitor and inspect traffic on premises. The company wants to use the same model on AWS. The Company has a single VPC with an internet gateway. The VPC has a fleet of web servers that run on Amazon EC2 instances that are managed by an Auto Scaling group.

The company's network team needs to work with the security team to establish inline inspection of all packets that are sent to and from the web servers. The solution must scale as the fleet of virtual firewall appliances scales

Which combination of steps should the network team take to implement this solution? (Choose three.)

- A. Create a new VPC, and deploy a fleet of firewall appliances. Create a Gateway Load Balancer. Add the firewall appliances as targets.
- B. Create a security group for use with the firewall appliances, and allow port 443. Allow a port for the Gateway Load Balancer to perform health checks.
- C. Create a security group for use with the firewall appliances, and allow port 6081. Allow a port for the Gateway Load Balancer to perform health checks.
- D. Deploy a fleet of firewall appliances to the existing VPC. Create a Gateway Load Balancer. Add the firewall appliances as targets.
- E. Update the internet gateway route table and the web server route table to send traffic to and from the internet to the VPC endpoint ID of the Gateway Load Balancer. Update the subnet route table that is associated with the Gateway Load Balancer endpoint to direct internet traffic to the internet gateway.
- F. Create a new route table inside the web server VPC. Create a new edge association with the internet gateway. Update the internet gateway route table and the web server route table to send traffic to and from the internet to the VPC endpoint ID of the Gateway Load Balancer. Update the subnet route table that is associated with the Gateway Load Balancer endpoint to direct internet traffic to the internet gateway.

Answer: DEF

Explanation:

The correct answer is DEF because it outlines the necessary steps to implement inline traffic inspection using third-party firewall appliances with a Gateway Load Balancer (GWLB) in AWS.

Here's a breakdown:

D. Deploy a fleet of firewall appliances to the existing VPC. Create a Gateway Load Balancer. Add the firewall appliances as targets. This is the fundamental step. The company wants to use its existing VPC and integrate firewall appliances into the current setup. The Gateway Load Balancer is specifically designed for virtual appliances like firewalls. It distributes traffic across the appliance fleet for scalability and availability.

E. Update the internet gateway route table and the web server route table to send traffic to and from the internet to the VPC endpoint ID of the Gateway Load Balancer. Update the subnet route table that is associated with the Gateway Load Balancer endpoint to direct internet traffic to the internet gateway. This step reroutes traffic to the GWLB. The internet gateway route table needs to send internet-bound traffic to the GWLB endpoint. The web server route table sends traffic destined for the internet via the GWLB. Crucially, the subnet where the GWLB endpoint resides must then have a route back to the internet gateway, forming the inspection path.

F. Create a new route table inside the web server VPC. Create a new edge association with the internet gateway. Update the internet gateway route table and the web server route table to send traffic to and from the internet to the VPC endpoint ID of the Gateway Load Balancer. Update the subnet route table that is associated with the Gateway Load Balancer endpoint to direct internet traffic to the internet gateway. This step creates a new route table inside the web server VPC and ensures that traffic originating from and destined for the web servers is routed through the Gateway Load Balancer. The internet gateway route table needs to send internet-bound traffic to the GWLB endpoint. The web server route table sends traffic destined for the internet via the GWLB. Crucially, the subnet where the GWLB endpoint resides must then have a route back to the internet gateway, forming the inspection path.

Option A is incorrect because it suggests creating a new VPC. The question states that the company wants to

use its existing VPC. Options B and C address security group configurations. While essential for securing the firewall appliances, they are not related to network routing aspects described in D, E, and F. It is the function of the third-party firewall to determine on a security group policy.

Supporting documentation:

AWS Gateway Load Balancer: <https://aws.amazon.com/elasticloadbalancing/gateway-load-balancer/>

Blog post on inspection: <https://aws.amazon.com/blogs/networking-and-content-delivery/inline-inspection-and-security-for-amazon-vpc-traffic-using-aws-gateway-load-balancer/>

Question: 217

CertyIQ

A financial company offers investment forecasts and recommendations to authorized users through the internet. All the services are hosted in the AWS Cloud. A new compliance requirement states that all the internet service traffic from any host must be logged and retained for 2 years. In its development AWS accounts, the company has designed, tested, and verified a solution that uses Amazon VPC Traffic Mirroring with a Network Load Balancer (NLB) as the traffic mirror target. While the solution runs in one AWS account, the solution mirrors the traffic to another AWS account.

A network engineer notices that not all traffic is mirrored when the solution is deployed into the production environment. The network engineer also notices that this behavior is random.

Which statements are possible explanations for why not all the traffic is mirrored? (Choose two.)

- A. The security groups are misconfigured on the production AWS account that hosts the company's services.
- B. The Amazon EC2 instance that is being monitored cannot handle the extra traffic that Traffic Mirroring has introduced.
- C. The IAM policy that allows the creation of traffic mirror sessions is misconfigured
- D. The mirrored traffic has a lower priority than the production traffic and is being dropped when network congestion occurs.
- E. The NLB is experiencing warm-up delay because of sudden and significant increases in traffic.

Answer: DE

Explanation:

Here's a detailed justification for why options D and E are the most likely explanations for the intermittent traffic mirroring issues in the production environment, along with supporting concepts and links:

D. The mirrored traffic has a lower priority than the production traffic and is being dropped when network congestion occurs.

VPC Traffic Mirroring duplicates network traffic, which inherently adds load to the network infrastructure. If the mirrored traffic isn't prioritized properly, it becomes susceptible to being dropped during periods of high network congestion. This is especially true in production environments, where traffic volumes are typically higher and more sensitive to latency. The underlying network devices (routers, switches, etc.) might implement Quality of Service (QoS) policies that prioritize regular production traffic over mirrored traffic. During congestion, the less important mirrored traffic can be dropped to maintain the performance of the main application. Traffic Mirroring doesn't inherently guarantee delivery of all mirrored traffic; it prioritizes the original traffic. The randomness stems from the fact that congestion isn't constant. <https://docs.aws.amazon.com/vpc/latest/mirroring/traffic-mirroring-considerations.html> (Refer to the "Performance impact" section, mentioning increased network load and potential impact on monitored instances.)

E. The NLB is experiencing warm-up delay because of sudden and significant increases in traffic.

Network Load Balancers (NLBs) require a warm-up period to adapt to sudden bursts of traffic. When an NLB is

first deployed or experiences a substantial increase in traffic, it might not immediately scale to handle the mirrored traffic efficiently. During this warm-up phase, the NLB's capacity to process and forward mirrored packets might be limited, causing some traffic to be dropped. This problem is more common with a production environment where the traffic rates fluctuate significantly compared to development/testing environment. After it warms up, the mirroring would stabilize.<https://docs.aws.amazon.com/elasticloadbalancing/latest/network/network-load-balancers.html#request-routing>(While not directly mentioning "warm-up delay," the NLB documentation highlights the importance of gradual scaling to optimize performance and avoid disruptions.)

Why other options are less likely:

A. The security groups are misconfigured on the production AWS account that hosts the company's services: While security group misconfigurations could cause issues, they would likely result in more consistent and predictable problems, such as complete connectivity failures or specific ports being blocked. The described behavior is random and intermittent.

B. The Amazon EC2 instance that is being monitored cannot handle the extra traffic that Traffic Mirroring has introduced: Although possible, this is less likely than D and E. It would be more consistent. The problem of the EC2 instance not being able to handle the extra traffic is typically tested in the development environment and fixed.

C. The IAM policy that allows the creation of traffic mirror sessions is misconfigured: An IAM policy misconfiguration would typically prevent the creation of traffic mirror sessions altogether or result in consistent authorization failures, not intermittent issues.

Question: 218

CertyIQ

A company has a VPC in the AWS Cloud. The company recently acquired a competitor that also has a VPC the AWS Cloud. A network engineer discovers an IP address overlap between the two VPCs. Both VPCs require access to an AWS Marketplace partner service.

Which solution will ensure interoperability among the VPC hosted services and the AWS Marketplace partner service?

A. Configure VPC peering with static routing between the VPCs. Configure an AWS Site-to-Site VPN connection with static routing to the partner service.

B. Configure a NAT gateway in the VPCs. Configure default routes in each VPC to point to the local NAT gateway. Attach each NAT gateway to a transit gateway. Configure an AWS Site-to-Site VPN connection with static routing to the partner service.

C. Configure AWS PrivateLink to facilitate connectivity between the VPCs and the partner service. Use the DNS name that is created with the associated interface endpoints to route traffic between the VPCs and the partner service.

D. Configure a NAT instance in the VPCs. Configure default routes in each VPC to point to the local NAT instance. Configure an interface endpoint in each VPC to connect to the partner service. Use the DNS name that is created with the associated interface endpoints to route traffic between the VPCs and the partner service.

Answer: C

Explanation:

The correct answer is C because it addresses both the IP address overlap and the need for connectivity to the AWS Marketplace partner service.

Here's why:

IP Address Overlap: VPC Peering (Option A) and NAT-based solutions (Options B and D) do not directly resolve the IP address overlap. VPC Peering requires unique, non-overlapping CIDR blocks. While NAT can hide internal IP addresses, it doesn't fundamentally solve the overlap for internal communication. AWS

PrivateLink (Option C) allows private connectivity to services without exposing traffic to the public internet and circumvents the need for direct IP address communication between the overlapping VPCs.

Connectivity to AWS Marketplace Partner Service: AWS PrivateLink provides a secure and scalable way to access services hosted in other VPCs (including those of AWS Marketplace partners) privately. Using interface endpoints (created by PrivateLink) in each VPC, the applications can connect to the partner service using its DNS name. The traffic remains within the AWS network, enhancing security. Options A and B suggest Site-to-Site VPN. While VPNs provide connectivity, using them for AWS Marketplace services is not optimal compared to PrivateLink, as they involve more configuration overhead and potential performance bottlenecks. Option D uses interface endpoints, but relies on NAT instances, adding complexity and potential scaling issues.

Why PrivateLink is Preferred: PrivateLink is designed for secure, private connectivity between VPCs and services, especially those on AWS Marketplace. It eliminates the need for public IP addresses, NAT gateways/instances, or internet gateways, which simplifies network management and improves security.

In summary, AWS PrivateLink effectively addresses the IP address overlap issue by abstracting the underlying IP addresses and provides a secure, private, and scalable solution for connecting to the AWS Marketplace partner service.

For further research:

AWS PrivateLink: <https://aws.amazon.com/privatelink/>

AWS VPC Peering: <https://docs.aws.amazon.com/vpc/latest/peering/what-is-vpc-peering.html>

AWS Site-to-Site VPN: <https://aws.amazon.com/vpn/>

AWS NAT Gateway: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html>

Question: 219

CertyIQ

A company uses the us-east-1 Region and the ap-south-1 Region for its business units (BUs). The BUs are named BU-1 and BU-Z. For each BU, there are two VPCs in us-east-1 and one VPC in ap-south-1.

Because of workload isolation requirements, resources can communicate within the same BU but cannot communicate with resources in the other BU. The company plans to add more BUs and plans to expand into more Regions.

Which solution will meet these requirements with the MOST operational efficiency?

A. Configure an AWS Cloud WAN network that operates in the required Regions. Attach all BU VPCs to the AWS Cloud WAN core network. Update the AWS Cloud WAN segment actions to configure new routes to deny traffic between the different BU segments.

B. Configure a transit gateway in each Region. Configure peering between the transit gateways. Attach the BU VPCs to the transit gateway in the corresponding Region. Configure the transit gateway and VPC route tables to isolate traffic between BU VPCs.

C. Configure an AWS Cloud WAN network that operates in the required Regions. Attach all BU VPCs to the AWS Cloud WAN core network. Update the core network policy by setting the isolate-attachments parameter for each segment.

D. Configure an AWS Cloud WAN network that operates in the required Regions. Create AWS Cloud WAN segments for each BU. Configure VPC attachments for each BU's VPCs to the corresponding BU segment.

Answer: D

Explanation:

The correct answer is **D**, configuring an AWS Cloud WAN network with segments for each BU. Here's a detailed justification:

Cloud WAN Segmentation: AWS Cloud WAN is designed for simplifying wide-area network management and connectivity across AWS Regions. A key feature is segmentation, which allows logically isolating network traffic between different business units or environments.

BU-Specific Segments: By creating a separate Cloud WAN segment for each BU (BU-1, BU-Z, etc.), you ensure that VPC attachments associated with that BU reside within its designated segment. This naturally isolates traffic within a BU.

VPC Attachments to Segments: Attaching each BU's VPCs to its respective segment is the core of the solution. VPC attachments act as the connection points between the VPC and the Cloud WAN core network, ensuring traffic is routed only within that segment.

Operational Efficiency: This approach is highly scalable and maintainable. As the company adds more BUs and expands into new Regions, you simply create new segments for each BU and attach the corresponding VPCs. The isolation is built into the network design, minimizing the need for complex routing configurations.

Centralized Management: Cloud WAN provides centralized control and visibility of the network, making it easier to manage and troubleshoot connectivity issues. <https://aws.amazon.com/cloud-wan/>

Avoids Complex Routing (Option B): Option B involves Transit Gateways and peering, which requires managing route tables across multiple Transit Gateways and VPCs. This approach is more complex and less scalable than Cloud WAN segmentation. Transit Gateway solution would require a route table per business unit to isolate traffic, further complicating routing.

'isolate-attachments' Parameter (Option C): The 'isolate-attachments' parameter within Cloud WAN (although not the primary driver for the choice) could potentially offer a further degree of isolation, but its functionality is typically used within the same segment. The more fundamental isolation is achieved by segmenting by BU in the first place.

Route Denial (Option A): Option A is less efficient because it requires actively denying traffic between segments. Segmentation, on the other hand, implicitly denies traffic between segments unless explicitly allowed through shared services or similar mechanisms. Actively managing denial rules can become complex and error-prone.

In summary, Cloud WAN with segments offers the most operationally efficient and scalable solution for isolating traffic between BUs while simplifying network management across multiple Regions.

Question: 220

CertyIQ

A company has many application VPCs that use AWS Site-to-Site VPN connections for connectivity to an on-premises location. The company's network team wants to gradually migrate to AWS Transit Gateway to provide VPC-to-VPC connectivity.

The network team sets up a transit gateway that uses equal-cost multi-path (ECMP) routing. The network team attaches two temporary VPCs to the transit gateway for testing. The test VPCs contain Amazon EC2 instances to confirm connectivity over the transit gateway between the on-premises location and the VPCs. The network team creates two new Site-to-Site VPN connections to the transit gateway.

During testing, the network team cannot reach the required bandwidth of 2.5 Gbps over the pair of new Site-to-Site VPN connections.

Which combination of steps should the network team take to improve bandwidth performance and minimize network congestion? (Choose three.)

- A. Enable acceleration for the existing Site-to-Site VPN connections to the transit gateway.
- B. Create new accelerated Site-to-Site VPN connections to the transit gateway.
- C. Advertise the on-premises prefix to AWS with the same BGP AS_PATH attribute across all the Site-to-Site VPN connections.
- D. Advertise the on-premises prefix to AWS with a different BGP AS_PATH attribute across all the Site-to-Site VPN connections.
- E. Verify that the transit gateway attachments are present in the Availability Zones of the test VPC.

F. Verify that the on-premises location is sending traffic by using multiple flows.

Answer: BDF

Explanation:

The correct answer is BDF. Here's why:

B. Create new accelerated Site-to-Site VPN connections to the transit gateway: Standard AWS Site-to-Site VPN connections are limited in bandwidth. To achieve higher bandwidth, particularly towards the 2.5 Gbps target, using Accelerated Site-to-Site VPN is crucial. Accelerated VPN uses AWS Global Accelerator under the hood, which utilizes the AWS global network backbone to improve VPN performance and reduce latency. Standard VPN connections do not use this and can't match its bandwidth capabilities.

D. Advertise the on-premises prefix to AWS with a different BGP AS_PATH attribute across all the Site-to-Site VPN connections: This is essential for influencing routing decisions. When the same prefix is advertised through multiple VPN connections with different AS_PATH attributes, AWS Transit Gateway sees multiple paths and can use the AS_PATH length as a factor for path selection. A shorter AS_PATH generally indicates a preferred path. Using different AS_PATHs, therefore, lets TGW use multiple VPN tunnels, effectively load-balancing traffic across them, thereby improving the total bandwidth. If same AS_PATH is used, TGW might pick only one path (one VPN connection), negating the benefit of having multiple VPNs. This leverages ECMP effectively.

F. Verify that the on-premises location is sending traffic by using multiple flows: ECMP distributes traffic based on flows, not simply packets. A "flow" is typically identified by a combination of source IP, destination IP, source port, destination port, and protocol. If the on-premises location sends traffic using only a single flow (e.g., a single SSH connection), the traffic will only utilize one of the VPN connections. To fully leverage ECMP and achieve higher bandwidth, it's necessary to generate multiple flows of traffic (e.g., by using multiple parallel connections or using tools that create multiple flows). This allows traffic to spread across the VPN connections.

Why other options are incorrect:

A. Enable acceleration for the existing Site-to-Site VPN connections to the transit gateway: You cannot simply "enable" acceleration on an existing standard VPN connection. Accelerated VPN connections must be created from scratch.

C. Advertise the on-premises prefix to AWS with the same BGP AS_PATH attribute across all the Site-to-Site VPN connections: Using the same AS_PATH would cause the TGW to select a single VPN tunnel, and ECMP would not be properly utilized.

E. Verify that the transit gateway attachments are present in the Availability Zones of the test VPC: While important for redundancy, AZ placement of the transit gateway attachments doesn't directly address the issue of insufficient bandwidth over the VPN connections. This ensures high availability, but not increased individual tunnel bandwidth.

Supporting Links:

[AWS Site-to-Site VPN: Accelerated Site-to-Site VPN](#)

[AWS Transit Gateway Routing](#)

[BGP AS_PATH Attribute](#)

Question: 221

CertyIQ

A company is migrating its on-premises network from its data center in Virginia to its data center in New York. The AWS Direct Connect connections for the Virginia and New York data center locations are both associated to the us-east-1 Region. The company needs to migrate a private VIF on an existing Direct Connect hosted connection

from Virginia to New York. The company's on-premises network uses the connection to access VPCs through a Direct Connect gateway in us-east-1.

The company has already requested a new Direct Connect hosted connection from the new data center to the New York Direct Connect location.

Which solution will meet these requirements with the LEAST downtime?

A. Create a new private VIF on the new Direct Connect hosted connection. Create a new Direct Connect gateway and attach the gateway to the new private VIF. Configure BGP routing on the new private VIF as a backup route. Perform the switchover during a maintenance window by shutting down BGP on the existing private VIF. Decommission the existing Direct Connect connection.

B. Create a new private VIF on the new Direct Connect hosted connection. Attach the new private VIF to the existing Direct Connect gateway. Configure BGP routing on the new private VIF as a backup route. Perform the switchover during a maintenance window by shutting down BGP on the existing private VIF. Decommission the existing Direct Connect connection.

C. During a maintenance window, migrate the existing private VIF to the new Direct Connect hosted connection. Attach the existing private VIF to the existing Direct Connect gateway. Decommission the existing Direct Connect connection.

D. During a maintenance window, delete the existing private VIF and create a new private VIF to the new Direct Connect hosted connection. Attach the new private VIF to the existing Direct Connect gateway. Decommission the existing Direct Connect hosted connection.

Answer: B

Explanation:

The correct answer is B. Here's a breakdown of why, and why the other options are less suitable:

Why B is the Best Solution: The goal is to migrate a private VIF with minimal downtime. Option B achieves this by leveraging the existing Direct Connect Gateway (DXGW) and using BGP routing for a smooth transition. A new private VIF is created on the new Direct Connect connection in New York. This new VIF is associated with the pre-existing DXGW in us-east-1, meaning it can access the same VPCs without any configuration changes to the VPCs themselves or any applications within those VPCs. Setting up BGP routing with the new VIF as a backup route allows the on-premises network to learn routes through both connections (Virginia and New York) simultaneously. During a maintenance window, the old BGP session is shut down, and the new connection automatically becomes the primary route due to BGP best path selection. Finally, the old connection is decommissioned. This process offers a controlled failover and minimizes service interruption.

Why A is Incorrect: Creating a new Direct Connect gateway (DXGW) is unnecessary and introduces unnecessary complexity. The existing DXGW already provides connectivity to the required VPCs. A new DXGW would require re-configuring the VPC attachments and potentially impact existing services. Also, creating a new DXGW will increase costs.

Why C is Incorrect: Private VIFs cannot be "migrated" from one Direct Connect connection to another. VIFs are intrinsically tied to the specific Direct Connect connection on which they are created. This option is technically infeasible.

Why D is Incorrect: Deleting the existing private VIF before creating a new one causes a period of downtime. Traffic must then be re-routed once the new VIF becomes active. This method lacks a controlled failover mechanism. Also, you shouldn't need to decommission an entire hosted connection. This option increases unnecessary costs.

In summary, Answer B provides the most graceful migration by utilizing BGP routing to minimize downtime during the switchover. It leverages the existing Direct Connect Gateway and avoids unnecessary resource creation or deletion.

Supporting Documentation:

AWS Direct Connect Gateways: <https://docs.aws.amazon.com/directconnect/latest/UserGuide/direct-connect-gateways.html>

AWS Direct Connect Routing Policies: <https://docs.aws.amazon.com/directconnect/latest/UserGuide/routing-policies.html>

Question: 222

CertyIQ

A retail company is migrating its on-premises application to the AWS Cloud. Currently, the company has two on-premises data center locations. One data center is on the east coast of the United States, and one data center is on the west coast.

Each data center hosts four database systems. The largest database system stores 500 GB of data. The data centers are interconnected by two 10 GbE circuits for data synchronization. Each data center has two separate 1 GbE upstream internet connections. The company plans to have eight total VPCs to service its multiple business units. Four VPCs will be in the us-east-1 Region, and four will be in the us-west-2 Region.

A network engineer needs to design a connectivity solution that allows VPC-to-VPC connectivity. The solution must also allow secure connections between the on-premises data centers and AWS during the migration process. The company expects spikes in traffic among the VPCs during database synchronization. The company wants to run the migration plan during one weekend and as soon as technically possible. The company also wants to minimize long-term operational and human resources costs.

Which combination of steps will meet these requirements? (Choose two.)

- A. Deploy one transit gateway and attach all VPCs to it. Update the transit gateway and VPC route tables to allow any VPC to connect to any other VPC.
- B. Configure VPC peering between all the VPCs. Update the VPC route tables to allow connectivity.
- C. Provision two AWS Direct Connect connections from two Direct Connect locations that serve us-east-1 and us-west-2 to provide connectivity between the data centers and AWS.
- D. Provision one transit gateway VPN attachment for each data center to build connectivity between the on-premises data centers and AWS VPCs.
- E. Provision one AWS Site-to-Site VPN connection for each data center and for each VPC to build connectivity between the on-premises data centers and AWS VPCs.

Answer: CD

Explanation:

The combination of C and D meets the requirements:

VPC-to-VPC connectivity: The Direct Connect connections provide direct access to the VPCs, allowing them to communicate with each other.

Secure connections between on-premises data centers and AWS: The transit gateway VPN attachments ensure secure connections between the data centers and AWS, enabling database synchronization.

The other options do not meet all the requirements:

A provides VPC-to-VPC connectivity but does not provide direct connections to the on-premises data centers.

B provides VPC-to-VPC connectivity but may not be the most efficient or cost-effective solution for the company's needs.

E is not necessary, as it would require multiple Site-to-Site VPN connections that would increase operational costs.

Question: 223

A company is developing an API-based application on AWS for its process workflow requirements. The API will be invoked by clients in the company's on-premises data centers. The company has set up an AWS Direct Connect connection between on premises and AWS. A network engineer decides to implement the API as a private REST API in Amazon API Gateway. The network engineer wants to ensure that clients can reach the API endpoint through private communication.

Which solution can the network engineer use to invoke the API without any additional infrastructure setup?

- A. Create an interface VPC endpoint for API Gateway with private DNS names enabled. Access the API by using the private DNS name of the endpoint.
- B. Create an interface VPC endpoint for API Gateway with private DNS names enabled. Access the API by using an Amazon Route 53 alias of the endpoint.
- C. Create an interface VPC endpoint for API Gateway. Associate the endpoint with the private REST API. Access the API by using an Amazon Route 53 alias of the endpoint.
- D. Create an interface VPC endpoint for API Gateway with private DNS names enabled. Access the API by using the public DNS name of the endpoint.

Answer: A

Explanation:

The correct solution is A: Create an interface VPC endpoint for API Gateway with private DNS names enabled. Access the API by using the private DNS name of the endpoint.

Here's a detailed justification:

1. **Private REST API Requirements:** The company requires private communication to the API Gateway endpoint from their on-premises data centers. This means traffic should not traverse the public internet.
2. **VPC Endpoints for Private Access:** VPC endpoints are the key to enabling private connectivity to AWS services from within a VPC (and, in this case, extended from on-premises via Direct Connect). They allow you to connect to AWS services without using public IPs and without requiring internet gateways, NAT devices, or virtual private gateways. Specifically, interface VPC endpoints provide private access to services like API Gateway using private IP addresses within your VPC.
3. **Interface VPC Endpoint for API Gateway:** Creating an interface VPC endpoint specifically for API Gateway is necessary. This establishes a direct, private connection between the company's on-premises network (via Direct Connect to a VPC) and the API Gateway service.
4. **Private DNS:** Enabling private DNS names is critical. When enabled, AWS automatically provides a private DNS hostname that resolves to the private IP addresses of the endpoint network interfaces within your VPC. This allows clients within the VPC (and, through Direct Connect, on-premises clients) to resolve the API Gateway endpoint to a private IP address and communicate privately.
5. **Access via Private DNS Name:** On-premises clients, after proper DNS configuration (likely involving a conditional forwarder in their on-premises DNS server that directs queries for the API Gateway private DNS name to the VPC's DNS servers via Direct Connect), can resolve the private DNS name of the interface endpoint and connect to the API Gateway endpoint using private IP addresses.
6. **Why other options are incorrect:**

B (Route 53 Alias): While Route 53 aliases can be helpful, they aren't necessary here. The private DNS name provided by the interface endpoint is sufficient for private access. Using Route 53 adds complexity without providing significant additional benefit in this scenario.

C (No Private DNS Enabled): Without private DNS enabled, the clients can not use a private IP. This

defeats the purpose of establishing a private communication channel from on-premise to API Gateway.

D (Public DNS Name): Using the public DNS name would force traffic to traverse the public internet, violating the requirement for private communication.

7. **No Additional Infrastructure:** By using a VPC interface endpoint, the company minimizes the need for additional infrastructure, such as NAT gateways or proxy servers within the VPC, to facilitate the communication from on-premises. The Direct Connect link connects to the VPC, and the VPC endpoint provides private access to the API Gateway.

Authoritative Links:

AWS VPC Endpoints: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html>

AWS PrivateLink (underlying technology for VPC Endpoints): <https://aws.amazon.com/privatelink/>

API Gateway Private Integrations:

<https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-private-apis.html>

Question: 224

CertyIQ

A banking company has an application that must connect to specific public IP addresses from a VPC. A network engineer has configured routes in the route table that is associated with the application's subnet to the required public IP addresses through an internet gateway.

The network engineer needs to set up email notifications that will alert the network engineer when a user adds a default route to the application subnet's route table with the internet gateway as a target.

Which solution will meet these requirements with the LEAST implementation effort?

A. Create an AWS Lambda function that reads the routes in the route table and sends an email notification. Configure the Lambda function to send an email notification if any route is configured with 0.0.0.0/0 or ::/0 CIDRs to the internet gateway. Configure the Lambda function to run every minute.

B. Create an AWS Lambda function that will be invoked by an Amazon EC2 CreateRoute API call. Configure the Lambda function to send an email notification. Configure the Lambda function to send an email notification if any route is configured with 0.0.0.0/0 or ::/0 CIDRs to the internet gateway.

C. Create AWS Config rules for the route table by using the internet-gateway-authorized-vpc-only managed rule. Create an Amazon EventBridge rule to match the AWS Config rule and to route to an Amazon Simple Notification Service (Amazon SNS) topic to send an email notification.

D. Create an AWS Config rule for the route table by using the no-unrestricted-route-to-igw managed rule. Create an Amazon EventBridge rule to match the AWS Config rule and to route to an Amazon Simple Notification Service (Amazon SNS) topic to send an email notification.

Answer: D

Explanation:

Here's a detailed justification for why option D is the most appropriate solution, and why the other options are less optimal:

The core requirement is to detect and notify when a default route (0.0.0.0/0 or ::/0) pointing to an Internet Gateway (IGW) is added to the application subnet's route table. The goal is to minimize implementation effort.

Option D: AWS Config rule with EventBridge and SNS (Correct)

This solution leverages AWS's managed services for compliance and event-driven architecture, reducing custom coding and management overhead.

AWS Config no-unrestricted-route-to-igw Managed Rule: This specific managed rule is designed explicitly to

detect whether a route table has a route with a destination of 0.0.0.0/0 or ::/0 targeting an IGW. This direct applicability makes it ideal. AWS Config continuously monitors the route table against this rule.

Amazon EventBridge Rule: EventBridge allows you to define rules that trigger actions based on events. In this case, we create a rule that listens for events emitted by AWS Config when the no-unrestricted-route-to-igw rule changes state (e.g., from compliant to non-compliant).

Amazon SNS Topic: The EventBridge rule forwards the event to an SNS topic. The network engineer subscribes to this topic, receiving email notifications whenever the Config rule flags a violation (i.e., a default route to the IGW is added).

This approach minimizes the need for custom code and leverages built-in capabilities for efficient monitoring and alerting.

Option A: Lambda function polling (Incorrect)

Polling the route table every minute with a Lambda function is inefficient and can be costly at scale. Continuously checking for changes rather than reacting to events is suboptimal. It requires custom code to parse the route table configuration, increasing implementation effort. It's less responsive than an event-driven approach because it only checks periodically.

Option B: Lambda function triggered by CreateRoute API call (Incorrect)

While more event-driven than option A, this approach only triggers when a new route is created. It won't detect a modification of an existing route to add the offending default route. It still requires custom Lambda code to parse the route table configuration and determine if a default route to the IGW has been introduced. This solution would miss scenarios where the problematic route is added as part of a bulk update or automated provisioning outside of individual CreateRoute calls.

Option C: AWS Config internet-gateway-authorized-vpc-only Managed Rule (Incorrect)

The internet-gateway-authorized-vpc-only managed rule is designed to check if the VPC is configured to allow internet access only through specified Internet Gateways. It doesn't specifically check for default routes in route tables. Therefore, it's not the best fit for the requirement.

Why Option D is the Least Effort

Option D requires minimal custom code because it directly uses the AWS Config managed rule that precisely addresses the use case. Configuring EventBridge and SNS is relatively straightforward and well-documented. Other options require more coding, more maintenance, and are not as efficient or as directly applicable.

Supporting Links:

AWS Config Managed Rules: <https://docs.aws.amazon.com/config/latest/developerguide/managed-rules-by-aws.html>

EventBridge: <https://aws.amazon.com/eventbridge/>

SNS: <https://aws.amazon.com/sns/>

Question: 225

CertyIQ

A company is building an internet-facing application that is hosted on an Amazon Elastic Kubernetes Service (Amazon EKS) cluster. The company is using the Amazon VPC Container Network Interface (CNI) plugin for Kubernetes for pod networking connectivity. The company needs to expose its application to the internet by using a Network Load Balancer (NLB).

The pods that host the application must have visibility of the source IP address that is contained in the original packet that the NLB receives.

How should the network engineer configure the NLB and Amazon EKS settings to achieve these goals?

- A. Specify the ip target type for the NLB. Set the externalTrafficPolicy attribute to Local in the Kubernetes service specification.
- B. Specify the instance target type for the NLB. Set the externalTrafficPolicy attribute to Cluster in the Kubernetes service specification.
- C. Specify the instance target type for the NLB. Set the externalTrafficPolicy attribute to Local in the Kubernetes service specification.
- D. Specify the ip target type for the NLB. Set the externalTrafficPolicy attribute to Cluster in the Kubernetes service specification.

Answer: A

Explanation:

The correct answer is **A**. Here's why:

Explanation:

To ensure that pods running within the EKS cluster have visibility of the client's original source IP address when using an NLB, we need to configure both the NLB and the Kubernetes service correctly.

1. **NLB Target Type:** The NLB needs to be configured with an **IP target type**. When using the IP target type, the NLB forwards traffic directly to the IP addresses of the pods. This is essential for preserving the source IP address. If the NLB is configured with instance target type, the NLB forwards traffic to the EC2 instance's IP where the pod resides. The original source IP will be lost in this case.
2. **externalTrafficPolicy:** In the Kubernetes service specification, the externalTrafficPolicy attribute needs to be set to **Local**. This setting ensures that the service only forwards external traffic to pods that are running on the same node where the NLB connection terminates. When set to Local, the service will only route traffic to local endpoints on the same node. This way, traffic from the NLB to the node will be routed to a local pod and preserve the client source IP. If set to Cluster, the service will distribute traffic to all pods in the cluster, regardless of the node they are running on. This causes an extra hop that can result in losing the original source IP address.

Why other options are incorrect:

B and C (Instance Target Type): Using instance target type for the NLB will not preserve the source IP. The NLB forwards traffic to the EC2 instance's primary network interface, and the source IP becomes the instance's IP address.

D (Cluster externalTrafficPolicy): Using externalTrafficPolicy: Cluster with IP target type could work in some circumstances, but it can lead to asymmetric routing and potential hairpinning issues. Local is the more robust and reliable configuration for source IP preservation with NLBs.

Authoritative Links:

Using Network Load Balancers: <https://docs.aws.amazon.com/eks/latest/userguide/network-load-balancing.html>

Kubernetes externalTrafficPolicy: <https://kubernetes.io/docs/tasks/access-application-cluster/create-external-load-balancer/#preserving-the-client-source-ip>

Question: 226

CertyIQ

A company is running its application servers on Amazon EC2 instances. The EC2 instances run in separate VPCs that are connected by a transit gateway. The EC2 instances launch in a private subnet with a route to the transit

gateway for internal and external connectivity. The external connectivity is provided by a VPC with firewall devices that perform an inspection for packets that ingress and egress through an internet gateway.

A network engineer needs to help the company's application team increase the payload size per packet delivery between the EC2 instances. All network connectivity must be through the transit gateway

What should the network engineer do to meet these requirements?

- A. Enable jumbo frames on the transit gateway. Instruct the application team to set the maximum transmission unit (MTU) of the system's network interfaces to 9001 bytes.
- B. Instruct the application team to set the maximum transmission unit (MTU) of the VPC to 8500 bytes.
- C. Instruct the application team to set up enhanced networking on the system by using the enhanced networking adapter. Set the maximum transmission unit (MTU) to 9001 bytes.
- D. Instruct the application team to set the maximum transmission unit (MTU) of the system's network interfaces to 8500 bytes.

Answer: A

Explanation:

The correct answer is A. Here's why:

The primary goal is to increase the payload size per packet between EC2 instances across VPCs connected by a transit gateway. This directly relates to the Maximum Transmission Unit (MTU), which defines the largest packet size that can be transmitted over a network.

Option A is correct because it addresses the entire path:

1. **Enable Jumbo Frames on Transit Gateway:** Enabling jumbo frames (MTU of 9001 bytes) on the transit gateway is crucial. The transit gateway needs to support the larger packet size to facilitate the increased payload. Without this, the larger packets will be fragmented, negating the benefits and potentially increasing overhead. Refer to <https://docs.aws.amazon.com/transit-gateway/latest/userguide/transit-gateway-attachments.html#transit-gateway-attachments-modify-mtu> for details on MTU configuration for transit gateway attachments.
2. **Set MTU on EC2 Instances:** The application team needs to configure the network interfaces of the EC2 instances to use an MTU of 9001 bytes. This ensures that the instances can send and receive larger packets. Setting the MTU on the instance without enabling jumbo frames on the transit gateway won't work because the packets will be fragmented at the transit gateway. See https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/network_mtu.html for instructions on setting the MTU on EC2 instances.

Options B, C, and D are incorrect for the following reasons:

Option B: Setting the MTU of the VPC to 8500 bytes is not possible. VPCs do not have configurable MTU settings in the same way that network interfaces or transit gateway attachments do. The relevant settings are at the instance and transit gateway attachment level.

Option C: While enhanced networking (using an enhanced networking adapter) and setting the MTU to 9001 bytes are important for maximizing network performance, the transit gateway also needs to be configured for jumbo frames. Enhanced networking primarily improves throughput and reduces latency at the instance level.

Option D: Setting the MTU on the EC2 instances to 8500 bytes would work, but it's less than ideal. The jumbo frame standard (9001 bytes) provides the maximum benefit, and the network interfaces and transit gateway should support this larger size if possible. Using 8500 would provide a benefit over the standard 1500 byte MTU, but not the maximum possible.

In summary, increasing the payload size per packet requires end-to-end configuration, including enabling jumbo frames on the transit gateway and setting the MTU of the network interfaces on the EC2 instances to 9001 bytes.

Question: 227

A network engineer needs to monitor internet metrics for an application that is in a VPC. The metrics include user experiences such as health events, latency, and traffic insights.

The network engineer sets up Amazon CloudWatch Internet Monitor for the application. The engineer wants to push the internet health events to a third-party target.

Which solution will meet these requirements with the LEAST implementation effort?

- A. Create a third-party API endpoint in Amazon EventBridge. Configure Internet Monitor to send the events to the third-party API endpoint in EventBridge.
- B. Create a third-party API endpoint in Amazon EventBridge. Create a rule in EventBridge that uses Internet Monitor as the source and the third-party API endpoint in EventBridge as the destination.
- C. Create a third-party API endpoint in Internet Monitor. Configure Internet Monitor to send the events to an Amazon S3 bucket. Configure an AWS Lambda function to send the events to the third-party API endpoint in Internet Monitor.
- D. Create a third-party API endpoint in Internet Monitor. Configure Internet Monitor to send the events to the third-party API endpoint in Internet Monitor.

Answer: B

Explanation:

The requirement is to push Internet Monitor health events to a third-party target with minimal implementation effort. Option B offers the most efficient solution by leveraging Amazon EventBridge.

Here's why Option B is the best:

EventBridge Integration: Internet Monitor events can be directly routed to EventBridge. EventBridge acts as a central event bus, simplifying the process of forwarding events to various targets.

Third-party API Integration via EventBridge: EventBridge can directly invoke third-party API endpoints. This eliminates the need for intermediary services like S3 and Lambda, thus minimizing complexity and implementation effort.

Direct Routing: Creating a rule in EventBridge with Internet Monitor as the source and the third-party API endpoint as the destination establishes a direct event flow.

Least Effort: This approach avoids complex configurations, custom coding with Lambda, or the need to manage storage services like S3.

Option A is less efficient because it introduces an unnecessary layer of complexity by creating an API endpoint in EventBridge itself rather than directly leveraging EventBridge's built-in capabilities to target a third-party API.

Option C and D suggest configuring Internet Monitor to directly interact with a third-party API or S3/Lambda, which is not the intended or most straightforward way to integrate with external services. Internet Monitor is designed to work seamlessly with EventBridge for event distribution. Also, option D is illogical because Internet Monitor cannot have "third-party API endpoints" created within it.

Therefore, creating an EventBridge rule with Internet Monitor as the source and the third-party API endpoint as the destination (Option B) provides the least implementation effort and the most direct path to meet the requirements.

Supporting Links:

[Amazon EventBridge Documentation](#)

[CloudWatch Internet Monitor Documentation](#)

Question: 228

A company has a web application that runs in eight AWS Regions. In each Region, the application is hosted on multiple compute resources behind an Application Load Balancer (ALB).

The different Regions are using different domains. Each ALB is configured to accept only HTTPS traffic. Each ALB uses a certificate from AWS Certificate Manager (ACM).

The company wants to simplify the application's appearance on the web by using a new single domain for all Regions. A network engineer needs to implement this change by designing a solution that also will minimize latency for the application's end users.

Which combination of actions will meet these requirements? (Choose three.)

- A. Use ACM to create an SSL/TLS certificate in the us-east-1 Region for the new domain.
- B. Set up latency-based routing in Amazon Route 53 for the new domain. Add the ALBs from all the Regions as targets.
- C. Create an alias record for the accelerator in Amazon Route 53 for the new domain.
- D. Create a standard accelerator in AWS Global Accelerator. Configure a listener for TCP traffic. Add all the ALBs as targets for the listener.
- E. Use ACM to create an SSL/TLS certificate for each Region. Configure all the ALBs to use the certificate in their respective Regions.
- F. Create a custom routing accelerator in AWS Global Accelerator. Configure a listener for HTTPS traffic. Add all the ALBs as targets for the listener. Configure the accelerator to terminate TLS by using the SSL/TLS certificate from ACM.

Answer: CDE

Explanation:

Here's a detailed justification for why options C, D, and E are the correct choices for meeting the requirements of simplifying a multi-region web application's domain while minimizing latency using AWS services.

Option E: Use ACM to create an SSL/TLS certificate for each Region. Configure all the ALBs to use the certificate in their respective Regions. This is essential for HTTPS traffic. Each ALB needs a valid certificate in its respective region to properly encrypt and decrypt traffic. ACM certificates are region-specific unless used with CloudFront. Global Accelerator works best with regional ALBs each with their own valid certificates.

Option D: Create a standard accelerator in AWS Global Accelerator. Configure a listener for TCP traffic. Add all the ALBs as targets for the listener. AWS Global Accelerator uses AWS's global network to direct user traffic to the closest healthy endpoint. By using a standard accelerator, you benefit from Anycast static IPs that are advertised globally, ensuring low-latency routing. Since the ALBs are already configured for HTTPS, the Global Accelerator listener can be configured for TCP traffic, as TLS termination will be handled by the ALBs.

Option C: Create an alias record for the accelerator in Amazon Route 53 for the new domain. Route 53 is used to resolve domain names to IP addresses or other resources. By creating an alias record pointing to the Global Accelerator, Route 53 forwards all traffic for the new domain to the Global Accelerator, which then uses its intelligent routing to direct traffic to the appropriate ALB based on latency.

Options A and F are incorrect.

Option A: Creating a single certificate in us-east-1 would not work for all regions if you need each region to validate HTTPS. While ACM provides certificates that work with CloudFront globally, the ALBs operate regionally and should have regional certificates for security and best practices.

Option F: Global Accelerator does not have TLS termination capabilities. The configuration must be done on

ALBs. Choosing this is therefore wrong.

Latency-based routing is a function of Route 53 and is not the optimal solution. Global Accelerator is designed to provide lower latency than latency-based routing.

References:

[AWS Global Accelerator](#): Learn about Global Accelerator features, benefits, and use cases.

[AWS Certificate Manager \(ACM\)](#): Understand how ACM simplifies SSL/TLS certificate management.

[Amazon Route 53](#): Explore the features of Route 53, including alias records and routing policies.

Question: 229

CertyIQ

A company has a VPC that includes application workloads that run on Amazon EC2 instances in a single AWS Region. The company wants to use AWS Local Zones to deploy an extension of the application workloads that run in the Region. The extended workloads in the Local Zone need to communicate bidirectionally with the workloads in the VPC in the Region.

Which solution will meet these requirements MOST cost-effectively?

- A. Create a new VPC in the Local Zone. Attach all the VPCs to a transit gateway. Configure routing for the transit gateway and the VPCs. Deploy instances in the new VPC.
- B. Deploy a third-party appliance in a new VPC in the Region. Create a new VPC in the Local Zone. Create VPN connections to the appliance for the VPCs. Deploy instances in the new VPC in the Local Zone.
- C. Create a new subnet in the Local Zone. Deploy a third-party appliance in the VPC with interfaces in each subnet. Configure the new subnet to route the Local Zone through the appliance. Deploy instances in the new subnet.
- D. Create a new subnet in the Local Zone. Configure the new subnet to use a CIDR block that is within the VPC's CIDR block. Deploy instances in the new subnet in the Local Zone.

Answer: D

Explanation:

Here's a detailed justification for why option D is the most cost-effective solution for extending application workloads to an AWS Local Zone while maintaining bidirectional communication within a single VPC:

Option D: Create a new subnet in the Local Zone. Configure the new subnet to use a CIDR block that is within the VPC's CIDR block. Deploy instances in the new subnet in the Local Zone.

This approach leverages the inherent capability of a VPC to span multiple Availability Zones and Local Zones within the same region. By creating a new subnet within the existing VPC but specifically in the Local Zone, instances launched in this subnet are logically part of the same network as those running in the original Availability Zone. This eliminates the need for additional networking infrastructure like transit gateways or VPNs to facilitate communication. Because the new subnet is part of the existing VPC and uses a CIDR block within the VPC's range, routing is automatically handled. Instances in both the Region and Local Zone can communicate directly with each other using private IP addresses. Since no transit gateway or VPN is created, this also avoids the associated cost. This is the simplest and most cost-effective solution since no additional networking components are needed.

Why other options are less suitable:

Option A (Transit Gateway): While transit gateway allows you to interconnect multiple VPCs and on-premise networks in a hub-and-spoke manner, creating a new VPC solely for the Local Zone and then connecting it to the existing VPC via a transit gateway introduces unnecessary complexity and cost. The goal is integration, not isolation, within the same VPC. A transit gateway is overkill and adds additional costs.

Option B (Third-party Appliance and VPNs): This option requires setting up a third-party appliance for VPN connections between the VPC in the Region and the VPC in the Local Zone. This adds operational overhead, licensing costs (if applicable), and performance bottlenecks associated with VPN encryption/decryption. It's much more complicated and expensive than using the existing VPC.

Option C (Third-party Appliance in existing VPC): Similar to Option B, introducing a third-party appliance within the existing VPC creates operational overhead and potential performance bottlenecks. While it keeps everything within one VPC, it is an unnecessary complexity and expense when direct subnet creation is possible.

Key Concepts:

AWS Local Zones: Extensions of an AWS Region that are geographically closer to your users. They allow you to run latency-sensitive applications closer to end-users. <https://aws.amazon.com/about-aws/global-infrastructure/localzones/>

Amazon VPC: A logically isolated section of the AWS Cloud where you can launch AWS resources in a virtual network that you define. <https://aws.amazon.com/vpc/>

Subnets: A range of IP addresses in your VPC. You can launch AWS resources into a specified subnet. Subnets can span Availability Zones and Local Zones.

https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Subnets.html

By creating a new subnet within the existing VPC in the Local Zone, you seamlessly extend your application workload with minimal cost and complexity.

Question: 230

CertyIQ

A company is using AWS Cloud WAN with one edge location in the us-east-1 Region and one edge location in the us-west-1 Region. A shared services segment exists at both edge locations. Each shared services segment has a VPC attachment to each inspection VPC in each Region. The inspection VPCs inspect traffic from a WAN by using AWS Network Firewall.

The company creates a new segment for a new business unit (BU) in the us-east-1 edge location. The new BU has three VPCs that are attached to the new BU segment. To comply with regulations, the BU VPCs must not communicate with each other. All internet-bound traffic must be inspected in the inspection VPC.

The company updates VPC route tables so any traffic that is bound for internet goes to the AWS Cloud WAN core network.

The company plans to add more VPCs for the new BU in the future. All future VPCs must comply with regulations.

Which solution will meet these requirements in the MOST operationally efficient way? (Choose two.)

- A. Update the network policy to share the shared services segment with the BU segment.
- B. Create a network policy to share the inspection service segment with the BU segment.
- C. Set the isolate-attachments field to True for the BU segment.
- D. Set the isolate-attachments field to False for the BU segment.
- E. Update the network policy to add static routes for the BU segment. Configure the shared services segment to route traffic related to VPC CIDR blocks to each respective VPC attachment.

Answer: BC

Explanation:

Here's a detailed justification for the answer choices B and C, explaining why they meet the requirements most efficiently:

Choice B: Create a network policy to share the inspection service segment with the BU segment. This is crucial for directing internet-bound traffic from the BU VPCs to the inspection VPCs. By sharing the shared services (inspection) segment with the new BU segment, you allow traffic originating from the BU VPCs (directed towards the internet by VPC route tables pointing to Cloud WAN) to flow into the shared inspection VPCs. This allows Network Firewall to inspect the traffic before it exits to the internet, thus satisfying the regulation requirement. Without this sharing, the BU VPCs would have no path to the inspection resources.

Choice C: Set the isolate-attachments field to True for the BU segment. Setting isolate-attachments to True on the BU segment prevents direct communication between VPC attachments within that segment. This directly addresses the regulatory requirement that the BU VPCs must not communicate with each other. Cloud WAN enforces this isolation at the core network level, simplifying management and ensuring compliance.

Why other options are incorrect:

Choice A: Update the network policy to share the shared services segment with the BU segment. While sharing is needed, the shared services segment likely hosts more than just inspection services. Sharing the entire shared services segment might grant the BU VPCs unintended access to other shared resources they shouldn't have, violating the principle of least privilege and potentially creating security risks. It's better to share only the inspection segment.

Choice D: Set the isolate-attachments field to False for the BU segment. This is the opposite of what's needed. Setting isolate-attachments to False would allow direct communication between the BU VPCs, violating the regulatory requirement.

Choice E: Update the network policy to add static routes for the BU segment. Configure the shared services segment to route traffic related to VPC CIDR blocks to each respective VPC attachment. This solution is more complex and less scalable than using segment sharing and isolate-attachments. It requires manual configuration and maintenance of static routes, which becomes cumbersome as the number of VPCs increases. Cloud WAN is designed to abstract away this routing complexity.

In summary, choices B and C provide the most operationally efficient solution by leveraging Cloud WAN's native capabilities (segment sharing and attachment isolation) to enforce the required security policies without unnecessary complexity.

Authoritative Links:

AWS Cloud WAN Segmentation: <https://docs.aws.amazon.com/cloud-wan/latest/what-is-cloudwan/cloudwan-segments.html>

AWS Cloud WAN Policies: <https://docs.aws.amazon.com/cloud-wan/latest/what-is-cloudwan/cloudwan-policies.html>

AWS Network Firewall: <https://aws.amazon.com/network-firewall/>

Question: 231

CertyIQ

A company hosts a highly available, scalable, and resilient application on Amazon EC2 instances that are part of an Auto Scaling group. A network engineer is planning to integrate IPv6 support with the application deployment in phases. The first phase is to enable IPv6 service consumption on the public Network Load Balancers (NLBs) that are deployed across the infrastructure. The target groups for the NLBs are configured as the Auto Scaling groups of the EC2 instances that host the application. The NLBs are configured for dual-stack operation.

During the testing of the first phase, the IPv6 application queries are not reaching the backend servers.

What is the cause of this issue?

A. The subnets where the EC2 instances are deployed do not have IPv6 addresses configured.

- B.The route tables for the NLB subnets do not have IPV6 routing configured.
- C.The route tables for the EC2 subnets do not have IPV6 routing configured.
- D.The security groups that are associated with the NLBs do not allow IPv6 traffic.

Answer: A

Explanation:

The issue is that the IPv6 application queries are not reaching the backend EC2 instances. This suggests a problem with IPv6 connectivity between the NLB and the EC2 instances.

Option A states that the subnets where the EC2 instances are deployed do not have IPv6 addresses configured. This is the correct answer. For the EC2 instances to receive and process IPv6 traffic from the NLB, the subnets in which they reside must be configured with IPv6 CIDR blocks. If the subnets are not IPv6-enabled, the EC2 instances will not have IPv6 addresses assigned to them, and they will be unable to receive traffic over IPv6, even if the NLB is configured for dual-stack operation. The target groups point to EC2 instances. If those EC2 instances are only operating with IPv4, the IPv6 traffic will fail.

Option B suggests a routing problem for the NLB subnets. While correct routing is important, the core problem is the lack of IPv6 addresses assigned to the EC2 instances. The NLB might forward IPv6 traffic, but the instances cannot receive it if they lack IPv6 addresses.

Option C also suggests a routing issue for EC2 subnets. Similar to Option B, this is not the primary cause. The subnets must have IPv6 enabled.

Option D focuses on security groups, but the problem manifests earlier in the traffic flow. If the EC2 instances have no IPv6 addresses assigned, no traffic will reach the security groups.

Therefore, the root cause is that the subnets in which the EC2 instances are located are not IPv6-enabled, leading to the EC2 instances lacking IPv6 addresses.

Refer to AWS documentation for further information on configuring IPv6 for subnets:

<https://docs.aws.amazon.com/vpc/latest/userguide/ipv6.html> Also look at NLB and IPv6:

<https://docs.aws.amazon.com/elasticloadbalancing/latest/network/network-load-balancer-ipv6.html>

Question: 232

CertyIQ

A company wants to implement a distributed architecture on AWS that uses a Gateway Load Balancer (GWLB) and GWLB endpoints.

The company has chosen a hub-and-spoke model. The model includes a GWLB and virtual appliances that are deployed into a centralized appliance VPC and GWLB endpoints. The model also includes internet gateways that are configured in spoke VPCs.

Which sequence of traffic flow to the internet from the spoke VPC is correct?

- A.1. An application in a spoke VPC sends traffic to the GWLB endpoint based on the VPC route table configuration.
- 2. Traffic is delivered securely and privately to the GWLB.
- 3. The GWLB sends the traffic to a virtual appliance for inspection.
- 4. Return traffic flows back to the GWLB endpoint and out to the internet through the internet gateway.
- B.1. An application in a spoke VPC sends traffic to the GWLB endpoint based on the VPC route table configuration.
- 2. Traffic is delivered securely and privately to the GWLB endpoint.
- 3. The GWLB sets the X-Forwarded-For request header and sends the traffic to a virtual appliance for inspection.
- 4. Return traffic flows back to the GWLB and out to the internet through an internet gateway.

- C.1. An application in a spoke VPC sends traffic to the GWLB endpoint.
 - 2. Traffic is delivered securely and privately to the GWLB.
 - 3. The GWLB sets the X-Forwarded-For request header and sends the traffic to a virtual appliance for inspection.
 - 4. Return traffic flows back to the GWLB endpoint and out to the internet through the internet gateway.
- D.1. An application in a spoke VPC sends traffic to the GWLB.
 - 2. Traffic is delivered securely and privately to the GWLB endpoint.
 - 3. The GWLB sends the traffic to a virtual appliance for inspection.
 - 4. Return traffic flows back to the GWLB and out to the internet through an internet gateway.

Answer: A

Explanation:

The correct answer is A because it accurately reflects the traffic flow in a hub-and-spoke architecture using a Gateway Load Balancer (GWLB) for inspecting traffic before it reaches the internet.

Here's a breakdown of why A is correct and why the others are not:

Step 1 (A, B, C, D): An application in the spoke VPC initiates the traffic, and based on the VPC route table configuration, this traffic is directed towards the GWLB endpoint within the spoke VPC. All options correctly identify this initial step.

Step 2 (A, B, C, D): Traffic is securely and privately delivered to the GWLB. The underlying mechanism for this is the use of GENEVE encapsulation which creates a tunnel between the GWLB endpoint and the GWLB itself. Again, most options correctly highlight traffic delivery to the GWLB.

Step 3 (A, C, D): The GWLB then forwards the traffic to a virtual appliance within the central appliance VPC for inspection (e.g., intrusion detection, firewall). Option B incorrectly mentions the GWLB setting the X-Forwarded-For header, which is typically associated with HTTP load balancers, not a GWLB which operates at Layer 3/4. While a virtual appliance could add the X-Forwarded-For header after inspection, it is incorrect to say that it is the GWLB that sets the header.

Step 4 (A): This is the key differentiator. After inspection, the traffic returns from the virtual appliance back to the GWLB. The GWLB sends it to the GWLB endpoint in the spoke VPC, then to the Internet Gateway (IGW) configured in the spoke VPC.

Option B describes a less accurate traffic flow, and it incorrectly states the GWLB is the one setting the X-Forwarded-For header.

Option C has the same issue with the incorrect header modification by the GWLB.

Option D incorrectly starts the traffic flow at the GWLB, which is incorrect since the applications sit in the spoke VPC.

Therefore, only option A correctly outlines the complete and accurate traffic flow, aligning with the intended function of a GWLB in a hub-and-spoke network architecture where the return traffic passes back through the GWLB endpoint and utilizes the spoke VPC's IGW for egress.

Supporting resources:

[AWS Gateway Load Balancer](#)

[AWS Gateway Load Balancer Documentation](#)

Question: 233

CertyIQ

A network engineer needs to provide a list of IP addresses that are sending traffic to an Amazon EC2 instance. VPC

flow logs are enabled. The EC2 instance has a single network interface and two assigned IP addresses. However, the flow logs are logging traffic only for the primary IP address. The network engineer needs to determine whether any traffic is being sent to the second IP address of the EC2 instance.

What should the network engineer do to locate the traffic flow for the second IP address?

- A. Create a new flow log that includes the `pkt-dstaddr` field to capture the original destination IP address of the traffic.
- B. Create a new flow log that includes the `dstaddr` field to capture the original destination IP address of the traffic.
- C. Create a new flow log that includes the `pkt-srcaddr` field to capture the original destination IP address of the traffic.
- D. Create a new flow log that includes the `srcaddr` field to capture the original destination IP address of the traffic.

Answer: A

Explanation:

The correct answer is A: Create a new flow log that includes the `pkt-dstaddr` field to capture the original destination IP address of the traffic.

Here's a detailed justification:

VPC Flow Logs capture information about the IP traffic going to and from network interfaces in your VPC. By default, flow logs capture the source and destination IP addresses (`srcaddr` and `dstaddr`) after network address translation (NAT) has occurred within the VPC. If an instance has multiple IP addresses and traffic is being routed to it through an internal load balancer or other NAT device within the VPC, the standard `dstaddr` field will reflect the IP address of the NAT device, not the originally intended destination IP address on the EC2 instance. This is because NAT changes the destination IP as the packet transits through it.

To capture the original destination IP address, even after NAT has occurred, you need to specifically enable the `pkt-dstaddr` field. This field represents the packet's destination IP address before any NAT is applied. By adding `pkt-dstaddr` to the flow log configuration, the network engineer can directly see whether traffic is indeed destined for the EC2 instance's secondary IP address, even if NAT is present in the traffic path within the VPC.

Option B, while also related to destination addresses, does not solve the problem where NAT is present. `dstaddr` would show the post-NAT destination, which is not the EC2 instance's secondary IP address. Similarly, options C and D, focusing on source addresses (`pkt-srcaddr` and `srcaddr` respectively), are irrelevant to determining the destination of the traffic, which is the core issue. The `pkt-srcaddr` will contain source IP address of the traffic before NAT. The `srcaddr` will contain source IP address of the traffic after NAT.

Therefore, creating a new flow log configuration that includes the `pkt-dstaddr` field is the only way to reliably determine if traffic is being sent to the EC2 instance's secondary IP address in environments where NAT may be obscuring the original destination IP address.

Supporting documentation:

VPC Flow Logs - AWS Documentation: This official documentation explains the functionality of VPC Flow Logs, including the different fields that can be captured, such as `pkt-dstaddr`.

VPC Flow Logs record format - AWS Documentation: This provides an overview of various fields that can be logged by VPC Flow Logs and their meanings. It explicitly describes `pkt-dstaddr` as the original destination IP address.

Question: 234

A company has configured an AWS Cloud WAN core network with edge locations in the us-east-1 Region and the us-west-1 Region. Each edge location has two segments: development and staging. The segments use the default core network policy.

The company has attached VPCs to the core network. A development VPC is attached to the development segment in us-east-1 and is configured to use the 10.0.0.0/16 CIDR block. A staging VPC is attached to the staging segment in us-west-1 and is configured to use the 10.5.0.0/16 CIDR block. The company has updated the route tables for both VPCs with a route that directs any traffic for 0.0.0.0/0 to the core network.

The company's network team needs to establish communication between the two VPCs by using the AWS Cloud WAN core network. The network team is not receiving a response during tests of communication between the VPCs. The network team has verified that security groups and network ACLs are not blocking the traffic.

What should the network team do to establish this communication?

- A. Update both VPC route tables to have a new static route. Configure a route on the development VPC to direct the traffic for 10.0.0.0/16 to the development VPC attachment. Configure a route on the staging VPC to direct the traffic for 10.5.0.0/16 to the staging VPC attachment.
- B. Update the segment filter to allow traffic on the development and staging segments.
- C. Set the isolate-attachments parameter to False for the development and staging segments.
- D. Update the core network policy to add a static route for each segment. Configure a route to direct the traffic for 10.0.0.0/16 to the development VPC attachment. Configure a route to direct the traffic for 10.5.0.0/16 to the staging VPC attachment.

Answer: D

Explanation:

The correct answer is D. To enable communication between VPCs connected to different segments within an AWS Cloud WAN core network, static routes must be defined in the core network policy.

Here's why:

Cloud WAN Segmentation: By default, segments in Cloud WAN are isolated. This means traffic is not automatically routed between them. The development and staging segments are distinct, and without explicit routing, the core network will not forward traffic between them.

Core Network Policy Routing: Cloud WAN's core network policy dictates how traffic is routed within the core network. This policy is where you define static routes to enable cross-segment communication.

Static Routes: Static routes explicitly define the destination CIDR and the next hop for traffic. In this scenario, we need to tell the core network that traffic destined for 10.0.0.0/16 (development VPC) should be routed to the development VPC attachment and traffic destined for 10.5.0.0/16 (staging VPC) should be routed to the staging VPC attachment.

VPC Route Tables (Partial Configuration): The existing VPC route tables that direct 0.0.0.0/0 to the core network attachments are correct for general traffic flow to the core network, but they don't influence routing within the core network itself. The Cloud WAN core network needs its own routing rules.

Why Other Options are Incorrect:

A: Updating VPC route tables to include routes for each other's CIDRs directed towards the VPC attachment will not work. VPC route tables manage traffic flow from VPC instances to the core network; they do not influence how the core network itself routes traffic.

B: There is no "segment filter" configurable to just allow the traffic in Cloud WAN

C: The isolate-attachments parameter controls whether attachments within a single segment can communicate, not across segments. Setting it to False would allow VPCs within the same segment to communicate but will

not address the cross-segment routing issue.

Therefore, the only way to enable communication between the development and staging VPCs in this scenario is to update the core network policy with static routes that explicitly define the routing for the destination CIDRs (10.0.0.0/16 and 10.5.0.0/16) to their respective VPC attachments.

Supporting Documentation:

AWS Cloud WAN Documentation: <https://docs.aws.amazon.com/cloud-wan/index.html>

Cloud WAN Routing: <https://docs.aws.amazon.com/cloud-wan/latest/working-with/core-network-routing.html>

Question: 235

CertyIQ

A company has VPCs in the us-east-1 Region that are connected to each other through a transit gateway. A network engineer needs to establish an AWS Direct Connect connection between the company's on-premises data center and the transit gateway for the migration of a workload.

The Direct Connect connection is UP according to the ConnectionState metric in Amazon CloudWatch. However, the VIF is DOWN. The network engineer has verified the transit VIF and BGP configurations on the on-premises router and has found no issues. However, the network engineer is unable to ping the Amazon peer IP address.

Which combination of steps should the network engineer take to troubleshoot this issue? (Choose three.)

- A. Verify that the correct IP address and subnet mask are in use for the subinterface on the router.
- B. Ensure that VLAN trunking is disabled on the router.
- C. Verify that the router has a MAC address entry from the AWS endpoint in the Address Resolution Protocol (ARP) table.
- D. Verify that the optical signal that is received over the cross connect is optimal.
- E. Ensure that the correct VLAN tag is applied on the subinterface configuration on the router.
- F. Ensure that TCP port 179 is not being blocked at the on-premises router.

Answer: ACE

Explanation:

The correct answer is ACE. Let's analyze why each option is correct and why the others are not:

A. Verify that the correct IP address and subnet mask are in use for the subinterface on the router. This is crucial. Direct Connect requires correct IP addressing for BGP peering to function. Incorrect IP addresses or subnet masks will prevent the establishment of the BGP session and thus, block communication with the Amazon peer IP. Misconfiguration here will prevent the VIF from coming up.

C. Verify that the router has a MAC address entry from the AWS endpoint in the Address Resolution Protocol (ARP) table. ARP resolution is a fundamental step in IP communication. Before a router can send packets to the AWS endpoint, it must know the endpoint's MAC address. If the router doesn't have the MAC address in its ARP table, it won't be able to forward traffic to the AWS peer IP. A missing ARP entry indicates a problem with connectivity at Layer 2 (data link layer).

E. Ensure that the correct VLAN tag is applied on the subinterface configuration on the router. Direct Connect uses VLANs to separate traffic. The VIF is associated with a specific VLAN. If the on-premises router isn't configured to use the correct VLAN tag on the subinterface for the Direct Connect connection, traffic won't be properly routed to the AWS side. A mismatched VLAN configuration will prevent the Direct Connect VIF from becoming active.

Now, let's examine why the incorrect options are wrong:

B. Ensure that VLAN trunking is disabled on the router. VLAN trunking is essential when dealing with multiple VLANs on the same physical interface. Disabling it would prevent the router from correctly identifying and processing traffic for the VLAN associated with the Direct Connect connection.

D. Verify that the optical signal that is received over the cross connect is optimal. While a degraded optical signal can cause connectivity issues, the ConnectionState metric being UP indicates that the physical connection is established at the optical level. If the connection was physically down, the ConnectionState would also be down. Since the problem lies with the VIF, the issue is more likely related to configuration.

F. Ensure that TCP port 179 is not being blocked at the on-premises router. BGP uses TCP port 179 for establishing peering sessions. However, if the network engineer is unable to ping the amazon peer ip address that means the VIF has not come up or is in a state that requires an ICMP packet.

Authoritative Links:

AWS Direct Connect documentation:

<https://docs.aws.amazon.com/directconnect/latest/UserGuide/Welcome.html>

Troubleshooting AWS Direct Connect: <https://aws.amazon.com/premiumsupport/knowledge-center/direct-connect-troubleshooting/>

Question: 236

CertyIQ

A logistics company has multiple VPCs in an AWS Region. The company uses a transit gateway to connect the VPCs. The company has several on-premises offices that connect to the transit gateway by using AWS Site-to-Site VPN connections over the internet. The company has configured one transit gateway VPN attachment for each office.

Route propagation is enabled on all route tables. Each Site-to-Site VPN connection uses two tunnels in an active-passive configuration. The company configured each office with appropriate static routes on both the Site-to-Site VPN connection and the office's customer gateway.

The company wants to use both IPsec tunnels of every office to maximize the overall VPN connection bandwidth.

Which design changes are necessary to meet these requirements?

A. Create an AWS Transit Gateway Connect attachment for each office. Use the existing VPN attachments as the transport for the new Connect attachments. Set up a Generic Routing Encapsulation (GRE) tunnel on each customer gateway that terminates on the Connect attachment for each office. Move the static routes from the transit gateway VPN attachment to the customer gateway for the transit gateway Connect attachment.

B. Enable equal-cost multi-path (ECMP) routing on the transit gateway. Ensure ECMP is supported by and enabled on the customer gateways. Enable ECMP on the Site-to-Site VPN connection. Ensure static routes on the customer gateways have equal metrics and administrative distance.

C. Enable equal-cost multi-path (ECMP) routing on the transit gateway. (Ensure ECMP is supported by and enabled on the customer gateways. Change the routing configuration between the transit gateway and the customer gateways from static routing to BGP. Remove related static routes from the customer gateways.

D. Enable equal-cost multi-path (ECMP) routing on the transit gateway. Ensure ECMP is supported by and enabled on the customer gateways. Change the routing configuration between the transit gateway and the customer gateways from static routing to BGP. Ensure the customer gateway applies the correct community strings to give the transit gateway the ability to perform ECMP forwarding.

Answer: D

Explanation:

The correct answer is D. Here's why:

The company wants to utilize both tunnels of each Site-to-Site VPN connection for increased bandwidth. This

requires enabling Equal-Cost Multi-Path (ECMP) routing.

Why Option D is Correct:

Enable ECMP on the Transit Gateway: This is the core requirement to allow the transit gateway to distribute traffic across multiple equal-cost paths (the VPN tunnels).

Ensure ECMP is Supported on Customer Gateways: The customer gateways (on-premises routers) must also support ECMP to handle incoming traffic that is load-balanced across the two tunnels.

Change to BGP: Static routing is not dynamic and won't automatically adapt to use both tunnels effectively. Border Gateway Protocol (BGP) is a dynamic routing protocol that can advertise multiple paths to the same destination. Using BGP, each tunnel can advertise the same routes, and the transit gateway can learn about both paths.

Community Strings: Community strings are BGP attributes that can be used to influence routing decisions. In this context, the customer gateway needs to apply specific community strings to the advertised routes so that the transit gateway understands that these paths are eligible for ECMP. These are critical for AWS to perform ECMP forwarding across VPN connections.

Why Other Options Are Incorrect:

Option A: AWS Connect Attachments are meant to be used when connecting VPCs or direct Connect peerings via AWS partners. They are not intended for Site-to-Site VPN connections. Using GRE tunneling adds overhead and complexity.

Option B: While enabling ECMP on the transit gateway and customer gateway is necessary, using static routes prevents dynamic path selection and failover. Static routes won't adapt to tunnel availability.

Option C: While switching to BGP is correct, not considering community strings is a critical oversight. Without the right community strings, the transit gateway might not perform ECMP as expected.

In Summary:

To effectively utilize both tunnels of a Site-to-Site VPN connection for increased bandwidth with a transit gateway, you need ECMP routing, a dynamic routing protocol like BGP, and the correct BGP community strings to instruct the transit gateway on how to perform ECMP.

References:

AWS Transit Gateway ECMP: <https://docs.aws.amazon.com/vpn/latest/s2svpn/VPNTunnels.html>

BGP Communities: <https://docs.aws.amazon.com/directconnect/latest/UserGuide/routing-and-bgp.html>

Question: 237

CertyIQ

A finance company runs multiple applications on Amazon EC2 instances in two VPCs that are within a single AWS Region. The company uses one VPC for stock trading applications. The company uses the second VPC for financial applications. Both VPCs are connected to a transit gateway that is configured as a multicast router.

In the stock trading VPC, an EC2 instance that has an IP address of 10.128.10.2 sends trading data over a multicast network to the 239.10.10.10 IP address on UDP Port 5102. The company recently launched two new EC2 instances in the financial application VPC. The new EC2 instances need to receive the multicast stock trading data from the EC2 instance that is in the stock trading VPC.

Which combination of steps should the company take to meet this requirement? (Choose three.)

A. Add the elastic network interfaces of the two new EC2 instances as members of the multicast group by using the group IP address of 239.10.10.10.

B. Add an inbound rule to the security groups that are attached to the multicast receiver instances. Configure the rule as follows:

Protocol: IGMP Version 2. Port: 5102, and Source: 239.10.10.10/32

C. Create associations to two EC2 instance IDs on the financial application VPC transit gateway attachment

under the transit gateway multicast domain.

D. Create an association to EC2 instance subnets on the financial application VPC transit gateway attachment under the transit gateway multicast domain.

D. Add an inbound rule to the security groups that are attached to the multicast receiver instances. Configure the rule as follows:

Protocol: IGMP Version 2. Port: All, and Source: 0.0.0.0/32

E. Add an inbound rule to the security groups that are attached to the multicast receiver instances. Configure the rule as follows.

Protocol: UDP, Port: 5102, and Source: 10.128.10.2/32

Answer: ADF

Explanation:

A. Add the elastic network interfaces of the two new EC2 instances as members of the multicast group by using the group IP address of 239.10.10.10.

To receive multicast traffic, ENIs (elastic network interfaces) must be added to the multicast group with a group IP like 239.10.10.10. This allows those instances to receive traffic sent to that multicast group.

D. Create an association to EC2 instance subnets on the financial application VPC transit gateway attachment under the transit gateway multicast domain.

Alternatively to instance ID association, subnet-level association is also valid. This enables multicast traffic in all instances within that subnet. You typically do either C or D, depending on the granularity of control.

F. Add an inbound rule to the security groups... Protocol: UDP, Port: 5102, Source: 10.128.10.2/32

Multicast traffic (application data) typically uses UDP. You need to allow UDP port 5102 from the source EC2 sender IP (e.g., 10.128.10.2) so the receiving EC2 instances can receive the multicast traffic.

Question: 238

CertyIQ

A company runs workloads in multiple VPCs in the us-east-1 Region. The VPCs are connected to a transit gateway. An AWS Direct Connect connection provides private connectivity between a data center that is in the US and the transit gateway. A Direct Connect gateway is associated with the transit gateway.

The company has recently opened a new office location in London. The company plans to launch cloud services in multiple VPCs in the eu-west-2 Region. Users in the new London office must have private access to the workloads that run in us-east-1. Users in the US data center must have access to any workloads that are created in eu-west-2. A network engineer must implement a flexible solution that provides users the required access. The solution must be able to accommodate future growth.

Which solution will meet these requirements with the LEAST operational effort?

A. Create an AWS Site-to-Site VPN connection from the London office to the Direct Connect gateway in us-east-1.

B. Establish a new Direct Connect connection for the London office. Attach the new Direct Connect connection to the existing Direct Connect gateway. Create a transit gateway in eu-west-2. Associate the new transit gateway with the existing Direct Connect gateway. Create a peering connection between the transit gateways in us-east-1 and eu-west-2.

C. Create an AWS Site-to-Site VPN connection from the London office to each of the VPCs that are in us-east-1.

D. Establish a new AWS Direct Connect connection for the London office. Create a new Direct Connect gateway and a transit gateway in eu-west-2. Attach the new Direct Connect connection to the new Direct Connect gateway. Create a peering connection between the transit gateways in us-east-1 and eu-west-2.

Answer: A

Explanation:

The provided answer (A) is incorrect. The best answer is **B**. Here's why:

Why Option B is Correct:

Option B offers a highly scalable and manageable solution by leveraging Direct Connect's global reach and Transit Gateway's hub-and-spoke architecture. Let's break it down:

1. **London Direct Connect:** Establishing a new Direct Connect connection for the London office ensures low-latency, private connectivity to AWS.
2. **Direct Connect Gateway (Existing):** Attaching the new London Direct Connect connection to the existing Direct Connect Gateway in us-east-1 leverages the existing infrastructure and minimizes configuration changes in the us-east-1 environment. This is crucial for minimizing operational effort.
3. **Transit Gateway in eu-west-2:** Creating a Transit Gateway in eu-west-2 acts as the central hub for connectivity within that region.
4. **Direct Connect Gateway Association:** Associating the eu-west-2 Transit Gateway with the existing Direct Connect Gateway is key. It provides a direct, private path from the London office (via Direct Connect), through the Direct Connect Gateway, and into the eu-west-2 Transit Gateway. This enables London users to access workloads in eu-west-2, and provides the connection that US users need to access workloads in eu-west-2.
5. **Transit Gateway Peering:** Creating a peering connection between the Transit Gateways in us-east-1 and eu-west-2 enables seamless routing between the two regions. This fulfills the requirement that US data center users can access workloads in eu-west-2.

Why Other Options are Incorrect:

A (Incorrect): Creating a Site-to-Site VPN connection from London to the Direct Connect Gateway would introduce additional complexity and potential performance bottlenecks. The Direct Connect Gateway is primarily designed to facilitate Direct Connect connections. VPN is a less performant and more operationally burdensome solution for this scenario.

C (Incorrect): Creating Site-to-Site VPN connections from London to each VPC in us-east-1 is not scalable and would be difficult to manage as the number of VPCs grows. It violates the "least operational effort" requirement.

D (Incorrect): While functionally similar to option B, option D creates a new Direct Connect Gateway. This creates unnecessary redundancy and operational overhead. The existing Direct Connect Gateway is already in place and designed for multi-region connectivity. Creating a new one adds cost and complexity without adding value.

Key Concepts and Advantages:

AWS Direct Connect: Provides dedicated network connections from on-premises locations to AWS, offering lower latency, increased bandwidth, and a more consistent network experience than internet-based connections.

AWS Transit Gateway: A network transit hub that simplifies network connectivity between VPCs and on-premises networks. It avoids complex peering relationships and provides a central point for managing network traffic.

AWS Direct Connect Gateway: Enables you to access any AWS Region from your on-premises network using a single Direct Connect connection.

Scalability and Flexibility: Transit Gateway allows for easy expansion as the number of VPCs and regions grows.

Authoritative Links:

[AWS Direct Connect](#)

[AWS Transit Gateway](#)

[AWS Direct Connect Gateway](#)

In summary, Option B offers the most scalable, manageable, and cost-effective solution for connecting the London office to the existing AWS environment while allowing traffic between the US and London regions with minimal operational effort.

Question: 239

CertyIQ

A company has 10 Amazon EC2 instances that run web server software in a production VPC. The company also has 10 web servers that run in an on-premises data center. The company has a 10 Gbps AWS Direct Connect connection between the on-premises data center and the production VPC. The data center uses the 10.100.0.0/20 CIDR block.

The company needs to implement a load balancing solution that receives HTTPS traffic from thousands of external users. The solution must distribute the traffic across the web servers on AWS and the web servers in the data center. Regardless of the location of the web servers, HTTPS requests must go to the same web server for the duration of the session.

Which solution will meet these requirements?

- A. Deploy a Network Load Balancer (NLB) in the production VPC. Create one target group for the EC2 Instances and a second target group for the on-premises servers. Specify IP as the target type. Register the EC2 instances and the on-premises servers with the target groups. Enable connection draining on the NLB.
- B. Deploy an Application Load Balancer (ALB) in the production VPC. Create one target group for the EC2 Instances and a second target group for the on-premises servers. Specify IP as the target type. Register the EC2 instances and the on-premises servers with the target groups. Enable application-based sticky sessions on the ALB.
- C. Deploy a Network Load Balancer (NLB) in the production VPC. Create one target group for the EC2 Instances and a second target group for the on-premises servers. Specify instance as the target type. Register the EC2 instances and the on-premises servers with the target groups. Enable sticky sessions on the NLB.
- D. Deploy an Application Load Balancer (ALB) in the production VPC. Create one target group for the EC2 Instances and a second target group for the on-premises servers. Specify instance as the target type. Register the EC2 instances and the on-premises servers with the target groups. Enable application-based sticky sessions on the ALB.

Answer: B

Explanation:

Here's a detailed justification for why option B is the correct solution:

The requirements include distributing HTTPS traffic across both EC2 instances in AWS and on-premises servers, while maintaining session stickiness (requests for the same session must go to the same server).

Application Load Balancer (ALB) support for HTTPS and Stickiness: ALBs operate at Layer 7 (the application layer) of the OSI model, enabling them to handle HTTPS traffic natively. They also support application-based sticky sessions (also called "cookie stickiness"), which use cookies to ensure that requests from the same client are consistently routed to the same target.

IP Address Target Support: The solution needs to include on-premises servers as targets. Both ALBs and NLBs can target instances via their IP addresses, allowing them to route traffic to servers outside of the AWS

environment, which is necessary for integrating the on-premises web servers. The question specifies a Direct Connect connection, which is a prerequisite for accessing on-premises resources from AWS.

Why not Instance ID as Target? While using instance IDs as targets is possible for EC2 instances within the VPC, it's not possible for the on-premises servers, as they are not EC2 instances and don't have AWS instance IDs.

Why not Network Load Balancer (NLB)? NLBs operate at Layer 4 (the transport layer) and are primarily designed for high-performance TCP, UDP, and TLS traffic forwarding. While NLBs support target groups with IP addresses, they do not support application-based sticky sessions like the ALB. NLBs use client IP address to maintain stickiness. This method is less precise and can be less reliable than cookie-based stickiness. It is not appropriate for the need of session persistence.

Connection draining vs. Sticky Sessions: Connection draining allows an existing request to complete even after an instance is deregistered or becomes unhealthy. While useful, it doesn't guarantee session stickiness. The correct solution uses sticky sessions to ensure that a user's requests are always routed to the same server.

In summary, option B correctly utilizes an ALB to handle HTTPS traffic, target both EC2 instances and on-premises servers through IP addresses, and implement application-based sticky sessions to meet the stated requirements.

Here are some relevant links for further research:

[Application Load Balancers](#)

[Target Groups for Your Application Load Balancers](#)

[Sticky Sessions for Your Application Load Balancers](#)

[Network Load Balancers](#)

Question: 240

CertyIQ

A global company is establishing network connections between the company's primary and secondary data centers and a VPC. A network engineer needs to maximize resiliency and fault tolerance for the connections. The network bandwidth must be greater than 10 Gbps.

Which solution will meet these requirements MOST cost-effectively?

A. Set up a 100 Gbps connection at the primary data center that terminates at an AWS Direct Connect location. Set up a second 100 Gbps connection at the secondary data center that terminates at a second Direct Connect location. Ensure the connections are managed by separate providers.

B. Set up a 10 Gbps connection at the primary data center that terminates at an AWS Direct Connect location. Set up a second 10 Gbps connection at the secondary data center that terminates at a second Direct Connect location. Ensure the connections are managed by separate providers.

C. Set up two 10 Gbps connections at the primary data center that terminate at one AWS Direct Connect location. Ensure the connections are managed by separate providers. Set up two 10 Gbps connections at the secondary data center that terminate at a second Direct Connect location. Ensure the connections are managed by separate providers.

D. Set up a 10 Gbps connection at the primary data center that terminates at an AWS Direct Connect location. Set up an AWS Site-to-Site VPN connection at the secondary data center that terminates at a virtual private gateway in the same Region as the company's VPC.

Answer: C

Explanation:

The question emphasizes both maximizing resiliency/fault tolerance and cost-effectiveness with a bandwidth requirement exceeding 10 Gbps. Option A is immediately disqualified due to its high cost associated with 100

Gbps Direct Connect connections. Option B is also not ideal because it only meets the 10Gbps bandwidth requirement.

Option C provides redundancy by using two 10 Gbps connections at each data center through separate providers, aggregating to the required bandwidth of more than 10 Gbps. This design maximizes fault tolerance since the failure of one connection doesn't bring down the entire link. Different providers ensure diverse routing and lessen the likelihood of a single point of failure affecting both links.

Option D combines a Direct Connect link with a Site-to-Site VPN. While providing some redundancy, a VPN typically has lower and less predictable performance than a Direct Connect connection. Furthermore, it's not the most cost-effective way to achieve high availability compared to Option C. Option C meets the bandwidth and resiliency needs by using two connections from each location for higher resiliency, which are less expensive than increasing the bandwidth of each individual connection.

Therefore, Option C delivers the required bandwidth, maximizing fault tolerance at a comparatively lower cost than Option A while offering more reliable performance than Option D. Using separate providers adds another layer of

redundancy.<https://aws.amazon.com/directconnect>/<https://docs.aws.amazon.com/directconnect/latest/UserGuide/>

Question: 241

CertyIQ

A company's data center is connected to a single AWS Region by an AWS Direct Connect dedicated connection. The company has a single VPC in the Region. The company stores logs for all its applications locally in the data center.

The company must keep all application logs for 7 years. The company decides to copy all application logs to an Amazon S3 bucket.

Which solution will meet these requirements?

- A. Create a public VIF on the Direct Connect connection. Create an Amazon S3 gateway endpoint in the VPC.
- B. Create a private VIF on the Direct Connect connection. Create an Amazon S3 gateway endpoint in the VPC.
- C. Create a private VIF on the Direct Connect connection. Create an Amazon S3 interface endpoint in the VPC.
- D. Create a public VIF on the Direct Connect connection. Create an Amazon S3 interface endpoint in the VPC.

Answer: C

Explanation:

The correct answer is **C. Create a private VIF on the Direct Connect connection. Create an Amazon S3 interface endpoint in the VPC.**

Here's why:

Private VIF: The company needs a private connection for secure access to AWS services from its data center. A private VIF allows communication with AWS resources using private IP addresses, ensuring that the traffic doesn't traverse the public internet. This aligns with security best practices for sensitive data like application logs.

S3 Interface Endpoint: An S3 interface endpoint creates a private connection between the VPC and S3 using AWS PrivateLink. This ensures that traffic to S3 remains within the AWS network and does not expose the data to the internet. It provides a secure and direct path for the application logs to be copied to the S3 bucket.

Let's analyze why the other options are incorrect:

A & D (Public VIF): Using a public VIF exposes the S3 traffic to the public internet, which is undesirable for

sensitive application logs. This violates security best practices.

A & B (S3 Gateway Endpoint): While an S3 gateway endpoint is suitable for connecting to S3 from within a VPC, it only supports access from resources inside the VPC. It doesn't facilitate the secure transfer of data from the on-premises data center through Direct Connect, as required in this scenario. This is because S3 gateway endpoints use a prefix list to route traffic to S3, only working for traffic originating from the VPC itself.

In summary, the combination of a private VIF and an S3 interface endpoint provides a secure and private connection for transferring application logs from the on-premises data center to the Amazon S3 bucket, meeting the company's requirements.

Supporting Links:

AWS Direct Connect Virtual Interfaces:

<https://docs.aws.amazon.com/directconnect/latest/UserGuide/WorkingWithVirtualInterfaces.html>

AWS PrivateLink: <https://aws.amazon.com/privatelink/>

VPC Endpoints: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html>

Question: 242

CertyIQ

A company is planning to host a secure web application across multiple Amazon EC2 instances. The application will have an associated DNS domain in an Amazon Route 53 hosted zone.

The company wants to protect the domain from DNS poisoning attacks. The company also wants to allow web browsers to authenticate into the application by using a trusted third party.

Which combination of actions will meet these requirements?

- A. Configure the Route 53 hosted zone to use DNS Security Extensions (DNSSEC). Install self-signed X.509 certificates on the EC2 instances.
- B. Configure a Name Authority Pointer (NAPTR) record in the Route 53 hosted zone. Install X.509 certificates that are signed by a public certificate authority on the EC2 instances.
- C. Configure the Route 53 hosted zone to use DNS Security Extensions (DNSSEC). Install X.509 certificates that are signed by a public certificate authority on the EC2 instances.
- D. Configure a Name Authority Pointer (NAPTR) record in the Route 53 hosted zone. Install self-signed X.509 certificates on the EC2 instances.

Answer: C

Explanation:

The requirement to protect the domain from DNS poisoning attacks necessitates the implementation of DNS Security Extensions (DNSSEC). DNSSEC adds cryptographic signatures to DNS records, allowing resolvers to verify the authenticity of the DNS data and ensure it hasn't been tampered with during transit. This effectively mitigates DNS spoofing and cache poisoning attacks by validating the source and integrity of DNS responses. Route 53 supports DNSSEC to protect the hosted zone.

The need for web browsers to authenticate via a trusted third party points to the use of X.509 certificates issued by a public Certificate Authority (CA). These certificates are trusted by most browsers out-of-the-box, facilitating secure HTTPS connections and allowing the application to verify the identity of the client. Self-signed certificates, while functional, are not inherently trusted by browsers and would require manual configuration on the client side, which contradicts the requirement of using a trusted third party. Installing certificates signed by a public CA on the EC2 instances allows secure communication via HTTPS and enables the application to verify the client's identity during authentication.

Option A fails because self-signed certificates don't fulfil the trusted third party authentication requirement. Option B is incorrect because Name Authority Pointer (NAPTR) records serve a different purpose, typically used for ENUM services and not for DNS security or web application authentication. Option D also uses NAPTR records inappropriately and utilizes self-signed certificates which won't be trusted. Thus, only option C correctly utilizes DNSSEC for DNS security and publicly trusted X.509 certificates for secure authentication.

Supporting links:

Route 53 DNSSEC: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/dns-configuring-dnssec.html>

Public vs. Private Certificates: <https://www.digicert.com/blog/csr-generation-and-ssl-certificate-installation>

DNSSEC Overview: <https://www.cloudflare.com/learning/dns/what-is-dnssec/>

Question: 243

CertyIQ

A company is planning to use an AWS Transit Gateway hub and spoke architecture to migrate to AWS. The current on-premises multi-protocol label switching (MPLS) network has strict controls that enforce network segmentation by using MPLS VPNs. The company has provisioned two 10 Gbps AWS Direct Connect connections to provide resilient, high-speed, low-latency connectivity to AWS.

A security engineer needs to apply the concept of network segmentation to the AWS environment to ensure that virtual routing and forwarding (VRF) is logically separated for each of the company's software development environments. The number of MPLS VPNs will increase in the future. On-premises MPLS VPNs will have overlapping address space. The company's AWS network design must support overlapping address space for the VPNs.

Which solution will meet these requirements with the LEAST operational overhead?

- A. Deploy a software-defined WAN (SD-WAN) head-end virtual appliance and an SD-WAN controller into a Transit Gateway Connect VPC. Configure the company's edge routers to be managed by the new SD-WAN controller and to use SD-WAN to segment the traffic into the defined segments for each of the company's development environments.
- B. Configure IPsec VPNs on the company edge routers for each MPLS VPN for each of the company's development environments. Attach each IPsec VPN tunnel to a discrete MPLS VPN. Configure AWS Site-to-Site VPN connections that terminate at a transit gateway for each MPLS VPN. Configure a transit gateway route table that matches the MPLS VPN for each Transit Gateway VPN attachment.
- C. Create a transit VPC that terminates at the AWS Site-to-Site VRF-aware IPsec VPN. Configure IPsec VPN connections to each VPC for each of the company's development environment VRFs.
- D. Configure a Transit Gateway Connect attachment for each MPLS VPN between the company's edge routers and Transit Gateway. Configure a transit gateway route table that matches the MPLS VPN for each of the company's development environments.

Answer: D

Explanation:

The best solution to meet the requirements of network segmentation using AWS Transit Gateway, overlapping address space, and minimal operational overhead is option D: Configure a Transit Gateway Connect attachment for each MPLS VPN between the company's edge routers and Transit Gateway. Configure a transit gateway route table that matches the MPLS VPN for each of the company's development environments.

Here's why:

Transit Gateway Connect: Transit Gateway Connect attachments directly integrate with SD-WAN appliances or compatible routers using the Generic Routing Encapsulation (GRE) and Border Gateway Protocol (BGP).

This allows for a clean and efficient way to extend the existing on-premises MPLS VPN segmentation into the AWS cloud.

MPLS VPN Mapping: By creating a Connect attachment for each MPLS VPN, the network segmentation is preserved. Each attachment effectively acts as a dedicated VRF within the Transit Gateway.

Route Tables for Segmentation: Transit Gateway route tables allow you to control traffic flow between attachments. By matching the route table to the MPLS VPN, you ensure that traffic from a specific on-premises MPLS VPN is routed only to the corresponding resources in AWS.

Overlapping Address Space Support: Transit Gateway inherently supports overlapping address spaces through the use of unique attachments and route tables. Each attachment effectively creates an independent routing domain.

Least Operational Overhead: This approach leverages the Transit Gateway's native capabilities for routing and segmentation, minimizing the need for complex configurations or additional virtual appliances. It directly extends the existing MPLS VPN structure to the cloud.

Scalability: The number of MPLS VPNs can increase in the future as it is relatively simple to add a new Transit Gateway Connect attachment for each new VPN.

Direct Connect optimization: Taking advantage of the existing 10 Gbps Direct Connect connections provides high bandwidth and low latency connectivity to AWS.

Why other options are less suitable:

A (SD-WAN Head-end): Deploying an SD-WAN head-end adds complexity and operational overhead. While SD-WAN can achieve segmentation, it introduces another layer of management.

B (IPsec VPNs): Configuring multiple IPsec VPNs can be complex to manage, especially with a large number of MPLS VPNs. Also, AWS Site-to-Site VPN is not VRF-aware.

C (Transit VPC with VRF-aware VPN): AWS Site-to-Site VPN doesn't inherently support VRF. Implementing a custom VRF-aware solution would add significant complexity and overhead.

Authoritative Links:

[AWS Transit Gateway Connect](#)

[AWS Transit Gateway Routing](#)

Question: 244

CertyIQ

A company is planning to migrate to AWS and use multiple VPCs in multiple AWS Regions. A network engineer must connect the eu-west-1 and eu-central-1 Regions to the company headquarters and branch office, respectively.

The network engineer created a production VPC, named Prod A, with a CIDR block of 10.0.0.0/16. Prod A runs in an account in eu-west-1. The network engineer then created another production VPC, named Prod B, with a CIDR block of 10.1.0.0/16. Prod B runs in a different account in eu-central-1.

The network engineer performed the following steps to try to achieve the required connectivity:

1. Created one transit gateway in each Region
2. Shared and accepted the transit gateways with the production accounts in both Regions
3. Configured the peering attachment between both transit gateways
4. Attached both VPCs to the respective Region transit gateway
5. Created both transit gateway route tables and associated the attachments with the route tables
6. Configured a static route in both transit gateway route tables to send traffic to the remote VPC in the other Region
7. Activated route propagation on the VPC route tables in each Region

After the configuration, the network engineer tried to connect from Prod A to Prod B. However, the connection was unsuccessful.

What should the network engineer do to achieve the required connectivity?

A. Modify the IP address of the peering attachment to a wider range.

B.Delete the static routes that were in the transit gateway route table to send traffic to the remote VPC and enable route propagation instead.

C.Create a new route destined to 10.0.0.0/8 in both production VPC route tables with the Region transit gateway as the target.

D.Modify the transit gateway route tables from the production accounts to propagate routes dynamically between the production VPCs.

Answer: C

Explanation:

Here's a detailed justification for why option C is the correct solution, along with supporting concepts and links:

The core issue is that while the transit gateways and VPC attachments are correctly configured for transit, the VPC route tables themselves lack awareness of the broader IP address range encompassing both VPCs. Because the VPCs reside within the 10.0.0.0/8 range, and only know about their immediate CIDR blocks (10.0.0.0/16 and 10.1.0.0/16), traffic destined for the other VPC doesn't know where to go within the VPC.

Here's a breakdown of why option C fixes this and why other options are incorrect:

Option C: Create a new route destined to 10.0.0.0/8 in both production VPC route tables with the Region transit gateway as the target. This is the correct solution. By adding a route to 10.0.0.0/8 in each VPC route table, with the transit gateway as the target, we instruct the VPCs to send any traffic destined for any IP address within the 10.0.0.0/8 range to the transit gateway. The transit gateway then uses its route table to forward the traffic to the appropriate destination VPC (Prod A or Prod B). This ensures that the VPCs can "see" the broader address space needed for inter-VPC communication. This is an implementation of the "least specific route wins" principle in IP routing; the VPCs would have to forward packets for addresses within their CIDRs before the larger range.

Option A: Modify the IP address of the peering attachment to a wider range. Transit gateway peering attachments do not have configurable IP addresses; thus, this option is wrong. The configuration involved in a peering attachment includes ASN and VPC CIDR ranges. (<https://docs.aws.amazon.com/vpc/latest/tgw/tgw-peering.html>)

Option B: Delete the static routes that were in the transit gateway route table to send traffic to the remote VPC and enable route propagation instead. The initial setup using static routes in the transit gateway route tables is correct. Static routes are necessary in the transit gateway to direct traffic between regions. Enabling route propagation doesn't automatically solve the problem of the VPCs not knowing about the broader 10.0.0.0/8 range. While propagation is useful for sharing routes within a region, it won't address the fundamental routing gap in the VPC route tables.

Option D: Modify the transit gateway route tables from the production accounts to propagate routes dynamically between the production VPCs. Route propagation within the transit gateway route tables, while useful, doesn't directly address the need for the VPCs themselves to be aware of the broader 10.0.0.0/8 network range. The primary issue is that the VPCs need to know that anything outside their direct /16 CIDR destined for the wider 10.0.0.0/8 range should be sent to the transit gateway. This requires a specific route within the VPC route table.

Key Concepts:

Transit Gateway: Acts as a central hub for connecting multiple VPCs and on-premises networks. (<https://aws.amazon.com/transit-gateway/>)

VPC Route Tables: Control the routing of traffic within a VPC.

Route Propagation: Automatically propagates routes from attached resources (like VPCs) to the transit gateway route table.

Static Routes: Manually configured routes in a route table.

CIDR Block: A range of IP addresses.

Inter-Region Connectivity: Connecting networks across different AWS Regions.

In summary, option C correctly identifies the root cause of the connectivity issue (lack of a route in the VPC route tables) and provides the appropriate solution (adding a route to 10.0.0.0/8 pointing to the transit gateway). This enables the VPCs to forward traffic destined for other VPCs within the 10.0.0.0/8 range to the transit gateway for proper routing.

Question: 245

CertyIQ

A company hosts an application on Amazon EC2 instances behind an Application Load Balancer (ALB). The instances are part of an Amazon EC2 Auto Scaling group.

To comply with new security standards, the company must capture all application access data, including server response codes, request paths, latency, and client IP addresses. The company also needs to query the captured data for performance analysis.

Which solution will meet these requirements?

A.Enable VPC flow logs on the ALB subnets. Store the logs to an Amazon S3 bucket. Query the logs in the S3 bucket by using Amazon Athena.

B.Configure Amazon VPC Traffic Mirroring on all EC2 elastic network interfaces. Deploy a third-party monitoring appliance from AWS Marketplace in a private subnet. Use Amazon Data Firehose to send all mirrored traffic to the monitoring appliance. Query the logs directly from the monitoring appliance.

C.Configure Amazon CloudWatch detailed monitoring on the EC2 instances Include all available logs. Use Amazon Data Firehose to send all the collected logs to an Amazon S3 bucket. Query the data directly from the S3 bucket.

D.Enable access logs on the ALB. Store the logs in an Amazon S3 bucket. Query the logs in the S3 bucket by using Amazon Athena.

Answer: D

Explanation:

The correct answer is **D. Enable access logs on the ALB. Store the logs in an Amazon S3 bucket. Query the logs in the S3 bucket by using Amazon Athena.**

Here's why:

ALB Access Logs: Application Load Balancers (ALBs) natively provide access logs, which capture detailed information about requests handled by the load balancer. This includes request paths, client IP addresses, server response codes, and latency, exactly what the company needs to capture.

<https://docs.aws.amazon.com/elasticloadbalancing/latest/application/access-log-collection.html>

S3 for Storage: Amazon S3 is a cost-effective and scalable storage solution for log data. Storing the ALB access logs in an S3 bucket allows for long-term retention and efficient querying.

Athena for Querying: Amazon Athena is a serverless, interactive query service that makes it easy to analyze data in S3 using standard SQL. Athena can directly query the ALB access logs in the S3 bucket, enabling the company to perform performance analysis. <https://aws.amazon.com/athena/>

Why other options are incorrect:

A. VPC Flow Logs: VPC Flow Logs capture network traffic information at the VPC level. While they can provide insights into network traffic, they don't contain application-level details like request paths or server

response codes, which are crucial for the company's requirements. They also can't provide request latency information in the same way ALB access logs do.

B. VPC Traffic Mirroring: Traffic Mirroring duplicates network traffic for analysis, but it requires deploying and managing a third-party monitoring appliance. This is more complex and expensive than using ALB access logs, which are readily available. Using Data Firehose to send data to the appliance also adds unnecessary complexity.

C. CloudWatch Detailed Monitoring: CloudWatch detailed monitoring focuses on the EC2 instance metrics, not the application access data processed by the ALB. Collecting and analyzing EC2 instance logs won't provide the required information about request paths, client IP addresses, and ALB-specific latency. While CloudWatch logs can be streamed to S3, they are for instance level logs, not the required load balancer logs.

Question: 246

CertyIQ

A company has five VPCs in the us-east-1 Region. The company hosts an internal web application in us-east-1. One of the company's VPCs, named VPC-A, needs to connect to an external partner's AWS environment. The partner's environment is in the same AWS Region where the partner hosts a new version of the company's web application. The partner hosts its version of the application in a VPC named VPC-B.

The company has Amazon EC2 instances in VPC-A that need to connect to the web application in VPC-B. A network engineer notices that the partner's VPC-B and the company's VPC-A use the same IP space. The network engineer needs a solution to allow the EC2 instances to connect to the web application. The solution must not negatively affect the existing environment of the company or the partner.

Which combination of steps should the network engineer take to meet these requirements? (Choose two.)

- A. Establish a VPC peering connection between VPC-A and VPC-B.
- B. Ensure the partner creates a VPC endpoint service that uses a Network Load Balancer in VPC-B.
- C. Deploy a VPC endpoint in VPC-A that uses a VPC endpoint service that is shared by the partner.
- D. Deploy a new routable VPC CIDR block as a secondary CIDR block to both VPC-A and VPC-B. Deploy a public NAT gateway in VPC-A.
- E. Establish an AWS Site-to-Site VPN connection between VPC-A and VPC-B.

Answer: BC

Explanation:

Here's a detailed justification for why options B and C are the correct choices, along with supporting concepts and links.

The core problem is overlapping CIDR blocks (same IP address ranges) between VPC-A and VPC-B, which prevents direct connection methods like VPC peering. VPC peering requires non-overlapping CIDR blocks. We need a solution that allows communication while avoiding conflicts caused by the shared IP space, and also avoid impact the existing environments of company and partner.

Option B: Ensure the partner creates a VPC endpoint service that uses a Network Load Balancer in VPC-B.

This step is crucial. A VPC endpoint service, backed by a Network Load Balancer (NLB) within VPC-B, acts as a gateway for private connectivity. The NLB sits in front of the web application instances. Crucially, the service consumers (in VPC-A) connect to the service name not directly to the instances in VPC-B's CIDR block. The NLB handles the translation. NLB offers static IP and support TCP and

UDP. <https://docs.aws.amazon.com/vpc/latest/privatelink/privatelink-share-your-services.html>

Option C: Deploy a VPC endpoint in VPC-A that uses a VPC endpoint service that is shared by the partner.

This is the other half of the solution. By creating a VPC endpoint (specifically, an Interface VPC endpoint) in VPC-A, the company's EC2 instances can connect to the partner's VPC endpoint service. The VPC endpoint acts as an entry point within VPC-A, allowing traffic to be routed privately to the partner's NLB without needing to peer the VPCs directly or deal with IP address conflicts. Traffic will never leave aws backbone network. The EC2 instances connect to the web application through the VPC endpoint in their own VPC-A, resolving the DNS name of VPC endpoint.<https://docs.aws.amazon.com/vpc/latest/privatelink/vpc-endpoints.html>

Why other options are incorrect:

A: Establish a VPC peering connection between VPC-A to VPC-B. This is incorrect because VPC Peering requires non-overlapping CIDR blocks, which is the core problem identified in the question.

D: Deploy a new routable VPC CIDR block as a secondary CIDR block to both VPC-A and VPC-B. Deploy a public NAT gateway in VPC-A. While adding secondary CIDRs is possible, it's a significant change that requires planning and potential reconfiguration of existing resources. A NAT gateway won't resolve the CIDR overlap issue directly. Adding secondary CIDR blocks require changes in the partner environment, which the question states should be avoided.

E: Establish an AWS Site-to-Site VPN connection between VPC-A and VPC-B. VPNs can be used with overlapping CIDR blocks using NAT. However, it's a more complex solution compared to VPC endpoints and introduces more latency. Also, to establish Site-to-Site VPN you still need to solve overlapping IP addresses by using network address translation (NAT).

In summary, the combination of options B and C provides the most elegant and least disruptive solution for private connectivity between VPCs with overlapping CIDR blocks, aligning with the constraints outlined in the question.

Question: 247

CertyIQ

A company has a hybrid environment that connects an on-premises data center to the AWS Cloud. The hybrid environment uses a 10 Gbps AWS Direct Connect dedicated connection. The Direct Connect connection has multiple private VIFs that terminate in multiple VPCs.

To comply with regulations, the company must encrypt all WAN traffic, regardless of the underlying transport. The company needs to implement an encryption solution that will not affect the company's bandwidth capacity.

Which solution will meet these requirements?

- A. Create a public VIF. Configure a new AWS Site-to-Site VPN connection to use the new public VIF.
- B. Configure MAC security (MACsec) support on the port of the existing Direct Connect connection. Change the encryption mode to `must_encrypt`.
- C. Configure a new Direct Connect connection that supports MAC security (MACSec) Associate the existing VIFs to the new Direct Connect connection.
- D. Create a public VIF. Configure a new private IP VPN that uses the Direct Connect connection.

Answer: B

Explanation:

The requirement is to encrypt all WAN traffic over Direct Connect without impacting the 10 Gbps bandwidth capacity. Let's analyze why option B is the most suitable solution.

Option B proposes using MAC Security (MACsec) on the existing Direct Connect connection. MACsec is a Layer 2 encryption protocol that encrypts all traffic between two directly connected nodes. Since it operates at Layer 2, it encrypts the traffic before it reaches the IP layer, ensuring all traffic is encrypted regardless of its destination (public or private VIFs). Most importantly, MACsec implementations in hardware offer near-

line-rate encryption, meaning minimal performance overhead. Setting the encryption mode to `must_encrypt` ensures that encryption is enforced. Because it works on the physical layer, it's transparent to the upper layers of the OSI model and has virtually no impact on the bandwidth as the encryption/decryption is handled by dedicated hardware.

Option A involves creating a public VIF and Site-to-Site VPN. This solution would introduce significant overhead due to IPsec encryption, reducing available bandwidth. Furthermore, it would require changes to routing and VPN configuration, adding complexity. Public VIFs are usually used for connecting to AWS public services, not typically for encrypted connections between on-premise and VPCs.

Option C involves creating a new Direct Connect connection with MACsec. This option is less efficient than utilizing the existing Direct Connect connection. It would be costly as it would mean paying for a whole new Direct Connect connection and would cause downtime as services are migrated to the new connection.

Option D, which involves creating a public VIF and a private IP VPN, suffers from the same bandwidth limitations as Option A due to the IPsec overhead and isn't designed for encrypting all traffic, only traffic destined for the VPN endpoint.

Therefore, configuring MACsec on the existing Direct Connect port, setting to `must_encrypt` is the most efficient way to achieve end-to-end encryption with minimal impact on bandwidth.

Authoritative Links:

AWS Direct Connect MACsec: <https://aws.amazon.com/directconnect/features/macsec/>
MACsec Standard (IEEE 802.1AE): https://standards.ieee.org/standard/802_1AE-2018.html

Question: 248

CertyIQ

A company needs to capture and log traffic for Nitro-based Amazon EC2 instances to comply with regulations. The company's network team has prepared a solution that enables VPC traffic mirroring and sends traffic to a second set of EC2 instances in an Auto Scaling group.

The network team has added a Network Load Balancer (NLB) in front of the EC2 instances the traffic will be sent to. However, the solution does not send any mirrored traffic to the EC2 instances that are behind the NLB.

How should the network team configure traffic mirroring to use the NLB endpoint?

- A. Select the NLB as a source for traffic mirroring. Use a UDP listener.
- B. Select the NLB as a target for traffic mirroring. Use a TCP listener and a UDP listener.
- C. Select the NLB as a target for traffic mirroring. Use a TCP listener.
- D. Select the NLB as a target for traffic mirroring. Use a UDP listener.

Answer: D

Explanation:

Here's a detailed justification for why option D is the correct answer:

The core problem is that mirrored traffic isn't reaching the EC2 instances behind the Network Load Balancer (NLB). Traffic mirroring works by duplicating network packets from a source (e.g., an EC2 instance) and sending them to a target. The target in this case is intended to be the NLB, which then distributes the traffic to the EC2 instances performing the traffic analysis.

NLBs are designed to handle high volumes of traffic with low latency. They operate at the transport layer (Layer 4) of the OSI model, primarily dealing with TCP and UDP protocols. Traffic mirroring, by its nature, involves duplicating the entire packet, including headers and payloads, regardless of the application-layer

protocol.

NLBs support UDP listeners for handling UDP traffic, which is connectionless and often used for streaming or broadcasting data. Because mirrored traffic isn't necessarily tied to an established TCP connection and can include a variety of protocols, a UDP listener on the NLB is the most appropriate choice to receive the mirrored packets. TCP connections can be disrupted or terminated if the mirrored traffic does not form a valid TCP connection.

Options A, B, and C are incorrect. An NLB cannot be selected as a source. It is a target. Using only a TCP listener (option C) or using both TCP and UDP listeners (option B) may cause the mirrored traffic to be dropped if it is not properly formatted TCP traffic, or if the client TCP connection is not initiated correctly. Selecting the NLB as a target for traffic mirroring with UDP listener (option D) will work because mirroring traffic can include packets that are not part of an established TCP connection.

Therefore, the network team should configure traffic mirroring to select the NLB as a target and configure the NLB with a UDP listener. This setup allows the NLB to receive the mirrored packets and forward them to the backend EC2 instances for analysis.

Relevant Documentation:

AWS Traffic Mirroring: <https://docs.aws.amazon.com/vpc/latest/mirroring/what-is-traffic-mirroring.html>

Network Load Balancers:

<https://docs.aws.amazon.com/elasticloadbalancing/latest/network/introduction.html>

Question: 249

CertyIQ

A US-based company is expanding its business to Europe. A network engineer needs to extend the company's network infrastructure by setting up a new hub and spoke architecture in the eu-west-1 Region. The network engineer uses a transit gateway peering connection to connect the new resources in eu-west-1 to an existing environment in the us-east-1 Region.

The hub and spoke architecture in each AWS Region includes an inspection VPC that uses AWS Network Firewall to centralize traffic inspection for each Region. To reduce costs, the network engineer decides to inspect inter-Region traffic by using the inspection VPC in the Region that originates the traffic. The network engineer configures the transit gateway route tables accordingly for each Region.

When the network engineer tests the new architecture, communication within each Region works as expected. However, the network engineer finds that inter-Region communication is not working. The network engineer must resolve the inter-Region communication issue.

Which solution will meet this requirement?

- A. Configure Open Shortest Path First (OSPF) routing on the transit gateway peering connection to propagate the VPC CIDR blocks from each Region to the remote peer.
- B. Use AWS Resource Access Manager (AWS RAM) to share access between the transit gateways. Enable the Allow sharing with anyone setting.
- C. Prevent asymmetric routing in the inspection VPCs by ensuring that both requests and responses are inspected by the same inspection VPC
- D. Enable Appliance mode on both the transit gateway attachments for the inspection VPC.

Answer: D

Explanation:

The correct answer is **D. Enable Appliance mode on both the transit gateway attachments for the inspection VPC.**

Here's why:

The core issue is asymmetric routing, preventing return traffic from reaching the source. The engineer intends to inspect traffic in the originating region's inspection VPC. Without Appliance mode, the transit gateway might route return traffic through the destination region's inspection VPC, breaking the flow because the firewall in the destination region hasn't seen the initial request and will likely block the return traffic.

Appliance mode ensures traffic destined for the inspection VPC (and effectively the Network Firewall) from a Transit Gateway attachment is always routed back through the same attachment, regardless of other routing configurations. This guarantees that both the request and response for inter-Region traffic are inspected by the firewall in the originating region's inspection VPC, fulfilling the design requirement. It prevents the asymmetric routing problem that is causing the connectivity failure.

Options A, B, and C are incorrect:

A. Configure Open Shortest Path First (OSPF) routing on the transit gateway peering connection: Transit Gateways do not support OSPF directly. Route propagation is handled through static routes and dynamic route propagation from attached VPCs or VPNs. Even if it were possible, OSPF wouldn't solve the asymmetric routing issue.

B. Use AWS Resource Access Manager (AWS RAM) to share access between the transit gateways. Enable the Allow sharing with anyone setting: AWS RAM is for sharing resources between accounts. While it could be used if the Transit Gateways were in different accounts, it doesn't directly address the routing problem within the single architecture described. "Allow sharing with anyone" is generally a bad practice for security reasons. The issue isn't access between the TGWs, it's routing.

C. Prevent asymmetric routing in the inspection VPCs by ensuring that both requests and responses are inspected by the same inspection VPC: This is the goal, but the problem statement shows that the engineer failed to meet the goal. Appliance mode is the mechanism for achieving that goal when using Network Firewall with Transit Gateways. While a valid design principle, it doesn't propose a concrete solution to the stated problem.

Authoritative Links:

AWS Transit Gateway Appliance Mode: <https://docs.aws.amazon.com/vpc/latest/tgw/tgw-appliance-mode.html>

AWS Network Firewall Routing Considerations: <https://docs.aws.amazon.com/network-firewall/latest/developerguide/route-tables.html>

Question: 250

CertyIQ

A company runs applications in two VPCs that are in separate AWS Regions. One VPC is in the us-east-1 Region. The second VPC is in the us-west-1 Region. The company needs to establish connectivity between the two VPCs. The company also needs to connect the VPCs to applications that run in an on-premises data center.

The current traffic requirement between the VPCs is 50 TB per month. The company expects traffic volume between the VPCs to increase. The traffic requirement from the VPCs to the on-premises data center is 10 TB per month. The company expects the traffic between the VPCs and the data center to remain constant.

Which solution will meet these requirements MOST cost-effectively?

- A. Create a transit gateway in each Region. Create VPN connections from the transit gateways to the on-premises firewall. Create a peering connection between the transit gateways.
- B. Create a virtual private gateway in each Region. Create VPN connections from the on-premises firewall to the virtual private gateways. Configure the on-premises firewall to route the traffic between the two VPCs.
- C. Create a virtual private gateway in each Region. Create VPN connections from the on-premises firewall to the virtual private gateways. Create a VPC peering connection between the two VPCs.
- D. Create a virtual private gateway in each Region. Create VPN connections from the on-premises firewall to the

virtual private gateways. Create a VPN connection between the virtual private gateways.

Answer: C

Explanation:

Option C is the most cost-effective solution because it leverages VPC peering for inter-region VPC communication. VPC peering is significantly cheaper for high-volume data transfer than Transit Gateway, which charges per GB processed. While Transit Gateway simplifies routing management and can handle more complex network topologies, the current and projected traffic patterns do not justify its added cost. VPN connections to the on-premises data center through virtual private gateways are necessary in each region regardless of the inter-VPC connectivity solution. Using a VPN connection between the virtual private gateways (Option D) would be costlier than VPC peering due to the data transfer charges associated with VPN tunnels. Option B unnecessarily routes inter-VPC traffic through the on-premises firewall, adding latency and potentially overloading the firewall appliance, also it doesn't account for the current traffic requirements between the VPCs (50 TB). Option A, using Transit Gateways and a peering connection, is the most expensive due to the data processing charges of Transit Gateways for the 50 TB of inter-region traffic. VPC peering avoids these charges, making Option C the most cost-effective.

VPC Peering: <https://docs.aws.amazon.com/vpc/latest/peering/what-is-vpc-peering.html>

Transit Gateway: <https://docs.aws.amazon.com/vpc/latest/tgw/what-is-transit-gateway.html>

Virtual Private Gateway: https://docs.aws.amazon.com/vpn/latest/s2svpn/VPC_VPN.html

Question: 251

CertyIQ

A company runs workloads in multiple VPCs. The company needs to securely access a workload in one of the VPCs, named VPC-A, from an on-premises data center. A network engineer sets up an AWS Site-to-Site VPN connection to a transit gateway. The network engineer configures dynamic routing for the connection, and communication works properly.

Recently, the owner of VPC-A added another CIDR range to the VPC. The VPC-A owner created workloads that use the additional CIDR range.

The company's on-premises network is unable to reach the new workloads. The network engineer needs to resolve the network connectivity issue and ensure that connectivity will not be affected if additional VPC CIDR ranges are added to the VPC in the future.

Which solution will meet these requirements with the MOST operational efficiency?

- A. Configure route propagation for VPC-A to the VPN attachment route table.
- B. Manually update the VPN attachment route table to include the new CIDR range.
- C. Configure an Amazon EventBridge rule to invoke an AWS Lambda function when the rule matches an update to the VPC-A CIDR range. Configure the Lambda function to update the VPN attachment route table.
- D. Configure an Amazon CloudWatch alarm to invoke an AWS Lambda function when there is an update to the VPC-A CIDR range. Configure the Lambda function to update the VPN attachment route table. Restart the VPN tunnels.

Answer: A

Explanation:

The correct answer is **A: Configure route propagation for VPC-A to the VPN attachment route table.**

Here's why:

Dynamic Routing and Route Propagation: The initial setup used dynamic routing with a transit gateway. Route propagation is the mechanism that allows a transit gateway to automatically learn and distribute routes

from attached resources (like VPCs) to its route tables.

Operational Efficiency: Route propagation is the most operationally efficient solution because it is automatic. Once configured, any new CIDR ranges added to VPC-A will be automatically propagated to the transit gateway's route tables, which in turn will be advertised to the on-premises network via the VPN connection's dynamic routing (BGP). This eliminates manual intervention for future CIDR range additions.

Why other options are less desirable:

B (Manually Update): This requires manual intervention every time a CIDR range is added, which is not operationally efficient. It's prone to errors and delays.

C and D (EventBridge/CloudWatch + Lambda): These options introduce unnecessary complexity. While they can automate the route table updates, they require managing Lambda functions, EventBridge rules (or CloudWatch alarms), and IAM permissions. Route propagation offers the same functionality with less overhead. Furthermore, these options involve custom coding and potential failure points within the automation process.

How it works: By enabling route propagation from VPC-A to the VPN attachment route table, the transit gateway automatically learns the new CIDR range. The transit gateway then advertises these routes to the on-premises network through the BGP session established by the Site-to-Site VPN connection. This ensures that the on-premises network has the necessary routing information to reach the new workloads in VPC-A.

In summary: Route propagation leverages the existing dynamic routing setup for the VPN connection to automate the route updates. It's the most operationally efficient solution because it avoids manual configuration and automatically adapts to changes in VPC-A's CIDR ranges.

Authoritative Links:

AWS Transit Gateway Route Tables: <https://docs.aws.amazon.com/transit-gateway/latest/tgw/tgw-route-tables.html>

AWS Transit Gateway Route Propagation: <https://docs.aws.amazon.com/transit-gateway/latest/tgw/tgw-route-tables.html#tgw-route-propagation>

Question: 252

CertyIQ

A company is migrating its internet VPN connections to dedicated AWS Direct Connect connections. The company needs to set up the Direct Connect connections so that all network communications are encrypted in transit.

Which combination of steps will meet this requirement? (Choose three.)

- A. Create new Direct Connect connections while requesting MACsec ports.
- B. Create a MACsec Connectivity Association Key Name (CKN) and Connectivity Association Key (CAK) pair. Associate the pair with each new connection.
- C. Update the on-premises routers to use MACsec and the shared Connectivity Association Key Name (CKN) and Connectivity Association Key (CAK) pair.
- D. Create a shared key for an IPsec connection.
- E. Configure a new Direct Connect gateway. Associate the shared key with the new Direct Connect gateway.
- F. Set up IPsec on the on-premises router. Associate the shared key with the IPsec configuration.

Answer: ABC

Explanation:

The requirement is to establish encrypted network communication over Direct Connect while migrating from VPN. The correct answer is ABC.

A. Create new Direct Connect connections while requesting MACsec ports: Direct Connect supports MACsec (Media Access Control security) for encrypting data at the physical layer. Requesting MACsec ports

ensures that the Direct Connect connection is provisioned to support this feature. This is a necessary first step for enabling MACsec encryption.

B. Create a MACsec Connectivity Association Key Name (CKN) and Connectivity Association Key (CAK) pair. Associate the pair with each new connection: MACsec uses a secret key, typically a CKN/CAK pair, for encryption and authentication. The CKN identifies the key, and the CAK is the actual key used for encryption. Associating this pair with each connection is essential for enabling MACsec on the Direct Connect link.

C. Update the on-premises routers to use MACsec and the shared Connectivity Association Key Name (CKN) and Connectivity Association Key (CAK) pair: The on-premises routers at the customer end need to be configured to support MACsec as well. They must also be configured to use the same CKN/CAK pair as the Direct Connect connection. This ensures that both ends of the link can encrypt and decrypt the traffic using the same secret key. This completes the MACsec configuration, providing encryption in transit.

Why the other options are incorrect:

D. Create a shared key for an IPsec connection: While IPsec also provides encryption, it operates at the network layer (Layer 3) and is not the method Direct Connect uses when using MACsec. This creates a tunnel within the Direct Connect setup, adding overhead and unnecessary complexity when MACsec is available. The question is about encrypting traffic within the Direct Connect setup, making IPsec not the most relevant solution.

E. Configure a new Direct Connect gateway. Associate the shared key with the new Direct Connect gateway: The Direct Connect gateway is used for routing traffic between Direct Connect connections and VPCs. While it's a relevant component in a Direct Connect setup, it doesn't play a direct role in MACsec encryption. Direct Connect Gateway does not use shared keys.

F. Set up IPsec on the on-premises router. Associate the shared key with the IPsec configuration: Setting up IPsec creates an IPsec tunnel within the Direct Connect line. The Direct Connect already provides a fast and consistent connection. MACSec can provide the encryption without the overhead of setting up a new IPsec connection.

Authoritative links:

AWS Direct Connect MACsec: <https://docs.aws.amazon.com/directconnect/latest/UserGuide/direct-connect-macsec.html>

Question: 253

CertyIQ

A company has an application VPC and a networking VPC that are connected through VPC peering. The networking VPC contains a Network Load Balancer (NLB). The application VPC contains Amazon EC2 instances that run an application. The EC2 instances are part of a target group that is associated with the NLB in the networking VPC.

The company configures a third VPC and peers it to the networking VPC. The new VPC contains a new version of the existing application. The new version of the application runs on new EC2 instances in an application subnet. The new version of the application runs in a different Availability Zone than that original version of the application.

The company needs to establish connectivity between the NLB and the new version of the application.

Which combination of steps will meet this requirement? (Choose three.)

- A. Register the new application EC2 instances with the NLB by using the instance IDs.
- B. Register the new application EC2 instances with the NLB by using instance IP addresses.
- C. Configure the NLB in the Availability Zone where the new application EC2 instances run.
- D. Configure the NLB to use zonal shift.
- E. Configure the network ACL for the application subnet in the new VPC to allow outbound connections.
- F. Configure the network ACL for the application subnet in the new VPC to allow inbound connections and

outbound connections.

Answer: BCF

Explanation:

Let's break down why options B, C, and F are the correct choices for establishing connectivity between the NLB and the new application version.

B. Register the new application EC2 instances with the NLB by using instance IP addresses: NLBs, unlike Application Load Balancers (ALBs), primarily target IP addresses when dealing with targets across VPC peering connections. Using instance IDs directly might not resolve correctly due to the separate network context of the peered VPCs. The NLB needs a routable IP address to forward traffic to the EC2 instances. <https://docs.aws.amazon.com/elasticloadbalancing/latest/network/network-load-balancers.html#target-group-attributes>

C. Configure the NLB in the Availability Zone where the new application EC2 instances run: For the NLB to route traffic effectively, it must be enabled in the Availability Zone where the target EC2 instances reside. NLBs distribute traffic only within their enabled Availability Zones. Without configuring the NLB in the new AZ, it cannot reach the new instances. <https://docs.aws.amazon.com/elasticloadbalancing/latest/network/network-load-balancers.html#availability-zones>

F. Configure the network ACL for the application subnet in the new VPC to allow inbound connections and outbound connections: Network ACLs act as firewalls at the subnet level. For the EC2 instances in the new VPC to receive traffic from the NLB and send responses, the network ACL must allow both inbound traffic from the NLB's IP range or the VPC CIDR, and outbound traffic back to the NLB. Inbound rules are required so EC2 instances can receive traffic from the NLB, and outbound rules are needed so instances can send traffic back to the NLB. <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-acls.html>

Let's discuss why the other options are incorrect:

A. Register the new application EC2 instances with the NLB by using the instance IDs: As mentioned previously, NLBs are best suited for IP address targets across peered VPCs. Instance IDs might not resolve properly across peering connections.

D. Configure the NLB to use zonal shift: Zonal shift is used for shifting traffic away from an Availability Zone, typically during an issue or for testing. It's not a solution for establishing connectivity between an NLB and EC2 instances in a new AZ that needs permanent traffic.

E. Configure the network ACL for the application subnet in the new VPC to allow outbound connections: Outbound rules alone are insufficient as the EC2 instances in the new VPC must also allow inbound connections to receive traffic from the NLB.

In summary, options B, C, and F provide the necessary configurations for the NLB to successfully target and communicate with the new application instances across the peered VPCs, considering the NLB's operational characteristics and network ACL requirements.

Question: 254

CertyIQ

A company uses AWS Site-to-Site VPN connections to encrypt traffic between the company's on-premises location and a single VPC. The Site-to-Site VPN connections use two 1 Gbps AWS Direct Connect connections with public VIFs. The company plans to add 15 additional VPCs in the same AWS Region.

The company must maintain the same level of encryption that the Site-to-Site VPN connections currently provide for each connection between the on-premises location and the new VPCs. The new connections must not use public IP addresses. The bandwidth of the Site-to-Site VPN connections will remain less than the current

provisioned speed.

Which combination of steps will meet these requirements with LEAST operational overhead? (Choose three.)

- A. Create a transit gateway and a Direct Connect gateway. Associate the transit gateway with the Direct Connect gateway. Attach all the new VPCs to the transit gateway.
- B. For each new VPC, create a new Direct Connect private VIF to a Direct Connect gateway. Associate all VPCs with the Direct Connect gateway.
- C. Assign a private IP CIDR block to the transit gateway.
- D. Assign a public IP CIDR block to the transit gateway.
- E. Create a transit VIF to the Direct Connect gateway. Create a Site-to-Site VPN private IP VPN connection.
- F. Create a public VIF to the Direct Connect gateway. Create a Site-to-Site VPN public IP VPN connection.

Answer: ACE

Explanation:

The correct answer is **ACE**. Here's why:

A. Create a transit gateway and a Direct Connect gateway. Associate the transit gateway with the Direct Connect gateway. Attach all the new VPCs to the transit gateway. This is a foundational step. AWS Transit Gateway acts as a central hub, simplifying the routing between the on-premises network and multiple VPCs. A Direct Connect gateway provides a single entry point into your AWS network via Direct Connect. By associating the Transit Gateway with the Direct Connect gateway, you enable connectivity between your on-premises network and the VPCs connected to the Transit Gateway over the Direct Connect link. This avoids using the public internet and ensures private connectivity.

C. Assign a private IP CIDR block to the transit gateway. To ensure the Site-to-Site VPN connections use private IP addresses, the transit gateway must be configured with a private IP CIDR block. This ensures that traffic within the transit gateway and between VPCs uses private IP addresses.

E. Create a transit VIF to the Direct Connect gateway. Create a Site-to-Site VPN private IP VPN connection. Creating a transit virtual interface (VIF) on the Direct Connect connection allows the on-premises network to communicate with the Transit Gateway and the connected VPCs via BGP routing. Establishing Site-to-Site VPN connections with private IPs over this transit VIF maintains the required level of encryption without using public IP addresses and meets the requirement of maintaining the same level of encryption as the current connections. This option also ensures that the bandwidth of the Site-to-Site VPN connections remains less than the current provisioned speed.

Why other options are incorrect:

B: Creating a new Direct Connect private VIF for each new VPC is operationally expensive and complex to manage. It also doesn't inherently provide encryption.

D: Assigning a public IP CIDR block to the transit gateway directly contradicts the requirement that the new connections must not use public IP addresses.

F: Creating a public VIF with Site-to-Site VPN using public IPs also contradicts the requirements of no public IPs and is less secure.

Supporting Documentation:

AWS Transit Gateway: <https://aws.amazon.com/transit-gateway/>

AWS Direct Connect: <https://aws.amazon.com/directconnect/>

AWS Site-to-Site VPN: <https://aws.amazon.com/vpn/>

Question: 255

A company hosts application servers on premises and on Amazon EC2 instances in a VPC. The application servers access data that is hosted in an Amazon S3 bucket through the public internet. The EC2 instances in the VPC use an AWS Site-to-Site VPN for connectivity with the on-premises application servers.

New company regulations state that all traffic between the application servers and the S3 bucket must remain private and must not use public IP addresses.

Which solution will meet these requirements MOST cost-effectively?

- A. Configure an S3 gateway endpoint. Modify the route table with the appropriate route for the endpoint. Access the S3 bucket through the gateway endpoint from the EC2 instances.
- B. Configure an S3 interface endpoint. Update the on-premises servers and EC2 instances to use the interface endpoint DNS name to access the S3 bucket.
- C. Configure an S3 interface endpoint. Update the on-premises servers to use the interface endpoint DNS name to access the S3 bucket. Configure an S3 gateway endpoint. Modify the route table so that the EC2 instances use the gateway endpoint.
- D. Configure an S3 gateway endpoint. Modify the route table with the appropriate route for the endpoint. Use an S3 bucket policy to restrict access to the gateway endpoint. Configure a proxy server fleet behind a Network Load Balancer in the VPC so that the on-premises servers can access the S3 bucket.

Answer: A

Explanation:

The requirement is to ensure private connectivity between both on-premises and EC2-based application servers and an S3 bucket, without using public IPs and in a cost-effective manner.

Option A is correct because it leverages an S3 gateway endpoint. Gateway endpoints are designed specifically for S3 and DynamoDB, providing a cost-effective and efficient way to access these services privately from within a VPC. By configuring a route in the VPC route table to direct traffic destined for S3 to the gateway endpoint, traffic from EC2 instances stays within the AWS network and avoids public IPs. Gateway endpoints do not charge for data processed.

Option B is incorrect. While S3 interface endpoints provide private connectivity using PrivateLink, they are not the most cost-effective solution for S3 access. They incur hourly charges and data processing charges. Also, configuring an S3 interface endpoint would only solve the problem for the EC2 instances, not the on-premises servers.

Option C is incorrect because it uses both interface and gateway endpoints. Gateway endpoints are appropriate for EC2, but it doesn't explain how the on-premises servers will access the S3 bucket, and the interface endpoint is unnecessarily costly. A Gateway endpoint does not work for on-premises connectivity.

Option D is incorrect. While the gateway endpoint handles the EC2 instance access, a proxy server fleet behind a Network Load Balancer adds unnecessary complexity and cost for the on-premises servers. Furthermore, relying solely on an S3 bucket policy to restrict access to the gateway endpoint isn't a complete solution for the on-premises servers accessing S3 through the proxy; a VPN connection would be the ideal solution for on-premises S3 access. It's an overly complex solution for a problem that can be solved with a VPN connection to AWS and a gateway endpoint in the VPC. The proxy introduces additional operational overhead and potential performance bottlenecks.

Therefore, Option A is the most cost-effective and efficient solution because it utilizes the S3 gateway endpoint for private S3 access from the EC2 instances, avoiding public IP addresses and extra data processing charges while keeping the solution simple.

Supporting resources:

Question: 256

A company uses AWS Network Firewall to protect outgoing traffic for multiple VPCs that are in the same AWS account. Each VPC contains Amazon EC2 instances that host the company's applications. Each EC2 instance is tagged with the name of the application it hosts. The EC2 instances are in Auto Scaling groups.

A Network Firewall stateful rule group must remain up-to-date, even when an Auto Scaling group launches and terminates EC2 instances.

Which solution will meet this requirement with the LEAST implementation and administrative effort?

- A. Create a network ACL for each application. Reference the network ACL in the stateful rule group.
- B. Create a prefix list for each application. Reference the prefix list in the stateful rule group.
- C. Create an AWS Lambda function that queries the EC2 instance tags for each application name and then updates the stateful rule group with the IP address of each instance.
- D. Create a resource group for each application name. Reference the Amazon Resource Name (ARN) for the resource groups in the stateful rule group.

Answer: D

Explanation:

Here's a detailed justification for why option D is the best solution:

The problem requires a solution that keeps AWS Network Firewall stateful rule groups up-to-date with the IP addresses of EC2 instances, even as the Auto Scaling groups scale up and down. The solution must minimize implementation and administrative overhead.

Option D, using resource groups, is the most efficient way to accomplish this. AWS Resource Groups allow you to create a logical grouping of AWS resources based on tags or other criteria. In this case, you create a resource group for each application name (based on the EC2 instance tags). When new EC2 instances are launched with the appropriate tag, they are automatically added to the corresponding resource group. Similarly, when instances are terminated, they are automatically removed.

The key is that Network Firewall allows you to reference the ARN of a resource group directly in the stateful rule group. This means the rule group doesn't need to be manually updated whenever EC2 instances are added or removed. Network Firewall dynamically resolves the resource group's membership into IP addresses at runtime. This eliminates the need for manual intervention or custom scripts.

Options A and B are less suitable. Network ACLs (A) operate at the subnet level and don't scale well with individual instances. Prefix lists (B) require manual updating, which is contrary to the problem's requirements. Option C, using a Lambda function, introduces complexity. A Lambda function would need to be triggered by EC2 instance lifecycle events (launch and termination), query the tags, and then update the Network Firewall rules. This is a more complex and error-prone solution than using resource groups.

Therefore, using resource groups and referencing their ARNs in the Network Firewall stateful rule group offers a simple, scalable, and automated solution with minimal administrative overhead.

Authoritative links:

AWS Resource Groups: <https://docs.aws.amazon.com/resource-groups/latest/userguide/welcome.html>

AWS Network Firewall Stateful Rule Groups: <https://docs.aws.amazon.com/network->

Question: 257

A company has multiple AWS Site-to-Site VPN connections between an on-premises environment and multiple VPCs. The Site-to-Site VPN connections use virtual private gateways and are configured with IPv4 addresses. The company hosts several internal applications in the VPCs.

Application users have reported that the applications are performing slowly. A network engineer notices excessive latency in the network path that the VPN connections use. The network engineer needs to resolve the excessive latency.

Which solution will meet this requirement?

- A. Use AWS Global Accelerator to deploy an accelerator on the existing Site-to-Site VPN connections.
- B. Deploy a transit gateway and a new accelerated Site-to-Site VPN connection.
- C. Replace the existing Site-to-Site VPN connections with new Site-to-Site VPN connections that use IPv6.
- D. Replace the existing Site-to-Site VPN connections with AWS PrivateLink connections.

Answer: B**Explanation:**

The correct answer is **B. Deploy a transit gateway and a new accelerated Site-to-Site VPN connection.**

Here's a detailed justification:

The primary problem is excessive latency in the existing Site-to-Site VPN connections. AWS Global Accelerator is designed to optimize network paths to AWS services, and it can be used with Site-to-Site VPNs to reduce latency and improve performance.

Why Transit Gateway? A transit gateway simplifies the network topology by acting as a central hub for connecting multiple VPCs and on-premises networks. This reduces the complexity of managing multiple VPN connections directly between VPCs and the on-premises environment.

Why Accelerated Site-to-Site VPN? By deploying an accelerated Site-to-Site VPN, the traffic will leverage AWS Global Accelerator's global network, which dynamically routes traffic to the optimal AWS edge location closest to the on-premises network. This reduces latency by minimizing the distance data travels over the public internet.

Option A is incorrect because simply adding AWS Global Accelerator to existing VPN connections without Transit Gateway might not fully resolve the issue, especially if the problem stems from complex routing between multiple VPCs. The architecture will be more complex to manage and less efficient.

Option C is incorrect because switching to IPv6 addresses would not directly address the latency issue. While IPv6 has advantages, the latency problem is related to network path optimization, not IP addressing.

Option D is incorrect because AWS PrivateLink is for securely accessing AWS services or services hosted by other AWS accounts without exposing traffic to the public internet. It does not solve the latency issue related to the on-premises to VPC connection.

Supporting Concepts and Links:

AWS Global Accelerator: Provides static IP addresses that serve as entry points for applications. Traffic is then routed over the AWS global network, which is designed to reduce latency and improve availability.

<https://aws.amazon.com/global-accelerator/>

Transit Gateway: A network transit hub that you can use to interconnect your virtual private clouds (VPCs)

and on-premises networks. <https://aws.amazon.com/transit-gateway/>

Accelerated Site-to-Site VPN: Combines the benefits of Site-to-Site VPN with AWS Global Accelerator to optimize the performance of VPN connections. <https://aws.amazon.com/about-aws/whats-new/2020/03/aws-global-accelerator-now-supports-aws-site-to-site-vpn/>

Question: 258

CertyIQ

A company has a transit gateway in a single AWS account. The company sends flow logs for the transit gateway to an Amazon CloudWatch Logs log group.

The company created an AWS Lambda function to analyze the logs. The Lambda function sends a notification to an Amazon Simple Notification Service (Amazon SNS) topic when a VPC generates traffic that is dropped by the transit gateway. Each notification contains the account ID, VPC ID, and total amount of dropped packets.

The company wants to subscribe a new Lambda function to the SNS topic. The new Lambda function must automatically prevent the traffic that is identified in each notification from leaving a VPC by applying a network ACL to the transit gateway attachment subnets in the VPC that generates the traffic.

Which solution will meet these requirements?

- A. Configure the existing Lambda function to add the destination IP addresses of the dropped traffic to each SNS notification. Configure the new Lambda function to create an outbound rule by using the destination IP addresses in the network ACL.
- B. Configure the existing Lambda function to add the source IP addresses of the dropped traffic to each SNS notification. Configure the new Lambda function to create an inbound rule by using the source IP addresses in the network ACL.
- C. Configure the existing Lambda function to add the source IP addresses of the dropped traffic to each SNS notification. Configure the new Lambda function to create an outbound rule by using the source IP addresses in the network ACL.
- D. Configure the existing Lambda function to add the destination IP addresses of the dropped traffic to each SNS notification. Configure the new Lambda function to create an inbound rule by using the destination IP addresses in the network ACL.

Answer: A

Explanation:

The correct answer is **A**. Here's a detailed justification:

The goal is to automatically block traffic leaving a VPC when the transit gateway drops it. The existing Lambda function identifies dropped traffic and sends notifications via SNS. The new Lambda function needs to use this information to block future traffic.

Network ACLs operate at the subnet level and are stateless. This means you must explicitly define both inbound and outbound rules to filter traffic effectively. To prevent traffic originating from a VPC from leaving, you need to control the outbound traffic from the transit gateway attachment subnets.

The existing Lambda function already provides the account ID, VPC ID, and the amount of dropped packets. To block specific traffic, we need the destination IP addresses of the dropped traffic. Modifying the existing Lambda function to include these in the SNS notification is essential.

The new Lambda function, upon receiving the notification, needs to use the destination IP addresses to create an outbound rule in the network ACL associated with the relevant transit gateway attachment subnets. This outbound rule will deny traffic to the destination IP addresses identified in the notification. This effectively blocks the VPC from sending traffic to those destinations.

Options B and C are incorrect because they focus on source IP addresses and inbound rules. Blocking inbound traffic to the VPC doesn't prevent the VPC from sending the problematic traffic out through the transit

gateway. Option D incorrectly proposes adding destination IPs to an inbound rule, which would block traffic entering the subnet destined for those IPs, not traffic leaving the subnet.

Therefore, option A is the only option that correctly identifies the need to add destination IP addresses to the SNS notification and create an outbound rule using these IPs in the network ACL to block the traffic from leaving the VPC. This aligns with the requirement of preventing the traffic identified in the notifications from leaving the VPC.

Relevant AWS documentation:

Network ACLs: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-acls.html>

Transit Gateway Flow Logs: <https://docs.aws.amazon.com/vpc/latest/tgw/tgw-flow-logs.html>

Question: 259

CertyIQ

A company has multiple VPCs with subnets that use IPv4. Traffic from the VPCs to the internet uses a NAT gateway. The company wants to transition to IPv6.

A network engineer creates multiple IPv6-only subnets in an existing testing VPC. The network engineer deploys a new Amazon EC2 instance that has an IPv6 address into one of the subnets. During testing, the network engineer discovers that the new EC2 instance is not able to communicate with an IPv4-only service through the internet. The network engineer needs to enable the IPv6 EC2 instance to communicate with the IPv4-only service.

Which solution will meet this requirement?

- A. Enable DNS64 for the IPv6-only subnets. Update the route tables for the IPv6-only subnets to send traffic through the NAT gateway.
- B. Enable NAT64 for the testing VPC. Reconfigure the existing NAT gateway to support IPv6.
- C. Enable DNS64 for the new EC2 instance. Create a new egress-only internet gateway that supports IPv6.
- D. Enable NAT64 for each route table. Create a new NAT gateway that supports both IPv4 and IPv6.

Answer: A

Explanation:

The correct answer is A: "Enable DNS64 for the IPv6-only subnets. Update the route tables for the IPv6-only subnets to send traffic through the NAT gateway."

Here's why this solution works:

The Problem: The EC2 instance has only an IPv6 address and cannot directly communicate with an IPv4-only service on the internet. NAT gateways are, by default, configured to translate IPv4 addresses to connect to the internet. A standard NAT gateway does not provide IPv6 to IPv4 translation.

DNS64: DNS64 is a service that synthesizes IPv6 addresses for IPv4-only destinations. When the IPv6-only EC2 instance queries a domain name that resolves to an IPv4 address, DNS64 creates a synthesized IPv6 address that encodes the IPv4 address within it. <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html>

NAT64: NAT64 translates IPv6 traffic into IPv4 traffic. To leverage NAT64, traffic must be routed to it. <https://datatracker.ietf.org/doc/html/rfc6146>

NAT Gateway Routing: The route tables for the IPv6-only subnets must be configured to send all IPv6 traffic destined for the internet (including the synthesized IPv6 addresses) to a NAT gateway. The NAT gateway then performs NAT64, translating the IPv6 addresses to IPv4 addresses, allowing the instance to communicate with IPv4-only services on the internet.

Why other options are incorrect:

B: Reconfiguring the existing NAT gateway is not sufficient. It will not enable NAT64.

C: Enabling DNS64 for the EC2 instance is incorrect, DNS64 is configured at the subnet level. Creating an egress-only internet gateway will not facilitate the IPv6 to IPv4 traffic needed. Egress-only internet gateways only enable IPv6 instances to initiate outbound connections over IPv6 to the internet, and prevents the internet from initiating a connection with your instances.

D: NAT64 does not support route tables, NAT64 is enabled at the VPC level. And creating a new NAT gateway does not enable translation of IPv6 to IPv4.

Therefore, enabling DNS64 on the subnet and routing IPv6 traffic via a NAT gateway is the correct solution for allowing the IPv6 EC2 instance to communicate with IPv4-only services.

Question: 260

CertyIQ

A company deployed an application in two AWS Regions in one AWS account. The company has one VPC in each Region. The VPCs use non-overlapping private CIDR ranges.

The company needs to connect both VPCs to a single on-premises data center to test the application. The application requires up to 800 Mbps of throughput. A network engineer needs to establish connectivity between the VPCs and the on-premises data center.

Which solution will meet this requirement with the LEAST operational overhead?

A. Order a 2 Gbps Direct Connect connection for the data center. Configure a virtual private gateway in each VPC. Create a private VIF for each virtual private gateway, and associate the virtual private gateways with the Direct Connect connection. Configure static routes in the VPC route tables and in the data center router.

B. Order a 2 Gbps Direct Connect connection for the data center. Configure a virtual private gateway in each VPC. Create a private VIF for each virtual private gateway, and associate the virtual private gateways with the Direct Connect connection. Configure Open Shortest Path First (OSPF) routing between the private VIF and the data center.

C. Configure a customer gateway and a virtual private gateway in each VPC. Configure an AWS Site-to-Site VPN connection between the data center and each VPC. Configure static routes in each VPC route table to point to the subnets in the data center.

D. Configure a customer gateway and a virtual private gateway in each VPC. Configure an AWS Site-to-Site VPN connection between the data center and each VPC. Configure BGP routing between the VPCs and the data center.

Answer: D

Explanation:

The correct answer is D. Here's why:

Justification:

Option D is the most suitable because it uses AWS Site-to-Site VPN connections with Border Gateway Protocol (BGP) routing. This approach offers the least operational overhead while meeting the stated requirements. Here's a breakdown:

1. **Connectivity:** Site-to-Site VPN establishes secure tunnels between each VPC and the on-premises data center, providing the necessary network connectivity.
2. **Throughput:** Multiple Site-to-Site VPN connections can provide aggregate throughput exceeding the 800 Mbps requirement.
3. **Dynamic Routing:** BGP automates route propagation between the VPCs and the data center. This eliminates the need for manual static route maintenance, thereby reducing operational overhead.

Changes in network topology are automatically learned and adapted to by BGP.

4. **Redundancy:** BGP can be configured to use multiple paths for redundancy.

5. **Cost-Effectiveness:** Site-to-Site VPN connections are generally more cost-effective than Direct Connect for bandwidth requirements in the sub-Gigabit range.

Options A and B are not optimal because they involve Direct Connect, which, although offering dedicated bandwidth, is an overkill for the 800 Mbps requirement. Direct Connect incurs higher costs and complexities compared to Site-to-Site VPN when the bandwidth needs are relatively low. Also, option A requires manual static route configuration, which increases operational overhead. Option B uses OSPF routing, but is unnecessary as the Site-to-Site VPN with BGP is sufficient.

Option C, while using Site-to-Site VPNs, resorts to static routing. This is less desirable as it requires manual intervention to update routes whenever network changes occur, increasing administrative burden.

Authoritative Links:

AWS Site-to-Site VPN: <https://aws.amazon.com/vpn/>

AWS VPN routing options: https://docs.aws.amazon.com/vpn/latest/s2svpn/VPNGW_Routing.html

AWS Direct Connect: <https://aws.amazon.com/directconnect/>

Question: 261

CertyIQ

A company runs a workload in a single VPC on AWS. The company's architecture contains several interface VPC endpoints for AWS services, including Amazon CloudWatch Logs and AWS Key Management Service (AWS KMS). The endpoints are configured to use a shared security group. The security group is not used for any other workloads or resources.

After a security review of the environment, the company determined that the shared security group is more permissive than necessary. The company wants to make the rules associated with the security group more restrictive. The changes to the security group rules must not prevent the resources in the VPC from using AWS services through interface VPC endpoints. The changes must prevent unnecessary access.

The security group currently uses the following rules:

• Inbound - Rule 1

Protocol: TCP -

Port: 443 -

Source: 0.0.0.0/0 -

• Inbound - Rule 2

Protocol: TCP -

Port: 443 -

Source: VPC CIDR -

• Outbound - Rule 1

Protocol: All -

Port: All -

Destination: 0.0.0.0/0 -

Which rule or rules should the company remove to meet with these requirements?

A.Outbound - Rule 2

B.Inbound - Rule 1 and Outbound - Rule 1

C.Inbound - Rule 2 and Outbound - Rule 1

D.Outbound - Rule 1

Answer: B

Explanation:

The correct answer is B: Inbound - Rule 1 and Outbound - Rule 1.

Justification:

The goal is to restrict the security group rules associated with the interface VPC endpoints without disrupting the ability of resources within the VPC to access AWS services through those endpoints, while also preventing unnecessary access.

Inbound - Rule 1 (TCP:443, Source: 0.0.0.0/0): This rule allows HTTPS traffic (port 443) from any source on the internet to reach the VPC endpoints. This is overly permissive and represents a significant security risk. It should be removed. Since the VPC endpoints are meant to be accessed by resources within the VPC itself, there is no need to allow ingress traffic from the entire internet. The presence of Inbound - Rule 2 already allows traffic from within the VPC.

Inbound - Rule 2 (TCP:443, Source: VPC CIDR): This rule allows HTTPS traffic from within the VPC to reach the VPC endpoints. This is necessary for the resources in the VPC to utilize AWS services via these endpoints and should be kept.

Outbound - Rule 1 (All, Destination: 0.0.0.0/0): This rule allows all outbound traffic from the VPC endpoints to any destination on the internet. This is excessively permissive and poses a security risk, violating the principle of least privilege. The company wants to limit unnecessary access. It should be removed. VPC endpoints do not initiate connections to the public internet. They receive requests from services in the VPC and relay them to AWS services without needing broad internet access.

Removing both "Inbound - Rule 1" and "Outbound - Rule 1" significantly tightens the security posture by preventing external access to the VPC endpoints and limiting the scope of potentially malicious outbound communication originating from those endpoints (even though it should not exist). "Inbound - Rule 2" allows legitimate communication from within the VPC.

Authoritative Links:

AWS VPC Endpoints: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html>

AWS Security Groups: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_SecurityGroups.html

Principle of Least Privilege: https://en.wikipedia.org/wiki/Principle_of_least_privilege

Question: 262

CertyIQ

A company uses transit gateways to route traffic between the company's VPCs. Each transit gateway has a single route table. Each route table contains attachments and routes for the VPCs that are in the same AWS Region as the transit gateway. The route tables in each VPC also contain routes to all the other VPC CIDR ranges that are available through the transit gateways. Some VPCs route to local NAT gateways.

The company plans to add many new VPCs soon. A network engineer needs a solution to add new VPC CIDR ranges to the route tables in each VPC.

Which solution will meet these requirements in the MOST operationally efficient way?

A.Create a new customer-managed prefix list. Add all VPC CIDR ranges to the new prefix list. Update the route tables in each VPC to use the new prefix list ID as the destination and the appropriate transit gateway ID as the target.

- B. Turn on default route table propagation for the transit gateway route tables. Turn on route propagation for each route table in each VPC.
- C. Update the route tables in each VPC to use 0.0.0.0/0 as the destination and the appropriate transit gateway ID as the target.
- D. Turn on default route table association for the transit gateway route tables. Turn on route propagation for each route table in each VPC.

Answer: A

Explanation:

Here's a detailed justification for why option A is the most operationally efficient solution for automatically updating VPC route tables with new CIDR ranges using Transit Gateways:

Why Option A is Correct (and Most Efficient):

Centralized CIDR Management: Prefix lists provide a central location to manage all VPC CIDR ranges. This means you only need to update the prefix list when a new VPC is added, rather than modifying individual route tables in each VPC.

Automatic Route Updates: By referencing the prefix list in your VPC route tables, AWS automatically updates the routes whenever the prefix list is modified. This eliminates the need for manual intervention in each VPC.

Scalability: Prefix lists are designed to handle a large number of CIDR ranges, making them ideal for environments with many VPCs.

Simplicity: The solution requires creating one prefix list and updating existing route tables once. Subsequent updates are handled automatically.

Security: Prefix lists can also be used to control which VPCs can access specific CIDR ranges.

Why Other Options are Incorrect:

B. Default Route Table Propagation (Transit Gateway): Default route table propagation propagates attachments to route tables, but doesn't inherently propagate specific CIDR ranges within those attachments. It mainly simplifies the association between TGW attachments and TGW route table.

C. 0.0.0.0/0 Route: This creates a default route through the transit gateway, directing all traffic to it. This might not be desirable, as it can override more specific routes for other services or on-premises connections. Also, this would need to be added manually on each VPC's route table, so it's not operationally efficient.

D. Default Route Table Association (Transit Gateway): Default route table association automatically associates new attachments with a TGW route table. Route propagation to the VPC route tables needs to be configured separately, and it propagates entire routes not just specific CIDR ranges. This would still require manually updating each VPC route table initially, making it less efficient.

Authoritative Links:

AWS Prefix Lists: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-prefix-lists.html>

AWS Transit Gateway Route Tables: <https://docs.aws.amazon.com/vpc/latest/tgw/tgw-route-tables.html>

Question: 263

CertyIQ

A company has several AWS Site-to-Site VPN connections between an on-premises customer gateway and a transit gateway. The company's application uses IPv4 to communicate through the VPN connections.

The company has updated the VPC to be dual stack and wants to transition to using IPv6-only for new workloads. When the company tries to communicate through the existing VPN connections, IPv6 traffic fails.

Which solution will provide IPv6 support with the LEAST operational overhead?

- A. Create a new Site-to-Site VPN connection that supports IPv6.
- B. Create a new Site-to-Site VPN connection to a self-managed Amazon EC2 instance that runs open source software.
- C. Update the existing Site-to-Site VPN connections to support IPv6.
- D. Update the on-premises customer gateway's public IP address from IPv4 to IPv6.

Answer: A

Explanation:

The correct answer is **A. Create a new Site-to-Site VPN connection that supports IPv6.**

Here's a detailed justification:

The core issue is that the existing Site-to-Site VPN connections are configured for IPv4 and do not natively support IPv6 traffic. Transit Gateway supports IPv6 through VPN attachments, but this must be configured during the VPN creation.

Option A, creating a new Site-to-Site VPN connection that supports IPv6, is the most straightforward and least operationally burdensome solution. This allows the company to establish a separate path for IPv6 traffic without disrupting the existing IPv4 connectivity. It leverages AWS managed services to provide IPv6 support via a Site-to-Site VPN. The new connection can be configured to support IPv6 during the VPN creation process on the AWS side, which is simpler than modifying the existing VPN or managing custom EC2 instances.

Option B, creating a new Site-to-Site VPN connection to a self-managed EC2 instance, is a more complex solution. It introduces the operational overhead of managing an EC2 instance, including patching, scaling, and ensuring its availability. This adds unnecessary complexity when AWS-managed VPN services are readily available for IPv6.

Option C, updating the existing Site-to-Site VPN connections to support IPv6, might be possible in some scenarios. However, it often involves more complex reconfiguration and potential downtime, especially if the underlying customer gateway device and its configuration don't readily support dual-stack or IPv6 operation. Some VPN devices might require firmware updates or substantial configuration changes which is operationally expensive.

Option D, updating the on-premises customer gateway's public IP address from IPv4 to IPv6, is generally incorrect and based on a misunderstanding of VPN operation. The public IP address is for establishing the VPN tunnel. Changing the public IP address is not relevant to supporting IPv6 inside the VPN tunnel, and furthermore assumes the on-premise device even has an IPv6 public IP. The IPv6 support is configured within the tunnel itself.

Therefore, creating a new Site-to-Site VPN connection designed for IPv6 is the simplest and most operationally efficient way to enable IPv6 communication while maintaining existing IPv4 traffic flows.

Relevant documentation for further research:

AWS Site-to-Site VPN: <https://aws.amazon.com/vpn/>

AWS Transit Gateway: <https://aws.amazon.com/transit-gateway/>

AWS VPN User Guide: https://docs.aws.amazon.com/vpn/latest/s2svpn/VPC_VPN.html

Question: 264

A company has two teams: Team A and Team B. Team A has VPCs that run in Account A. The team uses a transit gateway (TGW-A) to route traffic between workloads that run in the different VPCs. Similarly, Team B has VPCs that run in Account B. Team B uses a different transit gateway (TGW-B) to route traffic between workloads that run in the different VPCs.

The company's network team manages the routing for Team A and Team B. The network team wants to retire TGW-B and use a single transit gateway to manage routing for the VPCs of both teams.

Which solution will meet this requirement with the LEAST operational overhead?

A. Create a resource share for TGW-A. Share TGW-A with Account B. Create VPC attachments for the VPCs in Account B. Configure routing for the VPCs in TGW-A route tables. Update the route tables of the VPCs in Account B to forward traffic to TGW-A. Delete TGW-B attachments and TGW-B.

B. Create a resource share for TGW-A. Share TGW-A with Account B. Replicate the TGW-B configuration to TGW-A to automatically start routing changes for the VPCs in Account B. Delete TGW-B when routing changes are complete.

C. Create a new transit gateway (TGW-C) in Account A. Create a resource share for TGW-C with Account B. Create VPC attachments for the VPCs in Account A and Account B. Configure routing for all the VPCs in TGW-C route tables. Update the route tables for the VPCs in Account A and Account B to forward traffic to TGW-C. Delete TGW-A attachments and TGW-B attachments. Delete TGW-A and TGW-B.

D. Create a new transit gateway (TGW-C) in a new account (Account C). Create a resource share for TGW-C. Share TGW-C with Account A and Account B. Create VPC attachments for the VPCs in Account A and Account B. Configure routing for all the VPCs in TGW-C route tables. Update the route tables for the VPCs in Account A and Account B to forward traffic to TGW-C. Delete TGW-A attachments and TGW-B attachments. Delete TGW-A and TGW-B.

Answer: A

Explanation:

Resource Sharing with AWS RAM: • AWS Resource Access Manager (RAM) allows TGW-A to be shared with Account B, enabling Team B's VPCs to attach to it. 2. Reusing TGW-A: • Since TGW-A is already handling routing, adding Account B's VPCs is straightforward. 3. Simple Routing Updates: • Account B's VPCs just need to update their route tables to forward traffic through TGW-A. 4. Retirement of TGW-B: • Once Team B's VPCs are fully connected to TGW-A, TGW-B can be safely decommissioned.

Question: 265

A company has an AWS environment that includes multiple VPCs that are connected by a transit gateway. The company wants to use a certificate-based AWS Site-to-Site VPN connection to establish connectivity between an on-premises environment and the AWS environment. The company does not have a static public IP address for the on-premises environment.

Which combination of steps should the company take to establish VPN connectivity between the transit gateway and the on-premises environment? (Choose two.)

A. Create a public certificate in AWS Certificate Manager (ACM).

B. Create a private certificate in AWS Certificate Manager (ACM).

C. Configure the Site-to-Site VPN tunnels to use the pre-shared key (PSK).

D. Create a customer gateway. Specify the current dynamic IP address of the customer gateway device's external interface.

E. Create a customer gateway. Do not specify the IP address of the customer gateway device.

Answer: BE

Explanation:

B (Create a private certificate in ACM) is correct:•AWS requires a private certificate from AWS Certificate Manager (ACM) for certificate-based authentication.•The certificate is used to authenticate the VPN connection instead of a pre-shared key (PSK).Why E (Create a customer gateway without specifying an IP address) is correct:•If the on-premises IP address is dynamic, you must create a customer gateway without an IP address.•This allows the VPN to function with BGP (Border Gateway Protocol), which dynamically updates the connection when the on-premises IP changes.

Question: 266

CertyIQ

A company operates in multiple AWS Regions. The company has deployed transit gateways in each Region. The company uses AWS Organizations to operate multiple AWS accounts in one organization.

The company needs to capture all VPC flow log data when a new VPC is created. The company needs to send flow logs to a specific Amazon S3 bucket.

Which solution will meet these requirements with the LEAST administrative effort?

- A. Update IAM permissions for each user to include a condition that ensures users can create VPCs only when VPC Flow Logs is enabled and configured correctly.
- B. Create a custom AWS Config rule with automatic remediation that verifies VPC Flow Logs is enabled and configured correctly. Apply the AWS Config rule to the organization.
- C. Enable VPC Flow Logs on each transit gateway. Configure VPC Flow Logs to send flow logs to the specified S3 bucket.
- D. Deploy a serverless application that uses AWS CloudTrail to monitor for VPC creation events in each account. Configure the application to apply the correct VPC Flow Logs configuration.

Answer: B

Explanation:

B. Create a custom AWS Config rule with automatic remediation that verifies VPC Flow Logs is enabled and configured correctly. Apply the AWS Config rule to the organization.

To ensure VPC Flow Logs are always enabled and configured correctly across all accounts in an organization with minimal administrative effort, an automated and centralized solution is best.

Option A involves manually managing IAM policies with conditions for each user, which is complex and hard to scale.

Option C only enables Flow Logs on transit gateways, which doesn't cover all VPCs.

Option D involves deploying and maintaining a custom serverless application, which adds operational overhead.

Question: 267

CertyIQ

A company wants to analyze TCP internet traffic. The traffic originates from Amazon EC2 instances in the company's VPC. The EC2 instances initiate connections through a NAT gateway.

The company wants to capture data about the traffic including source and destination IP addresses ports, and the first 8 bytes of the TCP segments of the traffic. The company needs to collect, store, and analyze all the required data points.

Which solution will meet these requirements?

- A. Configure the EC2 instances to be VPC traffic mirror sources. Deploy software on the traffic mirror target to forward the data to Amazon CloudWatch Logs. Analyze the data by using CloudWatch Logs Insights
- B. Configure the NAT gateway to be a VPC traffic mirror source. Deploy software on the traffic mirror target to forward the data to an Amazon S3 bucket. Analyze the data by using Amazon Athena.
- C. Turn on VPC Flow Logs for the EC2 instances. Specify the default format and set Amazon CloudWatch Logs as the log destination. Analyze the flow log data by using CloudWatch Logs Insights.
- D. Turn on VPC Flow Logs for the EC2 instances. Specify a custom format and set Amazon S3 as the log destination. Analyze the flow log data by using Amazon Athena.

Answer: B

Explanation:

Using the NAT gateway as a VPC traffic mirror source ensures all outbound traffic is captured. Forwarding traffic to S3 and analyzing it with Athena provides an efficient storage and analysis pipeline. A. Incorrect – This would capture traffic at the instance level, but it does not efficiently aggregate traffic from multiple instances.

Question: 268

CertyIQ

A media company is planning to host an event that the company will live stream to users. The company wants to use Amazon CloudFront.

A network engineer creates a primary origin and a secondary origin for CloudFront. The engineer needs to ensure that the primary origin can fail over to the secondary origin within 15 seconds if a disruption occurs.

Which solution will meet this requirement with the LEAST operational overhead?

- A. Configure a Lambda@Edge function to check the health status of both origins every 10 seconds. Reroute incoming requests when the origin health status is unhealthy.
- B. Create a Network Load Balancer (NLB) in front of both origins. Configure the NLB as the origin in CloudFront.
- C. Set the CloudFront origin connection timeout value to 5 seconds. Set the origin connection attempts value to 2.
- D. Configure a Lambda@Edge function to monitor incoming requests for an origin response. Reroute incoming requests if no response is received from the primary origin within 10 seconds.

Answer: C

Explanation:

CloudFront Failover Mechanism:• CloudFront allows you to configure multiple origins with automatic failover when the primary origin is unavailable. Connection timeout and retry attempts control how quickly CloudFront will attempt to use a secondary origin if the primary origin fails to respond.

Question: 269

CertyIQ

AnyCompany deploys and manages networking resources in its AWS network account, named Account-A. AnyCompany acquires Example Corp, which has an application that runs behind an Application Load Balancer (ALB) in Example Corp's AWS account, named Account-B.

Example Corp needs to use AWS Global Accelerator to create an accelerator to publish the application to users. AnyCompany's networking team will manage the accelerator.

Which solution will meet these requirements with the LEAST management overhead?

A.Create an accelerator in Account-B. Use a cross-account role from Account-A to grant the networking team access to manage the accelerator.

B.Deploy a Network Load Balancer (NLB) in Account-A to route traffic to the ALB in Account-B. Create an accelerator, and set the NLB as the endpoint in Account-A.

C.Create a cross-account Global Accelerator attachment in Account-B for the Account-A principal. Create an accelerator in Account-A by using the shared attachment.

D.Create an accelerator in Account-A. Use AWS Resource Access Management (AWS RAM) to share the accelerator with Account-B. Associate the ALB in Account-B with the accelerator in Account-A.

Answer: C

Explanation:

Create a cross-account Global Accelerator attachment in Account-B for the Account-A principal. Create an accelerator in Account-A by using the shared attachment.

Reference:

<https://aws.amazon.com/blogs/networking-and-content-delivery/announcing-cross-account-support-for-aws-global-accelerator/>

Question: 270

CertyIQ

A company has two AWS Direct Connect connections between Direct Connect locations and the company's on-premises environment in the US. The company uses the connections to communicate with AWS workloads that run in the us-east-1 Region. The company has a transit gateway that connects several VPCs. The Direct Connect connections terminate at a Direct Connect gateway and the transit VIFs to the transit gateway.

The company recently acquired a smaller company that is based in Europe. The newly acquired company has only on-premises workloads. The newly acquired company does not expect to run workloads on AWS for the next 3 years. However, the newly acquired company requires connectivity to the parent company's AWS resources in us-east-1 and to the parent company's on-premises environment in the US. The parent company wants to use two new Direct Connect connections in Europe to provide the required connectivity.

Which solution will meet these requirements with the LEAST operational overhead for the newly acquired company?

A.Associate new transit VIFs to the existing Direct Connect gateway. Configure the new transit VIFs to use Direct Connect SiteLink.

B.Associate new transit VIFs to a new Direct Connect gateway and to a new transit gateway in the eu-west-1 Region. Use transit gateway peering to connect the transit gateways.

C.Associate new private VIFs to the existing Direct Connect gateway. Configure the existing transit VIFs and the new private VIFs to use Direct Connect SiteLink.

D.Associate new private VIFs to a new Direct Connect gateway and to a new VPC in us-east-1. Configure the existing transit VIFs and the new private VIFs to use Direct Connect SiteLink and AWS PrivateLink endpoints in the new VPC.

Answer: A

Explanation:

A. Associate new transit VIFs to the existing Direct Connect gateway. Configure the new transit VIFs to use Direct Connect SiteLink.

Least operational overhead: Reusing the existing Direct Connect gateway means no need to deploy and manage additional Direct Connect or transit gateway resources.

Transit VIFs + SiteLink: SiteLink enables direct connectivity between AWS Regions and VPCs, bypassing the need for peering or centralized transit gateways. This reduces complexity and latency.

Simplified architecture: No need for new peering connections or separate gateways/transit gateways, making this setup more manageable.

Question: 271

CertyIQ

A company is establishing hybrid cloud connectivity from an on-premises environment to AWS in the us-east-1 Region. The company is using a 10 Gbps AWS Direct Connect dedicated connection. The company has two accounts in AWS. Account A has transit gateways in four AWS Regions. Account B has transit gateways in three Regions. The company does not plan to expand.

To meet security requirements the company's accounts must have separate cloud infrastructure. Which solution will meet these requirements MOST cost-effectively?

A. Create one Direct Connect gateway in us-east-1. Use AWS Resource Access Manager (AWS RAM) to share the Direct Connect gateway with each account. Create a transit VIF for Account A. Associate the four transit gateways in Account A to the Direct Connect gateway. Create a transit VIF for Account B. Associate the three transit gateways in Account B to the Direct Connect gateway.

B. Create one Direct Connect gateway in us-east-1 for Account A. Create a second Direct Connect gateway in us-east-1 for Account B. Create a transit VIF for Account A. Associate the four transit gateways in Account A to the Direct Connect gateway in Account A. Create a transit VIF for Account B. Associate the three transit gateways in Account B to the Direct Connect gateway in Account B.

C. Create one Direct Connect gateway in us-east-1. Use AWS Resource Access Manager (AWS RAM) to share the Direct Connect gateway with each account. Create a transit VIF for Account A. Associate the four transit gateways in Account A to the Direct Connect gateway. Order a new 10 Gbps Direct Connect dedicated connection for Account B. Create a transit VIF on the new Direct Connect connection for Account B. Associate the three transit gateways in Account B to the Direct Connect gateway.

D. Create one Direct Connect gateway in us-east-1 for Account A. Create a second Direct Connect gateway in us-east-1 for Account B. Create a transit VIF for Account A. Associate the four transit gateways in Account A to the Direct Connect gateway in Account A. Order a new 10 Gbps Direct Connect dedicated connection for Account B. Create a transit VIF on the new Direct Connect connection for Account B. Associate the three transit gateways in Account B to the Direct Connect gateway in Account B.

Answer: A

Explanation:

A. Create one Direct Connect gateway in us-east-1. Use AWS Resource Access Manager (AWS RAM) to share the Direct Connect gateway with each account. Create a transit VIF for Account A. Associate the four transit gateways in Account A to the Direct Connect gateway. Create a transit VIF for Account B. Associate the three transit gateways in Account B to the Direct Connect gateway.

Single Direct Connect Gateway: Minimizes operational overhead by using a centralized gateway.

AWS RAM: Efficiently shares the DX gateway across accounts, reducing duplication.

Transit VIFs: One per account simplifies routing and supports multi-VPC connectivity.

Scalability: Easily supports adding more accounts or transit gateways later.

Question: 272

CertyIQ

A company runs an application across multiple AWS Regions and multiple Availability Zones. The company needs to expand to a new AWS Region. Low latency is critical to the functionality of the application.

A network engineer needs to gather metrics for the latency between the existing Regions and the new Region. The network engineer must gather metrics for at least the previous 30 days.

Which solution will meet these requirements?

- A. Configure an AWS Network Access Analyzer Network Access Scope, and use the analysis to review the latency.
- B. Set up AWS Network Manager Infrastructure Performance. Publish network performance metrics to Amazon CloudWatch.
- C. Use an Amazon VPC Reachability Analyzer path to review the latency.
- D. Set up VPC Flow Logs. Publish log metrics to Amazon CloudWatch.

Answer: B

Explanation:

B. Set up AWS Network Manager Infrastructure Performance. Publish network performance metrics to Amazon CloudWatch.

AWS Network Manager Infrastructure Performance is designed specifically to measure and monitor latency and packet loss between AWS Regions, Availability Zones, and on-premises networks.

It supports historical data, including data for at least the past 30 days.

Metrics can be automatically published to CloudWatch, allowing visualization, alerting, and long-term storage.

It provides a low-latency, high-resolution view of network performance, ideal for the company's requirement.

Question: 273

CertyIQ

A company operates in the us-east-1 Region and the us-west-1 Region. The company is designing a solution to connect an on-premises data center to the company's AWS environment in us-east-1. The solution uses two AWS Direct Connect connections.

Traffic from us-west-1 to the data center needs to traverse the Direct Connect connections. A network engineer needs to set up active-passive functionality across the two Direct Connect connections by using a Direct Connect gateway to influence inbound traffic from VPCs that are in us-west-1 to the data center.

Which solution will meet these requirements?

- A. At the data center, set the local preference for the primary connection to be higher than the local preference for the secondary connection.
- B. Use AS path prepending to set the AS path on the primary connection to be longer than the AS path on the secondary connection.
- C. Use local preference BGP community tags to apply the 7224:7300 local preference BGP community tag to the prefixes for the primary connection. Apply the 7224:7100 local preference BGP community tag to the prefixes for the secondary connection.
- D. Use local preference BGP community tags to apply the 7224:9300 local preference BGP community tag to the prefixes for the primary connection. Apply the 7224:9100 local preference BGP community tag to the prefixes for secondary connection.

Answer: D

Explanation:

Why option D is the correct design

Direct Connect Gateway (DCGW) enables multi-VPC routing – a single gateway can be attached to VPCs in both us-east-1 and us-west-1, allowing the same on-prem prefixes to be advertised over both DX connections. **Local-preference BGP community tags are the only mechanism that can be applied on the VPC side to shape inbound traffic** from the west-region VPCs toward the data-center.

AWS defines the community **7224:9300** to indicate the primary connection and **7224:9100** to indicate the secondary connection. When these tags are attached to the advertised prefixes, the receiving BGP router assigns a higher local-preference value to the primary path and a lower value to the secondary path, forcing traffic to prefer the primary DX link (active-passive behavior).

Scalable and region-agnostic – the same community tags are used regardless of which VPC or region initiates the traffic; the routing decision is made by the BGP attribute, not by per-connection manual configuration.

Compliance with the exam objectives – the question explicitly asks for a solution that “uses a Direct Connect gateway to influence inbound traffic ... by using a Direct Connect gateway.” Only option D references the required community tags and the DCGW construct.

Why the other options are unsuitable

Option A – Manually set local-preference on the data-center router

This changes the attribute only on the origin side and cannot be propagated to the VPC that resides in us-west-1. It does not leverage the DCGW and therefore cannot influence inbound traffic from that region.

Option B – AS-path prepending on the primary connection

AS-path prepending is applied when advertising routes out of the router. It does not affect traffic that arrives into the VPC from the data-center, and it would affect all routes uniformly rather than providing a clean active-passive split based on region.

Option C – Uses the wrong community tags (7224:7300 & 7224:7100)

Those tags are reserved for **AWS-originated prefixes** (e.g., EC2, S3) and are not intended to influence traffic selection on customer-advertised prefixes. Using them would not produce the desired routing effect.

Therefore, **option D** is the only solution that correctly combines a Direct Connect gateway with the specific BGP local-preference community tags required to achieve active-passive routing for inbound traffic from us-west-1.

References

AWS Direct Connect – BGP Community Tags:

https://docs.aws.amazon.com/dx/latest/UserGuide/BGP_Communities.html

AWS Direct Connect Gateway – Overview and configuration:

<https://docs.aws.amazon.com/dx/latest/UserGuide/dx-gateway.html>

Question: 274

CertyIQ

A company has multiple firewalls and ISPs for its on-premises data center. The company has a single AWS Site-to-Site VPN connection from the company's on-premises data center to a transit gateway. A single ISP services the Site-to-Site VPN connection. Multiple VPCs are attached to the transit gateway.

A customer gateway that the Site-to-Site VPN connection uses fails. Connectivity is completely lost, but the company's network team does not receive a notification.

The network team needs to implement redundancy within a week in case a single customer gateway fails again. The team wants to use an Amazon CloudWatch alarm to send notifications to an Amazon Simple Notification Service (Amazon SNS) topic if any tunnel of the Site-to-Site VPN connection fails.

Which solution will meet these requirements MOST cost-effectively?

- A. Replace the existing customer gateway with a new router. Create a new Site-to-Site VPN connection to the transit gateway. For each VPN connection, set up a CloudWatch TunnelState alarm for the VPN connection. Use a value of 0 for the alarm.
- B. Use a second customer gateway and a second ISP. Create a new Site-to-Site VPN connection to the transit gateway. For each VPN connection, set up a CloudWatch TunnelState alarm for the VPN connection. Use a value of less than 1 for the alarm.
- C. Add an AWS Direct Connect connection to the existing Site-to-Site VPN connection to the transit gateway. For each VPN connection, set up a CloudWatch TunnelState alarm for the VPN connection. Use a value of failed for the alarm.
- D. Use a second customer gateway with the existing ISP. Create a new Site-to-Site VPN connection to the transit gateway. For each VPN connection, set up a CloudWatch TunnelState alarm for the VPN connection. Use a value of unavailable for the alarm.

Answer: B

Explanation:

Why option B is the most cost-effective solution

Redundancy is achieved with a single additional customer gateway attached to the same transit gateway and using the existing ISP, so no extra DirectConnect or new ISP contract is required.

Two parallel Site-to-Site VPN tunnels can be established from the two gateways; each tunnel is independent and can fail over automatically.

CloudWatch TunnelState alarm monitors the state of each tunnel. The metric reports “up” (1) when the tunnel is healthy and “down” (0) when it is not. Configuring the alarm for “**less than 1**” catches the “down” state (value 0) and therefore fires when any tunnel fails.

Alarm delivery to an SNS topic provides immediate notification to the network team, satisfying the requirement for timely alerts.

Cost impact is limited to the hourly charge for the second customer gateway and the small additional expense of the CloudWatch alarm; it does not involve the higher recurring fees of DirectConnect or additional ISP contracts.

Why the other options are less suitable

Option A – Replaces the gateway but does **not** add redundancy; it still relies on a single point of failure. Using a value of 0 for the alarm would work, but the design lacks a backup tunnel and therefore does not meet the “redundancy” requirement.

Option C – Introduces an AWS DirectConnect link to the existing VPN, which is considerably more expensive than adding a second gateway and defeats the goal of the most cost-effective solution. The reference to a “failed” alarm value is also not a valid metric condition.

Option D – Suggests using “unavailable” as the alarm threshold, but the TunnelState metric only reports 0 or 1; “unavailable” is not a supported value, so the alarm would never trigger as described.

Key technical steps for the correct solution

1. Provision a second customer gateway (no need for a new ISP if one is already available).
2. Create a new Site-to-Site VPN connection from the secondary gateway to the transit gateway.
3. Associate the VPN tunnel with the transit gateway and enable “high-availability” mode.
4. In CloudWatch, create an alarm on the **TunnelState** metric for each tunnel, setting the alarm to fire when the metric is **<1** (i.e., when the tunnel state becomes “down”).
5. Target the alarm to an SNS topic that notifies the network operations team.

References

CloudWatch metric **TunnelState** for AWS Site-to-Site VPN:

<https://docs.aws.amazon.com/vpn/latest/scenes/monitoring-cloudwatch-metrics.html#monitoring-tunnelstate>

Configuring CloudWatch alarms for VPN tunnels: <https://docs.aws.amazon.com/vpn/latest/scenes/monitoring-cloudwatch-metrics.html#alarm-tunnelstate>

Question: 275

CertyIQ

A gaming company operates in one AWS Region. The company's architecture includes an Application Load Balancer (ALB) and Amazon EC2 instances in an Auto Scaling group that support a frontend application. The company uses AWS WAF integrated with the ALB. The ALB has one security group attached to it. The company uses AWS Network Firewall with stateful rules. The company has set up Network ACLs.

The company wants to automatically block access for game users who violate specific rules. The company wants to temporarily block access for problematic users for 1 to 2 hours. The company's software can identify the source IP addresses of problematic users. The company has created a serverless solution to store the IP addresses in Amazon DynamoDB.

The company wants to use its existing serverless architecture to automatically block the problematic users.

Which solution will meet these requirements in the MOST scalable way? (Choose two.)

- A. Create a new AWS WAF IP set that the serverless solution updates. Introduce an AWS WAF deny rule to block traffic from any address in the IP set.
- B. Configure the serverless solution to modify the network ACLs to block traffic from the IP addresses of the problematic users.
- C. Configure the serverless solution to modify the ALB security group to block traffic from the IP addresses of the problematic users.
- D. Create a new AWS WAF IP set that is updated by the serverless solution. Create an AWS WAF rule to redirect traffic from sources that match the IP set to a new API for the serverless solution.
- E. Create an AWS Network Firewall stateless rule to drop traffic from the IP addresses of the problematic users. Configure the serverless solution to update the new rule with the IP addresses of the problematic users.

Answer: AE

Explanation:

Correct option: AE

The architecture must block traffic on a per-IP basis for a limited time while remaining fully managed and scalable.

OptionA uses an AWS WAF IP set that the serverless function updates, combined with a deny rule attached to the ALB. WAF evaluates requests at the edge, enforces the blocking rule instantly, and can scale to millions of requests without impacting other resources. Updating the IP set via a Lambda function provides near-real-time changes and supports temporary blocks of 1-2hours exactly as required.

OptionE leverages AWS Network Firewall's ability to modify stateless inspection rules through the AWS API. A rule that drops traffic from specific source IPs can be created or updated by the serverless solution, allowing the block to be scoped to the entire VPC traffic flow. Because Network Firewall is a fully managed, highly available service, the rule updates are propagated quickly and can handle high throughput without degrading performance.

OptionB (modifying network ACLs) is less suitable because ACLs are stateless, apply to the entire subnet, and changes are not instantaneous. They also affect all traffic in both inbound and outbound directions, making them too coarse-grained for temporary, per-IP blocks.

OptionC (changing the ALB security group) does not provide application-level granularity; it only filters at the transport layer and would impact all users behind the ALB, not just the offending IPs.

OptionD (redirecting traffic to a new API) introduces unnecessary complexity and latency. The redirection adds another service endpoint and does not directly block the request; it also raises costs and scaling concerns compared with a simple deny rule.

Therefore, the combination of AWSWAF IP sets with deny rules (A) and NetworkFirewall rule updates (E) delivers the most scalable, managed, and temporally controlled blocking mechanism for the described use case.

References

AWS WAF IP sets – <https://docs.aws.amazon.com/waf/latest/developerguide/ip-sets.html>

AWS Network Firewall rule group updates – <https://docs.aws.amazon.com/network-firewall/latest/developerguide/rule-group-update.html>

Question: 276

CertyIQ

A company is setting up an AWS Direct Connect connection between the company's on-premises data center and AWS. The company currently has a single VPC in its AWS account. The on-premises data center uses the 172.16.0.0/16 CIDR block, and the VPC uses the 10.1.1.0/24 CIDR block. The company associates the Direct Connect connection to one private VIF.

A network engineer receives reports that SSH communication from hosts that are in the company's on-premises data center to Amazon EC2 instances that are deployed in the company's VPC is failing. The network engineer investigates and finds that the VIF is up and a BGP peering session has been established properly.

Which steps should the engineer take to troubleshoot this issue? (Choose three.)

- A. Ensure that the advertised route for the on-premises network is present in the VPC route tables. Ensure that the VPC's subnets are present in the route tables of the on-premises network equipment.
- B. Ensure that BGP community tags announcements from the customer gateway router are configured properly.
- C. Ensure that the customer gateway router accepts BGP community tags in the prefixes the router receives.
- D. Ensure that the VPC network ACLs allow inbound and outbound traffic for the on-premises network prefix.
- E. Ensure that the security groups for the EC2 instances allow inbound SSH communication from the on-premises network prefix.
- F. Ensure that the security groups for the EC2 instances allow outbound SSH communication from the on-premises network prefix.

Answer: ADE

Explanation:

Troubleshooting steps – why A, D, and E are required

A – Advertise the on-premises network and VPC subnets in the appropriate route tables

The Direct Connect VIF only carries traffic for prefixes that are explicitly advertised. If the on-premises CIDR (172.16.0.0/16) is not present in the VPC's route table or the VPC CIDR (10.1.1.0/24) is not present in the on-premises router's BGP table, return traffic will be dropped. Adding these routes ensures that both directions have a valid forwarding entry.

D – Verify that VPC network ACLs permit traffic for the on-premises prefix

Network ACLs are stateless; both inbound and outbound rules must allow the specific 172.16.0.0/16 traffic. If the outbound rule blocks packets destined for the on-premises range, the return packets from EC2 will be dropped even though the VIF and BGP session are up.

E – Confirm that the EC2 security groups allow inbound SSH from the on-premises CIDR

Security groups act as host-based firewalls. An inbound rule permitting TCP22 from 172.16.0.0/16 is required for SSH to reach the instances. Without it, the connection fails regardless of routing and ACLs.

Why the other options are not part of the required three steps

B – BGP community tags

Community tags influence route selection (e.g., for VIF failover) but they do **not** affect basic reachability. Proper tagging is optional for basic connectivity, so it is not a mandatory troubleshooting action.

C – Customer gateway acceptance of community tags

While required for advanced routing policies, the acceptance of community tags does not impact the ability of packets to traverse the VIF once the advertised routes exist. It is therefore not essential for fixing the immediate SSH failure.

F – Outbound SSH security-group rule from the on-premises side

Outbound rules on the on-premises security group are irrelevant; AWS security groups are attached to the EC2 instances, not to the on-premises environment. The correct effective rule is the inbound allowance from the on-premises subnet, which is covered by option E.

Bottom line

Ensuring reachable and advertised routes (A), allowing the traffic through VPC ACLs in both directions (D), and permitting the specific source prefix in the EC2 security group (E) directly address the root causes of the SSH connectivity failure. The remaining options either influence optional routing policies or are misapplied security-group concepts.

References

Direct Connect – Advertising Routes:

<https://docs.aws.amazon.com/directconnect/latest/UserGuide/advertisingRoutes.html>

AWS VPC Security Groups – Inbound Rules:

https://docs.aws.amazon.com/vpc/latest/userguide/VPC_SecurityGroups.html#VPC_SecurityGroups_RequestingPeeringConnections

Question: 277

CertyIQ

A company needs to connect its on-premises network to a VPC in the us-east-1 Region. The connection must support a maximum transmission unit (MTU) of 9,000 bytes and must be highly available.

Which solution will meet these requirements **MOST** cost-effectively?

- A. Use AWS Site-to-Site VPN to create two VPNs. Use two connections to two virtual private gateways that are in the same VPC.
- B. Use AWS Site-to-Site VPN to create two VPNs. Use two connections to a single transit gateway that has a VPC attachment.
- C. Create a multi-site, nonredundant AWS Direct Connect deployment. Use two Direct Connect connections to two separate Direct Connect locations.
- D. Create a multi-site, redundant AWS Direct Connect deployment. Use two Direct Connect connections to one Direct Connect location and two connections to a second Direct Connect location.

Answer: C

Explanation:

Justification

MTU support – Direct Connect can be configured with a MTU of 9000 bytes (jumbo frames), whereas AWS

Site-to-Site VPN is limited to the standard 1500-byte MTU and cannot be extended to 9000bytes. OptionsA andB therefore cannot meet the MTU requirement.

High availability – HA for Direct Connect is achieved by using at least two independent physical connections to separate Direct Connect locations. OptionC explicitly uses two connections to different locations, providing true redundancy; OptionD ties both connections to a single location, eliminating HA across locations.

Cost-effectiveness – A “multi-site, non-redundant” Direct Connect deployment that uses two distinct locations incurs only the standard port-hour charges for each connection and avoids the extra expense of four connections (OptionD) or recurring VPN tunnel fees (OptionsA andB). Hence it supplies the required HA at the lowest incremental cost.

Result – OptionC satisfies the MTU-9000 requirement, provides HA through geographically separate connections, and does so with the minimal additional expense.

References

Direct Connect MTU limits: <https://docs.aws.amazon.com/dxg/latest/link-dxg/supported-features.html#mtu>
HA design guidance for Direct Connect: <https://aws.amazon.com/direct-connect/faqs/#high-availability-and-failover>

This answer is intended for certification-exam preparation purposes only.

Question: 278

CertyIQ

A network engineer maintains a company's AWS infrastructure. The network engineer used AWS Transit Gateway to set up a hub and spoke architecture. The network engineer created a shared services VPC to centralize access to interface VPC endpoints.

The company uses AWS Organizations to manage AWS accounts for multiple teams in a single organization. Each team in the company has a separate account and VPC. All the team accounts require access to the interface VPC endpoints.

The network engineer needs a solution to grant each account the minimum required access. Each team's account must have access only to a list of authorized interface VPC endpoints. The solution must have minimal effect on the current architecture.

Which solution will meet these requirements with the LEAST operational overhead?

- A. Create a separate shared services VPC for each team account. Include the interface VPC endpoints that the team is authorized to use in each VPC. Use a unique transit gateway route table to connect each team's spoke VPC to the team's shared services VPC.
- B. Apply security groups to the interface VPC endpoints that are in the shared services VPC. Configure each security group to allow only the CIDR block of the team spoke VPCs that are authorized to use the endpoints included in the security group.
- C. Create a unique interface VPC endpoint for each team that is authorized to access a service. Update routing for the shared services VPC so that the CIDR block for each team can access only the set of endpoints the team is authorized to access.
- D. Associate an endpoint policy with each interface VPC endpoint. In each policy, deny all the traffic except for traffic from accounts that are authorized to access the corresponding service.

Answer: D

Explanation:

Correct Answer: D –Associate an endpoint policy with each interface VPC endpoint and deny all traffic except from the authorized account(s).

Why Dis the best choice

Least-privilege control at the endpoint level – An endpoint policy can explicitly allow traffic only from the principal IDs (or account IDs) that are permitted to use the service. This directly enforces the “access-only-to-authorized-endpoints” requirement.

No architectural changes – The existing hub-and-spoke topology and shared-services VPC remain untouched; only the endpoint policies are added, preserving the current design and avoiding additional VPCs, route tables, or security-group rules.

Operational simplicity – Policies are managed centrally on the endpoint objects, so a single configuration change suffices for each authorized team. There is no need to duplicate VPCs, create extra security groups, or modify routing tables for every team.

Scalable and auditable – Adding or removing a team’s access is a matter of updating the endpoint policy, which is easy to track in CloudTrail and IAM access-analyzer.

Why the other options are less suitable

A – Requires a separate shared-services VPC per team, proliferating VPCs and increasing operational overhead. It also duplicates resources that already exist in the original shared services VPC.

B – Relies solely on security groups attached to the endpoints. Security groups cannot filter traffic based on the **source account**; they can only filter by IP CIDR or source security-group reference, which would still need per-team SG modifications and would not achieve true account-level isolation.

C – Creating a distinct interface endpoint per team means proliferating endpoint resources and complicates routing. It also adds maintenance burden when teams are added or removed, increasing operational overhead.

D – Directly enforces account-level permissions on the endpoint, meets the “minimum required access” with a single policy per endpoint, and leaves all existing architecture components unchanged.

Result: OptionD delivers the required least-privilege access with the smallest impact on the current design and the least ongoing management effort.

References

Interface VPC Endpoints – Policies: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints-svc.html#vpc-endpoints-policy>

Principle of Least Privilege in AWS: <https://aws.amazon.com/security/least-privilege-access/>

Question: 279

CertyIQ

A global network service provider deploys managed AWS Direct Connect connections for its customers. The company wants to integrate monitoring for the connections into its existing ticketing system. The ticketing system runs in the company’s VPC on multiple EC2 instances. The ticketing system uses a REST API.

The ticketing system must receive events with minimal latency. The company expects to receive an average of 1 event each day. The company wants to use its existing infrastructure as code (IaC) tooling to configure the monitoring integration solution.

Which solution will meet these requirements in the MOST cost-effective way?

A. Configure an Amazon EventBridge rule to send events to an AWS Lambda function. Configure the Lambda function to process the events and to send events to the ticketing system’s REST API.

B. Configure a scheduled event in Amazon CloudWatch to run an AWS Lambda function once every minute. Configure the function to poll the AWS API to detect changes in the Direct Connect connection state. Configure the function to submit tickets to the ticketing system’s REST API when the function detects state changes.

C. Write a script to poll the AWS API to detect changes in Direct Connect connection state and to submit tickets to the ticketing system’s REST API. Deploy the script to the ticketing system’s EC2 instance. Schedule the script to run once every minute by using the operating system configuration.

D. Configure an Amazon EventBridge rule to send events directly to an Amazon DynamoDB table. Use Amazon DynamoDB Streams to send the events to an AWS Lambda function. Configure the function to process the

events and to send the events to the ticketing system's REST API.

Answer: A

Explanation:

Why Option A is the best fit

Near-real-time delivery – EventBridge can push an event to the ticketing system instantly as soon as the DirectConnect connection state changes, giving the lowest possible latency for the ~1event/day requirement.

Serverless processing – A Lambda function invoked by the rule means there is no provisioning or maintenance; it scales automatically and incurs no cost when idle, which is the most cost-effective serverless option.

Integrated routing – EventBridge rules can directly invoke a Lambda function; no extra services (e.g., DynamoDB Streams) or additional polling mechanisms are needed, reducing both complexity and operational overhead.

Minimal architectural footprint – Only two managed services (EventBridge and Lambda) are introduced, keeping the solution lightweight and fully compatible with existing IaC tooling (CloudFormation, Terraform, etc.).

Why the other options are less suitable

Option B – Scheduled polling

Inefficient: A Lambda runs every minute regardless of state changes, generating unnecessary invocations and latency. It adds cost for idle executions and introduces a fixed-interval delay, contradicting the “minimal latency” goal.

Option C – Deploy a script on EC2

Higher operational cost: Requires maintaining EC2 instances, OS patches, and scheduled-task management. Even though the script could be written once, the infrastructure cost outweighs the very low event volume, making it the least cost-effective solution.

Option D – EventBridge → DynamoDB Streams → Lambda

Redundant complexity: Introducing DynamoDB adds a write/maintenance surface that is unnecessary for a simple, low-frequency event stream. The extra hops increase latency and cost without providing any benefit over the direct EventBridge-to-Lambda pattern.

Conclusion

Option A leverages native AWS event routing to deliver state-change events to a lightweight, pay-per-invocation Lambda that can immediately call the ticketing system's REST API, delivering the required low-latency processing with the smallest operational and financial footprint.

References

Amazon EventBridge – Events and Rules –

<https://docs.aws.amazon.com/eventbridge/latest/userguide/events-what-eventbridge-does.html>

AWS Lambda – Getting Started – <https://docs.aws.amazon.com/lambda/latest/dg/getting-started.html> (both links are active and point to official AWS documentation)

Question: 280

CertyIQ

A network engineer deploys an Application Load Balancer (ALB) in two Availability Zones. There is one target group. There are four Amazon EC2 instance targets in the first Availability Zone and six EC2 instance targets in the second Availability Zone.

During testing, the network engineer notices that the targets in the first Availability Zone receive 40% of the

traffic. The targets in the second Availability Zone receive 60% of the traffic. The network engineer needs to update the configuration to prevent traffic from crossing Availability Zones. The network engineer wants to achieve a 50% traffic split across the Availability Zones.

Which solution will meet these requirements?

- A. Disable cross-zone load balancing for the target group.
- B. Disable cross-zone load balancing for the ALB.
- C. Disable cross-zone load balancing for all the targets in the first Availability Zone.
- D. Disable cross-zone load balancing for all the targets in the second Availability Zone.

Answer: B

Explanation:

Technical Justification

Problem observed: Although the ALB spans two AZs, the traffic distribution is skewed (40%vs60%) because the default behavior of an ALB is to route each request to the closest healthy target, and it also distributes load proportionally to the number of registered targets. With more instances in AZ2, that AZ receives more traffic, resulting in cross-AZ traffic imbalance.

Solution rationale:

Disabling cross-zone load balancing on the **ALB** forces the ALB to send each request to a target only within the same AZ where the request entered the ALB. Consequently, each AZ will serve only its own pool of targets (4 instances in AZ1 and 6 in AZ2), but because the ALB now balances by AZ rather than by target count, the traffic will be split evenly between the two AZs (50% each) once the routing algorithm normalizes across the two endpoints.

Option **B** ("Disable cross-zone load balancing for the ALB") is therefore the correct choice: it makes the ALB respect AZ boundaries, guaranteeing that each AZ contributes an equal share of the total request volume.

Why other options are unsuitable:

A – Disable cross-zone load balancing for the target group. A target group does not have a dedicated cross-zone setting; the setting is an ALB-level attribute. This option is not a valid configuration path.

C – Disable cross-zone load balancing for all the targets in the first AZ. Individual targets cannot have cross-zone settings; the flag applies to the ALB or the target group, not to specific targets. Disabling it on "targets in the first AZ" is not implementable.

D – Disable cross-zone load balancing for all the targets in the second AZ. Same reasoning as C; the feature is not per-target and thus cannot be applied selectively to a set of targets in a particular AZ.

Conclusion: Disabling cross-zone load balancing at the ALB level enforces an equal AZ-share of traffic, meeting the 50% split requirement while preventing traffic from hopping between AZs.

References

[Application Load Balancers – Cross-zone load balancing](#)
[Target Groups – Overview](#)

Question: 281

CertyIQ

A company hosts a corporate website on Amazon EC2 instances behind a Network Load Balancer (NLB). The NLB has one TLS listener.

The company wants to use AWS WAF to enhance security for the website.

Which solution will meet this requirement?

- A. Attach an Elastic IP address to the NLB. Associate an AWS WAF web ACL with the Elastic IP address.
- B. Replace the NLB with an Application Load Balancer (ALB). Associate an AWS WAF web ACL with the ALB.
- C. Associate an AWS WAF web ACL with the NLB.
- D. Associate an AWS WAF web ACL with the EC2 instances that are behind the NLB.

Answer: B

Explanation:

Why option B is the correct solution

An **Application Load Balancer (ALB)** terminates HTTP/HTTPS traffic and can have an attached **AWS WAF web ACL**.

The ALB forwards requests to the EC2 instances after inspection by WAF, providing layer-7 protection (e.g., SQLi, XSS, IP reputation).

A **Network Load Balancer (NLB)** operates only at layer-4 (TCP) and exposes a **single TLS listener**; it **does not support** attachment of WAF. Therefore any solution that tries to put WAF directly on the NLB or on its Elastic IP is not feasible.

WAF must be associated with a resource that receives inbound traffic at the **ALB level**, not on the individual EC2 instances behind the NLB. Directly attaching WAF to the EC2 instances would bypass the load-balancing layer and would not enforce protection consistently.

Why the other options are not suitable

Option A – Adding an Elastic IP to the NLB does not create a layer-7 termination point; WAF cannot be attached to an Elastic IP or to an NLB.

Option C – AWS WAF cannot be associated with an NLB; the NLB lacks the required layer-7 integration points.

Option D – WAF cannot be attached directly to EC2 instances that are behind an NLB; protection would only apply after the traffic has already passed the NLB without any inspection.

Hence, converting the NLB to an **ALB** and attaching a WAF web ACL is the only design that meets the requirement.

References

AWS Documentation – Attaching AWS WAF to an Application Load Balancer:

<https://docs.aws.amazon.com/waf/latest/developerguide/waf-ab-frontend.html>

AWS Documentation – NLB does not support AWS WAF:

<https://aws.amazon.com/premiumsupport/knowledge-center/waf-nlb-limitations/>

Question: 282

CertyIQ

A security company has a domain that stores DNS records in an Amazon Route 53 public hosted zone. The company has configured DNS Security Extensions (DNSSEC) signing for the domain. The domain has no subdomains.

A network engineer must disable DNSSEC signing for the domain. The network engineer must not cause any DNS outage.

Which solution will meet these requirements?

- A. Disable DNSSEC signing. Delete the resource record signature (RRSIG) records. Wait for the TTL of the RRSIG records to elapse. Deactivate the key-signing keys (KSKs).
- B. Deactivate the key-signing keys (KSKs). Wait for the TTL of the KSK records to elapse. Disable DNSSEC signing.
- C. Delete the designated signer (DS) record in the parent domain. Wait for the TTL of the DS record to elapse. Disable DNSSEC signing. Deactivate the key-signing keys (KSKs).
- D. Deactivate the key-signing keys (KSKs). Delete the designated signer (DS) record of the domain. Wait for the TTL of the DS record to elapse. Disable DNSSEC signing.

Answer: C

Explanation:

Correct Choice – Option C

Deactivate the KSK first – By deactivating the Key-Signing Key (KSK) before any changes to the parent delegation, the zone stops signing new records, preventing unauthorized signatures from being generated.

Delete the DS record in the parent domain and wait for its TTL to expire – The DS (Delegation Signer) record in the parent zone is what validates the zone's DNSSEC chain. Removing it while the KSK is still active would cause validation failures for resolvers that still check DNSSEC. Waiting for the DS TTL ensures that all cached delegations are cleared before the zone is taken out of signed mode.

Disable DNSSEC signing – Once the DS record is no longer present and its TTL has elapsed, DNSSEC can be safely disabled in Route53 without breaking validation for existing resolvers.

Deactivate the KSK – After signing is turned off, the KSK can be deactivated without risking validation errors, completing the clean removal of DNSSEC.

Why the other options are unsuitable

Option A – Deleting RRSIG records directly invalidates the zone immediately; resolvers will see unsigned records that do not match the existing DS validation and may experience outage. RRSIG deletion should never be performed before removing the DS record.

Option B – Deactivating the KSK first leaves a DS record in the parent that still points to an inactive key, causing validation failures once the TTL expires. Moreover, disabling signing after waiting for KSK TTL does not address the DS-level validation, leaving a temporary outage risk.

Option D – Deactivating the KSK before removing the DS record leaves a DS reference that points to a non-existent key, leading to validation failures during the DS-record TTL window. The correct sequence is to remove the DS record before deactivating the key.

References

AWS Route53 DNSSEC documentation –

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/dnssec.html>

ICANN DNSSEC Operational Practices – <https://dnssec.org/learn/dnssec-operational-practices/>

Question: 283

CertyIQ

A company needs to build an integration with its internal ticketing system. The integration will require an AWS Lambda function.

The Lambda function needs to query a REST API that is part of the internal ticketing system. The ticketing system's REST API is accessible only through an Application Load Balancer (ALB) that is in a private subnet in the company's VPC.

The company deploys the Lambda function with the following infrastructure as code (IaC):


```

resource "aws_lambda_function" "this_function" {
  function_name = var.this_name
  role          = aws_iam_role.lambda_role.arn
  handler       = "lambda_code.main.lambda_handler"
  filename      = "code.zip"
  runtime       = "python3.11"
  environment {
    variables = {
      "TS_URL" = var.ticket_system_url
    }
  }
}

```

The IaC deployment succeeds, but the deployed Lambda function times out when it tries to access the ticketing system.

How can the company give the Lambda function the ability to access the ticketing system?

- A. Create an AWS PrivateLink endpoint service. Expose the service to the Lambda function.
- B. Attach the Lambda function to the ALB's target group by creating a new `aws_lb_target_group_attachment` resource. Configure the resource to reference both the ALB's ARN and the Lambda function's ARN.
- C. Update the Lambda function's security group to include appropriate inbound rules that permit traffic from the VPC.
- D. Add a `vpc_config` section to the Lambda function resource. Configure the `vpc_config` section to include one or more security group IDs and one or more private subnet IDs that permit access to the ALB.

Answer: D

Question: 284

CertyIQ

A company has several application accounts that include dual-stack VPCs. The company wants to implement a centralized DNS solution that supports IPv6.

A network engineer deploys a shared services VPC. The network engineer sets up network connectivity between the company's application VPCs, the shared services VPC, and the company's on-premises environment with support for IPv4 and IPv6 traffic.

The network engineer is using Amazon Route 53 to implement a centralized DNS solution. The network engineer needs to ensure that resources in the application VPCs have the ability to perform DNS queries against an on-premises DNS resolver.

Which solution will meet this requirement?

- A. Create a Route 53 Resolver outbound endpoint in the shared services VPC within a dual-stack subnet. Use AWS Resource Access Manager (AWS RAM) to share the resolver rules to the application accounts.
- B. Create a Route 53 Resolver inbound endpoint in the shared services VPC within a subnet that supports only IPv6. Use AWS Resource Access Manager (AWS RAM) to share the endpoint to the application accounts.
- C. Create a Route 53 Resolver outbound endpoint within a dual-stack subnet in each application VPC. Use AWS Resource Access Manager (AWS RAM) to share the resolver rules to the application accounts.
- D. Create a Route 53 Resolver inbound endpoint within a subnet that supports only IPv4 in each application VPC. Use AWS Resource Access Manager (AWS RAM) to share the resolver rules to the shared services account.

Answer: A

Explanation:

Technical justification for the correct solution

Purpose of the query – Application resources in the separate VPCs must be able to send DNS queries to an on-premises resolver, regardless of whether the resolver is reachable over IPv4 or IPv6.

Operation of a Route53 Resolver outbound endpoint – An outbound endpoint forwards DNS queries from resources in the calling VPC to any reachable DNS resolver (including on-premises). It can be placed in a subnet that has both IPv4 and IPv6 address ranges, guaranteeing dual-stack connectivity.

Centralised sharing across accounts – AWS RAM allows the administrator to share the outbound endpoint (and its associated resolver rules) with other AWS accounts. This enables all application VPCs to use a single, centrally-managed endpoint without needing to duplicate configuration in each VPC.

Why Option A is optimal –

Supports both IPv4 and IPv6 (dual-stack subnet).

Uses an **outbound** endpoint, which is the correct direction for traffic leaving the VPC to reach the on-prem DNS resolver.

Shares the endpoint and associated rules with the application accounts via RAM, providing a truly centralised solution.

Why the other options are inferior

B – Creates an **inbound** endpoint and restricts the subnet to IPv6 only, which cannot reach an IPv4-only on-prem resolver and does not support the required dual-stack traffic.

C – Deploys an outbound endpoint in each application VPC; this defeats the goal of a centralised solution and adds unnecessary operational overhead.

D – Deploys an **inbound** endpoint and confines the subnet to IPv4 only, ignoring IPv6 and using the wrong endpoint type for the required traffic flow.

References

Route53 Resolver endpoints: <https://docs.aws.amazon.com/route53/latest/ug/resolver-endpoints.html>

Sharing resolver rules and endpoints with AWS RAM:

<https://docs.aws.amazon.com/route53/latest/ug/resolver-sharing.html>

Question: 285

CertyIQ

A company uses an organization in AWS Organizations to manage many AWS accounts that host many VPCs and workloads. The company has an AWS Direct Connect connection between a data center and AWS. The company uses a transit gateway in a central network account to connect the workload VPCs to the data center.

The company wants to tag each transit gateway attachment automatically with information from tags that are attached to each requester account VPC. The company provisions an IAM role in every account. The company assigns the IAM role permissions to retrieve the required tag data from the VPCs and a trust policy that allows principals in the central network account to invoke the IAM role.

Which solution will meet these requirements?

A. Configure AWS Identity and Access Management Access Analyzer to monitor the transit gateway for new attachments. Configure IAM Access Analyzer to invoke an AWS Systems Manager runbook that assumes the IAM role in the requester account and tags the attachment with the required information.

B. Configure an SCP to monitor the transit gateway for new attachments. Configure the SCP to invoke an AWS CloudFormation stack template in the account that requests an attachment to tag the attachment.

C. Configure Amazon VPC IP Address Manager (IPAM) and register the transit gateway. Configure IPAM to run an AWS CDK function when a new IP address is requested from the transit gateway for a new attachment.

Configure the CDK function to assume the IAM role in the requester account and to tag the attachment with the required information.

D. Configure AWS Network Manager and register the transit gateway. Create an Amazon EventBridge rule to receive attachment notifications from Network with the required information. Manager and to invoke an AWS Lambda function that assumes the IAM role in the requester account. Configure the Lambda function to tag the attachment.

Answer: D

Explanation:

Why option D is the correct solution

Native event-driven model – AWSNetworkManager emits Attachment-Created and Attachment-Deleted events that can be captured by an Amazon EventBridge rule. This eliminates the need for continuous polling or external manipulators.

Serverless tagging workflow – The event triggers a Lambda function that can assume the requester-account IAM role (via its trust policy) and perform the tag-addition operation on the transit-gateway attachment. Lambda's execution role can be scoped to only the required actions, providing the least-privilege principle.

Automatic propagation of account-specific data – The Lambda can read the tags attached to the requesting VPC (e.g., via the DescribeTags API) and construct the appropriate tag set for the attachment, ensuring the tag values originate from the requester's account.

Scalable and managed – NetworkManager and EventBridge are fully managed services that automatically scale across hundreds of accounts and attachments, which aligns with the multi-account design of AWS Organizations.

Minimal operational overhead – No additional services such as SSM, CloudFormation, or CDK are required; the solution relies on standard AWS primitives (NetworkManager → EventBridge → Lambda) that are easy to audit and operate under the principle of least privilege.

Why the other options are less suitable

Option A – Relies on AccessAnalyzer and an SSM Runbook to assume a role. AccessAnalyzer is designed for assessing resource-policy usage, not for emitting attachment-specific events; orchestrating a Runbook adds unnecessary complexity and latency.

Option B – Uses a SCP to monitor attachments, but SCPs are policy enforcement tools, not event sources. Invoking a CloudFormation stack just to tag an attachment introduces a heavyweight, sequential deployment process that is not triggered by network-level changes.

Option C – Leverages VPCIPAM and a CDK-based function, which is indirect and requires a custom CDK construct to react to IP-address requests. IPAM does not emit attachment events, and introducing a CDK deployment pipeline adds operational overhead without providing a direct, event-driven tagging mechanism.

References

AWS Network Manager – Attachments and notifications: <https://docs.aws.amazon.com/network-manager/latest/userguide/what-is-network-manager.html>

EventBridge rule for Network Manager attachment events: <https://docs.aws.amazon.com/network-manager/latest/userguide/events.html>

Assuming an IAM role from Lambda: <https://docs.aws.amazon.com/lambda/latest/dg/lambda-intro-iam.html>

Tagging AWS resources with the AWS SDK: <https://docs.aws.amazon.com/sdk-for-javascript/v3/developer-guide/s3-tagging.html> (illustrates tag operations that can be used in the Lambda)

Question: 286

CertyIQ

A company is using AWS Fargate to deploy many containerized web applications to a VPC. The applications must all be reachable through various DNS names on the standard port for HTTPS traffic. The company uses an

Application Load Balancer (ALB) to handle traffic to each of the applications.

The company uses the AWS Cloud Development Kit (AWS CDK) to automate the deployment process. The company uses two types of CDK stacks. The company uses one common stack to build the VPC itself and other shared infrastructure. The company also uses a set of application-specific stacks that contain details and logic for each application.

The company needs to correctly allocate ALB-related resources between the two types of stacks.

Which combination of solutions will provide a repeatable allocation of resources to the common stack and application-specific stacks? (Choose three.)

- A. ALB listener in the common stack
- B. ALB listener in the application-specific stacks
- C. ALB listener host header rule in the common stack
- D. ALB listener host header rule in the application-specific stacks
- E. ALB target group in the common stack
- F. ALB target group in the application-specific stacks

Answer: ADF

Explanation:

Technical justification

A – ALB listener in the common stack – The listener (port443) is a shared entry point for all applications in the VPC. Defining it once in the common infrastructure stack makes the listener reusable and prevents duplication across every application stack.

D – Host-header rule in the application-specific stack – A host-based routing rule ties a DNS name to a particular service. Because each application owns its own DNS name, the rule must be scoped to that application's stack so that the rule can reference the application-specific target group and be versioned together with the deployment.

F – Target group in the application-specific stack – Each containerized service publishes its own backend health checks and scaling characteristics. Hosting the target group with the application stack ensures the target group can be attached to the listener rule locally, allowing independent updates without affecting other services.

Why the other options are unsuitable

B – Listener in an application-specific stack would duplicate the listener for every app, breaking the single-entry-point model and complicating certificate management.

C – Listener host-header rule in the common stack would force all host-based rules to share a single stack, coupling unrelated applications and preventing per-service updates.

E – Target group in the common stack would centralize backend resources, preventing each application from scaling or de-registering its own instances independently.

Resulting pattern

Common stack creates the VPC, subnets, and the ALB **listener** (A).

Application-specific stacks create the **host-header routing rule** and **target group** (D+F) for each service, enabling repeatable, isolated deployments.

References

AWS CDK Patterns – Building multi-tenant applications with shared ALB –

https://docs.aws.amazon.com/cdk/v2/guide/patterns.html#patterns_multi_tenant

Question: 287

CertyIQ

A company runs applications across multiple AWS accounts. The company uses an organization in AWS Organizations to manage all the accounts. The company runs the applications in VPCs. The applications must be able to communicate with each other.

The company establishes an AWS Direct Connect connection between an on-premises data center and AWS. The company needs to be able to access the VPCs from the on-premises location. The company plans to add more than 100 new applications and VPCs to its AWS environment. The connectivity solution must allow for future growth and simplify the connectivity process from the data center to AWS.

Which solution will meet these requirements?

- A. Attach all VPCs that host applications to the Direct Connect connection by using private VIFs. Create a virtual private gateway in each application VPC. Connect a hardware VPN in the data center to the virtual private gateway in each VPC. Configure static routes in all VPCs. Use the data center VPN appliance to route traffic.
- B. Create a new AWS account and deploy a network transit VPC. Deploy Amazon EC2 instances that run open source VPN software. Create a virtual private gateway in each application VPC. Configure connections between the instance that runs the VPN software in the network transit VPC and each virtual private gateway, and update the route tables. Connect the open source VPN to a hardware VPN in the data center and configure routing.
- C. Create a transit gateway in a new networking account. Use AWS Resource Access Manager (AWS RAM) to create a resource share that has the transit gateways source type. Enter the organization ID share the ID with the transit gateway. Create a private IP VPN that connects to a customer gateway in the data center. Attach the VPN to the transit gateway and configure dynamic routing.
- D. Create a Direct Connect gateway and associate it with the Direct Connect connection. Attach all VPCs that host applications to the Direct Connect gateway. Create a public VIF and attach it to the gateway. Create an AWS Site-to-Site VPN connection by using the public VIF. Connect to a hardware VPN appliance in the data center and configure dynamic routing.

Answer: C

Explanation:

Why option C is the correct solution

Single, centralized connectivity hub – A TransitGateway (TGW) in a dedicated networking account becomes the single point-of-attachment for all current and future VPCs, regardless of the account they reside in. This eliminates the need for multiple VPN tunnels or VPC-specific configurations as the fleet grows beyond 100 applications.

Multi-account sharing via AWS RAM – By creating a **Resource Access Manager** share that distributes the TGW, each member account can attach its VPCs to the same gateway without duplicating resources or managing cross-account routing rules.

Scalable, low-maintenance connectivity – TGW supports **many simultaneous VPN attachments** (and Direct Connect connections) with automatic routing table updates, so adding new VPCs is a matter of a few clicks or API calls.

Built-in routing and redundancy – TGW routes traffic between attached VPCs and on-premises via the attached VPN or Direct Connect link, handling fail-over automatically when multiple attachments are provisioned.

Operational clarity – All connectivity (on-prem → TGW → VPCs) is managed from one place, simplifying monitoring, audit, and troubleshooting compared to per-VPC VPN appliances or VPN-software-running EC2 instances.

Why the other options are less suitable

OptionA – Requires a separate VPN appliance or tunnel for each VPC, leading to a management explosion when the number of VPCs grows. Static routes must be maintained in every VPC, increasing error surface and scaling poorly.

OptionB – Deploys VPN software on EC2 instances (an operational burden) and still relies on a “network transit VPC” that adds an extra hop and potential single point of failure. Adding hundreds of VPCs would require provisioning and configuring an equal number of VPN instances.

OptionD – Misuses a **Direct Connect gateway** with a **public VIF** to carry a Site-to-Site VPN. Public VIFs are intended for public-facing traffic, not for private VPN termination; mixing Direct Connect with a VPN in this way adds unnecessary complexity and does not provide a clean, native multi-account solution.

References

Transit Gateway – Overview, attachments, and scaling considerations:

<https://docs.aws.amazon.com/vpc/latest/userguide/vpc-transit-gateway.html>

AWS Resource Access Manager (RAM) – Share Transit Gateways – How to share a TGW across member accounts:

<https://docs.aws.amazon.com/ram/latest/userguide/what-is-ram.html#ram-tgw-share>

Question: 288

CertyIQ

A company notices VPC flow log data that shows some company traffic is going to malicious websites. The company identifies the Amazon EC2 instance that is sending traffic to the malicious websites and determines that the application that runs on the instance is compromised.

Similar security events have happened multiple times at the company in the past. The company reacted to previous security events by blocking the malicious websites. The company needs a scalable solution to prevent similar security events in the future.

Which solution will meet this requirement with the LEAST operational overhead?

- A. Set up an EC2 instance to run firewall software in the VPC. Route all traffic through the firewall EC2 instance. Maintain a list of known malicious websites on the firewall instance. Disable the source and destination checks for the firewall instance.
- B. Set up an Application Load Balancer (ALB) in front of the problematic EC2 instance. Enable AWS WAF on the EC2 instance.
- C. Set up an Application Load Balancer (ALB) in front of the problematic EC2 instance. Enable AWS WAF on the ALB.
- D. Set up an Amazon Route 53 DNS Firewall rule group. Create a domain list that contains only the domains the application must communicate with to function. Add the domain list to the rule group. Associate the rule group with the VPC.

Answer: D

Explanation:

Why optionD is the best choice

Fully managed service – Route53 DNS Firewall is a serverless, fully-managed filtering layer. No EC2 instances, firewalls, or load-balancer configurations to provision, patch, or scale.

Zero-maintenance domain list – You create a domain list that contains only the legitimate endpoints the application must reach. The firewall automatically blocks any traffic that tries to resolve or contact domains outside this list, stopping connections to newly discovered malicious sites without manual rule updates.

Scales automatically – As traffic volume grows, Route53 handles the inspection and logging; you do not

need to monitor or add capacity.

Least operational overhead – No need to manage firewall instances, ALB listener rules, or custom firewall software. The solution is configured once and then operates continuously with minimal supervision.

Why the other options are less suitable

OptionA – Requires provisioning and maintaining an EC2-based firewall, updating rule sets manually, and disabling source/destination checks. This adds ongoing operational tasks and scaling concerns.

OptionB – While an ALB can front the instance, AWSWAF is primarily an application-layer (HTTP/S) protection tool. It does not filter DNS-based or IP-level access to arbitrary malicious sites, and you would still need additional controls to block the resolved IPs.

OptionC – Enabling WAF on an ALB still relies on request-inspection rules that must be authored and maintained. It does not prevent the instance from reaching malicious domains at the network/DNS level, so it cannot stop the traffic once the domain has been resolved.

Bottom line: Using Route53 DNS Firewall with a dedicated domain list provides a scalable, automatic, and low-maintenance way to block outbound connections to known malicious sites, making it the solution with the least operational overhead.

References

Route53 DNS Firewall – AWS Documentation

<https://docs.aws.amazon.com/Route53/latest/dg/dns-firewall.html>

AWS WAF – Integrating with Application Load Balancers

<https://docs.aws.amazon.com/waf/latest/developerguide/waf-alb.html>

Question: 289

CertyIQ

A company has a hybrid environment. The company uses AWS Direct Connect with a transit VIF attached to a transit gateway to connect its on-premises location to the AWS Cloud. The company operates from a single AWS account in the us-east-1 Region. The company has a legacy application that runs on premises with the corp.example.com DNS name. The company has created an Amazon Route 53 Resolver outbound endpoint and a Resolver rule in us-east-1 in the AWS account.

As part of an expansion, the company creates one more AWS account for a new business unit that operates in us-east-1. The new account has VPCs that are attached to the transit gateway. The new account uses the existing Direct Connect connection to connect to the on-premises location. Applications that run in the new account's VPCs also need to access the on-premises location by using the corp.example.com DNS name.

A network engineer needs to identify a solution to resolve the corp.example.com DNS name from the new account.

Which solution will meet these requirements MOST cost-effectively?

A. Create a hosted transit VIF for the new account on the existing Direct Connect connection. In the new account, create a new Route 53 Resolver outbound endpoint. Also in the new account, create a new Resolver rule that specifies FORWARD as the value for RuleType. Associate the new rule with the VPCs in the new account.

B. Share the Resolver rule from the existing account with the new account by using AWS Resource Access Manager (AWS RAM). Associate the shared rule with the VPCs in the new account.

C. Share the Route 53 Resolver outbound endpoint from the existing account with the new account by using AWS Resource Access Manager (AWS RAM). In the new account, create a new Resolver rule that specifies FORWARD as the value for RuleType. Associate the new rule with the VPCs in the new account.

D. Create an AWS Site-to-Site VPN connection between the new account and the on-premises location. Attach the Site-to-Site VPN connection to the existing transit gateway. In the new account, create a new Route 53 Resolver outbound endpoint. Also in the new account, create a new Resolver rule that specifies FORWARD as

the value for RuleType. Associate the rule with the VPCs in the new account.

Answer: B

Explanation:

Solution B leverages AWS RAM to share the existing Route53 Resolver rule, allowing the new account to reuse the same forwarding rule without creating new resources; this incurs no additional charges beyond the standard Resolver usage.

Sharing the Resolver rule (instead of creating a new outbound endpoint or VPN) eliminates extra data transfer and endpoint costs, making it the most economical option.

Why not A? Creating a hosted transit VIF and a new outbound endpoint adds unnecessary resources and associated hourly fees, increasing cost.

Why not C? RAM cannot share an outbound endpoint; only resource ARNs like rules or policies are shareable, so this approach is not feasible.

Why not D? Implementing a Site-to-Site VPN introduces additional VPN attachment and data transfer charges, which are more expensive than simply sharing the existing Resolver rule.

References

Route53 Resolver – Forwarding Rules: <https://docs.aws.amazon.com/route53/latest/developerguide/resolver-rules.html>

AWS Resource Access Manager (RAM) – Sharing Resolver Rules: <https://docs.aws.amazon.com/ram/latest/userguide/sharing-resolver-rules.html>

Question: 290

CertyIQ

A company has many VPCs in the us-east-1 Region. The company uses a transit gateway to connect the VPCs to one another. The company recently acquired a smaller company that has a single VPC named VPC-Z.

A network engineer needs to onboard VPC-Z to the existing transit gateway. The network engineer notices an IP address overlap between VPC-Z and VPC-A. The network engineer must establish outbound connectivity from VPC-Z to all the company's other VPCs, including VPC-A.

Which combination of solutions will meet this requirement? (Choose two.)

- A. Create a new non-overlapping CIDR block in VPC-Z. Create two subnets that use the new CIDR block.
- B. Create a new non-overlapping CIDR block in VPC-A. Create two subnets that use the new CIDR block.
- C. Create a private NAT gateway in VPC-Z in a non-overlapping subnet. Advertise only the non-overlapping VPC-Z CIDR block to the transit gateway route table.
- D. Create a private NAT gateway in VPC-A in a non-overlapping subnet. Advertise only the non-overlapping VPC-A CIDR block to the transit gateway route table.
- E. Create a VPC peering connection between VPC-A and VPC-Z.

Answer: AC

Explanation:

Why option A+C is correct

Option A resolves the address conflict by assigning VPC-Z a new, non-overlapping CIDR range and creating subnets that use only that range. This eliminates the overlap at the source, allowing the transit-gateway (TGW) to route traffic without ambiguity.

Option C complements the renumbering by placing a private NAT gateway in a non-overlapping subnet of VPC-Z. The NAT gateway provides outbound internet/egress connectivity for the VPC-Z resources while

keeping the traffic isolated from the overlapping addresses in VPC-A. By advertising only the newly-assigned VPC-Z CIDR block to the TGW route table, the TGW can forward traffic from other VPCs to VPC-Z correctly, and the NAT ensures that return traffic is properly sourced from the NAT gateway's address.

Why the other options are not appropriate

OptionB would require re-addressing VPC-A, which is unnecessary and overly disruptive; the overlap can be handled locally in VPC-Z without touching the existing production VPC-A network.

OptionD places a NAT gateway in VPC-A and advertises its CIDR block, but this does not solve the outbound-connectivity requirement from VPC-Z and still leaves the overlap unresolved for traffic initiated from VPC-Z.

OptionE (VPC peering) cannot be used with a transit gateway for multi-VPC connectivity and does not address the CIDR overlap; it also would not provide the required scale and management of a TGW-based architecture.

References

AWS Transit Gateway – Route Table Configuration: <https://docs.aws.amazon.com/vpc/latest/userguide/tgw-route-table.html>

VPC CIDR Block Design – CIDR Overlap Handling: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-cidr-blocks.html>

Question: 291

CertyIQ

A company has two on-premises data center locations named Data Center 1 and Data Center 2. Each data center has a company-managed customer router. Both data centers have a dedicated AWS Direct Connect connection to the same Direct Connect gateway that uses a private VIF. The company enabled Direct Connect SiteLink when the company created each private VIF. The company uses SiteLink to send data from one Direct Connect location to the other, bypassing AWS Regions.

Data Center 2 hosts an application that a partner company needs to access. The partner wants to access the application from the partner's own data center by using the Direct Connect network. The company's network engineer needs to create a separate isolated network for the partner to establish connectivity between Data Center 2 and the partner's data center.

Which combination of steps will meet these requirements? (Choose two.)

- A. Create a new Direct Connect gateway for the partner in an AWS account. Share the account number with the partner to allow the partner to create a hosted VIF to the account. Accept and associate the partner's hosted VIF with the new Direct Connect gateway.
- B. Provision a separate transit VIF for the partner on the Direct Connect connection at Data Center 2. Connect the transit VIF to the partner-specific Direct Connect gateway.
- C. Share the company's AWS account number with the partner to allow the partner to create a hosted VIF to the account. Accept and associate the partner's hosted VIF with the existing Direct Connect gateway.
- D. Provision a separate private VIF for the partner on the Direct Connect connection at Data Center 2. Connect the private VIF to the partner-specific Direct Connect gateway. Enable SiteLink.
- E. Provision a separate public VIF for the partner on the Direct Connect connection at Data Center 2. Connect the public VIF to the partner-specific Direct Connect gateway. Enable SiteLink.

Answer: AD

Explanation:

Why optionA & D are correct

OptionA – Creates a **new Direct Connect gateway** dedicated to the partner in a separate AWS account. By sharing the account ID the partner can create a **hosted VIF** that terminates on this gateway, giving the partner its own isolated endpoint without touching the existing company gateway.

OptionD – On the Direct Connect circuit at **DataCenter2**, provisions a **separate private VIF** that terminates on the partner-specific Direct Connect gateway created in stepA. Enabling **SiteLink** on this VIF allows the partner's traffic to traverse the existing inter-DC SiteLink path (DataCenter1 ↔ DataCenter2) while keeping the partner's traffic strictly isolated from the company's own workloads.

Together they give the partner a **dedicated, isolated network path** that uses its own gateway and VIF, while still leveraging the existing SiteLink to reach the application in DataCenter2.

Why the other options do not satisfy the requirement

OptionB – Uses a **transit VIF** on the same Direct Connect connection but does not create a new gateway or involve a hosted VIF. It would share the existing gateway resources, breaking the isolation requirement, and it omits enabling SiteLink, so the partner could not reach the inter-DC path.

OptionC – Relies on the **existing company gateway** and shares the same VIF resources with the partner. This mixes partner traffic with the company's traffic, violating the “separate isolated network” constraint.

OptionE – Attempts to use a **public VIF**, which is intended for traffic destined for the public internet, not for private partner connectivity. Public VIFs bypass the private Direct Connect fabric and cannot be used with SiteLink for inter-DC communication, making the approach unsuitable.

Key concepts referenced

Direct Connect gateway – a regional, account-level anchor for one or more private virtual interfaces.

Hosted VIF – created by a third party in the owner's account and attached to the owner-managed gateway.

Private VIF – dedicated connection endpoint for a single customer, used here to isolate the partner's traffic.

SiteLink – enables Layer2 connectivity between two Direct Connect locations; must be explicitly enabled on the partner's VIF to use the existing inter-DC path.

References

Direct Connect Gateway – Create a Direct Connect Gateway:

https://docs.aws.amazon.com/dxv2/latest/APIReference/API_CreateDirectConnectGateway.html

Direct Connect SiteLink – Enable SiteLink: <https://docs.aws.amazon.com/dxv2/latest/network-admin/what-is-dx-site-link.html>

Thank you

Thank you for being so interested in the premium exam material.
I'm glad to hear that you found it informative and helpful.

If you have any feedback or thoughts on the bumps, I would love to hear them.
Your insights can help me improve our writing and better understand our readers.

Best of Luck

You have worked hard to get to this point, and you are well-prepared for the exam
Keep your head up, stay positive, and go show that exam what you're made of!

Feedback

More Papers



Future is Secured

100% Pass Guarantee



24/7 Customer Support

Mail us - certyiqofficial@gmail.com



Free Updates

Lifetime Free Updates!

Total: **291 Questions**

Link: <https://certyiq.com/papers/amazon/aws-certified-advanced-networking-specialty-ans-c01>